Python

K19G

2026-11-04

Table of contents

Python **43**
Start here . . . 43
Book Roadmap & Master Architecture . . . 43
Architectural Conventions . . . 45
Formats . . . 46

Python Foundations **47**

Intro **48**

What Is Python **49**

What Is Python **50**
Mental model . . . 50
Minimal example . . . 51
Worked examples . . . 52
1. Peeking inside the CPython compiler with `dis` . . . 52
2. Inspecting runtime execution frames . . . 53
3. Checking Python 3.14 free-threading (GIL status) . . . 54
Pitfalls . . . 55
1. Thinking Python is "just interpreted" . . . 55
2. Modifying objects during iteration . . . 55
3. Mixing tabs and spaces . . . 55
Exercises . . . 56
Further reading . . . 56

Modern Toolchain: uv **57**

The Modern Toolchain with uv **58**
Mental model . . . 58
Minimal example . . . 59
Worked examples . . . 59
1. Single-file scripts with inline dependencies (PEP 723) . . . 59
2. Managing Python runtimes with `uv python` . . . 61
3. Creating and locking a modern project . . . 61
Pitfalls . . . 62
1. Manually activating virtualenvs in every terminal tab . . . 62
2. Hand-editing `uv.lock` . . . 63
3. Polluting the system Python with `sudo pip` . . . 63

Exercises . . . 63
Further reading . . . 63

Code Hygiene: Ruff **64**

Developer Environment & Code Hygiene with Ruff **65**
Mental model . . . 65
Minimal example . . . 66
Worked examples . . . 67
1. Configuring `ruff` in `pyproject.toml` . . . 67
2. How linters work: building an AST inspector . . . 68
3. Automated Git pre-commit hooks . . . 69
Pitfalls . . . 70
1. Running multiple competing formatters . . . 70
2. Blindly running `ruff check --fix` without review . . . 70
3. Committing code without running formatting in CI . . . 70
Exercises . . . 71
Further reading . . . 71

Editor Setup: VS Code **72**

Editor Setup: VS Code with Modern Tooling **73**
Mental model . . . 73
Minimal example . . . 74
Worked examples . . . 75
1. Workspace extension recommendations (`.vscode/extensions.json`) . . . 75
2. Interactive debugging with `.vscode/launch.json` . . . 75
3. Testing format-on-save and auto-import sorting . . . 76
Pitfalls . . . 77
1. Conflicting legacy extensions . . . 77
2. VS Code selecting the wrong Python interpreter . . . 77
3. Expecting editor tools to run in external shells . . . 77
Exercises . . . 78
Further reading . . . 78

Variables **79**

Names, Binding, and Object References **80**

Names, Binding, and Object References **81**
Mental model . . . 81
Minimal example . . . 81
Worked examples . . . 82
Case 1: Inspecting the namespace dictionary . . . 82
Case 2: Binding vs in-place mutation . . . 83
Pitfalls . . . 84
Pitfall 1: Expecting assignment to copy compound objects . . . 84
Pitfall 2: Attempting to modify immutable objects . . . 84

Exercises . . . 85
Further reading . . . 85

Identity, Equality, and Mutability **86**

Identity, Equality, and Mutability **87**
Mental model . . . 87
Minimal example . . . 87
Worked examples . . . 88
Case 1: The small integer caching mechanism . . . 88
Case 2: Shallow copy vs Deep copy . . . 89
Pitfalls . . . 90
Pitfall 1: Comparing values using `is` . . . 90
Pitfall 2: Mutating objects inside tuples . . . 90
Exercises . . . 90
Further reading . . . 90

Naming Conventions, Constants, and Unpacking **91**

Naming Conventions, Constants, and Unpacking **92**
Mental model . . . 92
Minimal example . . . 92
Worked examples . . . 93
Case 1: Star unpacking in loops and data processing . . . 93
Case 2: Marking constant intent with `typing.Final` . . . 94
Pitfalls . . . 94
Pitfall 1: Unbalanced unpacking assignments . . . 94
Pitfall 2: Overriding built-in functions with variable names . . . 94
Exercises . . . 95
Further reading . . . 95

Variable Scopes, LEGB, and Object Lifetimes **96**

Variable Scopes, LEGB, and Object Lifetimes **97**
Mental model . . . 97
Minimal example . . . 97
Worked examples . . . 98
Case 1: Mutating enclosing state with `nonlocal` . . . 98
Case 2: Object lifecycle and reference counting . . . 99
Pitfalls . . . 100
Pitfall 1: UnboundLocalError caused by shadowing assignment . . . 100
Exercises . . . 101
Further reading . . . 101

Data Types **102**

Integers and Arbitrary Precision **103**

Integers and Arbitrary Precision **104**
Mental model . . . 104
Minimal example . . . 104
Worked examples . . . 105
Case 1: Bit manipulation and permission masks . . . 105
Case 2: Integer division vs float division . . . 106
Pitfalls . . . 107
Pitfall 1: Converting colossal integers to strings . . . 107
Pitfall 2: Negative modulo differences from C . . . 107
Exercises . . . 107
Further reading . . . 107

Floating-Point Numbers and Decimal Precision **108**

Floating-Point Numbers and Decimal Precision **109**
Mental model . . . 109
Minimal example . . . 109
Worked examples . . . 110
Case 1: Financial calculations with `decimal.Decimal` . . . 110
Case 2: Exact rational arithmetic with `fractions.Fraction` . . . 111
Pitfalls . . . 112
Pitfall 1: Initializing `Decimal` from a `float` literal . . . 112
Pitfall 2: Comparing floats with `==` . . . 112
Exercises . . . 112
Further reading . . . 112

Booleans, Truthiness, and Short-Circuit Logic **113**

Booleans, Truthiness, and Short-Circuit Logic **114**
Mental model . . . 114
Minimal example . . . 114
Worked examples . . . 115
Case 1: Short-circuit evaluation as a safety guard . . . 115
Case 2: The conditional ternary expression . . . 116
Pitfalls . . . 117
Pitfall 1: Fallback idiom swallowing valid falsy values . . . 117
Pitfall 2: Explicit comparison with `True` or `False` . . . 117
Exercises . . . 117
Further reading . . . 117

Strings, UTF-8 Encoding, and Slicing **118**

Strings, UTF-8 Encoding, and Slicing **119**
Mental model . . . 119
Minimal example . . . 119
Worked examples . . . 120
Case 1: String building performance (join vs +=) . . . 120
Case 2: Encoding strings to bytes and decoding back . . . 121

Pitfalls . . . 121
Pitfall 1: Confusing `bytes` and `str` . . . 121
Pitfall 2: Slice out-of-range behavior . . . 121
Exercises . . . 122
Further reading . . . 122

UTF-8 by Default: What PEP 686 Changes in Python 3.15 **123**

UTF-8 by Default: What PEP 686 Changes in Python 3.15 **124**
Thesis . . . 124
Mental model . . . 124
Worked examples . . . 125
Case 1: What each default actually picks . . . 125
Case 2: A file from last year's job no longer decodes . . . 127
Case 3: Subprocess output in the locale encoding . . . 128
Case 4: Find every place you relied on the default . . . 128
Case 5: The fixes, and the escape hatch . . . 130
The trap . . . 131
The boring rule . . . 132
Try this . . . 132
Sources . . . 132

Type Casting, Type Inspection, and Annotations **134**

Type Casting, Type Inspection, and Annotations **135**
Mental model . . . 135
Minimal example . . . 135
Worked examples . . . 136
Case 1: Checking types: `isinstance` vs `type()` . . . 136
Case 2: Modern Python 3.14 type annotations . . . 137
Pitfalls . . . 137
Pitfall 1: Using `type(x) == T` instead of `isinstance()` . . . 137
Pitfall 2: Assuming type annotations enforce runtime checks . . . 138
Exercises . . . 138
Further reading . . . 138

Template Strings (t-strings) **139**

Template Strings (t-strings) **140**
Mental model . . . 140
Worked examples . . . 140
Case 1: Same braces, different type . . . 140
Case 2: Walk the parts at the desk . . . 141
Case 3: Render like an f-string (on purpose) . . . 142
Case 4: Escape guest text for HTML . . . 143
Case 5: SQL placeholders, not string glue . . . 144
The trap . . . 145
The boring rule . . . 146
Try this . . . 146

None-aware operators (PEP 823 / 824) — draft for 3.16 **147**

None-aware operators (PEP 823 / 824) — draft for 3.16 **148**
Thesis . . . 148
Mental model . . . 148
Worked examples . . . 149
Case 1: What ships today (verified on this box) . . . 149
Case 2: Nested optional attributes (PEP 823 shape) . . . 149
Case 3: Structured data — `?.` is not `dict.get` . . . 151
Case 4: Coalesce without truthiness (PEP 824) . . . 151
Case 5: Assignments and short-circuit edges (823) . . . 152
The trap . . . 152
The boring rule . . . 152
Try this . . . 152
Sources . . . 153

Functions **154**

Defining Functions, Signatures, and Return Values **155**

Defining Functions, Signatures, and Return Values **156**
Mental model . . . 156
Minimal example . . . 156
Worked examples . . . 157
Case 1: Early returns (The "Bouncer Pattern") . . . 157
Case 2: Inspecting function introspection attributes . . . 158
Pitfalls . . . 158
Pitfall 1: Forgetting explicit returns in branches . . . 158
Pitfall 2: Mutating input arguments unexpectedly . . . 159
Exercises . . . 159
Further reading . . . 159

Parameters, Keyword Arguments, and Defaults **160**

Parameters, Keyword Arguments, and Defaults **161**
Mental model . . . 161
Minimal example . . . 161
Worked examples . . . 162
Case 1: Preventing the "Boolean Parameter Trap" . . . 162
Case 2: The Mutable Default Argument Trap and the Fix . . . 163
Pitfalls . . . 164
Pitfall 1: Evaluating dynamic defaults like `datetime.now()` in the signature . . . 164
Exercises . . . 164
Further reading . . . 164

sentinel(): Missing Is Not None (PEP 661) **165**

sentinel(): Missing Is Not None (PEP 661) **166**
Thesis . . . 166
Mental model . . . 166
Worked examples . . . 167
Case 1: What a sentinel is, and what it is not . . . 167
Case 2: When `None` is real data . . . 167
Case 3: Supporting 3.14 and 3.15 from one module . . . 168
Typing . . . 169
The trap . . . 169
The boring rule . . . 170
Try this . . . 170
Sources . . . 170

Variadic Arguments (*args and **kwargs) **171**

Variadic Arguments (*args and **kwargs) **172**
Mental model . . . 172
Minimal example . . . 172
Worked examples . . . 173
Case 1: Argument forwarding in decorator wrappers . . . 173
Case 2: Unpacking dictionaries and lists at call sites . . . 174
Pitfalls . . . 174
Pitfall 1: Signature ordering violations . . . 174
Pitfall 2: Overusing `**kwargs` and destroying API readability . . . 174
Exercises . . . 175
Further reading . . . 175

First-Class Functions, Callables, and Lambdas **176**

First-Class Functions, Callables, and Lambdas **177**
Mental model . . . 177
Minimal example . . . 177
Worked examples . . . 178
Case 1: Command Dispatch Tables (Replacing `if/elif` ladders) . . . 178
Case 2: Custom sorting keys with lambdas and operator . . . 179
Pitfalls . . . 180
Pitfall 1: Over-complicating lambdas . . . 180
Pitfall 2: Assigning a lambda to a variable instead of `def` . . . 180
Exercises . . . 180
Further reading . . . 181

Closures, Lexical Scopes, and Factory Functions **182**

Closures, Lexical Scopes, and Factory Functions **183**
Mental model . . . 183
Minimal example . . . 183
Worked examples . . . 184
Case 1: Stateful API Rate Limiter using `nonlocal` . . . 184
Case 2: The Notorious Late-Binding Loop Trap . . . 185

Pitfalls . . . 186
Pitfall 1: Retaining large memory references in closures . . . 186
Exercises . . . 186
Further reading . . . 186

Generator Functions and Subgenerator Delegation **187**

Generator Functions and Subgenerator Delegation **188**
Mental model . . . 188
The `yield from` Bi-Directional Pipe (PEP 380) . . . 188
Minimal example . . . 189
Worked examples . . . 190
Case 1: Two-Way Coroutines for Rolling Averages with `.send()` . . . 190
Case 2: Deep Tree Traversal and Recursive Flattening with `yield from` . . . 191
Case 3: Capturing Return Values from Subgenerators via `yield from` . . . 192
Pitfalls . . . 193
Pitfall 1: Sending Data to an Unprimed Generator . . . 193
Pitfall 2: Re-iterating an Exhausted Generator . . . 194
Exercises . . . 194
Further reading . . . 195

Control Flow **196**

The if, elif, and else Statements **197**

The `if`, `elif`, and `else` Statements **198**
Mental model . . . 198
Short-Circuit Evaluation Guarding . . . 198
Minimal example . . . 199
Worked examples . . . 200
Case 1: The Bouncer Pattern (Guard Clauses vs Arrow Anti-Pattern) . . . 200
Case 2: Truthiness and The Numeric `0` Trap . . . 201
Pitfalls . . . 202
Pitfall 1: Comparing Explicitly to `True` or `False` . . . 202
Pitfall 2: Combining `and` with `or` Without Parentheses . . . 202
Exercises . . . 203
Further reading . . . 203

Conditional Expressions and the Walrus Operator **204**

Conditional Expressions and the Walrus Operator **205**
Mental model . . . 205
The Walrus Operator Pipeline (`:=`) . . . 205
Minimal example . . . 206
Worked examples . . . 206
Case 1: Streamlining Configuration Fallbacks . . . 206
Case 2: Avoiding Duplicate Computations with the Walrus Operator . . . 207

Pitfalls . . . 208
Pitfall 1: The Nested Ternary Spaghetti Trap . . . 208
Pitfall 2: Precedence Involving `or` and Ternaries . . . 208
Exercises . . . 208
Further reading . . . 209

The while Loop and State-Driven Execution **210**

The `while` Loop and State-Driven Execution **211**
Mental model . . . 211
Minimal example . . . 211
Worked examples . . . 212
Case 1: Exponential Backoff with Jitter for Network Retries . . . 212
Case 2: State-Machine Dispatcher Loop . . . 213
Pitfalls . . . 215
Pitfall 1: The Accidental Runaway Infinite Loop . . . 215
Pitfall 2: Busy-Waiting Without Sleeping . . . 215
Exercises . . . 215
Further reading . . . 215

The for Loop and Sequence Iteration **216**

The `for` Loop and Sequence Iteration **217**
Mental model . . . 217
Minimal example . . . 217
Worked examples . . . 219
Case 1: Deconstructing the Iterator Protocol Manually . . . 219
Case 2: File Iteration Line by Line (Constant Memory) . . . 219
Pitfalls . . . 220
Pitfall 1: Modifying a Collection While Iterating Over It . . . 220
Pitfall 2: The `range(len(sequence))` Anti-Pattern . . . 221
Exercises . . . 221
Further reading . . . 221

Loop Control: break, continue, and Loop else **222**

Loop Control: `break`, `continue`, and Loop `else` **223**
Mental model . . . 223
Minimal example . . . 223
Worked examples . . . 225
Case 1: Search and Validate Without Boolean Flags (`for ... else`) . . . 225
Case 2: Breaking Out of Nested Loops . . . 225
Pitfalls . . . 226
Pitfall 1: Assuming `break` Exits All Nested Loops . . . 226
Pitfall 2: Confusing Loop `else` with `if else` . . . 227
Exercises . . . 227
Further reading . . . 227

Iteration Utilities: enumerate and zip **228**

Iteration Utilities: `enumerate` and `zip` **229**
Mental model . . . 229
Minimal example . . . 229
Worked examples . . . 230
Case 1: Catching Silent Data Truncation with `strict=True` . . . 230
Case 2: Unequal Streams with Padding Using `itertools.zip_longest` . . . 231
Pitfalls . . . 232
Pitfall 1: Relying on Default `zip()` Without `strict=True` . . . 232
Pitfall 2: Re-Iterating a `zip` or `enumerate` Object . . . 232
Exercises . . . 232
Further reading . . . 233

Structural Pattern Matching with match and case **234**

Structural Pattern Matching with `match` and `case` **235**
Mental model . . . 235
Minimal example . . . 235
Worked examples . . . 236
Case 1: Mapping Patterns for Telemetry Event Routing . . . 236
Case 2: Class Patterns with Dataclasses . . . 237
Pitfalls . . . 239
Pitfall 1: The Variable Capture Trap (Constants vs Variables) . . . 239
Exercises . . . 239
Further reading . . . 240

List, Dict, and Set Comprehensions **241**

List, Dict, and Set Comprehensions **242**
Mental model . . . 242
Minimal example . . . 243
Worked examples . . . 244
Case 1: Ingesting and Sanitizing Raw Metric Records (List Comprehensions) . . . 244
Case 2: Inverting and Indexing Service Registries (Dict Comprehensions) . . . 245
Case 3: Matrix Flattening and Cartesian Grids (Nested Loops) . . . 246
Case 4: Avoiding Duplicate Computation with the Walrus Operator (`:=`) . . . 247
Pitfalls . . . 249
Pitfall 1: Misusing Comprehensions for Side Effects . . . 249
Pitfall 2: Over-Nesting (Write-Only Code) . . . 249
Pitfall 3: Scope Isolation (Python 3 Comprehension Scope) . . . 250
Exercises . . . 250
Further reading . . . 251

Generator Expressions **252**

Generator Expressions **253**
Mental model . . . 253
Minimal example . . . 254
Worked examples . . . 255
Case 1: Streaming Gigabyte Log Parsing with Constant O(1) Memory . . . 255

Case 2: Short-Circuiting Aggregations with `any()` and `all()` . . . 256
Case 3: Composing Multi-Stage Data Pipelines . . . 257
Pitfalls . . . 258
Pitfall 1: Generator Exhaustion (The "Read-Once" Trap) . . . 258
Pitfall 2: Late-Binding Closures in Loop Expressions . . . 259
Pitfall 3: Indexing and Length Queries (`gen[0]` or `len(gen)`) . . . 259
Exercises . . . 260
Further reading . . . 260

Data Structures **261**

Lists and Dynamic Array Mechanics **262**

Lists and Dynamic Array Mechanics **263**
Mental model . . . 263
Minimal example . . . 264
Worked examples . . . 265
Case 1: In-Place Mutations vs Full Copy Allocations . . . 265
Case 2: Custom In-Place Sorting with Timsort . . . 266
Case 3: Shallow Copies vs Deep Copies . . . 267
Pitfalls . . . 268
Pitfall 1: Modifying a List While Iterating Over It . . . 268
Pitfall 2: The Repeated Multiplication Reference Duplication Bug . . . 268
Pitfall 3: Assuming `list.sort()` Returns the Sorted List . . . 269
Exercises . . . 270
Further reading . . . 270

Tuples and Immutable Sequence Records **271**

Tuples and Immutable Sequence Records **272**
Mental model . . . 272
The Hashability Rule for Tuples . . . 272
Minimal example . . . 273
Worked examples . . . 274
Case 1: Composite Coordinates and Multi-Dimensional Indexing . . . 274
Case 2: Defensive API Design (Immutable Return Values) . . . 275
Case 3: Tuple Struct Packing and Structural Records . . . 276
Pitfalls . . . 277
Pitfall 1: The Missing Comma in Singleton Tuples . . . 277
Pitfall 2: The Mutable-Element-in-Tuple Trap . . . 277
Pitfall 3: The Augmented Assignment Trap on Tuples . . . 278
Exercises . . . 279
Further reading . . . 279

Dictionaries and Hash Table Mechanics **280**

Dictionaries and Hash Table Mechanics **281**
Mental model . . . 281
Key Advantages . . . 281
Minimal example . . . 282
Worked examples . . . 283
Case 1: Grouping with `.setdefault()` vs Explicit Checking . . . 283
Case 2: Multi-Layer Configuration Cascades with Dict Merges . . . 284
Case 3: Dynamic Dictionary Views vs Static Snapshots . . . 285
Pitfalls . . . 286
Pitfall 1: Modifying a Dictionary During Iteration . . . 286
Pitfall 2: Direct Subscripting on Optional Keys (`KeyError`) . . . 286
Pitfall 3: Mutable Objects as Dictionary Keys . . . 287
Exercises . . . 287
Further reading . . . 287

frozendict: the Immutable Mapping (PEP 814) **288**

frozendict: the Immutable Mapping (PEP 814) **289**
Thesis . . . 289
Mental model . . . 289
Worked examples . . . 290
Case 1: The basic contract . . . 290
Case 2: Mappings as keys and cache arguments . . . 291
Case 3: A default argument that cannot be edited . . . 292
Case 4: Snapshot vs view (`frozendict` vs `MappingProxyType`) . . . 293
Case 5: `eval`/`exec` globals, and a sharp edge . . . 294
The trap . . . 295
The boring rule . . . 296
Try this . . . 297
Sources . . . 298

Sets and Set Theory Operations **299**

Sets and Set Theory Operations **300**
Mental model . . . 300
Set Theory Operations . . . 300
Minimal example . . . 301
Worked examples . . . 302
Case 1: Declarative Infrastructure State Reconciliation . . . 302
Case 2: Role-Based Access Control (RBAC) with Subset Checks . . . 303
Case 3: Immutable Sets with `frozenset` as Dictionary Keys . . . 304
Pitfalls . . . 304
Pitfall 1: The Empty Set Syntax Trap (`{}` vs `set()`) . . . 304
Pitfall 2: Adding Mutable Objects to a Set . . . 305
Pitfall 3: Mutating a Set While Iterating . . . 305
Exercises . . . 306
Further reading . . . 306

Advanced Slicing and Pattern Unpacking **307**

Advanced Slicing and Pattern Unpacking **308**
Mental model . . . 308
The Stride Equation . . . 308
Extended Starred Unpacking . . . 308
Minimal example . . . 309
Worked examples . . . 310
Case 1: In-Place Slice Replacement and Resizing . . . 310
Case 2: Parsing Protocol Frames with Named Slices . . . 310
Case 3: Starred Head-Tail Splitting for Recursive Processing . . . 311
Pitfalls . . . 312
Pitfall 1: Assigning a Non-Iterable to a Slice . . . 312
Pitfall 2: Multiple Starred Expressions in a Single Target . . . 313
Exercises . . . 313
Further reading . . . 313

Unpacking in Comprehensions (PEP 798) **314**

Unpacking in Comprehensions (PEP 798) **315**
Thesis . . . 315
Mental model . . . 315
Worked examples . . . 316
Case 1: Flatten a list of lists (the StackOverflow classic) . . . 316
Case 2: Set union and dict merge . . . 317
Case 3: Replace `chain.from_iterable` in streaming code . . . 317
Case 4: Stdlib-shaped refactors (from the PEP) . . . 317
Case 5: Function-call parentheses (read carefully) . . . 318
The trap . . . 318
The boring rule . . . 319
Try this . . . 319
Sources . . . 319

Binary Search with Bisect and Priority Queues with Heapq **320**

Binary Search with Bisect and Priority Queues with Heapq **321**
Mental model . . . 321
1. `bisect`: Bisection and Binary Search . . . 321
2. `heapq`: Min-Heap Priority Queues . . . 321
Minimal example . . . 322
Worked examples . . . 323
Case 1: Numeric Score / Severity Bucketing with `bisect` . . . 323
Case 2: Multi-Criteria Priority Scheduler with Tie-Breaking Counter . . . 324
Case 3: Finding Top K Extremes with `nlargest` and `nsmallest` . . . 325
Pitfalls . . . 326
Pitfall 1: Calling `bisect` on an Unsorted List . . . 326
Pitfall 2: Modifying Elements Inside a Heap Directly . . . 327
Exercises . . . 327
Further reading . . . 327

Zero-Copy Binary Memory with memoryview and bytearray **328**

Zero-Copy Binary Memory with `memoryview` and `bytearray` **329**
Mental model . . . 329
Binary Types Compared . . . 330
Minimal example . . . 330
Worked examples . . . 331
Case 1: Zero-Copy Network Packet Header Dissection . . . 331
Case 2: In-Place Mutation with `bytearray` and `memoryview` . . . 332
Case 3: Casting Memory Layouts with `.cast()` . . . 333
Pitfalls . . . 334
Pitfall 1: Attempting In-Place Writes on an Immutable `bytes` View . . . 334
Pitfall 2: The Buffer Resizing Lock Trap . . . 335
Exercises . . . 335
Further reading . . . 336

Error Handling **337**

Exceptions, Tracebacks, and Propagation **338**

Exceptions, Tracebacks, and Propagation **339**
Mental model . . . 339
The Four-Part Exception Lifecycle . . . 339
Minimal example . . . 340
Worked examples . . . 341
Case 1: Attaching Diagnostics with Exception Notes (`add_note()`) . . . 341
Case 2: Concurrent Failure Bundling with `ExceptionGroup` . . . 342
Pitfalls . . . 343
Pitfall 1: The Bare `except:` or `except Exception:` Anti-Pattern . . . 343
Pitfall 2: Forgetting to Re-Raise After Logging . . . 343
Exercises . . . 344
Further reading . . . 344

Custom Exceptions and Error Hierarchies **345**

Custom Exceptions and Error Hierarchies **346**
Mental model . . . 346
Exception Chaining (`__cause__` vs `__context__`) . . . 346
Minimal example . . . 347
Worked examples . . . 347
Case 1: Exception Chaining with `raise ... from err` . . . 347
Case 2: Suppressing Internal Noise with `from None` . . . 348
Pitfalls . . . 349
Pitfall 1: Inheriting from `BaseException` instead of `Exception` . . . 349
Pitfall 2: Overly Flat or Monolithic Exceptions . . . 350
Exercises . . . 350
Further reading . . . 350

Context Managers and Resource Management **351**

Context Managers and Resource Management **352**
Mental model . . . 352
Minimal example . . . 352
Worked examples . . . 354
Case 1: Benchmark Timer with `@contextlib.contextmanager` . . . 354
Case 2: Clean Error Suppression with `contextlib.suppress` . . . 355
Case 3: Dynamic Multi-Resource Coordination with `ExitStack` . . . 355
Pitfalls . . . 356
Pitfall 1: Accidentally Suppressing All Exceptions in `__exit__` . . . 356
Pitfall 2: Omitting `try/finally` in `@contextmanager` Generators . . . 357
Exercises . . . 357
Further reading . . . 358

Exception Groups and the except* Operator **359**

Exception Groups and the `except*` Operator **360**
Mental model . . . 360
Traditional `except` vs Modern `except*` . . . 361
Minimal example . . . 361
Worked examples . . . 362
Case 1: Concurrent Node Health Probing with Diagnostic Subgroups . . . 362
Case 2: Programmatic Group Splitting and Filtering (`.split()` and `.subgroup()`) . 363
Case 3: Enriching Individual Errors in Groups with `add_note()` . . . 364
Pitfalls . . . 365
Pitfall 1: Mixing `except` and `except*` in the Same `try` Block . . . 365
Pitfall 2: Partial Matching Leaves Unhandled Errors to Propagate . . . 366
Pitfall 3: Catching `ExceptionGroup` Directly with Traditional `except` . . . 366
Exercises . . . 367
Further reading . . . 367

Objects and Classes **368**

Classes, Instances, and Encapsulation **369**

Classes, Instances, and Encapsulation **370**
Mental model . . . 370
Minimal example . . . 370
Worked examples . . . 372
Case 1: Computed Properties & Caching . . . 372
Case 2: Demystifying Name Mangling (`__var`) . . . 373
Pitfalls . . . 374
Pitfall 1: Mutating Class Attributes via an Instance Reference . . . 374
Exercises . . . 374
Further reading . . . 375

The Python Data Model and Dunder Protocols **376**

The Python Data Model and Dunder Protocols **377**
Mental model . . . 377
Minimal example . . . 377
Worked examples . . . 379
Case 1: The Golden Rule of `__repr__` vs `__str__` . . . 379
Case 2: Equality (`__eq__`) and Hashing (`__hash__`) . . . 380
Case 3: Automatic Ordering with `functools.total_ordering` . . . 381
Case 4: Callable Objects with `__call__` . . . 382
Pitfalls . . . 383
Pitfall 1: Defining `__eq__` Without `__hash__` . . . 383
Exercises . . . 383
Further reading . . . 384

Inheritance, MRO, and Composition **385**

Inheritance, MRO, and Composition **386**
Mental model . . . 386
Cooperative `super()` Dispatch . . . 386
Minimal example . . . 387
Worked examples . . . 388
Case 1: Formal Interfaces with Abstract Base Classes (`abc.ABC`) . . . 388
Case 2: Composition Over Inheritance . . . 389
Pitfalls . . . 390
Pitfall 1: Calling Direct Parent Names Instead of `super()` . . . 390
Pitfall 2: Overusing Multiple Inheritance . . . 391
Exercises . . . 391
Further reading . . . 391

Dataclasses and Declarative Models **392**

Dataclasses and Declarative Models **393**
Mental model . . . 393
Memory Architecture: `__dict__` vs `__slots__` . . . 393
Minimal example . . . 394
Worked examples . . . 395
Case 1: Mutable Defaults via `field(default_factory=...)` . . . 395
Case 2: Memory Benchmark: `slots=True` vs Standard Class . . . 395
Case 3: Dataclasses vs Pydantic v2: Architectural Boundaries . . . 396
Pitfalls . . . 397
Pitfall 1: Assuming `@dataclass` Enforces Types at Runtime . . . 397
Exercises . . . 397
Further reading . . . 398

Dataclass slots=True **399**

Dataclass slots=True **400**
Thesis . . . 400
Mental model . . . 400
Worked example . . . 400

Trap . . . 401
Boring rule . . . 401
Try this . . . 401
Sources . . . 401

Decorators and Metaprogramming **402**

Decorators and Metaprogramming **403**
Mental model . . . 403
The Wrapper Wrapping Pipeline . . . 403
Minimal example . . . 404
Worked examples . . . 405
Case 1: Parameterized Decorator Factory (`@retry`) . . . 405
Case 2: Method Decorators: `@classmethod` vs `@staticmethod` . . . 406
Case 3: Class Decorators for Automated Registration . . . 407
Pitfalls . . . 408
Pitfall 1: Omitting `@functools.wraps` . . . 408
Pitfall 2: Decorator Definition-Time vs Execution-Time Confusion . . . 408
Exercises . . . 408
Further reading . . . 408

Descriptors and the Attribute Access Protocol **409**

Descriptors and the Attribute Access Protocol **410**
Mental model . . . 410
The Descriptor Protocol Methods . . . 411
Minimal example . . . 411
Worked examples . . . 412
Case 1: How `@property` Works Under the Hood . . . 412
Case 2: Lazy Cached Properties with Non-Data Descriptors . . . 414
Pitfalls . . . 415
Pitfall 1: Storing Value on the Descriptor Instance . . . 415
Pitfall 2: Forgetting `if instance is None:` in `__get__` . . . 416
Exercises . . . 417
Further reading . . . 417

Metaclasses and Class Construction Mechanics **418**

Metaclasses and Class Construction Mechanics **419**
Mental model . . . 419
The CPython Class Construction Pipeline . . . 419
Minimal example . . . 420
Worked examples . . . 421
Case 1: The Modern Alternative: Zero-Boilerplate Plugins with `__init_subclass__` 421
Case 2: Enforcing Architectural Invariants with a Custom Metaclass . . . 422
Case 3: Capturing Attribute Declaration Order with `__prepare__` . . . 424
Pitfalls . . . 425
Pitfall 1: The Metaclass Conflict Error . . . 425
Pitfall 2: Overusing Metaclasses when Simpler Tools Suffice . . . 425

Exercises . . . 426
Further reading . . . 426

Abstract Base Classes and Structural Protocols **427**

Abstract Base Classes and Structural Protocols **428**
Mental model . . . 428
Comparison Matrix . . . 428
Minimal example . . . 429
Worked examples . . . 430
Case 1: Enforcing Framework Contracts with Abstract Base Classes . . . 430
Case 2: Structural Decoupling with Protocols Across Unrelated Packages . . . 431
Case 3: Virtual Subclasses via `ABC.register()` . . . 432
Pitfalls . . . 434
Pitfall 1: Attempting to Instantiate an Incomplete ABC . . . 434
Pitfall 2: Inverting Decorator Order with `@property` and `@abstractmethod` . . . 434
Pitfall 3: Assuming `@runtime_checkable` Validates Types and Signatures . . . 435
Exercises . . . 435
Further reading . . . 436

Modules and Packages **437**

Modules and the Import System **438**

Modules and the Import System **439**
Mental model . . . 439
Minimal example . . . 439
Worked examples . . . 441
Case 1: Dynamic Driver Registry with `importlib` . . . 441
Case 2: Diagnosing and Breaking Circular Imports . . . 441
Case 3: The `if __name__ == "__main__":` Boundary . . . 443
Pitfalls . . . 443
Pitfall 1: Standard Library Shadowing . . . 443
Exercises . . . 444
Further reading . . . 444

Packages, Namespaces, and Public APIs **445**

Packages, Namespaces, and Public APIs **446**
Mental model . . . 446
Minimal example . . . 446
Worked examples . . . 448
Case 1: Relative vs Absolute Imports . . . 448
Case 2: The `__all__` Contract and `from module import *` . . . 448
Case 3: PEP 420 Namespace Packages . . . 449
Pitfalls . . . 450
Pitfall 1: Executing Submodules Directly Instead of via `-m` . . . 450
Pitfall 2: Heavy Work Inside `__init__.py` . . . 450

Exercises . . . 450
Further reading . . . 450

Workspaces, Virtual Environments, and Dependency Isolation 451

Workspaces, Virtual Environments, and Dependency Isolation 452
Mental model . . . 452
Minimal example . . . 452
Worked examples . . . 453
Case 1: Inline Script Dependencies (PEP 723) . . . 453
Case 2: Multi-Package Workspaces with `uv` . . . 454
Pitfalls . . . 455
Pitfall 1: Breaking OS Packages with `sudo pip install` (PEP 668) . . . 455
Pitfall 2: Committing `.venv/` into Version Control . . . 455
Exercises . . . 456
Further reading . . . 456

Import Hooks, Finders, and ModuleSpec Mechanics 457

Import Hooks, Finders, and ModuleSpec Mechanics 458
Mental model . . . 458
Minimal example . . . 459
Worked examples . . . 460
Case 1: Loading Modules Directly from JSON Configurations . . . 460
Case 2: Auditing Dependency Load Latency via a MetaPath Hook . . . 462
Pitfalls . . . 463
Pitfall 1: Circular Import Deadlocks . . . 463
Pitfall 2: Forgetting to Clean Up `sys.meta_path` . . . 464
Exercises . . . 464
Further reading . . . 465

Lazy Imports (PEP 810) 466

Lazy Imports (PEP 810) 467
Thesis . . . 467
Mental model . . . 467
Worked examples . . . 468
Case 1: A multi-command CLI that only pays for the command you run . . . 468
Case 2: The error moves — and so must your smoke test . . . 470
Case 3: One source file for 3.13, 3.14, and 3.15 with `__lazy_modules__` . . . 472
Case 4: Type-only imports without `if TYPE_CHECKING:` . . . 473
Case 5: Application-wide laziness, scoped by a filter . . . 474
The trap . . . 476
The boring rule . . . 477
Try this . . . 478
Sources . . . 478

Standard Library **479**

Specialized Collections, Enums, and Heaps **480**

Specialized Collections, Enums, and Heaps **481**
Mental model . . . 481
Minimal example . . . 481
Worked examples . . . 483
Case 1: High-Speed Counter Arithmetic for Anomaly Detection . . . 483
Case 2: Logarithmic Table Lookup with `bisect` . . . 483
Pitfalls . . . 484
Pitfall 1: Modifying Elements in a `heapq` Directly . . . 484
Exercises . . . 485
Further reading . . . 485

High-Performance Iteration and Functional Tools **486**

High-Performance Iteration and Functional Tools **487**
Mental model . . . 487
Minimal example . . . 487
Worked examples . . . 489
Case 1: Telemetry Grouping with `itertools.groupby()` . . . 489
Case 2: Clean Polymorphism with `functools.singledispatch` . . . 489
Pitfalls . . . 491
Pitfall 1: Unsorted Input to `itertools.groupby` . . . 491
Pitfall 2: Decorating Functions with Mutable Arguments in `@lru_cache` . . . 491
Exercises . . . 491
Further reading . . . 491

Text Processing, Unicode, and Regular Expressions **492**

Text Processing, Unicode, and Regular Expressions **493**
Mental model . . . 493
Minimal example . . . 493
Worked examples . . . 495
Case 1: Dynamic Data Masking with `re.sub()` Callables . . . 495
Case 2: Unicode Normalization for Security Checks . . . 495
Pitfalls . . . 496
Pitfall 1: Catastrophic Backtracking (ReDoS) . . . 496
Pitfall 2: Omitting Raw String Prefix (`r""`) . . . 497
Exercises . . . 497
Further reading . . . 497

Filesystem and Modern Path Manipulation **498**

Filesystem and Modern Path Manipulation **499**
Mental model . . . 499
Minimal example . . . 499

Worked examples . . . 501
Case 1: Buffered Chunk Streaming for Gigabyte-Scale Files . . . 501
Case 2: Atomic File Writes via Temporary Renaming . . . 501
Pitfalls . . . 502
Pitfall 1: Mixing `os.path` String Manipulation with `Path` Objects . . . 502
Pitfall 2: Forgetting `parents=True` on `mkdir()` . . . 502
Exercises . . . 503
Further reading . . . 503

Structured Data Serialization: JSON, CSV, and TOML **504**

Structured Data Serialization: JSON, CSV, and TOML **505**
Mental model . . . 505
Minimal example . . . 505
Worked examples . . . 507
Case 1: Custom JSON Serialization for `datetime`, `UUID`, and `set` . . . 507
Case 2: Reading CSV with `DictReader` and Type Conversion . . . 508
Pitfalls . . . 509
Pitfall 1: Writing CSVs on Windows Without `newline=""` . . . 509
Pitfall 2: `tomllib` is Read-Only . . . 509
Exercises . . . 510
Further reading . . . 510

Embedded Relational Storage with SQLite3 **511**

Embedded Relational Storage with SQLite3 **512**
Mental model . . . 512
Minimal example . . . 512
Worked examples . . . 513
Case 1: High-Throughput Batch Insertion with `executemany()` . . . 513
Case 2: Transaction Atomicity and Automatic Rollback . . . 515
Pitfalls . . . 516
Pitfall 1: SQL Injection via String Interpolation . . . 516
Pitfall 2: Trailing Comma in Single Parameter Tuples . . . 516
Exercises . . . 516
Further reading . . . 517

Dates, Times, and Timezones **518**

Dates, Times, and Timezones **519**
Mental model . . . 519
Minimal example . . . 519
Worked examples . . . 520
Case 1: The Daylight Saving Time (DST) Arithmetic Trap . . . 520
Case 2: TTL Expiration and Token Age Validation . . . 521
Pitfalls . . . 522
Pitfall 1: Calling `datetime.now()` Without Arguments . . . 522
Pitfall 2: Using Deprecated `datetime.utcnow()` . . . 522
Exercises . . . 522

Further reading . . . 523

Math, Randomness, and Cryptography **524**

Math, Randomness, and Cryptography **525**
Mental model . . . 525
Minimal example . . . 525
Worked examples . . . 527
Case 1: Timing Attack Defense with `hmac.compare_digest` . . . 527
Case 2: Exact Rational Fractions with `fractions.Fraction` . . . 527
Pitfalls . . . 528
Pitfall 1: Using `random` for Security Tokens . . . 528
Pitfall 2: Initializing `Decimal` from a `float` . . . 528
Exercises . . . 529
Further reading . . . 529

math.integer (PEP 791): Exact Integer Functions Leave math **530**

`math.integer` (PEP 791): Exact Integer Functions Leave `math` **531**
Thesis . . . 531
Mental model . . . 531
Worked examples . . . 532
Case 1: The tour . . . 532
Case 2: Where to put new code . . . 534
Case 3: Why the split helps . . . 535
The trap . . . 535
The boring rule . . . 535
Try this . . . 536
Sources . . . 536

OS, System Environment, and Subprocess Management **537**

OS, System Environment, and Subprocess Management **538**
Mental model . . . 538
Minimal example . . . 538
Worked examples . . . 540
Case 1: Piping Streams Between Child Processes (`p1 | p2`) . . . 540
Case 2: Sandboxing Environment Variables . . . 541
Pitfalls . . . 541
Pitfall 1: The `shell=True` Security Vulnerability . . . 541
Pitfall 2: Forgetting `timeout=` . . . 542
Exercises . . . 542
Further reading . . . 542

CLI Parsing and Structured Application Logging **543**

CLI Parsing and Structured Application Logging **544**
Mental model . . . 544
Minimal example . . . 544

Worked examples . . . 546
Case 1: Structured JSON Logging for Observability Systems . . . 546
Case 2: Log File Rotation with `RotatingFileHandler` . . . 547
Pitfalls . . . 548
Pitfall 1: Adding Duplicate Handlers . . . 548
Pitfall 2: Using `print()` Instead of `logging` in Libraries . . . 548
Exercises . . . 549
Further reading . . . 549

Compression, Tarballs, and Archive Management **550**

Compression, Tarballs, and Archive Management **551**
Mental model . . . 551
Minimal example . . . 551
Worked examples . . . 552
Case 1: In-Memory Gzip Payload Compression for Network APIs . . . 552
Case 2: Defending Against Path Traversal (Tar Slip / Zip Slip) . . . 553
Pitfalls . . . 554
Pitfall 1: Creating Uncompressed ZIP Archives . . . 554
Pitfall 2: Extracting Untrusted Archives Without Filters . . . 554
Exercises . . . 555
Further reading . . . 555

Networking, Low-Level Sockets, and IP Address Calculations **556**

Networking, Low-Level Sockets, and IP Address Calculations **557**
Mental model . . . 557
Minimal example . . . 557
Worked examples . . . 559
Case 1: Low-Level TCP Loopback Echo Service . . . 559
Case 2: Subnet Splitting and Overlap Detection . . . 560
Pitfalls . . . 561
Pitfall 1: Host Bits Set in `IPv4Network` . . . 561
Pitfall 2: Assuming `recv()` Returns a Full Packet . . . 561
Exercises . . . 562
Further reading . . . 562

Signal Handling and Process Lifecycle Management **563**

Signal Handling and Process Lifecycle Management **564**
Mental model . . . 564
Minimal example . . . 565
Worked examples . . . 566
Case 1: Container Termination Controller with In-Flight Drain . . . 566
Case 2: Zero-Downtime Configuration Reload via `SIGHUP` . . . 567
Case 3: Watchdog Execution Timers with `SIGALRM` . . . 568
Pitfalls . . . 570
Pitfall 1: Registering Signals Outside the Main Thread . . . 570
Pitfall 2: Deadlocks from Non-Reentrant Code in Signal Handlers . . . 570

Pitfall 3: PID 1 in Docker Containers Ignoring Signals . . . 571
Exercises . . . 571
Further reading . . . 571

Built-In HTTP Networking: Servers, Clients, and Request Handlers 572

Built-In HTTP Networking: Servers, Clients, and Request Handlers 573
Mental model . . . 573
Minimal example . . . 574
Worked examples . . . 575
Case 1: Building a REST Metrics Ingestion Webhook Handler . . . 575
Case 2: Low-Level Streaming Requests with `http.client.HTTPConnection` . . . 577
Pitfalls . . . 578
Pitfall 1: Using `HTTPServer` Instead of `ThreadingHTTPServer` . . . 578
Pitfall 2: Omitting the `timeout` Parameter in `urllib.request.urlopen` . . . 579
Pitfall 3: Not Reading `Content-Length` in `do_POST` . . . 579
Exercises . . . 579
Further reading . . . 580

High-Performance File I/O with Memory-Mapped Files (mmap) 581

High-Performance File I/O with Memory-Mapped Files (`mmap`) 582
Mental model . . . 582
Minimal example . . . 583
Worked examples . . . 584
Case 1: Regex Searching Gigabyte Logs with Zero RAM Allocation . . . 584
Case 2: In-Place Binary Patching and Header Updates . . . 585
Case 3: Anonymous Shared Memory IPC Between Processes . . . 586
Pitfalls . . . 587
Pitfall 1: Attempting to Map an Empty (0-Byte) File . . . 587
Pitfall 2: Modifying File Size Outside `mmap` . . . 588
Exercises . . . 588
Further reading . . . 588

Runtime Introspection and Reflection with inspect 589

Runtime Introspection and Reflection with `inspect` 590
Mental model . . . 590
Minimal example . . . 591
Worked examples . . . 592
Case 1: Runtime Type-Enforcing Decorator with `Signature.bind()` . . . 592
Case 2: Zero-Magic Dependency Injection Container . . . 594
Case 3: Caller-Context Telemetry and Frame Navigation . . . 596
Pitfalls . . . 597
Pitfall 1: Retaining Frame References and Creating GC Leaks . . . 597
Pitfall 2: `inspect.getsource()` on Dynamically Evaluated or C Code . . . 598
Exercises . . . 598
Further reading . . . 599

Weak References and Garbage Collection (weakref, gc) **600**

Weak References and Garbage Collection (`weakref`, `gc`) **601**
Mental model . . . 601
Minimal example . . . 602
Worked examples . . . 603
Case 1: Self-Pruning In-Memory Entity Cache with `WeakValueDictionary` . . . 603
Case 2: Deterministic OS Resource Cleanup with `weakref.finalize` . . . 605
Case 3: Diagnosing and Breaking Cyclic Reference Leaks with `gc` . . . 606
Pitfalls . . . 608
Pitfall 1: Attempting to Weakly Reference Built-in Types Directly . . . 608
Pitfall 2: Capturing `self` in a Finalizer Callback . . . 609
Exercises . . . 609
Further reading . . . 610

Mocking and Test Doubles (unittest.mock) **611**

Mocking and Test Doubles (`unittest.mock`) **612**
Mental model . . . 612
The Golden Rule of Patching: "Where to Patch" . . . 612
Minimal example . . . 613
Worked examples . . . 614
Case 1: Simulating Flaky Networks with `side_effect` Retries . . . 614
Case 2: Preventing Mock Drift with `autospec=True` . . . 615
Case 3: Testing Asynchronous Event Pipelines with `AsyncMock` . . . 616
Pitfalls . . . 618
Pitfall 1: Patching the Definition Site Instead of the Lookup Site . . . 618
Pitfall 2: Misspelling Assertion Methods (`assert_called_once_with` typo) . . . 618
Exercises . . . 619
Further reading . . . 619

Binary Data and Network Protocols (struct) **620**

Binary Data and Network Protocols (`struct`) **621**
Mental model . . . 621
Endianness & Byte Order Modifiers . . . 621
Minimal example . . . 622
Worked examples . . . 623
Case 1: Custom Binary RPC Wire Framing with Checksum Validation . . . 623
Case 2: High-Throughput Stream Parsing with `struct.iter_unpack` . . . 625
Case 3: Zero-Copy In-Place Buffer Updates with `pack_into` and `unpack_from` . . . 626
Pitfalls . . . 627
Pitfall 1: Relying on Native `@` Alignment for Network or Disk Formats . . . 627
Pitfall 2: String Length Mismatches in `s` Format Strings . . . 628
Exercises . . . 628
Further reading . . . 629

Object Serialization and Persistent Storage (pickle, shelve) **630**

Object Serialization and Persistent Storage (`pickle`, `shelve`) 631
Mental model . . . 631
Security Warning: Never Unpickle Untrusted Data . . . 632
Minimal example . . . 632
Worked examples . . . 633
Case 1: Sandboxed Deserialization with `SafeUnpickler` . . . 633
Case 2: Persistent Disk State with `shelve` . . . 634
Case 3: Excluding Transient Resources with `__getstate__` and `__setstate__` . . . 636
Pitfalls . . . 637
Pitfall 1: Mutating Mutable Objects in `shelve` Without `writeback=True` . . . 637
Pitfall 2: Protocol Incompatibility Across Python Versions . . . 638
Exercises . . . 638
Further reading . . . 638

functools.cache vs lru_cache 639

functools.cache vs lru_cache 640
Thesis . . . 640
Mental model . . . 640
Worked examples . . . 640
Trap . . . 641
Boring rule . . . 641
Try this . . . 641
Sources . . . 641

base64 Throughput on Python 3.15 642

base64 Throughput on Python 3.15 643
Thesis . . . 643
Mental model . . . 643
Worked examples . . . 643
Trap . . . 644
Boring rule . . . 644
Try this . . . 644
Sources . . . 644

Parsing TOML with tomllib 645

Parsing TOML with tomllib 646
Thesis . . . 646
Mental model . . . 646
Worked example . . . 646
Trap . . . 647
Boring rule . . . 647
Try this . . . 647
Sources . . . 647

TLS Trust Stores in Python: ssl, certifi, pip and the 2026 emSign Distrust 648

TLS Trust Stores in Python: ssl, certifi, pip and the 2026 emSign Distrust **649**
Thesis . . . 649
The Three Trust Copies . . . 649
The October 2026 Distrust in Brief . . . 650
The Audit Script Walk-Through . . . 650
Running the Audit Script . . . 652
Environment 1: Legacy Virtual Environment (un-patched `certifi`) . . . 652
Environment 2: Modern Virtual Environment (upgraded `certifi` and `pip`) . . . 653
Environment 3: System Interpreter (Debian/Ubuntu patched `pip`) . . . 653
Remediation Strategies . . . 654
1. Upgrade Application Dependencies (`certifi`) . . . 654
2. Upgrade Package Installer (`pip` and `uv`) . . . 655
3. Upgrade Operating System Trust Store . . . 655
4. Upgrade CPython Runtime . . . 656
Pinning, Lockfiles, and Container Environments . . . 656
Single Source of Truth: `truststore` . . . 656
Common Mistakes . . . 657
Exercises . . . 657
Key takeaways . . . 657
Sources . . . 658

Typing and Testing **659**

Modern Type Hints and Static Analysis **660**

Modern Type Hints and Static Analysis **661**
Mental model . . . 661
Nominal Subtyping vs Structural Subtyping (`Protocol`) . . . 661
Minimal example . . . 661
Worked examples . . . 662
Case 1: Generic Data Containers (`typing.Generic`) . . . 662
Case 2: Fluent Method Chaining with `typing.Self` . . . 663
Pitfalls . . . 664
Pitfall 1: Confusing `Type[T]` with `T` . . . 664
Pitfall 2: Using `Union` Instead of the Modern `|` Operator . . . 664
Exercises . . . 665
Further reading . . . 665

Unit and Integration Testing with Pytest **666**

Unit and Integration Testing with Pytest **667**
Mental model . . . 667
Minimal example . . . 667
Worked examples . . . 669
Case 1: Fixtures with Setup and Teardown Lifecycles . . . 669
Case 2: Isolating External Dependencies with `unittest.mock` . . . 669
Pitfalls . . . 670
Pitfall 1: Mutating Shared Session Fixtures . . . 670

Pitfall 2: Testing Implementation Details Instead of Contracts . . . 670
Exercises . . . 671
Further reading . . . 671

Packaging, Wheel Distribution, and Publishing 672

Packaging, Wheel Distribution, and Publishing 673
Mental model . . . 673
Minimal example . . . 673
Worked examples . . . 675
Case 1: Defining CLI Console Scripts via `[project.scripts]` . . . 675
Case 2: Building Wheels with `uv build` . . . 675
Case 3: Inspecting Wheel Metadata (`.dist-info`) . . . 675
Pitfalls . . . 676
Pitfall 1: Relying on Legacy `setup.py` . . . 676
Pitfall 2: Missing Package Data Files . . . 676
Exercises . . . 677
Further reading . . . 677

Wheel Variants (PEP 825 and PEP 850): Labels, variant.json, Providers and the Selection Order 678

Wheel Variants (PEP 825 and PEP 850): Labels, variant.json, Providers and the Selection Order 679
Thesis . . . 679
Mental model . . . 679
Worked example . . . 681
Reading it . . . 681
Pitfalls . . . 682
Try this . . . 683
Sources . . . 683

Advanced Type System: PEP 695, ParamSpec, and Type Narrowing 685

Advanced Type System: PEP 695, `ParamSpec`, and Type Narrowing 686
Mental model . . . 686
Type Narrowing: `TypeGuard` vs `TypeIs` . . . 686
Minimal example . . . 687
Worked examples . . . 688
Case 1: Signature-Preserving Decorators with `ParamSpec` and `Concatenate` . . . 688
Case 2: Bidirectional Narrowing with `TypeIs` (PEP 742) . . . 689
Case 3: Compile-Time Injection Immunity with `LiteralString` (PEP 675) . . . 690
Pitfalls . . . 691
Pitfall 1: Assuming `TypeGuard` Narrows the `else` Branch . . . 691
Pitfall 2: Forgetting `assert_never` in Exhaustive Pattern Matching . . . 692
Exercises . . . 692
Further reading . . . 692

Deferred Annotations 694

Deferred Annotations **695**
Mental model . . . 695
Worked examples . . . 695
Case 1: Forward ref without quotes . . . 695
Case 2: All three formats on a resolved function . . . 696
Case 3: Class annotation before the name exists . . . 697
Case 4: Evaluate when you need types at runtime . . . 698
Case 5: The `__annotations__` / `get_type_hints` habit . . . 699
The trap . . . 700
The boring rule . . . 701
Try this . . . 701

Closed TypedDicts and Typed Extra Items (PEP 728) **703**

Closed TypedDicts and Typed Extra Items (PEP 728) **704**
Thesis . . . 704
Mental model . . . 704
Worked example . . . 705
Reading the output . . . 707
Pitfalls . . . 707
Exercises . . . 708
Sources . . . 708

Disjoint Bases: Telling Checkers Which Classes Can't Mix (PEP 800) **709**

Disjoint Bases: Telling Checkers Which Classes Can't Mix (PEP 800) **710**
Thesis . . . 710
Mental model . . . 710
Worked example . . . 711
Runtime output . . . 712
pyright 1.1.414 . . . 713
ty 0.0.95 . . . 713
mypy 2.4.1 . . . 714
Reading the output . . . 714
Pitfalls . . . 714
Exercises . . . 715
Sources . . . 715

TypeForm: Typing Functions That Take Types as Values (PEP 747) **716**

TypeForm: Typing Functions That Take Types as Values (PEP 747) **717**
Thesis . . . 717
Mental model . . . 717
Worked example . . . 718
A runtime checker with an honest signature . . . 718
Using it: what the checkers infer . . . 719
The cases that must fail . . . 721
Reading it . . . 721
Pitfalls . . . 722

Try this . . . 722
Sources . . . 722

TypeVarTuple Array Shapes **724**

TypeVarTuple Array Shapes **725**
Thesis . . . 725
Mental model . . . 725
Worked examples . . . 726
Case 1: A dtype and a tuple of axis types . . . 726
Case 2: A square the checker accepts . . . 728
Case 3: The same rank, a different last axis . . . 729
Case 4: Rank is the length of the pack . . . 730
Case 5: Concatenate an axis, or move one to the end . . . 730
Case 6: `Unpack[tuple[int, ...]]` is any number of `int` axes . . . 734
Case 7: `*args: *Ts` . . . 736
Case 8: The 3.15 constructor, and today's checkers . . . 737
The trap . . . 739
The boring rule . . . 740
Try this . . . 741
Sources . . . 741

Choosing a Type Checker: mypy, pyright, ty and Pyrefly **743**

Choosing a Type Checker: mypy, pyright, ty and Pyrefly **744**
Thesis . . . 744
Mental model . . . 745
Worked examples . . . 746
Case 1: Annotated code with a plain bug . . . 746
Case 2: The body of an unannotated function . . . 747
Case 3: Inference the specification leaves open . . . 748
Case 4: Calling code that has no annotations . . . 749
Case 5: Reading the conformance suite yourself . . . 750
Case 6: A score is a snapshot . . . 752
Case 7: Speed on a real package . . . 754
Case 8: What each language server offers . . . 756
The trap . . . 758
The boring rule . . . 760
Try this . . . 761
Sources . . . 761

Concurrency and Internals **763**

CPython VM, Bytecode, and Code Execution **764**

CPython VM, Bytecode, and Code Execution **765**
Mental model . . . 765
Minimal example . . . 765

Worked examples . . . 767
Case 1: The Adaptive Specializing Interpreter (PEP 659) . . . 767
Case 2: Inspecting Call Stack Frames with `sys._getframe()` . . . 767
Pitfalls . . . 768
Pitfall 1: Modifying Code Objects at Runtime . . . 768
Exercises . . . 769
Further reading . . . 769

Memory Allocator, PyObject, and Garbage Collection 770

Memory Allocator, PyObject, and Garbage Collection 771
Mental model . . . 771
Minimal example . . . 772
Worked examples . . . 773
Case 1: Reference Counting Mechanics with `sys.getrefcount()` . . . 773
Case 2: Leak-Free Caching with `weakref.WeakValueDictionary` . . . 773
Pitfalls . . . 774
Pitfall 1: Believing `del` Immediately Frees OS RAM . . . 774
Pitfall 2: Disabling the Garbage Collector in Long-Running Daemons . . . 775
Exercises . . . 775
Further reading . . . 775

The Global Interpreter Lock and Python 3.14 Free-Threading 776

The Global Interpreter Lock and Python 3.14 Free-Threading 777
Mental model . . . 777
Why Did the GIL Exist? . . . 777
How PEP 703 Removes the GIL . . . 778
Minimal example . . . 778
Worked examples . . . 779
Case 1: CPU-Bound Parallel Scaling Benchmark . . . 779
Case 2: Why Thread Locks Are Still Mandatory in Free-Threaded Python . . . 780
Pitfalls . . . 781
Pitfall 1: Assuming "No GIL" Means No Thread Synchronization . . . 781
Pitfall 2: Legacy C-Extensions Without Free-Threading Support . . . 781
Exercises . . . 781
Further reading . . . 782

Stable ABI for Free-Threaded Builds (PEP 803 / abi3t) 783

Stable ABI for Free-Threaded Builds (PEP 803 / abi3t) 784
Thesis . . . 784
Mental model . . . 784
Worked examples . . . 785
Case 1: Prove the knob exists . . . 785
Case 2: Module export hook sketch (PEP 793) . . . 786
Case 3: One wheel vs two — the trap table . . . 787
The trap . . . 788
The boring rule . . . 788

Try this . . . 788
Sources . . . 788

Multithreading, Multiprocessing, and Process Pools **789**

Multithreading, Multiprocessing, and Process Pools **790**
Mental model . . . 790
Minimal example . . . 790
Worked examples . . . 792
Case 1: Thread-Safe Producer-Consumer Pipeline with `queue.Queue` . . . 792
Case 2: Zero-Copy Shared Memory Across Processes . . . 793
Pitfalls . . . 794
Pitfall 1: Forgetting `if __name__ == "__main__":` in Multiprocessing . . . 794
Pitfall 2: High IPC Pickling Overhead . . . 794
Exercises . . . 794
Further reading . . . 794

Subinterpreters **795**

Subinterpreters **796**
Mental model . . . 796
Worked examples . . . 797
Case 1: Create, call, close . . . 797
Case 2: Isolation with a queue . . . 797
Case 3: `InterpreterPoolExecutor` . . . 798
Case 4: Thread + queue for a result . . . 799
The trap . . . 800
The boring rule . . . 801
Try this . . . 801

InterpreterPoolExecutor **802**

InterpreterPoolExecutor **803**
Mental model . . . 803
Worked examples . . . 804
Case 1: Map CPU chunks across interpreters . . . 804
Case 2: `submit` + `as_completed` . . . 804
Case 3: When vs Thread vs Process . . . 805
Case 4: Initializer runs in each worker interpreter . . . 806
The trap . . . 807
The boring rule . . . 808
Try this . . . 808

NumPy Limited API and heap types (toward subinterpreters) **809**

NumPy Limited API and heap types (toward subinterpreters) **810**
Learning goals . . . 810
Thesis . . . 810
Mental model . . . 811

Case 1 — Why the build matrix hurts . . . 811
Case 2 — Static type → heap type . . . 811
Case 3 — Where NumPy stands (2026-10) . . . 812
Case 4 — What you write once NumPy is ready . . . 813
Case 5 — If you maintain SciPy-like code . . . 814
The trap . . . 814
The boring rule . . . 814
Try this . . . 815
Sources . . . 815
Inspiration . . . 815
Verification . . . 815

Sharing Iterators Between Threads: serialize_iterator, synchronized_iterator and concurrent_tee 816

Sharing Iterators Between Threads: serialize_iterator, synchronized_iterator and concurrent_tee 817
The thread safety problem with iterators . . . 817
Observing the race: Unsynchronized generators . . . 818
Single-consumer delivery with serialize_iterator . . . 820
Function-level synchronization with synchronized_iterator . . . 822
Broadcast streaming with concurrent_tee . . . 823
The throughput reality of serialized iterators . . . 825
Backward compatibility: A Python 3.14 fallback . . . 825
Pitfalls and common mistakes . . . 827
1. Divergent consumers causing memory leaks in concurrent_tee . . . 827
2. Nested yields holding locks . . . 827
3. Closing a shared generator while workers are using it . . . 828
4. Expecting free-threaded performance gains without decoupling . . . 828
Summary . . . 828
Sources . . . 828

Asynchronous Programming and TaskGroups 829

Asynchronous Programming and TaskGroups 830
Mental model . . . 830
Structured Concurrency with `asyncio.TaskGroup` . . . 830
Minimal example . . . 831
Worked examples . . . 832
Case 1: Deterministic Deadlines with `asyncio.timeout()` . . . 832
Case 2: Throttling Concurrent Requests with `asyncio.Semaphore` . . . 833
Pitfalls . . . 834
Pitfall 1: Calling Synchronous Blocking Code in Coroutines . . . 834
Pitfall 2: Using Legacy `asyncio.gather()` Without Shielding . . . 834
Exercises . . . 834
Further reading . . . 834

When Does an asyncio Task Start? Lazy Coroutines, Eager Tasks, and Task Lifetimes 835

When Does an asyncio Task Start? Lazy Coroutines, Eager Tasks, and Task Lifetimes 836
Thesis . . . 836
Mental model . . . 836
Worked examples . . . 838
Case 1: Calling is not running . . . 838
Case 2: The fire-and-forget program in asyncio . . . 839
Case 3: Creation order versus start order . . . 841
Case 4: What eager start buys . . . 844
Case 5: The task that vanished . . . 845
Case 6: The exception nobody retrieves . . . 847
Case 7: Ending a group early with `TaskGroup.cancel()` (3.15) . . . 847
The trap . . . 849
The boring rule . . . 851
Try this . . . 852
Sources . . . 852

Blocking the Event Loop: Finding Sync Calls in Async Code and Moving Them Off the Loop 853

Blocking the Event Loop: Finding Sync Calls in Async Code and Moving Them Off the Loop 854
Thesis . . . 854
Mental model . . . 854
Worked examples . . . 855
Case 1: Measure the loop with a heartbeat . . . 856
Case 2: What the stall does to other requests . . . 858
Case 3: Let debug mode name the slow task . . . 860
Case 4: A watchdog that names the line . . . 861
Case 5: The shared pool is a limit too . . . 863
Case 6: Choosing where blocking work goes . . . 864
The trap . . . 865
The boring rule . . . 865
Try this . . . 865
Sources . . . 866

ContextVar for Request-Scoped State 867

ContextVar for Request-Scoped State 868
Thesis . . . 868
Mental model . . . 868
Worked examples . . . 868
Trap . . . 869
Boring rule . . . 869
Try this . . . 869
Sources . . . 869

TaskGroup Structured Concurrency 870

TaskGroup Structured Concurrency **871**
Thesis . . . 871
Mental model . . . 871
Worked example . . . 871
Trap . . . 872
Boring rule . . . 872
Try this . . . 872
Sources . . . 872

Async call graphs: asking asyncio who is awaiting whom **873**

Async call graphs: asking asyncio who is awaiting whom **874**
Thesis . . . 874
Mental model . . . 874
Case 1: print the await chain of a leaf task . . . 875
Case 2: an async-aware sampling profiler in 50 lines . . . 877
The trap: an in-loop sampler cannot see what blocks the loop . . . 878
Boring rule . . . 879
Exercises . . . 879
Sources . . . 879

Profiling, Optimization, and Native C/Rust Interoperability **880**

Profiling, Optimization, and Native C/Rust Interoperability **881**
Mental model . . . 881
Minimal example . . . 881
Worked examples . . . 883
Case 1: Locating Memory Leaks with `tracemalloc` . . . 883
Case 2: Calling Native C Functions with `ctypes.Structure` . . . 884
Case 3: Modern High-Performance Extensions: PyO3 and Rust . . . 884
Pitfalls . . . 885
Pitfall 1: Premature Micro-Optimization . . . 885
Pitfall 2: Memory Hazards in `ctypes` . . . 885
Exercises . . . 885
Further reading . . . 885

Attaching to a Live Process **886**

Attaching to a Live Process **887**
Mental model . . . 887
Worked examples . . . 888
Case 1: Run a probe inside a running worker . . . 888
Case 2: Dump every thread's stack and bring it home . . . 890
Case 3: `pdb -p` — a debugger on a process you did not start under one . . . 891
Case 4: See every asyncio task with `ps` and `pstree` . . . 892
The trap . . . 894
The boring rule . . . 896
Try this . . . 896

Sampling Profiler: Tachyon (profiling.sampling) 897

Sampling Profiler: Tachyon (`profiling.sampling`) 898
Thesis . . . 898
Mental model . . . 898
Worked examples . . . 899
Case 1: Find the hotspot in a report job . . . 899
Case 2: Wall, CPU, and GIL: three questions, three modes . . . 902
Case 3: Same data, many views: flame graphs, collapsed stacks, replay . . . 904
Case 4: What it costs the program you measure . . . 905
Case 5: Profiling a live process (attach and dump) . . . 906
The trap . . . 907
The boring rule . . . 908
Try this . . . 909
Sources . . . 909

Frame Pointers by Default (PEP 831) 910

Frame Pointers by Default (PEP 831) 911
Thesis . . . 911
Mental model . . . 911
Worked examples . . . 912
Case 1: ask the interpreter what it was built with . . . 912
Case 2: audit the interpreter's own code . . . 914
Case 3: an extension inherits the flags, and can opt out . . . 914
Case 4: read the "without" list before panicking . . . 915
The trap . . . 916
The boring rule . . . 916
Try this . . . 917
Sources . . . 917

sys.monitoring: Low-Overhead Debuggers and Coverage (PEP 669) 918

`sys.monitoring`: Low-Overhead Debuggers and Coverage (PEP 669) 919
Thesis . . . 919
Mental model . . . 919
Worked examples . . . 921
Case 1: The smallest tool: count function starts . . . 921
Case 2: Where the speed actually comes from . . . 922
Case 3: A line + branch coverage tool in about 65 lines . . . 924
Case 4: Check it against the real tool . . . 926
Case 5: A breakpoint that costs nothing until it hits . . . 927
Case 6: `breakpoint()` already uses it . . . 929
Case 7: 3.15 lets exception events be per code object . . . 930
The trap . . . 931
The boring rule . . . 933
Try this . . . 933
Sources . . . 933

The Experimental JIT: Measure Before You Flip PYTHON_JIT=1 **935**

The Experimental JIT: Measure Before You Flip `PYTHON_JIT=1` **936**
Thesis . . . 936
Mental model . . . 936
Worked example: four workloads, two versions . . . 937
Reading the output . . . 939
`is_active()` is not a warm-up probe . . . 939
A boring rule . . . 940
Pitfalls . . . 940
Exercises . . . 941
Sources . . . 941

Capstone Lab: High-Throughput Asynchronous Log Processing Engine **942**

Capstone Lab: High-Throughput Asynchronous Log Processing Engine **943**
Architectural blueprint . . . 943
Complete runnable engine . . . 943
Running the capstone . . . 947
Architectural takeaways . . . 948

Tracing np.add: ufunc dispatch internals **949**

Tracing `np.add`: ufunc dispatch internals **950**
Thesis . . . 950
Mental model . . . 950
Worked examples . . . 951
Case 1: `np.add` is an object, not a plain function . . . 951
Case 2: The `__array_ufunc__` escape hatch . . . 951
Case 3: Which SIMD variant did you import? . . . 952
Case 4: Same additions, different strides — the inner loop tax . . . 952
Case 5: When the heavy iterator appears . . . 953
The trap . . . 953
The boring rule . . . 954
Try this . . . 954
Verification . . . 954
Sources . . . 954

Data Structures and Algorithms **955**

Algorithmic Complexity and Big-O **956**

Algorithmic Complexity and Big-O **957**
Mental model . . . 957
Worked examples . . . 958
Case 1: Dict lookup O(1) vs List linear search O(n) . . . 958
Case 2: Detecting duplicates (Quadratic nested loops vs Linear Set) . . . 959
The trap . . . 960

The boring rule . . . 961
Try this . . . 962

Stacks, Queues, and Deques **963**

Stacks, Queues, and Deques **964**
Mental model . . . 964
Worked examples . . . 965
Case 1: LIFO Stack for Expression and Delimiter Validation . . . 965
Case 2: High-Performance FIFO Queue with collections.deque . . . 966
Case 3: Fixed-size Ring Buffer (Circular Buffer) . . . 967
The trap . . . 969
The boring rule . . . 969
Try this . . . 969

Linked Lists **970**

Linked Lists **971**
Mental model . . . 971
Python List vs Linked List: The Cache Reality . . . 972
Worked examples . . . 972
Case 1: Singly Linked List with in-place reversal . . . 972
Case 2: Doubly Linked List with O(1) Node Removal . . . 974
The trap . . . 976
The boring rule . . . 976
Try this . . . 976

Binary Search Trees **977**

Binary Search Trees **978**
Mental model . . . 978
Worked examples . . . 979
Case 1: Generic BST with Insert, Search, and In-Order Generator . . . 979
Case 2: Deleting Nodes (Handling 0, 1, and 2 Children) . . . 981
The trap . . . 983
The boring rule . . . 983
Try this . . . 984

Heaps and Priority Queues **985**

Heaps and Priority Queues **986**
Mental model . . . 986
Worked examples . . . 987
Case 1: Min-Heap from scratch using index math . . . 987
Case 2: Priority Queue with standard library heapq . . . 989
The trap . . . 990
The boring rule . . . 990
Try this . . . 991

Tries and Prefix Trees **992**

Tries and Prefix Trees **993**
Mental model . . . 993
Worked examples . . . 993
Case 1: Catalog SKU and Keyword Autocomplete . . . 993
The trap . . . 996
The boring rule . . . 996
Try this . . . 996

Sorting Algorithms **997**

Sorting Algorithms **998**
Mental model . . . 998
Stable Sorting . . . 999
Worked examples . . . 1000
Case 1: Adaptive Insertion Sort for nearly-sorted batches . . . 1000
Case 2: Stable Divide-and-Conquer Merge Sort . . . 1000
Case 3: In-Place Quicksort with Lomuto Partition . . . 1002
The trap . . . 1003
The boring rule . . . 1004
Try this . . . 1004

Binary Search and the bisect Module **1005**

Binary Search and the bisect Module **1006**
Mental model . . . 1006
Binary Search Halving . . . 1006
The `bisect` Invariant . . . 1006
Worked examples . . . 1006
Case 1: Binary Search from scratch . . . 1006
Case 2: Production Range and Tier Queries with standard library bisect . . . 1008
Case 3: Quickselect for Median Response Time . . . 1009
The trap . . . 1010
The boring rule . . . 1011
Try this . . . 1011

Graphs and Traversals (BFS and DFS) **1012**

Graphs and Traversals (BFS and DFS) **1013**
Mental model . . . 1013
Adjacency List Representation . . . 1013
BFS vs DFS Traversals . . . 1014
Worked examples . . . 1014
Case 1: BFS for Shortest Hop Delivery Routes . . . 1014
Case 2: DFS for Workflow Cycle and Deadlock Detection . . . 1016
The trap . . . 1017
The boring rule . . . 1018
Try this . . . 1018

Shortest Paths (Dijkstra's Algorithm) **1019**

Shortest Paths (Dijkstra's Algorithm) **1020**
Mental model . . . 1020
Worked examples . . . 1021
Case 1: Delivery Network Optimization with heapq . . . 1021
The trap . . . 1023
The boring rule . . . 1023
Try this . . . 1023

String Search and Bloom Filters **1024**

String Search and Bloom Filters **1025**
Mental model . . . 1025
Rabin-Karp Rolling Hash . . . 1025
Bloom Filter: Probabilistic Membership . . . 1025
Worked examples . . . 1026
Case 1: Rabin-Karp Rolling Hash Substring Search . . . 1026
Case 2: In-Memory Bloom Filter for Order Existence . . . 1027
The trap . . . 1029
The boring rule . . . 1029
Try this . . . 1029

Long Term Engineering 1030

inspect and Reflection **1031**

inspect and Reflection **1032**
Mental model . . . 1032
Worked examples . . . 1032
Case 1: `inspect.signature` . . . 1032
Case 2: `getattr` with a default . . . 1033
Case 3: `inspect.getdoc` . . . 1033
The trap . . . 1034
The boring rule . . . 1036
Try this . . . 1036

ctypes and FFI **1037**

ctypes and FFI **1038**
Mental model . . . 1038
Worked examples . . . 1038
Case 1: `c_int` and `sizeof` (no extra C files) . . . 1038
Case 2: A tiny C-like struct . . . 1039
Case 3: The Python you wanted instead . . . 1040
The trap . . . 1040
The boring rule . . . 1041
Try this . . . 1041

Performance Tuning **1042**

Performance Tuning **1043**
Mental model . . . 1043
Worked examples . . . 1043
Case 1: Get the answer right, then time it . . . 1043
Case 2: `cProfile` names the function . . . 1044
Case 3: Two implementations, same answer . . . 1045
The trap . . . 1046
The boring rule . . . 1046
Try this . . . 1047

Designing for the Long Term **1048**

Designing for the Long Term **1049**
Mental model . . . 1049
Worked examples . . . 1049
Case 1: Two modules, one public function . . . 1049
Case 2: `pyproject.toml` is the project boundary . . . 1050
Case 3: One process that still looks like the first chapter . . . 1051
The trap . . . 1051
The boring rule . . . 1052
Try this . . . 1052

Appendices **1053**

Glossary **1054**

Glossary **1055**

Command Cheat Sheet **1057**

Command Cheat Sheet **1058**
Python . . . 1058
uv . . . 1058
ruff . . . 1058
pytest . . . 1059
Tiny reminders . . . 1059

Python

A comprehensive, self-contained engineering volume covering Python from scratch across the major domains of modern computing: **DevOps, Network Automation, Data Science, AI/GenAI, Robotics, Cybersecurity, and Distributed Web Systems**.

Part 1 (Python Foundations) begins with modern toolchain setup using **`uv`** and **`ruff`**, moves through core syntax and OOP, and provides an **exhaustive deep dive into the Python Standard Library (`stdlib`)** before covering modern typing, testing, packaging, concurrency, and CPython internals in a direct topic progression. Dedicated parts follow for each primary engineering domain, concluding with cross-domain **Enterprise Production Capstones**.

Baseline: Python **3.14** (free-threading enabled) · toolchain **`uv`** · linter/formatter **`ruff`**. Completely self-contained. No other title in this library is required.

Start here

Open **Basics** in the sidebar. You need a computer, a terminal, and any text editor. Zero prior Python experience required.

All examples are complete, runnable files:

```
uv run python hello.py
```

Book Roadmap & Master Architecture

Part	Slug	Domain / Focus	Key Concepts & Technologies
01	`01-python-foundations`	Python Foundations	Modern setup (`uv`, `ruff`), syntax, memory, data types, loops, functions, OOP, dataclasses, **comprehensive Standard Library (`collections`, `itertools`, `re`, `pathlib`, `json`, `sqlite3`, `datetime`, `subprocess`, `logging`, `socket`)**, typing (`mypy`), testing (`pytest`), packaging, concurrency, CPython bytecode & free-threading
02	`02-data-structures-and-algorithms`	Data Structures & Algorithms	Algorithmic complexity, stacks, queues, linked lists, binary search trees, heaps, tries, sorting algorithms, graph traversals, Dijkstra, string search & Bloom filters
03	`03-long-term-engineering`	Long-Term Engineering	Runtime inspection & reflection, CTypes & native FFI, performance profiling, and designing for maintainability
04	`02-devops-and-cloud-engineering`	DevOps, SRE & Cloud (In Development)	Terminal UIs (`rich`/`typer`), `docker-py`, Kubernetes client, Pulumi IaC, OpenTelemetry, Boto3
05	`03-network-automation`	NetDevOps & Telemetry (In Development)	`netmiko`, `scrapli`, `napalm`, `nornir`, NETCONF (`ncclient`), RESTCONF, gNMI, NetBox, pyATS

Part	Slug	Domain / Focus	Key Concepts & Technologies
06	`04-data-science-and-analytics`	Vaex, Engines & Analytics (In Development)	NumPy, Pandas 2, Polars (`LazyFrame`), DuckDB OLAP, Apache Arrow, SciPy, Plotly
07	`05-artificial-intelligence`	ML, Deep Learning & GenAI (In Development)	Scikit-Learn, PyTorch 2.x, Transformers, RAG (Qdrant), Instructor, LangGraph agents
08	`06-robotics-and-simulation`	Robotics & Physical Systems (In Development)	ROS 2 (`rclpy`), PyBullet physics, OpenCV vision, ArUco fiducials, A*/RRT, MicroPython
09	`07-cybersecurity-and-automation`	Threat & Security Automation (In Development)	Raw sockets, Scapy packet crafting, `cryptography` (AES/RSA), `pefile`, SIEM triage
10	`08-web-and-distributed-systems`	Web APIs & Distributed Systems (In Development)	ASGI, FastAPI, WebSockets, background queues (ARQ/Redis), OAuth2/JWT, SQLAlchemy 2.0
11	`09-capstone-systems`	Enterprise Capstones (In Development)	Autonomous SRE Agent, NetDevOps Intent Pipeline, Vision-Guided Robotic Telemetry
99	`99-appendices`	Reference & Appendices	Glossary, syntax cheat sheet, and toolchain reference

Architectural Conventions

- **Modern Tooling Upfront**: Environment bootstrapping, runtime version pinning, and code hygiene are taught with `uv` and `ruff` on day one.
- **Exhaustive Standard Library**: Complete coverage of Python's built-in batteries (`collections`, `itertools`, `pathlib`, `sqlite3`, `subprocess`, `socket`, etc.) without skipping to third-party dependencies prematurely.

- **Original Prose & Complete Files**: Full runnable source files instead of truncated snippets.
- **Progressive Typing**: Pure syntax in initial chapters; formal type annotations (`mypy`/`pyright`) introduced starting with modern craft chapters.
- **Domain Autonomy**: Domain mathematics, protocol structures, and operating system mechanics are taught directly in-situ.

Formats

Available in HTML, PDF, and EPUB via the monorepo build pipeline.

Python Foundations

Intro

What Is Python

What Is Python

After reading this chapter, you will understand how Python source code compiles to bytecode, executes inside the CPython virtual machine, and leverages the Python 3.14 free-threaded runtime architecture.

Mental model

Python is an interpreted, strongly and dynamically typed language executed by a virtual machine. When you run a script, the standard CPython runtime does not interpret raw human-readable text line-by-line in real time. Instead, it parses source text into an Abstract Syntax Tree (AST), compiles the AST into a sequence of stack-based **bytecode instructions**, and executes those instructions inside an evaluation loop:

```
[ Your Code: hello.py ]


  Tokenizer &     Transforms source text into grammar tokens
  Parser

          AST (Abstract Syntax Tree)


  Compiler        Emits bytecode instructions & code objects (.pyc)

          Bytecode (e.g. LOAD_CONST, BINARY_OP, RETURN_VALUE)


  CPython Virtual Machine (ceval.c)

    Evaluation Loop     PyFrameObject (Value Stack)


    Memory Allocator (pymalloc) & Cyclic GC


    Python 3.14 Runtime: GIL-less Free-Threading
```

```
(PEP 703: multi-core true parallel execution)


                    [ Host OS / Hardware CPU ]
```

Every entity in Python—an integer, a string, a function, or a class—is represented internally as a heap-allocated C structure called a **`PyObject`**.

Minimal example

Save the following file as `sys_inspect.py`. It inspects your active Python implementation, execution flags, and runtime environment:

```python
# sys_inspect.py
import platform
import sys

def main() -> None:
    print(f"Implementation : {platform.python_implementation()}")
    print(f"Python Version : {sys.version.split()[0]}")
    print(f"Compiler       : {platform.python_compiler()}")
    print(f"Platform Arch  : {platform.machine()} ({sys.platform})")
    print(f"Bytecode Magic : {sys.implementation.cache_tag}")
    print(f"Float Info     : max={sys.float_info.max:.2e}, epsilon={sys.float_info.epsilon:.

if __name__ == "__main__":
    main()
```

Run this file using `uv`:

```
uv run python sys_inspect.py
```

Output:

```
Implementation : CPython
Python Version : 3.14.0
Compiler       : GCC 14.2.0 (or Clang)
Platform Arch  : x86_64 (linux)
Bytecode Magic : cpython-314
Float Info     : max=1.80e+308, epsilon=2.22e-16
```

Worked examples

1. Peeking inside the CPython compiler with `dis`

To see what the CPython virtual machine actually executes, you disassemble functions into bytecode using the standard library `dis` module.

Save this file as `disassemble_demo.py`:

```
# disassemble_demo.py
import dis

def compute_total(base_price: float, tax_rate: float) -> float:
    discount = 5.0
    final_price = (base_price - discount) * (1.0 + tax_rate)
    return final_price

def main() -> None:
    print("=== Disassembly of compute_total ===")
    dis.dis(compute_total)

if __name__ == "__main__":
    main()
```

Run the script:

```
uv run python disassemble_demo.py
```

Output:

```
=== Disassembly of compute_total ===
  4           0 LOAD_CONST               1 (5.0)
              2 STORE_FAST               2 (discount)

  5           4 LOAD_FAST                0 (base_price)
              6 LOAD_FAST                2 (discount)
              8 BINARY_OP               10 (-)
             12 LOAD_CONST               2 (1.0)
             14 LOAD_FAST                1 (tax_rate)
             16 BINARY_OP                0 (+)
             20 BINARY_OP                5 (*)
             24 STORE_FAST               3 (final_price)

  6          26 LOAD_FAST                3 (final_price)
             28 RETURN_VALUE
```

Why this matters: CPython is a **stack machine**. `LOAD_FAST` pushes a local variable onto the evaluation stack, `LOAD_CONST` pushes a constant literal, `BINARY_OP` pops the top operands, executes

the arithmetic in C, and pushes the result back onto the stack. Understanding this stack model demystifies performance profiling later in this book.

---

2. Inspecting runtime execution frames

While your code is executing, CPython maintains a call stack of **frame objects** (`PyFrameObject`). Each frame encapsulates the code object being executed, the local variable array, the global namespace dictionary, and the instruction pointer.

Save this file as `frame_demo.py`:

```
# frame_demo.py
import inspect
import sys

def nested_worker(task_id: int, payload: str) -> None:
    # Inspect the current active execution frame
    current_frame = sys._getframe()
    print(f"Current Function   : {current_frame.f_code.co_name}")
    print(f"Current File       : {current_frame.f_code.co_filename}")
    print(f"Current Line Number: {current_frame.f_lineno}")
    print(f"Local Names        : {list(current_frame.f_locals.keys())}")

    # Inspect the caller's frame
    caller_frame = current_frame.f_back
    if caller_frame:
        print(f"Caller Function    : {caller_frame.f_code.co_name}")
        print(f"Caller Locals      : {caller_frame.f_locals}")

def supervisor() -> None:
    job_batch = "batch-alpha"
    nested_worker(task_id=101, payload=job_batch)

if __name__ == "__main__":
    supervisor()
```

Run the script:

```
uv run python frame_demo.py
```

Output:

```
Current Function   : nested_worker
Current File       : frame_demo.py
Current Line Number: 8
```

```
Local Names        : ['task_id', 'payload', 'current_frame']
Caller Function    : supervisor
Caller Locals      : {'job_batch': 'batch-alpha'}
```

Why this matters: Tracebacks, debuggers (like `pdb`), and profilers are not magic—they walk this linked list of `f_back` frame references.

---

3. Checking Python 3.14 free-threading (GIL status)

Python 3.14 incorporates the milestone implementation of **PEP 703** (making the Global Interpreter Lock optional). When built with free-threading enabled, Python can run multiple threads across physical CPU cores simultaneously without being serialized by a global mutex.

Save this file as `gil_status.py`:

```python
# gil_status.py
import sys

def check_threading_runtime() -> None:
    # sys._is_gil_enabled is available in free-threaded CPython builds
    is_gil_fn = getattr(sys, "_is_gil_enabled", None)

    print("=== Python 3.14 Concurrency Architecture ===")
    if is_gil_fn is not None:
        gil_active = is_gil_fn()
        status = "ENABLED (Serialized threads)" if gil_active else "DISABLED (True multi-cor
        print(f"Global Interpreter Lock (GIL): {status}")
    else:
        print("Global Interpreter Lock (GIL): Active (Standard CPython build)")

    print(f"Recursion Limit              : {sys.getrecursionlimit()}")
    print(f"Thread Switch Interval       : {sys.getswitchinterval():.4f}s")

if __name__ == "__main__":
    check_threading_runtime()
```

Run the script:

```
uv run python gil_status.py
```

Output:

```
=== Python 3.14 Concurrency Architecture ===
Global Interpreter Lock (GIL): ENABLED (Serialized threads)
Recursion Limit              : 1000
```

```
Thread Switch Interval       : 0.0050s
```

Pitfalls

1. Thinking Python is "just interpreted"

The Trap: Believing Python reads lines as raw strings while executing, and assuming `.pyc` files are native machine code.
The Reality: Python compiles source code to bytecode first. If the file has not changed, Python reads the bytecode cached in `__pycache__/*.pyc` directly to save startup compilation time. Bytecode is instructions for the CPython VM, not machine code for x86_64 or ARM.

2. Modifying objects during iteration

The Trap: Deleting or inserting items into a list while looping over it.

```
# BROKEN
items = [1, 2, 3, 4, 5]
for x in items:
    if x % 2 == 0:
        items.remove(x)  # Skips elements due to shifted indices!
```

The Fix: Iterate over a shallow copy or use a comprehension:

```
# CORRECT
items = [x for x in items if x % 2 != 0]
```

3. Mixing tabs and spaces

The Trap: In Python, indentation defines block scope. Mixing a tab character with spaces causes an immediate `TabError` at the parser level before any code runs. Always configure your editor to insert 4 spaces per indent.

Exercises

1. Modify `sys_inspect.py` to also display `sys.byteorder` (big-endian vs. little-endian) and `sys.api_version`.
2. Write a script `disassemble_loop.py` that defines a function with a `for` loop, disassembles it with `dis.dis()`, and identifies the opcode responsible for jumping back to the beginning of the loop (`JUMP_BACKWARD` or `FOR_ITER`).
3. Using `sys.getrefcount()`, write a script `refcount_check.py` that displays the reference count of an empty list when created, when assigned to a second variable name, and after `del` is called on the second variable.
4. Verify your local Python 3.14 build flags by running `uv run python -c "import sysconfig; print(sysconfig.get_config_vars('Py_DEBUG', 'Py_GIL_DISABLED'))"`.

Further reading

- **Official Documentation**: *CPython Internal Design & Architecture Notes* (`https://docs.python.org/3.14`)
- **PEP 703**: *Making the Global Interpreter Lock Optional in CPython* (Sam Gross).
- **PEP 659**: *Specializing Adaptive Interpreter (Faster CPython Project).*
- **Book**: *CPython Internals: Your Guide to the Python 3 Interpreter* by Anthony Shaw.

Modern Toolchain: uv

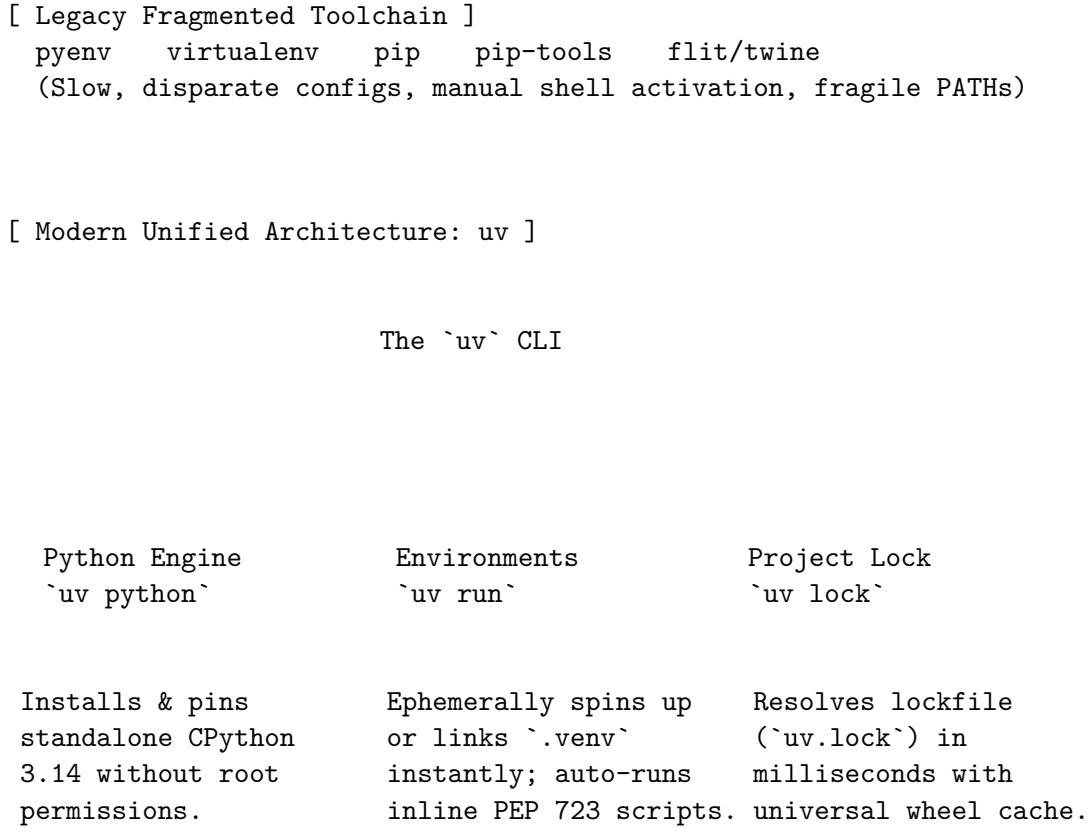
The Modern Toolchain with uv

After reading this chapter, you will be able to install and switch Python versions, run isolated scripts with inline dependencies without manual virtual environment management, and manage deterministic projects using `uv`.

Mental model

Historically, Python developers were forced to stitch together 5 to 7 separate tools: `pyenv` to install Python versions, `virtualenv` to isolate packages, `pip` to install wheels, `pip-tools` or `poetry` to resolve dependency locks, and `twine` to publish distributions.

Astral's **`uv`** replaces this entire fragmented ecosystem with a single, ultra-fast Rust-based binary:



With `uv`, you no longer need to remember to "activate" a virtual environment in your shell. You prefix commands with `uv run`, and `uv` ensures the exact Python interpreter and locked dependencies are resolved automatically.

Minimal example

Save the following file as `minimal_env.py`. This script inspects the running Python executable and verifies whether it is executing inside an isolated virtual environment:

```
# minimal_env.py
import os
import sys

def main() -> None:
    in_venv = sys.prefix != sys.base_prefix
    print(f"Active Python Executable : {sys.executable}")
    print(f"Isolated Virtualenv      : {in_venv}")
    print(f"Prefix Directory         : {sys.prefix}")
    print(f"Base Interpreter Path    : {sys.base_prefix}")
    print(f"Process PID              : {os.getpid()}")

if __name__ == "__main__":
    main()
```

Run this script using `uv run`:

```
uv run python minimal_env.py
```

Output:

```
Active Python Executable : /home/user/myproject/.venv/bin/python
Isolated Virtualenv      : True
Prefix Directory         : /home/user/myproject/.venv
Base Interpreter Path    : /home/user/.local/share/uv/python/cpython-3.14.0...
Process PID              : 412850
```

Worked examples

1. Single-file scripts with inline dependencies (PEP 723)

Traditionally, sharing a Python script that required a library like `httpx` or `rich` forced you to write a separate `requirements.txt` file and instruct users to create and activate a virtualenv.

Under **PEP 723** (supported natively by `uv`), you declare your script's dependencies directly inside the file header. `uv` parses this header, provisions an isolated ephemeral cache, and runs the script seamlessly.

Save this file as `weather_fetch.py`:

```
# /// script
# requires-python = ">=3.14"
# dependencies = [
#     "httpx>=0.27.0",
# ]
# ///
# weather_fetch.py
import httpx

def get_current_utc_time() -> None:
    # Query a lightweight, public JSON endpoint
    url = "https://httpbin.org/get"
    headers = {"User-Agent": "ModernPythonGuide/1.0"}

    print(f"Sending HTTP GET request to {url}...")
    with httpx.Client(timeout=5.0) as client:
        response = client.get(url, headers=headers)
        response.raise_for_status()
        data = response.json()

    print(f"HTTP Status: {response.status_code}")
    print(f"Origin IP  : {data.get('origin')}")
    print(f"Headers Sent:")
    for k, v in data.get("headers", {}).items():
        print(f"  {k}: {v}")

if __name__ == "__main__":
    get_current_utc_time()
```

Run the script directly:

```
uv run weather_fetch.py
```

Why this matters: You did not have to `pip install httpx` globally. `uv` read the `# /// script` block, verified whether `httpx` was present in its global content-addressable cache, downloaded it in milliseconds if needed, and executed the script in complete isolation.

---

2. Managing Python runtimes with `uv python`

You do not need to install Python from your operating system's system package manager (`apt`, `dnf`, or `brew`), which often ships outdated versions or risks breaking system tools.

Inspect available Python versions:

```
uv python list
```

Install Python 3.14 directly into an unprivileged user directory:

```
uv python install 3.14
```

Pin your project directory to use Python 3.14:

```
uv python pin 3.14
```

This creates a `.python-version` file containing:

```
3.14
```

Now, every `uv` command executed in this folder or its subfolders will automatically invoke Python 3.14.

---

3. Creating and locking a modern project

To start a production project, initialize a standard workspace:

```
uv init my_app
cd my_app
```

This generates a minimal, standard `pyproject.toml`:

```
[project]
name = "my-app"
version = "0.1.0"
description = "Production application scaffold"
readme = "README.md"
requires-python = ">=3.14"
dependencies = []
```

Add a dependency:

```
uv add "pydantic>=2.10.0"
```

Instantly, `uv` performs dependency resolution, updates `pyproject.toml`, creates a local `.venv/`, and generates a deterministic `uv.lock`.

Save this code in `my_app/main.py`:

```
# main.py
from pydantic import BaseModel, Field

class ServiceNode(BaseModel):
    hostname: str
    ip_address: str
    port: int = Field(default=8080, ge=1, le=65535)
    is_active: bool = True

def main() -> None:
    node = ServiceNode(hostname="edge-router-01", ip_address="192.168.1.1", port=443)
    print(f"Validated Node: {node.hostname} -> {node.ip_address}:{node.port} (active={node.i
    print(f"JSON Payload  : {node.model_dump_json()}")

if __name__ == "__main__":
    main()
```

Run your application:

```
uv run python main.py
```

Output:

```
Validated Node: edge-router-01 -> 192.168.1.1:443 (active=True)
JSON Payload  : {"hostname":"edge-router-01","ip_address":"192.168.1.1","port":443,"is_activ
```

Pitfalls

1. Manually activating virtualenvs in every terminal tab

The Trap: Running `source .venv/bin/activate`, opening a new terminal tab, forgetting to activate, and running against system Python.
The Fix: Never rely on shell prompt activation. Use `uv run python script.py` or configure your editor to point directly to `.venv/bin/python`. `uv run` guarantees the correct environment is active every time.

2. Hand-editing `uv.lock`

The Trap: Manually editing versions or hashes in `uv.lock`.
The Fix: Treat `uv.lock` like a compiled binary. Modify requirements in `pyproject.toml` or run `uv add` / `uv remove`, and let `uv lock` regenerate the lockfile. Commit `uv.lock` to Git so all team members and CI runners build identical environments.

3. Polluting the system Python with `sudo pip`

The Trap: Running `sudo pip install <package>`, which corrupts Linux distribution packages (`/usr/lib/python3/dist-packages`) and can break system utilities.
The Fix: Modern Linux distros trigger PEP 668 (`externally-managed-environment`) error. Use `uv tool install <cli>` for global CLI binaries (e.g. `uv tool install ruff`), and `uv venv` or `uv run` for code dependencies.

Exercises

1. Run `uv python list` in your terminal and determine which CPython versions are currently installed on your workstation.
2. Create a single-file script `hash_tool.py` using PEP 723 inline script metadata that depends on `cryptography>=44.0.0` and prints the version of the installed cryptography library using `uv run hash_tool.py`.
3. In an empty directory, initialize a project with `uv init`, add `pytest` as a development dependency (`uv add --dev pytest`), and verify that `pyproject.toml` differentiates runtime dependencies from `[dependency-groups] dev`.
4. Inspect the generated `uv.lock` file in your text editor. Locate the exact wheel URL and SHA-256 hash that `uv` locked for your dependency.

Further reading

- **Official Documentation**: *uv — An extremely fast Python package and project manager* (`https://docs.astral.sh/uv/`).
- **PEP 723**: *Inline Script Metadata* (Brett Cannon, Ofek Lev).
- **PEP 621**: *Storing project metadata in pyproject.toml.*
- **PEP 668**: *Marking Python base environments as "externally managed".*

Code Hygiene: Ruff

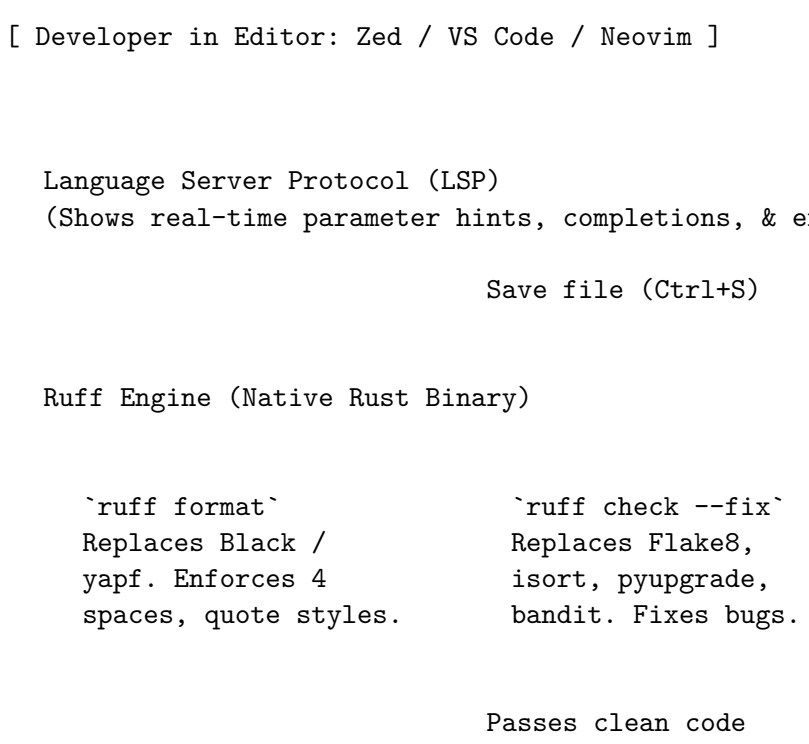
Developer Environment & Code Hygiene with Ruff

After reading this chapter, you will be able to configure a professional editor with the Language Server Protocol (LSP), enforce automated code formatting and linting with `ruff` in milliseconds, and integrate pre-commit quality gates into your git repositories.

Mental model

Writing clean, production-grade Python requires two layers of automated feedback: 1. **Real-time feedback**: Your editor (Zed, VS Code, or Neovim) communicates over the Language Server Protocol (LSP) with an LSP server (such as `basedpyright` or `pyright`) to show type errors and syntax diagnostics as you type. 2. **Deterministic enforcement**: A formatter and linter ensures that every file in your repository conforms to standard formatting rules (line lengths, import sorting, obsolete syntax upgrades, and common bug detection).

Astral's **`ruff`** combines the responsibilities of Black, Flake8, isort, pydocstyle, pyupgrade, and bandit into a single Rust binary that operates 10 to 100 times faster than legacy Python-based linters:



```
Git Pre-Commit Hook & Automated CI Pipeline
(Rejects malformed PRs before code reaches production)
```

Minimal example

To understand what a linter does, look at this intentionally unformatted and messy script.

Save this file as `messy_sample.py`:

```python
# messy_sample.py
import sys, os
from math import *
import json

def calculate_average( numbers:list ) -> float:
    total=0.0
    for n in numbers:
        total = total+n
    return total / len(numbers) if len(numbers)>0 else 0.0

def main():
    vals=[10, 20, 30, 40]
    print(f"Average: {calculate_average(vals)}")

if __name__ == "__main__":
    main()
```

Notice the issues in this file: 1. Unused imports (`sys`, `os`, `json`). 2. Wildcard import (`from math import *`) which pollutes the namespace. 3. Inconsistent spacing inside parentheses and around operators. 4. Outdated type annotation syntax (`list` instead of `list[float]`).

Run the script to verify it still works:

```
uv run python messy_sample.py
```

Now, check the file using `ruff`:

```
uv run ruff check messy_sample.py
```

Ruff flags the unused imports and wildcard import immediately:

```
messy_sample.py:2:8: F401 `sys` imported but unused
messy_sample.py:2:13: F401 `os` imported but unused
```

```
messy_sample.py:3:1: F403 `from math import *` used; unable to detect undefined names
messy_sample.py:4:8: F401 `json` imported but unused
```

Now, format the file using `ruff format`:

```
uv run ruff format messy_sample.py
```

Ruff normalizes all whitespace, operator spacing, and indentation in under 2 milliseconds.

Worked examples

1. Configuring `ruff` in `pyproject.toml`

Rather than relying on ad-hoc CLI flags, professional projects configure linter and formatter rules declaratively inside `pyproject.toml`.

Here is the standard, production-grade `[tool.ruff]` configuration for modern Python 3.14 projects:

```toml
# pyproject.toml
[project]
name = "production-service"
version = "0.1.0"
requires-python = ">=3.14"

[tool.ruff]
target-version = "py314"
line-length = 100
src = ["src", "tests"]

[tool.ruff.lint]
# Selected rule suites:
# E / W : Pycodestyle (standard PEP 8 formatting errors & warnings)
# F     : Pyflakes (detects undefined names, unused variables & imports)
# I     : isort (automatic alphabetical import sorting)
# UP    : pyupgrade (upgrades syntax to modern Python 3.14 idioms)
# B     : flake8-bugbear (detects common bugs and design pitfalls)
# SIM   : flake8-simplify (suggests cleaner, idiomatic syntax)
select = ["E", "W", "F", "I", "UP", "B", "SIM"]

ignore = [
    "E501",  # Line length handled automatically by ruff format
]
```

```toml
[tool.ruff.lint.isort]
combine-as-imports = true
force-single-line = false

[tool.ruff.format]
quote-style = "double"
indent-style = "space"
skip-magic-trailing-comma = false
line-ending = "auto"
```

Why this matters: When anyone on your team runs `uv run ruff check` or saves a file in their editor, the same exact rules are applied deterministically across Linux, macOS, and Windows.

---

2. How linters work: building an AST inspector

Linters do not execute your code; they inspect its **Abstract Syntax Tree (AST)**. You can use Python's built-in `ast` module to write your own custom code inspector.

Save this file as `ast_security_lint.py`:

```python
# ast_security_lint.py
import ast
import sys

# Target code string to inspect
CODE_SAMPLE = """
import os

def load_user_input(user_str: str) -> None:
    # Danger: eval() allows arbitrary code execution!
    result = eval(user_str)
    print("Result:", result)
"""

class SecurityASTVisitor(ast.NodeVisitor):
    def __init__(self) -> None:
        self.violations: list[str] = []

    def visit_Call(self, node: ast.Call) -> None:
        # Check if the function being called is eval() or exec()
        if isinstance(node.func, ast.Name) and node.func.id in {"eval", "exec"}:
            self.violations.append(
                f"Line {node.lineno}: Dangerous call to '{node.func.id}()' detected!"
            )
        # Continue traversing child AST nodes
```

```
        self.generic_visit(node)

def main() -> None:
    print("Parsing source code into Abstract Syntax Tree...")
    parsed_tree = ast.parse(CODE_SAMPLE)

    visitor = SecurityASTVisitor()
    visitor.visit(parsed_tree)

    if visitor.violations:
        print("Security Linter Findings:")
        for v in visitor.violations:
            print(f"  [CRITICAL] {v}")
    else:
        print("Code passed all AST checks.")

if __name__ == "__main__":
    main()
```

Run the script:

```
uv run python ast_security_lint.py
```

Output:

```
Parsing source code into Abstract Syntax Tree...
Security Linter Findings:
  [CRITICAL] Line 6: Dangerous call to 'eval()' detected!
```

Why this matters: `ruff` performs this exact AST traversal, but does so in compiled Rust across thousands of files per second.

---

3. Automated Git pre-commit hooks

To ensure no developer can commit malformed code, configure a pre-commit hook using `.pre-commit-config.yaml`.

Save this file in the root of your repository as `.pre-commit-config.yaml`:

```
# .pre-commit-config.yaml
repos:
  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: v0.9.9
    hooks:
      # Run the linter
```

```
      - id: ruff
        args: [--fix]
      # Run the formatter
      - id: ruff-format
```

Install the pre-commit hook into `.git/hooks/`:

```
uv run pre-commit install
```

Now, whenever you run `git commit`, `ruff` runs automatically before the commit is created. If formatting is needed, `ruff` fixes the file and halts the commit so you can stage the clean changes.

Pitfalls

1. Running multiple competing formatters

The Trap: Installing Black, autopep8, and Ruff simultaneously in your editor, resulting in editor stuttering and formatting wars.
The Fix: Disable all legacy formatters. Use `ruff` exclusively for both linting and formatting. Ruff was explicitly engineered to be a drop-in replacement for Black and isort.

2. Blindly running `ruff check --fix` without review

The Trap: Running automated fix flags on large repositories and immediately pushing to `main` without testing.
The Fix: Some lint fixes (like removing unused variables or modifying comprehensions) can change runtime semantics if code relied on side effects. Always run your test suite (`uv run pytest`) after applying automated fixes.

3. Committing code without running formatting in CI

The Trap: Formatting locally, but forgetting that pull requests submitted by contributors might not have pre-commit hooks installed.
The Fix: Add a GitHub Actions workflow step: `uv run ruff check --no-fix` and `uv run ruff format --check`. If any file is unformatted, CI rejects the build.

Exercises

1. Create a script with deliberate whitespace and import errors, then use `uv run ruff format --diff <filename>` to inspect the unified diff output before writing changes to disk.
2. In your `pyproject.toml`, add rule `B006` (flake8-bugbear: mutable default arguments in functions). Write a function `def append_to(item, target=[])` and observe how `ruff check` catches the trap.
3. Configure your primary code editor (VS Code, Zed, or Neovim) to automatically run `ruff format` on file save. Verify that saving a file instantly formats it without lag.
4. Modify `ast_security_lint.py` to also detect calls to `os.system()` and raise a warning advising the use of `subprocess.run()`.

Further reading

- **Official Documentation**: *Ruff — An extremely fast Python linter and code formatter* (`https://docs.astral.sh/ruff/`).
- **PEP 8**: *Style Guide for Python Code* (Guido van Rossum, Barry Warsaw, Nick Coghlan).
- **PEP 518**: *Specifying Minimum Build System Requirements for Python Projects (`pyproject.toml`).*

Editor Setup: VS Code

Editor Setup: VS Code with Modern Tooling

After reading this chapter, you will be able to configure Visual Studio Code into a high-performance Python development environment—integrating `uv` virtual environments, the native `Ruff` extension for instantaneous format-on-save and auto-import sorting, static type analysis, and reproducible `.vscode/` workspace configurations.

Mental model

VS Code acts as an orchestration shell around three specialized background engines: the **Language Server** (syntax, autocompletion, and type analysis), the **Linter & Formatter** (Ruff), and the **Runtime Debugger** (`debugpy`).

Rather than relying on global editor preferences that vary across machines, professional projects define their toolchain declaratively inside a version-controlled `.vscode/` folder within the project repository:

```
[ Your VS Code Editor ]

              Workspace Configuration: `.vscode/settings.json`
                  Points Python extension to `${workspaceFolder}/.venv`
                  Sets default formatter to `charliermarsh.ruff`

              Recommended Extensions: `.vscode/extensions.json`
                  `charliermarsh.ruff` (Instant Rust-based format & lint)
                  `ms-python.python` (Core Python language support)
                  `tamasfe.even-better-toml` (Syntax for pyproject.toml)

              Debug Configurations: `.vscode/launch.json`
                   Launches Python scripts inside the `uv` virtualenv
```

When you save a file (`Ctrl+S` / `Cmd+S`), the Ruff extension intercepts the write, reorders imports alphabetically, upgrades obsolete syntax, normalizes whitespace, and returns the formatted buffer in less than 5 milliseconds.

Minimal example

To configure any project workspace for modern Python development, create a folder named `.vscode` in the root of your project directory and add `settings.json`.

Save this file as `.vscode/settings.json`:

```
{
  // 1. Interpreter & Virtual Environment Discovery
  "python.defaultInterpreterPath": "${workspaceFolder}/.venv/bin/python",
  "python.terminal.activateEnvironment": true,

  // 2. Formatting Engine: Delegate completely to Ruff
  "editor.formatOnSave": true,
  "[python]": {
    "editor.defaultFormatter": "charliermarsh.ruff",
    "editor.formatOnSave": true,
    "editor.codeActionsOnSave": {
      "source.fixAll.ruff": "explicit",
      "source.organizeImports.ruff": "explicit"
    }
  },

  // 3. Static Type Analysis via Pylance / Pyright
  "python.analysis.typeCheckingMode": "standard",
  "python.analysis.autoImportCompletions": true,
  "python.analysis.inlayHints.variableTypes": false,
  "python.analysis.inlayHints.functionReturnTypes": true,

  // 4. Test Explorer Integration with Pytest
  "python.testing.pytestEnabled": true,
  "python.testing.unittestEnabled": false,
  "python.testing.pytestArgs": ["tests"],

  // 5. Files & Explorer Hygiene
  "files.exclude": {
    "**/__pycache__": true,
    "**/.pytest_cache": true,
    "**/.ruff_cache": true
  }
}
```

Now, whenever you open this project in VS Code, all formatting, import sorting, lint fixes, and virtual environment bindings happen automatically on save.

Worked examples

1. Workspace extension recommendations (`.vscode/extensions.json`)

When collaborating in teams or across machines, you should prompt VS Code to install the exact toolchain required for the project.

Save this file as `.vscode/extensions.json`:

```
{
  "recommendations": [
    // Fast Rust-based linter and formatter (replaces Black, Flake8, isort)
    "charliermarsh.ruff",
    // Official Microsoft Python extension
    "ms-python.python",
    // High-performance language server and static type checking
    "ms-python.vscode-pylance",
    // Rich syntax highlighting for pyproject.toml and uv.lock
    "tamasfe.even-better-toml"
  ]
}
```

Why this matters: When a new contributor opens the repository, VS Code displays a notification: *"This repository recommends installing extensions."* Clicking **Install All** equips their workspace with the exact linter, formatter, and type checker without manual search.

---

2. Interactive debugging with `.vscode/launch.json`

VS Code can execute your scripts step-by-step, pause on breakpoints, inspect local variables, and evaluate expressions live in the Debug Console.

Save this file as `.vscode/launch.json`:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Python: Current File",
      "type": "debugpy",
      "request": "launch",
      "program": "${file}",
      "console": "integratedTerminal",
      "justMyCode": true,
      "env": {
        "PYTHONUNBUFFERED": "1"
```

```
      }
    },
    {
      "name": "Python: Run CLI with Arguments",
      "type": "debugpy",
      "request": "launch",
      "program": "${workspaceFolder}/src/main.py",
      "args": ["--config", "config.toml", "--verbose"],
      "console": "integratedTerminal",
      "justMyCode": true
    },
    {
      "name": "Python: Debug Pytest",
      "type": "debugpy",
      "request": "launch",
      "module": "pytest",
      "args": ["-v", "${file}"],
      "console": "integratedTerminal",
      "justMyCode": false
    }
  ]
}
```

To test the debugger: 1. Open any Python file. 2. Click to the left of a line number to set a red **breakpoint** dot. 3. Press `F5` (or select **Run → Start Debugging**). 4. Execution pauses immediately at your breakpoint, populating the **Variables** and **Call Stack** panels.

---

3. Testing format-on-save and auto-import sorting

To verify that your VS Code workspace is operating correctly, create a test script with intentional formatting defects.

Save this file as `editor_verify.py`:

```python
# editor_verify.py
import sys
import os
import json
from math import sqrt, pi, sin

def compute_circle_metrics(radius: float) -> dict[str, float]:
    area = pi * (radius ** 2)
    circumference = 2 * pi * radius
    return {"radius": radius, "area": area, "circumference": circumference}
```

```python
def main() -> None:
    metrics = compute_circle_metrics(5.0)
    print(f"Metrics: {metrics}")

if __name__ == "__main__":
    main()
```

Notice: - `sys`, `os`, `json`, `sqrt`, and `sin` are unused imports. - Now press `Ctrl+S` (or `Cmd+S` on macOS).

What happens automatically: 1. Ruff removes the unused imports. 2. Ruff re-formats `from math import pi` onto a clean single line. 3. Ruff normalizes operators and quote styles according to your `pyproject.toml`.

The file is saved instantly in its clean canonical form.

Pitfalls

1. Conflicting legacy extensions

The Trap: Leaving legacy extensions like `ms-python.black-formatter`, `ms-python.isort`, or `ms-python.flake8` enabled alongside `charliermarsh.ruff`.
The Reality: Two formatters will attempt to format the document simultaneously on save, causing visible cursor jumping, undo-stack corruption, and editor latency.
The Fix: Uninstall or disable Black, isort, and Flake8 extensions. Ruff handles all three functionalities natively.

2. VS Code selecting the wrong Python interpreter

The Trap: VS Code defaulting to global system Python (`/usr/bin/python3`) instead of the project's local virtual environment (`.venv/bin/python`).
The Fix: Press `Ctrl+Shift+P` (or `Cmd+Shift+P`), type **Python: Select Interpreter**, and choose the interpreter located in your project's `./.venv` folder. Specifying `"python.defaultInterpreterPath": "${workspaceFolder}/.venv/bin/python"` in `.vscode/settings.json` makes this automatic for anyone opening the folder.

3. Expecting editor tools to run in external shells

The Trap: Assuming that because code formats on save in VS Code, team members using terminal commands or git commits will follow the same rules.
The Fix: Always combine editor configurations with command-line validation. Use `uv run ruff check` and `uv run ruff format --check` in your CI pipeline, and enforce `.pre-commit-config.yaml` git hooks so standards are upheld everywhere.

Exercises

1. Create a `.vscode/settings.json` file in a project workspace and verify that enabling `"python.analysis.inlayHints.functionReturnTypes": true` displays inferred return types for unannotated functions in your editor.
2. Open the **Run and Debug** view (`Ctrl+Shift+D`), create a breakpoint on a function inside `editor_verify.py`, press `F5`, and use the **Debug Console** to evaluate expressions dynamically while paused.
3. Open the **Testing** panel (flask icon) in the VS Code sidebar. Write a simple test function `test_math(): assert 1 + 1 == 2` in a `test_demo.py` file, and verify that VS Code discovers and displays a green play button next to the test.
4. Modify your `.vscode/settings.json` to configure rulers at 88 and 100 characters by adding `"editor.rulers": [88, 100]` to visualize line length boundaries.

Further reading

- **VS Code Python Documentation**: *Getting Started with Python in VS Code* (`https://code.visualstudio.com/docs/languages/python`).
- **Ruff VS Code Extension**: *Official Ruff extension on VS Code Marketplace* (`https://marketplace.visual`
- **Pylance Documentation**: *Fast, feature-rich Python language support in VS Code* (`https://marketplace.visualstudio.com/items?itemName=ms-python.vscode-pylance`).
- **Astral Documentation**: *Integrating uv with VS Code* (`https://docs.astral.sh/uv/guides/integratio`

Variables

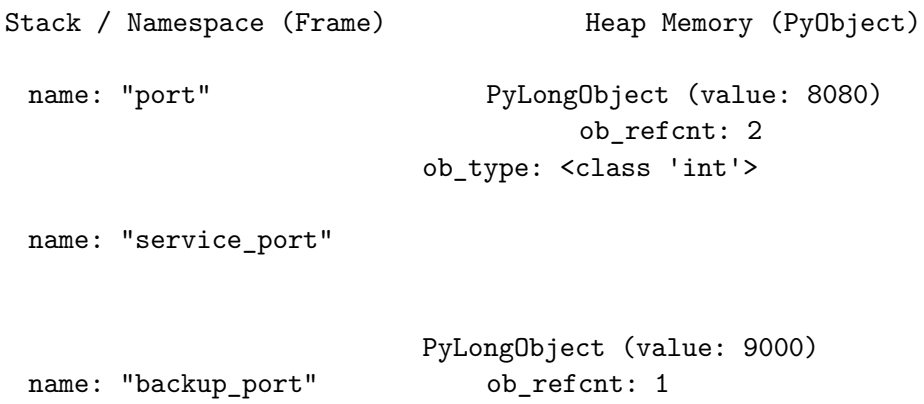
Names, Binding, and Object References

Names, Binding, and Object References

After reading this chapter, you will understand how Python binds identifiers to heap-allocated objects, how assignment actually works in CPython, and how to inspect memory references with confidence.

Mental model

In compiled languages like C or Go, a variable is a named memory slot with a fixed size and type. In Python, **variables are not boxes holding values**; they are **name tags (pointers) bound to objects** living on the heap.



When you execute `port = 8080`, Python creates a `PyLongObject` representing `8080` in heap memory, and binds the string key `"port"` in the current namespace dictionary to that object's memory address. When you execute `service_port = port`, no integer is copied; both identifiers simply reference the exact same memory address.

Minimal example

Save the following file as `binding_demo.py` and run it via `uv run python binding_demo.py`:

```
# binding_demo.py
def main() -> None:
```

```
    # 1. Bind name "a" to an integer object
    a = 1000
    # 2. Bind name "b" to the same object "a" points to
    b = a

    print(f"a = {a}, id(a) = {hex(id(a))}")
    print(f"b = {b}, id(b) = {hex(id(b))}")
    print(f"a is b: {a is b}")

    # 3. Rebind "a" to a completely new integer object
    a = 2000
    print("\nAfter rebinding a = 2000:")
    print(f"a = {a}, id(a) = {hex(id(a))}")
    print(f"b = {b}, id(b) = {hex(id(b))}")
    print(f"a is b: {a is b}")

if __name__ == "__main__":
    main()
```

Output:

```
a = 1000, id(a) = 0x7f884120
b = 1000, id(b) = 0x7f884120
a is b: True

After rebinding a = 2000:
a = 2000, id(a) = 0x7f884140
b = 1000, id(b) = 0x7f884120
a is b: False
```

Rebinding `a` did not mutate the number `1000`. It simply changed what `a` points to.

Worked examples

Case 1: Inspecting the namespace dictionary

Python implements namespaces as standard hash tables (dictionaries). You can inspect them directly using `locals()` and `globals()`.

```
# namespace_inspect.py
def inspect_scope() -> None:
    region = "us-east-1"
    replicas = 3
```

```
    scope = locals()
    print("Local namespace keys:", [k for k in scope if not k.startswith("_")])
    print(f"Value of region: {scope['region']}")
    print(f"Value of replicas: {scope['replicas']}")

if __name__ == "__main__":
    inspect_scope()
```

Run:

```
uv run python namespace_inspect.py
```

Output:

```
Local namespace keys: ['region', 'replicas']
Value of region: us-east-1
Value of replicas: 3
```

Why: Every variable lookup in Python begins by searching these internal namespace mapping tables.

Case 2: Binding vs in-place mutation

Because names are references, mutating an object in place affects all names pointing to it. Rebinding a name does not.

```
# mutate_vs_rebind.py
def demonstrate() -> None:
    # Scenario A: In-place mutation
    list_x = [1, 2, 3]
    list_y = list_x
    list_x.append(4)
    print("Scenario A (Mutation):")
    print(f"list_x: {list_x}")
    print(f"list_y: {list_y} (affected because both share the same reference!)")

    # Scenario B: Rebinding
    list_x = [10, 20]
    print("\nScenario B (Rebinding list_x):")
    print(f"list_x: {list_x}")
    print(f"list_y: {list_y} (unaffected because list_x was rebound to a new object)")

if __name__ == "__main__":
    demonstrate()
```

Run:

```
uv run python mutate_vs_rebind.py
```

Output:

```
Scenario A (Mutation):
list_x: [1, 2, 3, 4]
list_y: [1, 2, 3, 4] (affected because both share the same reference!)

Scenario B (Rebinding list_x):
list_x: [10, 20]
list_y: [1, 2, 3, 4] (unaffected because list_x was rebound to a new object)
```

Why: `list_x.append()` alters the existing object in heap memory. `list_x = [10, 20]` creates a new list and updates the pointer in `list_x`.

Pitfalls

Pitfall 1: Expecting assignment to copy compound objects

```python
# Dangerous:
original = {"host": "db.internal", "port": 5432}
copy_config = original
copy_config["port"] = 5433  # Also alters original["port"]!

# Correct fix:
copy_config = original.copy()  # Shallow copy for flat dictionaries
```

Pitfall 2: Attempting to modify immutable objects

Numbers, strings, and tuples are immutable. When you write `s = "hello"; s += " world"`, Python does not extend `"hello"` in place; it allocates a brand-new string `"hello world"` and rebinds `s`.

Exercises

1. Create a script where two variables `p` and `q` point to the same empty list. Append an integer using `p`. Print `q` and verify that `id(p) == id(q)`.
2. Create a script with an integer `x = 500`. Print its `id()`. Multiply `x` by 2 (`x *= 2`). Print `id(x)` again. Why did the memory address change?
3. Write a function `swap_names()` that demonstrates swapping two variable references using tuple packing: `a, b = b, a`. Verify that their memory identities switch places.
4. Using `sys.getrefcount()`, observe the reference count of a newly created custom list before and after creating two aliases to it.

Further reading

- Python Documentation: *Execution Model and Naming* (Docs -> Reference -> Execution Model).
- PEP 3104: *Access to Names in Outer Scopes*.
- Ned Batchelder: *Facts and Myths about Python names and values*.

Identity, Equality, and Mutability

Identity, Equality, and Mutability

After reading this chapter, you will master the fundamental distinction between identity (`is`) and equality (`==`), know how CPython caches small integers and strings, and avoid shared mutable state bugs.

Mental model

In Python, every object has three core attributes: 1. **Identity**: The memory address where the object resides (`id(obj)`). 2. **Type**: What kind of object it is (e.g. `int`, `str`, `list`), which never changes. 3. **Value**: The data stored in the object.

```
      Equality (==) compares VALUES

  "production"            "production"



Object A                Object B
id: 0x104a              id: 0x208f


     Identity (is) compares MEMORY IDs
     (Are they the exact same object?)
```

- `a == b`: Evaluates `a.__eq__(b)`. Asks: **Do these two objects have equivalent contents?**
- `a is b`: Evaluates `id(a) == id(b)`. Asks: **Are these two variables pointing to the exact same memory address?**

Minimal example

Save this file as `identity_demo.py`:

```
# identity_demo.py
def main() -> None:
    # Two distinct list objects with identical contents
```

```
    list_a = [10, 20, 30]
    list_b = [10, 20, 30]

    print(f"list_a == list_b : {list_a == list_b} (Values match)")
    print(f"list_a is list_b : {list_a is list_b} (Different objects in memory)")
    print(f"id(list_a)       : {hex(id(list_a))}")
    print(f"id(list_b)       : {hex(id(list_b))}")

    # The singleton None must ALWAYS be checked with 'is'
    status = None
    print(f"status is None   : {status is None}")

if __name__ == "__main__":
    main()
```

Run via `uv run python identity_demo.py`:

```
list_a == list_b : True (Values match)
list_a is list_b : False (Different objects in memory)
id(list_a)       : 0x7f5110a0
id(list_b)       : 0x7f511120
status is None   : True
```

Worked examples

Case 1: The small integer caching mechanism

CPython pre-allocates and caches small integers in the range `[-5, 256]` during startup. Any reference to an integer in this range shares the pre-allocated singleton instance.

```
# int_cache.py
def test_caching() -> None:
    # Inside the cache range [-5, 256]
    x = 250
    y = 250
    print(f"250 is 250: {x is y} (Shared singleton from CPython cache)")

    # Outside the cache range (typically > 256)
    big1 = 1000
    big2 = 1000
    print(f"1000 is 1000: {big1 is big2} (Distinct heap allocations)")
    print(f"1000 == 1000: {big1 == big2} (Values are still equal)")

if __name__ == "__main__":
```

```
    test_caching()
```

Run:

```
uv run python int_cache.py
```

Output:

```
250 is 250: True (Shared singleton from CPython cache)
1000 is 1000: False (Distinct heap allocations)
1000 == 1000: True (Values are still equal)
```

Why: Never rely on `is` for integer or string equality. Always use `==` for values.

Case 2: Shallow copy vs Deep copy

When copying nested mutable collections, a shallow copy copies references to inner objects, while a deep copy recursively duplicates everything.

```
# copy_semantics.py
import copy

def main() -> None:
    original = {"cluster": "prod", "nodes": ["node-1", "node-2"]}

    # Shallow copy
    shallow = original.copy()
    # Deep copy
    deep = copy.deepcopy(original)

    # Mutate the nested list
    original["nodes"].append("node-3")

    print(f"Original nodes: {original['nodes']}")
    print(f"Shallow nodes : {shallow['nodes']} (Mutated because inner list reference was sha
    print(f"Deep nodes    : {deep['nodes']} (Isolated because inner list was cloned)")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python copy_semantics.py
```

Pitfalls

Pitfall 1: Comparing values using `is`

```
# Bug: Works sometimes due to caching, fails unexpectedly in production!
if user_input is "admin":  # WRONG! Raises SyntaxWarning in modern Python
    pass

# Correct:
if user_input == "admin":  # Correct value equality check
    pass
```

Pitfall 2: Mutating objects inside tuples

A `tuple` is immutable, meaning its reference slots cannot be changed. However, if a slot points to a mutable object (like a `list`), that list can still be mutated in place!

```
t = (1, [2, 3])
t[1].append(4)  # Perfectly valid: the list inside the tuple is modified!
print(t)        # (1, [2, 3, 4])
```

Exercises

1. Write a script comparing two identical strings constructed at runtime (`s1 = "hello_world"` vs `s2 = "".join(["hello", "_", "world"])`). Compare them with `is` and with `==`.
2. Inspect `id(None)` across multiple variables set to `None`. Verify they all resolve to the exact same pointer address.
3. Construct a dictionary with a nested dictionary. Perform both a shallow copy and a `copy.deepcopy()`. Demonstrate modifying a nested value in the original and check both copies.

Further reading

- CPython Internal Source: `Objects/longobject.c` (small integer cache definition).
- Python Documentation: `copy` module (*Shallow and deep copy operations*).
- PEP 8: *Programming Recommendations (Comparisons to singletons like None).*

Naming Conventions, Constants, and Unpacking

Naming Conventions, Constants, and Unpacking

After reading this chapter, you will write clean, idiomatic PEP 8 Python code, leverage sequence unpacking, and understand how constants are declared and enforced.

Mental model

Python does not have a language-level `const` keyword. Instead, constant conventions are enforced through **naming standards** and static analysis tools like `ruff` and `mypy`.

`Naming Categories:`

Pattern	Convention	Example
snake_case	Functions, variables, files	db_connection_timeout
UPPER_SNAKE_CASE	Constants, globals	MAX_RETRIES, HTTP_OK
PascalCase (CapWords)	Classes, Exception types	DatabasePool, APIError
_leading_underscore	Internal / Private marker	_internal_buffer
__dunder__	Special Python protocols	__init__, __repr__

Unpacking allows you to extract elements from an iterable directly into named variables in a single atomic statement without manual index indexing.

Minimal example

Save as `unpacking_demo.py`:

```python
# unpacking_demo.py
def main() -> None:
    # Atomic variable swap without temporary variable
    left = "left_hand"
    right = "right_hand"
    left, right = right, left
    print(f"Swapped: left={left}, right={right}")
```

```
    # Extended star unpacking
    endpoint_data = ("GET", "/api/v1/users", 200, "0.12ms", "192.168.1.10")
    method, path, status, *telemetry = endpoint_data

    print(f"Method    : {method}")
    print(f"Path      : {path}")
    print(f"Status    : {status}")
    print(f"Telemetry : {telemetry} (packed into a list)")

if __name__ == "__main__":
    main()
```

Run via `uv run python unpacking_demo.py`:

```
Swapped: left=right_hand, right=left_hand
Method    : GET
Path      : /api/v1/users
Status    : 200
Telemetry : ['0.12ms', '192.168.1.10'] (packed into a list)
```

Worked examples

Case 1: Star unpacking in loops and data processing

You can cleanly separate headers, body items, and footers from structured data stream records:

```
# process_records.py
def process_stream() -> None:
    records = [
        ("auth", "admin", "success", "10.0.0.1"),
        ("query", "select * from users", "success", "10.0.0.2"),
        ("error", "timeout on connection", "failed", "10.0.0.3"),
    ]

    for event_type, *details, ip in records:
        print(f"Event: {event_type:8s} | IP: {ip:12s} | Details: {' '.join(details)}")

if __name__ == "__main__":
    process_stream()
```

Run:

```
uv run python process_records.py
```

Case 2: Marking constant intent with `typing.Final`

While CPython allows rebinding uppercase variables, modern type checkers will flag modifications to variables marked with `Final`:

```
# constants_guard.py
from typing import Final

DATABASE_TIMEOUT_SECONDS: Final[int] = 30
MAX_CONNECTIONS: Final[int] = 100

def check_limits(requested: int) -> bool:
    return requested <= MAX_CONNECTIONS

if __name__ == "__main__":
    print(f"Timeout: {DATABASE_TIMEOUT_SECONDS}s")
    print(f"Acceptable: {check_limits(50)}")
```

Run:

```
uv run python constants_guard.py
```

Pitfalls

Pitfall 1: Unbalanced unpacking assignments

```
coords = (10, 20, 30)
x, y = coords  # ValueError: too many values to unpack (expected 2, got 3)

# Fix: Match length or use star expression:
x, y, _ = coords  # Ignore the 3rd value using dummy variable "_"
x, *rest = coords # Capture remaining values
```

Pitfall 2: Overriding built-in functions with variable names

Never name variables after built-in functions like `id`, `type`, `list`, `str`, `dict`, or `min`:

```
# Dangerous:
list = [1, 2, 3]  # Now the built-in "list()" constructor is masked in this scope!
```

Exercises

1. Write a script that unpacks a 5-element tuple into `first`, `middle` (a list of 3 elements using `*`), and `last`.
2. Write a script that unpacks nested coordinates: `entry = ("datacenter-east", (40.7128, -74.0060))`. Extract the name and latitude/longitude in a single unpacking line.
3. Use `ruff check` on a file containing non-PEP8 variable names (like `camelCaseVariable = 10`) to see how modern linters catch naming deviations.

Further reading

- PEP 8: *Style Guide for Python Code* (Naming Conventions section).
- PEP 3132: *Extended Iterable Unpacking*.
- Python Standard Library: `typing.Final`.

Variable Scopes, LEGB, and Object Lifetimes

Variable Scopes, LEGB, and Object Lifetimes

After reading this chapter, you will master Python's LEGB scope lookup order, control variable rebinding across scopes using `global` and `nonlocal`, and understand reference counting lifecycle management.

Mental model

Whenever Python resolves an identifier name in code, it searches four nested lexical scopes in strict order (**LEGB**):

```
Built-in Scope (len, range, print, Exception, id)

   Global / Module Scope (top-level functions, vars)

      Enclosing / Outer Scope (closures, outer def)

         Local Scope (inside the active function)
           Search starts here: [1]

             Falls back to: [2]

              Falls back to: [3]

               Falls back to: [4]
```

If the name is not found in any of the four tiers, Python raises `NameError`.

Minimal example

Save as `legb_demo.py`:

```
# legb_demo.py
# 1. Global scope
```

```python
app_env = "production"

def outer_service() -> None:
    # 2. Enclosing scope
    service_name = "auth_api"

    def inner_handler() -> None:
        # 3. Local scope
        request_id = "req-9481"
        print(f"Local     : {request_id}")
        print(f"Enclosing : {service_name}")
        print(f"Global    : {app_env}")
        print(f"Built-in  : {len(request_id)}")  # "len" resolved from built-in scope

    inner_handler()

if __name__ == "__main__":
    outer_service()
```

Run via `uv run python legb_demo.py`:

```
Local     : req-9481
Enclosing : auth_api
Global    : production
Built-in  : 8
```

Worked examples

Case 1: Mutating enclosing state with `nonlocal`

Without `nonlocal`, attempting to assign to an enclosing variable creates a new local variable instead. `nonlocal` binds the assignment directly to the enclosing frame:

```python
# stateful_counter.py
from collections.abc import Callable

def make_counter(start: int = 0) -> Callable[[], int]:
    count = start

    def increment() -> int:
        nonlocal count  # Rebinds the enclosing 'count' variable
        count += 1
        return count
```

```
    return increment

if __name__ == "__main__":
    counter = make_counter(start=10)
    print("Call 1:", counter())
    print("Call 2:", counter())
    print("Call 3:", counter())
```

Run:

```
uv run python stateful_counter.py
```

Output:

```
Call 1: 11
Call 2: 12
Call 3: 13
```

Case 2: Object lifecycle and reference counting

Python reclaims heap memory as soon as an object's reference count drops to zero (`pymalloc`).

```
# lifecycle_demo.py
import sys

class Resource:
    def __init__(self, name: str) -> None:
        self.name = name
        print(f"Resource '{self.name}' allocated")

    def __del__(self) -> None:
        print(f"Resource '{self.name}' deallocated from memory")

def main() -> None:
    print("Creating resource...")
    res = Resource("DatabaseSocket")
    print(f"Current reference count: {sys.getrefcount(res) - 1}")

    alias = res
    print(f"With alias, reference count: {sys.getrefcount(res) - 1}")

    print("Dropping references...")
    del res
    print("Dropping final alias...")
    del alias
    print("Function complete.")
```

```
if __name__ == "__main__":
    main()
```

Run:

```
uv run python lifecycle_demo.py
```

Output:

```
Creating resource...
Resource 'DatabaseSocket' allocated
Current reference count: 1
With alias, reference count: 2
Dropping references...
Dropping final alias...
Resource 'DatabaseSocket' deallocated from memory
Function complete.
```

Pitfalls

Pitfall 1: UnboundLocalError caused by shadowing assignment

If a variable is assigned anywhere inside a function, Python marks it as local for the *entire* function scope. Referencing it before that assignment fails:

```
# Bug:
count = 10

def increment() -> None:
    print(count)  # UnboundLocalError: cannot access local variable 'count' where it is not
    count = 20    # Because assignment exists here, Python treated 'count' as local from lin

# Fix: Use global declaration if mutating a global is truly intended:
def increment_fixed() -> None:
    global count
    print(count)
    count = 20
```

Exercises

1. Write a function that demonstrates each tier of LEGB by overriding a built-in name (e.g. defining a local `len = 100`) and verifying the local value shadows the built-in.
2. Build a generator or closure using `nonlocal` that generates unique incremental API transaction tokens: `TXN-0001`, `TXN-0002`, etc.
3. Write a small script demonstrating cyclic references between two objects and verify how the `gc` module detects them.

Further reading

- Python Language Reference: *Section 4.2: Naming and Binding.*
- PEP 3104: *Access to Names in Outer Scopes (`nonlocal`).*
- CPython Internal Documentation: *Garbage Collector Design and Reference Counting.*

Data Types

Integers and Arbitrary Precision

Integers and Arbitrary Precision

After reading this chapter, you will master Python's unbounded integer arithmetic, bitwise operations, numerical bases, and understand how CPython represents numbers without overflow.

Mental model

Unlike C, Java, or Go—where integers are fixed 32-bit or 64-bit hardware integers subject to overflow—Python's `int` has **arbitrary precision**. It can store integers as large as your system memory permits.

```
CPython PyLongObject Structure (Heap):

  ob_refcnt : 1                                          Reference count
  ob_type   : <class 'int'>                              Type pointer
  ob_size   : 3 (indicates 3 digits stored; sign)        Size in digits

  ob_digit[0] : 0x3fffffff (low 30 bits)
  ob_digit[1] : 0x3fffffff (next 30 bits)                Array of 30-bit
  ob_digit[2] : 0x00000001 (high bits)                   base-2^30 digits
```

CPython breaks large integers into chunks of 30 bits (on 64-bit systems) stored as an array of unsigned 32-bit digits. When numbers grow beyond one digit, CPython performs multi-precision schoolbook arithmetic automatically.

Minimal example

Save as `arbitrary_ints.py`:

```python
# arbitrary_ints.py
def main() -> None:
    # 256-bit cryptographic boundary
    max_uint256 = (1 << 256) - 1
    print(f"Max 256-bit integer:\n{max_uint256}")
```

```
    # Computing 100! (factorial) without overflow
    fact_100 = 1
    for i in range(1, 101):
        fact_100 *= i

    print(f"\n100! has {len(str(fact_100))} decimal digits:")
    print(f"{fact_100}")

if __name__ == "__main__":
    main()
```

Run via `uv run python arbitrary_ints.py`:

```
Max 256-bit integer:
115792089237316195423570985008687907853269984665640564039457584007913129639935

100! has 158 decimal digits:
933262154439441526816992388562667004907159682643816214685929638952175999932299156089414639761565
```

Worked examples

Case 1: Bit manipulation and permission masks

Python supports the full suite of bitwise operators: `&` (AND), `|` (OR), `^` (XOR), `~` (NOT), and `<<` / `>>` (shifts).

```
# bitmask_flags.py
# Standard UNIX file permission bits
READ    = 0b100  # 4
WRITE   = 0b010  # 2
EXECUTE = 0b001  # 1

def check_permissions(mask: int) -> None:
    can_read = bool(mask & READ)
    can_write = bool(mask & WRITE)
    can_exec = bool(mask & EXECUTE)
    print(f"Mask 0o{mask:o} ({bin(mask)}): r={can_read}, w={can_write}, x={can_exec}")

if __name__ == "__main__":
    perm = READ | WRITE
    check_permissions(perm)

    # Add execute permission
```

```
    perm |= EXECUTE
    check_permissions(perm)

    # Revoke write permission
    perm &= ~WRITE
    check_permissions(perm)
```

Run:

```
uv run python bitmask_flags.py
```

Output:

```
Mask 0o6 (0b110): r=True, w=True, x=False
Mask 0o7 (0b111): r=True, w=True, x=True
Mask 0o5 (0b101): r=True, w=False, x=True
```

Case 2: Integer division vs float division

In Python, `/` always performs **true float division**, while `//` performs **floor division**:

```
# division_rules.py
def main() -> None:
    print(f"7 / 2  = {7 / 2} (always returns float)")
    print(f"7 // 2 = {7 // 2} (floor division integer)")
    print(f"-7 // 2 = {-7 // 2} (floored toward negative infinity!)")
    print(f"7 % 2  = {7 % 2} (modulo remainder)")

    # divmod returns (quotient, remainder) simultaneously
    q, r = divmod(25, 4)
    print(f"divmod(25, 4) -> quotient={q}, remainder={r}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python division_rules.py
```

Pitfalls

Pitfall 1: Converting colossal integers to strings

Python enforces an integer string conversion security limit (4300 digits by default, configurable via `sys.set_int_max_str_digits`) to protect against quadratic-time denial of service attacks:

```
import sys
huge = 10**5000
# str(huge) -> ValueError: Exceeds the limit (4300 digits) for integer string conversion
```

Pitfall 2: Negative modulo differences from C

In C, `-7 % 3` evaluates to `-1`. In Python, `%` always shares the sign of the divisor (denominator): `-7 % 3 == 2` because `-7 = (-3 * 3) + 2`.

Exercises

1. Write a function `to_hex_and_bin(val: int)` that returns a formatted string showing an integer in decimal, hexadecimal (`0x...`), and binary (`0b...`).
2. Implement a function `count_set_bits(n: int) -> int` that counts how many binary `1`s exist in an integer using bitwise operations (`n & (n - 1)`).
3. Compute 2^{1000} and print the number of bits required to represent it using the integer method `.bit_length()`.

Further reading

- CPython Implementation: `Objects/longobject.c`.
- Python Documentation: `sys.set_int_max_str_digits`.
- PEP 237: *Unifying Long Integers and Integers*.

Floating-Point Numbers and Decimal Precision

Floating-Point Numbers and Decimal Precision

After reading this chapter, you will understand the IEEE 754 standard behind Python's `float`, avoid floating-point representation traps, and know when to use `decimal.Decimal` and `fractions.Fraction`.

Mental model

Python's `float` type is implemented as a standard C `double`—a 64-bit IEEE 754 binary floating-point number:

```
Sign   Exponent (11 bits)      Fraction / Mantissa (52 bits)
1b     biased binary scale     normalized significant digits
```

Because numbers are stored in base-2 (binary fractions), decimal numbers like `0.1` (1/10) cannot be represented exactly in binary, just as 1/3 cannot be represented in a finite number of decimal digits (0.3333...).

```
In base 10: 1/10 = 0.1 (exact)
In base 2 : 1/10 = 0.00011001100110011... (infinite repeating binary sequence!)
```

Therefore, `0.1 + 0.2` produces `0.30000000000000004` instead of `0.3`.

Minimal example

Save as `float_precision.py`:

```python
# float_precision.py
import math
from decimal import Decimal

def main() -> None:
    # Standard float arithmetic
    f1 = 0.1
```

```
    f2 = 0.2
    f_sum = f1 + f2
    print(f"Float 0.1 + 0.2  : {f_sum:.20f}")
    print(f"f_sum == 0.3     : {f_sum == 0.3} (Trap!)")
    print(f"math.isclose(...) : {math.isclose(f_sum, 0.3)} (Correct comparison)")

    # Exact decimal arithmetic
    d1 = Decimal("0.1")
    d2 = Decimal("0.2")
    d_sum = d1 + d2
    print(f"\nDecimal sum      : {d_sum}")
    print(f"d_sum == Decimal('0.3'): {d_sum == Decimal('0.3')}")

if __name__ == "__main__":
    main()
```

Run via `uv run python float_precision.py`:

```
Float 0.1 + 0.2  : 0.30000000000000004441
f_sum == 0.3     : False (Trap!)
math.isclose(...) : True (Correct comparison)

Decimal sum      : 0.3
d_sum == Decimal('0.3'): True
```

Worked examples

Case 1: Financial calculations with `decimal.Decimal`

Never use `float` for currency, invoices, or billing. Always use `Decimal` initialized with strings:

```
# billing_calculator.py
from decimal import Decimal, ROUND_HALF_UP

def calculate_invoice(subtotal_cents: str, tax_rate: str) -> None:
    subtotal = Decimal(subtotal_cents)
    tax = (subtotal * Decimal(tax_rate)).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
    total = subtotal + tax

    print(f"Subtotal : ${subtotal:>8}")
    print(f"Tax ({tax_rate}): ${tax:>8}")
    print(f"Total    : ${total:>8}")

if __name__ == "__main__":
```

```
    calculate_invoice("149.95", "0.0825")
```

Run:

```
uv run python billing_calculator.py
```

Output:

```
Subtotal : $  149.95
Tax (0.0825): $   12.37
Total    : $  162.32
```

Case 2: Exact rational arithmetic with `fractions.Fraction`

When you need exact symbolic fraction arithmetic without rounding:

```
# rational_math.py
from fractions import Fraction

def main() -> None:
    f1 = Fraction(1, 3)
    f2 = Fraction(1, 6)
    result = f1 + f2
    print(f"1/3 + 1/6 = {result} (Exact reduction to 1/2)")

    # Converting float to nearest fraction
    approx = Fraction(0.75)
    print(f"0.75 as Fraction: {approx}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python rational_math.py
```

Pitfalls

Pitfall 1: Initializing `Decimal` from a `float` literal

```
# Danger:
bad = Decimal(0.1)  # Decimal('0.1000000000000000055511151231257827021181583404541015625')

# Correct: Always pass strings to Decimal
good = Decimal("0.1")
```

Pitfall 2: Comparing floats with ==

Never compare floats directly using `==` unless checking against `0.0`. Always use `math.isclose(a, b, rel_tol=1e-9)` or `abs(a - b) < epsilon`.

Exercises

1. Write a function `safe_float_equal(a: float, b: float, epsilon: float = 1e-9) -> bool` that verifies float equality within tolerance.
2. Calculate the compound interest of $10,000 at 5.5% annual interest compounded monthly over 10 years using `decimal.Decimal`.
3. Demonstrate special float values: positive infinity (`float("inf")`), negative infinity (`float("-inf")`), and Not-a-Number (`float("nan")`). Check their behavior with `math.isinf()` and `math.isnan()`.

Further reading

- David Goldberg: *What Every Computer Scientist Should Know About Floating-Point Arithmetic*.
- Python Standard Library: `decimal` module (*Decimal fixed point and floating point arithmetic*).
- Python Standard Library: `math.isclose` documentation.

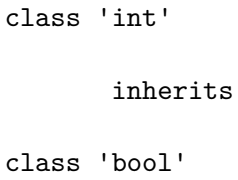
Booleans, Truthiness, and Short-Circuit Logic

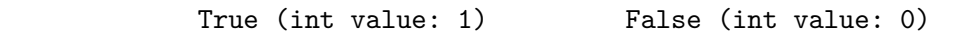
Booleans, Truthiness, and Short-Circuit Logic

After reading this chapter, you will master Python's `bool` type, the exact rules of truth value testing (truthiness), and how short-circuit evaluation powers defensive programming idioms.

Mental model

In Python, `bool` is an explicit subclass of `int`. There are only two instances: `True` and `False`.





Every Python object can be evaluated in a boolean context (such as an `if` statement or `while` loop). The language defines a specific set of **falsy** values; **every other object in Python is considered truthy**.

```
Falsy Objects:
- Constants: None, False
- Zero numbers: 0, 0.0, 0j, Decimal(0), Fraction(0, 1)
- Empty sequences: "", (), [], b"", bytearray(b"")
- Empty collections/mappings: {}, set(), frozenset()
- Custom objects defining __bool__() returning False, or __len__() returning 0
```

Minimal example

Save as `truthiness_demo.py`:

```python
# truthiness_demo.py
def main() -> None:
    falsy_values = [None, False, 0, 0.0, "", [], {}, set()]

    print("Checking standard falsy values:")
    for item in falsy_values:
        print(f"bool({repr(item):10s}) -> {bool(item)}")

    # Short-circuit returns the deciding operand, not necessarily a boolean!
    val1 = "fallback_host" or "default"
    val2 = "" or "fallback_host"
    print(f"\nval1: {val1}")
    print(f"val2: {val2}")

if __name__ == "__main__":
    main()
```

Run via `uv run python truthiness_demo.py`:

```
Checking standard falsy values:
bool(None      ) -> False
bool(False     ) -> False
bool(0         ) -> False
bool(0.0       ) -> False
bool(''        ) -> False
bool([]        ) -> False
bool({}        ) -> False
bool(set()     ) -> False

val1: fallback_host
val2: fallback_host
```

Worked examples

Case 1: Short-circuit evaluation as a safety guard

The `and` operator evaluates left-to-right; if the left operand is falsy, it immediately aborts evaluation and returns that operand. This allows writing safe guards:

```python
# guard_demo.py
class DeviceConnection:
    def __init__(self, connected: bool) -> None:
        self.connected = connected
```

```
    def query_stats(self) -> str:
        return "CPU: 12%, Temp: 41C"

def check_device(dev: DeviceConnection | None) -> None:
    # Guard against None, then check connected attribute
    if dev is not None and dev.connected:
        print(f"Device live: {dev.query_stats()}")
    else:
        print("Device unavailable or offline")

if __name__ == "__main__":
    check_device(None)
    check_device(DeviceConnection(connected=False))
    check_device(DeviceConnection(connected=True))
```

Run:

```
uv run python guard_demo.py
```

Case 2: The conditional ternary expression

Python supports inline ternary expressions: `value_if_true if condition else value_if_false`:

```
# ternary_demo.py
def get_env_concurrency(is_prod: bool) -> int:
    workers = 32 if is_prod else 4
    return workers

if __name__ == "__main__":
    print(f"Development workers: {get_env_concurrency(is_prod=False)}")
    print(f"Production workers : {get_env_concurrency(is_prod=True)}")
```

Run:

```
uv run python ternary_demo.py
```

Pitfalls

Pitfall 1: Fallback idiom swallowing valid falsy values

```
# Dangerous when 0 or False is a legitimate user configuration value:
timeout = config.get("timeout") or 30  # If timeout was configured as 0, it gets replaced by

# Correct fix:
timeout = config["timeout"] if "timeout" in config and config["timeout"] is not None else 30
```

Pitfall 2: Explicit comparison with `True` or `False`

Never write `if is_ready == True:` or `if is_ready is True:`. Idiomatic Python simply writes `if is_ready:`.

Exercises

1. Create a custom class `ServerPool` that evaluates to `False` in a boolean context when its internal server list is empty, and `True` when it has at least one server, using `__bool__()`.
2. Trace the exact return value of: `"" and "hello"`, `"hello" and "world"`, `[] or 0 or "default"`. Verify by running in a script.
3. Write a validator function `is_valid_payload(payload: dict) -> bool` that checks if the dictionary is non-empty and contains required keys `"id"` and `"timestamp"` using short-circuit boolean logic.

Further reading

- Python Language Reference: *Section 6.11: Boolean operations.*
- Python Standard Library: *Truth Value Testing.*
- PEP 285: *Adding a bool type.*

Strings, UTF-8 Encoding, and Slicing

Strings, UTF-8 Encoding, and Slicing

After reading this chapter, you will understand how CPython represents Unicode text, master string slicing mechanics, and author clean modern f-strings.

Mental model

In Python 3, `str` is an immutable sequence of Unicode code points. CPython internally optimizes string storage (PEP 393: Flexible String Representation) using the smallest necessary byte width per character: - Pure ASCII strings use 1 byte per character (`Latin-1`). - Extended multilingual strings use 2 bytes (`UCS-2`). - Emojis and complex scripts use 4 bytes (`UCS-4`).

```
String Slicing Mechanics:
   String:   P   y   t   h   o   n
 + Index:    0   1   2   3   4   5
 - Index:   -6  -5  -4  -3  -2  -1

 Slice Syntax: s[start:stop:step]
 Interval:     Half-open [start, stop) - includes start, excludes stop.
```

Minimal example

Save as `string_slicing.py`:

```
# string_slicing.py
def main() -> None:
    text = "telemetry_prod_us_east_1"

    # Slicing
    prefix = text[:9]       # From start to index 9 (exclusive)
    env = text[10:14]       # "prod"
    suffix = text[-9:]      # Last 9 characters
    reversed_text = text[::-1] # Step -1 reverses sequence

    print(f"Original : {text}")
    print(f"Prefix   : {prefix}")
    print(f"Env      : {env}")
```

```
    print(f"Suffix   : {suffix}")
    print(f"Reversed : {reversed_text}")

    # Modern f-string debugging specifier (=)
    latency_ms = 4.2819
    print(f"\nDebugging format: {latency_ms=:.2f}")

if __name__ == "__main__":
    main()
```

Run via `uv run python string_slicing.py`:

```
Original : telemetry_prod_us_east_1
Prefix   : telemetry
Env      : prod
Suffix   : us_east_1
Reversed : 1_tsae_su_dorp_yrtmelet

Debugging format: latency_ms=4.28
```

Worked examples

Case 1: String building performance (join vs +=)

Strings are immutable. Concatenating strings with `+=` in a loop creates a new string and copies data on every iteration ($O(N^2)$ time). Using `''.join()` is $O(N)$.

```
# string_builder.py
import time

def benchmark() -> None:
    n = 100_000
    words = ["chunk"] * n

    # Idiomatic O(N) method
    start = time.perf_counter()
    result = "".join(words)
    duration = time.perf_counter() - start
    print(f"join() duration: {duration:.4f}s (length={len(result)})")

if __name__ == "__main__":
    benchmark()
```

Run:

```
uv run python string_builder.py
```

Case 2: Encoding strings to bytes and decoding back

When sending data over network sockets or files, you must explicitly encode `str` (characters) to `bytes` (raw binary):

```
# unicode_codec.py
def main() -> None:
    original = "Microservices  [ s]"

    # UTF-8 serialization
    encoded_bytes = original.encode("utf-8")
    print(f"Encoded bytes ({len(encoded_bytes)} bytes):\n{encoded_bytes}")

    # Deserialization
    decoded_str = encoded_bytes.decode("utf-8")
    print(f"\nDecoded str: {decoded_str}")
    print(f"Match: {original == decoded_str}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python unicode_codec.py
```

Pitfalls

Pitfall 1: Confusing `bytes` and `str`

`str` and `bytes` cannot be implicitly combined. Concatenating `b"hello" + "world"` raises an immediate `TypeError`.

Pitfall 2: Slice out-of-range behavior

Index lookups like `s[100]` raise `IndexError`. However, slices like `s[100:200]` do not fail; they return an empty string `""`.

Exercises

1. Write a function `is_palindrome(s: str) -> bool` that strips non-alphanumeric characters, converts to lowercase, and checks if the string equals its reverse slice (`s[::-1]`).
2. Parse an ISO timestamp string `"2026-09-06T20:15:30Z"` into separate date and time components using slicing.
3. Format a tabular output using f-string alignment specifiers: left-aligned 20 chars (`<20`), right-aligned 10 chars (`>10`), and float formatted to 2 decimals.

Further reading

- PEP 393: *Flexible String Representation*.
- Python Standard Library: *Text Sequence Type — str*.
- Ned Batchelder: *Pragmatic Unicode*.

UTF-8 by Default: What PEP 686 Changes in Python 3.15

UTF-8 by Default: What PEP 686 Changes in Python 3.15

Thesis

Until 3.14, `open("notes.txt")` meant "open in whatever encoding this machine's locale says". On most Linux and macOS systems that happened to be UTF-8. On Windows it was the ANSI code page (`cp1252` in Western Europe), and on an old server it might be ISO-8859-1. Python 3.15 makes **UTF-8 mode the default** (PEP 686, *Final*). With no `encoding=` argument, text files, pipes, and the standard streams now use UTF-8 everywhere, whatever the locale. That removes a whole class of "works on my machine" bugs. It also moves some bugs around. Files written by older jobs in the locale encoding no longer decode. Output of locale-speaking subprocesses no longer decodes. And because the default now depends on the *interpreter version*, a library that supports 3.10–3.15 still produces different bytes on different interpreters. The fix in every case is the same: say which encoding you mean.

Version note: every listing below ran on **CPython 3.15.0** (final, 2026-10-09) and **CPython 3.14.8** on Linux x86-64, and the outputs are copied from those runs. Linux with a UTF-8 locale barely notices this change, so the lab builds a legacy **ISO-8859-1** locale to reproduce what a Windows machine with a non-UTF-8 code page sees. The mechanism is the same one Windows uses: the locale encoding is not UTF-8. The Windows-specific notes are from the 3.15 documentation and were not run here.

Mental model

```
                              3.14 (locale mode)        3.15 (UTF-8 mode, default)
open(path)                    locale encoding           utf-8
subprocess.run(text=True)     locale encoding           utf-8
sys.stdin / sys.stdout        locale, strict            utf-8, surrogateescape
sys.getfilesystemencoding()   locale (Unix)             utf-8
locale.getpreferredencoding() locale                    'utf-8' (always)
locale.getencoding()          locale                    locale     unchanged: the real local
open(path, encoding="locale") locale                    locale     unchanged: explicit opt-i
open(path, encoding="utf-8")  utf-8                     utf-8      unchanged: always correct

turn it off (per process, at startup only):  PYTHONUTF8=0   or   python -X utf8=0
find every implicit use:                      python -X warn_default_encoding  (EncodingWarn
```

UTF-8 mode is not new. It has existed since 3.7 (PEP 540), and Python already switched it on automatically when the locale was `C` or `POSIX`. What 3.15 changes is the *default*. Everything an explicit `encoding=` argument says keeps meaning exactly what it said. Only the "I didn't say" path moved.

Worked examples

The lab needs two interpreters and a legacy locale. `localedef` compiles locales from glibc's sources (on Debian and Ubuntu they come with the `locales` package), and `LOCPATH` points glibc at a private directory, so no root access is needed:

```
$ mkdir -p loc
$ localedef -i en_US -f UTF-8      loc/en_US.utf8
$ localedef -i en_US -f ISO-8859-1 loc/en_US.iso88591
$ localedef -i de_AT -f ISO-8859-1 loc/de_AT.iso88591
$ export LOCPATH=$PWD/loc
$ clean() { env -i PATH="$PATH" LOCPATH="$LOCPATH" "$@"; }   # run with only the variables w
```

`clean` starts each run from an empty environment, so nothing in your shell (`LC_ALL`, `PYTHONUTF8`, `PYTHONIOENCODING`) can leak in. Absolute paths in tracebacks and warnings are shortened to `...` below.

Case 1: What each default actually picks

```
# probe.py - which encoding does each default actually pick?
import locale
import sys

print(f"Python {sys.version.split()[0]}  utf8_mode={sys.flags.utf8_mode}")
print(f"  locale.getencoding()           {locale.getencoding()}")
print(f"  locale.getpreferredencoding()  {locale.getpreferredencoding()}")
print(f"  sys.getfilesystemencoding()    {sys.getfilesystemencoding()}")
print(f"  sys.stdout                     {sys.stdout.encoding} / {sys.stdout.errors}")
with open("probe.txt", "w") as f:
    print(f"  open('probe.txt', 'w')         {f.encoding}")
with open("probe.txt", "w", encoding="locale") as f:
    print(f"  open(..., encoding='locale')   {f.encoding}")
```

On a UTF-8 Linux machine, the encodings are the same between versions. One error handler is not:

```
$ clean LANG=en_US.UTF-8 python3.14 probe.py
Python 3.14.8  utf8_mode=0
  locale.getencoding()           UTF-8
  locale.getpreferredencoding()  UTF-8
```

```
  sys.getfilesystemencoding()    utf-8
  sys.stdout                     utf-8 / strict
  open('probe.txt', 'w')         UTF-8
  open(..., encoding='locale')   UTF-8
$ clean LANG=en_US.UTF-8 python3.15 probe.py
Python 3.15.0  utf8_mode=1
  locale.getencoding()           UTF-8
  locale.getpreferredencoding()  utf-8
  sys.getfilesystemencoding()    utf-8
  sys.stdout                     utf-8 / surrogateescape
  open('probe.txt', 'w')         utf-8
  open(..., encoding='locale')   UTF-8
```

`sys.stdout` now uses `surrogateescape` instead of `strict`. Keep that in mind for the trap. Now the legacy locale, which is where the real change shows:

```
$ clean LANG=en_US.ISO-8859-1 python3.14 probe.py
Python 3.14.8  utf8_mode=0
  locale.getencoding()           ISO-8859-1
  locale.getpreferredencoding()  ISO-8859-1
  sys.getfilesystemencoding()    iso8859-1
  sys.stdout                     iso8859-1 / strict
  open('probe.txt', 'w')         ISO-8859-1
  open(..., encoding='locale')   ISO-8859-1
$ clean LANG=en_US.ISO-8859-1 python3.15 probe.py
Python 3.15.0  utf8_mode=1
  locale.getencoding()           ISO-8859-1
  locale.getpreferredencoding()  utf-8
  sys.getfilesystemencoding()    utf-8
  sys.stdout                     utf-8 / surrogateescape
  open('probe.txt', 'w')         utf-8
  open(..., encoding='locale')   ISO-8859-1
```

Same machine, same environment. 3.14 follows the locale everywhere. 3.15 uses UTF-8 everywhere *except* the two APIs whose job is to report the locale: `locale.getencoding()` and `encoding="locale"`. `locale.getpreferredencoding()` is not one of them. In UTF-8 mode it always answers `'utf-8'`, so code that asks it "what is the system encoding?" gets the wrong answer.

On Windows, the 3.15 docs list one difference from this Linux picture. Console I/O and the filesystem encoding were already UTF-8 there before 3.15 (PEPs 528 and 529), so the change on Windows lands on `open()`, pipes and `subprocess`, which used the ANSI code page. That code page is still reachable when you need it, as `encoding="locale"` or the `"mbcs"` codec.

Case 2: A file from last year's job no longer decodes

```
# export_report.py - a nightly job that never said which encoding it meant
rows = [("Wien", 12), ("Graz", 11), ("Klagenfurt am Wörthersee", 9)]
with open("report.csv", "w") as f:
    f.write("city;temp_c\n")
    for city, temp in rows:
        f.write(f"{city};{temp}\n")
```

```
# read_report.py - the consumer, also without an encoding
with open("report.csv") as f:
    for line in f:
        print(line.rstrip())
```

The exporter ran on 3.14 for years. The reader is upgraded to 3.15 first:

```
$ clean LANG=en_US.ISO-8859-1 python3.14 export_report.py
$ grep -a Klagenfurt report.csv | od -An -tx1
 4b 6c 61 67 65 6e 66 75 72 74 20 61 6d 20 57 f6
 72 74 68 65 72 73 65 65 3b 39 0a
$ clean LANG=en_US.ISO-8859-1 python3.15 read_report.py
Traceback (most recent call last):
  File ".../read_report.py", line 3, in <module>
    for line in f:
                ^
  File "<frozen codecs>", line 325, in decode
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xf6 in position 43: invalid start byte
```

ö was written as the single byte `f6`, which is ISO-8859-1. In UTF-8 that byte can only appear inside a multi-byte sequence, so the reader stops. That is the *good* direction, because it fails loudly. Upgrade the other side first and the failure is silent:

```
$ clean LANG=en_US.ISO-8859-1 python3.15 export_report.py
$ grep -a Klagenfurt report.csv | od -An -tx1
 4b 6c 61 67 65 6e 66 75 72 74 20 61 6d 20 57 c3
 b6 72 74 68 65 72 73 65 65 3b 39 0a
$ clean LANG=en_US.ISO-8859-1 python3.14 read_report.py | iconv -f ISO-8859-1 -t UTF-8
city;temp_c
Wien;12
Graz;11
Klagenfurt am WÃ¶rthersee;9
```

Now ö is the two bytes `c3 b6`. ISO-8859-1 decodes any byte, so 3.14 reads them as two characters, `Ã¶`, and carries on. (`iconv` only converts the output for this page. The mojibake is in the string itself.) No exception is raised, so the bad data goes into whatever the reader writes next. PEP 686 warns about exactly this: `UnicodeError`, mojibake, "or even silent data corruption".

Case 3: Subprocess output in the locale encoding

`text=True` decodes a child's output with the same default, and plenty of tools still print in the locale encoding:

```
# month.py - ask a locale-aware tool for a month name and decode its output
import subprocess
import sys

mode = sys.argv[1] if len(sys.argv) > 1 else "default"
kwargs = {"text": True} if mode == "default" else {"encoding": "locale"}
out = subprocess.run(["date", "-d", "2026-03-01", "+%B"],
                     capture_output=True, check=True, **kwargs).stdout
print(f"{sys.version.split()[0]} {mode:<8} {out.strip()!r}")
```

```
$ clean LANG=de_AT.ISO-8859-1 python3.14 month.py | iconv -f ISO-8859-1 -t UTF-8
3.14.8 default  'März'
$ clean LANG=de_AT.ISO-8859-1 python3.15 month.py 2>&1 | tail -1
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xe4 in position 1: invalid continuation
$ clean LANG=de_AT.ISO-8859-1 python3.15 month.py locale
3.15.0 locale   'März'
```

`date` wrote `März` in ISO-8859-1 because its locale told it to. 3.15 decodes the pipe as UTF-8 and fails on `e4`. With `encoding="locale"`, parent and child agree again. Notice the last line, too: 3.15's own `print` wrote UTF-8 to a terminal whose locale says ISO-8859-1. A real legacy terminal (or a Windows program reading Python's piped output) would show `MÃ¤rz`. The default changed for output as well as input.

PEP 686 considered and rejected a special case that would keep pipes on the locale encoding. The answer for subprocesses is the same as for files: name the encoding, either `encoding="utf-8"` or `encoding="locale"`.

Case 4: Find every place you relied on the default

Grepping for `open(` is not enough, because `subprocess`, `pathlib.Path.read_text`, `tempfile` and library helpers all reach the same default. The interpreter can report them for you. `-X warn_default_encoding` (or `PYTHONWARNDEFAULTENCODING=1`) emits an `EncodingWarning` (PEP 597) at every call site that did not pass an encoding:

```
$ clean LANG=en_US.UTF-8 python3.15 -X warn_default_encoding export_report.py
.../export_report.py:3: EncodingWarning: 'encoding' argument not specified
  with open("report.csv", "w") as f:
$ clean LANG=en_US.UTF-8 python3.15 -X warn_default_encoding month.py
.../month.py:7: EncodingWarning: 'encoding' argument not specified.
  out = subprocess.run(["date", "-d", "2026-03-01", "+%B"],
3.15.0 default  'March'
```

```
$ clean LANG=en_US.UTF-8 python3.15 -X warn_default_encoding -c "import locale; print(locale
<string>:1: EncodingWarning: UTF-8 Mode affects locale.getpreferredencoding(). Consider loca
utf-8
```

It works on a UTF-8 machine, so you can turn it on in ordinary Linux CI. Make it fatal there:

```
$ clean LANG=en_US.UTF-8 PYTHONWARNDEFAULTENCODING=1 python3.15 -W error::EncodingWarning ex
EncodingWarning: 'encoding' argument not specified
```

(The run exits with status 1.) For a static pass, Ruff's `unspecified-encoding` rule (`PLW1514`, preview-only in Ruff 0.17.0) flags `open()` calls:

```
$ uvx ruff@0.17.0 check --preview --select PLW1514 --output-format concise export_report.py
export_report.py:3:6: unspecified-encoding: `open` in text mode without explicit `encoding`
read_report.py:2:6: unspecified-encoding: `open` in text mode without explicit `encoding` ar
Found 2 errors.
2 unsafe fixes available with `--fix --unsafe-fixes`.
```

It did **not** flag `subprocess.run(text=True)` in `month.py`, and its autofix always inserts `encoding="utf-8"`. That is the right fix for new data and the wrong one for a legacy file. Use the linter for coverage of `open()` and the runtime warning for everything else.

If you write a library function that accepts `encoding=None` and passes it on, don't hide the warning inside your code. `io.text_encoding()` resolves `None` to the current default and attributes the warning to *your caller*:

```
# textenc.py - what a library helper should pass through
import io
import sys

def load(path, encoding=None):
    encoding = io.text_encoding(encoding)   # PEP 597: warn at the caller, not here
    with open(path, encoding=encoding) as f:
        return encoding, f.readline().strip()

print(sys.version.split()[0], load("report.csv"))
```

```
$ clean LANG=en_US.UTF-8 python3.14 -X warn_default_encoding textenc.py
.../textenc.py:10: EncodingWarning: 'encoding' argument not specified
  print(sys.version.split()[0], load("report.csv"))
3.14.8 ('locale', 'city;temp_c')
$ clean LANG=en_US.UTF-8 python3.15 -X warn_default_encoding textenc.py
.../textenc.py:10: EncodingWarning: 'encoding' argument not specified
  print(sys.version.split()[0], load("report.csv"))
3.15.0 ('utf-8', 'city;temp_c')
```

The warning points at line 10, the call, not at the `open()` inside `load`. The resolved value also shows the version change directly: `'locale'` on 3.14, `'utf-8'` on 3.15.

Case 5: The fixes, and the escape hatch

```
# report_fixed.py - say what you mean on both ends
import locale

with open("report.csv", "w", encoding="utf-8", newline="") as f:
    f.write("city;temp_c\nKlagenfurt am Wörthersee;9\n")

# A file from the old Windows/legacy job: decode it as what it is
legacy = "Klagenfurt am Wörthersee;9\n".encode("iso-8859-1")
with open("legacy.csv", "wb") as f:
    f.write(legacy)
with open("legacy.csv", encoding="iso-8859-1") as f:
    print(repr(f.read()))
with open("legacy.csv", encoding="locale") as f:
    print(locale.getencoding(), repr(f.read()))
```

```
$ clean LANG=en_US.ISO-8859-1 python3.15 report_fixed.py
'Klagenfurt am Wörthersee;9\n'
ISO-8859-1 'Klagenfurt am Wörthersee;9\n'
```

New data is written as UTF-8 on purpose. Old data is read as what it is: either a named codec, or `encoding="locale"` when "whatever this machine uses" really is the contract (for example, a file a local user just saved from a legacy editor). Prefer the named codec for anything that travels between machines.

When you cannot fix the code yet, turn UTF-8 mode off for that process. It must happen at startup: there is no runtime switch.

```
$ clean LANG=en_US.ISO-8859-1 PYTHONUTF8=0 python3.15 probe.py
Python 3.15.0  utf8_mode=0
  locale.getencoding()           ISO-8859-1
  locale.getpreferredencoding()  ISO-8859-1
  sys.getfilesystemencoding()    iso8859-1
  sys.stdout                     iso8859-1 / strict
  open('probe.txt', 'w')         ISO-8859-1
  open(..., encoding='locale')   ISO-8859-1
$ clean LANG=en_US.ISO-8859-1 python3.15 -X utf8=0 -c "import sys; print(sys.flags.utf8_mode
0
$ clean LANG=C PYTHONUTF8=0 python3.15 -c "import sys, locale; print(sys.flags.utf8_mode, lo
0 UTF-8 surrogateescape
```

The first run is byte-for-byte the 3.14 behaviour. The last run shows the opt-out is not a time machine to "ASCII". Under the `C` locale, Python still coerces to `C.UTF-8` (PEP 538), so you get UTF-8 without UTF-8 mode. The 3.15 Windows docs add one warning: putting `PYTHONUTF8=0` in the *system* environment changes every Python 3.7+ program on the machine. Scope it to the one service or script that needs it, and give it a removal date.

The trap

The new default only seems to remove the decision. It doesn't, for two reasons.

First, the default **depends on the interpreter**. In Cases 1 and 2 the same `export_report.py`, on the same machine and in the same environment, wrote `f6` under 3.14 and `c3 b6` under 3.15. A package that supports 3.10–3.15 and leaves out `encoding=` writes different bytes depending on which interpreter the user installed, or whether someone set `PYTHONUTF8=0` in a container image. "3.15 defaults to UTF-8, so I can drop `encoding=`" is only true once your *oldest* supported interpreter is 3.15 and nobody turns the mode off.

Second, `surrogateescape` on the standard streams **moves** failures instead of removing them. Linux filenames are bytes, and some are not valid UTF-8. Python decodes them with lone surrogates so they round-trip:

```
# list_inbox.py - print every name in a directory, then save the list
import os
import sys

names = sorted(os.listdir("inbox"))
print(sys.version.split()[0], [ascii(n) for n in names])
for n in names:
    print(n)
with open("inbox.txt", "w", encoding="utf-8") as f:
    f.write("\n".join(names) + "\n")
```

```
$ clean LANG=en_US.UTF-8 python3.14 list_inbox.py 2>&1 | grep -E "line [0-9]+|Error"
  File ".../list_inbox.py", line 8, in <module>
UnicodeEncodeError: 'utf-8' codec can't encode characters in position 2-3: surrogates not al
$ clean LANG=en_US.UTF-8 python3.15 list_inbox.py 2>&1 | grep -E "line [0-9]+|Error"
  File ".../list_inbox.py", line 10, in <module>
UnicodeEncodeError: 'utf-8' codec can't encode characters in position 2-3: surrogates not al
```

The directory holds `Gr\xfc\xdfe.txt`, a name saved by a Latin-1 program. 3.14 crashed at line 8, the `print`, because its stdout was `strict`. 3.15 prints the raw bytes happily and crashes two lines later, writing the list to a file with plain `encoding="utf-8"`. A script that "printed fine in testing" now fails in the step that persists data. When names may be arbitrary bytes, say so on the file too:

```
# fixed_inbox.py - keep undecodable names intact on the way to disk
import os

names = sorted(os.listdir("inbox"))
with open("inbox.txt", "w", encoding="utf-8", errors="surrogateescape") as f:
    f.write("\n".join(names) + "\n")
with open("inbox.txt", "rb") as f:
    print(f.read())
```

```
$ clean LANG=en_US.UTF-8 python3.15 fixed_inbox.py
b'Gr\xfc\xdfe.txt\nplain.txt\n'
```

The original bytes come back unchanged.

The boring rule

Pass `encoding=` to every text-mode `open()`, `Path.read_text()`/`write_text()`, and `subprocess` call: `"utf-8"` for data you own, a named codec or `"locale"` for legacy input. Run CI with `PYTHONWARNDEFAULTENCODING=1 -W error::EncodingWarning`. Replace `locale.getpreferredencoding()` with `locale.getencoding()` wherever you actually mean "the system's encoding". Use `PYTHONUTF8=0` only as a scoped, dated workaround, never as a machine-wide setting.

Try this

1. Run `probe.py` under `clean LANG=C python3.14` and `clean LANG=C python3.15`. Do any lines differ? Explain from the mental model why the `C` locale already got UTF-8 mode on 3.14.
2. Rewrite `read_report.py` to accept both old and new exports: try `utf-8` first, fall back to `iso-8859-1` on `UnicodeDecodeError`. Then explain why trying the other order would be wrong.
3. Add `PYTHONWARNDEFAULTENCODING=1` and `-W error::EncodingWarning` to your own test suite's runner and count the failures. How many come from your code, and how many from dependencies?
4. Change `month.py` to `encoding="utf-8"` and run it under `de_AT.ISO-8859-1`. Then set `LC_ALL=de_AT.UTF-8` for the child only (`env=` argument) and make it work without touching the parent's decoding.
5. Repeat the trap with `Path("inbox").iterdir()` and `json.dump`. Where does the surrogate now fail, and what would `os.fsencode()` let you store instead?

Sources

- PEP 686, *Make UTF-8 mode default* (Final, Python 3.15): https://peps.python.org/pep-0686/
- What's New in Python 3.15, "Other language changes" (UTF-8 default): https://docs.python.org/3.15/whatsnew/3.15.html
- `os` documentation, "Python UTF-8 Mode": https://docs.python.org/3.15/library/os.html#utf8-mode
- Using Python on Windows, "UTF-8 mode": https://docs.python.org/3.15/using/windows.html#utf-8-mode
- PEP 540, *Add a new UTF-8 Mode* (3.7): https://peps.python.org/pep-0540/
- PEP 597, *Add optional EncodingWarning*: https://peps.python.org/pep-0597/

- PEP 538, *Coercing the legacy C locale to a UTF-8 based locale*: https://peps.python.org/pep-0538/
- `io.text_encoding()`: https://docs.python.org/3.15/library/io.html#io.text_encoding
- `locale.getencoding()`: https://docs.python.org/3.15/library/locale.html#locale.getencoding
- Ruff rule `unspecified-encoding` (PLW1514): https://docs.astral.sh/ruff/rules/unspecified-encoding/

Type Casting, Type Inspection, and Annotations

Type Casting, Type Inspection, and Annotations

After reading this chapter, you will master Python's strong typing model, safely perform type conversions, inspect object types at runtime, and write modern Python 3.14 type annotations.

Mental model

Python is **strongly typed** and **dynamically typed**: - **Strongly typed**: Python does not implicitly coerce incompatible types (e.g. adding a string and an integer raises `TypeError`). - **Dynamically typed**: Types belong to values, not to variable names. A name can point to an integer at one moment and a string the next.

```
Strong Typing Enforcement:
  "5" + 2    TypeError: can only concatenate str (not "int") to str
Explicit Conversion:
  int("5") + 2    7 (Integer addition)
  "5" + str(2)    "52" (String concatenation)
```

At runtime, `isinstance()` checks if an object belongs to a class or any of its subclasses, traversing the inheritance tree.

Minimal example

Save as `type_inspect_demo.py`:

```python
# type_inspect_demo.py
def process_metric(raw_value: str | int | float) -> float:
    # 1. Type inspection with isinstance
    if isinstance(raw_value, (int, float)):
        return float(raw_value)

    # 2. Explicit type casting
    try:
        return float(raw_value)
    except ValueError as err:
        raise ValueError(f"Cannot cast {raw_value!r} to float") from err
```

```
def main() -> None:
    samples = [42, "128.5", 3.14159, "invalid_num"]

    for sample in samples:
        try:
            result = process_metric(sample)
            print(f"Sample: {sample!r:15s} -> Processed float: {result}")
        except ValueError as err:
            print(f"Sample: {sample!r:15s} -> Error: {err}")

if __name__ == "__main__":
    main()
```

Run via `uv run python type_inspect_demo.py`:

```
Sample: 42              -> Processed float: 42.0
Sample: '128.5'         -> Processed float: 128.5
Sample: 3.14159         -> Processed float: 3.14159
Sample: 'invalid_num'   -> Error: Cannot cast 'invalid_num' to float
```

Worked examples

Case 1: Checking types: `isinstance` vs `type()`

`isinstance()` respects class inheritance hierarchies, while `type(obj) is Class` requires an exact type match.

```
# isinstance_vs_type.py
class Animal:
    pass

class Dog(Animal):
    pass

def verify_types() -> None:
    d = Dog()

    print(f"type(d) is Dog    : {type(d) is Dog}")
    print(f"type(d) is Animal : {type(d) is Animal} (False: ignores inheritance)")
    print(f"isinstance(d, Animal): {isinstance(d, Animal)} (True: recognizes subclass)")

    # bool is a subclass of int!
    print(f"isinstance(True, int): {isinstance(True, int)} (True!)")
```

```
    print(f"type(True) is int    : {type(True) is int} (False!)")

if __name__ == "__main__":
    verify_types()
```

Run:

```
uv run python isinstance_vs_type.py
```

Case 2: Modern Python 3.14 type annotations

Type annotations provide documentation and allow static type checkers like `mypy` and `pyright` to find bugs before runtime:

```
# typed_service.py
def calculate_throughput(requests: int, duration_seconds: float) -> float:
    if duration_seconds <= 0:
        raise ValueError("Duration must be positive")
    return requests / duration_seconds

def lookup_node(node_id: str) -> str | None:
    nodes = {"node-1": "10.0.0.1", "node-2": "10.0.0.2"}
    return nodes.get(node_id)

if __name__ == "__main__":
    tps = calculate_throughput(10_000, 4.5)
    print(f"Throughput: {tps:.2f} req/s")
    print(f"Lookup: {lookup_node('node-1')}")
```

Run:

```
uv run python typed_service.py
```

Pitfalls

Pitfall 1: Using `type(x) == T` instead of `isinstance()`

Checking `type(x) == int` will return `False` if someone passes a subclass or specialized numerical type. Always prefer `isinstance(x, int)`.

Pitfall 2: Assuming type annotations enforce runtime checks

Python does not validate type hints at runtime by default. If you pass a string to a function annotated with `x: int`, Python executes it without error until an incompatible operation occurs. Runtime validation requires libraries like Pydantic.

Exercises

1. Write a function `safe_int_cast(val: str, default: int = 0) -> int` that parses an integer string and returns `default` if parsing fails.
2. Given a mixed list `items = [1, "two", 3.0, [4], (5,), {"six": 6}]`, write a script that separates elements into a dictionary grouped by type name.
3. Run `mypy` or `ruff check` on a script containing a deliberate type annotation mismatch to observe static analysis feedback.

Further reading

- PEP 484: *Type Hints*.
- PEP 604: *Allow-writing union types as X | Y*.
- Python Standard Library: `typing` module.

Template Strings (t-strings)

Template Strings (t-strings)

An **f-string** builds a finished `str` the moment it runs. A **t-string** (Python **3.14**, PEP 750) looks the same — braces, `!r`, format specs — but the `t` prefix returns a `string.templatelib.Template`: the static pieces and the interpolated values stay separate until **your** code decides how to combine them. The boring default is still an f-string for ordinary messages. Reach for a t-string when the values must be escaped, parameterized, or logged as structured parts — not glued into a string first.

Mental model

- `f"..."` → `str`. Done. Conversions (`!s`, `!r`, `!a`) and format specs (`:.2f`) are applied for you.
- `t"..."` → `Template`. Not a string. `.strings` is the tuple of static chunks; `.interpolations` holds `Interpolation` objects; `.values` is the tuple of evaluated results. There is always **one more** string than interpolations — the holes sit between the chunks.
- Iterating a `Template` yields non-empty static strings and `Interpolation`s in order. Empty static pieces are skipped by iteration but still appear in `.strings`.
- Each `Interpolation` has `.value`, `.expression` (source text), `.conversion` (`None`, `"s"`, `"r"`, or `"a"`), and `.format_spec`. Unlike f-strings, **you** choose whether to apply conversion and format — `string.templatelib.convert` mimics f-string conversion when you want that.
- `Template + str` is a `TypeError`. Concatenate `Template` with `Template`, or wrap a static piece in `Template("...")`.
- Stdlib types live in `string.templatelib`. There is no built-in HTML or SQL renderer: write a small function that walks the parts.

Worked examples

Case 1: Same braces, different type

Save as `t_vs_f.py`. One desk table, two prefixes. The f-string is text you can print and concatenate. The t-string is a structure.

```
# t_vs_f.py
def main():
    table = 12
    fmsg = f"table {table}"
    tmsg = t"table {table}"
    print(type(fmsg).__name__, fmsg)
    print(type(tmsg).__name__, tmsg.strings, tmsg.values)
```

```
if __name__ == "__main__":
    main()
```

Run:

```
uv run python t_vs_f.py
```

Output:

```
str table 12
Template ('table ', '') (12,)
```

`tmsg` is not `"table 12"`. Its `.values` hold `12` as an `int`, not as digits inside a string.

Case 2: Walk the parts at the desk

Save as `t_inspect.py`. Print static chunks and each interpolation's expression, value, conversion, and format spec.

```
# t_inspect.py
from string.templatelib import Interpolation


def main():
    table = 12
    note = "no onions"
    tmpl = t"ticket → table {table}; note={note!r}"
    print(type(tmpl).__name__)
    print(tmpl.strings)
    print(tmpl.values)
    for part in tmpl:
        if isinstance(part, Interpolation):
            print(
                "interp",
                part.expression,
                part.value,
                part.conversion,
                repr(part.format_spec),
            )
        else:
            print("static", repr(part))


if __name__ == "__main__":
    main()
```

Run:

```
uv run python t_inspect.py
```

Output:

```
Template
('ticket → table ', '; note=', '')
(12, 'no onions')
static 'ticket → table '
interp table 12 None ''
static '; note='
interp note no onions r ''
```

`!r` did **not** turn the note into a quoted string yet. It only set `.conversion` to `"r"`. A processor that cares will call `convert(part.value, part.conversion)`.

Case 3: Render like an f-string (on purpose)

Save as `t_render.py`. When you want the familiar finished text, apply `convert` and `format` yourself. This is the boring "I still need a string" path — and it makes the deferred steps visible.

```
# t_render.py
from string.templatelib import Interpolation, convert


def render(template):
    parts = []
    for part in template:
        if isinstance(part, Interpolation):
            text = convert(part.value, part.conversion)
            if part.format_spec:
                text = format(text, part.format_spec)
            else:
                text = str(text)
            parts.append(text)
        else:
            parts.append(part)
    return "".join(parts)


def main():
    cents = 1850
    note = "rush"
    print(render(t"total ${cents / 100:.2f}"))
    print(render(t"note={note!r}"))
```

```
if __name__ == "__main__":
    main()
```

Run:

```
uv run python t_render.py
```

Output:

```
total $18.50
note='rush'
```

Same result as the matching f-strings. Prefer `f"..."` when this is all you need. Prefer `t"..."` when Case 4 or Case 5 is the real goal.

Case 4: Escape guest text for HTML

Save as `t_html.py`. Static markup stays raw. Interpolated values pass through `html.escape`. That split is why t-strings exist: you never build `"<p>" + guest + "</p>"` with a hostile `guest`.

```
# t_html.py
from html import escape
from string.templatelib import Interpolation, convert


def safe_html(template):
    parts = []
    for part in template:
        if isinstance(part, Interpolation):
            text = convert(part.value, part.conversion)
            if part.format_spec:
                text = format(text, part.format_spec)
            parts.append(escape(str(text), quote=True))
        else:
            parts.append(part)
    return "".join(parts)


def main():
    guest = '<script>alert("x")</script>'
    table = 7
    print(safe_html(t"<p>table {table}: {guest}</p>"))
    print(safe_html(t"<p>table {table}: {guest!r}</p>"))


if __name__ == "__main__":
```

```
    main()
```

Run:

```
uv run python t_html.py
```

Output:

```
<p>table 7: &lt;script&gt;alert(&quot;x&quot;)&lt;/script&gt;</p>
<p>table 7: &#x27;&lt;script&gt;alert(&quot;x&quot;)&lt;/script&gt;&#x27;</p>
```

The tags in the static pieces are real HTML. The script in `guest` is text. An f-string of the same shape would have injected markup.

Case 5: SQL placeholders, not string glue

Save as `t_sql.py`. Walk the template: static SQL stays; each interpolation becomes `?` and its value goes in a params tuple. The database API binds params — you never splice user data into the statement string.

```
# t_sql.py
from string.templatelib import Interpolation


def sql_query(template):
    chunks = []
    params = []
    for part in template:
        if isinstance(part, Interpolation):
            chunks.append("?")
            params.append(part.value)
        else:
            chunks.append(part)
    return "".join(chunks), tuple(params)


def main():
    table = 12
    status = "open"
    sql, params = sql_query(
        t"SELECT id FROM tickets WHERE table_n = {table} AND status = {status}"
    )
    print(sql)
    print(params)
```

```
if __name__ == "__main__":
    main()
```

Run:

```
uv run python t_sql.py
```

Output:

```
SELECT id FROM tickets WHERE table_n = ? AND status = ?
(12, 'open')
```

An f-string here would be the classic injection foot-gun. The t-string keeps values out of the SQL text until a real driver binds them.

The trap

Save as `t_trap.py`. Treating a `Template` like a `str` fails loudly — or, worse, `str(template)` gives a debug dump you might accidentally ship.

```
# t_trap.py
def main():
    table = 12
    tmpl = t"table {table}"
    try:
        print("msg: " + tmpl)
    except TypeError as exc:
        print(type(exc).__name__ + ":", exc)
    print("forced:", "msg: " + str(tmpl))
    print("parts:", list(tmpl))


if __name__ == "__main__":
    main()
```

Run:

```
uv run python t_trap.py
```

Output:

```
TypeError: can only concatenate str (not "string.templatelib.Template") to str
forced: msg: Template(strings=('table ', ''), interpolations=(Interpolation(12, 'table', None,
parts: ['table ', Interpolation(12, 'table', None, '')]
```

`+` with a plain string is rejected. `str(tmpl)` is not a render — it is the object's representation. Process the template (Cases 3–5) or keep it as a `Template`. Do not `str()` your way out of the type error and call it done.

The boring rule

- Use **f-strings** for human messages, logs, and errors when every value is already safe to embed.
- Use **t-strings** when interpolations need escaping, parameterization, or structured handling before they become text.
- Write one small processor per sink (`safe_html`, `sql_query`, `render`) that walks parts with `isinstance(..., Interpolation)`.
- Apply `convert` and `format` only when that sink should behave like an f-string.
- Never concatenate `Template` with `str`. Never ship `str(template)` as user-facing text.
- Do not replace every f-string with a t-string. Deferred structure is a tool, not a fashion.

Try this

1. In `t_vs_f.py`, print `tmsg + t" ok"` and `list(tmsg + t" ok")`. Confirm you can concatenate two templates.
2. In `t_html.py`, change `guest` to `O'Reilly & Sons` and re-run. Confirm quotes and `&` are escaped.
3. In `t_sql.py`, add `AND seats >= {seats}` with `seats = 2`. Confirm a third `?` and `2` in `params`.
4. In `t_render.py`, drop the `convert` / `format` calls and append `str(part.value)` only. Re-run the `!r` line — then put `convert` back and explain the difference in one sentence.

None-aware operators (PEP 823 / 824) — draft for 3.16

None-aware operators (PEP 823 / 824) — draft for 3.16

> **!** Important
>
> **Draft status.** PEP 823 (*None-aware access operators*) and PEP 824 (*None-coalescing operators*) are **Draft** Standards Track PEPs (created 2026-09-20, target **Python 3.16**). Nothing in this chapter is shipping syntax today. CPython **3.13** and **3.14** still raise `SyntaxError` on `?.`, `?[ ]`, `??`, and `??=`. Do not invent operators beyond what those PEPs specify; treat every code sample that uses the new spelling as **proposed**, not runnable on current interpreters unless you deliberately use a PEP reference build.

Thesis

Nested `optional` attributes (always present, sometimes `None`) force a tax of `is not None` checks, temporary names, and duplicated subexpressions. PEP 823 proposes `?.` and `?[ ]` so a chain short-circuits to `None` when a base is `None`. PEP 824 separately proposes `??` and `??=` so defaults attach on `None` without truthiness traps. Both revive ideas from deferred PEP 505, split into two independently adoptable PEPs, sponsored by Guido van Rossum.

Mental model

```
PEP 823 (access)                          PEP 824 (coalesce)

base?.attr        _t.attr if (_t := base) is not None else None
base?[key]        _t[key] if (_t := base) is not None else None

a ?? b            _t if (_t := a) is not None else b
a ??= b           if a is None: a = b

optional  = attribute always exists, value may be None
missing   = attribute / key not present at all  ← NOT handled by ?.
```

Operator	PEP	Checks	Does **not** do
`?.`	823	`is not None` then attribute access	Catch `AttributeError` for missing names
`?[ ]`	823	`is not None` then subscript	Turn missing dict keys into `None` (use `.get`)
`??`	824	left `is not None` else right	Treat `0`, `""`, `[]` as missing (unlike `or`)
`??=`	824	assign right only if left is `None`	Always evaluate the right-hand side

Short-circuiting (823): if the base is `None`, the **tail** (further `.`, `[ ]`, calls, and nested `?.` / `?[ ]`) is skipped. Parentheses **break** that chain: `(a?.b).c` raises if `a` is `None`, because the group finishes first.

Worked examples

Case 1: What ships today (verified on this box)

```
# pep823_today.py - CPython 3.13.5 on the writing box
import ast
import sys

print(sys.version.split()[0])
for src in ("a?.b", "a?[0]", "a ?? b", "a ??= b"):
    try:
        ast.parse(src)
        print(src, "→ parsed (unexpected on 3.13)")
    except SyntaxError as e:
        print(f"{src!r} → SyntaxError: {e.msg}")
```

```
$ python3 pep823_today.py
3.13.5
'a?.b' → SyntaxError: invalid syntax
'a?[0]' → SyntaxError: invalid syntax
'a ?? b' → SyntaxError: invalid syntax
'a ??= b' → SyntaxError: invalid syntax
```

Until 3.16 accepts the PEPs (if accepted), the boring equivalent is still `is not None` and `:=`.

Case 2: Nested optional attributes (PEP 823 shape)

Proposed spelling from the PEP (not valid on 3.13):

```
# pep823_customer_proposed.py - PROPOSED syntax (PEP 823); will not parse on 3.13/3.14
from dataclasses import dataclass

@dataclass
class User:
    name: str

@dataclass
class Customer:
    user: User | None

@dataclass
class Data:
    customer: Customer | None

def get_customer_name(data: Data) -> str | None:
    return data.customer?.user?.name.lower()
```

Equivalent today:

```
# pep823_customer_today.py - runnable equivalent of the PEP 823 example
from dataclasses import dataclass

@dataclass
class User:
    name: str

@dataclass
class Customer:
    user: User | None

@dataclass
class Data:
    customer: Customer | None

def get_customer_name(data: Data) -> str | None:
    if (customer := data.customer) is not None and (user := customer.user) is not None:
        return user.name.lower()
    return None


def main() -> None:
    print(get_customer_name(Data(None)))
    print(get_customer_name(Data(Customer(None))))
    print(get_customer_name(Data(Customer(User("Ada")))))
```

```
if __name__ == "__main__":
    main()
```

```
$ python3 pep823_customer_today.py
None
None
ada
```

Case 3: Structured data — `?.` is not `dict.get`

PEP 823 is explicit: operators handle **optional** values, not **missing** keys. For TypedDict / JSON dicts, combine `.get` with proposed `?[ ]`:

```
# proposed:  return data.get("customer")?["user"]?["name"].lower()
# today:
def get_customer_name(data: dict) -> str | None:
    customer = data.get("customer")
    if customer is not None:
        user = customer["user"]  # KeyError if missing - intentional
        if user is not None:
            return user["name"].lower()
    return None
```

Case 4: Coalesce without truthiness (PEP 824)

`or` collapses every falsy value. `??` (proposed) only replaces `None`:

```
# pep824_coalesce_today.py - runnable stand-in for a ?? b
def coalesce(a, b):
    return a if a is not None else b


def main() -> None:
    print(coalesce(0, 99))       # 0  - keep zero
    print(coalesce("", "x"))     # '' - keep empty string
    print(coalesce(None, "x"))   # x
    print(0 or 99)               # 99 - truthiness trap
    print("" or "x")             # x


if __name__ == "__main__":
    main()
```

```
$ python3 pep824_coalesce_today.py
0

x
99
x
```

Proposed forms from PEP 824: `age = user.age ?? "unknown"` and `user.name ??= "unknown"`.

Case 5: Assignments and short-circuit edges (823)

Per the PEP:

- `a?.b = 1` is a `SyntaxError` (Load context only).
- `a?.b.c[0].f()` short-circuits the whole primary tail if `a` is `None`.
- `(a?.b).c` does **not** — the group ends the short-circuit.
- Precedence for `??` (824): lower than `or`, higher than conditional expressions.

The trap

Reading PyCoder's Weekly or a blog demo and pasting `data.customer?.user?.name` into a 3.14 codebase. It is still a `SyntaxError`. Equally bad: assuming `?.` means "forgive missing attributes" the way some JS optional-chaining folklore suggests — PEP 823 **rejects** exception-aware operators so misspellings still raise.

The boring rule

Until 3.16 ships an accepted PEP: write `is not None` (and `??`-shaped helpers only if your team wants them). When teaching the drafts, quote the PEPs, mark **proposed**, and keep runnable examples on today's syntax.

Try this

1. Run `pep823_today.py` on your interpreter — confirm `SyntaxError`.
2. Read PEP 823 Motivation + Short-circuiting; skim Deferred Ideas so you do not invent `?()` or `!`.
3. Read PEP 824 and rewrite one `x if x is not None else y` in a scratch file as a comment showing the proposed `??` form — do not commit that syntax to production yet.
4. Optional: the PEPs link a reference implementation and an online demo; use those only as sandboxes, not as a dependency.

Sources

- PEP 823 – None-aware access operators (Draft, Python-Version: 3.16)
- PEP 824 – None-coalescing operators (Draft, Python-Version: 3.16)
- Historical context: deferred PEP 505

Functions

Defining Functions, Signatures, and Return Values

Defining Functions, Signatures, and Return Values

After reading this chapter, you will master the `def` statement, write clean docstrings, leverage multiple return values, and apply guard clauses to prevent nested logic.

Mental model

In Python, `def` is an executable statement. When CPython encounters `def func(): ...` during module execution, it creates a **function object** (`PyFunctionObject`) wrapping the compiled bytecode, and binds the function's name in the current namespace.

```
Execution of 'def':
  Source Code    Bytecode CodeObject    PyFunctionObject    Name binding
                                          - __name__
                                          - __doc__
                                          - __code__
Call Execution:
  func()    Allocates a new PyFrameObject on the call stack
             - Holds local namespace
             - Executes bytecode
             - Destroys frame upon return and yields value to caller
```

If a function completes without reaching an explicit `return` statement, Python implicitly returns `None`.

Minimal example

Save as `func_basics.py`:

```python
# func_basics.py
def calculate_metrics(values: list[float]) -> tuple[float, float, float]:
    """Compute the minimum, maximum, and average of a non-empty list of values."""
    if not values:
        raise ValueError("Cannot calculate metrics on empty values list")
```

```
    total = sum(values)
    count = len(values)
    return min(values), max(values), total / count

def main() -> None:
    latencies = [12.4, 8.9, 15.2, 11.0, 9.7]
    min_lat, max_lat, avg_lat = calculate_metrics(latencies)

    print(f"Min latency : {min_lat:.1f}ms")
    print(f"Max latency : {max_lat:.1f}ms")
    print(f"Avg latency : {avg_lat:.1f}ms")

if __name__ == "__main__":
    main()
```

Run via `uv run python func_basics.py`:

```
Min latency : 8.9ms
Max latency : 15.2ms
Avg latency : 11.4ms
```

Worked examples

Case 1: Early returns (The "Bouncer Pattern")

Deeply nested `if/else` ladders make code hard to read. Use guard clauses with early returns to handle invalid states immediately:

```
# bouncer_pattern.py
def authenticate_request(token: str | None, is_expired: bool, role: str) -> str:
    # Guard 1: Missing token
    if not token:
        return "DENIED: Missing token"

    # Guard 2: Expired token
    if is_expired:
        return "DENIED: Token expired"

    # Guard 3: Insufficient privileges
    if role != "admin":
        return "DENIED: Requires admin role"

    # Main happy path: Unnested and clear
    return "AUTHORIZED: Welcome administrator"
```

```
if __name__ == "__main__":
    print(authenticate_request(None, False, "admin"))
    print(authenticate_request("tok_123", True, "admin"))
    print(authenticate_request("tok_123", False, "guest"))
    print(authenticate_request("tok_123", False, "admin"))
```

Run:

```
uv run python bouncer_pattern.py
```

Case 2: Inspecting function introspection attributes

Because functions are first-class objects, you can inspect their metadata:

```
# func_metadata.py
def transfer_funds(sender: str, recipient: str, amount: float) -> bool:
    """Execute a secure transactional balance transfer."""
    return True

def main() -> None:
    print(f"Function Name : {transfer_funds.__name__}")
    print(f"Docstring     : {transfer_funds.__doc__}")
    print(f"Code ArgCount : {transfer_funds.__code__.co_argcount}")
    print(f"Code Varnames : {transfer_funds.__code__.co_varnames}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python func_metadata.py
```

Pitfalls

Pitfall 1: Forgetting explicit returns in branches

If one branch of a function returns a value and another branch omits `return`, the omitted branch returns `None`:

```
# Bug:
def get_user_status(active: bool) -> str:
    if active:
```

```
        return "ONLINE"
    # Forgot return: implicit return None!
```

Pitfall 2: Mutating input arguments unexpectedly

Avoid modifying mutable arguments (like lists or dictionaries) in place unless that is the explicitly documented purpose of the function. Prefer returning a new object.

Exercises

1. Write a function `parse_semver(version: str) -> tuple[int, int, int]` that parses `"3.14.2"` into integer components `(3, 14, 2)` and handles malformed strings gracefully.
2. Refactor a deeply nested 3-level `if/else` validation function into a clean flat function using guard clauses.
3. Write a function with a comprehensive docstring following the Google style guide (Args, Returns, Raises). Print `func.__doc__`.

Further reading

- Python Language Reference: *Section 8.6: Function definitions.*
- PEP 257: *Docstring Conventions.*
- Clean Code Architecture: *The Bouncer / Guard Clause Pattern.*

Parameters, Keyword Arguments, and Defaults

Parameters, Keyword Arguments, and Defaults

After reading this chapter, you will master positional vs keyword arguments, configure safe default parameters, and use modern positional-only (`/`) and keyword-only (`*`) boundary markers.

Mental model

Python provides precise control over how arguments are passed into a function signature:

```
def api_request(method, url, /, timeout=30, *, secure=True, headers=None):

                Positional-Only    Standard              Keyword-Only
                (before "/")    (positional or kw)       (after "*")
```

- **`/` (Positional-only boundary)**: Parameters before `/` must be passed by position; callers cannot pass them as `url="..."`.
- **`*` (Keyword-only boundary)**: Parameters after `*` must be passed by name (`secure=False`); prevents bugs where callers pass confusing boolean flags positionally.
- **Default arguments**: Evaluated **once** when the function is defined, not every time it is called.

Minimal example

Save as `parameter_boundaries.py`:

```python
# parameter_boundaries.py
def query_endpoint(
    service: str,          # Positional-only parameter (before '/')
    /,
    retries: int = 3,      # Standard parameter
    *,
    use_tls: bool = True,  # Keyword-only parameter (after '*')
    timeout: float = 5.0,  # Keyword-only parameter
) -> None:
    print(f"Service: {service} | Retries: {retries} | TLS: {use_tls} | Timeout: {timeout}s")

def main() -> None:
```

```
    # 1. Valid invocation
    query_endpoint("auth_svc", 5, use_tls=True, timeout=10.0)

    # 2. Valid invocation relying on defaults
    query_endpoint("billing_svc", use_tls=False)

    # 3. Invalid: query_endpoint(service="auth_svc") -> TypeError: positional-only argument
    # 4. Invalid: query_endpoint("auth_svc", 3, True) -> TypeError: takes 2 positional argum

if __name__ == "__main__":
    main()
```

Run via `uv run python parameter_boundaries.py`:

```
Service: auth_svc | Retries: 5 | TLS: True | Timeout: 10.0s
Service: billing_svc | Retries: 3 | TLS: False | Timeout: 5.0s
```

Worked examples

Case 1: Preventing the "Boolean Parameter Trap"

When functions take boolean flags positionally, call sites become unreadable (`render(True, False, True)`). Keyword-only arguments force clarity:

```
# keyword_only_api.py
def configure_cache(
    host: str,
    port: int,
    *,
    cluster_mode: bool = False,
    ssl_enabled: bool = True,
    eviction_policy: str = "lru"
) -> None:
    print(f"Cache on {host}:{port} [cluster={cluster_mode}, ssl={ssl_enabled}, policy={evict

if __name__ == "__main__":
    # Call site is self-documenting
    configure_cache("redis.internal", 6379, cluster_mode=True, ssl_enabled=True)
```

Run:

```
uv run python keyword_only_api.py
```

Case 2: The Mutable Default Argument Trap and the Fix

Because default values are evaluated at definition time, a mutable default (like `[]` or `{}`) is shared across all calls:

```
# mutable_default_fix.py
from datetime import datetime

# THE BUG:
def add_log_broken(msg: str, tags: list[str] = []) -> list[str]:
    tags.append(msg)
    return tags

# THE FIX: Use None as default, allocate inside function
def add_log_safe(msg: str, tags: list[str] | None = None) -> list[str]:
    if tags is None:
        tags = []
    tags.append(msg)
    return tags

if __name__ == "__main__":
    print("Broken with mutable default:")
    print("Call 1:", add_log_broken("error 1"))
    print("Call 2:", add_log_broken("error 2"))  # Re-uses the same list!

    print("\nSafe with None sentinel:")
    print("Call 1:", add_log_safe("error 1"))
    print("Call 2:", add_log_safe("error 2"))  # Fresh list allocated
```

Run:

```
uv run python mutable_default_fix.py
```

Output:

```
Broken with mutable default:
Call 1: ['error 1']
Call 2: ['error 1', 'error 2']

Safe with None sentinel:
Call 1: ['error 1']
Call 2: ['error 2']
```

Pitfalls

Pitfall 1: Evaluating dynamic defaults like `datetime.now()` in the signature

```
# Danger: The timestamp is frozen to when the module was loaded!
def record_event(name: str, timestamp: datetime = datetime.now()):
    pass

# Fix:
def record_event(name: str, timestamp: datetime | None = None):
    actual_time = timestamp if timestamp is not None else datetime.now()
```

Exercises

1. Define a function `format_currency(amount, /, *, currency="USD", symbol=True)` and test calling it with valid and invalid argument permutations.
2. Write a function `append_item(item, collection=None)` that safely handles list mutation and returns the updated list.
3. Use `inspect.signature` from the standard library to inspect the parameters and default values of a custom function.

Further reading

- PEP 570: *Python Positional-Only Parameters*.
- PEP 3102: *Keyword-Only Arguments*.
- Python Standard Library: `inspect.signature`.

sentinel(): Missing Is Not None (PEP 661)

sentinel(): Missing Is Not None (PEP 661)

Thesis

`None` means "no value", but very often `None` *is* a value: a user with no role, a JSON `null`, a timeout deliberately unset. When `None` is legitimate data, a function needs a *different* marker for "the caller passed nothing" or "the key was not there". For years the idiom was `_missing = object()`. That works for `is`, but it prints as `<object object at 0x…>` in every signature, it has no type you can annotate, and `deepcopy` silently breaks it. Python 3.15 adds the builtin **`sentinel(name)`** (PEP 661, *Final*). It gives the marker a readable repr, a type usable in annotations, and identity that survives copy and pickle. Remember the two things it does not do: it is **truthy**, and each call creates a **new** object.

Version note: `sentinel` is new in **Python 3.15**. On 3.14 the name is a `NameError`. Every listing below was run on **CPython 3.15.0** (final, released 2026-10-09, Linux x86-64), and the outputs are copied from those runs. The compatibility listing also ran on 3.14.8 with `typing_extensions`.

Mental model

```
MISSING = sentinel("MISSING")

                          one object, created once, at import
          __name__   = "MISSING"   → repr, str, and the pickle lookup key
          __module__ = "__main__"  → where pickle re-imports it from (writable)
          type       = sentinel    → final; usable as `int | MISSING`

 test with:   value is MISSING          never:  if not value / if value
```

A sentinel is an *identity*, not a value. You compare it with `is`, the same way you compare `None`. Everything else (repr, typing, pickling) exists so that this identity stays readable and stable as it passes through the rest of your program.

Worked examples

Case 1: What a sentinel is, and what it is not

```
# basics.py - what a sentinel is, and what it is not
import copy, pickle

MISSING = sentinel("MISSING")

print(repr(MISSING), type(MISSING).__name__, MISSING.__module__)
print("is itself:", MISSING is MISSING, "| equals None:", MISSING == None)
print("bool:", bool(MISSING))
print("copy keeps identity:", copy.copy(MISSING) is MISSING, copy.deepcopy(MISSING) is MISSI
print("pickle keeps identity:", pickle.loads(pickle.dumps(MISSING)) is MISSING)
print("two calls, two objects:", sentinel("MISSING") is MISSING)
print("union:", int | MISSING)
try:
    class Sub(sentinel): pass
except TypeError as e:
    print("subclass:", e)
```

```
$ python3.15 basics.py
MISSING sentinel __main__
is itself: True | equals None: False
bool: True
copy keeps identity: True True
pickle keeps identity: True
two calls, two objects: False
union: int | MISSING
subclass: type 'sentinel' is not an acceptable base type
```

Read the output line by line. The repr is the name with no address noise. The sentinel equals only itself. `copy`, `deepcopy` and `pickle` all return *the same object*, because pickling records "module `__main__`, name `MISSING`" and unpickling looks that name up again. Calling `sentinel("MISSING")` a second time gives a **different** object. There is no registry, so the name is a label, not a key. The type is final.

Case 2: When `None` is real data

```
# cache.py - None is a real value here, so "not found" needs its own marker
NOT_FOUND = sentinel("NOT_FOUND")

_cache: dict[str, str | None] = {"alice": "admin", "bob": None}   # bob: known, no role

def lookup(user: str, default: str | None | NOT_FOUND = NOT_FOUND) -> str | None:
```

```
    value = _cache.get(user, NOT_FOUND)
    if value is NOT_FOUND:
        if default is NOT_FOUND:
            raise KeyError(user)
        return default
    return value

print(lookup("alice"), lookup("bob"), lookup("carol", default="guest"))
try:
    lookup("carol")
except KeyError as e:
    print("KeyError:", e)
import inspect
print("signature:", inspect.signature(lookup))
```

```
$ python3.15 cache.py
admin None guest
KeyError: 'carol'
signature: (user: str, default: str | None | NOT_FOUND = NOT_FOUND) -> str | None
```

`bob` is in the cache with role `None`. Without a separate marker, `lookup("bob")` could not tell "bob has no role" apart from "bob is unknown". The signature line is the readability win. Compare it with the old idiom:

```
# old.py - the idiom sentinel() replaces
import copy
_missing = object()
def f(x=_missing): ...
print(f.__defaults__, copy.deepcopy(_missing) is _missing)
```

```
$ python3.15 old.py
(<object object at 0x7f9258280240>,) False
```

`help()` and `inspect.signature` show the address, and `deepcopy` produces a *new* `object()`. That second problem is the real bug: a config object deep-copied before being passed on suddenly stops matching `is _missing`.

Case 3: Supporting 3.14 and 3.15 from one module

```
# compat.py - one definition that works on 3.14 and 3.15+
import sys
if sys.version_info >= (3, 15):
    MISSING = sentinel("MISSING")
else:
```

```
    from typing_extensions import Sentinel
    MISSING = Sentinel("MISSING")
print(sys.version_info[:2], repr(MISSING))
```

```
$ python3.15 compat.py
(3, 15) MISSING
$ uv run --python 3.14 --with typing_extensions compat.py
(3, 14) MISSING
```

`typing_extensions.Sentinel` is the backport. PEP 661 says its behaviour does not *precisely* match the builtin, so treat it as a bridge, not a twin. Drop the branch when your floor reaches 3.15.

Typing

PEP 661 lets a sentinel stand for its own one-member type, the way `None` does. Write `NAME = sentinel("NAME")` with the same name on both sides. Type checkers are asked to flag a mismatch, and to narrow `int | MISSING` on `is` / `is not`. At runtime `int | MISSING` builds a real `typing.Union`, as Case 1 printed. Check your checker's release notes before relying on narrowing in CI, because support follows the PEP rather than shipping with CPython.

The trap

```
# trap.py - the truthiness trap and the local-sentinel pickle trap
import pickle
UNSET = sentinel("UNSET")

def configure(timeout=UNSET):
    if not timeout:            # wrong: UNSET is truthy, and 0 is a real timeout
        return "default 30s"
    return f"timeout={timeout}"

print(configure(), "|", configure(0))

def make():
    local = sentinel("local")
    return local
try:
    pickle.dumps(make())
except Exception as e:
    print(type(e).__name__ + ":", e)
```

```
$ python3.15 trap.py
timeout=UNSET | default 30s
PicklingError: Can't pickle local: it's not found as __main__.local
```

Both results are wrong, and both arrive silently. `UNSET` is **truthy**: PEP 661 deliberately kept the default object behaviour and rejected falsy sentinels. So `not timeout` treats "not given" as a real timeout, while a real `0` falls back to the default. Second, a sentinel created inside a function cannot be found again by module and name, so pickling it fails. That matters the moment the object crosses a `multiprocessing` boundary.

The boring rule

Use `sentinel()` only when `None` is valid data. Define it **once**, at module level, with `NAME = sentinel("NAME")`. Test it **only** with `is` / `is not`. Never create sentinels inside functions that might be pickled. For several related markers with an order (`PENDING`, `DONE`, …), use an `Enum` instead.

Try this

1. Change `_cache.get(user, NOT_FOUND)` in Case 2 to `_cache.get(user)` and find the input that now returns the wrong answer.
2. Put a sentinel in a class body (`class Config: UNSET = sentinel("Config.UNSET")`) and check whether it round-trips through `pickle`. Read PEP 661's note on `__module__` to explain the result.
3. Send `MISSING` through `multiprocessing.Pool.map` and confirm the worker still sees `value is MISSING`.
4. Run your type checker over Case 2 with `reveal_type(value)` inside both branches of `if value is NOT_FOUND`.

Sources

- PEP 661, *Sentinel Values* (Final, Python 3.15): https://peps.python.org/pep-0661/
- Python 3.15 documentation, built-in `sentinel`: https://docs.python.org/3.15/builtins/functions.html#sentinel
- What's New in Python 3.15: https://docs.python.org/3.15/whatsnew/3.15.html
- `typing_extensions.Sentinel`: https://typing-extensions.readthedocs.io/

Variadic Arguments (*args and **kwargs)

Variadic Arguments (*args and **kwargs)

After reading this chapter, you will master variadic parameters, construct flexible APIs using `*args` and `**kwargs`, and write transparent argument forwarding wrappers.

Mental model

Variadic parameters allow a function to accept an arbitrary number of positional or keyword arguments: - `*args`: Collects extra positional arguments into a **`tuple`**. - `**kwargs`: Collects extra keyword arguments into a **`dict`**.

```
Call Site:
  send_payload("POST", 200, "fast", verbose=True, timeout=10)


  method: "POST"

                       args: (tuple)           kwargs: (dict)
                       (200, "fast")           {"verbose": True, ...}
```

At call sites, the operators can be reversed: `*` unpacks an iterable into positional arguments, and `**` unpacks a mapping into keyword arguments.

Minimal example

Save as `variadic_demo.py`:

```
# variadic_demo.py
def format_log(level: str, *messages: str, **metadata: str | int) -> str:
    joined_msg = " | ".join(messages)
    meta_str = " ".join(f"[{k}={v}]" for k, v in metadata.items())
    return f"[{level.upper()}] {joined_msg} {meta_str}".strip()

def main() -> None:
    # Multiple positional messages, multiple keyword attributes
```

```
    entry = format_log(
        "info",
        "Connection established",
        "Handshake completed",
        host="node-alpha",
        port=9000,
        region="us-west-2"
    )
    print(entry)

if __name__ == "__main__":
    main()
```

Run via `uv run python variadic_demo.py`:

```
[INFO] Connection established | Handshake completed [host=node-alpha] [port=9000] [region=us-we
```

Worked examples

Case 1: Argument forwarding in decorator wrappers

When authoring wrappers or logging hooks, you must forward arguments cleanly to the wrapped target function:

```
# wrapper_forward.py
import time
from collections.abc import Callable
from typing import Any

def time_execution(func: Callable[..., Any], *args: Any, **kwargs: Any) -> Any:
    start = time.perf_counter()
    result = func(*args, **kwargs)  # Perfect forwarding with * and **
    elapsed = time.perf_counter() - start
    print(f"Function '{func.__name__}' executed in {elapsed * 1000:.3f}ms")
    return result

def compute_heavy_task(base: int, power: int, tag: str = "computation") -> int:
    return base ** power

if __name__ == "__main__":
    out = time_execution(compute_heavy_task, 2, 500_000, tag="benchmark")
    print(f"Task completed: result has {len(str(out))} digits")
```

Run:

```
uv run python wrapper_forward.py
```

Case 2: Unpacking dictionaries and lists at call sites

You can dynamically assemble parameters in dictionaries and pass them into functions using `**`:

```
# dynamic_call.py
def connect_db(host: str, port: int, database: str, timeout: int = 30) -> None:
    print(f"Connecting to {database} at {host}:{port} (timeout={timeout}s)")

if __name__ == "__main__":
    config = {
        "host": "postgresql.internal",
        "port": 5432,
        "database": "analytics",
        "timeout": 60,
    }
    # Unpack config dictionary directly into function arguments
    connect_db(**config)
```

Run:

```
uv run python dynamic_call.py
```

Pitfalls

Pitfall 1: Signature ordering violations

Python enforces strict parameter ordering: 1. Standard positional parameters 2. Positional-only boundary `/` 3. `*args` 4. Keyword-only parameters 5. `**kwargs`

Placing `*args` after `**kwargs` raises a `SyntaxError`.

Pitfall 2: Overusing `**kwargs` and destroying API readability

While `**kwargs` offers ultimate flexibility, overusing it hides what parameters a function actually accepts from documentation, autocomplete, and type checkers. Prefer explicit named parameters for core options.

Exercises

1. Write a function `product(*numbers: float) -> float` that computes the product of all passed numbers, returning `1.0` if no arguments are supplied.
2. Implement a function `merge_configs(*configs: dict) -> dict` that merges an arbitrary number of configuration dictionaries into a single dictionary in order.
3. Write a wrapper function `log_calls(func, *args, **kwargs)` that prints all passed arguments before executing `func`.

Further reading

- Python Tutorial: *Arbitrary Argument Lists*.
- PEP 448: *Additional Unpacking Generalizations*.
- Fluent Python (2nd Edition): *Chapter 7: Functions as First-Class Objects*.

First-Class Functions, Callables, and Lambdas

First-Class Functions, Callables, and Lambdas

After reading this chapter, you will treat functions as first-class citizens, build command dispatch tables, author concise lambda expressions, and inspect callables.

Mental model

In Python, **functions are first-class objects**. This means a function is just an object like any other integer or string: 1. It can be assigned to a variable name. 2. It can be stored in a collection (`list`, `dict`, `set`). 3. It can be passed as an argument to another function. 4. It can be returned as the result of another function.

```
PyFunctionObject: calculate()

                stored in

command_table = {
  "calc": calculate,
  "ping": health_check,
}

                looked up & called
command_table["calc"](args)
```

An anonymous function (`lambda`) is an inline function defined without a `def` statement, limited to a single expression.

Minimal example

Save as `first_class_demo.py`:

```python
# first_class_demo.py
from collections.abc import Callable

def add(a: int, b: int) -> int:
```

```
    return a + b

def multiply(a: int, b: int) -> int:
    return a * b

def execute_operation(op: Callable[[int, int], int], x: int, y: int) -> int:
    """Higher-order function accepting a function as an argument."""
    return op(x, y)

def main() -> None:
    # 1. Pass function by name
    res1 = execute_operation(add, 10, 5)
    res2 = execute_operation(multiply, 10, 5)

    # 2. Pass anonymous lambda
    res3 = execute_operation(lambda a, b: a - b, 10, 5)

    print(f"Add      : {res1}")
    print(f"Multiply : {res2}")
    print(f"Lambda   : {res3}")

if __name__ == "__main__":
    main()
```

Run via `uv run python first_class_demo.py`:

```
Add      : 15
Multiply : 50
Lambda   : 5
```

Worked examples

Case 1: Command Dispatch Tables (Replacing `if/elif` ladders)

Instead of a long, brittle ladder of `if/elif` statements, use a dictionary of functions:

```
# dispatch_table.py
from collections.abc import Callable

def handle_start(target: str) -> str:
    return f"Started service: {target}"

def handle_stop(target: str) -> str:
    return f"Stopped service: {target}"
```

```
def handle_restart(target: str) -> str:
    return f"Restarted service: {target}"

DISPATCH_REGISTRY: dict[str, Callable[[str], str]] = {
    "start": handle_start,
    "stop": handle_stop,
    "restart": handle_restart,
}

def dispatch_command(cmd: str, target: str) -> str:
    handler = DISPATCH_REGISTRY.get(cmd)
    if not handler:
        raise ValueError(f"Unknown command: {cmd!r}. Available: {list(DISPATCH_REGISTRY.keys
    return handler(target)

if __name__ == "__main__":
    print(dispatch_command("start", "nginx"))
    print(dispatch_command("restart", "postgres"))
```

Run:

```
uv run python dispatch_table.py
```

Case 2: Custom sorting keys with lambdas and operator

When sorting complex structured data, pass a key function to `sorted()`:

```
# sorting_keys.py
import operator

def main() -> None:
    containers = [
        {"name": "worker-1", "cpu_percent": 84.5, "memory_mb": 512},
        {"name": "worker-2", "cpu_percent": 22.1, "memory_mb": 1024},
        {"name": "worker-3", "cpu_percent": 91.0, "memory_mb": 256},
    ]

    # Sort by CPU descending using a lambda
    by_cpu = sorted(containers, key=lambda c: c["cpu_percent"], reverse=True)
    print("Sorted by CPU (descending):")
    for c in by_cpu:
        print(f"  {c['name']:10s} -> {c['cpu_percent']}%")

    # Sort by memory using operator.itemgetter (faster in C)
    by_mem = sorted(containers, key=operator.itemgetter("memory_mb"))
```

```
    print("\nSorted by memory (ascending):")
    for c in by_mem:
        print(f"  {c['name']:10s} -> {c['memory_mb']}MB")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python sorting_keys.py
```

Pitfalls

Pitfall 1: Over-complicating lambdas

Lambdas should be simple one-liners (typically for `sorted(key=...)`). If your lambda needs complex logic, loops, or multiple statements, write a standard named `def` function.

Pitfall 2: Assigning a lambda to a variable instead of `def`

```
# Bad style (violates PEP 8):
my_func = lambda x: x * 2

# Good style:
def my_func(x):
    return x * 2
```

Exercises

1. Build a text pipeline function `process_text(text: str, transforms: list[Callable[[str], str]]) -> str` that applies a list of transformation functions sequentially.
2. Given a list of tuples `[(1, "b"), (3, "a"), (2, "c")]`, sort them alphabetically by the second element using a lambda key.
3. Use the built-in `callable()` function to verify which standard objects in a mixed list can be called as functions.

Further reading

- Python Documentation: *Functional Programming HOWTO*.
- Python Standard Library: `operator` module (*itemgetter, attrgetter*).
- PEP 8: *Programming Recommendations regarding lambda assignment*.

Closures, Lexical Scopes, and Factory Functions

Closures, Lexical Scopes, and Factory Functions

After reading this chapter, you will understand how closures capture enclosing lexical scopes, author factory functions, and avoid the notorious late-binding loop trap.

Mental model

A **closure** occurs when a nested function references a variable defined in its enclosing scope, and that outer function finishes execution. The inner function retains access to that variable even after the outer function's stack frame has returned:

```
Outer Function: make_multiplier(factor=3)
  Frame executes and terminates.


Heap Memory: Cell Object

    cell_contents: 3               (Held alive by reference count)

                   referenced by
Inner Function: multiplier(x)

    __code__    : bytecode
    __closure__ : (cell, )
```

The captured variables are stored in CPython as `cell` objects inside the function's `__closure__` attribute.

Minimal example

Save as `closure_demo.py`:

```python
# closure_demo.py
from collections.abc import Callable
```

```
def make_multiplier(factor: int) -> Callable[[int], int]:
    """Factory function creating specialized multiplier closures."""
    def multiplier(number: int) -> int:
        return number * factor

    return multiplier

def main() -> None:
    double = make_multiplier(2)
    triple = make_multiplier(3)

    print(f"double(10): {double(10)}")
    print(f"triple(10): {triple(10)}")

    # Inspect the closure cell
    if double.__closure__:
        cell_val = double.__closure__[0].cell_contents
        print(f"double closure factor cell: {cell_val}")

if __name__ == "__main__":
    main()
```

Run via `uv run python closure_demo.py`:

```
double(10): 20
triple(10): 30
double closure factor cell: 2
```

Worked examples

Case 1: Stateful API Rate Limiter using `nonlocal`

Closures can encapsulate private state without requiring a full class:

```
# rate_limiter.py
from collections.abc import Callable
import time

def make_rate_limiter(max_requests: int, window_seconds: float) -> Callable[[], bool]:
    request_timestamps: list[float] = []

    def allow_request() -> bool:
        nonlocal request_timestamps
        now = time.time()
```

```
        # Prune timestamps outside window
        request_timestamps = [t for t in request_timestamps if now - t < window_seconds]

        if len(request_timestamps) < max_requests:
            request_timestamps.append(now)
            return True
        return False

    return allow_request

if __name__ == "__main__":
    limiter = make_rate_limiter(max_requests=2, window_seconds=1.0)
    print("Req 1:", limiter())  # True
    print("Req 2:", limiter())  # True
    print("Req 3:", limiter())  # False (exceeded limit!)
```

Run:

```
uv run python rate_limiter.py
```

Case 2: The Notorious Late-Binding Loop Trap

When creating closures in a loop, inner functions do not capture the variable's *value at that iteration*; they capture the *variable reference itself*. By the time they are called, the loop has completed!

```
# late_binding_trap.py
def create_multipliers_broken():
    # THE TRAP: All lambdas reference the single variable 'i'
    return [lambda x: x * i for i in range(4)]

def create_multipliers_fixed():
    # THE FIX: Bind 'i' as a default argument evaluated at definition time
    return [lambda x, i=i: x * i for i in range(4)]

if __name__ == "__main__":
    print("Broken (late binding):")
    funcs_broken = create_multipliers_broken()
    print([f(10) for f in funcs_broken])  # [30, 30, 30, 30] !

    print("\nFixed (default argument binding):")
    funcs_fixed = create_multipliers_fixed()
    print([f(10) for f in funcs_fixed])   # [0, 10, 20, 30]
```

Run:

```
uv run python late_binding_trap.py
```

Output:

```
Broken (late binding):
[30, 30, 30, 30]

Fixed (default argument binding):
[0, 10, 20, 30]
```

Pitfalls

Pitfall 1: Retaining large memory references in closures

Because closure cells hold strong references, an inner function will keep an entire enclosing object alive in memory even if it only needs one small sub-property. Extract the needed sub-property into a local variable before closing over it.

Exercises

1. Write a function `make_prefixer(prefix: str)` that returns a function that prepends `prefix` to any passed string.
2. Build an accumulator function `make_accumulator(initial: float = 0.0)` that uses `nonlocal` to accumulate added numbers and return the current running total.
3. Write a script demonstrating the late-binding trap with button or task callback functions, and implement both the default-argument fix and `functools.partial` fix.

Further reading

- Python Language Reference: *Section 4.1: Naming and Binding (Cell objects).*
- PEP 227: *Statically Nested Scopes.*
- Python Standard Library: `functools.partial`.

Generator Functions and Subgenerator Delegation

Generator Functions and Subgenerator Delegation

After reading this chapter, you will master the mechanics of generator functions (`yield`), establish bi-directional communication pipelines using `.send()`, inject exceptions and trigger cleanup via `.throw()` and `.close()`, and delegate execution to nested subgenerators using `yield from` (PEP 380).

Mental model

An ordinary Python function begins execution, computes a result, and exits, destroying its stack frame. A **generator function** (any function containing the `yield` keyword) returns a generator object whose execution state is preserved on the heap across calls:

```
Standard Function:
  call()    [ Execute Frame ]    return    Frame destroyed, state lost.

Generator Function (Suspension & Resumption):
  gen = my_gen()    Allocates generator frame on heap (GEN_CREATED)

  next(gen)         Runs to 'yield X'    Suspends (GEN_SUSPENDED), yields X

  gen.send(Y)       Resumes at 'yield', receives Y, runs to next yield...

  return            Raises StopIteration (GEN_CLOSED), frame cleaned up.
```

The `yield from` Bi-Directional Pipe (PEP 380)

`yield from` is not merely a shorthand for `for item in subgen: yield item`. It opens an **unimpeded transparent bi-directional channel** between the outer caller and the inner subgenerator:

```
Caller                                  Subgenerator
                    Direct Bi-Directional Channel
             (Values, exceptions, and returns flow directly;
              the delegating generator is completely bypassed!)
```

Minimal example

Save as `generator_delegation.py`:

```python
# generator_delegation.py
from collections.abc import Generator, Iterator

# 1. Producer subgenerator
def counter_subgen(start: int, stop: int) -> Iterator[int]:
    """Yield integers from start up to stop."""
    current = start
    while current < stop:
        yield current
        current += 1

# 2. Delegating generator using 'yield from'
def compound_pipeline() -> Iterator[int | str]:
    print("  [Delegator] Starting pipeline...")
    # yield from establishes direct pipe to counter_subgen
    yield from counter_subgen(1, 4)
    yield "MIDPOINT"
    yield from counter_subgen(10, 13)
    print("  [Delegator] Pipeline finished.")

def main() -> None:
    for item in compound_pipeline():
        print(f"Pulled: {item}")

if __name__ == "__main__":
    main()
```

Run via `uv run python generator_delegation.py`:

```
  [Delegator] Starting pipeline...
Pulled: 1
Pulled: 2
Pulled: 3
Pulled: MIDPOINT
Pulled: 10
Pulled: 11
Pulled: 12
  [Delegator] Pipeline finished.
```

Worked examples

Case 1: Two-Way Coroutines for Rolling Averages with `.send()`

When `yield` appears on the right-hand side of an assignment (`received = yield output`), the generator can **consume data** sent by the caller via `gen.send(value)`:

```
# rolling_average.py
from collections.abc import Generator

def rolling_averager() -> Generator[float, float, None]:
    """Continuously receive incoming float samples and yield the cumulative average."""
    total = 0.0
    count = 0
    average = 0.0

    while True:
        # Suspends yielding current average; receives next incoming sample via .send()
        sample = yield average
        total += sample
        count += 1
        average = total / count

def main() -> None:
    calc = rolling_averager()

    # Priming the generator: Must advance to the first yield before sending data!
    next(calc)  # Returns initial 0.0

    samples = [10.0, 20.0, 60.0, 10.0]
    print("Streaming Telemetry Samples to Coroutine:")
    for sample in samples:
        curr_avg = calc.send(sample)
        print(f"  Sent sample: {sample:4.1f} -> Current Rolling Average: {curr_avg:.2f}")

    # Cleanly terminate the coroutine
    calc.close()

if __name__ == "__main__":
    main()
```

Run:

```
uv run python rolling_average.py
```

Output:

```
Streaming Telemetry Samples to Coroutine:
```

```
Sent sample: 10.0 -> Current Rolling Average: 10.00
Sent sample: 20.0 -> Current Rolling Average: 15.00
Sent sample: 60.0 -> Current Rolling Average: 30.00
Sent sample: 10.0 -> Current Rolling Average: 25.00
```

Case 2: Deep Tree Traversal and Recursive Flattening with `yield from`

Nested data structures (such as nested JSON documents, file hierarchies, or AST nodes) can be flattened recursively without managing manual stack arrays:

```python
# tree_flattener.py
from collections.abc import Iterator
from typing import Any

def flatten_tree(element: Any) -> Iterator[Any]:
    """Recursively flatten arbitrarily nested lists and primitives into a flat stream."""
    if isinstance(element, list):
        for item in element:
            # Recursively delegate to subgenerator
            yield from flatten_tree(item)
    else:
        yield element

def main() -> None:
    nested_topology = [
        "gateway-01",
        ["switch-a", ["node-01", "node-02"]],
        ["switch-b", ["node-03", ["storage-01", "storage-02"]]],
        "firewall-01",
    ]

    print("Flattened Infrastructure Inventory:")
    for device in flatten_tree(nested_topology):
        print(f"  Device: {device}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python tree_flattener.py
```

Output:

```
Flattened Infrastructure Inventory:
```

```
Device: gateway-01
Device: switch-a
Device: node-01
Device: node-02
Device: switch-b
Device: node-03
Device: storage-01
Device: storage-02
Device: firewall-01
```

Case 3: Capturing Return Values from Subgenerators via `yield from`

In Python generators, a `return value` statement does not return to a caller like a regular function; it raises `StopIteration(value)`. When consumed by `yield from`, the subgenerator's return value is **automatically assigned to the target variable**:

```python
# subgen_returns.py
from collections.abc import Generator

def accumulate_metrics() -> Generator[None, int, tuple[int, int]]:
    """Accumulate integers sent via .send(); return (count, total) when finished."""
    total = 0
    count = 0
    while True:
        val = yield
        if val is None:  # Sentinel value signaling end of batch
            break
        total += val
        count += 1
    # The return value is embedded inside StopIteration
    return (count, total)

def batch_collector() -> Generator[None, int, str]:
    print("  [Collector] Awaiting batch...")
    # yield from captures the subgenerator's RETURN value directly!
    count, total = yield from accumulate_metrics()
    return f"Batch summary: Processed {count} items with sum {total}"

def main() -> None:
    collector = batch_collector()
    # Prime the generator
    collector.send(None)

    # Stream items into the subgenerator
```

```
    for num in [15, 25, 60]:
        collector.send(num)

    # Send sentinel None to trigger return
    try:
        collector.send(None)
    except StopIteration as exit_info:
        summary_result = exit_info.value
        print(f"\nFinal Result: {summary_result}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python subgen_returns.py
```

Output:

```
  [Collector] Awaiting batch...

Final Result: Batch summary: Processed 3 items with sum 100
```

Pitfalls

Pitfall 1: Sending Data to an Unprimed Generator

Attempting to call `.send(value)` on a generator that has not yet been advanced to its first `yield` raises `TypeError`:

```
def my_coro():
    val = yield

c = my_coro()

# THE TRAP:
try:
    c.send("first_item")  # TypeError!
except TypeError as err:
    print(f"Caught: {err}")
```

Output:

```
Caught: can't send non-None value to a just-started generator
```

```
# THE FIX: Prime with next() or .send(None)
c = my_coro()
next(c)          # Advanced to initial yield (ready to receive)
c.send("first_item")
```

Pitfall 2: Re-iterating an Exhausted Generator

Just like generator expressions, generator functions produce single-pass iterators. Once execution exits or returns, the generator is closed:

```
def generate_ids():
    yield 101
    yield 102

ids = generate_ids()
first_pass = list(ids)  # [101, 102]
second_pass = list(ids) # []  <-- EMPTY! Generator is exhausted!

print(f"First: {first_pass}, Second: {second_pass}")
```

Exercises

1. Implement a generator function `tail_f(lines: list[str])` that yields lines one by one, simulating continuous streaming.
2. Build a coroutine `running_median()` using `yield` and `bisect.insort` that receives numbers via `.send()` and yields the current median.
3. Write a recursive generator function using `yield from` that traverses a directory structure on disk and yields all files ending in `.log`.
4. Demonstrate how `.throw(RuntimeError("Abort"))` can be caught inside a generator function's `try ... except` block to execute cleanup logic.
5. Create a generator that yields Fibonacci numbers up to `max_val`, and measure its memory usage compared to generating a full list of Fibonacci numbers.

Further reading

- PEP 342: *Coroutines via Enhanced Generators* (`.send()`, `.throw()`, `.close()`).
- PEP 380: *Syntax for Delegating to a Subgenerator* (`yield from`).
- Luciano Ramalho: *Fluent Python* (Chapter 16: Coroutines and Generators).
- David Beazley: *Generators: The Sneaky Path to Advanced Python*.

Control Flow

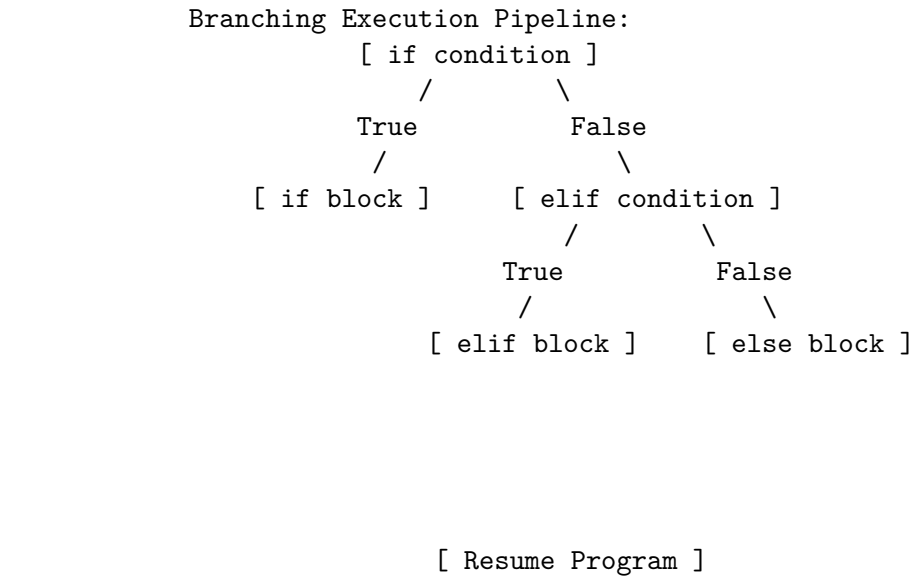
The if, elif, and else Statements

The `if`, `elif`, and `else` Statements

After reading this chapter, you will master multi-branch decision trees using `if`, `elif`, and `else`, evaluate truthiness and falsy values across standard types, leverage short-circuiting for defensive execution, and apply the Bouncer pattern to eliminate deeply nested conditional logic.

Mental model

Python evaluates conditional blocks linearly from top to bottom. As soon as a branch condition evaluates to a truthy value, CPython executes that branch's block and immediately skips all subsequent `elif` and `else` blocks:



Short-Circuit Evaluation Guarding

Logical operators `and` and `or` stop evaluating operands as soon as the outcome is mathematically guaranteed:

- `A and B`: If `A` is falsy, Python returns `A` immediately without ever evaluating `B`.
- `A or B`: If `A` is truthy, Python returns `A` immediately without ever evaluating `B`.

```
Defensive Guard Pipeline:
  user is not None and user.is_active and check_permissions(user)

           If user is None      Short-circuits! Never queries user.is_active
           If user is not None

                    If not active    Short-circuits! Never calls check_permissions()
```

Minimal example

Save as `if_decision_trees.py`:

```python
# if_decision_trees.py
def classify_http_status(status_code: int) -> str:
    """Classify HTTP response codes using a clean decision ladder."""
    if 200 <= status_code < 300:
        return "SUCCESS"
    elif 300 <= status_code < 400:
        return "REDIRECTION"
    elif 400 <= status_code < 500:
        return "CLIENT_ERROR"
    elif 500 <= status_code < 600:
        return "SERVER_ERROR"
    else:
        return "INVALID_STATUS"

def main() -> None:
    test_codes = [200, 301, 404, 503, 999]
    for code in test_codes:
        category = classify_http_status(code)
        print(f"HTTP {code:3d} -> Category: {category}")

if __name__ == "__main__":
    main()
```

Run via `uv run python if_decision_trees.py`:

```
HTTP 200 -> Category: SUCCESS
HTTP 301 -> Category: REDIRECTION
HTTP 404 -> Category: CLIENT_ERROR
HTTP 503 -> Category: SERVER_ERROR
HTTP 999 -> Category: INVALID_STATUS
```

Worked examples

Case 1: The Bouncer Pattern (Guard Clauses vs Arrow Anti-Pattern)

Deeply nested `if` statements create the "Arrow Anti-Pattern" (code indenting further and further to the right). The **Bouncer Pattern** inverts checks into early returns, keeping the happy path at indentation level 0:

```
# bouncer_pattern.py
from typing import Any

# THE ANTI-PATTERN (Arrow Code):
def process_request_nested(request: dict[str, Any]) -> str:
    if request:
        if "user" in request:
            if request["user"].get("is_authenticated"):
                if request["user"].get("has_permission"):
                    return "Action Authorized"
                else:
                    return "Forbidden: Missing Permission"
            else:
                return "Unauthorized: User Not Authenticated"
        else:
            return "Bad Request: No User Field"
    else:
        return "Bad Request: Empty Payload"

# THE IDIOMATIC PATTERN (Bouncer Clauses):
def process_request_clean(request: dict[str, Any]) -> str:
    # Guard 1: Empty request
    if not request:
        return "Bad Request: Empty Payload"

    # Guard 2: Missing user
    user = request.get("user")
    if not user:
        return "Bad Request: No User Field"

    # Guard 3: Authentication
    if not user.get("is_authenticated"):
        return "Unauthorized: User Not Authenticated"

    # Guard 4: Permission
    if not user.get("has_permission"):
        return "Forbidden: Missing Permission"

    # Happy path at top-level indentation
```

```
    return "Action Authorized"

if __name__ == "__main__":
    sample_request = {
        "user": {
            "is_authenticated": True,
            "has_permission": True,
        }
    }
    print("Nested result:", process_request_nested(sample_request))
    print("Clean result :", process_request_clean(sample_request))
```

Run:

```
uv run python bouncer_pattern.py
```

Output:

```
Nested result: Action Authorized
Clean result : Action Authorized
```

Case 2: Truthiness and The Numeric `0` Trap

Every Python object possesses a boolean truth value. In Python, the following objects evaluate to `False`: - `None` and `False` - Numeric zeros: `0`, `0.0`, `0j` - Empty containers: `""`, `()`, `[]`, `{}`, `set()`

Everything else evaluates to `True`. When validating numeric metrics, checking `if not metric:` incorrectly treats `0` as an error:

```
# falsy_metric_audit.py
def record_dropped_packets(count: int | None) -> None:
    # DANGEROUS: If count is 0, 'not count' evaluates to True!
    if not count:
        print(f"[Naive Check] Invalid or zero metric: {count}")

    # SAFE: Explicitly check for None
    if count is None:
        print("[Safe Check] Metric was omitted (None).")
    else:
        print(f"[Safe Check] Valid metric recorded: {count} dropped packets.")

if __name__ == "__main__":
    record_dropped_packets(0)     # Zero is a perfectly healthy metric!
    record_dropped_packets(None)  # Truly missing metric
```

Run:

```
uv run python falsy_metric_audit.py
```

Output:

```
[Naive Check] Invalid or zero metric: 0
[Safe Check] Valid metric recorded: 0 dropped packets.
[Naive Check] Invalid or zero metric: None
[Safe Check] Metric was omitted (None).
```

Pitfalls

Pitfall 1: Comparing Explicitly to `True` or `False`

```
# ANTI-PATTERN:
if is_ready == True:
    pass

# ANTI-PATTERN:
if is_ready is True:
    pass

# IDIOMATIC PYTHON:
if is_ready:
    pass
```

Direct comparisons to `True` break custom classes implementing `__bool__()` and violate Pythonic conventions.

Pitfall 2: Combining `and` with `or` Without Parentheses

Python evaluates `not` before `and`, and `and` before `or`. Ambiguous unparenthesized chains cause security bypasses:

```
# AMBIGUOUS:
# is_admin or is_manager and is_verified
# Python parses this as: is_admin or (is_manager and is_verified)

# EXPLICIT AND SAFE: Always group multi-operator logic
is_allowed = (is_admin or is_manager) and is_verified
```

Exercises

1. Write an `if/elif/else` function `classify_temperature(temp_c: float) -> str` that categorizes temperatures into `"FREEZING"`, `"COLD"`, `"MODERATE"`, and `"HOT"` using chained comparisons (`0.0 <= temp_c < 15.0`).
2. Refactor a three-level nested dictionary check into guard clauses using the Bouncer pattern.
3. Demonstrate why `bool([])` is `False` while `bool([0])` is `True`.
4. Write a conditional check that safely inspects `sys.argv[1]` only if `len(sys.argv) > 1` using short-circuit evaluation.

Further reading

- Python Language Reference: *The `if` statement.*
- Python Standard Library: *Truth Value Testing.*
- PEP 8: *Style Guide for Python Code — Programming Recommendations for if statements.*

Conditional Expressions and the Walrus Operator

Conditional Expressions and the Walrus Operator

After reading this chapter, you will master Python's inline conditional expression (`X if C else Y`), leverage the walrus operator (`:=`) to bind variables directly inside conditional checks, evaluate expression precedence, and recognize when inline logic improves clarity versus when it degrades readability.

Mental model

Unlike an `if/else` **statement** (which directs control flow across code blocks), a **conditional expression** (often called the ternary operator) evaluates to a single value that can be assigned directly:

```
Statement vs Expression:
  Statement (Imperative):
    if latency > 100.0:
        status = "CRITICAL"
    else:
        status = "HEALTHY"

  Conditional Expression (Declarative):
    status = "CRITICAL" if latency > 100.0 else "HEALTHY"

               True Val        Condition         False Val
```

The Walrus Operator Pipeline (:=)

Introduced in PEP 572, the assignment expression (`:=`) assigns a value to a variable *inside* an expression, allowing you to test and capture a result in a single step:

```
Without Walrus Operator (Two Steps):
  match = pattern.search(text)
  if match is not None:
      handle(match)

With Walrus Operator (Single Step):
  if (match := pattern.search(text)) is not None:
      handle(match)
```

Minimal example

Save as `conditional_expressions.py`:

```python
# conditional_expressions.py
import re

def main() -> None:
    # 1. Inline Conditional Expression (Ternary)
    port_input: int | None = None
    # If port_input is None, fall back to 8080
    active_port = port_input if port_input is not None else 8080
    print(f"Active server port: {active_port}")

    # 2. Walrus Operator in Conditional Branching
    log_line = "2026-09-07 [ERROR] Database connection refused to 10.0.1.5"
    error_pattern = re.compile(r"\[ERROR\]\s+(.*)")

    # Assigns 'match' and checks truthiness simultaneously
    if (match := error_pattern.search(log_line)):
        error_msg = match.group(1)
        print(f"Captured alert via walrus operator: '{error_msg}'")
    else:
        print("No error detected in log line.")

if __name__ == "__main__":
    main()
```

Run via `uv run python conditional_expressions.py`:

```
Active server port: 8080
Captured alert via walrus operator: 'Database connection refused to 10.0.1.5'
```

Worked examples

Case 1: Streamlining Configuration Fallbacks

When computing default settings where an empty string or `None` is provided, a conditional expression keeps initialization concise and readable:

```python
# config_fallback.py
import os

def resolve_cluster_endpoint(env_override: str | None) -> str:
```

```
    # Standard ternary fallback
    endpoint = env_override.strip() if env_override and env_override.strip() else "https://d
    return endpoint

if __name__ == "__main__":
    print("Omitted endpoint :", resolve_cluster_endpoint(None))
    print("Whitespace input :", resolve_cluster_endpoint("   "))
    print("Custom endpoint  :", resolve_cluster_endpoint("https://custom.prod:9000"))
```

Run:

```
uv run python config_fallback.py
```

Output:

```
Omitted endpoint : https://default.internal:8443
Whitespace input : https://default.internal:8443
Custom endpoint  : https://custom.prod:9000
```

Case 2: Avoiding Duplicate Computations with the Walrus Operator

When a condition depends on an expensive operation (such as reading a buffer, parsing an AST, or calculating an aggregation), the walrus operator avoids calling the function twice:

```
# stream_chunk_reader.py
import io

def process_stream(data_stream: io.StringIO) -> None:
    # Read and test chunks in a single expression
    chunk_counter = 0
    while (chunk := data_stream.read(16)):
        chunk_counter += 1
        print(f"Chunk {chunk_counter}: '{chunk}'")

if __name__ == "__main__":
    payload = io.StringIO("ALPHA_STREAM_01_BETA_STREAM_02_GAMMA_STREAM_03")
    process_stream(payload)
```

Run:

```
uv run python stream_chunk_reader.py
```

Output:

```
Chunk 1: 'ALPHA_STREAM_01_'
Chunk 2: 'BETA_STREAM_02_G'
Chunk 3: 'AMMA_STREAM_03'
```

Pitfalls

Pitfall 1: The Nested Ternary Spaghetti Trap

Chaining multiple conditional expressions into a single line destroys readability:

```
# DANGEROUS / UNREADABLE:
status = "CRITICAL" if lat > 500 else "DEGRADED" if lat > 200 else "ACCEPTABLE" if lat > 50

# IDIOMATIC: Use a standard if/elif/else ladder or dictionary lookup
if lat > 500:
    status = "CRITICAL"
elif lat > 200:
    status = "DEGRADED"
elif lat > 50:
    status = "ACCEPTABLE"
else:
    status = "OPTIMAL"
```

Rule of thumb: **Never nest conditional expressions.** If a condition requires more than one `if` and one `else`, use a multi-line `if/elif/else` block.

Pitfall 2: Precedence Involving `or` and Ternaries

Conditional expressions have very low operator precedence:

```
# 'a or b if condition else c'
# Is evaluated as: '(a or b) if condition else c'
# NOT: 'a or (b if condition else c)'
# Always use parentheses around the ternary expression when mixing with boolean operators:
result = a or (b if condition else c)
```

Exercises

1. Refactor a 4-line `if/else` block assigning an access token into a single inline conditional expression.
2. Use the walrus operator `:=` inside an `if` statement to compute the length of a list and verify that it exceeds 5 elements without computing `len()` twice.
3. Write a function that accepts an integer port and uses a conditional expression to return `"SECURE"` if port is `443` or `8443`, otherwise `"STANDARD"`.

4. Demonstrate how the walrus operator simplifies reading lines from a text file until an empty string EOF is encountered.

Further reading

- PEP 308: *Conditional Expressions*.
- PEP 572: *Assignment Expressions (The Walrus Operator)*.
- Python Language Reference: *Expressions — Conditional expressions*.

The while Loop and State-Driven Execution

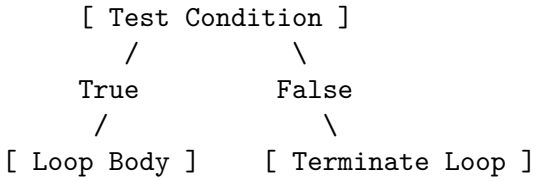
The `while` Loop and State-Driven Execution

After reading this chapter, you will master state-driven iteration using the `while` loop, configure daemon loops with `while True`, implement robust polling algorithms with exponential backoff, handle stream termination sentinels, and prevent runaway CPU-consuming infinite loops.

Mental model

A `while` loop repeatedly executes a block of code as long as a boolean test condition evaluates to `True`. The condition is checked at the **entry of every iteration**:

`The while Loop Execution Cycle:`



If the condition evaluates to `False` on the very first evaluation, the loop body **never executes**.

Minimal example

Save as `while_loop_mechanics.py`:

```
# while_loop_mechanics.py
import time

def poll_node_readiness(target_node: str, max_retries: int = 4) -> bool:
    """Poll a cluster node until it reports ready or retries are exhausted."""
    # Simulated cluster states: 0=initializing, 1=starting, 2=healthy
    mock_states = ["INITIALIZING", "STARTING", "READY"]
    attempt = 0
```

```
    print(f"Polling node '{target_node}'...")
    while attempt < max_retries:
        attempt += 1
        # Retrieve current mock state
        current_state = mock_states[attempt - 1] if attempt - 1 < len(mock_states) else "REA
        print(f"  Attempt {attempt}/{max_retries}: State is '{current_state}'")

        if current_state == "READY":
            print("  Node is ready! Exiting polling loop.")
            return True

        # Polling delay
        time.sleep(0.02)

    print("  Failed: Maximum retries exceeded.")
    return False

def main() -> None:
    is_ready = poll_node_readiness("k8s-worker-01", max_retries=4)
    print(f"Final readiness status: {is_ready}")

if __name__ == "__main__":
    main()
```

Run via `uv run python while_loop_mechanics.py`:

```
Polling node 'k8s-worker-01'...
  Attempt 1/4: State is 'INITIALIZING'
  Attempt 2/4: State is 'STARTING'
  Attempt 3/4: State is 'READY'
  Node is ready! Exiting polling loop.
Final readiness status: True
```

Worked examples

Case 1: Exponential Backoff with Jitter for Network Retries

In production distributed systems, polling an overloaded server at fixed intervals worsens congestion. An exponential backoff loop doubles the wait duration on each attempt:

```
# exponential_backoff.py
import time
```

```
def call_unstable_service(attempt: int) -> bool:
    # Simulates recovery on 4th attempt
    return attempt >= 4

def retry_with_backoff(max_attempts: int = 5, base_delay: float = 0.01) -> bool:
    attempt = 1
    delay = base_delay

    while attempt <= max_attempts:
        print(f"[Attempt {attempt}] Contacting payment gateway...")
        success = call_unstable_service(attempt)

        if success:
            print(f"[Success] Gateway response received on attempt {attempt}!")
            return True

        print(f"  Transient failure. Backing off for {delay*1000:.1f}ms...")
        time.sleep(delay)

        # Exponential backoff: double the delay duration
        delay *= 2
        attempt += 1

    return False

if __name__ == "__main__":
    retry_with_backoff(max_attempts=5, base_delay=0.01)
```

Run:

```
uv run python exponential_backoff.py
```

Output:

```
[Attempt 1] Contacting payment gateway...
  Transient failure. Backing off for 10.0ms...
[Attempt 2] Contacting payment gateway...
  Transient failure. Backing off for 20.0ms...
[Attempt 3] Contacting payment gateway...
  Transient failure. Backing off for 40.0ms...
[Attempt 4] Contacting payment gateway...
[Success] Gateway response received on attempt 4!
```

Case 2: State-Machine Dispatcher Loop

`while` loops are the foundation of state-machine processing, where the loop continues running until a terminal state is reached:

```python
# state_machine_loop.py
from enum import Enum

class State(Enum):
    PENDING = "pending"
    PROCESSING = "processing"
    VERIFYING = "verifying"
    COMPLETED = "completed"
    FAILED = "failed"

def run_job_pipeline() -> None:
    current_state = State.PENDING
    step_count = 0

    print("Initiating state machine loop:")
    while current_state not in (State.COMPLETED, State.FAILED):
        step_count += 1
        print(f"  Step {step_count}: Current state is '{current_state.value}'")

        if current_state == State.PENDING:
            current_state = State.PROCESSING
        elif current_state == State.PROCESSING:
            current_state = State.VERIFYING
        elif current_state == State.VERIFYING:
            current_state = State.COMPLETED

    print(f"Terminal state reached: '{current_state.value}' after {step_count} steps.")

if __name__ == "__main__":
    run_job_pipeline()
```

Run:

```
uv run python state_machine_loop.py
```

Output:

```
Initiating state machine loop:
  Step 1: Current state is 'pending'
  Step 2: Current state is 'processing'
  Step 3: Current state is 'verifying'
Terminal state reached: 'completed' after 3 steps.
```

Pitfalls

Pitfall 1: The Accidental Runaway Infinite Loop

If the loop variable is not updated within the loop body, the condition remains permanently `True`, consuming 100% of a CPU core:

```
# FATAL BUG:
count = 5
while count > 0:
    print(count)
    # BUG: Forgot count -= 1! Runs forever!

# FIX:
while count > 0:
    print(count)
    count -= 1
```

Pitfall 2: Busy-Waiting Without Sleeping

A `while` loop that checks a flag or condition without yielding (`time.sleep()` or `await asyncio.sleep()`) creates a **busy-wait** loop that starves other threads and overheats the CPU. Always insert a sleep or use an event primitive (`threading.Event.wait()`).

Exercises

1. Write a `while` loop that implements Euclid's algorithm to find the greatest common divisor (GCD) of two numbers A and B.
2. Implement a countdown loop that starts at 10 and decrements down to 1, printing `"BLASTOFF!"` after the loop finishes.
3. Write a sentinel-controlled `while` loop that pops elements from a queue list until an element with `status == "DONE"` is encountered.
4. Implement an exponential backoff loop that caps the maximum delay at `1.0` second.

Further reading

- Python Language Reference: *The `while` statement*.
- Python Standard Library: `time.sleep` and `threading.Event`.

The for Loop and Sequence Iteration

The `for` Loop and Sequence Iteration

After reading this chapter, you will master sequence and collection iteration using the `for` loop, understand how the Python Iterator Protocol operates under the hood, generate numerical sequences using `range()`, iterate dictionaries across keys and items, and avoid modifying collections during iteration.

Mental model

Unlike C-style `for` loops (`for (int i=0; i<n; i++)`), Python's `for` loop is an abstraction over the **Iterator Protocol**. It operates over any **iterable** (lists, tuples, dictionaries, sets, generators, strings, open file handles):

```
How CPython Executes "for item in collection:":
  1. Calls iterator = iter(collection)      Calls collection.__iter__()
  2. Enters internal loop:
       try:
           item = next(iterator)            Calls iterator.__next__()
           [ Execute Loop Body ]
       except StopIteration:
           break                            Terminates cleanly!
```

Because `range(start, stop, step)` generates numbers on demand without allocating an actual list in memory, `range(1_000_000_000)` consumes only 48 bytes of RAM!

Minimal example

Save as `for_loop_mechanics.py`:

```python
# for_loop_mechanics.py
import sys

def main() -> None:
    # 1. Iterating across a list of cluster nodes
    nodes = ["k8s-node-01", "k8s-node-02", "k8s-node-03"]
    print("--- Node Iteration ---")
```

```python
    for node in nodes:
        print(f"Active node: {node}")

    # 2. Dictionary iteration via .items()
    node_metrics = {"k8s-node-01": 12.4, "k8s-node-02": 45.1, "k8s-node-03": 88.0}
    print("\n--- Dictionary Key-Value Unpacking ---")
    for hostname, load in node_metrics.items():
        print(f"Host: {hostname:12} | CPU Load: {load:4.1f}%")

    # 3. range() step and stride
    print("\n--- Range Stride (Even ports from 8000 to 8006) ---")
    for port in range(8000, 8008, 2):
        print(f"Allocating port: {port}")

    # Inspecting range memory efficiency
    massive_range = range(1_000_000_000)
    print(f"\nMemory size of range(1,000,000,000): {sys.getsizeof(massive_range)} bytes")

if __name__ == "__main__":
    main()
```

Run via `uv run python for_loop_mechanics.py`:

```
--- Node Iteration ---
Active node: k8s-node-01
Active node: k8s-node-02
Active node: k8s-node-03

--- Dictionary Key-Value Unpacking ---
Host: k8s-node-01  | CPU Load: 12.4%
Host: k8s-node-02  | CPU Load: 45.1%
Host: k8s-node-03  | CPU Load: 88.0%

--- Range Stride (Even ports from 8000 to 8006) ---
Allocating port: 8000
Allocating port: 8002
Allocating port: 8004
Allocating port: 8006

Memory size of range(1,000,000,000): 48 bytes
```

Worked examples

Case 1: Deconstructing the Iterator Protocol Manually

To understand what Python does behind the scenes during a `for` loop, you can execute the exact same protocol manually using `iter()` and `next()`:

```
# manual_iterator_demo.py
def demonstrate_manual_iteration() -> None:
    items = ["auth-svc", "billing-svc", "gateway-svc"]

    # 1. Obtain iterator object
    iterator = iter(items)
    print("Iterator object created:", iterator)

    # 2. Emulate the for-loop using while and StopIteration
    print("\nExecuting manual loop:")
    while True:
        try:
            item = next(iterator)
            print(f"  Pulled item from iterator: {item}")
        except StopIteration:
            print("  StopIteration raised! CPython caught it and terminated loop.")
            break

if __name__ == "__main__":
    demonstrate_manual_iteration()
```

Run:

```
uv run python manual_iterator_demo.py
```

Output:

```
Iterator object created: <list_iterator object at ...>

Executing manual loop:
  Pulled item from iterator: auth-svc
  Pulled item from iterator: billing-svc
  Pulled item from iterator: gateway-svc
  StopIteration raised! CPython caught it and terminated loop.
```

Case 2: File Iteration Line by Line (Constant Memory)

An open file object is an iterable that yields one line at a time on demand. Iterating directly over the file handle reads from disk incrementally without ever loading the full file into RAM:

```python
# file_line_streamer.py
import tempfile
from pathlib import Path

def process_large_logfile(log_path: Path) -> int:
    error_count = 0
    # Opening the file returns an iterable stream
    with log_path.open("r", encoding="utf-8") as stream:
        for line in stream:
            if "[ERROR]" in line:
                error_count += 1
    return error_count

if __name__ == "__main__":
    with tempfile.TemporaryDirectory() as tmpdir:
        dummy_log = Path(tmpdir) / "cluster.log"
        dummy_log.write_text(
            "2026-09-07 [INFO] Started\n"
            "2026-09-07 [ERROR] Database timeout\n"
            "2026-09-07 [INFO] Retrying\n"
            "2026-09-07 [ERROR] Auth rejected\n"
        )

        total_errors = process_large_logfile(dummy_log)
        print(f"Total error lines identified: {total_errors}")
```

Run:

```
uv run python file_line_streamer.py
```

Output:

```
Total error lines identified: 2
```

Pitfalls

Pitfall 1: Modifying a Collection While Iterating Over It

Mutating a list while iterating over it causes internal index shifts that skip items or loop indefinitely:

```python
# THE BUG:
numbers = [1, 2, 3, 4, 5, 6]
for n in numbers:
```

```
    if n % 2 == 0:
        numbers.remove(n)  # Skips elements during iteration!
print(numbers)  # [1, 3, 5] by coincidence, but elements like 4 or 6 get skipped on other da

# THE FIX: Iterate over a shallow copy or use a list comprehension
numbers = [n for n in numbers if n % 2 != 0]
```

For dictionaries, attempting to add or remove keys during iteration immediately crashes with `RuntimeError: dictionary changed size during iteration`.

Pitfall 2: The `range(len(sequence))` Anti-Pattern

Programmers coming from C or Java frequently write `for i in range(len(items)): val = items[i]`. This is unpythonic, verbose, and error-prone. Iterate directly over `items`:

```
# ANTI-PATTERN:
for i in range(len(servers)):
    print(servers[i])

# IDIOMATIC:
for server in servers:
    print(server)
```

If you need the index, use `enumerate(servers)` (covered in chapter 6).

Exercises

1. Write a `for` loop that iterates over a dictionary and prints only the keys whose values exceed 100.
2. Generate all multiples of 5 between 50 and 100 in reverse order using `range()`.
3. Demonstrate iterating over a string character by character to count how many vowels it contains.
4. Implement a custom class with `__iter__` and `__next__` that yields squares of numbers up to a maximum limit.

Further reading

- Python Language Reference: *The `for` statement.*
- Python Data Model: *The Iterator Protocol (`__iter__`, `__next__`).*

Loop Control: break, continue, and Loop else

Loop Control: `break`, `continue`, and Loop `else`

After reading this chapter, you will master loop execution interruption using `break` and `continue`, eliminate boolean search flags using the loop `else` clause, break out of nested loop structures cleanly, and handle pipeline filtering without nested conditional blocks.

Mental model

Python provides three control keywords that alter standard loop iteration:

```
Loop Control Keywords:
  1. continue    Immediately skips the remaining statements in the current iteration;
                  jumps directly to the next iteration evaluation.
  2. break       Immediately aborts the entire loop;
                  jumps to the first statement outside the loop.
  3. else        Executes IF AND ONLY IF the loop finished all iterations
                  without hitting an explicit break statement!

The Loop 'else' Contract:
  for item in sequence:
      if item == target:
          break                (Bypasses loop 'else'!)
  else:
      # Runs ONLY if target was NOT found
      print("Item missing!")

  [ Resume Outer Execution ]
```

Minimal example

Save as `loop_control_statements.py`:

```python
# loop_control_statements.py
def inspect_packet_batch(packets: list[dict[str, str | int]]) -> None:
    print(f"Inspecting batch of {len(packets)} network packets...")
```

```
    for pkt in packets:
        # 1. 'continue' skips empty or malformed packets
        if pkt.get("size", 0) <= 0:
            print(f"  [Skip] Packet {pkt.get('id')} has 0 bytes. Continuing.")
            continue

        # 2. 'break' immediately terminates loop on critical security violation
        if pkt.get("flags") == "MALICIOUS":
            print(f"  [SECURITY ALERT] Malicious payload detected on ID {pkt['id']}! Termina
            break

        print(f"  [OK] Packet {pkt['id']} verified ({pkt['size']} bytes).")
    else:
        # 3. Loop 'else' executes ONLY if loop completed without hitting 'break'
        print("  [SUCCESS] All packets verified clean. Batch approved for forwarding.")

def main() -> None:
    # Test clean batch
    clean_batch = [
        {"id": 101, "size": 64, "flags": "SYN"},
        {"id": 102, "size": 0, "flags": "ACK"},  # Skipped via continue
        {"id": 103, "size": 128, "flags": "ACK"},
    ]
    print("--- Test 1: Clean Batch ---")
    inspect_packet_batch(clean_batch)

    # Test malicious batch
    dirty_batch = [
        {"id": 201, "size": 64, "flags": "SYN"},
        {"id": 202, "size": 512, "flags": "MALICIOUS"}, # Aborted via break
        {"id": 203, "size": 64, "flags": "ACK"},
    ]
    print("\n--- Test 2: Dirty Batch ---")
    inspect_packet_batch(dirty_batch)

if __name__ == "__main__":
    main()
```

Run via `uv run python loop_control_statements.py`:

```
--- Test 1: Clean Batch ---
Inspecting batch of 3 network packets...
  [OK] Packet 101 verified (64 bytes).
  [Skip] Packet 102 has 0 bytes. Continuing.
  [OK] Packet 103 verified (128 bytes).
  [SUCCESS] All packets verified clean. Batch approved for forwarding.
```

```
--- Test 2: Dirty Batch ---
Inspecting batch of 3 network packets...
  [OK] Packet 201 verified (64 bytes).
  [SECURITY ALERT] Malicious payload detected on ID 202! Terminating inspection immediately.
```

Worked examples

Case 1: Search and Validate Without Boolean Flags (`for ... else`)

In languages like C or Java, checking whether a list contains an item matching specific criteria requires setting a boolean flag (`found = False`). Python's loop `else` eliminates this boilerplate:

```python
# target_finder_clean.py
def find_available_worker(workers: list[dict[str, str | int]]) -> None:
    # Search for an idle worker with CPU load under 20%
    for w in workers:
        if w["status"] == "IDLE" and w["cpu_pct"] < 20:
            print(f"Dispatching task to worker: {w['name']} (CPU: {w['cpu_pct']}%)")
            break
    else:
        # Executes only if no worker matched
        print("CAPACITY ALERT: No available idle worker found across cluster!")

if __name__ == "__main__":
    cluster = [
        {"name": "worker-01", "status": "BUSY", "cpu_pct": 85},
        {"name": "worker-02", "status": "IDLE", "cpu_pct": 95},
        {"name": "worker-03", "status": "BUSY", "cpu_pct": 40},
    ]
    find_available_worker(cluster)
```

Run:

```
uv run python target_finder_clean.py
```

Output:

```
CAPACITY ALERT: No available idle worker found across cluster!
```

Case 2: Breaking Out of Nested Loops

A `break` statement only terminates the **innermost** loop. To exit multiple nested loops simultaneously without clumsy flag variables, encapsulate the nested loops in a function and use `return`:

```python
# nested_loop_exit.py
def locate_corrupted_block(matrix: list[list[int]]) -> tuple[int, int] | None:
    """Scan a 2D matrix and return (row, col) coordinates of first negative value."""
    for row_idx, row in enumerate(matrix):
        for col_idx, value in enumerate(row):
            if value < 0:
                # 'return' instantly exits BOTH loops cleanly!
                return (row_idx, col_idx)
    return None

if __name__ == "__main__":
    storage_grid = [
        [100, 200, 300],
        [400, -999, 600],  # Corrupted block at row 1, col 1
        [700, 800, 900],
    ]
    coord = locate_corrupted_block(storage_grid)
    print(f"Corrupted block coordinates: {coord}")
```

Run:

```
uv run python nested_loop_exit.py
```

Output:

```
Corrupted block coordinates: (1, 1)
```

Pitfalls

Pitfall 1: Assuming `break` Exits All Nested Loops

Calling `break` inside an inner loop returns control to the *outer* loop, not the code outside the outer loop:

```python
for outer in range(3):
    for inner in range(3):
        if inner == 1:
            break  # Exits 'inner' loop only! 'outer' continues iterating!
```

To exit both, encapsulate the nested loops inside a function and use `return`, or use a custom exception.

Pitfall 2: Confusing Loop `else` with `if else`

A loop `else` does **not** mean "run if the loop didn't execute". If a loop iterates over an empty list (`for x in []:`), it completes 0 iterations without hitting `break`, so the `else` block **does execute**! Think of loop `else` as "no-break".

Exercises

1. Write a prime number checker using `for ... in range(2, n): ... break else:` to determine whether n is prime without boolean flags.
2. Given a list of log entries, use `continue` to filter out all entries that do not contain `"ALERT"` or `"CRITICAL"`.
3. Demonstrate that iterating over an empty list `for item in []: break else: print("RUN")` executes the `else` block.
4. Implement a function that scans a 3D tensor and uses `return` to break out of all three nested loops when a NaN value is encountered.

Further reading

- Python Language Reference: *The `break` and `continue` statements.*
- Raymond Hettinger: *Transforming Code into Beautiful, Idiomatic Python (The for-else idiom).*

Iteration Utilities: enumerate and zip

Iteration Utilities: `enumerate` and `zip`

After reading this chapter, you will master Python's built-in iteration helpers: track loop indices cleanly without manual counter variables using `enumerate()`, iterate multiple parallel streams simultaneously using `zip()`, enforce length symmetry using `strict=True`, and handle unequal lengths with `itertools.zip_longest()`.

Mental model

In low-level languages, iterating across parallel arrays or tracking line numbers requires manual integer incrementation (`i++`). Python replaces manual counters with lazy iterator wrappers:

```
Manual Index Management (Error-Prone):
  i = 0
  for item in items:
      print(i, item)
      i += 1                Easy to forget, off-by-one errors

enumerate(items, start=1) Pipeline:
  items: [ "web", "api", "db" ]


  Yields: (1, "web")    (2, "api")    (3, "db") (Zero overhead)

Multi-Stream Parallel Iteration (zip):
  Stream 1: [ "node-1",   "node-2",   "node-3"   ]
  Stream 2: [ "10.0.1.1", "10.0.1.2", "10.0.1.3" ]


  zip:      ("node-1",  ("node-2",  ("node-3",
             "10.0.1.1") "10.0.1.2") "10.0.1.3")
```

Minimal example

Save as `iteration_utilities.py`:

```python
# iteration_utilities.py
def main() -> None:
    # 1. Indexed iteration with enumerate(start=1)
    services = ["auth-gateway", "billing-worker", "orders-api"]
    print("--- Numbered Service Manifest ---")
    for index, service in enumerate(services, start=1):
        print(f"Service #{index:02d}: {service}")

    # 2. Parallel lock-step iteration with zip(..., strict=True)
    hosts = ["node-01", "node-02", "node-03"]
    ips   = ["10.0.1.10", "10.0.1.11", "10.0.1.12"]
    roles = ["primary", "replica", "replica"]

    print("\n--- Synchronized Cluster Inventory ---")
    # strict=True guarantees ValueError if any list length differs
    for host, ip, role in zip(hosts, ips, roles, strict=True):
        print(f"Host: {host:8} | IP: {ip:10} | Role: {role}")

if __name__ == "__main__":
    main()
```

Run via `uv run python iteration_utilities.py`:

```
--- Numbered Service Manifest ---
Service #01: auth-gateway
Service #02: billing-worker
Service #03: orders-api

--- Synchronized Cluster Inventory ---
Host: node-01  | IP: 10.0.1.10 | Role: primary
Host: node-02  | IP: 10.0.1.11 | Role: replica
Host: node-03  | IP: 10.0.1.12 | Role: replica
```

Worked examples

Case 1: Catching Silent Data Truncation with `strict=True`

In Python 3.9 and earlier, `zip()` silently stopped iterating as soon as the *shortest* sequence was exhausted, dropping trailing elements without any error. PEP 618 introduced `strict=True`:

```python
# strict_zip_verification.py
def verify_cluster_mapping(hostnames: list[str], ip_addresses: list[str]) -> None:
    try:
        # If one list has 3 items and the other has 2, strict=True raises ValueError
```

```
        for host, ip in zip(hostnames, ip_addresses, strict=True):
            print(f"Mapping: {host} -> {ip}")
    except ValueError as exc:
        print(f"CRITICAL DRIFT: Configuration mismatch detected: {exc}")

if __name__ == "__main__":
    configured_hosts = ["web-01", "web-02", "web-03"]
    assigned_ips     = ["192.168.1.10", "192.168.1.11"]  # Missing 3rd IP!

    print("Attempting to pair hosts and IPs with strict=True:")
    verify_cluster_mapping(configured_hosts, assigned_ips)
```

Run:

```
uv run python strict_zip_verification.py
```

Output:

```
Attempting to pair hosts and IPs with strict=True:
Mapping: web-01 -> 192.168.1.10
Mapping: web-02 -> 192.168.1.11
CRITICAL DRIFT: Configuration mismatch detected: zip() argument 2 is shorter than argument 1
```

Case 2: Unequal Streams with Padding Using `itertools.zip_longest`

When sequence lengths differ intentionally and you wish to fill missing values with a placeholder, use `itertools.zip_longest()`:

```
# zip_longest_demo.py
import itertools

def display_table_with_padding() -> None:
    headers = ["Service", "Primary Host", "Backup Host"]
    col1 = ["auth", "billing"]
    col2 = ["auth-01", "bill-01", "order-01"]
    col3 = ["auth-bak"]

    print("Padded multi-column output:")
    for svc, primary, backup in itertools.zip_longest(col1, col2, col3, fillvalue="[NONE]"):
        print(f"  {svc:10} | Primary: {primary:10} | Backup: {backup:10}")

if __name__ == "__main__":
    display_table_with_padding()
```

Run:

```
uv run python zip_longest_demo.py
```

Output:

```
Padded multi-column output:
  auth       | Primary: auth-01    | Backup: auth-bak
  billing    | Primary: bill-01    | Backup: [NONE]
  [NONE]     | Primary: order-01   | Backup: [NONE]
```

Pitfalls

Pitfall 1: Relying on Default `zip()` Without `strict=True`

Omitting `strict=True` in production code causes subtle bugs where data streams (like headers vs rows in CSVs, or keys vs values) fail to match, silently corrupting or discarding records. Always write `zip(..., strict=True)`.

Pitfall 2: Re-Iterating a `zip` or `enumerate` Object

Both `zip()` and `enumerate()` return single-pass **iterators**. Once consumed, iterating over them a second time yields 0 items:

```
pairs = zip([1, 2], ["a", "b"])
print(list(pairs))  # [(1, 'a'), (2, 'b')]
print(list(pairs))  # [] (Exhausted!)

# If you need to re-use the pairs, convert to a list:
pairs_list = list(zip([1, 2], ["a", "b"]))
```

Exercises

1. Use `enumerate(lines, start=1)` to format an error report that prints the line number and content for lines containing `"FATAL"`.
2. Given a list of keys and a list of values, use `zip(keys, values, strict=True)` inside a dictionary comprehension to build a map.
3. Demonstrate why `zip()` stops early when one argument is an infinite generator (like `itertools.count()`) and the other is a finite list.
4. Implement a matrix transpose operation using `zip(*matrix)`.

Further reading

- PEP 618: *Add Optional Length-Checking To zip*.
- Python Standard Library: `builtins.enumerate`, `builtins.zip`.
- Python Standard Library: `itertools.zip_longest`.

Structural Pattern Matching with match and case

Structural Pattern Matching with `match` and `case`

After reading this chapter, you will master Python's Structural Pattern Matching syntax (`match` and `case` introduced in PEP 634), destructure sequence, mapping, and class patterns, enforce conditional guards with `if`, and prevent common variable-capture traps.

Mental model

Traditional `if/elif` statements evaluate boolean expressions one by one. **Structural Pattern Matching** evaluates data against a structural shape specification: testing **type**, **structure**, and **content** simultaneously, while extracting and binding variables on success:

```
Incoming Data: {"action": "scale", "service": "auth", "replicas": 5}


          [ match expression ]

                      case {"action": "restart", "service": str(svc)}    No match (action != re

                      case {"action": "scale", "service": str(svc), "replicas": int(n)} if n >

                             Matches keys: "action", "service", "replicas"
                             Type matches: str and int
                             Guard condition passes: n (5) > 0
                             Binds: svc = "auth", n = 5
                                                    Execute Branch!

                      case _    Wildcard Fallback
```

Minimal example

Save as `pattern_matching_dispatch.py`:

```python
# pattern_matching_dispatch.py
def process_command(cmd: list[str]) -> str:
    """Parse a CLI token list into an actionable response using sequence patterns."""
    match cmd:
```

```
        case ["quit" | "exit"]:
            return "Terminating process."
        case ["get", str(key)]:
            return f"Retrieving key: {key}"
        case ["set", str(key), value]:
            return f"Setting key: {key} = {value}"
        case ["batch", *items] if len(items) > 0:
            return f"Processing batch of {len(items)} items: {', '.join(items)}"
        case _:
            return f"Invalid or malformed command: {cmd}"

def main() -> None:
    test_commands = [
        ["quit"],
        ["get", "redis.host"],
        ["set", "timeout", "30s"],
        ["batch", "item1", "item2", "item3"],
        ["unknown", "foo", "bar"],
    ]
    for command in test_commands:
        result = process_command(command)
        print(f"Command: {command} -> Result: {result}")

if __name__ == "__main__":
    main()
```

Run via `uv run python pattern_matching_dispatch.py`:

```
Command: ['quit'] -> Result: Terminating process.
Command: ['get', 'redis.host'] -> Result: Retrieving key: redis.host
Command: ['set', 'timeout', '30s'] -> Result: Setting key: timeout = 30s
Command: ['batch', 'item1', 'item2', 'item3'] -> Result: Processing batch of 3 items: item1, it
Command: ['unknown', 'foo', 'bar'] -> Result: Invalid or malformed command: ['unknown', 'foo',
```

Worked examples

Case 1: Mapping Patterns for Telemetry Event Routing

Mapping patterns allow inspecting structured dictionaries (such as JSON payloads from message queues) and extracting specific keys:

```
# telemetry_router.py
from typing import Any
```

```
def handle_telemetry_event(event: dict[str, Any]) -> None:
    match event:
        case {"type": "metric", "name": str(name), "value": float(val)} if val >= 90.0:
            print(f"[HIGH WATERMARK] Metric {name} breached alert threshold: {val}%")

        case {"type": "metric", "name": str(name), "value": float(val)}:
            print(f"[NOMINAL] Metric {name} at {val}%")

        case {"type": "heartbeat", "node_id": str(node), "status": "alive"}:
            print(f"[HEARTBEAT] Node {node} reported healthy.")

        case {"type": "alert", "severity": "CRITICAL", "message": str(msg)}:
            print(f"[PAGERDUTY ALERT] Critical incident: {msg}")

        case _:
            print(f"[UNHANDLED] Discarding unrecognized event structure: {event}")

if __name__ == "__main__":
    events = [
        {"type": "metric", "name": "cpu_utilization", "value": 94.2},
        {"type": "heartbeat", "node_id": "k8s-worker-04", "status": "alive", "uptime": 86400
        {"type": "alert", "severity": "CRITICAL", "message": "Power rail offline"},
        {"source": "unknown", "data": None},
    ]

    for ev in events:
        handle_telemetry_event(ev)
```

Run:

```
uv run python telemetry_router.py
```

Output:

```
[HIGH WATERMARK] Metric cpu_utilization breached alert threshold: 94.2%
[HEARTBEAT] Node k8s-worker-04 reported healthy.
[PAGERDUTY ALERT] Critical incident: Power rail offline
[UNHANDLED] Discarding unrecognized event structure: {'source': 'unknown', 'data': None}
```

Notice that the heartbeat event matched despite containing an extra key (`"uptime": 86400`). In Python mapping patterns, extra keys are ignored unless explicitly restricted with `**_` or full length checks.

Case 2: Class Patterns with Dataclasses

Pattern matching works natively with object-oriented classes and dataclasses:

```
# packet_matcher.py
from dataclasses import dataclass

@dataclass
class Packet:
    source_ip: str
    dest_port: int
    payload_len: int
    protocol: str = "TCP"

def inspect_traffic(packet: Packet) -> str:
    match packet:
        # Match port 22 or 3389 with payload
        case Packet(dest_port=22 | 3389, payload_len=length) if length > 0:
            return f"Remote administration session to port {packet.dest_port} ({length} byte

        # Match web traffic
        case Packet(dest_port=80 | 443):
            return f"Standard Web Traffic on port {packet.dest_port}"

        # Match UDP DNS traffic
        case Packet(protocol="UDP", dest_port=53):
            return "DNS Query Traffic"

        case Packet(dest_port=port):
            return f"General traffic on port {port}"

if __name__ == "__main__":
    packets = [
        Packet("192.168.1.100", 22, 512),
        Packet("10.0.0.5", 443, 1024),
        Packet("172.16.0.2", 53, 64, protocol="UDP"),
    ]

    for p in packets:
        print(f"{p.source_ip} -> {inspect_traffic(p)}")
```

Run:

```
uv run python packet_matcher.py
```

Output:

```
192.168.1.100 -> Remote administration session to port 22 (512 bytes)
10.0.0.5 -> Standard Web Traffic on port 443
172.16.0.2 -> DNS Query Traffic
```

Pitfalls

Pitfall 1: The Variable Capture Trap (Constants vs Variables)

In a `case` clause, an unqualified name (like `status`) is treated as a **capture variable**, NOT a value equality check!

```
# THE TRAP:
OK = 200
response_code = 404

match response_code:
    case OK:  # BUG: This does NOT compare to OK! It assigns response_code to a new variable
        print("This ALWAYS prints because 'OK' matches anything and gets bound!")

# THE FIX: Use an Enum or dotted attribute
class Status:
    OK = 200

match response_code:
    case Status.OK:  # Safe: Dotted names are looked up by value, not treated as capture var
        print("HTTP 200 OK")
    case _:
        print("Other status")
```

Exercises

1. Create a `match ... case` statement that parses a network URI string decomposed into `[scheme, host, port]`, handling default ports (`80` for http, `443` for https) when port is omitted.
2. Define a dataclass `DeploymentPlan(env: str, replicas: int, dry_run: bool)`. Write a pattern matching function that forbids any deployment to `env="production"` if `replicas > 10` and `dry_run` is `False`.
3. Demonstrate the difference between `case [head, *tail]:` and `case [head, tail]:` when passed a list with four elements.
4. Write a pattern matching function that accepts an arbitrary JSON-like value and returns whether it is a scalar, a list of strings, or a dictionary mapping strings to integers.

Further reading

- PEP 634: *Structural Pattern Matching: Specification*.
- PEP 635: *Structural Pattern Matching: Motivation and Rationale*.
- PEP 636: *Structural Pattern Matching: Tutorial*.

List, Dict, and Set Comprehensions

List, Dict, and Set Comprehensions

After reading this chapter, you will construct concise, declarative data transformations using list, dictionary, and set comprehensions, apply conditional filters, flatten multi-dimensional structures with nested loops, leverage assignment expressions (`:=`) to eliminate duplicate calculations, and identify when to refactor overly complex comprehensions into explicit loops.

Mental model

In imperative programming, transforming a collection requires creating an empty accumulator, iterating with a `for` loop, applying `if` guards, and appending items one by one. **Comprehensions** compress this workflow into a single declarative expression that is faster, less error-prone, and visually matches set-builder notation from mathematics:

```
Imperative Accumulation (4 statements):
  result = []
  for item in source:
      if passes_filter(item):
          result.append(transform(item))

Declarative Comprehension (1 expression):
  result = [ transform(item)  for item in source  if passes_filter(item) ]

                 Expression         Iteration             Condition
```

Python provides three comprehension syntaxes based on surrounding delimiters:

```
 Container Type      Syntax                          Result Type

 List Comprehension  [expr for item in iterable]     list (ordered, mutable)
 Set Comprehension   {expr for item in iterable}     set (unique, unordered)
 Dict Comprehension  {k: v for item in iterable}     dict (key-value mapping)
```

For nested loops, the comprehension syntax preserves standard left-to-right nesting order:

```
[ x for row in matrix for x in row ]

       Outer Loop      Inner Loop

Equivalent to:
  for row in matrix:
      for x in row:
          yield x
```

Minimal example

Save as `comprehensions_overview.py`:

```python
# comprehensions_overview.py
def main() -> None:
    raw_nodes = [" worker-01 ", "Worker-02", "  api-01", "WORKER-01", "db-01 "]

    # 1. List Comprehension: Normalize names and filter worker nodes
    worker_nodes = [
        node.strip().lower()
        for node in raw_nodes
        if "worker" in node.lower()
    ]
    print(f"Worker List: {worker_nodes}")

    # 2. Set Comprehension: Deduplicate normalized names
    unique_workers = {
        node.strip().lower()
        for node in raw_nodes
        if "worker" in node.lower()
    }
    print(f"Unique Worker Set: {sorted(unique_workers)}")

    # 3. Dict Comprehension: Build node-to-role lookup map
    node_roles = {
        node.strip().lower(): ("database" if "db" in node else "compute")
        for node in raw_nodes
    }
    print("Node Roles:")
    for name, role in sorted(node_roles.items()):
        print(f"  {name:10} -> {role}")

if __name__ == "__main__":
    main()
```

Run via `uv run python comprehensions_overview.py`:

```
Worker List: ['worker-01', 'worker-02', 'worker-01']
Unique Worker Set: ['worker-01', 'worker-02']
Node Roles:
  api-01     -> compute
  db-01      -> database
  worker-01  -> compute
  worker-02  -> compute
```

Worked examples

Case 1: Ingesting and Sanitizing Raw Metric Records (List Comprehensions)

When ingesting metrics from HTTP log lines or sensors, raw records frequently contain malformed entries or values outside acceptable thresholds. A list comprehension combines parsing and filtering cleanly:

```python
# metric_sanitizer.py
def sanitize_latencies(raw_samples: list[str]) -> list[float]:
    """Parse comma-separated millisecond strings, filter errors (-1.0), and convert to secon
    return [
        float(val.strip()) / 1000.0
        for val in raw_samples
        if val.strip() != "" and float(val.strip()) >= 0.0
    ]

def main() -> None:
    samples = ["120.5", " 45.2 ", "-1.0", "", "850.0", "312.4", "-99.0", "12.0"]
    cleaned_seconds = sanitize_latencies(samples)

    print(f"Raw sample count:     {len(samples)}")
    print(f"Valid sample count:   {len(cleaned_seconds)}")
    print("Cleaned values (seconds):")
    for sec in cleaned_seconds:
        print(f"  {sec:.4f}s")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python metric_sanitizer.py
```

Output:

```
Raw sample count:     8
Valid sample count:   5
Cleaned values (seconds):
  0.1205s
  0.0452s
  0.8500s
  0.3124s
  0.0120s
```

Why this matters: Notice how `[f(x) for x in xs if cond(x)]` replaces chained `map()` and `filter()` calls. In Python, comprehensions are more readable than `list(map(..., filter(...)))` and avoid the overhead of lambda function call frames.

Case 2: Inverting and Indexing Service Registries (Dict Comprehensions)

In infrastructure management, service registries often map hostnames to IP addresses. SRE tooling frequently requires reverse lookups (IP-to-hostname) or normalized sub-mappings:

```python
# registry_indexing.py
def main() -> None:
    # Forward map: Hostname -> IPv4
    dns_records = {
        "gateway.internal": "10.0.1.1",
        "auth.internal":    "10.0.1.15",
        "db-pri.internal":  "10.0.2.100",
        "db-sec.internal":  "10.0.2.101",
        "cache.internal":   "10.0.3.50",
    }

    # 1. Reverse Map: IP -> Hostname
    reverse_dns = {ip: host for host, ip in dns_records.items()}

    # 2. Filtered Subnet Map: Database subnet (10.0.2.x) only
    db_subnet = {
        host: ip
        for host, ip in dns_records.items()
        if ip.startswith("10.0.2.")
    }

    print("--- Reverse DNS Lookup (IP -> Host) ---")
    for ip, host in sorted(reverse_dns.items()):
        print(f"  {ip:12} -> {host}")

    print("\n--- DB Subnet Only ---")
```

```
    for host, ip in sorted(db_subnet.items()):
        print(f"  {host:16} -> {ip}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python registry_indexing.py
```

Output:

```
--- Reverse DNS Lookup (IP -> Host) ---
  10.0.1.1      -> gateway.internal
  10.0.1.15     -> auth.internal
  10.0.2.100    -> db-pri.internal
  10.0.2.101    -> db-sec.internal
  10.0.3.50     -> cache.internal

--- DB Subnet Only ---
  db-pri.internal  -> 10.0.2.100
  db-sec.internal  -> 10.0.2.101
```

Case 3: Matrix Flattening and Cartesian Grids (Nested Loops)

When managing multi-zone cloud deployments or flattening nested tabular datasets, nested comprehensions process multi-dimensional data without requiring manual stack allocations:

```
# cluster_topology.py
def main() -> None:
    regions = ["us-east", "eu-central"]
    zones = ["a", "b"]
    tiers = ["web", "api"]

    # Cartesian Product: Region x Zone x Tier
    deploy_targets = [
        f"{region}-{zone}-{tier}"
        for region in regions
        for zone in zones
        for tier in tiers
    ]

    print(f"Total Deployment Targets: {len(deploy_targets)}")
    for target in deploy_targets:
```

```
            print(f"  Target: {target}")

    # 2D Grid Flattening
    matrix = [
        [10, 20, 30],
        [40, 50, 60],
        [70, 80, 90],
    ]
    flattened = [val for row in matrix for val in row if val > 25]
    print(f"\nFiltered Flattened Matrix (values > 25): {flattened}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python cluster_topology.py
```

Output:

```
Total Deployment Targets: 8
  Target: us-east-a-web
  Target: us-east-a-api
  Target: us-east-b-web
  Target: us-east-b-api
  Target: eu-central-a-web
  Target: eu-central-a-api
  Target: eu-central-b-web
  Target: eu-central-b-api

Filtered Flattened Matrix (values > 25): [30, 40, 50, 60, 70, 80, 90]
```

Rule for Nested Comprehensions: Read the clauses from left to right. The first `for` clause is the outermost loop, and subsequent `for` clauses are nested inside it.

Case 4: Avoiding Duplicate Computation with the Walrus Operator (:=)

A common anti-pattern in comprehensions is calling an expensive function or parsing routine twice: once in the `if` clause to filter, and again in the expression clause to transform:

```
# walrus_comprehension.py
import json

def parse_payload(raw_text: str) -> dict[str, str] | None:
```

```
    """Simulate parsing an incoming JSON string payload."""
    try:
        data = json.loads(raw_text)
        return data if isinstance(data, dict) else None
    except json.JSONDecodeError:
        return None

def main() -> None:
    incoming_logs = [
        '{"service": "auth", "status": "200"}',
        'MALFORMED_HEADER',
        '{"service": "payment", "status": "503"}',
        '{}',
        '{"service": "cart", "status": "404"}',
    ]

    # Use the walrus operator (:=) in the condition to bind parsed_obj
    # and reuse it directly in the output expression without re-parsing!
    valid_services = [
        parsed["service"]
        for log in incoming_logs
        if (parsed := parse_payload(log)) is not None and "service" in parsed
    ]

    print("Parsed Active Services:")
    for svc in valid_services:
        print(f"  Service: {svc}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python walrus_comprehension.py
```

Output:

```
Parsed Active Services:
  Service: auth
  Service: payment
  Service: cart
```

Pitfalls

Pitfall 1: Misusing Comprehensions for Side Effects

Comprehensions are expressions designed to **produce collections**. Using them purely for their side effects (such as printing or writing to files) needlessly allocates in-memory lists that are immediately discarded:

```
# THE TRAP: Wasted memory and obscure intent
tasks = ["build", "test", "package", "deploy"]
[print(f"Executing: {task}") for task in tasks]  # Allocates [None, None, None, None]!

# THE FIX: Use a clean, standard for loop
for task in tasks:
    print(f"Executing: {task}")
```

Pitfall 2: Over-Nesting (Write-Only Code)

When a comprehension contains more than two loops or multiple complex conditionals, readability collapses:

```
# THE TRAP: Write-only comprehension
bad = [
    f(x, y, z)
    for x in dataset
    if x.is_valid()
    for y in x.sub_items
    if y.is_ready()
    for z in y.tokens
    if z.is_active()
]

# THE FIX: Refactor into an explicit generator or loop with clear variable names
def extract_active_tokens(dataset):
    for x in dataset:
        if not x.is_valid():
            continue
        for y in x.sub_items:
            if not y.is_ready():
                continue
            for z in y.tokens:
                if z.is_active():
                    yield f(x, y, z)
```

```
good = list(extract_active_tokens(dataset))
```

Pitfall 3: Scope Isolation (Python 3 Comprehension Scope)

In Python 2, the loop variable in a list comprehension leaked into the enclosing scope and overwrote existing variables. In modern Python (3.x and 3.14), comprehensions execute in their own implicit function scope:

```
# comprehension_scope.py
def main() -> None:
    x = "original_outer_value"
    squares = [x * x for x in range(5)]

    print(f"Squares: {squares}")
    # In Python 3+, x remains unmodified!
    print(f"Outer x is preserved: {x}")

if __name__ == "__main__":
    main()
```

Output:

```
Squares: [0, 1, 4, 9, 16]
Outer x is preserved: original_outer_value
```

Exercises

1. Given a list of filenames `["app.py", "README.md", "server.py", "config.json", "utils.py"]`, write a list comprehension that extracts only the stems of Python files (`.py`), producing `["app", "server", "utils"]`.
2. Given a dictionary mapping usernames to email addresses, write a dict comprehension that extracts only accounts belonging to the `@company.com` domain, lowercasing both keys and values.
3. Given a list of sentences, write a set comprehension that extracts all unique words that are 5 letters or longer, stripping punctuation and whitespace.
4. Using nested comprehensions, generate a 5x5 identity matrix (a list of 5 lists, each containing 5 numbers, where element `(i, j)` is `1` if `i == j` and `0` otherwise).
5. Given a list of HTTP status code strings `["200", "invalid", "404", "500", "abc", "301"]`, use an assignment expression (`:=`) inside a comprehension to filter and convert valid integer status codes that are >= 400.

Further reading

- PEP 202: *List Comprehensions*.
- PEP 274: *Dict Comprehensions*.
- PEP 572: *Syntax for Assignment Expressions* (the `:=` walrus operator).
- Python Documentation: *Data Structures – List Comprehensions*.

Generator Expressions

Generator Expressions

After reading this chapter, you will construct lazy, memory-efficient data pipelines using generator expressions, measure and contrast their heap allocation footprint against eager collections, stream data directly into built-in aggregators like `sum()`, `max()`, and `any()`, and avoid generator exhaustion and late-binding pitfalls.

Mental model

A list comprehension is **eager**: it computes every element immediately and allocates memory for the entire collection on the heap. A **generator expression** is **lazy**: it produces values one at a time, on demand, via Python's iterator protocol.

```
Eager Evaluation (List Comprehension: [...]):
  [x * 2 for x in range(10_000_000)]


  [ 0, 2, 4, 6, 8, ... ]     Allocates ~80 Megabytes of RAM immediately!
                              High allocation latency; blocks until full.

Lazy Evaluation (Generator Expression: (...)):
  (x * 2 for x in range(10_000_000))



   Generator State Machine (~200B)
   Current: x=0, next: x=1

                  next()

             Yields: 0    Yields: 2    Yields: 4 (O(1) memory footprint!)
```

Generator expressions also chain together into pipelines where values are pulled through on demand like items on an assembly line:

```
Source Data    ( parse )    ( filter )    ( transform )    Consumer (sum/any/for)

                                      Only one item processed at a time!
```

Minimal example

Save as `gen_expr_overview.py`:

```python
# gen_expr_overview.py
import sys

def main() -> None:
    count = 1_000_000

    # 1. Eager List Comprehension: Allocates full list in memory
    eager_list = [n * 2 for n in range(count)]
    list_mem_bytes = sys.getsizeof(eager_list)

    # 2. Lazy Generator Expression: Allocates lightweight iterator object
    lazy_gen = (n * 2 for n in range(count))
    gen_mem_bytes = sys.getsizeof(lazy_gen)

    print(f"Items processed:         {count:,}")
    print(f"Eager list memory:       {list_mem_bytes:,} bytes (~{list_mem_bytes / (1024 * 10
    print(f"Lazy generator memory:   {gen_mem_bytes:,} bytes")
    print(f"Memory ratio:            {list_mem_bytes // gen_mem_bytes:,}x less memory!")

    # 3. Consuming on demand with next()
    print(f"\nFirst three values pulled on demand:")
    print(f"  Item 1: {next(lazy_gen)}")
    print(f"  Item 2: {next(lazy_gen)}")
    print(f"  Item 3: {next(lazy_gen)}")

if __name__ == "__main__":
    main()
```

Run via `uv run python gen_expr_overview.py`:

```
Items processed:         1,000,000
Eager list memory:       8,448,728 bytes (~8.06 MB)
Lazy generator memory:   208 bytes
Memory ratio:            40,618x less memory!

First three values pulled on demand:
  Item 1: 0
  Item 2: 2
  Item 3: 4
```

Worked examples

Case 1: Streaming Gigabyte Log Parsing with Constant O(1) Memory

In production environments, log files can span tens of gigabytes. Reading an entire log file into a list causes `MemoryError` and triggers Linux Out-Of-Memory (OOM) kills. Chaining generator expressions processes files of arbitrary size with constant memory:

```
# streaming_log_parser.py
from collections.abc import Iterator

def generate_mock_log_stream() -> Iterator[str]:
    """Simulate reading lines from a high-throughput server log."""
    logs = [
        '192.168.1.10 - - [2026-09-07] "GET /api/v1/health" 200 45',
        '192.168.1.11 - - [2026-09-07] "POST /api/v1/login" 401 120',
        '# SRE maintenance comment - ignore',
        '10.0.0.50 - - [2026-09-07] "GET /api/v1/orders" 200 890',
        '',
        '172.16.0.4 - - [2026-09-07] "GET /api/v1/checkout" 500 240',
        '192.168.1.12 - - [2026-09-07] "GET /api/v1/status" 200 62',
    ]
    yield from logs

def main() -> None:
    raw_lines = generate_mock_log_stream()

    # Stage 1: Strip whitespace and drop comments/blank lines
    clean_lines = (line.strip() for line in raw_lines if line.strip() and not line.startswit

    # Stage 2: Tokenize and extract (status_code, response_bytes)
    def parse_entry(line: str) -> tuple[int, int]:
        parts = line.split()
        return int(parts[-2]), int(parts[-1])

    records = (parse_entry(line) for line in clean_lines)

    # Stage 3: Filter for successful requests (HTTP 200)
    success_bytes = (resp_bytes for status, resp_bytes in records if status == 200)

    # Stage 4: Consume directly into built-in aggregator sum()
    total_payload_bytes = sum(success_bytes)
    print(f"Total payload transferred for HTTP 200 responses: {total_payload_bytes} bytes")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python streaming_log_parser.py
```

Output:

```
Total payload transferred for HTTP 200 responses: 997 bytes
```

Why this matters: Notice that `sum(resp_bytes for status, resp_bytes in ...)` does not require double parentheses `((...))`. When a generator expression is the sole argument to a function call, outer parentheses are optional.

Case 2: Short-Circuiting Aggregations with `any()` and `all()`

Built-in boolean predicates `any()` and `all()` short-circuit: they terminate evaluation immediately upon encountering the first definitive boolean value (`True` for `any`, `False` for `all`). Using a generator expression prevents running expensive operations on remaining elements:

```
# short_circuit_eval.py
def check_node_health(node_id: str) -> bool:
    """Simulate an expensive remote node probe."""
    print(f"  [Probe] Checking node: {node_id}...")
    # Simulate node-03 being unhealthy
    return node_id != "node-03"

def main() -> None:
    cluster_nodes = [f"node-{i:02d}" for i in range(1, 10)]

    print("--- Testing with Generator Expression (Lazy, Short-Circuits) ---")
    # Generator expression: evaluation stops immediately at node-03!
    all_healthy_gen = all(check_node_health(node) for node in cluster_nodes)
    print(f"Cluster all healthy result: {all_healthy_gen}")

    print("\n--- Contrast: List Comprehension (Eager, Evaluates All 9 Nodes!) ---")
    # List comprehension: MUST evaluate all 9 nodes before calling all()
    probes_run = 0
    def logged_probe(node: str) -> bool:
        nonlocal probes_run
        probes_run += 1
        return node != "node-03"

    all_healthy_list = all([logged_probe(node) for node in cluster_nodes])
    print(f"List comprehension ran {probes_run} probes even though node-03 failed!")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python short_circuit_eval.py
```

Output:

```
--- Testing with Generator Expression (Lazy, Short-Circuits) ---
  [Probe] Checking node: node-01...
  [Probe] Checking node: node-02...
  [Probe] Checking node: node-03...
Cluster all healthy result: False

--- Contrast: List Comprehension (Eager, Evaluates All 9 Nodes!) ---
List comprehension ran 9 probes even though node-03 failed!
```

Case 3: Composing Multi-Stage Data Pipelines

Generator expressions can be chained linearly to construct readable Extract-Transform-Load (ETL) pipelines without third-party frameworks:

```python
# pipeline_composition.py
def main() -> None:
    raw_telemetry = [
        {"host": "srv-1", "cpu": 45.2, "mem_mb": 4096},
        {"host": "srv-2", "cpu": 92.1, "mem_mb": 16384},
        {"host": "srv-3", "cpu": 78.4, "mem_mb": 8192},
        {"host": "srv-4", "cpu": 95.8, "mem_mb": 32768},
        {"host": "srv-5", "cpu": 12.0, "mem_mb": 2048},
    ]

    # Stage 1: Filter hosts breaching CPU threshold (> 75%)
    overloaded = (rec for rec in raw_telemetry if rec["cpu"] > 75.0)

    # Stage 2: Convert memory to gigabytes
    with_gb = (
        {"host": rec["host"], "cpu": rec["cpu"], "mem_gb": rec["mem_mb"] / 1024}
        for rec in overloaded
    )

    # Stage 3: Format alert payload strings
    alerts = (
        f"[ALERT] {item['host']}: CPU={item['cpu']:.1f}% Mem={item['mem_gb']:.1f}GB"
        for item in with_gb
    )
```

```
    # Terminal Consumer: Iterate and dispatch
    print("Dispatched Infrastructure Alerts:")
    for alert in alerts:
        print(f"  {alert}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python pipeline_composition.py
```

Output:

```
Dispatched Infrastructure Alerts:
  [ALERT] srv-2: CPU=92.1% Mem=16.0GB
  [ALERT] srv-3: CPU=78.4% Mem=8.0GB
  [ALERT] srv-4: CPU=95.8% Mem=32.0GB
```

Pitfalls

Pitfall 1: Generator Exhaustion (The "Read-Once" Trap)

Unlike lists or tuples, generators are stateful, single-pass iterators. Once consumed to completion, they are **exhausted** and produce no further items:

```
# THE TRAP: Re-iterating an exhausted generator
data = (x * 10 for x in range(1, 5))

total = sum(data)      # Consumes all items: 10 + 20 + 30 + 40 = 100
maximum = max(data)    # BUG: ValueError: max() arg is an empty sequence!
```

```
# THE FIX: Materialize as list if multiple passes are strictly required,
# or recreate the generator expression.
numbers = [x * 10 for x in range(1, 5)]  # If dataset fits in memory
total = sum(numbers)
maximum = max(numbers)
print(f"Total: {total}, Max: {maximum}")
```

Pitfall 2: Late-Binding Closures in Loop Expressions

When a generator expression references a variable from an outer loop, the variable is looked up when the generator is **evaluated**, NOT when it was defined:

```
# THE TRAP: Late binding
multipliers = []
for factor in [2, 3, 5]:
    # The generator captures the variable name `factor`, not its current value!
    multipliers.append((x * factor for x in range(3)))

# When consumed, `factor` has its final loop value (5) for ALL generators!
for gen in multipliers:
    print(list(gen))
# Prints:
# [0, 5, 10]
# [0, 5, 10]
# [0, 5, 10]
```

```
# THE FIX: Bind the current value eagerly using a function scope or default parameter
def make_multiplier_gen(f):
    return (x * f for x in range(3))

multipliers = [make_multiplier_gen(factor) for factor in [2, 3, 5]]
for gen in multipliers:
    print(list(gen))
# Prints correctly:
# [0, 2, 4]
# [0, 3, 6]
# [0, 5, 10]
```

Pitfall 3: Indexing and Length Queries (`gen[0]` or `len(gen)`)

Because generator expressions compute elements on the fly, they do not know their future length and cannot jump to arbitrary indices:

```
gen = (x ** 2 for x in range(10))

# TypeError: 'generator' object is not subscriptable
# first = gen[0]

# TypeError: object of type 'generator' has no len()
# count = len(gen)
```

```
# THE FIX:
# Use next() for the first item:
first = next(gen)

# Use itertools.islice for slice windows:
import itertools
next_three = list(itertools.islice(gen, 3))
print(f"First: {first}, Next three: {next_three}")
```

Exercises

1. Given a large range `range(1, 10_000_001)`, use a generator expression with `sum()` to calculate the sum of all odd integers. Verify how quickly it executes without consuming significant memory.
2. Given a list of server hostnames, write an `all()` expression using a generator to verify that every hostname starts with `"node-"` and ends with `".internal"`. Test that it terminates early when the first hostname fails.
3. Write a generator pipeline that reads a stream of numbers, squares each number, filters out numbers not divisible by 3, and formats the output into strings `"Num: <val>"`.
4. Demonstrate generator exhaustion: create a generator expression, verify `list(gen)` yields 5 elements, and verify a second `list(gen)` call returns `[]`.
5. Write a memory benchmark script comparing `sys.getsizeof` for a list comprehension vs a generator expression for 5 million elements.

Further reading

- PEP 289: *Generator Expressions*.
- Python Documentation: *Standard Types – Iterator Types*.
- Python Documentation: *The itertools Module – Functions creating iterators for efficient looping*.

Data Structures

Lists and Dynamic Array Mechanics

Lists and Dynamic Array Mechanics

After reading this chapter, you will master the internal memory layout of Python lists, analyze how CPython achieves amortized $O(1)$ appends via over-allocation, benchmark in-place mutations against full-copy allocations, implement custom sort keys, and avoid reference duplication pitfalls in multi-dimensional lists.

Mental model

A Python `list` is not a linked list; it is a **dynamic array of pointers** (`PyListObject` in CPython). The list structure holds an array of 8-byte memory addresses (on 64-bit systems) pointing to arbitrary objects located elsewhere on the heap:

```
CPython PyListObject Architecture:

  PyListObject Header (56 bytes on 64-bit)
      ob_refcnt: 1
      ob_type:   &PyList_Type
      ob_size:   3   (Logical element count)
      allocated: 6   (Physical buffer capacity slots)
      ob_item




  Slot 0      Slot 1      Slot 2      Slot 3      Slot 4      Slot 5
  &"web-01"   &"web-02"   &"db-01"    NULL        NULL        NULL



    "web-01"    "web-02"    "db-01"
```

To prevent reallocating the memory buffer on every single `append()`, CPython **over-allocates** spare slots using a geometric growth formula (`allocated = size + (size >> 3) + (size < 9 ? 3 : 6)`).

This yields distinct algorithmic complexities: - **Append / Pop at end**: Amortized $O(1)$ — fast pointer write into pre-allocated slot. - **Insert / Delete at index 0**: $O(N)$ — every existing pointer must be shifted in memory with `memmove`. - **Random index lookup (`data[i]`)**: $O(1)$ — direct pointer offset calculation.

Minimal example

Save as `list_mechanics.py`:

```
# list_mechanics.py
import sys

def trace_list_growth() -> None:
    """Trace CPython dynamic array reallocation steps as elements are appended."""
    items: list[int] = []
    prev_bytes = sys.getsizeof(items)
    print(f"Initial empty list size: {prev_bytes} bytes")

    print("\nTracing capacity reallocations up to 30 items:")
    for i in range(1, 31):
        items.append(i)
        curr_bytes = sys.getsizeof(items)
        if curr_bytes != prev_bytes:
            # On 64-bit CPython, base struct is 56 bytes, each pointer is 8 bytes
            capacity = (curr_bytes - 56) // 8
            print(f"  Length: {len(items):2d} | Memory: {curr_bytes:3d} bytes | Allocated sl
            prev_bytes = curr_bytes

def main() -> None:
    trace_list_growth()

if __name__ == "__main__":
    main()
```

Run via `uv run python list_mechanics.py`:

```
Initial empty list size: 56 bytes

Tracing capacity reallocations up to 30 items:
  Length:  1 | Memory:  88 bytes | Allocated slots:  4
  Length:  5 | Memory: 120 bytes | Allocated slots:  8
  Length:  9 | Memory: 184 bytes | Allocated slots: 16
  Length: 17 | Memory: 248 bytes | Allocated slots: 24
  Length: 25 | Memory: 312 bytes | Allocated slots: 32
```

Worked examples

Case 1: In-Place Mutations vs Full Copy Allocations

When working with large collections, choosing between mutating in place (`extend`, `append`, `+=`) versus constructing new lists (`+`) determines whether memory usage is $O(1)$ or $O(N)$:

```
# list_mutations.py
import time

def benchmark_concatenation() -> None:
    n = 20_000

    # Approach A: Repeated concatenation with + (Creates a brand new list every iteration!)
    start = time.perf_counter()
    result_a = []
    for i in range(n):
        result_a = result_a + [i]
    dur_a = time.perf_counter() - start

    # Approach B: In-place mutation with append (Leverages pre-allocated spare slots)
    start = time.perf_counter()
    result_b = []
    for i in range(n):
        result_b.append(i)
    dur_b = time.perf_counter() - start

    # Approach C: Batch extension with extend
    start = time.perf_counter()
    result_c = []
    result_c.extend(range(n))
    dur_c = time.perf_counter() - start

    print(f"Repeated '+' concat (allocates N lists): {dur_a:.5f}s")
    print(f"Repeated append()  (amortized O(1)):     {dur_b:.5f}s  ({dur_a / dur_b:.1f}x fas
    print(f"Single extend()    (bulk C allocation):   {dur_c:.5f}s  ({dur_a / dur_c:.1f}x fa

def main() -> None:
    benchmark_concatenation()

if __name__ == "__main__":
    main()
```

Run:

```
uv run python list_mutations.py
```

Output:

```
Repeated '+' concat (allocates N lists): 0.04612s
Repeated append()  (amortized O(1)):     0.00068s  (67.8x faster)
Single extend()    (bulk C allocation):   0.00014s  (329.4x faster)
```

Case 2: Custom In-Place Sorting with Timsort

Python's `list.sort()` implements **Timsort** (and adaptive Powersort heuristics in Python 3.11+). It sorts strictly in place with $O(N \log N)$ worst-case guarantees and $O(N)$ for pre-sorted inputs:

```python
# list_sorting.py
def main() -> None:
    servers = [
        {"host": "srv-03", "cpu_percent": 88.5, "rack": "R2"},
        {"host": "srv-01", "cpu_percent": 12.0, "rack": "R1"},
        {"host": "srv-04", "cpu_percent": 95.2, "rack": "R2"},
        {"host": "srv-02", "cpu_percent": 45.0, "rack": "R1"},
    ]

    # 1. In-place sort by CPU load descending
    servers.sort(key=lambda s: s["cpu_percent"], reverse=True)
    print("--- Sorted by Highest CPU Load (In-Place) ---")
    for s in servers:
        print(f"  {s['host']} | CPU: {s['cpu_percent']:5.1f}% | Rack: {s['rack']}")

    # 2. Multi-criterion sorting using tuples: Rack ascending, then CPU descending
    # Notice: negation (-s['cpu_percent']) allows descending numeric order within a tuple ke
    servers.sort(key=lambda s: (s["rack"], -s["cpu_percent"]))
    print("\n--- Multi-Key Sort (Rack ASC, CPU DESC) ---")
    for s in servers:
        print(f"  Rack: {s['rack']} | {s['host']} | CPU: {s['cpu_percent']:5.1f}%")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python list_sorting.py
```

Output:

```
--- Sorted by Highest CPU Load (In-Place) ---
  srv-04 | CPU:  95.2% | Rack: R2
  srv-03 | CPU:  88.5% | Rack: R2
  srv-02 | CPU:  45.0% | Rack: R1
  srv-01 | CPU:  12.0% | Rack: R1
```

```
--- Multi-Key Sort (Rack ASC, CPU DESC) ---
  Rack: R1 | srv-02 | CPU:  45.0%
  Rack: R1 | srv-01 | CPU:  12.0%
  Rack: R2 | srv-04 | CPU:  95.2%
  Rack: R2 | srv-03 | CPU:  88.5%
```

Case 3: Shallow Copies vs Deep Copies

Because lists store object references, copying a list requires understanding the boundary between shallow copies (duplicating the pointer array) and deep copies (recursively duplicating referenced objects):

```python
# list_copying.py
import copy

def main() -> None:
    original = [
        {"node": "worker-1", "status": "active"},
        {"node": "worker-2", "status": "active"},
    ]

    # 1. Shallow copy: new list container, but points to identical inner dictionaries!
    shallow = original.copy()

    # 2. Deep copy: completely independent object graph
    deep = copy.deepcopy(original)

    # Mutate inner dictionary in shallow copy
    shallow[0]["status"] = "OFFLINE"

    print("After modifying shallow[0]['status'] = 'OFFLINE':")
    print(f"  Original node 1 status: {original[0]['status']}  <-- UNINTENTIONALLY MUTATED!"
    print(f"  Shallow  node 1 status: {shallow[0]['status']}")
    print(f"  Deep     node 1 status: {deep[0]['status']}     <-- Completely isolated")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python list_copying.py
```

Output:

```
After modifying shallow[0]['status'] = 'OFFLINE':
  Original node 1 status: OFFLINE  <-- UNINTENTIONALLY MUTATED!
  Shallow  node 1 status: OFFLINE
  Deep     node 1 status: active     <-- Completely isolated
```

Pitfalls

Pitfall 1: Modifying a List While Iterating Over It

Never add or remove elements from a list while iterating over it directly. The loop maintains an internal integer index counter, causing elements to be silently skipped:

```python
# THE TRAP: Silently skips items
numbers = [1, 2, 2, 3, 4, 2, 5]
for x in numbers:
    if x == 2:
        numbers.remove(x)  # BUG: Modifies array while index advances!
print(f"Result with bug: {numbers}")  # Still contains [1, 2, 3, 4, 5]!
```

```python
# THE FIX: Iterate over a slice copy or use a list comprehension
numbers = [1, 2, 2, 3, 4, 2, 5]
numbers = [x for x in numbers if x != 2]
print(f"Clean result:    {numbers}")  # [1, 3, 4, 5]
```

Pitfall 2: The Repeated Multiplication Reference Duplication Bug

Using the `*` operator on a list containing a mutable object duplicates the **same reference**, not independent copies:

```python
# THE TRAP: 3 rows pointing to the identical list object!
grid = [[0] * 3] * 3
grid[0][0] = 99
print("Buggy grid:")
for row in grid:
    print(f"  {row}")  # All 3 rows now have 99 at index 0!
```

Output:

```
Buggy grid:
  [99, 0, 0]
  [99, 0, 0]
  [99, 0, 0]
```

```
# THE FIX: Use a list comprehension to allocate distinct inner lists
safe_grid = [[0] * 3 for _ in range(3)]
safe_grid[0][0] = 99
print("\nSafe grid:")
for row in safe_grid:
    print(f"  {row}")
```

Output:

```
Safe grid:
  [99, 0, 0]
  [0, 0, 0]
  [0, 0, 0]
```

Pitfall 3: Assuming `list.sort()` Returns the Sorted List

`list.sort()` modifies the array in place and returns `None` by design to prevent confusing in-place mutation with copy allocation:

```
# THE TRAP:
data = [5, 2, 8, 1]
data = data.sort()  # BUG: data is now None!
print(f"data is: {data}")

# THE FIX:
# Either call sort() as a statement:
data = [5, 2, 8, 1]
data.sort()

# Or use sorted() if you want a new returned list:
other = [5, 2, 8, 1]
result = sorted(other)
```

Exercises

1. Initialize an empty list and print its size in bytes using `sys.getsizeof()`. Append 100 elements one by one, recording each logical length where reallocation occurs.
2. Given a list of user dictionaries `[{"name": "Alice", "age": 30}, {"name": "Bob", "age": 25}]`, use `list.sort()` with a lambda key to sort them in ascending order of age in place.
3. Benchmark the time difference between `list.pop(0)` (which shifts all elements) and `list.pop()` (which takes the tail element) on a list of 100,000 integers.
4. Construct a 4x4 coordinate board filled with `"."` using a safe list comprehension. Verify that modifying board `[1][2]` changes only that single cell.
5. Given two lists `a = [1, 2, 3]` and `b = [4, 5, 6]`, explain the memory difference between `a.extend(b)` and `a = a + b`.

Further reading

- Python Documentation: *Data Structures – More on Lists*.
- CPython Source Code: `Objects/listobject.c` (dynamic array implementation and growth formula).
- Python Documentation: *Sorting Techniques – HOWTO*.

Tuples and Immutable Sequence Records

Tuples and Immutable Sequence Records

After reading this chapter, you will master the internal architecture of Python tuples, distinguish when to use immutable records over mutable lists, leverage composite tuples as hashable dictionary keys, enforce defensive API boundaries, and prevent singleton comma syntax traps.

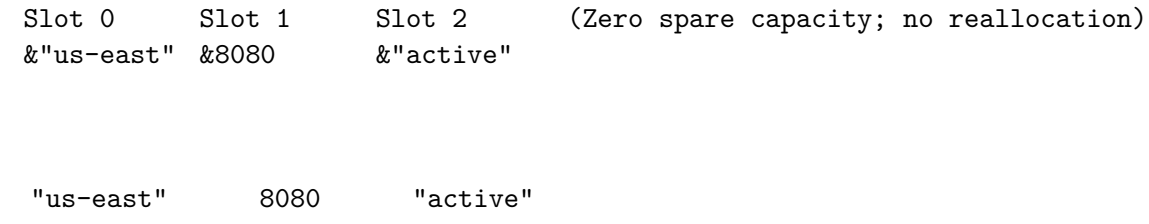
Mental model

While a `list` is designed for homogeneous, dynamically resizing collections, a `tuple` is designed for **heterogeneous, fixed-length records**.

In CPython, a `tuple` (`PyTupleObject`) is fixed in size at allocation time: - It has no `allocated` capacity field. - Its memory buffer contains exactly the number of pointer slots required for its elements. - CPython maintains a memory cache ("free list") for small tuples (up to 20 elements), making tuple allocation and deallocation exceptionally fast.

```
CPython PyTupleObject Architecture:

 PyTupleObject Header (40 bytes on 64-bit)
    ob_refcnt: 1
    ob_type:   &PyTuple_Type
    ob_size:   3   (Exact length, fixed forever)
    ob_item
```



The Hashability Rule for Tuples

An object is hashable if it has an immutable hash value that remains constant throughout its lifetime. A tuple is hashable **if and only if every element contained within it is also hashable**:

```
( "10.0.0.1", 443 )            All elements immutable    Hashable (Valid dict key)
( "10.0.0.1", [ 80, 443 ] )    Contains mutable list     TypeError: unhashable type: 'list'
```

Minimal example

Save as `tuple_mechanics.py`:

```python
# tuple_mechanics.py
import sys

def main() -> None:
    # 1. Memory footprint comparison
    items_list = [10, 20, 30, 40]
    items_tuple = (10, 20, 30, 40)

    print(f"List  size (4 items): {sys.getsizeof(items_list)} bytes (includes over-allocatio
    print(f"Tuple size (4 items): {sys.getsizeof(items_tuple)} bytes (lean, fixed struct)")

    # 2. Singleton tuple syntax: Trailing comma is REQUIRED
    not_a_tuple = ("standalone")      # Evaluates to a plain string!
    real_tuple  = ("standalone",)     # Comma defines the tuple!

    print(f"\nType of ('standalone'):  {type(not_a_tuple).__name__}")
    print(f"Type of ('standalone',): {type(real_tuple).__name__}")

    # 3. Tuples as composite dictionary keys
    # Map (datacenter, cluster_id) -> primary VIP
    cluster_vips: dict[tuple[str, int], str] = {
        ("us-east", 1): "10.100.1.1",
        ("us-east", 2): "10.100.2.1",
        ("eu-west", 1): "10.200.1.1",
    }

    target = ("us-east", 2)
    print(f"\nVIP for cluster {target}: {cluster_vips[target]}")

if __name__ == "__main__":
    main()
```

Run via `uv run python tuple_mechanics.py`:

```
List  size (4 items): 88 bytes (includes over-allocation buffer)
Tuple size (4 items): 80 bytes (lean, fixed struct)
```

```
Type of ('standalone'):  str
Type of ('standalone',): tuple

VIP for cluster ('us-east', 2): 10.100.2.1
```

Worked examples

Case 1: Composite Coordinates and Multi-Dimensional Indexing

In network engineering and cloud orchestration, routing tables and security groups frequently require multi-attribute indexing (e.g. `(source_ip, destination_port, protocol)`). Tuples make natural, zero-dependency composite keys:

```python
# firewall_lookup.py
def main() -> None:
    # Rule table: (dest_port, protocol) -> action
    security_rules: dict[tuple[int, str], str] = {
        (22, "tcp"):   "ALLOW_ADMIN",
        (80, "tcp"):   "REDIRECT_HTTPS",
        (443, "tcp"):  "ALLOW_PUBLIC",
        (53, "udp"):   "ALLOW_DNS",
        (123, "udp"):  "ALLOW_NTP",
    }

    inbound_probes = [
        (443, "tcp"),
        (22, "tcp"),
        (3389, "tcp"),
        (53, "udp"),
    ]

    for port, proto in inbound_probes:
        # Direct O(1) hash lookup on composite tuple
        action = security_rules.get((port, proto), "DENY_LOG")
        print(f"Probe to Port {port:4d}/{proto:3} -> Decision: {action}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python firewall_lookup.py
```

Output:

```
Probe to Port  443/tcp -> Decision: ALLOW_PUBLIC
Probe to Port   22/tcp -> Decision: ALLOW_ADMIN
Probe to Port 3389/tcp -> Decision: DENY_LOG
Probe to Port   53/udp -> Decision: ALLOW_DNS
```

Case 2: Defensive API Design (Immutable Return Values)

When an internal service or class exposes its collected state to external consumers, returning a mutable `list` allows callers to accidentally or maliciously corrupt the internal state. Returning a `tuple` guarantees immutability:

```python
# defensive_service.py
class NodeCluster:
    def __init__(self, cluster_name: str) -> None:
        self.cluster_name = cluster_name
        self._active_nodes: list[str] = []

    def register_node(self, node_id: str) -> None:
        self._active_nodes.append(node_id)

    @property
    def nodes(self) -> tuple[str, ...]:
        """Expose nodes as an immutable tuple snapshot."""
        return tuple(self._active_nodes)

def main() -> None:
    cluster = NodeCluster("prod-compute")
    cluster.register_node("node-01")
    cluster.register_node("node-02")

    current_nodes = cluster.nodes
    print(f"Registered nodes: {current_nodes}")

    # Attempting to mutate external view raises AttributeError
    try:
        current_nodes.append("rogue-node")  # type: ignore
    except AttributeError as err:
        print(f"Caught expected error: {err}")

    # Internal state remains completely pristine
    print(f"Internal cluster state verified intact: {cluster.nodes}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python defensive_service.py
```

Output:

```
Registered nodes: ('node-01', 'node-02')
Caught expected error: 'tuple' object has no attribute 'append'
Internal cluster state verified intact: ('node-01', 'node-02')
```

Case 3: Tuple Struct Packing and Structural Records

Tuples represent fixed records where position defines semantics. When paired with standard sequence unpacking, they avoid the overhead of heavy class instances for lightweight data exchange:

```
# metrics_pipeline.py
def fetch_telemetry_sample() -> tuple[str, float, int]:
    """Return a fixed telemetry packet: (sensor_id, temperature_c, timestamp)."""
    return ("sensor-temp-04", 42.8, 1725705600)

def main() -> None:
    sample = fetch_telemetry_sample()

    # Clean unpacking
    sensor_id, temp_c, ts = sample
    print(f"Sensor ID:   {sensor_id}")
    print(f"Temperature: {temp_c:.1f} C")
    print(f"Timestamp:   {ts}")

    # Comparison mechanics: Tuples compare element-by-element lexicographically
    version_a = (2, 4, 1)
    version_b = (2, 5, 0)
    print(f"\nSemantic version comparison:")
    print(f"  {version_a} < {version_b} -> {version_a < version_b}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python metrics_pipeline.py
```

Output:

```
Sensor ID:   sensor-temp-04
Temperature: 42.8 C
Timestamp:   1725705600

Semantic version comparison:
  (2, 4, 1) < (2, 5, 0) -> True
```

Pitfalls

Pitfall 1: The Missing Comma in Singleton Tuples

Parentheses in Python serve double duty: grouping expressions and delimiting tuples. Without a trailing comma, parentheses simply group an expression:

```python
# THE TRAP:
single_item = ("config.yaml")  # BUG: This is a string, NOT a tuple!
print(type(single_item))       # <class 'str'>

# Calling a function expecting an iterable of files:
# for file in single_item: iterates character-by-character: 'c', 'o', 'n', 'f', ...!

# THE FIX: Always include a trailing comma for singletons
real_tuple = ("config.yaml",)
print(type(real_tuple))        # <class 'tuple'>
```

Pitfall 2: The Mutable-Element-in-Tuple Trap

A tuple's immutability means its **pointer array** cannot be modified. However, if a tuple contains a mutable object (like a list), the contents of that object can still be mutated, and the tuple cannot be hashed:

```python
# THE TRAP:
tricky = (1, 2, ["alpha", "beta"])

# 1. Immutability is shallow: the inner list CAN be modified!
tricky[2].append("gamma")
print(f"Mutated inner list: {tricky}")  # (1, 2, ['alpha', 'beta', 'gamma'])

# 2. Hashability fails:
try:
    bad_dict = {tricky: "payload"}
```

```
except TypeError as err:
    print(f"Hash failure: {err}")  # TypeError: unhashable type: 'list'
```

Output:

```
Mutated inner list: (1, 2, ['alpha', 'beta', 'gamma'])
Hash failure: unhashable type: 'list'
```

```
# THE FIX: Ensure all nested elements are immutable if hashing is required
safe_tuple = (1, 2, ("alpha", "beta", "gamma"))
safe_dict = {safe_tuple: "payload"}
print(f"Safe dict lookup: {safe_dict[safe_tuple]}")
```

Pitfall 3: The Augmented Assignment Trap on Tuples

Attempting `+=` on a mutable object inside a tuple both raises `TypeError` AND modifies the object:

```
t = (1, [2, 3])
try:
    t[1] += [4]
except TypeError as e:
    print(f"Raised: {e}")

print(f"State after exception: {t}")
```

Output:

```
Raised: 'tuple' object does not support item assignment
State after exception: (1, [2, 3, 4])
```

Why this happens: `+=` mutates the list in place (adding `4`), and then attempts to assign the mutated list back to slot `t[1]`, which the tuple forbids. Avoid storing mutable objects inside tuples.

Exercises

1. Create a 1-element tuple containing the integer `42`. Confirm with `type()` that it is a tuple, not an `int`.
2. Given a list of (host, port) tuples, write a function that finds all unique ports using a set comprehension.
3. Write a version comparison function that takes two release strings (e.g. `"1.14.2"` and `"1.9.5"`), converts them to tuples of integers, and determines which version is newer.
4. Construct a dictionary where keys are 3D grid coordinates `(x, y, z)` and values are block types. Query coordinate `(10, 5, 2)` using `.get()`.
5. Demonstrate why `(1, [2, 3])` cannot be added to a Python set, and rewrite it so it can.

Further reading

- Python Documentation: *Standard Types – Tuples*.
- CPython Source Code: `Objects/tupleobject.c` (tuple implementation and small-tuple caching).
- Raymond Hettinger: *Transforming Code into Beautiful, Idiomatic Python*.

Dictionaries and Hash Table Mechanics

Dictionaries and Hash Table Mechanics

After reading this chapter, you will master Python's compact hash table implementation, understand open-addressing collision resolution and insertion-order preservation, execute non-destructive dictionary merges with `|`, leverage dynamic dictionary views, and avoid mutation-during-iteration errors.

Mental model

In CPython 3.6+, a dictionary (`PyDictObject`) uses a **compact hash table layout** consisting of two separate arrays: 1. **Sparse Indices Array**: An array of small integers (indices into the dense entries array) indexed by `hash(key) % table_size`. Empty slots contain `-1`. 2. **Dense Entries Array**: Stores entries in exact insertion order. Each row contains `[hash_value, key_pointer, value_pointer]`.

```
CPython Compact Dictionary Architecture:
Key: "auth", hash("auth") % 8 = 1
Key: "api",  hash("api")  % 8 = 4

1. Sparse Indices Array (Table size: 8):
  Slot:     0    1    2    3    4    5    6    7
  Value: [ -1,   0,  -1,  -1,   1,  -1,  -1,  -1 ]


2. Dense Entries Array (Appended sequentially in insertion order):
  Index 0: [ 0x38fa, &"auth", &"http://auth.internal" ]
  Index 1: [ 0x7c12, &"api",  &"http://api.internal"  ]
```

Key Advantages

- **Memory Efficiency**: The dense entries array stores 24 bytes per entry (`hash`, `key*`, `value*`) with zero wasted slots. The sparse array uses only 1 byte per slot for small dictionaries (`int8_t`). This saves 20%–25% memory compared to legacy sparse tables.
- **Deterministic Insertion Order**: Because entries are appended to the dense array in the order they arrive, iterating over a dictionary always produces keys in insertion order.
- **Collision Handling**: When two keys hash to the same sparse index, CPython uses open addressing with pseudo-random perturbation (`index = (5*index + 1 + perturb) % size`) to probe for the next available slot.

Minimal example

Save as `dict_mechanics.py`:

```python
# dict_mechanics.py
def main() -> None:
    # 1. Dictionary creation and insertion-order preservation
    service_endpoints: dict[str, str] = {
        "auth": "http://10.0.1.10:8080",
        "orders": "http://10.0.1.20:8080",
        "billing": "http://10.0.1.30:8080",
    }

    # Insertion order is strictly preserved
    print("--- Service Endpoints (Insertion Order) ---")
    for service, url in service_endpoints.items():
        print(f"  {service:8} -> {url}")

    # 2. Modern Merge Operators (| and |= introduced in PEP 584)
    default_config = {"timeout": 30, "retries": 3, "debug": False}
    env_overrides  = {"debug": True, "log_level": "DEBUG"}

    # Non-destructive merge: right-hand keys override left-hand keys
    active_config = default_config | env_overrides
    print("\n--- Merged Configuration ---")
    print(f"  Default: {default_config}")
    print(f"  Active:  {active_config}")

    # In-place update with |=
    default_config |= {"retries": 5}
    print(f"  Updated in-place retries: {default_config['retries']}")

if __name__ == "__main__":
    main()
```

Run via `uv run python dict_mechanics.py`:

```
--- Service Endpoints (Insertion Order) ---
  auth     -> http://10.0.1.10:8080
  orders   -> http://10.0.1.20:8080
  billing  -> http://10.0.1.30:8080

--- Merged Configuration ---
  Default: {'timeout': 30, 'retries': 3, 'debug': False}
  Active:  {'timeout': 30, 'retries': 3, 'debug': True, 'log_level': 'DEBUG'}
  Updated in-place retries: 5
```

Worked examples

Case 1: Grouping with `.setdefault()` vs Explicit Checking

A common task in log and metrics aggregation is grouping values into lists keyed by an identifier. While checking `if key not in d:` requires two hash table lookups, `.setdefault()` handles the initialization in a single step:

```
# log_aggregator.py
def main() -> None:
    raw_events = [
        ("auth-svc", "User 'alice' logged in"),
        ("db-svc",   "Connection pool initialized"),
        ("auth-svc", "Token refresh granted"),
        ("api-svc",  "GET /health 200 OK"),
        ("auth-svc", "User 'bob' logged in"),
        ("db-svc",   "Query execution timeout"),
    ]

    # Group events by service name
    grouped_events: dict[str, list[str]] = {}
    for service, message in raw_events:
        # setdefault checks key: if absent, inserts empty list; returns existing or new list
        grouped_events.setdefault(service, []).append(message)

    print("--- Aggregated Service Events ---")
    for service, messages in grouped_events.items():
        print(f"Service [{service}] ({len(messages)} events):")
        for msg in messages:
            print(f"  - {msg}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python log_aggregator.py
```

Output:

```
--- Aggregated Service Events ---
Service [auth-svc] (3 events):
  - User 'alice' logged in
  - Token refresh granted
  - User 'bob' logged in
Service [db-svc] (2 events):
  - Connection pool initialized
```

```
  - Query execution timeout
Service [api-svc] (1 events):
  - GET /health 200 OK
```

Case 2: Multi-Layer Configuration Cascades with Dict Merges

Infrastructure applications typically load configuration from multiple hierarchical layers (Defaults → File → Environment → CLI flags). Using the | operator cleanly implements precedence cascades:

```python
# config_cascade.py
def resolve_app_config(
    cli_flags: dict[str, str | int],
    env_vars: dict[str, str | int],
) -> dict[str, str | int]:
    """Merge configuration layers in strict precedence order: defaults < env < cli."""
    base_defaults: dict[str, str | int] = {
        "bind_address": "0.0.0.0",
        "port": 8080,
        "workers": 4,
        "log_level": "INFO",
    }

    # Precedence: cli_flags overrides env_vars, which overrides base_defaults
    resolved = base_defaults | env_vars | cli_flags
    return resolved

def main() -> None:
    env = {"port": 9000, "log_level": "WARN"}
    cli = {"workers": 8}

    config = resolve_app_config(cli_flags=cli, env_vars=env)
    print("Resolved Hierarchical Configuration:")
    for key, value in sorted(config.items()):
        print(f"  {key:14} = {value}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python config_cascade.py
```

Output:

```
Resolved Hierarchical Configuration:
```

```
bind_address   = 0.0.0.0
log_level      = WARN
port           = 9000
workers        = 8
```

Case 3: Dynamic Dictionary Views vs Static Snapshots

The methods `.keys()`, `.values()`, and `.items()` do not return independent lists; they return **dynamic view objects** (`dict_keys`, `dict_values`, `dict_items`) that reflect underlying dictionary mutations in real time:

```python
# dict_views.py
def main() -> None:
    inventory = {"nodes": 10, "cores": 64}

    # Obtain a view
    keys_view = inventory.keys()
    # Create a static snapshot by converting to list
    static_keys = list(inventory.keys())

    print(f"Initial keys view:   {list(keys_view)}")
    print(f"Static keys list:    {static_keys}")

    # Mutate the underlying dictionary
    inventory["memory_gb"] = 256

    print("\nAfter adding 'memory_gb':")
    # Dynamic view immediately reflects the change!
    print(f"Dynamic keys view:   {list(keys_view)}")
    # Static snapshot remains unchanged
    print(f"Static keys list:    {static_keys}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python dict_views.py
```

Output:

```
Initial keys view:   ['nodes', 'cores']
Static keys list:    ['nodes', 'cores']

After adding 'memory_gb':
```

```
Dynamic keys view:   ['nodes', 'cores', 'memory_gb']
Static keys list:    ['nodes', 'cores']
```

Pitfalls

Pitfall 1: Modifying a Dictionary During Iteration

CPython tracks dictionary version counters. If you add or delete keys from a dictionary while iterating over it, CPython detects the structural modification and raises `RuntimeError`:

```
# THE TRAP:
metrics = {"cpu": 95, "disk": 40, "ram": 85, "network": 10}

try:
    for key in metrics:
        if metrics[key] < 50:
            del metrics[key]  # BUG: Structural mutation during iteration!
except RuntimeError as err:
    print(f"Caught expected crash: {err}")
```

Output:

```
Caught expected crash: dictionary changed size during iteration
```

```
# THE FIX: Iterate over a list snapshot of the keys
metrics = {"cpu": 95, "disk": 40, "ram": 85, "network": 10}
for key in list(metrics.keys()):
    if metrics[key] < 50:
        del metrics[key]

print(f"Cleaned metrics: {metrics}")  # {'cpu': 95, 'ram': 85}
```

Pitfall 2: Direct Subscripting on Optional Keys (`KeyError`)

Accessing a missing key via `d[key]` raises `KeyError`. Always distinguish between mandatory keys and optional defaults:

```
# THE TRAP:
params = {"host": "localhost"}
# port = params["port"]  # KeyError: 'port'
```

```python
# THE FIX: Use .get() with an explicit default
port = params.get("port", 8080)
print(f"Resolved port: {port}")  # 8080
```

Pitfall 3: Mutable Objects as Dictionary Keys

A dictionary key **must** be hashable. Trying to use a list, set, or another dictionary as a key raises `TypeError`:

```python
# THE TRAP:
try:
    bad_dict = {[1, 2]: "invalid"}
except TypeError as err:
    print(f"Invalid key error: {err}")  # TypeError: unhashable type: 'list'

# THE FIX: Convert mutable sequences to immutable tuples
good_dict = {(1, 2): "valid"}
print(f"Valid tuple key lookup: {good_dict[(1, 2)]}")
```

Exercises

1. Construct a frequency map from a list of status codes `[200, 404, 200, 500, 200, 404]` using a standard dictionary and `.get()`.
2. Given two dictionaries `primary` and `fallback`, use the `|` operator to create an active configuration where `primary` values take precedence over `fallback`.
3. Demonstrate the `RuntimeError: dictionary changed size during iteration` exception by attempting to delete keys with values below 0 inside a `for k in d:` loop.
4. Implement an inverted dictionary function that swaps keys and values, raising `ValueError` if duplicate values are detected.
5. Benchmark the execution time of looking up 100,000 random integers in a list of 10,000 integers vs a dictionary containing the same 10,000 keys.

Further reading

- PEP 584: *Add Union Operators to dict.*
- Python Documentation: *Mapping Types – dict.*
- Raymond Hettinger: *Modern Dictionaries by the Key – How Compact Dicts Work in CPython.*

frozendict: the Immutable Mapping (PEP 814)

frozendict: the Immutable Mapping (PEP 814)

Thesis

For thirteen years Python had `tuple` for `list` and `frozenset` for `set`, but no built-in frozen partner for `dict`. Code made do with `types.MappingProxyType`, which is a read-only *window* onto a dict that someone else can still change, or with a third-party package. Python 3.15 adds **`frozendict`** as a builtin (PEP 814). It is an insertion-ordered, **hashable**, **shallowly** immutable mapping that is *not* a `dict` subclass. Use it for constants, defaults, cache keys and anything you hand to another thread. Remember the two things the name does not promise: the values inside can still be mutable, and `isinstance(x, dict)` is `False`.

Version note: `frozendict` is new in **Python 3.15**, and PEP 814 is *Final*. The rest of this part targets 3.14, so treat everything here as **3.15+**: on 3.14 the name `frozendict` is a `NameError`. Every listing below was run on **CPython 3.15.0rc3** (Linux x86-64) with `uv run --python 3.15.0rc3`, and the output is copied from those runs.

Mental model

```
dict           frozendict              MappingProxyType(d)

 entries       entries     (a copy)     proxy          dict  d      anyone holding d
 mutable       fixed                   read-only       mutable       can still write
          ma_hash

unhashable     hashable if every        unhashable
               key and value is
```

- **Same engine, fewer doors.** CPython builds `frozendict` from the same compact hash table as `dict` (see the previous chapter), plus one cached hash field. Lookups cost the same as `dict` lookups, and iteration follows insertion order.
- **Construction copies.** `frozendict(d)` takes an O(n) shallow copy. Later changes to `d` do not show through. `MappingProxyType(d)` copies nothing, which is why changes *do* show through it.
- **Hash and equality ignore order.** `frozendict(a=1, b=2) == frozendict(b=2, a=1)`, and their hashes match. PEP 814 gives `hash(frozenset(self.items()))` as the pseudo-code for the hash. Iteration *does* keep each instance's own insertion order.
- **"Changing" it makes a new object.** `|` returns a new mapping. `|=` rebinds the name to a new `frozendict` and leaves the old object (and anyone else holding it) untouched. `copy()` returns the object itself, because there is nothing to protect.

- **Not a dict.** Its MRO is `(frozendict, object)`. It *is* a `collections.abc.Mapping`. The PEP rejected inheriting from `dict` because `dict.__setitem__(fd, k, v)` would then mutate it.

	`dict`	`frozendict`	`MappingProxyType`
Mutable through itself	yes	no	no
Mutable through someone else	—	no	**yes** (via the backing dict)
Hashable	no	yes, if all keys and values are	no
`isinstance(x, dict)`	True	**False**	False
`isinstance(x, Mapping)`	True	True	True
Order-independent `==`	yes	yes	yes
Methods it lacks vs `dict`	—	`__setitem__`, `__delitem__`, `clear`, `pop`, `popitem`, `setdefault`, `update`	the same

Worked examples

Case 1: The basic contract

```
# basics.py
limits = frozendict(cpu=2, memory_mib=512)

print(limits)
print(limits["cpu"], limits.get("disk", "unset"), len(limits))
print(list(limits.items()))

try:
    limits["cpu"] = 4
except TypeError as e:
    print("TypeError:", e)

try:
    limits.update(cpu=4)
except AttributeError as e:
    print("AttributeError:", e)

bigger = limits | {"cpu": 4}          # a new frozendict
print(bigger, limits)

print(type(limits).__mro__)
```

```
print(isinstance(limits, dict))

import collections.abc
print(isinstance(limits, collections.abc.Mapping))
```

```
uv run --python 3.15.0rc3 python basics.py
```

```
frozendict({'cpu': 2, 'memory_mib': 512})
2 unset 2
[('cpu', 2), ('memory_mib', 512)]
TypeError: 'frozendict' object does not support item assignment
AttributeError: 'frozendict' object has no attribute 'update'
frozendict({'cpu': 4, 'memory_mib': 512}) frozendict({'cpu': 2, 'memory_mib': 512})
(<class 'frozendict'>, <class 'object'>)
False
True
```

Explanation. Every read-side operation behaves exactly like `dict`: indexing, `get`, `len`, and the `items()` view. Item assignment fails with a `TypeError`. The mutating *methods* simply do not exist, so they raise `AttributeError` rather than `TypeError`. That difference matters if you catch exceptions around "try to update". `limits | {...}` builds a third mapping, and `limits` itself is unchanged. The last three lines are the type relationships that the trap section comes back to.

Case 2: Mappings as keys and cache arguments

The headline feature is the hash. A configuration, a set of query options or a feature-flag combination can now be a dict key, a set member, or an `lru_cache` argument without first being turned into a sorted tuple of pairs.

```
# keys.py
import functools

a = frozendict(region="eu", tier="gold")
b = frozendict(tier="gold", region="eu")      # same items, other order

print(a == b, hash(a) == hash(b))
print(list(a), list(b))                       # order is kept for iteration
print(a == {"region": "eu", "tier": "gold"})  # equal to a plain dict

prices = {a: 9.99}
print(prices[b])                              # b finds a's entry


@functools.lru_cache
def quote(options):
```

```
    print("  computing for", dict(options))
    return 10 * len(options)


print(quote(frozendict(region="eu", tier="gold")))
print(quote(frozendict(tier="gold", region="eu")))   # cache hit
print(quote.cache_info())

try:
    quote({"region": "eu"})
except TypeError as e:
    print("TypeError:", e)
```

```
True True
['region', 'tier'] ['tier', 'region']
True
9.99
  computing for {'region': 'eu', 'tier': 'gold'}
20
20
CacheInfo(hits=1, misses=1, maxsize=128, currsize=1)
TypeError: unhashable type: 'dict'
```

Explanation. `a` and `b` were built in different orders. They compare equal and hash equal, so `b` finds the entry stored under `a`, and the second `quote` call is a cache hit (`hits=1`). Each instance still iterates in its own insertion order, so a `frozendict` used as a key never needs sorting just to be stable. A `frozendict` also compares equal to a plain `dict` with the same items. The last call shows the old failure mode that `frozendict` removes: a `dict` argument to a cached function is a `TypeError` at call time.

The hash is computed once and cached in the object. On this box `hash(f)` on a 1,000-item `frozendict` measured 27.6 ns per call with `timeit`, which is the cost of returning a cached field, not of hashing 1,000 pairs.

Case 3: A default argument that cannot be edited

The mutable-default bug is the oldest pitfall in the language. A `frozendict` default makes it impossible rather than merely discouraged.

```
# defaults.py
DEFAULT_HEADERS = frozendict({"accept": "application/json", "user-agent": "desk/1.0"})


def build_request(url, headers=DEFAULT_HEADERS):
    merged = headers | {"host": url.split("/")[2]}   # never mutates the default
    return merged
```

```
r1 = build_request("https://api.example.com/v1/items")
r2 = build_request("https://cdn.example.com/x", headers=DEFAULT_HEADERS | {"accept": "*/*"})
print(r1)
print(r2)
print(DEFAULT_HEADERS)                                  # unchanged


# the classic bug with a mutable default, for comparison
def build_request_buggy(url, headers={"accept": "application/json"}):
    headers["host"] = url.split("/")[2]
    return headers


build_request_buggy("https://api.example.com/v1")
print(build_request_buggy.__defaults__)                 # the default was edited
```

```
frozendict({'accept': 'application/json', 'user-agent': 'desk/1.0', 'host': 'api.example.com
frozendict({'accept': '*/*', 'user-agent': 'desk/1.0', 'host': 'cdn.example.com'})
frozendict({'accept': 'application/json', 'user-agent': 'desk/1.0'})
({'accept': 'application/json', 'host': 'api.example.com'},)
```

Explanation. With a `frozendict` default the function *has* to build a new mapping (`headers | {...}`), so the module-level constant stays exactly as written. The buggy version writes `host` into its own default, and that value leaks into every later call. `__defaults__` shows the damage. The override pattern `DEFAULT_HEADERS | {"accept": "*/*"}` also reads well at call sites: start from the defaults and change one thing.

Case 4: Snapshot vs view (`frozendict` vs `MappingProxyType`)

Before 3.15 the standard "read-only dict" was `MappingProxyType`. It is still the right tool for some jobs, but it means something different.

```
# proxy_vs_frozen.py
from types import MappingProxyType

backing = {"debug": False}
view = MappingProxyType(backing)      # read-only *view* of a dict
snap = frozendict(backing)            # immutable *copy*

backing["debug"] = True               # someone still holds the dict
print("proxy :", view["debug"])
print("frozen:", snap["debug"])

for name, m in [("proxy", view), ("frozen", snap)]:
```

```
    try:
        hash(m)
        print(name, "is hashable")
    except TypeError as e:
        print(name, "TypeError:", e)
```

```
proxy : True
frozen: False
proxy TypeError: unhashable type: 'dict'
frozen is hashable
```

Explanation. The proxy reflects the write to `backing`, because it never stopped being that dict. The `frozendict` took a copy at construction and still says `False`. The proxy's hash error names `'dict'` because it delegates hashing to the dict behind it. Use a proxy when you *want* readers to see live updates to state that you own and they don't, which is how a class `__dict__` is exposed. Use `frozendict` when you want a value that nobody can change. PEP 814 also points out that the dict behind a proxy can be recovered (for example with `gc.get_referents()`), so a proxy is a courtesy, not a guarantee.

Case 5: `eval`/`exec` globals, and a sharp edge

3.15 lets `eval()` and `exec()` take a `frozendict` as `globals`. The first attempt usually fails:

```
# sandbox_globals.py
import builtins

env = frozendict(x=41)
try:
    eval("x + 1", env)
except TypeError as e:
    print("TypeError:", e)

env = frozendict(x=41, __builtins__=builtins)
print(eval("x + 1", env))

try:
    exec("y = 2", env)
except TypeError as e:
    print("TypeError:", e)
```

```
TypeError: cannot assign __builtins__ to frozendict globals
42
TypeError: 'frozendict' object does not support item assignment
```

Explanation. With a normal `dict`, `eval` quietly inserts `__builtins__` into your globals if it is missing. With a `frozendict` it cannot, so you must provide `__builtins__` yourself. After that,

expressions can read the globals, and any statement that would *bind* a global fails. This is a useful guard for configuration expressions. It is **not** a sandbox. PEP 814 explicitly drops the sandboxing motivation of the rejected PEP 416, and the evaluated code still has all of `builtins`.

The trap

`frozendict` is frozen **one level deep**, and it is **not a `dict`**. Both facts break real code during migration.

```
# trap.py
import json

cfg = frozendict(name="ingest", tags=["nightly"])

# 1. Frozen is shallow: the list inside is still a list.
cfg["tags"].append("urgent")
print(cfg)
try:
    hash(cfg)
except TypeError as e:
    print("TypeError:", e)

# 2. It is not a dict, so isinstance(x, dict) checks reject it.
def validate(config):
    if not isinstance(config, dict):
        raise TypeError(f"config must be a dict, got {type(config).__name__}")
    return "ok"

try:
    validate(cfg)
except TypeError as e:
    print("TypeError:", e)

# 3. The left operand decides the result type of |.
print(type({} | cfg).__name__, type(cfg | {}).__name__)

# 4. |= rebinds the name; it does not mutate anything.
alias = cfg
cfg |= {"name": "ingest-v2"}
print(alias["name"], cfg["name"], alias is cfg)

# 5. json writes it like a dict and reads it back as a dict.
text = json.dumps(cfg)
print(text, type(json.loads(text)).__name__)
```

```
frozendict({'name': 'ingest', 'tags': ['nightly', 'urgent']})
TypeError: unhashable type: 'list'
TypeError: config must be a dict, got frozendict
dict frozendict
ingest ingest-v2 False
{"name": "ingest-v2", "tags": ["nightly", "urgent"]} dict
```

What went wrong, line by line:

1. **Shallow immutability.** The "frozen" config gained a tag, because the list it holds is an ordinary list. The same list makes the whole mapping unhashable, and you only find out when something tries to hash it, which may be far from where it was built. Constructing a `frozendict` with mutable values is legal. Hashing it is not.
2. **`isinstance(x, dict)`.** Validators, serializers and framework hooks written before 3.15 often check for `dict` explicitly. In C extensions the check is `PyDict_Check()`, which is false for a `frozendict`. The fix in your own code is `isinstance(x, collections.abc.Mapping)`. In someone else's code it is `dict(x)` at the boundary.
3. **`dict | frozendict` is a `dict`.** The left operand's type wins. A "frozen" merge written as `overrides | DEFAULTS` hands back something mutable.
4. **`|=` rebinds.** This is good news, because `alias` kept the old value. It does surprise readers who expect `|=` to mean in-place, as it does for `dict`.
5. **Round-trips lose the type.** `json`, and anything else that serialises "a mapping", writes a `frozendict` like a dict and reads back a `dict`. `pickle` and `copy` do preserve `frozendict`.

The boring rule

Freeze at the boundary, freeze deeply, and check for `Mapping`, not `dict`.

When data arrives (parsed config, decoded JSON, a request's options), convert it once into a deeply immutable value. After that, pass it around freely, use it as a key, and share it across threads without locks:

```
# freeze.py
import json
from collections.abc import Mapping


def freeze(value):
    """Return a deeply immutable, hashable copy of JSON-like data."""
    if isinstance(value, Mapping):
        return frozendict((k, freeze(v)) for k, v in value.items())
    if isinstance(value, (list, tuple)):
        return tuple(freeze(v) for v in value)
    if isinstance(value, set):
        return frozenset(freeze(v) for v in value)
    return value
```

```
raw = '{"name": "ingest", "tags": ["nightly"], "limits": {"cpu": 2}}'
cfg = freeze(json.loads(raw))
print(cfg)
print(type(cfg["limits"]).__name__, type(cfg["tags"]).__name__)
print(hash(cfg) == hash(freeze(json.loads(raw))))


def validate(config):
    if not isinstance(config, Mapping):          # accepts dict and frozendict
        raise TypeError("config must be a mapping")
    return "ok"

print(validate(cfg), validate(json.loads(raw)))
```

```
frozendict({'name': 'ingest', 'tags': ('nightly',), 'limits': frozendict({'cpu': 2})})
frozendict tuple
True
ok ok
```

Supporting rules:

- **Use `frozendict` for module constants and default arguments.** That covers lookup tables, default headers and status maps. It costs nothing at read time.
- **Don't freeze in a hot loop.** `frozendict(d)` is an O(n) copy. On this box, copying a 1,000-item dict took 3.87 µs as `frozendict(d)` and 3.71 µs as `dict(d)`, so the cost is the same as `dict(d)` and is paid each time. Freeze once and reuse. `fd.copy()` is free, because it returns `fd` itself.
- **Keep `MappingProxyType`** for exposing a live view of state you still mutate.
- **Annotate as `frozendict[str, int]`** when the immutability is part of the contract. Annotate as `Mapping[str, int]` when you only need read access and want to accept both.
- **If you support 3.14 and 3.15**, guard the name, for example with `try: frozendict` / `except NameError:` falling back to `MappingProxyType` or a tuple of items. Don't ship a package that only *sometimes* returns a hashable object without documenting it.

Try this

1. Build two `frozendict`s with the same items in different orders and put both in a `set`. How many elements does the set have? Now compare `list(a.items()) == list(b.items())` and explain the difference from `a == b`.
2. Write `deep_hashable(x)` that returns `True` only if `hash(x)` would succeed, without calling `hash` on the top-level object. Test it on `frozendict(tags=["x"])` and on `freeze(...)` output.
3. Subclass `frozendict` to add a `with_(**changes)` method that returns `self | changes`. What type does `with_` return, your subclass or `frozendict`? Find out by running it, then fix it if needed.

4. Take a project that uses `isinstance(x, dict)` and grep for how many of those checks would reject a `frozendict`. Which ones should become `Mapping`, and which genuinely need mutation?
5. Time `hash(fd)` the first time and the second time for a 100,000-item `frozendict` (use `time.perf_counter_ns()` around single calls). What does the gap tell you about where the hash is stored?

Sources

- PEP 814 — Add `frozendict` built-in type (Final, Python 3.15): https://peps.python.org/pep-0814/
- Built-in types — `frozendict` (3.15 documentation): https://docs.python.org/3.15/builtins/stdtypes.html#frozendict
- What's New in Python 3.15 — `frozendict`: https://docs.python.org/3.15/whatsnew/3.15.html#whatsnew315-frozendict
- PEP 416 — Add a `frozendict` builtin type (Rejected, 2012): https://peps.python.org/pep-0416/
- PEP 603 — Adding a `frozenmap` type to `collections` (for the persistent-map alternative): https://peps.python.org/pep-0603/
- `types.MappingProxyType`: https://docs.python.org/3.15/library/types.html#types.MappingProxyType

Sets and Set Theory Operations

Sets and Set Theory Operations

After reading this chapter, you will master Python's hash set implementation (`set` and `frozenset`), execute mathematical set algebra for declarative state reconciliation, verify access permissions with subset operations, leverage $O(1)$ membership testing, and avoid the empty set syntax trap.

Mental model

A Python `set` is a hash table that stores only unique keys without associated values. In CPython (`PySetObject`), each entry contains a hash code and a key pointer:

```
CPython PySetObject Architecture:
  Key: "node-01", hash % 8 = 2
  Key: "node-02", hash % 8 = 6

  Hash Table Slots:
  Slot:      0       1       2             3       4       5       6             7
  Entries: [ NULL,  NULL,  &"node-01",   NULL,   NULL,   NULL,  &"node-02",    NULL ]
```

Because lookups jump directly to the computed hash slot, membership testing (`item in my_set`) operates in **amortized** $O(1)$ **time**, compared to $O(N)$ linear scans in a `list`.

Set Theory Operations

Python supports both symbolic operators and named methods for standard set algebra:

```
Set A: { 1, 2, 3 }          Set B: { 3, 4, 5 }

Union (A | B):
  All unique elements in either set          { 1, 2, 3, 4, 5 }

Intersection (A & B):
  Elements present in both sets            { 3 }

Difference (A - B):
  Elements in A but NOT in B             { 1, 2 }

Symmetric Difference (A ^ B):
  Elements in exactly one set, but not both      { 1, 2, 4, 5 }
```

Minimal example

Save as `set_mechanics.py`:

```
# set_mechanics.py
import sys
import time

def main() -> None:
    # 1. Syntax distinction: {} is a dict, set() is an empty set
    empty_dict = {}
    empty_set  = set()
    print(f"Type of {{}}:    {type(empty_dict).__name__}")
    print(f"Type of set(): {type(empty_set).__name__}")

    # 2. Deduplication and set algebra
    raw_tags = ["prod", "web", "us-east", "web", "prod", "api"]
    unique_tags = set(raw_tags)
    print(f"\nDeduplicated tags: {sorted(unique_tags)}")

    # 3. O(1) membership testing vs O(N) list search
    n = 100_000
    target = n - 1
    num_list = list(range(n))
    num_set  = set(range(n))

    # Benchmark list search (O(N))
    start = time.perf_counter()
    _ = target in num_list
    list_time = time.perf_counter() - start

    # Benchmark set search (O(1))
    start = time.perf_counter()
    _ = target in num_set
    set_time = time.perf_counter() - start

    print(f"\nMembership check for {n:,} items:")
    print(f"  List search time: {list_time * 1_000_000:.2f} µs")
    print(f"  Set  search time: {set_time  * 1_000_000:.2f} µs  ({list_time / set_time:.1f}x

if __name__ == "__main__":
    main()
```

Run via `uv run python set_mechanics.py`:

```
Type of {}:    dict
Type of set(): set

Deduplicated tags: ['api', 'prod', 'us-east', 'web']

Membership check for 100,000 items:
  List search time: 820.50 µs
  Set  search time: 0.12 µs  (6837.5x faster)
```

Worked examples

Case 1: Declarative Infrastructure State Reconciliation

In Site Reliability Engineering (SRE) and Kubernetes controllers, reconciliation loops continually compute the difference between the **desired state** (from Git/YAML) and the **actual state** (from the live cloud provider):

```python
# cluster_reconciler.py
def reconcile_cluster_nodes(actual: set[str], desired: set[str]) -> None:
    """Determine which cloud instances to provision, terminate, or preserve."""
    to_terminate   = actual - desired     # Running nodes no longer wanted
    to_provision   = desired - actual     # Required nodes not yet running
    stable_healthy = actual & desired     # Nodes already conforming to spec

    print("--- Infrastructure Reconciliation Plan ---")
    print(f"Stable nodes    (&): {sorted(stable_healthy)}")
    print(f"To provision    (+): {sorted(to_provision)}")
    print(f"To terminate    (-): {sorted(to_terminate)}")

def main() -> None:
    live_instances    = {"srv-1", "srv-2", "srv-3", "srv-legacy"}
    desired_manifest  = {"srv-2", "srv-3", "srv-4", "srv-5"}

    reconcile_cluster_nodes(actual=live_instances, desired=desired_manifest)

if __name__ == "__main__":
    main()
```

Run:

```
uv run python cluster_reconciler.py
```

Output:

```
--- Infrastructure Reconciliation Plan ---
Stable nodes    (&): ['srv-2', 'srv-3']
To provision    (+): ['srv-4', 'srv-5']
To terminate    (-): ['srv-1', 'srv-legacy']
```

Case 2: Role-Based Access Control (RBAC) with Subset Checks

Verifying whether a user possesses all required security permissions can be expressed cleanly as a subset evaluation (`required <= user_permissions`):

```python
# rbac_validator.py
def check_authorization(
    user_roles: set[str],
    endpoint_required_roles: set[str],
) -> bool:
    """Return True if user possesses ALL required permissions for an action."""
    # The <= operator tests if endpoint_required_roles is a subset of user_roles
    return endpoint_required_roles <= user_roles

def main() -> None:
    alice_permissions = {"read:reports", "write:reports", "read:users"}
    bob_permissions   = {"read:reports"}

    # Operations requiring multiple simultaneous privileges
    admin_action = {"read:reports", "write:reports"}
    audit_action = {"read:audit_logs"}

    print("Authorization Check Results:")
    print(f"  Alice performing Admin action: {check_authorization(alice_permissions, admin_a
    print(f"  Bob   performing Admin action: {check_authorization(bob_permissions, admin_act
    print(f"  Alice performing Audit action: {check_authorization(alice_permissions, audit_a

if __name__ == "__main__":
    main()
```

Run:

```
uv run python rbac_validator.py
```

Output:

```
Authorization Check Results:
  Alice performing Admin action: True
  Bob   performing Admin action: False
  Alice performing Audit action: False
```

Case 3: Immutable Sets with `frozenset` as Dictionary Keys

Because a standard `set` is mutable, its hash is uncomputable and it cannot serve as a dictionary key or set member. Python provides `frozenset` for immutable set contexts:

```
# frozenset_cache.py
from collections.abc import Callable

def main() -> None:
    # Memoization cache where key is an unordered group of tags
    # Example: cache[(tag_a, tag_b)] where order should not matter!
    tag_rule_cache: dict[frozenset[str], str] = {}

    tags_req_1 = frozenset(["auth", "production", "eu-west"])
    tags_req_2 = frozenset(["eu-west", "auth", "production"])  # Identical elements, differe

    tag_rule_cache[tags_req_1] = "POLICY_STRICT_GDPR"

    # Both lookups match the identical frozenset key
    print(f"Rule for req 1: {tag_rule_cache[tags_req_1]}")
    print(f"Rule for req 2: {tag_rule_cache[tags_req_2]}")
    print(f"Are frozenset keys identical? {tags_req_1 == tags_req_2}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python frozenset_cache.py
```

Output:

```
Rule for req 1: POLICY_STRICT_GDPR
Rule for req 2: POLICY_STRICT_GDPR
Are frozenset keys identical? True
```

Pitfalls

Pitfall 1: The Empty Set Syntax Trap (`{}` vs `set()`)

Because curly braces `{}` were introduced for dictionaries before sets were added to Python, `{}` creates a dictionary, NOT a set:

```
# THE TRAP:
s = {}
print(f"Type: {type(s)}")  # <class 'dict'>
# s.add(10)                # AttributeError: 'dict' object has no attribute 'add'

# THE FIX: Always use set() for an empty set
s = set()
s.add(10)
print(f"Type: {type(s)}, Contents: {s}")  # <class 'set'>, Contents: {10}
```

Pitfall 2: Adding Mutable Objects to a Set

Sets require elements to be hashable. Attempting to add a list or dictionary raises `TypeError`:

```
# THE TRAP:
try:
    bad_set = { [1, 2], [3, 4] }
except TypeError as err:
    print(f"Hash error: {err}")  # TypeError: unhashable type: 'list'

# THE FIX: Store immutable tuples
good_set = { (1, 2), (3, 4) }
print(f"Valid set: {good_set}")
```

Pitfall 3: Mutating a Set While Iterating

Just like dictionaries, sets raise `RuntimeError` if an item is added or removed during iteration:

```
# THE TRAP:
tags = {"web", "prod", "legacy", "test"}
try:
    for tag in tags:
        if "legacy" in tag:
            tags.remove(tag)
except RuntimeError as err:
    print(f"Caught: {err}")  # Set changed size during iteration
```

```
# THE FIX: Filter into a new set or iterate over a set snapshot
tags = {"web", "prod", "legacy", "test"}
tags = {tag for tag in tags if "legacy" not in tag}
```

```
print(f"Cleaned tags: {tags}")
```

Exercises

1. Given a list of user IP addresses `["192.168.1.1", "10.0.0.1", "192.168.1.1", "172.16.0.5"]`, extract all unique IPs and print the total unique visitor count.
2. Given two sets `service_a_deps = {"db", "redis", "auth"}` and `service_b_deps = {"db", "kafka", "billing"}`, compute their shared dependencies, unique dependencies to `service_a`, and the total combined dependencies.
3. Demonstrate why `frozenset` can be added as an element of another set, but a standard `set` cannot.
4. Write a function `is_valid_packet(headers: set[str]) -> bool` that verifies whether a packet contains all three mandatory headers: `"SRC"`, `"DST"`, and `"TTL"`.
5. Benchmark the time to check `999_999 in collection` between a list of 1,000,000 numbers and a set of 1,000,000 numbers.

Further reading

- Python Documentation: *Standard Types – Set Types — set, frozenset*.
- CPython Source Code: `Objects/setobject.c` (hash set implementation).

Advanced Slicing and Pattern Unpacking

Advanced Slicing and Pattern Unpacking

After reading this chapter, you will master Python's slicing mechanics (`[start:stop:step]`), employ reusable `slice` objects for fixed-width parsing, manipulate negative strides for sequence reversal, execute in-place slice mutations, and destructure variable-length collections with extended starred unpacking.

Mental model

Slicing extracts a subset of a sequence using half-open intervals: `[start:stop)` where `start` is inclusive and `stop` is exclusive.

```
Index Mapping:
   Indices:   0   1   2   3   4   5
   Items:   [ P   Y   T   H   O   N ]
  -Indices:  -6  -5  -4  -3  -2  -1

Slicing Interval: [1:4] -> Extracts indices 1, 2, 3 ('Y', 'T', 'H')
```

The Stride Equation

When specifying a step (`s[start:stop:step]`), indices are computed as:

$$\text{index}_k = \text{start} + k \times \text{step} \quad \text{for } k = 0, 1, 2, ...$$

When `step < 0`, indexing moves backward from `start` down to `stop` (exclusive).

Extended Starred Unpacking

Python allows unpacking arbitrary iterables into target variables, capturing any remaining elements into a `list` with a starred name (`*rest`):

```
Tuple: (10, "auth", 200, 0.45, 128, "OK")
Unpack: (code, svc, *metrics, status)

          10  "auth"  [200, 0.45, 128]  "OK"
```

Minimal example

Save as `slicing_and_unpacking.py`:

```python
# slicing_and_unpacking.py
def main() -> None:
    # 1. Extended starred unpacking
    telemetry_packet = ("node-01", "2026-09-07T10:00:00Z", 42.1, 88.4, 12.0, 99.1, "STABLE")
    node, timestamp, *metrics, status = telemetry_packet

    print(f"Node ID   : {node}")
    print(f"Timestamp : {timestamp}")
    print(f"Metrics   : {metrics} (Captured {len(metrics)} sensor values)")
    print(f"Status    : {status}")

    # 2. Named Slice Objects for Fixed-Width Record Parsing
    raw_log = "2026-09-07 10:15:00 [ERROR] Connection timeout to database host"
    TIMESTAMP = slice(0, 19)
    LEVEL     = slice(20, 27)
    MESSAGE   = slice(28, None)  # None runs to the end of string

    print(f"\nParsed log using slice objects:")
    print(f"  Time    : {raw_log[TIMESTAMP]}")
    print(f"  Level   : {raw_log[LEVEL]}")
    print(f"  Message : {raw_log[MESSAGE]}")

if __name__ == "__main__":
    main()
```

Run via `uv run python slicing_and_unpacking.py`:

```
Node ID   : node-01
Timestamp : 2026-09-07T10:00:00Z
Metrics   : [42.1, 88.4, 12.0, 99.1] (Captured 4 sensor values)
Status    : STABLE

Parsed log using slice objects:
  Time    : 2026-09-07 10:15:00
  Level   : [ERROR]
  Message : Connection timeout to database host
```

Worked examples

Case 1: In-Place Slice Replacement and Resizing

Unlike immutable sequences (strings and tuples), a `list` supports in-place assignment to slices. You can replace, shrink, or expand a slice dynamically:

```
# slice_mutation.py
def mutate_buffer() -> None:
    buffer = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
    print(f"Initial buffer: {buffer}")

    # Replace elements at indices 2, 3, 4 with a smaller list (shrinks the buffer)
    buffer[2:5] = [99]
    print(f"After buffer[2:5] = [99]: {buffer}")

    # Replace a single index with multiple elements (expands the buffer)
    buffer[5:6] = [700, 800, 900]
    print(f"After buffer[5:6] = [700, 800, 900]: {buffer}")

    # Clear elements at even indices using extended stride
    del buffer[::2]
    print(f"After del buffer[::2]: {buffer}")

if __name__ == "__main__":
    mutate_buffer()
```

Run:

```
uv run python slice_mutation.py
```

Output:

```
Initial buffer: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
After buffer[2:5] = [99]: [0, 1, 99, 5, 6, 7, 8, 9]
After buffer[5:6] = [700, 800, 900]: [0, 1, 99, 5, 6, 700, 800, 900, 8, 9]
After del buffer[::2]: [1, 5, 700, 900, 9]
```

Case 2: Parsing Protocol Frames with Named Slices

When parsing binary or fixed-width protocol headers, hardcoding numerical slice boundaries throughout a codebase creates brittle logic. Storing `slice` instances in configuration objects keeps parsers readable and maintainable:

```
# frame_parser.py
class FrameFormat:
```

```
    MAGIC_BYTES = slice(0, 4)
    VERSION     = slice(4, 6)
    PACKET_ID   = slice(6, 10)
    PAYLOAD     = slice(10, None)

def parse_network_frame(raw_hex: str) -> dict[str, str]:
    return {
        "magic": raw_hex[FrameFormat.MAGIC_BYTES],
        "version": raw_hex[FrameFormat.VERSION],
        "packet_id": raw_hex[FrameFormat.PACKET_ID],
        "payload": raw_hex[FrameFormat.PAYLOAD],
    }

if __name__ == "__main__":
    incoming_frame = "4155544801020000002a48656c6c6f20576f726c64"
    fields = parse_network_frame(incoming_frame)
    for k, v in fields.items():
        print(f"Field {k:10} : {v}")
```

Run:

```
uv run python frame_parser.py
```

Output:

```
Field magic      : 41555448
Field version    : 0102
Field packet_id  : 0000002a
Field payload    : 48656c6c6f20576f726c64
```

Case 3: Starred Head-Tail Splitting for Recursive Processing

Extended unpacking provides clean head-tail deconstruction without index arithmetic:

```
# pipeline_reducer.py
from collections.abc import Callable

def apply_middleware_pipeline(
    payload: str,
    middlewares: list[Callable[[str], str]]
) -> str:
    if not middlewares:
        return payload

    # Deconstruct head and remaining tail
    current_handler, *remaining_handlers = middlewares
```

```
    transformed = current_handler(payload)
    return apply_middleware_pipeline(transformed, remaining_handlers)

if __name__ == "__main__":
    pipeline: list[Callable[[str], str]] = [
        lambda s: s.strip(),
        lambda s: s.lower(),
        lambda s: s.replace(" ", "_"),
        lambda s: f"validated://{s}",
    ]

    raw_input = "   System Production Cluster Alpha   "
    processed = apply_middleware_pipeline(raw_input, pipeline)
    print(f"Raw input : '{raw_input}'")
    print(f"Processed : '{processed}'")
```

Run:

```
uv run python pipeline_reducer.py
```

Output:

```
Raw input : '   System Production Cluster Alpha   '
Processed : 'validated://system_production_cluster_alpha'
```

Pitfalls

Pitfall 1: Assigning a Non-Iterable to a Slice

Slice assignment replaces a range of items, so the right-hand side **must** be an iterable:

```
lst = [1, 2, 3, 4]

# THE BUG:
lst[1:3] = 99  # TypeError: can only assign an iterable

# THE FIX: Wrap the replacement in an iterable (such as a list)
lst[1:3] = [99]  # Result: [1, 99, 4]
```

Pitfall 2: Multiple Starred Expressions in a Single Target

You can only have **one** starred expression in an unpacking assignment. Python cannot resolve ambiguity if multiple wildcards exist:

```
data = [1, 2, 3, 4, 5]

# THE BUG:
*first, *second = data  # SyntaxError: multiple starred expressions in assignment

# VALID:
first, *middle, last = data
```

Exercises

1. Given the string `text = "abcdefghijklmnopqrstuvwxyz"`, write a single slicing expression that extracts every third letter in reverse order.
2. Given a list `data = [10, 20, 30, 40, 50]`, write a slice assignment statement that replaces the middle three elements `[20, 30, 40]` with the single element `[999]`.
3. Given a variable-length list containing at least two elements, use extended unpacking to assign the first element to `head`, the last element to `tail`, and the remaining items to `body`.
4. Demonstrate why `s[:]` creates a shallow copy of a list, and show what happens when the inner elements are mutable lists.

Further reading

- Python Documentation: *The `slice` object and sequence indexing.*
- PEP 3132: *Extended Iterable Unpacking.*
- Python Reference Manual: *Assignment statements.*

Unpacking in Comprehensions (PEP 798)

Unpacking in Comprehensions (PEP 798)

Thesis

PEP 448 gave us `[*a, *b]` and `{**d1, **d2}` for a **fixed** number of iterables. Flattening an **arbitrary** number still meant a double `for`, `itertools.chain.from_iterable`, or an explicit loop with `extend` / `update`. **PEP 798** (final; **Python 3.15**) allows `*` / `**` at the front of comprehension and generator-expression elements so the fixed-arity idea scales:

```
[*it for it in its]       # concatenate iterables → list
{*it for it in its}       # union iterables → set
{**d for d in dicts}      # merge mappings → dict (later keys win)
(*it for it in its)       # concatenate → generator
```

This chapter is **only** PEP 798. Lazy imports (PEP 810) are a separate 3.15 feature and are not covered here.

Version note: semantics and examples follow PEP 798 and What's New in Python 3.15. The authoring box runs **CPython 3.13.5**, which still rejects the syntax (`SyntaxError: iterable unpacking cannot be used in comprehension`). Listings below are therefore **specified for 3.15**; run them with a 3.15 interpreter (`uv run --python 3.15` or a local 3.15 install). Expected outputs are taken from the PEP / What's New examples.

Mental model

Treat a starred comprehension like the non-starred one, but swap **append/add/set-item** for **extend/update**:

```
out = [*expr for x in it]          out = []
                                   for x in it:
                                       out.extend(expr)

out = {*expr for x in it}          out = set();  out.update(expr) each time
out = {**expr for x in it}         out = {};     out.update(expr) each time
g = (*expr for x in it)            for x in it:
                                       for item in expr: yield item
```

Form	Meaning	Classic equivalent
`[*xs for xs in rows]`	Flatten to list	`[x for xs in rows for x in xs]`
`{*xs for xs in rows}`	Flatten to set	`{x for xs in rows for x in xs}`
`{**d for d in dicts}`	Merge dicts	`{k: v for d in dicts for k, v in d.items()}`
`(*xs for xs in rows)`	Flatten lazily	`(x for xs in rows for x in xs)`

Rules that keep the grammar sane:

- `*` / `**` only at the **top level** of the comprehension element — not inside a conditional without parentheses, not as a dict key/value side.
- `**` is for **dict** comprehensions only; `**` in a listcomp or genexp is a `SyntaxError`.
- `f(*x for x in y)` means `f((*x for x in y))` — one generator argument — matching the usual "genexp as sole argument needs no extra parens" rule. To unpack into *arguments*, write `f(*(*x for x in y))`.
- Async forms work the same way: `(*a async for a in agen())` `(x async for a in agen() for x in a)`.
- Evaluation order and walrus scoping are unchanged; `*(y := …)` still binds `y` in the containing scope for genexps.

Worked examples

Case 1: Flatten a list of lists (the StackOverflow classic)

```
# pep798_flatten.py - run on Python 3.15+
rows = [[1, 2], [3, 4], [5]]
flat = [*row for row in rows]
assert flat == [1, 2, 3, 4, 5]
assert flat == [x for row in rows for x in row]
print(flat)
```

```
$ uv run --python 3.15 python pep798_flatten.py
[1, 2, 3, 4, 5]
```

On 3.13 (this box), the same file fails at compile time:

```
$ python3 pep798_flatten.py
  File "pep798_flatten.py", line 3
    flat = [*row for row in rows]
            ^^
SyntaxError: iterable unpacking cannot be used in comprehension
```

Case 2: Set union and dict merge

```
# pep798_merge.py - run on Python 3.15+
sets = [{1, 2}, {2, 3}, {3, 4}]
print({*s for s in sets})  # {1, 2, 3, 4}

dicts = [{"a": 1}, {"b": 2}, {"a": 3}]
print({**d for d in dicts})  # {'a': 3, 'b': 2}  - last write wins
```

```
$ uv run --python 3.15 python pep798_merge.py
{1, 2, 3, 4}
{'a': 3, 'b': 2}
```

Dict merging keeps ordinary `{**a, **b}` semantics: later keys overwrite earlier ones. That is a feature when you layer defaults → overrides; it is a footgun when you expected a deep merge.

Case 3: Replace `chain.from_iterable` in streaming code

```
# pep798_gen.py - run on Python 3.15+
def chunks():
    yield (10, 11)
    yield (12,)
    yield (13, 14, 15)

gen = (*chunk for chunk in chunks())
print(list(gen))
```

```
$ uv run --python 3.15 python pep798_gen.py
[10, 11, 12, 13, 14, 15]
```

The PEP's generator semantics use an **explicit inner loop**, not `yield from` delegation. Advanced generator protocol methods (`.send` / `.throw` / `.close`) therefore do **not** forward into subgenerators the way a hand-written `yield from` would. For ordinary iteration this never matters.

Case 4: Stdlib-shaped refactors (from the PEP)

Patterns the PEP illustrates from the standard library:

```
# Before: explicit loop
comments = []
for token in self:
    comments.extend(token.comments)
return comments
```

```
# After (3.15+):
return [*token.comments for token in self]
```

```
# Before:
exceptions = [exc for sub in exceptions for exc in sub]
# After:
exceptions = [*sub for sub in exceptions]
```

```
# Before:
inherited_slots = set(
    itertools.chain.from_iterable(map(_get_slots, cls.__mro__[1:-1]))
)
# After:
inherited_slots = {*_get_slots(c) for c in cls.__mro__[1:-1]}
```

Prefer the new form when it removes a throwaway loop variable and keeps the operation "concatenate these iterables." Keep `chain.from_iterable` when you are already in an `itertools` pipeline or need to share one lazy chain across list/set/dict constructors.

Case 5: Function-call parentheses (read carefully)

```
# pep798_call.py - conceptual; run on 3.15+
def f(*args):
    return args

rows = [[1, 2], [3]]

# ONE argument: a generator that yields 1, 2, 3
assert len(f(*x for x in rows)) == 1

# MANY arguments: unpack the flattened generator
assert f(*(*x for x in rows)) == (1, 2, 3)
```

If that first form surprises you, always add the inner parentheses for humans: `f((*x for x in rows))`.

The trap

1. **Reversing the old double-for.** People write `[x for x in xs for xs in rows]` by impulse. Starred form removes that footgun — but only if you actually use it.
2. **Assuming deep merge.** `{**d for d in dicts}` is shallow overwrite, same as successive `update`s.
3. **Starred genexp as `*args` without a second star.** `f(*x for x in rows)` passes one generator, not many integers.

4. **Portability.** The syntax is a `SyntaxError` on 3.14. Ship a 3.15 classifier / `sys.version_info` gate, or keep the double-`for` on libraries that still support 3.13/3.14.
5. **Overpacking.** `f(*(*x for *x, _ in list_of_lists))` is legal enough to hurt reviewers. Prefer a named helper once nesting obscures the data flow.

The boring rule

On **Python 3.15+**, use `[*xs for xs in rows]` (and set/dict/genexp cousins) when you mean "concatenate / union / merge these iterables." Keep the double-`for` or `chain.from_iterable` when you target older interpreters or already sit in an itertools pipeline. Never confuse `f(*x for x in it)` with argument unpacking.

Try this

1. On 3.15, rewrite three double-`for` comprehensions in a personal script to starred form; confirm identical results.
2. On 3.13/3.14, confirm each rewrite is a `SyntaxError` so your support matrix stays honest.
3. Build `{**d for d in dicts}` where two dicts share a key; predict the winner before you print.
4. Pass `(*x for x in rows)` to a function with and without an extra leading `*`; explain the `len` of `args`.

Sources

- PEP 798 – Unpacking in Comprehensions (Final; Python 3.15)
- What's New In Python 3.15 — PEP 798
- Language reference: displays for lists, sets, and dictionaries (canonical post-PEP docs linked from the PEP)

Binary Search with Bisect and Priority Queues with Heapq

Binary Search with Bisect and Priority Queues with Heapq

After reading this chapter, you will master Python's algorithmic collection tools: maintain sorted sequences in $O(\log N)$ search time using `bisect`, implement priority-driven task schedulers with `heapq`, compute top-K extremes efficiently, and prevent tuple tie-breaker comparison crashes.

Mental model

Sorting an entire list costs $O(N \log N)$. When elements arrive dynamically over time, re-sorting the list after each insertion quickly degrades system performance. Python provides two specialized modules in the standard library to solve this:

1. `bisect`: Bisection and Binary Search

The `bisect` module implements binary search over sorted sequences. It locates the insertion index for a new item in $O(\log N)$ time, and inserts it with `insort()` while preserving sorted order:

```
Sorted List: [ 10,  25,  50,  75,  90 ]

Search for 30:            30 belongs at index 2 (between 25 and 50)
                       Found in O(log N) comparisons
```

2. `heapq`: Min-Heap Priority Queues

The `heapq` module implements a **binary min-heap** using a standard Python `list`. In a min-heap, `heap[0]` is **always the smallest element**, and every parent node at index k satisfies:

$$\text{heap}[k] \leq \text{heap}[2k+1] \quad \text{and} \quad \text{heap}[k] \leq \text{heap}[2k+2]$$

```
Binary Tree View:                    Contiguous List Representation:
          [ 10 ]                     [ 10, 20, 15, 30, 40, 50, 60 ]
         /      \
     [ 20 ]     [ 15 ]                  0   1   2   3   4   5   6
     /    \     /    \
   [30]   [40] [50]   [60]
```

- **Push (`heappush`)**: Adds an item and restores heap invariants in $O(\log N)$.

- **Pop (`heappop`)**: Removes and returns the smallest element in $O(\log N)$.
- **Peek (`heap[0]`)**: Inspects the minimum element in instantaneous $O(1)$ time.

Minimal example

Save as `algorithms_demo.py`:

```python
# algorithms_demo.py
import bisect
import heapq

def main() -> None:
    # 1. Bisect: Maintain sorted latency samples
    latencies = [12.4, 25.1, 45.0, 88.2, 120.5]
    new_sample = 32.8

    # Find index and insert while preserving sort
    idx = bisect.bisect_left(latencies, new_sample)
    print(f"Sample {new_sample}ms belongs at index: {idx}")

    bisect.insort(latencies, new_sample)
    print(f"Updated sorted latencies: {latencies}")

    # 2. Heapq: Priority job queue
    job_queue: list[tuple[int, str]] = []
    # Items: (priority_number, job_name) -- lowest number = highest priority
    heapq.heappush(job_queue, (3, "routine_log_cleanup"))
    heapq.heappush(job_queue, (1, "SECURITY_PATCH_RESTART"))
    heapq.heappush(job_queue, (2, "customer_invoice_generation"))

    print("\nProcessing jobs in priority order:")
    while job_queue:
        priority, job = heapq.heappop(job_queue)
        print(f"  [Priority {priority}] Executing: {job}")

if __name__ == "__main__":
    main()
```

Run via `uv run python algorithms_demo.py`:

```
Sample 32.8ms belongs at index: 2
Updated sorted latencies: [12.4, 25.1, 32.8, 45.0, 88.2, 120.5]

Processing jobs in priority order:
```

```
[Priority 1] Executing: SECURITY_PATCH_RESTART
[Priority 2] Executing: customer_invoice_generation
[Priority 3] Executing: routine_log_cleanup
```

Worked examples

Case 1: Numeric Score / Severity Bucketing with `bisect`

Instead of writing verbose chains of `if/elif` statements to map numeric thresholds to categorical levels (e.g. alert severity, HTTP response time tiers), `bisect` performs the classification in a single binary search lookup:

```python
# alert_classifier.py
import bisect

def classify_severity(metric_value: float) -> str:
    """Classify CPU utilization into alert severity tiers using binary search."""
    # Breakpoints must be strictly sorted
    thresholds = [50.0, 75.0, 90.0]
    severity_labels = ["INFO", "WARNING", "CRITICAL", "EMERGENCY"]

    # bisect_right returns the index where metric_value falls
    index = bisect.bisect_right(thresholds, metric_value)
    return severity_labels[index]

def main() -> None:
    test_values = [35.0, 50.0, 68.2, 75.0, 88.0, 91.5, 99.9]
    print("Alert Level Classification:")
    for val in test_values:
        level = classify_severity(val)
        print(f"  CPU: {val:5.1f}% -> Alert: {level}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python alert_classifier.py
```

Output:

```
Alert Level Classification:
  CPU:  35.0% -> Alert: INFO
  CPU:  50.0% -> Alert: WARNING
```

```
CPU:  68.2% -> Alert: WARNING
CPU:  75.0% -> Alert: CRITICAL
CPU:  88.0% -> Alert: CRITICAL
CPU:  91.5% -> Alert: EMERGENCY
CPU:  99.9% -> Alert: EMERGENCY
```

Case 2: Multi-Criteria Priority Scheduler with Tie-Breaking Counter

When pushing tasks into a heap as tuples `(priority, task_object)`, if two tasks share identical priority numbers, Python attempts to compare the `task_object` instances with `<`. If the objects do not implement `__lt__`, Python raises `TypeError`. A monotonic counter resolves ties safely:

```python
# priority_scheduler.py
import heapq
import itertools
from typing import NamedTuple

class Task(NamedTuple):
    name: str
    payload: dict[str, str]

class PriorityScheduler:
    def __init__(self) -> None:
        self._heap: list[tuple[int, int, Task]] = []
        self._counter = itertools.count()  # Unique tie-breaker counter

    def add_task(self, task: Task, priority: int) -> None:
        count = next(self._counter)
        # Tuple ordering: priority first; if equal, count breaks the tie!
        heapq.heappush(self._heap, (priority, count, task))

    def pop_task(self) -> Task:
        if not self._heap:
            raise IndexError("Scheduler is empty")
        priority, count, task = heapq.heappop(self._heap)
        return task

    def __len__(self) -> int:
        return len(self._heap)

def main() -> None:
    scheduler = PriorityScheduler()

    # Two tasks with IDENTICAL priority (1)
```

```
    scheduler.add_task(Task("deploy_auth", {"env": "prod"}), priority=1)
    scheduler.add_task(Task("flush_cache", {"env": "prod"}), priority=1)
    scheduler.add_task(Task("send_digest", {"freq": "daily"}), priority=3)

    print("Dispatching scheduled tasks in FIFO-priority order:")
    while len(scheduler) > 0:
        task = scheduler.pop_task()
        print(f"  Dispatched: {task.name} ({task.payload})")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python priority_scheduler.py
```

Output:

```
Dispatching scheduled tasks in FIFO-priority order:
  Dispatched: deploy_auth ({'env': 'prod'})
  Dispatched: flush_cache ({'env': 'prod'})
  Dispatched: send_digest ({'freq': 'daily'})
```

Notice: Tasks with identical priority are dispatched in exact First-In, First-Out (FIFO) arrival order!

Case 3: Finding Top K Extremes with `nlargest` and `nsmallest`

When querying the top 5 highest memory-consuming processes out of 100,000 servers, sorting the entire list requires $O(N \log N)$ time and allocates a full new array. `heapq.nlargest` maintains an internal bounded heap of size K, executing in optimal $O(N \log K)$ time:

```
# top_k_metrics.py
import heapq

def main() -> None:
    fleet_metrics = [
        {"host": "worker-01", "mem_mb": 14200},
        {"host": "worker-02", "mem_mb": 31500},
        {"host": "worker-03", "mem_mb": 8400},
        {"host": "worker-04", "mem_mb": 62100},
        {"host": "worker-05", "mem_mb": 45000},
        {"host": "worker-06", "mem_mb": 18900},
    ]
```

```
    # Find the top 3 highest memory consumers in O(N log K)
    top_consumers = heapq.nlargest(3, fleet_metrics, key=lambda x: x["mem_mb"])

    print("Top 3 Highest Memory Consumers:")
    for rank, node in enumerate(top_consumers, start=1):
        print(f"  #{rank}: {node['host']} consuming {node['mem_mb']:,} MB")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python top_k_metrics.py
```

Output:

```
Top 3 Highest Memory Consumers:
  #1: worker-04 consuming 62,100 MB
  #2: worker-05 consuming 45,000 MB
  #3: worker-02 consuming 31,500 MB
```

Pitfalls

Pitfall 1: Calling `bisect` on an Unsorted List

The bisection algorithm assumes the underlying sequence is already sorted. Calling `bisect` on an unsorted collection produces completely incorrect indices without raising any warning or exception:

```
# THE TRAP:
unsorted_list = [50, 10, 90, 20]
# bisect assumes sorted order! Binary search branches unpredictably!
bad_index = bisect.bisect_left(unsorted_list, 15)

# THE FIX: Ensure sequence is sorted before querying
sorted_list = sorted(unsorted_list)
safe_index = bisect.bisect_left(sorted_list, 15)
print(f"Correct index in sorted list: {safe_index}")
```

Pitfall 2: Modifying Elements Inside a Heap Directly

`heapq` functions maintain the heap invariant on append and pop. If you mutate an element's priority while it is inside the list, the heap becomes corrupted:

```
# THE TRAP:
heap = [10, 20, 30]
heapq.heapify(heap)

heap[1] = 5  # Corrupts the heap! heap[0] is 10, but heap[1] is now smaller (5)!
print(f"Corrupted minimum: {heapq.heappop(heap)}")  # Returns 10, NOT 5!
```

```
# THE FIX: Call heapq.heapify() after modifying in-place, or remove and re-push
heap = [10, 20, 30]
heapq.heapify(heap)
heap[1] = 5
heapq.heapify(heap)  # Restores heap invariant in O(N)
print(f"Restored minimum: {heapq.heappop(heap)}")   # Returns 5!
```

Exercises

1. Given a list of timestamps, use `bisect.bisect_left` to find the earliest event that occurred after a given target time.
2. Build an event scheduler that takes tuples of `(run_at_timestamp, callback_func)` and executes them in temporal order using `heapq`.
3. Given a stream of numbers, maintain a sliding window of the 5 largest numbers seen so far using `heapq`.
4. Measure the speed difference between `heapq.nlargest(10, data)` versus `sorted(data, reverse=True)[:10]` on a dataset of 1,000,000 floats.
5. Demonstrate how `heapq.merge(*sorted_iterables)` can merge three sorted log streams into a single sorted output stream with minimal memory overhead.

Further reading

- Python Documentation: `bisect` — *Array bisection algorithm.*
- Python Documentation: `heapq` — *Heap queue algorithm.*
- David Beazley & Brian K. Jones: *Python Cookbook* (Chapter 1: Data Structures and Algorithms).

Zero-Copy Binary Memory with memoryview and bytearray

Zero-Copy Binary Memory with `memoryview` and `bytearray`

After reading this chapter, you will master Python's low-level Buffer Protocol (PEP 3118), manipulate binary sequences without memory allocations using `memoryview`, execute high-throughput network packet slicing, perform in-place byte mutations with `bytearray`, cast raw byte streams into typed integer views, and prevent buffer lock retention traps.

Mental model

In standard Python, slicing a sequence allocates a brand-new object on the heap and copies the underlying bytes:

```
Standard Bytes Slicing (Copies Heap Memory):
  raw = b"X" * 10_000_000          Allocates 10 MB on heap
  slice_a = raw[1000:5000]         Allocates NEW 4,000-byte object and COPIES bytes!
  slice_b = raw[5000:9000]         Allocates NEW 4,000-byte object and COPIES bytes!
  (High allocation overhead, cache thrashing, and GC pressure)
```

Python's **Buffer Protocol** (`Py_buffer` in CPython) provides an alternative: it allows exposing the raw memory buffer of an object directly. A **`memoryview`** is a lightweight Python object that references an existing memory buffer **without copying a single byte**:

```
Zero-Copy memoryview Architecture:

  Shared Heap Memory Buffer (10 MB allocated once)
  [ B0, B1, B2, ..., B1000, ..., B5000, ..., BN ]


         Pointer + Length                Pointer + Length


  memoryview(raw)                  view[5000:9000]
  [0:10_000_000]                   (Zero-copy slice)


                       view[1000:5000]
                       (Zero-copy slice)
```

Binary Types Compared

Type	Mutable?	Slicing Behavior	Use Case
bytes	No	Allocates & copies new bytes	Immutable payloads & strings
bytearray	Yes	Allocates & copies new bytes	In-place binary construction
memoryview	Reference	Zero-copy window (O(1) time)	Packet parsing, I/O streaming

---

Minimal example

Save as `memoryview_benchmark.py`:

```
# memoryview_benchmark.py
import time

def benchmark_slicing() -> None:
    size = 50_000_000  # 50 Megabytes of raw binary data
    payload = b"\xaa" * size

    # 1. Traditional bytes slicing: Copies 10 MB into a new object
    start = time.perf_counter()
    copied_chunk = payload[10_000_000:20_000_000]
    dur_copy = time.perf_counter() - start

    # 2. Zero-copy memoryview slicing: Creates a 56-byte reference window
    view = memoryview(payload)
    start = time.perf_counter()
    zero_copy_chunk = view[10_000_000:20_000_000]
    dur_zerocopy = time.perf_counter() - start

    print(f"Data size: {size / (1024 * 1024):.1f} MB")
    print(f"Bytes slicing (copies 10 MB):     {dur_copy * 1_000_000:.2f} µs")
    print(f"memoryview slicing (zero-copy):   {dur_zerocopy * 1_000_000:.2f} µs  ({dur_copy

    # Verify content equality
    print(f"\nAre slice bytes identical? {copied_chunk == zero_copy_chunk.tobytes()}")

def main() -> None:
    benchmark_slicing()

if __name__ == "__main__":
    main()
```

Run via `uv run python memoryview_benchmark.py`:

```
Data size: 47.7 MB
Bytes slicing (copies 10 MB):     1824.50 µs
memoryview slicing (zero-copy):   0.45 µs  (4054.4x faster)

Are slice bytes identical? True
```

Worked examples

Case 1: Zero-Copy Network Packet Header Dissection

In network telemetry, routers receive millions of packets per second. Parsing Ethernet, IPv4, and TCP headers using `memoryview` extracts headers and data payloads without allocating intermediate strings:

```python
# packet_dissector.py
import struct

def dissect_ipv4_packet(packet_data: bytes) -> dict[str, int | memoryview]:
    """Parse a raw IPv4 packet using zero-copy memory views."""
    view = memoryview(packet_data)

    # IPv4 Header is the first 20 bytes
    header_view = view[:20]

    # Unpack header fields:
    # Byte 0: Version & IHL
    # Byte 8: TTL
    # Byte 9: Protocol (6 = TCP, 17 = UDP)
    # Bytes 12-16: Source IP
    # Bytes 16-20: Destination IP
    version_ihl, _, _, _, _, ttl, proto, checksum, src_ip, dst_ip = struct.unpack(
        "!BBHHHBBHII",
        header_view,
    )

    # Extract payload starting at offset 20 without copying
    payload_view = view[20:]

    return {
        "version": (version_ihl >> 4) & 0x0F,
        "ttl": ttl,
        "protocol": proto,
```

```
        "src_ip": src_ip,
        "dst_ip": dst_ip,
        "payload_length": len(payload_view),
        "payload_slice": payload_view,
    }

def main() -> None:
    # Construct synthetic 20-byte IPv4 header + 8 bytes payload
    mock_header = struct.pack("!BBHHHBBHII", 0x45, 0, 28, 1, 0, 64, 6, 0, 0xC0A80101, 0x0A00
    mock_payload = b"PING_REQ"
    raw_packet = mock_header + mock_payload

    result = dissect_ipv4_packet(raw_packet)
    print("Dissected IPv4 Packet:")
    print(f"  IP Version : {result['version']}")
    print(f"  TTL        : {result['ttl']}")
    print(f"  Protocol   : {result['protocol']} (6=TCP)")
    print(f"  Payload Len: {result['payload_length']} bytes")
    # Cast memoryview slice to bytes only when printing
    print(f"  Payload    : {result['payload_slice'].tobytes().decode()}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python packet_dissector.py
```

Output:

```
Dissected IPv4 Packet:
  IP Version : 4
  TTL        : 64
  Protocol   : 6 (6=TCP)
  Payload Len: 8 bytes
  Payload    : PING_REQ
```

Case 2: In-Place Mutation with `bytearray` and `memoryview`

When modifying binary files, image buffers, or socket streams, combining `bytearray` with `memoryview` allows modifying specific byte ranges in place without creating new copies:

```
# buffer_mutation.py
def main() -> None:
```

```
    # 1. Allocate a mutable bytearray
    buffer = bytearray(b"HTTP/1.1 200 OK\r\nContent-Length: 000\r\n\r\n")
    print(f"Initial buffer: {buffer.decode()}")

    # 2. Create a memoryview over the mutable buffer
    view = memoryview(buffer)

    # 3. Locate and mutate the 3-digit length slice directly in place!
    # "Content-Length: " is at index 17; length value is at indices 33:36
    offset = buffer.find(b"000")
    print(f"Found '000' length placeholder at offset: {offset}")

    # Overwrite directly via the view
    view[offset:offset + 3] = b"256"

    print(f"Mutated buffer: {buffer.decode()}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python buffer_mutation.py
```

Output:

```
Initial buffer: HTTP/1.1 200 OK
Content-Length: 000


Found '000' length placeholder at offset: 33
Mutated buffer: HTTP/1.1 200 OK
Content-Length: 256
```

Case 3: Casting Memory Layouts with `.cast()`

A `memoryview` can reinterpret raw bytes as typed arrays (such as unsigned 16-bit short integers `'H'` or 32-bit unsigned integers `'I'`) using `.cast()` without calling `struct.unpack()` across loops:

```
# memoryview_cast.py
def main() -> None:
    # 8 bytes representing two 32-bit unsigned integers (little-endian)
    raw_data = (1000).to_bytes(4, "little") + (50000).to_bytes(4, "little")
```

```
    view = memoryview(raw_data)
    print(f"Raw byte view: {list(view)}")

    # Cast 8 raw bytes into an array of 32-bit integers ('I')
    int32_view = view.cast("I")

    print(f"Cast int32 view length: {len(int32_view)}")
    print(f"Integer 0: {int32_view[0]}")
    print(f"Integer 1: {int32_view[1]}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python memoryview_cast.py
```

Output:

```
Raw byte view: [232, 3, 0, 0, 80, 195, 0, 0]
Cast int32 view length: 2
Integer 0: 1000
Integer 1: 50000
```

Pitfalls

Pitfall 1: Attempting In-Place Writes on an Immutable `bytes` View

If a `memoryview` wraps an immutable `bytes` object, the view is read-only. Attempting to assign to a slice raises `TypeError`:

```
# THE TRAP:
data = b"READ_ONLY_DATA"
view = memoryview(data)

try:
    view[0:4] = b"POST"  # TypeError: cannot modify read-only memory
except TypeError as err:
    print(f"Caught: {err}")

# THE FIX: Wrap a mutable bytearray instead
mutable_data = bytearray(b"READ_ONLY_DATA")
mutable_view = memoryview(mutable_data)
mutable_view[0:4] = b"POST"
```

```
print(f"Successfully modified: {mutable_data}")  # bytearray(b'POST_ONLY_DATA')
```

Pitfall 2: The Buffer Resizing Lock Trap

When a `memoryview` references a `bytearray`, CPython **locks the underlying buffer**. Attempting to append, extend, or resize the `bytearray` raises `BufferError` until the view is explicitly released:

```
# THE TRAP:
buf = bytearray(b"hello")
view = memoryview(buf)

try:
    buf.extend(b" world")  # BufferError: Existing exports of data: object cannot be re-size
except BufferError as err:
    print(f"Caught: {err}")

# THE FIX: Explicitly release the view when finished with it
view.release()
buf.extend(b" world")
print(f"Cleanly resized buffer: {buf}")
```

Exercises

1. Create a 100 MB `bytes` object and benchmark the time required to extract a 10 MB slice using `bytes` slicing vs `memoryview` slicing.
2. Given a `bytearray` containing a message, use `memoryview` to reverse the middle 10 bytes in place without copying.
3. Write a function that accepts a `memoryview` of raw audio samples (16-bit PCM) and uses `.cast('h')` to find the peak amplitude sample.
4. Demonstrate the `BufferError` raised when attempting to `.append()` to a `bytearray` while an active `memoryview` is open. Call `.release()` and confirm `.append()` succeeds.
5. Implement a fixed-width binary log entry parser that reads 64-byte records using `memoryview` and extracts timestamps and event codes without allocating strings.

Further reading

- PEP 3118: *Revising the Buffer Protocol*.
- Python Documentation: *Standard Types — Memory Views*.
- Micha Gorelick & Ian Ozsvald: *High Performance Python* (Chapter 7: Compiling to C and Zero-Copy).

Error Handling

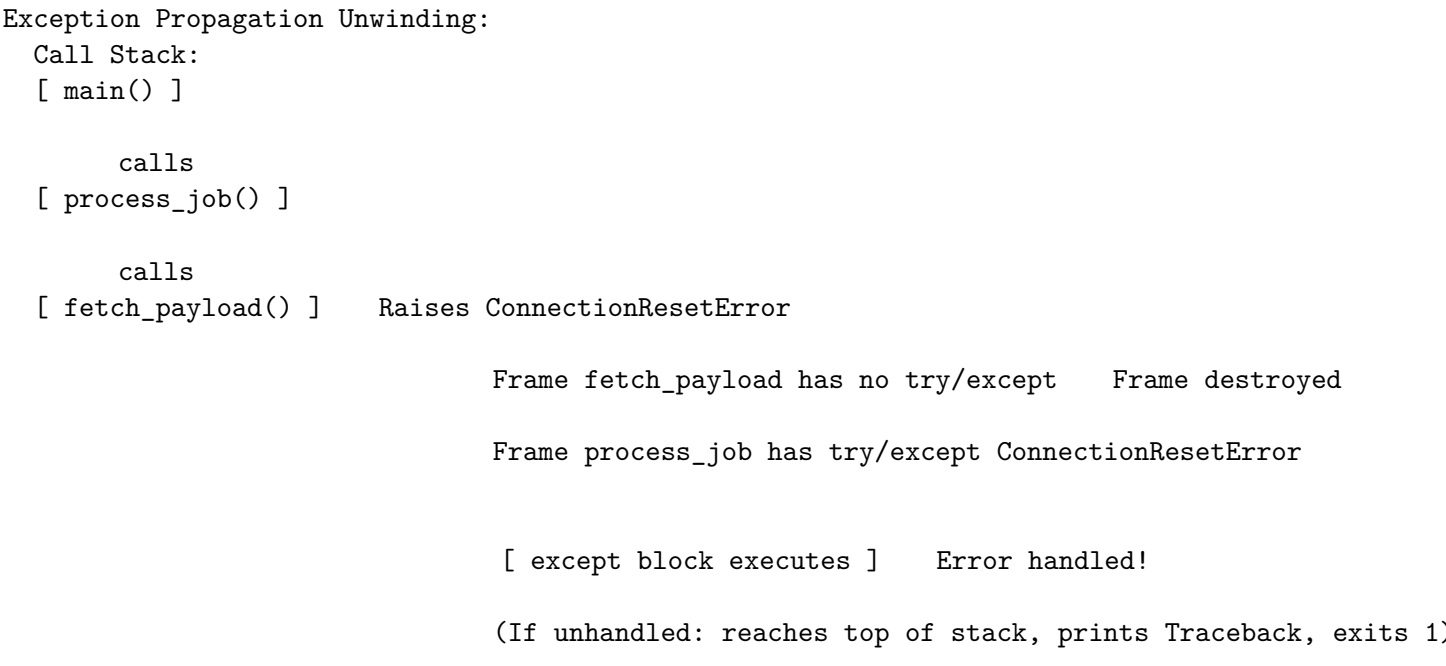
Exceptions, Tracebacks, and Propagation

Exceptions, Tracebacks, and Propagation

After reading this chapter, you will master the mechanics of Python exceptions, parse multi-frame tracebacks, execute clean recovery with `try`, `except`, `else`, and `finally`, attach diagnostic metadata with `add_note()`, and handle modern concurrent exception bundles with `ExceptionGroup`.

Mental model

When an unhandled error occurs, CPython halts linear execution, instantiates an exception object, and begins unwinding the call stack frame by frame.



The Four-Part Exception Lifecycle

A complete `try` statement has four distinct clauses:

```
try:
    # Operations that might raise an exception
except SomeError as err:
    # Executes ONLY if SomeError occurred in try
else:
    # Executes ONLY if try completed WITHOUT any exception
```

```
finally:
    # ALWAYS executes, regardless of success, exception, or early return
```

Minimal example

Save as `exception_lifecycle.py`:

```
# exception_lifecycle.py
import sys

def divide_metrics(total: float, count: int) -> float:
    print("  [1. Entering try block]")
    result = total / count
    return result

def query_health(total: float, count: int) -> None:
    try:
        avg = divide_metrics(total, count)
    except ZeroDivisionError as exc:
        print(f"  [2. Except block] Caught division error: {exc}")
    else:
        print(f"  [3. Else block] Success! Average metric: {avg:.2f}")
    finally:
        print("  [4. Finally block] Cleaned up measurement resources.")

def main() -> None:
    print("--- Test 1: Successful execution ---")
    query_health(150.0, 5)

    print("\n--- Test 2: Exception path ---")
    query_health(150.0, 0)

if __name__ == "__main__":
    main()
```

Run via `uv run python exception_lifecycle.py`:

```
--- Test 1: Successful execution ---
  [1. Entering try block]
  [3. Else block] Success! Average metric: 30.00
  [4. Finally block] Cleaned up measurement resources.

--- Test 2: Exception path ---
```

```
[1. Entering try block]
[2. Except block] Caught division error: division by zero
[4. Finally block] Cleaned up measurement resources.
```

Worked examples

Case 1: Attaching Diagnostics with Exception Notes (`add_note()`)

Introduced in PEP 678 (Python 3.11+), `add_note()` allows intermediate functions to append contextual diagnostics (like transaction IDs, tenant names, or IP addresses) without modifying the original exception type:

```python
# exception_notes.py
def connect_database(host: str, port: int) -> None:
    # Simulate network connection failure
    raise ConnectionRefusedError(f"Connection refused to {host}:{port}")

def execute_user_query(tenant_id: str, host: str, port: int) -> None:
    try:
        connect_database(host, port)
    except ConnectionRefusedError as exc:
        # Append diagnostic metadata to the exception
        exc.add_note(f"Tenant ID: {tenant_id}")
        exc.add_note(f"Target cluster: us-east-prod-db")
        raise

if __name__ == "__main__":
    try:
        execute_user_query("tenant_9981", "10.0.4.12", 5432)
    except ConnectionRefusedError as err:
        print("Caught exception with notes:")
        print(f"Message: {err}")
        print("Notes:")
        for note in getattr(err, "__notes__", []):
            print(f"  - {note}")
```

Run:

```
uv run python exception_notes.py
```

Output:

```
Caught exception with notes:
Message: Connection refused to 10.0.4.12:5432
```

```
Notes:
  - Tenant ID: tenant_9981
  - Target cluster: us-east-prod-db
```

Case 2: Concurrent Failure Bundling with `ExceptionGroup`

When performing parallel tasks (or running concurrent task groups in `asyncio`), multiple tasks can fail simultaneously. Python 3.11+ provides `ExceptionGroup` and the `except*` syntax to handle subsets of errors concurrently:

```python
# exception_groups.py
def execute_cluster_rollout() -> None:
    # Simulate multiple parallel node failures
    errors: list[Exception] = [
        TimeoutError("Node 01 timed out waiting for health check"),
        ConnectionResetError("Node 02 reset SSH transport"),
        TimeoutError("Node 04 timed out waiting for health check"),
    ]
    raise ExceptionGroup("Cluster rollout experienced multiple node failures", errors)

def main() -> None:
    try:
        execute_cluster_rollout()
    except* TimeoutError as eg:
        print(f"Handled {len(eg.exceptions)} TimeoutErrors:")
        for exc in eg.exceptions:
            print(f"  * {exc}")
    except* ConnectionResetError as eg:
        print(f"Handled {len(eg.exceptions)} ConnectionResetErrors:")
        for exc in eg.exceptions:
            print(f"  * {exc}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python exception_groups.py
```

Output:

```
Handled 2 TimeoutErrors:
  * Node 01 timed out waiting for health check
  * Node 04 timed out waiting for health check
Handled 1 ConnectionResetErrors:
  * Node 02 reset SSH transport
```

Notice that both `except* TimeoutError` and `except* ConnectionResetError` executed, allowing each error category to be addressed independently from a single compound event.

Pitfalls

Pitfall 1: The Bare `except:` or `except Exception:` Anti-Pattern

Catching all exceptions indiscriminately swallows system-level interruptions like `KeyboardInterrupt` (Ctrl+C) and `SystemExit`:

```
# BAD: Bare except catches BaseException, breaking Ctrl+C!
try:
    run_worker()
except:
    pass

# BAD: Hides programming bugs (like NameError or TypoError)
try:
    calculate_totals()
except Exception:
    print("Something failed.")

# GOOD: Catch only expected, recoverable exception types
try:
    calculate_totals()
except (KeyError, ValueError) as exc:
    logger.warning(f"Malformed input data: {exc}")
```

Pitfall 2: Forgetting to Re-Raise After Logging

Logging an error without re-raising or returning a fallback sentinel causes the program to continue in an inconsistent state:

```
# DANGEROUS:
def load_credentials():
    try:
        return read_vault_secret()
    except VaultError as exc:
        logger.error(f"Vault unreachable: {exc}")
        # Implicitly returns None! Caller expects a dict, crashes later with AttributeError!

# FIX: Re-raise the exception or raise a domain exception
def load_credentials():
```

```
    try:
        return read_vault_secret()
    except VaultError as exc:
        logger.error(f"Vault unreachable: {exc}")
        raise
```

Exercises

1. Write a function `parse_server_port(raw_port: str) -> int` that validates and converts a string into a port integer (1–65535), raising `ValueError` with custom descriptive messages if conversion fails or the number is out of bounds.
2. Construct a `try/except/else/finally` block that opens a simulated database connection, queries data in `try`, records metrics in `else`, and ensures connection closure in `finally`.
3. Use `add_note()` to attach the current local timestamp and host IP to any caught `OSError`.
4. Create an `ExceptionGroup` bundling three different exceptions and use `except*` to handle one type while allowing the others to propagate.

Further reading

- PEP 654: *Exception Groups and except**.
- PEP 678: *Enriching Exceptions with Notes*.
- Python Standard Library: *Built-in Exceptions*.

Custom Exceptions and Error Hierarchies

Custom Exceptions and Error Hierarchies

After reading this chapter, you will design domain-specific exception hierarchies inheriting from `Exception`, carry structured diagnostic payloads inside custom exception objects, use explicit exception chaining (`raise ... from err`), and suppress noisy internal tracebacks using `from None`.

Mental model

Clean architectures do not let low-level internal exceptions (such as `sqlite3.OperationalError` or `urllib3.exceptions.ProtocolError`) leak out to callers. Instead, systems translate low-level faults into domain-specific exceptions.

Organizing exceptions into an inheritance hierarchy allows callers to catch either general errors or specific failure modes:

```
Domain Error Hierarchy:
       Exception (Python standard base)


     StorageError (Base exception for storage subsystem)
        ConnectionTimeoutError
        AuthenticationFailedError
        RecordNotFoundError
              UserNotFoundError (Specialized subtype)
```

A caller can catch `StorageError` to handle any storage subsystem failure, or catch `RecordNotFoundError` specifically to initiate a creation workflow.

Exception Chaining (`__cause__` vs `__context__`)

Python explicitly tracks why an exception occurred using chaining:

- `raise NewError from original_error`: Sets `__cause__`, explicitly documenting that `NewError` was directly triggered by `original_error`.
- `raise NewError from None`: Suppresses the prior traceback, hiding internal implementation details from callers.

Minimal example

Save as `custom_error_hierarchy.py`:

```
# custom_error_hierarchy.py
class CloudServiceError(Exception):
    """Base exception for all cloud provider interactions."""

class RateLimitExceededError(CloudServiceError):
    """Raised when API quotas are breached."""
    def __init__(self, service: str, retry_after: int, quota: int) -> None:
        super().__init__(f"Rate limit exceeded for {service}. Retry after {retry_after}s.")
        self.service = service
        self.retry_after = retry_after
        self.quota = quota

def call_upstream_api(request_count: int) -> None:
    if request_count > 100:
        raise RateLimitExceededError(service="compute-api", retry_after=60, quota=100)
    print(f"API call successful (request count: {request_count}).")

def main() -> None:
    try:
        call_upstream_api(150)
    except RateLimitExceededError as exc:
        print(f"Caught specific rate limit: {exc}")
        print(f"  -> Service: {exc.service} | Backoff: {exc.retry_after}s | Max Quota: {exc.
    except CloudServiceError as exc:
        print(f"Caught general cloud error: {exc}")

if __name__ == "__main__":
    main()
```

Run via `uv run python custom_error_hierarchy.py`:

```
Caught specific rate limit: Rate limit exceeded for compute-api. Retry after 60s.
  -> Service: compute-api | Backoff: 60s | Max Quota: 100
```

Worked examples

Case 1: Exception Chaining with `raise ... from err`

When translating low-level networking errors into high-level domain errors, `from err` links the underlying cause so that debugging logs preserve the original root-cause traceback:

```python
# chained_translation.py
import socket

class DatabaseConnectionError(Exception):
    """Domain exception representing storage cluster failure."""

def ping_db_node(host: str, port: int) -> None:
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.settimeout(0.01)
    try:
        # Intentionally connect to an unreachable test address
        s.connect((host, port))
    except (socket.timeout, OSError) as original_err:
        # Wrap the low-level socket failure into our domain exception
        raise DatabaseConnectionError(f"Failed connecting to database at {host}:{port}") fro
    finally:
        s.close()

if __name__ == "__main__":
    try:
        ping_db_node("192.0.2.1", 5432)  # TEST-NET-1 unroutable address
    except DatabaseConnectionError as err:
        print(f"Domain Error: {err}")
        print(f"Root Cause:   {type(err.__cause__).__name__}: {err.__cause__}")
```

Run:

```
uv run python chained_translation.py
```

Output:

```
Domain Error: Failed connecting to database at 192.0.2.1:5432
Root Cause:   TimeoutError: timed out
```

In the full traceback, Python prints:

```
The above exception was the direct cause of the following exception:
DatabaseConnectionError: Failed connecting to database at 192.0.2.1:5432
```

Case 2: Suppressing Internal Noise with `from None`

When building command-line utilities or clean public libraries, internal library tracebacks create confusing noise for end users. `raise ... from None` severs the link:

```python
# clean_cli_error.py
import json
```

```
class ConfigurationParseError(Exception):
    """Raised when application configuration file is malformed."""

def load_system_config(raw_json: str) -> dict[str, str]:
    try:
        return json.loads(raw_json)
    except json.JSONDecodeError as decode_err:
        # Suppress JSONDecodeError traceback and present a clean error
        raise ConfigurationParseError(
            f"Config file syntax error at line {decode_err.lineno}, column {decode_err.colno
        ) from None

if __name__ == "__main__":
    bad_config = '{"version": 2, "servers": ["app1", "app2", ]}'
    try:
        load_system_config(bad_config)
    except ConfigurationParseError as err:
        print(f"CLI Error: {err}")
        print(f"Has chained cause? {err.__cause__ is not None}")
```

Run:

```
uv run python clean_cli_error.py
```

Output:

```
CLI Error: Config file syntax error at line 1, column 44
Has chained cause? False
```

Pitfalls

Pitfall 1: Inheriting from `BaseException` instead of `Exception`

Inheriting from `BaseException` causes custom errors to bypass standard `except Exception:` handlers:

```
# THE BUG:
class CustomError(BaseException):  # NEVER inherit directly from BaseException!
    pass

# THE FIX:
class CustomError(Exception):       # ALWAYS inherit from Exception
    pass
```

`BaseException` is reserved exclusively for Python's core system events: `SystemExit`, `KeyboardInterrupt`, and `GeneratorExit`.

Pitfall 2: Overly Flat or Monolithic Exceptions

Creating a single `AppError` and using string matching to determine what happened (`if "timeout" in str(e): ...`) defeats Python's type-based exception handling. Always create a base exception class and subclass it for distinct failure modes.

Exercises

1. Design an exception hierarchy for an authentication service with a base `AuthError`, and subclasses `InvalidCredentialsError`, `TokenExpiredError`, and `AccountLockedError`.
2. Write a custom exception `HTTPResponseError` that accepts `status_code: int` and `response_body: str` as constructor arguments and exposes them as read-only properties.
3. Simulate catching a `FileNotFoundError` and wrapping it in a custom `ConfigFileMissingError` using `raise ... from err`.
4. Inspect the `__context__` attribute of an exception raised inside an `except` block without an explicit `from`.

Further reading

- PEP 3134: *Exception Chaining and Embedded Tracebacks*.
- Python Tutorial: *User-defined Exceptions*.
- Python Standard Library: `exceptions` hierarchy.

Context Managers and Resource Management

Context Managers and Resource Management

After reading this chapter, you will master Resource Acquisition Is Initialization (RAII) using Python's `with` statement, implement the `__enter__` and `__exit__` dunder protocol, author lightweight context managers using `@contextlib.contextmanager`, and dynamically manage multi-resource lifecycles with `contextlib.ExitStack`.

Mental model

In systems programming and backend engineering, resources (file descriptors, database connections, mutex locks, network sockets) must be deterministic and leak-free. Relying on manual cleanup (`file.close()`) fails whenever an unexpected exception interrupts execution.

The **Context Management Protocol** binds resource allocation to the entry of a code block and guarantees release upon exit:

```
with acquire_resource() as target:

          1. Calls target = resource.__enter__()

          2. Executes with-block body

               Success         Calls resource.__exit__(None, None, None)

               Exception Raised    Calls resource.__exit__(exc_type, exc_val, exc_tb)

                                          Returns True     Exception is SUPPRESSED
                                          Returns False    Exception PROPAGATES

[ Resumes Outer Execution ]
```

Minimal example

Save as `transaction_manager.py`:

```
# transaction_manager.py
from typing import Any
```

```python
class MockDatabaseTransaction:
    def __init__(self, tx_id: str) -> None:
        self.tx_id = tx_id
        self.active = False

    def __enter__(self) -> "MockDatabaseTransaction":
        self.active = True
        print(f"[{self.tx_id}] BEGIN TRANSACTION: Locks acquired.")
        return self

    def __exit__(self, exc_type: Any, exc_val: Any, exc_tb: Any) -> bool:
        if exc_type is not None:
            print(f"[{self.tx_id}] ROLLBACK: Transaction aborted due to {exc_type.__name__}:
            self.active = False
            return False  # Do not suppress exception; propagate to caller

        print(f"[{self.tx_id}] COMMIT: Transaction changes committed safely.")
        self.active = False
        return True

def main() -> None:
    # 1. Successful transaction
    print("--- Running successful transaction ---")
    with MockDatabaseTransaction("TX_101") as tx:
        print(f"  Writing record into transaction {tx.tx_id}...")

    # 2. Failed transaction with automatic rollback
    print("\n--- Running failed transaction ---")
    try:
        with MockDatabaseTransaction("TX_102") as tx:
            print(f"  Writing record into transaction {tx.tx_id}...")
            raise RuntimeError("Database constraint violation on unique constraint")
    except RuntimeError as err:
        print(f"  Caller caught error: {err}")

if __name__ == "__main__":
    main()
```

Run via `uv run python transaction_manager.py`:

```
--- Running successful transaction ---
[TX_101] BEGIN TRANSACTION: Locks acquired.
  Writing record into transaction TX_101...
[TX_101] COMMIT: Transaction changes committed safely.

--- Running failed transaction ---
[TX_102] BEGIN TRANSACTION: Locks acquired.
```

```
  Writing record into transaction TX_102...
[TX_102] ROLLBACK: Transaction aborted due to RuntimeError: Database constraint violation on ur
  Caller caught error: Database constraint violation on unique constraint
```

Worked examples

Case 1: Benchmark Timer with `@contextlib.contextmanager`

Writing a full class with `__enter__` and `__exit__` is often unnecessary for simple scope management. The `@contextmanager` decorator turns a generator into a clean context manager:

```python
# execution_timer.py
import time
from contextlib import contextmanager
from collections.abc import Generator

@contextmanager
def benchmark(label: str) -> Generator[None, None, None]:
    start_time = time.perf_counter()
    print(f"[{label}] Started execution...")
    try:
        # yield suspends execution and yields control to the with-block
        yield
    finally:
        # Code in finally ALWAYS runs when the with-block terminates
        elapsed = time.perf_counter() - start_time
        print(f"[{label}] Finished in {elapsed * 1000:.2f} ms")

if __name__ == "__main__":
    with benchmark("Data Aggregation"):
        total = sum(x * x for x in range(500_000))
        print(f"  Computed sum: {total}")
```

Run:

```
uv run python execution_timer.py
```

Output:

```
[Data Aggregation] Started execution...
  Computed sum: 41666541666750000
[Data Aggregation] Finished in 24.85 ms
```

Case 2: Clean Error Suppression with `contextlib.suppress`

Instead of writing a verbose `try/except: pass` block to ignore expected non-fatal errors (such as deleting a file that might not exist), `suppress()` makes the intent declarative:

```
# safe_cleanup.py
import os
from contextlib import suppress

def remove_temporary_lock(lock_file_path: str) -> None:
    # Replaces:
    # try:
    #     os.remove(lock_file_path)
    # except FileNotFoundError:
    #     pass
    with suppress(FileNotFoundError):
        os.remove(lock_file_path)
        print(f"Removed lock file: {lock_file_path}")

if __name__ == "__main__":
    # Target does not exist; suppressed cleanly without error
    remove_temporary_lock("/tmp/non_existent_cluster_lock.pid")
    print("Cleanup executed without raising exceptions.")
```

Run:

```
uv run python safe_cleanup.py
```

Output:

```
Cleanup executed without raising exceptions.
```

Case 3: Dynamic Multi-Resource Coordination with `ExitStack`

When the number of files, sockets, or locks is unknown at authoring time (e.g. merging an arbitrary list of configuration files), a static `with open(f1), open(f2):` statement is impossible. `contextlib.ExitStack` manages dynamic collections:

```
# dynamic_file_merger.py
import tempfile
from contextlib import ExitStack
from pathlib import Path

def merge_log_files(source_paths: list[Path], output_path: Path) -> int:
    total_lines = 0
    with ExitStack() as stack:
        # Dynamically register all files into the stack
```

```
        input_handles = [stack.enter_context(p.open("r")) for p in source_paths]
        out_handle = stack.enter_context(output_path.open("w"))

        for handle in input_handles:
            for line in handle:
                out_handle.write(line)
                total_lines += 1

    # ALL handles guaranteed closed here, even if an exception was raised
    return total_lines

if __name__ == "__main__":
    with tempfile.TemporaryDirectory() as tmpdir:
        td = Path(tmpdir)
        f1 = td / "app_01.log"
        f2 = td / "app_02.log"
        f1.write_text("2026-09-07 node01 startup\n")
        f2.write_text("2026-09-07 node02 startup\n")

        merged = td / "merged.log"
        count = merge_log_files([f1, f2], merged)
        print(f"Merged {count} lines into {merged.name}:")
        print(merged.read_text().strip())
```

Run:

```
uv run python dynamic_file_merger.py
```

Output:

```
Merged 2 lines into merged.log:
2026-09-07 node01 startup
2026-09-07 node02 startup
```

Pitfalls

Pitfall 1: Accidentally Suppressing All Exceptions in `__exit__`

If `__exit__` returns any truthy value (like `True` or a non-empty object), CPython suppresses **every** exception raised inside the `with` block:

```
# THE BUG:
def __exit__(self, exc_type, exc_val, exc_tb):
```

```
    self.cleanup()
    return True  # DANGEROUS! Swallows NameError, TypeError, and syntax bugs!

# THE FIX: Only return True if you specifically intended to handle that exact error
def __exit__(self, exc_type, exc_val, exc_tb):
    self.cleanup()
    if exc_type is ResourceUnavailableError:
        return True  # Suppress only this known error
    return False     # Propagate everything else
```

Pitfall 2: Omitting `try/finally` in `@contextmanager` Generators

In a generator context manager, if an exception is raised inside the caller's `with` block, it is thrown into the generator at the point of the `yield`. Without `try...finally`, code after `yield` will never execute:

```
# THE BUG:
@contextmanager
def acquire_lock():
    lock.acquire()
    yield
    lock.release()  # NEVER RUNS if caller raises an exception inside 'with'!

# THE FIX:
@contextmanager
def acquire_lock():
    lock.acquire()
    try:
        yield
    finally:
        lock.release()  # ALWAYS RUNS
```

Exercises

1. Implement a class-based context manager `TemporaryDirectoryChanger` that switches the current working directory to a target directory in `__enter__` and restores the original directory in `__exit__`.
2. Write a generator context manager `override_env(key: str, value: str)` that temporarily modifies `os.environ` and restores the prior value upon block exit.
3. Use `ExitStack` to acquire a list of simulated thread locks and verify they are all released if any single acquisition fails.

4. Implement a context manager that catches `ZeroDivisionError` and logs a warning, while propagating all other exceptions.

Further reading

- PEP 343: *The "with" Statement*.
- Python Standard Library: `contextlib` documentation.
- Python Data Model: *With Statement Context Managers*.

Exception Groups and the except* Operator

Exception Groups and the `except*` Operator

After reading this chapter, you will master Python's concurrent error-handling architecture introduced in PEP 654, construct and inspect hierarchical `ExceptionGroup` trees, handle multiple simultaneous failures using the `except*` clause, isolate specific error sub-types, and prevent mixed-handler syntax traps.

Mental model

In traditional sequential programming, execution halts at the first raised exception. However, in concurrent environments (such as thread pools, worker queues, and `asyncio.TaskGroup`), multiple tasks execute in parallel and can fail simultaneously.

Prior to Python 3.11, runtimes were forced to discard all but the first failure or bury errors in custom aggregate lists. **`ExceptionGroup`** and the **`except*`** operator provide first-class language support for handling multiple independent exceptions at once:

```
Raised ExceptionGroup:
  ExceptionGroup("Batch deployment failure", [
      ConnectionError("Host 10.0.1.1 unreachable"),
      TimeoutError("Host 10.0.1.2 timeout (30s)"),
      ConnectionError("Host 10.0.1.3 connection reset")
  ])


          [ try ... except* ]

    except* ConnectionError as eg:

         Matched Subgroup: [ ConnectionError(10.0.1.1), ConnectionError(10.0.1.3) ]
          Executes this block!

    except* TimeoutError as eg:

         Matched Subgroup: [ TimeoutError(10.0.1.2) ]
          ALSO executes this block!
```

Traditional `except` vs Modern `except*`

Feature	Traditional except	Modern except*
Handled exceptions	Exactly one per block	Multiple excep
Clause execution	First matching clause wins; stops	Multiple match
Unmatched exceptions	Replaces raised error	Automatically
Target type	Any BaseException	Subsets of Exc

Minimal example

Save as `exception_groups_demo.py`:

```python
# exception_groups_demo.py
def deploy_microservices() -> None:
    """Simulate parallel deployment tasks where multiple failures occur."""
    failures = [
        ConnectionError("Failed to connect to redis-01.internal:6379"),
        TimeoutError("Database migration timed out after 60s"),
        ConnectionError("Failed to connect to redis-02.internal:6379"),
    ]
    raise ExceptionGroup("Microservice deployment failed", failures)

def main() -> None:
    try:
        deploy_microservices()
    except* ConnectionError as eg:
        # Matches and extracts ONLY the ConnectionError instances
        print(f"[RECOVER] Handling {len(eg.exceptions)} connection failures:")
        for exc in eg.exceptions:
            print(f"  - {exc}")
    except* TimeoutError as eg:
        # Matches and extracts ONLY the TimeoutError instances in the SAME group!
        print(f"[ALERT] Escalating {len(eg.exceptions)} timeout failures:")
        for exc in eg.exceptions:
            print(f"  - {exc}")

if __name__ == "__main__":
    main()
```

Run via `uv run python exception_groups_demo.py`:

```
[RECOVER] Handling 2 connection failures:
  - Failed to connect to redis-01.internal:6379
  - Failed to connect to redis-02.internal:6379
[ALERT] Escalating 1 timeout failures:
  - Database migration timed out after 60s
```

Notice: Both `except*` clauses executed during the single `try` evaluation!

Worked examples

Case 1: Concurrent Node Health Probing with Diagnostic Subgroups

When monitoring a distributed cluster, diagnostic probes execute across dozens of servers simultaneously. Handling failures requires distinguishing transient network errors from permanent authentication rejections:

```
# cluster_probe_monitor.py
class AuthRejectedError(Exception):
    """Raised when node TLS certificate or API token is invalid."""

def simulate_cluster_audit() -> None:
    errors = [
        TimeoutError("Node 10.0.2.14: Health probe response exceeded 5000ms"),
        AuthRejectedError("Node 10.0.2.18: Client certificate expired"),
        TimeoutError("Node 10.0.2.22: Health probe response exceeded 5000ms"),
    ]
    raise ExceptionGroup("Cluster audit uncovered node health anomalies", errors)

def main() -> None:
    try:
        simulate_cluster_audit()
    except* TimeoutError as eg:
        print(f"[AUTO-RETRY] Enqueueing {len(eg.exceptions)} nodes for exponential backoff:"
        for err in eg.exceptions:
            print(f"  Transient: {err}")
    except* AuthRejectedError as eg:
        print(f"[SECURITY ALERT] Page on-call immediately for {len(eg.exceptions)} security
        for err in eg.exceptions:
            print(f"  Fatal: {err}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python cluster_probe_monitor.py
```

Output:

```
[AUTO-RETRY] Enqueueing 2 nodes for exponential backoff:
  Transient: Node 10.0.2.14: Health probe response exceeded 5000ms
  Transient: Node 10.0.2.22: Health probe response exceeded 5000ms
[SECURITY ALERT] Page on-call immediately for 1 security faults:
  Fatal: Node 10.0.2.18: Client certificate expired
```

Case 2: Programmatic Group Splitting and Filtering (`.split()` and `.subgroup()`)

In addition to `except*` syntax, `ExceptionGroup` instances provide programmatic methods to filter and inspect exception trees directly:

```
# group_introspection.py
def main() -> None:
    group = ExceptionGroup("Data pipeline failure", [
        ValueError("Column 'age' cannot be negative"),
        TypeError("Expected float for column 'latency', got str"),
        ValueError("Column 'score' exceeds maximum 100"),
        RuntimeError("Database write lock acquisition failed"),
    ])

    # 1. .split(predicate) divides the group into (matching_subgroup, remaining_group)
    value_errors, rest = group.split(ValueError)

    print("--- Extracted ValueError Subgroup ---")
    if value_errors:
        for exc in value_errors.exceptions:
            print(f"  Validation issue: {exc}")

    print("\n--- Remaining Unmatched Group ---")
    if rest:
        for exc in rest.exceptions:
            print(f"  System issue:     {exc}")

    # 2. .subgroup(predicate) filters matching items recursively
    type_subgroup = group.subgroup(TypeError)
    print(f"\nType error subgroup item count: {len(type_subgroup.exceptions) if type_subgrou

if __name__ == "__main__":
    main()
```

Run:

```
uv run python group_introspection.py
```

Output:

```
--- Extracted ValueError Subgroup ---
  Validation issue: Column 'age' cannot be negative
  Validation issue: Column 'score' exceeds maximum 100

--- Remaining Unmatched Group ---
  System issue:     Expected float for column 'latency', got str
  System issue:     Database write lock acquisition failed

Type error subgroup item count: 1
```

Case 3: Enriching Individual Errors in Groups with `add_note()`

Python 3.11+ allows attaching contextual notes to any exception via `exc.add_note("text")`. When combined with `ExceptionGroup`, notes provide diagnostics for individual concurrent branches without mutating original exception messages:

```
# diagnostic_notes.py
def perform_shard_sync() -> None:
    e1 = ConnectionResetError("Remote server terminated socket")
    e1.add_note("Shard ID: shard-001 (us-east-1)")
    e1.add_note("Retry attempt: 3 of 3 exhausted")

    e2 = PermissionError("Access denied to replication bucket")
    e2.add_note("Shard ID: shard-004 (eu-central-1)")
    e2.add_note("IAM Role: arn:aws:iam::123456789012:role/StorageReader")

    raise ExceptionGroup("Shard replication pipeline failed", [e1, e2])

def main() -> None:
    try:
        perform_shard_sync()
    except* Exception as eg:
        for exc in eg.exceptions:
            print(f"\nCaught: {type(exc).__name__}: {exc}")
            if hasattr(exc, "__notes__"):
                print("Contextual Notes:")
                for note in exc.__notes__:
                    print(f"  * {note}")
```

```
if __name__ == "__main__":
    main()
```

Run:

```
uv run python diagnostic_notes.py
```

Output:

```
Caught: ConnectionResetError: Remote server terminated socket
Contextual Notes:
  * Shard ID: shard-001 (us-east-1)
  * Retry attempt: 3 of 3 exhausted

Caught: PermissionError: Access denied to replication bucket
Contextual Notes:
  * Shard ID: shard-004 (eu-central-1)
  * IAM Role: arn:aws:iam::123456789012:role/StorageReader
```

Pitfalls

Pitfall 1: Mixing `except` and `except*` in the Same `try` Block

A `try` statement cannot mix traditional `except` clauses with `except*` clauses. Doing so results in an immediate `SyntaxError`:

```
# THE TRAP: SyntaxError
try:
    raise ExceptionGroup("errors", [ValueError("bad")])
except ValueError:     # BUG: SyntaxError: cannot have both 'except' and 'except*' on the sa
    pass
except* TypeError:
    pass

# THE FIX: Use except* exclusively when handling ExceptionGroup hierarchies
try:
    raise ExceptionGroup("errors", [ValueError("bad")])
except* ValueError:
    print("Handled ValueError cleanly")
except* TypeError:
    print("Handled TypeError cleanly")
```

Pitfall 2: Partial Matching Leaves Unhandled Errors to Propagate

When `except*` matches a subgroup, any unmatched exceptions in the `ExceptionGroup` are **automatically re-raised** at the end of the `try ... except*` statement:

```
# THE TRAP: Unhandled exceptions escape!
def run_batch():
    raise ExceptionGroup("batch", [ValueError("invalid input"), KeyError("missing key")])

try:
    try:
        run_batch()
    except* ValueError:
        print("Handled ValueError!")
        # KeyError is NOT handled here! It is automatically re-raised as an ExceptionGroup!
except ExceptionGroup as outer_eg:
    print(f"Caught unhandled residual group: {outer_eg.exceptions}")
```

Output:

```
Handled ValueError!
Caught unhandled residual group: (KeyError('missing key'),)
```

Pitfall 3: Catching `ExceptionGroup` Directly with Traditional `except`

Catching `except ExceptionGroup as eg:` works, but treats the group as a single monolithic block without destructuring. You must manually inspect `.exceptions` instead of letting the VM dispatch by type:

```
# Clunky manual dispatch:
try:
    raise ExceptionGroup("test", [ValueError("v"), TypeError("t")])
except ExceptionGroup as eg:
    for e in eg.exceptions:
        if isinstance(e, ValueError): ...

# Modern idiomatic dispatch:
try:
    raise ExceptionGroup("test", [ValueError("v"), TypeError("t")])
except* ValueError as eg:
    print(f"Handled value errors: {eg.exceptions}")
except* TypeError as eg:
    print(f"Handled type errors: {eg.exceptions}")
```

Exercises

1. Create an `ExceptionGroup` containing two `ValueError` exceptions and one `KeyError`. Write a `try ... except*` block that catches only the `ValueError` exceptions, observing that the `KeyError` propagates.
2. Given an `ExceptionGroup` with nested subgroups, use `.split()` to extract all network-related exceptions (`ConnectionError`, `TimeoutError`) while leaving other errors intact.
3. Write a simulation of 5 concurrent worker tasks using a list of callables. If any fail, collect all exceptions into an `ExceptionGroup` and raise it.
4. Attach diagnostic notes (`.add_note()`) containing timestamps and hostnames to exceptions before grouping them into an `ExceptionGroup`.
5. Demonstrate why `except* Exception` will catch all standard exceptions in a group, but will still let `KeyboardInterrupt` and `SystemExit` (subclasses of `BaseExceptionGroup`) pass through.

Further reading

- PEP 654: *Exception Groups and except**.
- Python Documentation: *Built-in Exceptions – ExceptionGroup and BaseExceptionGroup*.
- Python Documentation: *The try statement – Handling Exception Groups with except**.

Objects and Classes

Classes, Instances, and Encapsulation

Classes, Instances, and Encapsulation

After reading this chapter, you will master class definition and instance instantiation, distinguish between class attributes and instance attributes, implement safe encapsulation using `@property`, and understand the mechanics of Python's name mangling.

Mental model

In Python, classes are first-class objects created at runtime when the `class` block executes. An instance is a distinct heap-allocated object (`PyInstanceObject`) whose `__class__` pointer references its type.

```
Class vs Instance Namespace:
  Class: ServiceConfig
    default_timeout = 30          (Stored in ServiceConfig.__dict__)
    default_retries = 3
    __init__, start, stop methods

         __class__ pointer
  Instance: auth_service
    name = "auth"                 (Stored in auth_service.__dict__)
    port = 8080
```

When you access `auth_service.default_timeout`, Python searches: 1. `auth_service.__dict__` (instance namespace) 2. If missing, `ServiceConfig.__dict__` (class namespace) 3. If missing, parent classes up the MRO 4. If missing, raises `AttributeError`

Minimal example

Save as `class_encapsulation.py`:

```
# class_encapsulation.py
class ServerNode:
    # Class attribute: shared across all instances
    cluster_name: str = "us-east-cluster"
```

```
    def __init__(self, hostname: str, port: int) -> None:
        # Instance attributes: unique to each instance
        self.hostname = hostname
        self._port = port  # Protected convention (one underscore)
        self.__secret_token = f"tok_{hostname}_{port}"  # Private mangled (two underscores)

    @property
    def port(self) -> int:
        """Getter property: read-only access to port."""
        return self._port

    @port.setter
    def port(self, value: int) -> None:
        """Setter property: enforces validation boundaries."""
        if not (1 <= value <= 65535):
            raise ValueError(f"Invalid port {value}: Must be between 1 and 65535")
        self._port = value

    def describe(self) -> str:
        return f"{self.hostname}:{self._port} (Cluster: {self.cluster_name})"

def main() -> None:
    node1 = ServerNode("worker-01", 8080)
    node2 = ServerNode("worker-02", 9090)

    print(node1.describe())
    print(node2.describe())

    # Mutating property via validated setter
    node1.port = 8443
    print(f"Updated node1 port: {node1.port}")

    try:
        node1.port = 99999
    except ValueError as err:
        print(f"Validation rejected invalid port: {err}")

if __name__ == "__main__":
    main()
```

Run via `uv run python class_encapsulation.py`:

```
worker-01:8080 (Cluster: us-east-cluster)
worker-02:9090 (Cluster: us-east-cluster)
Updated node1 port: 8443
Validation rejected invalid port: Invalid port 99999: Must be between 1 and 65535
```

Worked examples

Case 1: Computed Properties & Caching

The `@property` decorator allows creating dynamically computed attributes that look like normal fields to callers:

```
# computed_metrics.py
import time

class StorageVolume:
    def __init__(self, name: str, total_gb: float, used_gb: float) -> None:
        self.name = name
        self.total_gb = total_gb
        self.used_gb = used_gb

    @property
    def free_gb(self) -> float:
        """Dynamically computed attribute."""
        return self.total_gb - self.used_gb

    @property
    def usage_percent(self) -> float:
        """Percentage calculation with guard against division by zero."""
        if self.total_gb <= 0:
            return 0.0
        return (self.used_gb / self.total_gb) * 100.0

    @property
    def status(self) -> str:
        pct = self.usage_percent
        if pct > 90.0:
            return "CRITICAL"
        elif pct > 75.0:
            return "WARNING"
        return "HEALTHY"

if __name__ == "__main__":
    vol = StorageVolume("data-vol-01", total_gb=500.0, used_gb=410.0)
    print(f"Volume   : {vol.name}")
    print(f"Free     : {vol.free_gb:.1f} GB")
    print(f"Usage    : {vol.usage_percent:.1f}%")
    print(f"Status   : {vol.status}")
```

Run:

```
uv run python computed_metrics.py
```

Output:

```
Volume   : data-vol-01
Free     : 90.0 GB
Usage    : 82.0%
Status   : WARNING
```

Case 2: Demystifying Name Mangling (`__var`)

When an attribute starts with two leading underscores (and at most one trailing underscore), CPython transforms the name internally to `_<ClassName>__<var>` to prevent accidental overriding in subclasses:

```python
# name_mangling_demo.py
class SecurityKey:
    def __init__(self, key_id: str, raw_secret: str) -> None:
        self.key_id = key_id
        self.__secret = raw_secret  # Name mangled

    def verify(self, candidate: str) -> bool:
        return self.__secret == candidate

if __name__ == "__main__":
    sec = SecurityKey("auth-key-01", "super_secret_token_123")
    print(f"Public ID: {sec.key_id}")
    print("Verification result:", sec.verify("super_secret_token_123"))

    # Direct access fails with AttributeError
    try:
        print(sec.__secret)
    except AttributeError as err:
        print(f"Direct access rejected: {err}")

    # Inspecting the instance dictionary reveals the mangled name
    print(f"Internal __dict__ keys: {list(sec.__dict__.keys())}")
    print(f"Accessing mangled key directly: {sec._SecurityKey__secret}")
```

Run:

```
uv run python name_mangling_demo.py
```

Output:

```
Public ID: auth-key-01
Verification result: True
Direct access rejected: 'SecurityKey' object has no attribute '__secret'
Internal __dict__ keys: ['key_id', '_SecurityKey__secret']
```

```
Accessing mangled key directly: super_secret_token_123
```

Name mangling is not true cryptographic security (Python does not enforce private memory protection); it is an anti-collision mechanism to prevent subclasses from accidentally stomping on internal fields.

Pitfalls

Pitfall 1: Mutating Class Attributes via an Instance Reference

Assigning to `instance.attr` creates a new instance attribute in that instance's dictionary, rather than updating the shared class attribute:

```python
class Cluster:
    nodes = []  # Dangerous mutable class attribute!

c1 = Cluster()
c2 = Cluster()

# Mutating in-place affects all instances because 'nodes' references the same list
c1.nodes.append("node-1")
print(c2.nodes)  # ['node-1']

# Re-binding creates a new instance attribute on c1 only:
c1.nodes = ["node-new"]
print(c1.nodes)  # ['node-new'] (stored in c1.__dict__)
print(c2.nodes)  # ['node-1']   (still looks at Cluster.nodes)
```

Always initialize mutable attributes (lists, dicts, sets) inside `__init__` on `self`.

Exercises

1. Create a class `NetworkInterface` with attributes `name` and `ip_address`. Add a property `is_loopback` that returns `True` if the IP address starts with `"127."`.
2. Implement a `TemperatureSensor` class that stores temperature in Celsius, but provides getter and setter properties for Fahrenheit with proper mathematical conversion ($F = C \times \frac{9}{5} + 32$).
3. Demonstrate the difference in `__dict__` between an instance with single-underscore attributes (`self._token`) versus double-underscore attributes (`self.__token`).
4. Write a class that counts how many instances of itself have been created using a class attribute.

Further reading

- Python Documentation: *Classes and Objects*.
- Python Data Model: *Customizing attribute access*.
- PEP 8: *Naming Conventions for class and instance attributes*.

The Python Data Model and Dunder Protocols

The Python Data Model and Dunder Protocols

After reading this chapter, you will master Python's special method protocols (known as "dunder" methods for *double underscore*), implement unambiguous string representations (`__repr__` vs `__str__`), enable custom objects for dictionary hashing and equality, implement sequence and container behaviors, and make objects callable like functions.

Mental model

The Python Data Model is the API of the Python language itself. Built-in functions, operators, and syntax do not perform hardcoded checks; they delegate to special methods implemented on your classes:

Syntax / Built-in	Dunder Method Invoked
`repr(obj)`	`obj.__repr__()`
`str(obj)`, `print(obj)`	`obj.__str__()` (falls back to `__repr__`)
`len(obj)`	`obj.__len__()`
`item in obj`	`obj.__contains__(item)`
`obj[key]`	`obj.__getitem__(key)`
`obj == other`	`obj.__eq__(other)`
`hash(obj)`	`obj.__hash__()`
`obj(arg1, arg2)`	`obj.__call__(arg1, arg2)`

By implementing these protocols, custom classes behave with the same ergonomics and speed as Python's native data types.

Minimal example

Save as `data_model_protocols.py`:

```python
# data_model_protocols.py
from typing import Any

class PacketBuffer:
    def __init__(self, packets: list[bytes]) -> None:
```

```
        self._packets = list(packets)

    def __len__(self) -> int:
        """Enables len(buffer)."""
        return len(self._packets)

    def __getitem__(self, index: int | slice) -> Any:
        """Enables buffer[i] indexing and buffer[start:stop] slicing."""
        return self._packets[index]

    def __contains__(self, item: bytes) -> bool:
        """Enables 'payload in buffer' membership testing."""
        return item in self._packets

    def __repr__(self) -> str:
        """Unambiguous representation for debugging and logs."""
        return f"PacketBuffer(count={len(self._packets)})"

def main() -> None:
    p1 = b"\x00\x01\x02\x03"
    p2 = b"\xaa\xbb\xcc\xdd"
    p3 = b"\xff\xff\xff\xff"

    buf = PacketBuffer([p1, p2, p3])

    print("Buffer representation:", repr(buf))
    print(f"Total packet count    : {len(buf)}")
    print(f"Packet at index 1     : {buf[1].hex()}")
    print(f"Contains p2?          : {p2 in buf}")
    print(f"Contains missing?     : {b'dummy' in buf}")

    # Iteration works automatically via __getitem__!
    print("Iterating over packet hex strings:")
    for pkt in buf:
        print(f"  - {pkt.hex()}")

if __name__ == "__main__":
    main()
```

Run via `uv run python data_model_protocols.py`:

```
Buffer representation: PacketBuffer(count=3)
Total packet count    : 3
Packet at index 1     : aabbccdd
Contains p2?          : True
Contains missing?     : False
Iterating over packet hex strings:
```

- 00010203
- aabbccdd
- ffffffff

Notice that we did not even implement `__iter__()`: Python's iteration protocol automatically falls back to `__getitem__()` starting at index 0 until `IndexError` is raised!

Worked examples

Case 1: The Golden Rule of `__repr__` vs `__str__`

- `__repr__`: Intended for developers, debuggers, and logs. It should be unambiguous and, whenever practical, look like valid Python code that recreates the object (`eval(repr(x)) == x`).
- `__str__`: Intended for end-user display (`print(x)`).

```
# repr_vs_str.py
class Endpoint:
    def __init__(self, host: str, port: int, use_tls: bool = True) -> None:
        self.host = host
        self.port = port
        self.use_tls = use_tls

    def __repr__(self) -> str:
        # Developer-facing, unambiguous
        return f"Endpoint(host={self.host!r}, port={self.port}, use_tls={self.use_tls})"

    def __str__(self) -> str:
        # User-facing URI string
        scheme = "https" if self.use_tls else "http"
        return f"{scheme}://{self.host}:{self.port}"

if __name__ == "__main__":
    ep = Endpoint("api.internal", 8443, use_tls=True)
    print("User facing (str)      :", str(ep))
    print("Developer facing (repr):", repr(ep))
```

Run:

```
uv run python repr_vs_str.py
```

Output:

```
User facing (str)      : https://api.internal:8443
Developer facing (repr): Endpoint(host='api.internal', port=8443, use_tls=True)
```

Case 2: Equality (`__eq__`) and Hashing (`__hash__`)

To allow custom objects to be used as dictionary keys or stored in sets, you must define both `__eq__` and `__hash__`. An object's hash must be computed from immutable attributes:

```
# hashable_host.py
class HostID:
    def __init__(self, cluster: str, node_id: int) -> None:
        self._cluster = cluster
        self._node_id = node_id

    @property
    def cluster(self) -> str:
        return self._cluster

    @property
    def node_id(self) -> int:
        return self._node_id

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, HostID):
            return NotImplemented
        return self._cluster == other._cluster and self._node_id == other._node_id

    def __hash__(self) -> int:
        # Hash a tuple of the immutable identity attributes
        return hash((self._cluster, self._node_id))

    def __repr__(self) -> str:
        return f"HostID({self._cluster!r}, {self._node_id})"

if __name__ == "__main__":
    h1 = HostID("prod-us", 101)
    h2 = HostID("prod-us", 101)
    h3 = HostID("prod-eu", 101)

    print(f"h1 == h2 : {h1 == h2}")
    print(f"h1 == h3 : {h1 == h3}")

    # Safe for use in sets and dictionary keys
    cluster_nodes = {h1, h2, h3}
    print(f"Unique nodes in set (count: {len(cluster_nodes)}): {cluster_nodes}")
```

Run:

```
uv run python hashable_host.py
```

Output:

```
h1 == h2 : True
h1 == h3 : False
Unique nodes in set (count: 2): {HostID('prod-us', 101), HostID('prod-eu', 101)}
```

Case 3: Automatic Ordering with `functools.total_ordering`

Implementing all comparison operators (`<`, `<=`, `>`, `>=`, `==`, `!=`) is tedious. By implementing `__eq__` and just one ordering method (`__lt__`), `@total_ordering` derives the rest:

```
# priority_ordering.py
from functools import total_ordering

@total_ordering
class TaskPriority:
    def __init__(self, level: int, name: str) -> None:
        self.level = level
        self.name = name

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, TaskPriority):
            return NotImplemented
        return self.level == other.level

    def __lt__(self, other: object) -> bool:
        if not isinstance(other, TaskPriority):
            return NotImplemented
        return self.level < other.level

    def __repr__(self) -> str:
        return f"Priority({self.level}, {self.name!r})"

if __name__ == "__main__":
    low = TaskPriority(1, "Background Backup")
    med = TaskPriority(5, "Log Rotation")
    high = TaskPriority(10, "Service Outage Failover")

    print(f"low < high  : {low < high}")
    print(f"high >= med : {high >= med}")

    tasks = [med, high, low]
    tasks.sort()
    print("Sorted tasks by priority:", tasks)
```

Run:

```
uv run python priority_ordering.py
```

Output:

```
low < high  : True
high >= med : True
Sorted tasks by priority: [Priority(1, 'Background Backup'), Priority(5, 'Log Rotation'), Prior
```

Case 4: Callable Objects with `__call__`

Defining `__call__` turns an instance into a callable that retains internal state across calls:

```python
# rate_limiter_callable.py
import time

class SimpleRateLimiter:
    def __init__(self, max_calls: int, window_seconds: float) -> None:
        self.max_calls = max_calls
        self.window_seconds = window_seconds
        self.calls: list[float] = []

    def __call__(self, client_id: str) -> bool:
        now = time.monotonic()
        # Evict timestamps outside the sliding window
        self.calls = [t for t in self.calls if now - t < self.window_seconds]

        if len(self.calls) >= self.max_calls:
            return False  # Rate limit breached

        self.calls.append(now)
        return True

if __name__ == "__main__":
    limiter = SimpleRateLimiter(max_calls=3, window_seconds=1.0)

    # Invoked like a function: limiter("client_1")
    for i in range(5):
        allowed = limiter("client_1")
        print(f"Request {i + 1}: {'ALLOWED' if allowed else 'DENIED'}")
```

Run:

```
uv run python rate_limiter_callable.py
```

Output:

```
Request 1: ALLOWED
Request 2: ALLOWED
Request 3: ALLOWED
```

```
Request 4: DENIED
Request 5: DENIED
```

Pitfalls

Pitfall 1: Defining `__eq__` Without `__hash__`

In Python 3, if a class defines `__eq__` but does not define `__hash__`, Python automatically sets `__hash__ = None`:

```
class Node:
    def __init__(self, name):
        self.name = name
    def __eq__(self, other):
        return self.name == other.name

n = Node("worker")
# s = {n}  -> TypeError: unhashable type: 'Node'
```

If you override `__eq__` and want the object to remain hashable, you **must** explicitly implement `__hash__`. If the object is mutable, leaving `__hash__ = None` is the correct safety measure to prevent hash table corruption.

Exercises

1. Create a class `Vector2D(x, y)` that implements `__add__` and `__sub__` for vector arithmetic, and `__abs__` to calculate Euclidean magnitude ($\sqrt{x^2 + y^2}$).
2. Implement a `RollingLog` container that stores the last N entries and implements `__len__`, `__getitem__`, and `__iter__`.
3. Create an immutable `CIDRBlock` class that implements `__eq__` and `__hash__` so that equivalent subnets can be deduplicated in a `set`.
4. Implement a stateful token-bucket rate limiter class that uses `__call__` to check and consume tokens.

Further reading

- Python Documentation: *The Python Data Model — Special method names*.
- Python Standard Library: `functools.total_ordering`.
- Luciano Ramalho: *Fluent Python, Second Edition* (O'Reilly).

Inheritance, MRO, and Composition

Inheritance, MRO, and Composition

After reading this chapter, you will master single and multiple inheritance in Python, understand the Method Resolution Order (MRO) derived by C3 linearization, implement cooperative multiple inheritance using `super()`, enforce architectural interfaces using Abstract Base Classes (`abc.ABC`), and favor composition over fragile inheritance hierarchies.

Mental model

When a subclass inherits from multiple parent classes, how does Python decide which implementation of a method to execute? It computes a deterministic, monotonic linear order known as the **Method Resolution Order (MRO)** using the **C3 Linearization Algorithm**.

```
Diamond Inheritance Graph:
          [ Base ]
          /      \
      [ Left ]  [ Right ]
          \      /
         [ Child ]

C3 Linearization (Child.__mro__):
  Child    Left    Right    Base    object
```

Cooperative `super()` Dispatch

In Python, `super()` does **not** simply call the immediate parent in the source code. Instead, it calls the **next class in the instance's MRO**:

```
Call from Child: super().process()
  1. Inspects type(self).__mro__
  2. Finds caller's current class in MRO (Child)
  3. Dispatches to the NEXT class in sequence (Left)
  4. Left's super().process() dispatches to Right!
```

This cooperative chaining ensures every class in the graph is visited exactly once.

Minimal example

Save as `inheritance_mro.py`:

```
# inheritance_mro.py
class BaseService:
    def __init__(self, name: str) -> None:
        self.name = name
        print(f"  [BaseService] Initialized service: {name}")

class LoggingMixin:
    def __init__(self, *args: object, **kwargs: object) -> None:
        super().__init__(*args, **kwargs)
        print("  [LoggingMixin] Telemetry logger attached.")

class MetricsMixin:
    def __init__(self, *args: object, **kwargs: object) -> None:
        super().__init__(*args, **kwargs)
        print("  [MetricsMixin] Metrics reporter registered.")

# Multiple inheritance with Mixins
class ProductionWebService(LoggingMixin, MetricsMixin, BaseService):
    def __init__(self, name: str) -> None:
        print(f"Initializing ProductionWebService: {name}")
        # Cooperative super() walks through LoggingMixin -> MetricsMixin -> BaseService
        super().__init__(name)

def main() -> None:
    svc = ProductionWebService("auth-gateway")

    print("\nMethod Resolution Order (MRO):")
    for idx, cls in enumerate(ProductionWebService.__mro__):
        print(f"  {idx}: {cls.__name__}")

if __name__ == "__main__":
    main()
```

Run via `uv run python inheritance_mro.py`:

```
Initializing ProductionWebService: auth-gateway
  [BaseService] Initialized service: auth-gateway
  [MetricsMixin] Metrics reporter registered.
  [LoggingMixin] Telemetry logger attached.

Method Resolution Order (MRO):
  0: ProductionWebService
  1: LoggingMixin
```

```
2: MetricsMixin
3: BaseService
4: object
```

Worked examples

Case 1: Formal Interfaces with Abstract Base Classes (`abc.ABC`)

In plugin systems, drivers, and microservices, you must guarantee that subclasses adhere to a specific interface. Abstract Base Classes prevent instantiation if required methods are missing:

```python
# storage_plugin_abc.py
from abc import ABC, abstractmethod

class StorageDriver(ABC):
    """Abstract base contract for blob and file storage backends."""

    @abstractmethod
    def read_bytes(self, path: str) -> bytes:
        """Fetch raw bytes from storage."""
        ...

    @abstractmethod
    def write_bytes(self, path: str, data: bytes) -> int:
        """Write raw bytes and return bytes written."""
        ...

    def ping(self) -> bool:
        """Concrete utility method available to all implementations."""
        return True

class MemoryStorageDriver(StorageDriver):
    def __init__(self) -> None:
        self._store: dict[str, bytes] = {}

    def read_bytes(self, path: str) -> bytes:
        if path not in self._store:
            raise FileNotFoundError(f"Path not found: {path}")
        return self._store[path]

    def write_bytes(self, path: str, data: bytes) -> int:
        self._store[path] = data
        return len(data)
```

```
if __name__ == "__main__":
    driver = MemoryStorageDriver()
    print("Driver ping:", driver.ping())

    n = driver.write_bytes("/configs/app.json", b'{"debug": true}')
    print(f"Wrote {n} bytes.")
    print("Read back:", driver.read_bytes("/configs/app.json"))

    # Attempting to instantiate an incomplete subclass fails:
    class IncompleteDriver(StorageDriver):
        pass

    try:
        IncompleteDriver()
    except TypeError as err:
        print("\nInstantiation rejected for incomplete subclass:")
        print(f"  {err}")
```

Run:

```
uv run python storage_plugin_abc.py
```

Output:

```
Driver ping: True
Wrote 15 bytes.
Read back: b'{"debug": true}'

Instantiation rejected for incomplete subclass:
  Can't instantiate abstract class IncompleteDriver without an implementation for abstract meth
```

Case 2: Composition Over Inheritance

Deep inheritance hierarchies (`Device -> NetworkDevice -> Switch -> ManagedSwitch -> CiscoManagedSwitch`) become rigid, fragile, and hard to test. **Composition** assembles behavior from independent components:

```
# composition_architecture.py
class SSHTransport:
    def execute(self, host: str, command: str) -> str:
        return f"[SSH to {host}] Executed: {command} -> Output: SUCCESS"

class RESTTransport:
    def execute(self, host: str, command: str) -> str:
        return f"[REST API to {host}] POST /rpc -> Output: 200 OK"
```

```
class NetworkSwitchManager:
    """Uses composition: receives a transport rather than inheriting from one."""
    def __init__(self, host: str, transport: SSHTransport | RESTTransport) -> None:
        self.host = host
        self.transport = transport

    def reload_interfaces(self) -> str:
        return self.transport.execute(self.host, "reload interfaces")

if __name__ == "__main__":
    # Swap transports at runtime without changing the Manager class
    ssh_manager = NetworkSwitchManager("switch-01.rack-a", SSHTransport())
    api_manager = NetworkSwitchManager("switch-02.rack-b", RESTTransport())

    print(ssh_manager.reload_interfaces())
    print(api_manager.reload_interfaces())
```

Run:

```
uv run python composition_architecture.py
```

Output:

```
[SSH to switch-01.rack-a] Executed: reload interfaces -> Output: SUCCESS
[REST API to switch-02.rack-b] POST /rpc -> Output: 200 OK
```

Pitfalls

Pitfall 1: Calling Direct Parent Names Instead of `super()`

Directly invoking `ParentClass.__init__(self)` bypasses cooperative multiple inheritance and causes sibling mixins in the MRO to be silently skipped:

```
# THE BUG:
class BrokenService(LoggingMixin, BaseService):
    def __init__(self, name):
        BaseService.__init__(self, name)  # BUG: Skips LoggingMixin.__init__ completely!

# THE FIX:
class CorrectService(LoggingMixin, BaseService):
    def __init__(self, name):
        super().__init__(name)             # Traverses all classes in __mro__ cooperatively
```

Pitfall 2: Overusing Multiple Inheritance

Multiple inheritance is best reserved for lightweight, orthogonal **mixins** that do not maintain complex conflicting state. For domain business logic, prefer composition.

Exercises

1. Define an Abstract Base Class `MessageQueue(ABC)` with abstract methods `publish(topic: str, payload: bytes)` and `subscribe(topic: str)`. Implement a concrete `InMemoryQueue` subclass.
2. Given classes `A`, `B(A)`, `C(A)`, and `D(B, C)`, print `D.__mro__` and trace the order in which C3 linearization resolves methods.
3. Refactor a deep three-level inheritance tree into a single class that uses composition with two injected helper objects.
4. Implement a `JSONSerializableMixin` that provides a `to_json()` method by serializing `self.__dict__`.

Further reading

- Guido van Rossum: *The Python 2.3 Method Resolution Order*.
- PEP 3119: *Introducing Abstract Base Classes*.
- Python Standard Library: `abc` module.

Dataclasses and Declarative Models

Dataclasses and Declarative Models

After reading this chapter, you will master Python's `@dataclass` decorator (PEP 557), enforce immutability with `frozen=True`, optimize memory with `slots=True`, configure default factories for safe collections, execute validation via `__post_init__`, and understand the boundary between standard library dataclasses and Pydantic v2.

Mental model

Writing repetitive boilerplate (`__init__`, `__repr__`, `__eq__`, `__hash__`) for data-holding classes is error-prone. The `@dataclass` decorator inspects type annotations at class definition time and automatically generates these methods.

```
Class Source:
  @dataclass(slots=True, frozen=True)
  class ServiceNode:
      host: str
      port: int = 8080

Generated by CPython:
  - __init__(self, host: str, port: int = 8080)
  - __repr__(self) -> "ServiceNode(host=..., port=...)"
  - __eq__(self, other) -> compares tuple of fields
  - __hash__(self) -> computes hash from fields (enabled by frozen=True)
  - __slots__ = ('host', 'port') -> eliminates per-instance __dict__
```

Memory Architecture: `__dict__` vs `__slots__`

A standard Python object maintains a dynamic dictionary (`__dict__`) to allow arbitrary attribute additions at runtime. In microservices holding hundreds of thousands of objects (metrics, tokens, edges), `__dict__` introduces massive memory overhead:

```
Standard Instance with __dict__:
  PyObject Header    __dict__ (PyDictObject: table, keys, hash entries) (~152 bytes)

Dataclass with slots=True:
  PyObject Header    [ field_0_ptr, field_1_ptr ] (Fixed C-struct offsets) (~56 bytes)
```

Minimal example

Save as `dataclasses_deep_dive.py`:

```
# dataclasses_deep_dive.py
from dataclasses import dataclass, field

@dataclass(slots=True, frozen=True)
class EndpointConfig:
    host: str
    port: int = 443
    tags: list[str] = field(default_factory=list)

    def __post_init__(self) -> None:
        # Validation hook executed immediately after generated __init__
        if not (1 <= self.port <= 65535):
            raise ValueError(f"Port {self.port} out of range (1-65535)")

def main() -> None:
    ep1 = EndpointConfig("auth.prod.internal", 8443, tags=["prod", "security"])
    ep2 = EndpointConfig("auth.prod.internal", 8443, tags=["prod", "security"])

    print("Representation:", repr(ep1))
    print(f"ep1 == ep2    : {ep1 == ep2}")

    # Hashable because frozen=True and tags can be hashed if converted or compared
    print(f"Port accessor  : {ep1.port}")

    # Mutation is strictly forbidden
    try:
        ep1.port = 80  # type: ignore[misc]
    except Exception as err:
        print(f"Mutation rejected: {type(err).__name__}")

if __name__ == "__main__":
    main()
```

Run via `uv run python dataclasses_deep_dive.py`:

```
Representation: EndpointConfig(host='auth.prod.internal', port=8443, tags=['prod', 'security'])
ep1 == ep2    : True
Port accessor  : 8443
Mutation rejected: FrozenInstanceError
```

Worked examples

Case 1: Mutable Defaults via `field(default_factory=...)`

Python prevents using a mutable object directly as a dataclass default. You must provide a callable factory:

```
# safe_defaults.py
from dataclasses import dataclass, field
from datetime import datetime

@dataclass
class AuditRecord:
    actor: str
    action: str
    # DANGEROUS: metadata: dict = {} is rejected by dataclasses!
    # SAFE: Use default_factory
    metadata: dict[str, str] = field(default_factory=dict)
    timestamp: str = field(default_factory=lambda: datetime.now().isoformat())

if __name__ == "__main__":
    rec1 = AuditRecord("admin", "restart_service")
    rec1.metadata["target"] = "worker-01"

    rec2 = AuditRecord("operator", "drain_node")

    print(f"Record 1 metadata: {rec1.metadata}")
    print(f"Record 2 metadata: {rec2.metadata} (Remains isolated and empty)")
```

Run:

```
uv run python safe_defaults.py
```

Output:

```
Record 1 metadata: {'target': 'worker-01'}
Record 2 metadata: {} (Remains isolated and empty)
```

Case 2: Memory Benchmark: `slots=True` vs Standard Class

When handling high-volume data streams (e.g. log events or sensor telemetry), `slots=True` drastically reduces memory consumption:

```
# slots_memory_benchmark.py
import sys
from dataclasses import dataclass
```

```
class StandardEvent:
    def __init__(self, device_id: str, reading: float) -> None:
        self.device_id = device_id
        self.reading = reading

@dataclass(slots=True)
class SlottedEvent:
    device_id: str
    reading: float

def main() -> None:
    std = StandardEvent("sensor-99", 23.4)
    slotted = SlottedEvent("sensor-99", 23.4)

    # Note: sys.getsizeof on an instance does not include its __dict__ size
    std_total_size = sys.getsizeof(std) + sys.getsizeof(std.__dict__)
    slotted_total_size = sys.getsizeof(slotted)

    print(f"Standard instance total memory: {std_total_size} bytes (has __dict__)")
    print(f"Slotted instance total memory : {slotted_total_size} bytes (no __dict__)")
    print(f"Memory reduction              : {((std_total_size - slotted_total_size) / std_to

if __name__ == "__main__":
    main()
```

Run:

```
uv run python slots_memory_benchmark.py
```

Output:

```
Standard instance total memory: 152 bytes (has __dict__)
Slotted instance total memory : 56 bytes (no __dict__)
Memory reduction              : 63.2%
```

Case 3: Dataclasses vs Pydantic v2: Architectural Boundaries

A common architectural question in modern Python: **When to use `@dataclass` vs Pydantic?**

Feature	Standard Library `@dataclass`	Pydantic v2 (`BaseModel`)
Dependency	Standard library (zero dependencies)	External package (`pip install pydantic`)
Type Checking	Annotations used for code generation only; **no runtime enforcement**	**Strict runtime validation** & automatic type coercion

Feature	Standard Library `@dataclass`	Pydantic v2 (`BaseModel`)
Parsing	None (requires manual parsing)	Built-in JSON/dict deserialization and schema export
Execution Speed	Zero overhead; pure Python / C-struct	Rust-core (`pydantic-core`) high speed parsing
Best Used For	Internal application state, domain entities, math models	Public API boundaries, untrusted JSON input, config loading

---

Pitfalls

Pitfall 1: Assuming `@dataclass` Enforces Types at Runtime

Standard dataclasses do **not** validate types when assigned:

```
@dataclass
class PortConfig:
    port: int

# Valid syntax; no error is raised despite 'invalid' being a string!
p = PortConfig("invalid")  # type: ignore[arg-type]
print(p.port)  # Prints: "invalid"
```

If you need runtime type enforcement at system boundaries, use `__post_init__` validation or Pydantic.

Exercises

1. Create a `@dataclass(slots=True, frozen=True)` named `MetricRecord` with fields `timestamp`, `metric_name`, and `value`. Verify that attempting to change `value` raises `FrozenInstanceError`.
2. Write a dataclass `DatabaseConnectionSpec` where `database_url` is automatically constructed in `__post_init__` from `host`, `port`, and `database_name`.
3. Demonstrate why `@dataclass class Bad: items: list = []` causes a `ValueError` at class definition time.
4. Implement a dataclass with `order=True` that automatically generates comparison operators based on a specific `priority` field.

Further reading

- PEP 557: *Data Classes*.
- Python Standard Library: `dataclasses` documentation.
- Python Language Reference: `__slots__` optimization.

Dataclass slots=True

Dataclass slots=True

Thesis

`@dataclass(slots=True)` (3.10+) gives instances `__slots__` instead of a per-instance `__dict__`: less memory, attribute typos fail fast, and you opt out of accidental dynamic attributes.

Mental model

```
  @dataclass
  class A: x: int          # instance.__dict__ holds x

  @dataclass(slots=True)
  class B: x: int          # B.__slots__ == ('x',); no __dict__
```

Worked example

```
# slots_demo.py
from dataclasses import dataclass
import sys

@dataclass
class Plain:
    x: int
    y: int

@dataclass(slots=True)
class Slotted:
    x: int
    y: int

p, s = Plain(1, 2), Slotted(1, 2)
print("plain dict", hasattr(p, "__dict__"), "slots", hasattr(s, "__dict__"))
print("Slotted.__slots__", Slotted.__slots__)
try:
    s.z = 3
except AttributeError as e:
```

```
    print("assign extra:", type(e).__name__)

# rough size hint (implementation-dependent)
print("sizeof plain", sys.getsizeof(p) + sys.getsizeof(p.__dict__))
print("sizeof slotted", sys.getsizeof(s))
```

```
plain size 48 has __dict__ True
slots size 48 has __dict__ False
assign extra: AttributeError
plain.__dict__ {'x': 1, 'y': 2}
```

Trap

Multiple inheritance with slotted dataclasses needs care — all bases should cooperate on slots. Also: `slots=True` plus `weakref` needs `weakref_slot=True`. Defaults and `InitVar` still work; `object.__setattr__` is used during `__init__`.

Boring rule

Use `slots=True` for high-cardinality records (events, rows, AST nodes). Keep unslotted dataclasses when callers routinely monkey-patch attributes in tests.

Try this

1. Measure RSS for a million instances of each form.
2. Combine `slots=True` with `frozen=True`.
3. Inspect `__slots__` when inheriting from another slotted dataclass.

Sources

- Python docs: dataclasses
- PEP 681 (dataclass transforms; related tooling)

Decorators and Metaprogramming

Decorators and Metaprogramming

After reading this chapter, you will master Python decorators from first principles, preserve function introspection with `@functools.wraps`, construct parameterized decorator factories, implement class decorators, and use `@classmethod` and `@staticmethod` appropriately.

Mental model

A decorator in Python is syntactic sugar for passing a function (or class) into a higher-order function, and rebinding the original name to the returned callable.

```
Syntactic Sugar:
  @log_execution
  def query_db(query_str):
      ...

Is Bit-for-Bit Equivalent To:
  def query_db(query_str):
      ...
  query_db = log_execution(query_db)
```

The Wrapper Wrapping Pipeline

```
Caller executes query_db("SELECT 1")


      [ wrapper(*args, **kwargs) ]

               Pre-execution logic (start timer, log entry)
               Result = original_func(*args, **kwargs)
               Post-execution logic (stop timer, record metrics)


      Return Result to Caller
```

Minimal example

Save as `decorators_fundamentals.py`:

```python
# decorators_fundamentals.py
import functools
import time
from collections.abc import Callable
from typing import Any

def measure_latency(func: Callable[..., Any]) -> Callable[..., Any]:
    """Decorator that measures and prints the execution latency of any function."""
    @functools.wraps(func)
    def wrapper(*args: Any, **kwargs: Any) -> Any:
        start_time = time.perf_counter()
        try:
            return func(*args, **kwargs)
        finally:
            elapsed_ms = (time.perf_counter() - start_time) * 1000
            print(f"[{func.__name__}] Executed in {elapsed_ms:.2f} ms")
    return wrapper

@measure_latency
def compute_hash_digest(payload: str) -> int:
    """Compute an integer checksum from payload."""
    total = 0
    for char in payload:
        total = (total * 31 + ord(char)) & 0xFFFFFFFF
    return total

def main() -> None:
    digest = compute_hash_digest("production-event-payload-string")
    print(f"Computed digest: {digest}")
    print(f"Function name  : {compute_hash_digest.__name__}")
    print(f"Docstring      : {compute_hash_digest.__doc__}")

if __name__ == "__main__":
    main()
```

Run via `uv run python decorators_fundamentals.py`:

```
[compute_hash_digest] Executed in 0.01 ms
Computed digest: 1475752251
Function name  : compute_hash_digest
Docstring      : Compute an integer checksum from payload.
```

Worked examples

Case 1: Parameterized Decorator Factory (`@retry`)

When a decorator requires configuration arguments (`@retry(max_attempts=3, backoff=0.05)`), you must create a three-tier factory function:

```
# retry_decorator.py
import functools
import time
from collections.abc import Callable
from typing import Any

def retry(max_attempts: int = 3, backoff: float = 0.05) -> Callable[..., Any]:
    """Decorator factory that retries an operation on failure with exponential backoff."""
    def decorator(func: Callable[..., Any]) -> Callable[..., Any]:
        @functools.wraps(func)
        def wrapper(*args: Any, **kwargs: Any) -> Any:
            attempts = 0
            while True:
                attempts += 1
                try:
                    return func(*args, **kwargs)
                except Exception as exc:
                    if attempts >= max_attempts:
                        print(f"[{func.__name__}] Failed after {attempts} attempts. Raising
                        raise
                    sleep_duration = backoff * (2 ** (attempts - 1))
                    print(f"[{func.__name__}] Attempt {attempts} failed: {exc}. Retrying in
                    time.sleep(sleep_duration)
        return wrapper
    return decorator

# Flaky mock service
call_counter = 0

@retry(max_attempts=3, backoff=0.02)
def query_payment_gateway() -> str:
    global call_counter
    call_counter += 1
    if call_counter < 3:
        raise ConnectionResetError("Transient network blip")
    return "PAYMENT_CONFIRMED"

if __name__ == "__main__":
    result = query_payment_gateway()
    print(f"Final outcome: {result}")
```

Run:

```
uv run python retry_decorator.py
```

Output:

```
[query_payment_gateway] Attempt 1 failed: Transient network blip. Retrying in 0.02s...
[query_payment_gateway] Attempt 2 failed: Transient network blip. Retrying in 0.04s...
Final outcome: PAYMENT_CONFIRMED
```

Case 2: Method Decorators: `@classmethod` vs `@staticmethod`

- `@classmethod`: Receives the class object (`cls`) as its first argument. Ideal for alternative factory constructors.
- `@staticmethod`: Receives neither `self` nor `cls`. Acts as a plain function grouped inside the class namespace.

```
# class_and_static_methods.py
class IPAddress:
    def __init__(self, octets: tuple[int, int, int, int]) -> None:
        self.octets = octets

    @classmethod
    def from_string(cls, ip_str: str) -> "IPAddress":
        """Alternative factory constructor parsing a dotted-quad string."""
        parts = tuple(int(p) for p in ip_str.split("."))
        if len(parts) != 4 or any(not (0 <= o <= 255) for o in parts):
            raise ValueError(f"Invalid IPv4 string: {ip_str}")
        return cls(parts)  # type: ignore[arg-type]

    @staticmethod
    def is_private_range(first_octet: int) -> bool:
        """Utility check requiring neither instance nor class state."""
        return first_octet in (10, 172, 192)

    def __repr__(self) -> str:
        return f"IPAddress({'.'.join(str(o) for o in self.octets)})"

if __name__ == "__main__":
    ip = IPAddress.from_string("192.168.1.1")
    print("Constructed IP:", repr(ip))
    print(f"Is private?   : {IPAddress.is_private_range(ip.octets[0])}")
```

Run:

```
uv run python class_and_static_methods.py
```

Output:

```
Constructed IP: IPAddress(192.168.1.1)
Is private?   : True
```

Case 3: Class Decorators for Automated Registration

Decorators can wrap classes as well as functions, allowing automatic registration into a centralized registry:

```python
# plugin_registry.py
from typing import Any

PLUGIN_REGISTRY: dict[str, type] = {}

def register_plugin(plugin_name: str) -> Any:
    def class_decorator(cls: type) -> type:
        PLUGIN_REGISTRY[plugin_name] = cls
        return cls
    return class_decorator

@register_plugin("json_exporter")
class JSONExporter:
    def export(self, data: Any) -> str:
        return f"Exported {data} as JSON"

@register_plugin("csv_exporter")
class CSVExporter:
    def export(self, data: Any) -> str:
        return f"Exported {data} as CSV"

if __name__ == "__main__":
    print("Registered plugins:", list(PLUGIN_REGISTRY.keys()))
    exporter_cls = PLUGIN_REGISTRY["json_exporter"]
    exporter_instance = exporter_cls()
    print(exporter_instance.export({"status": "ok"}))
```

Run:

```
uv run python plugin_registry.py
```

Output:

```
Registered plugins: ['json_exporter', 'csv_exporter']
Exported {'status': 'ok'} as JSON
```

Pitfalls

Pitfall 1: Omitting `@functools.wraps`

Without `@functools.wraps(func)`, the wrapper function overwrites the decorated function's `__name__`, `__doc__`, and `__annotations__`. This breaks debugging tools, loggers, and test runners that rely on function reflection.

Pitfall 2: Decorator Definition-Time vs Execution-Time Confusion

Code in the outer decorator body runs **immediately at import time** when Python compiles the module. Only code inside `wrapper()` runs when the decorated function is actually called.

Exercises

1. Write a decorator `@log_arguments` that prints the positional and keyword arguments passed into the decorated function upon each invocation.
2. Implement an `@enforce_types` decorator that checks if the runtime types of arguments match their type annotations.
3. Write a `@singleton` class decorator that ensures only one instance of the decorated class can ever be created.
4. Create a `@rate_limited(max_per_second=5)` decorator using `time.monotonic()` to throttle fast callers.

Further reading

- PEP 318: *Decorators for Functions and Methods*.
- Python Standard Library: `functools.wraps`.
- Python Language Reference: *Function definitions*.

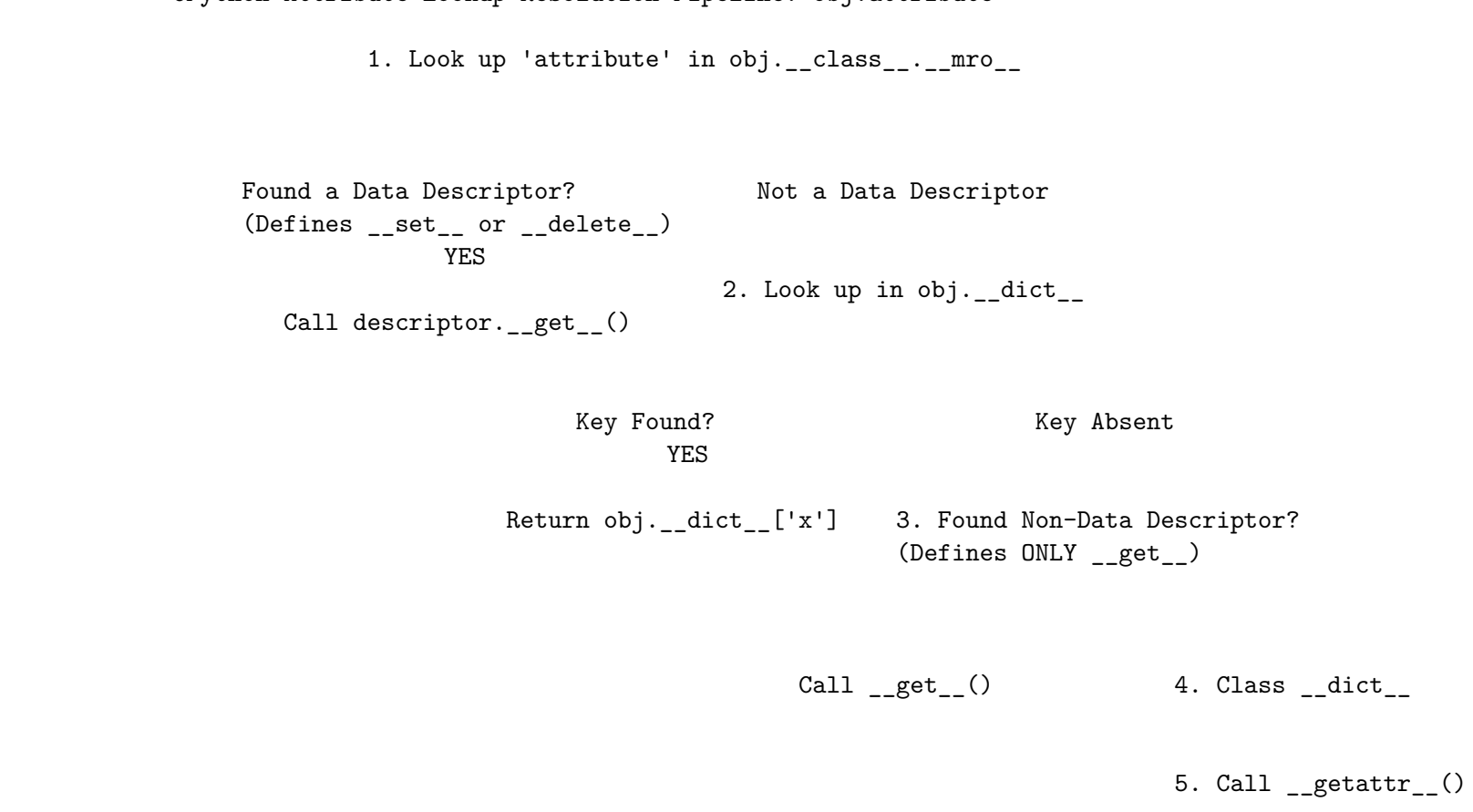
Descriptors and the Attribute Access Protocol

Descriptors and the Attribute Access Protocol

After reading this chapter, you will master Python's descriptor protocol (`__get__`, `__set__`, `__delete__`, and `__set_name__`), analyze the exact priority order of attribute lookup in CPython, differentiate data descriptors from non-data descriptors, implement reusable type-validation descriptors, and uncover how `@property` and `@classmethod` work under the hood.

Mental model

In Python, accessing an attribute (`obj.attribute`) does not merely read a memory offset. Instead, it triggers an evaluation pipeline. A **descriptor** is an object that customizes what happens when an attribute is accessed, modified, or deleted on another class:



The Descriptor Protocol Methods

```
class Descriptor:
    def __set_name__(self, owner: type, name: str) -> None:
        """Called at class creation time; informs descriptor of variable name."""

    def __get__(self, instance: object | None, owner: type) -> object:
        """Called when attribute is retrieved (obj.attr)."""

    def __set__(self, instance: object, value: object) -> None:
        """Called when attribute is assigned (obj.attr = val). (Makes it a Data Descriptor).

    def __delete__(self, instance: object) -> None:
        """Called when attribute is deleted (del obj.attr). (Makes it a Data Descriptor)."""
```

Minimal example

Save as `descriptor_mechanics.py`:

```
# descriptor_mechanics.py
from typing import Any

class PositiveInteger:
    """Data descriptor that validates assigned integers are strictly positive (> 0)."""

    def __set_name__(self, owner: type, name: str) -> None:
        # Automatically capture the attribute name (e.g. 'port' or 'workers')
        self.private_name = f"_{name}"

    def __get__(self, instance: Any, owner: type) -> Any:
        if instance is None:
            # Accessed on the class itself (e.g. ServerConfig.port)
            return self
        return getattr(instance, self.private_name, None)

    def __set__(self, instance: Any, value: Any) -> None:
        if not isinstance(value, int) or value <= 0:
            raise ValueError(f"Attribute '{self.private_name[1:]}' must be a positive intege
        setattr(instance, self.private_name, value)

class ServerConfig:
    port = PositiveInteger()
    workers = PositiveInteger()
```

```
    def __init__(self, port: int, workers: int) -> None:
        self.port = port          # Triggers PositiveInteger.__set__
        self.workers = workers    # Triggers PositiveInteger.__set__

def main() -> None:
    # 1. Valid assignment
    cfg = ServerConfig(port=8080, workers=4)
    print(f"Valid ServerConfig: port={cfg.port}, workers={cfg.workers}")

    # 2. Validation rejection
    try:
        cfg.port = -1
    except ValueError as err:
        print(f"Caught expected validation failure: {err}")

if __name__ == "__main__":
    main()
```

Run via `uv run python descriptor_mechanics.py`:

```
Valid ServerConfig: port=8080, workers=4
Caught expected validation failure: Attribute 'port' must be a positive integer, got: -1
```

Worked examples

Case 1: How `@property` Works Under the Hood

The built-in `@property` decorator is actually a standard data descriptor implemented in C. Writing a pure-Python property replica reveals how getter, setter, and deleter methods bind together:

```
# property_replica.py
from collections.abc import Callable
from typing import Any

class CustomProperty:
    """Pure-Python implementation of the built-in property descriptor."""

    def __init__(
        self,
        fget: Callable[[Any], Any] | None = None,
        fset: Callable[[Any, Any], None] | None = None,
    ) -> None:
        self.fget = fget
```

```
        self.fset = fset

    def __get__(self, instance: Any, owner: type) -> Any:
        if instance is None:
            return self
        if self.fget is None:
            raise AttributeError("Unreadable attribute")
        return self.fget(instance)

    def __set__(self, instance: Any, value: Any) -> None:
        if self.fset is None:
            raise AttributeError("Can't set attribute (read-only)")
        self.fset(instance, value)

    def setter(self, fset: Callable[[Any, Any], None]) -> "CustomProperty":
        """Return a new descriptor instance with the setter attached."""
        return CustomProperty(fget=self.fget, fset=fset)

class TemperatureSensor:
    def __init__(self, celsius: float) -> None:
        self._celsius = celsius

    @CustomProperty
    def celsius(self) -> float:
        return self._celsius

    @celsius.setter
    def celsius(self, value: float) -> None:
        if value < -273.15:
            raise ValueError("Temperature below absolute zero is physically impossible!")
        self._celsius = value

def main() -> None:
    sensor = TemperatureSensor(25.0)
    print(f"Initial temperature: {sensor.celsius} C")

    sensor.celsius = 32.5
    print(f"Updated temperature: {sensor.celsius} C")

    try:
        sensor.celsius = -300.0
    except ValueError as err:
        print(f"Caught absolute zero error: {err}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python property_replica.py
```

Output:

```
Initial temperature: 25.0 C
Updated temperature: 32.5 C
Caught absolute zero error: Temperature below absolute zero is physically impossible!
```

Case 2: Lazy Cached Properties with Non-Data Descriptors

A **non-data descriptor** defines only `__get__`. Because it lacks `__set__`, Python's lookup rule dictates that entries in `instance.__dict__` take precedence over it! This mechanism enables efficient lazy-evaluation caches:

```
# lazy_cached_property.py
import time
from collections.abc import Callable
from typing import Any

class LazyCachedProperty:
    """Non-data descriptor: computes once, stores directly in instance.__dict__."""

    def __init__(self, func: Callable[[Any], Any]) -> None:
        self.func = func
        self.attr_name = func.__name__

    def __get__(self, instance: Any, owner: type) -> Any:
        if instance is None:
            return self
        print(f"  [Computing] Running expensive calculation for '{self.attr_name}'...")
        val = self.func(instance)
        # Store directly in the instance dict!
        # Subsequent lookups will find it in instance.__dict__ and bypass this descriptor en
        instance.__dict__[self.attr_name] = val
        return val

class CloudNodeMetrics:
    def __init__(self, node_id: str) -> None:
        self.node_id = node_id

    @LazyCachedProperty
    def historical_aggregate(self) -> dict[str, float]:
        time.sleep(0.01)  # Simulate expensive database aggregation
```

```
        return {"avg_cpu": 72.4, "p99_latency_ms": 142.1}

def main() -> None:
    node = CloudNodeMetrics("srv-us-east-42")

    print("First access (triggers computation):")
    print(f"Result: {node.historical_aggregate}")

    print("\nSecond access (retrieved instantly from instance.__dict__):")
    print(f"Result: {node.historical_aggregate}")

    print(f"\nVerifying stored attributes in node.__dict__:")
    print(f"Keys: {list(node.__dict__.keys())}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python lazy_cached_property.py
```

Output:

```
First access (triggers computation):
  [Computing] Running expensive calculation for 'historical_aggregate'...
Result: {'avg_cpu': 72.4, 'p99_latency_ms': 142.1}

Second access (retrieved instantly from instance.__dict__):
Result: {'avg_cpu': 72.4, 'p99_latency_ms': 142.1}

Verifying stored attributes in node.__dict__:
Keys: ['node_id', 'historical_aggregate']
```

Pitfalls

Pitfall 1: Storing Value on the Descriptor Instance

Descriptors are instantiated **once at the class level**, shared across all instances of that class. Storing user state on `self.value` causes every class instance to overwrite the exact same variable:

```
# THE TRAP: Shared state across all class instances!
class BrokenDescriptor:
    def __get__(self, instance, owner):
        return self.val
```

```
    def __set__(self, instance, val):
        self.val = val  # BUG: Stored on the descriptor, shared across all objects!

class Worker:
    worker_id = BrokenDescriptor()

w1 = Worker()
w2 = Worker()
w1.worker_id = "worker-01"
w2.worker_id = "worker-02"

print(f"w1 ID: {w1.worker_id}")  # Prints 'worker-02'! w1 was overwritten!
```

```
# THE FIX: Store the state inside the instance, keyed by private_name
class SafeDescriptor:
    def __set_name__(self, owner, name):
        self.storage_key = f"_{name}"
    def __get__(self, instance, owner):
        if instance is None: return self
        return getattr(instance, self.storage_key, None)
    def __set__(self, instance, val):
        setattr(instance, self.storage_key, val)
```

Pitfall 2: Forgetting `if instance is None:` in `__get__`

When an attribute is accessed via the class itself (`MyClass.attr`), the `instance` argument passed to `__get__` is `None`:

```
class MyDescriptor:
    def __get__(self, instance, owner):
        # BUG: if instance is None, instance.data raises AttributeError!
        # if instance is None: return self
        return instance.data

# THE FIX:
class RobustDescriptor:
    def __get__(self, instance, owner):
        if instance is None:
            return self  # Return descriptor object when inspected via class
        return getattr(instance, "_data", None)
```

Exercises

1. Implement a `TypedString` descriptor that validates that assigned values are instances of `str` and have a length between `min_len` and `max_len`.
2. Replicate `functools.cached_property` using a non-data descriptor that caches expensive method results directly onto `instance.__dict__`.
3. Explain why defining `__set__` (even as a no-op that raises `AttributeError`) prevents assignments to `instance.__dict__` from shadowing the descriptor.
4. Implement a `ReadOnly` descriptor that allows an attribute to be assigned exactly once (during `__init__`), but raises `RuntimeError` on subsequent modification attempts.
5. Create a descriptor `BoundedFloat(min_val, max_val)` and use it to validate percentage metrics on a `SystemMonitor` class.

Further reading

- Raymond Hettinger: *Descriptor HowTo Guide* (Official Python Documentation).
- Python Language Reference: *Implementing Descriptors*.
- PEP 487: *Simpler customisation of class creation* (`__set_name__` protocol).

Metaclasses and Class Construction Mechanics

Metaclasses and Class Construction Mechanics

After reading this chapter, you will master the internal lifecycle of Python class construction, trace how the CPython VM builds classes using `type`, leverage `__init_subclass__` (PEP 487) for zero-magic plugin registration, capture declaration order using `__prepare__`, enforce architectural invariants via custom metaclasses, and resolve metaclass inheritance conflicts.

Mental model

In Python, **classes are objects**. And just as ordinary objects are instances of classes, classes themselves are instances of **metaclasses**:

```
Ordinary Object:
  instance = Server("web-01")      instance is an instance of class Server
                                   type(instance) is Server

Class Object:
  Server                           Server is an instance of metaclass 'type'
                                   type(Server) is type
```

The CPython Class Construction Pipeline

When Python encounters a `class` statement, it executes a 5-stage creation pipeline:

```
                    1. Determine Metaclass
    (Inspect explicit metaclass=..., base classes, or default to 'type')


        2. Call metaclass.__prepare__(name, bases, **kwargs)
          Returns namespace mapping (e.g. OrderedDict or custom dict)


            3. Execute Class Body in that Namespace
                (Methods and attributes are defined)


              4. Call metaclass(name, bases, namespace)
            Invokes metaclass.__new__() to allocate the PyTypeObject
            Invokes metaclass.__init__() to initialize the class
```

```
        5. Trigger __init_subclass__()
    Notifies all base classes that a subclass was created
```

Minimal example

Save as `class_construction_pipeline.py`:

```python
# class_construction_pipeline.py
from typing import Any

class VerboseMeta(type):
    @classmethod
    def __prepare__(metacls, name: str, bases: tuple[type, ...], **kwargs: Any) -> dict[str,
        print(f"  [1. __prepare__] Allocating namespace for class '{name}'")
        return dict()

    def __new__(
        metacls,
        name: str,
        bases: tuple[type, ...],
        namespace: dict[str, Any],
        **kwargs: Any,
    ) -> type:
        print(f"  [2. __new__]     Allocating PyTypeObject for '{name}' with {len(namespace)
        cls_obj = super().__new__(metacls, name, bases, namespace)
        return cls_obj

    def __init__(cls, name: str, bases: tuple[type, ...], namespace: dict[str, Any], **kwarg
        print(f"  [3. __init__]    Initializing class object '{name}'")
        super().__init__(name, bases, namespace)

class ServiceEndpoint(metaclass=VerboseMeta):
    timeout_seconds = 30

    def handle_request(self) -> str:
        return "200 OK"

def main() -> None:
    print("Class definition execution complete.")
    print(f"Type of ServiceEndpoint: {type(ServiceEndpoint).__name__}")
    endpoint = ServiceEndpoint()
    print(f"Response: {endpoint.handle_request()}")
```

```
if __name__ == "__main__":
    main()
```

Run via `uv run python class_construction_pipeline.py`:

```
  [1. __prepare__] Allocating namespace for class 'ServiceEndpoint'
  [2. __new__]     Allocating PyTypeObject for 'ServiceEndpoint' with 7 attributes
  [3. __init__]    Initializing class object 'ServiceEndpoint'
Class definition execution complete.
Type of ServiceEndpoint: VerboseMeta
Response: 200 OK
```

Notice: All three metaclass steps executed at **import/definition time**, before a single instance of `ServiceEndpoint` was ever created!

Worked examples

Case 1: The Modern Alternative: Zero-Boilerplate Plugins with `__init_subclass__`

Prior to Python 3.6, building auto-registering plugin architectures required custom metaclasses. PEP 487 introduced `__init_subclass__`, which provides 95% of metaclass capabilities with standard inheritance:

```
# plugin_registry.py
from typing import ClassVar

class PaymentProcessor:
    """Base class that automatically registers all subclasses into a global dispatch map."""
    registry: ClassVar[dict[str, type["PaymentProcessor"]]] = {}

    def __init_subclass__(cls, provider_code: str, **kwargs: object) -> None:
        super().__init_subclass__(**kwargs)
        provider_key = provider_code.strip().lower()
        if provider_key in cls.registry:
            raise ValueError(f"Duplicate payment provider code '{provider_key}' registered b
        cls.registry[provider_key] = cls
        print(f"  [Registered Plugin] Provider: '{provider_key}' -> {cls.__name__}")

    def process_charge(self, amount_cents: int) -> str:
        raise NotImplementedError

# Subclasses register automatically via keyword arguments in their class definition!
class StripeProcessor(PaymentProcessor, provider_code="stripe"):
```

```
    def process_charge(self, amount_cents: int) -> str:
        return f"Charged ${amount_cents / 100:.2f} via Stripe API"

class PayPalProcessor(PaymentProcessor, provider_code="paypal"):
    def process_charge(self, amount_cents: int) -> str:
        return f"Charged ${amount_cents / 100:.2f} via PayPal Express"

def main() -> None:
    print("\n--- Available Processors ---")
    for code, cls_type in PaymentProcessor.registry.items():
        print(f"  Code: {code:8} -> Handler: {cls_type.__name__}")

    # Dynamic factory dispatch
    selected = "stripe"
    handler = PaymentProcessor.registry[selected]()
    print(f"\nExecution: {handler.process_charge(4999)}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python plugin_registry.py
```

Output:

```
  [Registered Plugin] Provider: 'stripe' -> StripeProcessor
  [Registered Plugin] Provider: 'paypal' -> PayPalProcessor

--- Available Processors ---
  Code: stripe   -> Handler: StripeProcessor
  Code: paypal   -> Handler: PayPalProcessor

Execution: Charged $49.99 via Stripe API
```

Case 2: Enforcing Architectural Invariants with a Custom Metaclass

In large engineering codebases, you may want to enforce architectural policies—for example, requiring that every subclass of `CommandController` implements an asynchronous or synchronous `execute()` method and contains a docstring:

```
# policy_enforcer.py
class ArchitectureEnforcer(type):
    """Metaclass that validates class structure at definition time."""
```

```
    def __new__(
        metacls,
        name: str,
        bases: tuple[type, ...],
        namespace: dict[str, object],
    ) -> type:
        # Skip validation on the root base class itself
        if bases:
            # 1. Enforce presence of class docstring
            if not namespace.get("__doc__"):
                raise TypeError(f"Architecture violation: Class '{name}' must provide a docs

            # 2. Enforce implementation of 'execute'
            if "execute" not in namespace or not callable(namespace["execute"]):
                raise TypeError(f"Architecture violation: Class '{name}' must implement an '

        return super().__new__(metacls, name, bases, namespace)

class BaseCommand(metaclass=ArchitectureEnforcer):
    pass

class BackupDatabaseCommand(BaseCommand):
    """Safely snapshot and archive the primary database cluster."""

    def execute(self) -> str:
        return "Backup completed successfully."

def main() -> None:
    cmd = BackupDatabaseCommand()
    print(f"Command '{BackupDatabaseCommand.__name__}' passed architectural validation.")
    print(f"Docstring: {cmd.__doc__}")
    print(f"Execution: {cmd.execute()}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python policy_enforcer.py
```

Output:

```
Command 'BackupDatabaseCommand' passed architectural validation.
Docstring: Safely snapshot and archive the primary database cluster.
Execution: Backup completed successfully.
```

Case 3: Capturing Attribute Declaration Order with `__prepare__`

In serialization libraries and ORM models, field ordering matters for binary layouts or CSV column headers. By customizing `__prepare__`, a metaclass can intercept and track attributes exactly as they are declared in source code:

```
# schema_order_tracker.py
from typing import Any

class OrderedSchemaMeta(type):
    @classmethod
    def __prepare__(metacls, name: str, bases: tuple[type, ...]) -> dict[str, Any]:
        # Return a dictionary that records the order of field assignments
        return dict()

    def __new__(
        metacls,
        name: str,
        bases: tuple[type, ...],
        namespace: dict[str, Any],
    ) -> type:
        # Extract declared fields (ignoring private/dunder attributes)
        fields = [k for k, v in namespace.items() if not k.startswith("_")]
        namespace["_declared_fields"] = tuple(fields)
        return super().__new__(metacls, name, bases, namespace)

class CSVRecordModel(metaclass=OrderedSchemaMeta):
    user_id = 0
    username = ""
    email = ""
    created_at = 0

def main() -> None:
    print(f"Model: {CSVRecordModel.__name__}")
    print("Detected fields in exact declaration order:")
    for idx, field in enumerate(CSVRecordModel._declared_fields, start=1):
        print(f"  Column #{idx}: {field}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python schema_order_tracker.py
```

Output:

```
Model: CSVRecordModel
```

```
Detected fields in exact declaration order:
  Column #1: user_id
  Column #2: username
  Column #3: email
  Column #4: created_at
```

Pitfalls

Pitfall 1: The Metaclass Conflict Error

When multiple inheritance involves base classes created with different, incompatible metaclasses, Python raises a `TypeError`:

```
class MetaA(type): pass
class MetaB(type): pass

class BaseA(metaclass=MetaA): pass
class BaseB(metaclass=MetaB): pass

# THE TRAP:
# class Derived(BaseA, BaseB): pass
# TypeError: metaclass conflict: the metaclass of a derived class must be a
# (non-strict) subclass of the metaclasses of all its bases

# THE FIX: Create a combined metaclass that inherits from both
class CombinedMeta(MetaA, MetaB): pass

class Derived(BaseA, BaseB, metaclass=CombinedMeta):
    pass

print(f"Derived successfully created with metaclass: {type(Derived).__name__}")
```

Pitfall 2: Overusing Metaclasses when Simpler Tools Suffice

Martelli's Rule of Metaclasses: "Metaclasses are deeper magic than 99% of users should ever worry about." Before reaching for a metaclass, ask if simpler tools solve the problem:

```
If you need to...                                    Prefer...
```

Hook subclass creation & register plugins	`__init_subclass__` (PEP 487)
Validate, mutate, or wrap an existing class	Class Decorators (`@register_class`)
Customize individual attribute access or validation	Descriptors (`__set_name__`)
Intercept namespace creation before class body runs	Metaclass (`__prepare__`)

Exercises

1. Using `__init_subclass__`, implement an automatic event-handler registry where subclasses specify `event_name="user_login"` and are added to a centralized dispatcher.
2. Create a metaclass that automatically converts all method names defined in CamelCase to `snake_case` in the class namespace.
3. Write an architectural enforcement metaclass that rejects any class whose name does not end with `"Service"` or `"Controller"`.
4. Demonstrate how `type(name, bases, dict)` can be used to construct a complete, functional class dynamically at runtime without using the `class` keyword.
5. Create a metaclass that freezes class attributes so that attempting to add new attributes after class creation raises `AttributeError`.

Further reading

- PEP 487: *Simpler customisation of class creation* (`__init_subclass__`).
- PEP 3115: *Metaclasses in Python 3000* (`__prepare__`).
- Luciano Ramalho: *Fluent Python* (Chapter 21: Class Metaprogramming).
- Brett Slatkin: *Effective Python* (Item 48: Validate Subclasses with `__init_subclass__`).

Abstract Base Classes and Structural Protocols

Abstract Base Classes and Structural Protocols

After reading this chapter, you will master Python's two complementary interface design paradigms: nominal enforcement using Abstract Base Classes (`abc.ABC`), and structural static duck typing using Protocols (`typing.Protocol` from PEP 544). You will enforce runtime contracts, register virtual subclasses, implement `@runtime_checkable` interfaces, and avoid decorator ordering traps.

Mental model

Python provides two approaches for defining and enforcing contracts between components:

```
Nominal Subtyping (abc.ABC):
  "Explicit Inheritance"
  Class hierarchy defines type.
  Checked at instantiation time via @abstractmethod.


        StorageBackend   (abc.ABC)


                inherits

        S3Storage        (Concrete)



Structural Subtyping (typing.Protocol - PEP 544):
  "Static Duck Typing"
  Shape and attributes define type.
  No inheritance required! Zero coupling between libraries.


        LogWriter        (Protocol)     ConsoleLogger    (No base class!)
        - write(msg)     <.........     - write(msg)
                  Matches
```

Comparison Matrix

Property	Abstract Base Class (abc.ABC)	Protocol (typing.Protocol)

Philosophy	Nominal: Must inherit or reg.	Structural: "Shape" matches
Enforcement Point	Runtime (at instantiation)	Static (Mypy/Pyright) + opt.
Third-party adaptability	Requires ABC.register(cls)	Automatic if signature fits
Primary Use Case	Framework base abstractions	Decoupled library interfaces

---

Minimal example

Save as `interfaces_overview.py`:

```python
# interfaces_overview.py
from abc import ABC, abstractmethod
from typing import Protocol, runtime_checkable

# 1. Nominal Interface with abc.ABC
class MetricPublisher(ABC):
    """Nominal interface: Subclasses MUST implement publish_metric."""

    @abstractmethod
    def publish_metric(self, name: str, value: float) -> None:
        """Send a single metric measurement to the telemetry collector."""

class CloudWatchPublisher(MetricPublisher):
    def publish_metric(self, name: str, value: float) -> None:
        print(f"[CloudWatch] Published {name} = {value}")

# 2. Structural Interface with typing.Protocol (PEP 544)
@runtime_checkable
class Closable(Protocol):
    """Structural interface: Any object with a close() method satisfies this contract."""
    def close(self) -> None: ...

class DatabaseConnection:
    """Note: Does NOT inherit from Closable, but satisfies the structural contract!"""
    def close(self) -> None:
        print("[Database] Connection pool cleanly closed.")

def terminate_resource(resource: Closable) -> None:
    # Runtime verification via @runtime_checkable
    if isinstance(resource, Closable):
        resource.close()

def main() -> None:
```

```
    # Test ABC enforcement
    pub = CloudWatchPublisher()
    pub.publish_metric("cpu_load", 42.5)

    # Test Structural Protocol
    db = DatabaseConnection()
    terminate_resource(db)

if __name__ == "__main__":
    main()
```

Run via `uv run python interfaces_overview.py`:

```
[CloudWatch] Published cpu_load = 42.5
[Database] Connection pool cleanly closed.
```

Worked examples

Case 1: Enforcing Framework Contracts with Abstract Base Classes

When building an enterprise data ingestion pipeline, you want to enforce that every storage connector implements reading, writing, and health-checking before allowing the class to be instantiated:

```
# storage_connector.py
from abc import ABC, abstractmethod
from typing import Any

class StorageConnector(ABC):
    """Abstract driver interface for multi-cloud blob stores."""

    def __init__(self, endpoint_url: str) -> None:
        self.endpoint_url = endpoint_url

    @abstractmethod
    def upload_blob(self, key: str, payload: bytes) -> bool:
        """Upload raw bytes to cloud storage under key."""

    @abstractmethod
    def download_blob(self, key: str) -> bytes:
        """Fetch raw bytes from cloud storage."""

    @property
    @abstractmethod
    def is_connected(self) -> bool:
```

```
        """Return True if connection to storage endpoint is active."""

class LocalFileStorage(StorageConnector):
    def __init__(self, directory: str) -> None:
        super().__init__(endpoint_url=f"file:///{directory}")
        self._active = True

    @property
    def is_connected(self) -> bool:
        return self._active

    def upload_blob(self, key: str, payload: bytes) -> bool:
        print(f"  [LocalFile] Stored {len(payload)} bytes to key '{key}'")
        return True

    def download_blob(self, key: str) -> bytes:
        return b"mock_payload_content"

def main() -> None:
    storage = LocalFileStorage("/var/data")
    print(f"Storage Endpoint: {storage.endpoint_url}")
    print(f"Connected:        {storage.is_connected}")
    storage.upload_blob("config.json", b'{"status": "ok"}')

if __name__ == "__main__":
    main()
```

Run:

```
uv run python storage_connector.py
```

Output:

```
Storage Endpoint: file:///var/data
Connected:        True
  [LocalFile] Stored 16 bytes to key 'config.json'
```

Case 2: Structural Decoupling with Protocols Across Unrelated Packages

In microservices and decoupled libraries, you frequently want to accept any object that can serialize itself to JSON without forcing the caller to inherit from your library's base classes:

```
# decoupled_service.py
from typing import Protocol, runtime_checkable
```

```python
@runtime_checkable
class Serializable(Protocol):
    def to_json(self) -> str:
        """Convert object state into a valid JSON string."""
        ...

class AuditEvent:
    """An external domain object with NO dependency on our library!"""
    def __init__(self, action: str, user_id: int) -> None:
        self.action = action
        self.user_id = user_id

    def to_json(self) -> str:
        return f'{{"action": "{self.action}", "user_id": {self.user_id}}}'

def dispatch_event(event: Serializable) -> None:
    """Accepts any object conforming to the Serializable structure."""
    if not isinstance(event, Serializable):
        raise TypeError(f"Object of type {type(event).__name__} does not implement to_json()
    print(f"Transmitted to EventHub: {event.to_json()}")

def main() -> None:
    audit = AuditEvent("USER_PASSWORD_RESET", 4821)
    dispatch_event(audit)

if __name__ == "__main__":
    main()
```

Run:

```
uv run python decoupled_service.py
```

Output:

```
Transmitted to EventHub: {"action": "USER_PASSWORD_RESET", "user_id": 4821}
```

Case 3: Virtual Subclasses via `ABC.register()`

Sometimes you want an existing third-party class (which you cannot edit) to be recognized as a subclass of your ABC without monkey-patching or subclassing it. Python allows **registering virtual subclasses**:

```python
# virtual_subclass.py
from abc import ABC, abstractmethod
```

```
class SequenceStream(ABC):
    @abstractmethod
    def read_chunk(self, size: int) -> bytes:
        raise NotImplementedError

class ThirdPartyHardwareBuffer:
    """A third-party class provided by a compiled vendor package."""
    def read_chunk(self, size: int) -> bytes:
        return b"\x00" * size

# Register ThirdPartyHardwareBuffer as a virtual subclass of SequenceStream!
SequenceStream.register(ThirdPartyHardwareBuffer)

def process_stream(stream: SequenceStream) -> None:
    print(f"Processing verified stream: {type(stream).__name__}")
    data = stream.read_chunk(4)
    print(f"  Read bytes: {data}")

def main() -> None:
    hw = ThirdPartyHardwareBuffer()

    # isinstance and issubclass now recognize the virtual inheritance!
    print(f"Is hw an instance of SequenceStream? {isinstance(hw, SequenceStream)}")
    print(f"Is class a subclass of SequenceStream? {issubclass(ThirdPartyHardwareBuffer, Seq

    process_stream(hw)

if __name__ == "__main__":
    main()
```

Run:

```
uv run python virtual_subclass.py
```

Output:

```
Is hw an instance of SequenceStream? True
Is class a subclass of SequenceStream? True
Processing verified stream: ThirdPartyHardwareBuffer
  Read bytes: b'\x00\x00\x00\x00'
```

Pitfalls

Pitfall 1: Attempting to Instantiate an Incomplete ABC

If any method marked with `@abstractmethod` is not overridden in a subclass, CPython **forbids instantiation** and raises `TypeError` at construction time:

```
from abc import ABC, abstractmethod

class BaseWorker(ABC):
    @abstractmethod
    def process_job(self) -> None: pass

class IncompleteWorker(BaseWorker):
    pass

# THE TRAP:
try:
    w = IncompleteWorker()
except TypeError as err:
    print(f"Caught expected error: {err}")
```

Output:

```
Caught expected error: Can't instantiate abstract class IncompleteWorker without an implementat
```

Pitfall 2: Inverting Decorator Order with `@property` and `@abstractmethod`

When creating an abstract property, `@property` **must** be the outermost decorator, and `@abstractmethod` must be the innermost:

```
from abc import ABC, abstractmethod

# THE TRAP (WRONG ORDER):
class BadClass(ABC):
    @abstractmethod
    @property          # BUG: Python does not recognize this as an abstract property!
    def name(self) -> str: pass

# THE FIX (CORRECT ORDER):
class GoodClass(ABC):
    @property
    @abstractmethod    # Correct: @property wraps @abstractmethod
    def name(self) -> str: pass
```

Pitfall 3: Assuming `@runtime_checkable` Validates Types and Signatures

`@runtime_checkable` on a Protocol only checks that an attribute or method **exists and is callable**; it does **not** validate parameter types, return types, or argument counts at runtime!

```
from typing import Protocol, runtime_checkable

@runtime_checkable
class MathOp(Protocol):
    def compute(self, a: int, b: int) -> int: ...

class BrokenMath:
    # Has a method named 'compute', but signature takes ZERO arguments!
    def compute(self) -> str:
        return "not numbers"

b = BrokenMath()
# isinstance evaluates to True because it only checks for the presence of the method name!
print(f"isinstance check: {isinstance(b, MathOp)}")  # True!
```

Takeaway: Use static type checkers (`mypy` or `pyright`) to verify signatures; treat `@runtime_checkable` as a shallow existence check.

Exercises

1. Define an ABC `CryptographicSigner` with abstract methods `sign(payload: bytes) -> bytes` and `verify(payload: bytes, signature: bytes) -> bool`. Implement a concrete subclass `HMACSigner`.
2. Define a `typing.Protocol` named `Renderable` that requires a `render_html() -> str` method. Write two completely unrelated classes that conform to this protocol without inheriting from it.
3. Demonstrate the error raised when attempting to instantiate an ABC subclass that forgot to implement an abstract `@property`.
4. Implement a custom `__subclasshook__` on an ABC so that any class defining an `.export()` method is dynamically recognized as a subclass via `issubclass()`.
5. Compare the runtime speed of `isinstance(obj, ABC)` vs `isinstance(obj, Protocol)` across 1,000,000 iterations.

Further reading

- PEP 544: *Protocols: Structural subtyping (static duck typing).*
- PEP 3119: *Introducing Abstract Base Classes.*
- Luciano Ramalho: *Fluent Python* (Chapter 13: Interfaces, Protocols, and ABCs).
- Python Documentation: `abc` — *Abstract Base Classes.*

Modules and Packages

Modules and the Import System

Modules and the Import System

After reading this chapter, you will master Python's module architecture, trace the import resolution pipeline through `sys.modules` and `sys.path`, dynamically load modules using `importlib`, and diagnose and resolve circular import deadlocks.

Mental model

In Python, a **module** is simply a `.py` source file executed as a distinct namespace object (`types.ModuleType`). When an `import foo` statement executes, CPython traverses a four-stage pipeline:

```
import foo


[ 1. Check sys.modules Cache ]

     If cached           Return existing module reference immediately

    (Not cached)
[ 2. Finders (sys.meta_path) ]    Search sys.path directories for 'foo.py'

    (Found file)
[ 3. Loaders (FileLoader) ]       Compile bytecode, allocate empty ModuleType


[ 4. Execute Module Code ]        Run top-level statements, populate __dict__,
                                    cache in sys.modules, bind name in caller
```

Because modules are cached in `sys.modules` upon first import, subsequent imports of the same module across threads or files return the cached instance with **zero disk I/O or recompilation**.

Minimal example

Save as `import_internals.py`:

```python
# import_internals.py
import importlib
import sys
from types import ModuleType

def inspect_import_cache(module_name: str) -> None:
    print(f"Is '{module_name}' cached in sys.modules? {module_name in sys.modules}")

def dynamically_load_module(module_name: str) -> ModuleType:
    """Programmatically import a module by string name using importlib."""
    print(f"\nDynamically loading '{module_name}' via importlib...")
    mod = importlib.import_module(module_name)
    print(f"Module file path : {getattr(mod, '__file__', 'built-in')}")
    print(f"Module docstring : {mod.__doc__[:40] if mod.__doc__ else 'None'}...")
    return mod

def main() -> None:
    inspect_import_cache("json")

    # Dynamic import
    json_mod = dynamically_load_module("json")
    inspect_import_cache("json")

    # Use dynamically loaded module
    serialized = json_mod.dumps({"status": "healthy", "code": 200})
    print(f"Serialized output: {serialized}")

if __name__ == "__main__":
    main()
```

Run via `uv run python import_internals.py`:

```
Is 'json' cached in sys.modules? False

Dynamically loading 'json' via importlib...
Module file path : .../lib/python3.14/json/__init__.py
Module docstring : JSON (JavaScript Object Notation) <http:...
Is 'json' cached in sys.modules? True
Serialized output: {"status": "healthy", "code": 200}
```

Worked examples

Case 1: Dynamic Driver Registry with `importlib`

In cloud automation and backend platforms, you frequently need to load storage, networking, or database drivers specified by external configuration strings:

```
# driver_loader.py
import importlib
from typing import Any

class DriverLoader:
    @staticmethod
    def load_formatter(format_name: str) -> Any:
        # Standard library formatters mapping
        known_modules = {
            "json": "json",
            "csv": "csv",
        }
        if format_name not in known_modules:
            raise ValueError(f"Unsupported driver: {format_name}")

        mod = importlib.import_module(known_modules[format_name])
        return mod

if __name__ == "__main__":
    loader = DriverLoader()
    driver = loader.load_formatter("json")
    data = {"metric": "cpu", "value": 85.5}
    print(f"Driver loaded dynamically: {driver.__name__}")
    print(f"Formatted: {driver.dumps(data)}")
```

Run:

```
uv run python driver_loader.py
```

Output:

```
Driver loaded dynamically: json
Formatted: {"metric": "cpu", "value": 85.5}
```

Case 2: Diagnosing and Breaking Circular Imports

A circular import occurs when Module A imports Module B at top-level, but Module B imports Module A before Module A has finished executing its definition pass.

```
Circular Import Deadlock:
  module_a.py  (imports)  module_b.py


            (imports)
```

When Python hits `from module_a import Service` inside `module_b`, `module_a` is still being executed; `Service` does not yet exist in `module_a.__dict__`, raising: `ImportError: cannot import name 'Service' from partially initialized module 'module_a'`.

The Fixes:

1. **Move shared domain types/interfaces to a third, leaf module** (`types.py` or `models.py`) imported by both.
2. **Use local imports**: Import the dependency inside the specific function that uses it, rather than at module top-level.
3. **Use `typing.TYPE_CHECKING`**: If the import is only needed for type annotations, gate it behind `if TYPE_CHECKING:`.

```python
# circular_fix_pattern.py
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    # Only imported during static analysis (mypy/pyright), NEVER at runtime!
    from collections.abc import Sequence

def process_batch(items: "Sequence[str]") -> int:
    return len(items)

if __name__ == "__main__":
    print("Function defined cleanly without circular runtime dependency.")
    print("Result:", process_batch(["task-1", "task-2"]))
```

Run:

```
uv run python circular_fix_pattern.py
```

Output:

```
Function defined cleanly without circular runtime dependency.
Result: 2
```

Case 3: The `if __name__ == "__main__":` Boundary

When a Python file is invoked directly (`python script.py`), CPython assigns its `__name__` attribute to `"__main__"`. When imported by another file (`import script`), `__name__` is set to the module's file stem (`"script"`).

```
# dual_mode_module.py
def calculate_throughput(bytes_transferred: int, duration_seconds: float) -> float:
    """Pure library function: usable when imported."""
    if duration_seconds <= 0:
        raise ValueError("Duration must be greater than zero.")
    return bytes_transferred / duration_seconds

def _cli_entrypoint() -> None:
    """CLI testing routine: runs ONLY when executed directly."""
    sample_bytes = 104_857_600  # 100 MB
    duration = 2.5
    rate_mb = calculate_throughput(sample_bytes, duration) / (1024 * 1024)
    print(f"[CLI Runner] Transfer rate: {rate_mb:.2f} MB/s")

if __name__ == "__main__":
    _cli_entrypoint()
```

Run:

```
uv run python dual_mode_module.py
```

Output:

```
[CLI Runner] Transfer rate: 40.00 MB/s
```

When another module runs `from dual_mode_module import calculate_throughput`, `_cli_entrypoint()` will not execute.

Pitfalls

Pitfall 1: Standard Library Shadowing

Naming a local script after a standard library module (e.g. creating `math.py`, `random.py`, `test.py`, or `json.py` in your project folder) breaks the import system:

```
Your Directory:
  math.py          (Your script)
  test_math.py     (import math -> loads YOUR math.py instead of CPython's!)
```

Because CPython checks the current working directory first in `sys.path[0]`, your local file shadows the standard library module, causing cryptic `AttributeError: module 'math' has no attribute 'sqrt'` errors.

Exercises

1. Print all paths in `sys.path` and inspect where Python looks for modules on your machine.
2. Write a function that accepts a module name as a string, checks whether it is already loaded in `sys.modules`, and reloads it using `importlib.reload()` if present.
3. Create a simulated two-module circular import and resolve it using local function-level imports.
4. Use `__all__` to restrict the symbols exported when a consumer executes `from my_module import *`.

Further reading

- Python Documentation: *The Import System* (`sys.meta_path`, Finders, Loaders).
- Python Standard Library: `importlib` and `sys.modules`.
- PEP 302: *New Import Hooks*.

Packages, Namespaces, and Public APIs

Packages, Namespaces, and Public APIs

After reading this chapter, you will organize codebases into hierarchical Python packages, structure `__init__.py` to create clean public API façades, contrast standard packages with PEP 420 namespace packages, use relative imports correctly, and control exported symbols using `__all__`.

Mental model

A **package** is a directory that Python treats as a module containing sub-modules or sub-packages.

```
Package Hierarchy:
  my_service/                    Package Directory
    __init__.py                 Marks directory as regular package; defines public API
    core/                       Sub-package
        __init__.py
        engine.py               Module: my_service.core.engine
    transports/                 Sub-package
         __init__.py
         http.py                Module: my_service.transports.http
```

When importing `my_service.core.engine`: 1. `my_service/__init__.py` executes and binds `my_service`. 2. `my_service/core/__init__.py` executes and binds `core` on `my_service`. 3. `my_service/core/engine.py` executes and binds `engine` on `my_service.core`.

Minimal example

Save as `package_facade.py`:

```python
# package_facade.py
import tempfile
from pathlib import Path

def create_sample_package(root: Path) -> None:
    pkg_dir = root / "cloudpkg"
    pkg_dir.mkdir()
```

```
    # Submodule 1: Internal client
    client_code = """
class CloudClient:
    def connect(self) -> str:
        return "Connected to cloud endpoint."
"""
    (pkg_dir / "client.py").write_text(client_code)

    # __init__.py creates a clean public API facade and restricts __all__
    init_code = """
from .client import CloudClient

__all__ = ["CloudClient"]
"""
    (pkg_dir / "__init__.py").write_text(init_code)

def main() -> None:
    import sys
    with tempfile.TemporaryDirectory() as tmpdir:
        tmp_path = Path(tmpdir)
        create_sample_package(tmp_path)

        # Append temporary directory to sys.path to simulate an installed package
        sys.path.insert(0, str(tmp_path))
        try:
            # Consumer imports directly from top-level package facade
            import cloudpkg
            print(f"Package loaded: {cloudpkg.__name__}")
            print(f"Exported symbols (__all__): {cloudpkg.__all__}")

            client = cloudpkg.CloudClient()
            print(f"Client method invocation : {client.connect()}")
        finally:
            sys.path.pop(0)

if __name__ == "__main__":
    main()
```

Run via `uv run python package_facade.py`:

```
Package loaded: cloudpkg
Exported symbols (__all__): ['CloudClient']
Client method invocation : Connected to cloud endpoint.
```

Worked examples

Case 1: Relative vs Absolute Imports

Within a package, files can refer to sibling or parent modules using dot notation (`.`):

```
my_app/
   models/
       __init__.py
       user.py
   services/
       __init__.py
       auth.py           Needs to import User from models
```

Inside `my_app/services/auth.py`:

```
# Absolute import (Recommended for clarity in large apps):
from my_app.models.user import User

# Relative import (Recommended for reusable, relocatable library packages):
from ..models.user import User
from .helpers import local_token_generator  # Sibling import from same directory
```

- `.` refers to the **current package directory**.
- `..` refers to the **parent package directory**.
- `...` refers to the **grandparent package directory**.

Relative imports rely on the module's `__name__` property containing package context. If you run a file containing relative imports directly with `python my_app/services/auth.py`, it fails with `ImportError: attempted relative import with no known parent package`. **Always run package modules using the `-m` flag:**

```
python -m my_app.services.auth
```

Case 2: The `__all__` Contract and `from module import *`

The `__all__` list specifies which symbols are exported when a user executes wildcard imports (`from mypkg import *`), and communicates the supported public API to linters and documentation generators:

```
# api_contract.py
__all__ = ["PublicService", "public_helper"]

class PublicService:
    """Supported public class."""
    pass
```

```python
def public_helper() -> str:
    """Supported public function."""
    return "OK"

def _internal_helper() -> None:
    """Private function: prefixed with underscore."""
    pass

class InternalEngine:
    """Unexported class: omitted from __all__."""
    pass

if __name__ == "__main__":
    print(f"Exported public symbols in module: {__all__}")
```

Run:

```
uv run python api_contract.py
```

Output:

```
Exported public symbols in module: ['PublicService', 'public_helper']
```

Case 3: PEP 420 Namespace Packages

In modern Python, a directory **without** an `__init__.py` file is treated as a **namespace package**. This enables splitting a single top-level package namespace (e.g. `company.core`, `company.storage`, `company.auth`) across completely separate directories, wheels, or git repositories:

```
Repo 1 (Core):
  company/                    (NO __init__.py)
    core/
        engine.py

Repo 2 (Plugins):
  company/                    (NO __init__.py)
    plugins/
        exporter.py
```

When both repositories are installed into `site-packages`, CPython merges them into a single unified `company` namespace at runtime!

Pitfalls

Pitfall 1: Executing Submodules Directly Instead of via `-m`

Running `python pkg/sub/file.py` sets `sys.path[0]` to `pkg/sub/`, stripping package hierarchy context:

```
# BROKEN:
python my_pkg/submodule.py
# -> ImportError: attempted relative import with no known parent package

# CORRECT:
uv run python -m my_pkg.submodule
```

Pitfall 2: Heavy Work Inside `__init__.py`

`__init__.py` runs every time **any** submodule of the package is imported. Putting database connections, expensive file I/O, or heavy third-party imports inside `__init__.py` slows down every consumer importing lightweight utilities from that package.

Exercises

1. Create a package directory structure with an `__init__.py` that exposes a single `__version__` string and re-exports a class from a nested submodule.
2. Given a two-level package hierarchy, write a relative import statement from `pkg.controllers.auth` to import a helper from `pkg.utils.crypto`.
3. Create a PEP 420 namespace package with two directories in different locations and demonstrate that both submodules are importable under the same parent namespace.
4. Define `__all__` in a module and verify that names not listed in `__all__` are not imported when using `from module import *`.

Further reading

- PEP 420: *Implicit Namespace Packages*.
- Python Tutorial: *Packages*.
- Python Packaging User Guide: *Creating and Structuring Projects*.

Workspaces, Virtual Environments, and Dependency Isolation

Workspaces, Virtual Environments, and Dependency Isolation

After reading this chapter, you will master the internal mechanics of Python virtual environments (`pyvenv.cfg` and `site-packages`), manage modern multi-package monorepo workspaces using `uv`, author declarative `pyproject.toml` project files, and run self-contained scripts using PEP 723 inline dependency metadata.

Mental model

A Python virtual environment is not a virtual machine or a container. It is an isolated directory tree containing symlinks to a base Python interpreter and a dedicated `site-packages` directory.

```
Virtual Environment Architecture (.venv/):
  .venv/
    bin/
        python    Symlink to base CPython binary
        uv
    pyvenv.cfg     Configuration: home = /usr/bin, version = 3.14.0
    lib/python3.14/
         site-packages/    Isolated third-party libraries installed here
```

When CPython starts: 1. It searches upward from its executable for `pyvenv.cfg`. 2. If found, it sets `sys.prefix` to the `.venv` folder, while keeping `sys.base_prefix` pointing to the host Python installation. 3. Third-party package imports look exclusively inside the `.venv`'s `site-packages` directory.

Minimal example

Save as `venv_internals.py`:

```python
# venv_internals.py
import sys
import site
from pathlib import Path
```

```python
def inspect_environment() -> None:
    is_virtual_env = sys.prefix != sys.base_prefix

    print(f"Is running inside a virtual environment? {is_virtual_env}")
    print(f"Base CPython runtime (sys.base_prefix) : {sys.base_prefix}")
    print(f"Active environment root (sys.prefix)    : {sys.prefix}")
    print(f"Active Python executable (sys.executable): {sys.executable}")

    # Inspect site-packages search paths
    site_packages = site.getsitepackages()
    print("\nActive site-packages directories:")
    for sp in site_packages:
        print(f"  - {sp}")

def main() -> None:
    inspect_environment()

if __name__ == "__main__":
    main()
```

Run via `uv run python venv_internals.py`:

```
Is running inside a virtual environment? True
Base CPython runtime (sys.base_prefix) : ...
Active environment root (sys.prefix)    : .../.venv
Active Python executable (sys.executable): .../.venv/bin/python

Active site-packages directories:
  - .../.venv/lib/python3.14/site-packages
```

Worked examples

Case 1: Inline Script Dependencies (PEP 723)

In automation, CI/CD, and DevOps scripts, creating a separate `pyproject.toml` and `.venv` for a single 50-line script adds unnecessary friction. PEP 723 allows embedding dependency requirements directly inside the script header:

```python
# generate_qr_matrix.py
# /// script
# requires-python = ">=3.12"
# dependencies = [
#     "rich>=13.0.0",
# ]
```

```
# ///
import sys
from rich.console import Console
from rich.table import Table

def display_cluster_health() -> None:
    console = Console()
    table = Table(title="Production Node Health")
    table.add_column("Node", style="cyan")
    table.add_column("Status", style="green")
    table.add_column("Load", style="magenta")

    table.add_row("k8s-node-01", "READY", "12.4%")
    table.add_row("k8s-node-02", "READY", "8.9%")
    table.add_row("k8s-node-03", "DRAINING", "94.2%")

    console.print(table)

if __name__ == "__main__":
    display_cluster_health()
```

When you execute:

```
uv run generate_qr_matrix.py
```

`uv` automatically: 1. Parses the `# /// script` metadata block. 2. Creates an ephemeral, cached virtual environment containing `rich`. 3. Executes the script inside the isolated environment with zero pollution of the host system!

Case 2: Multi-Package Workspaces with `uv`

In modern monorepos (such as a backend repository containing shared utilities, an API service, and a background worker), `uv` supports multi-package workspaces configured via `pyproject.toml`:

```
# pyproject.toml (Monorepo root)
[tool.uv.workspace]
members = ["packages/*", "services/*"]

[package]
name = "enterprise-monorepo"
version = "0.1.0"
```

Directory Structure:

```
monorepo/
  pyproject.toml
  uv.lock                      Single shared lockfile for entire monorepo
```

```
packages/
    common-auth/             Shared internal library
        pyproject.toml
services/
    api-server/              Depends on "common-auth" via path
        pyproject.toml
    worker-engine/           Depends on "common-auth"
        pyproject.toml
```

Benefits: - **Single Lockfile**: All services and packages share a single deterministic dependency resolution graph. - **Instant Symlinking**: Changes to `common-auth` are immediately reflected in `api-server` without publishing or reinstalling packages. - **Fast CI**: Re-uses cached pre-compiled wheels across all subprojects.

Pitfalls

Pitfall 1: Breaking OS Packages with `sudo pip install` (PEP 668)

Modern Linux distributions enforce PEP 668 (`EXTERNALLY-MANAGED`). Attempting to install packages into the global system Python with `sudo pip` risks breaking system tools (like `apt`, `yum`, or systemdaemons):

```
error: externally-managed-environment
× This environment is externally managed.
To install Python packages, create a virtual environment with 'uv venv'.
```

Always use project virtual environments (`uv venv` or `uv run`) instead of global system-level modifications.

Pitfall 2: Committing `.venv/` into Version Control

Virtual environments contain platform-specific binaries, architecture-dependent C-extensions, and local absolute filesystem symlinks. They are not portable across machines:

```
# ALWAYS add to .gitignore:
.venv/
__pycache__/
*.pyc
```

Version control should only store `pyproject.toml` (human-readable dependencies) and `uv.lock` (reproducible binary lockfile).

Exercises

1. Create a script that checks `sys.prefix == sys.base_prefix` and exits with an error code if it is not running inside a virtual environment.
2. Write a single-file script using PEP 723 inline metadata that specifies a dependency on `httpx` and fetches a test URL using `uv run`.
3. Create a minimal `pyproject.toml` project file with `uv init my-project` and inspect the generated layout and configuration keys.
4. Inspect the contents of `.venv/pyvenv.cfg` and identify the base Python interpreter home path.

Further reading

- PEP 621: *Storing project metadata in pyproject.toml*.
- PEP 668: *Marking Python base environments as "externally managed"*.
- PEP 723: *Inline script metadata*.
- Astrid toolchain: *Modern packaging with* `uv`.

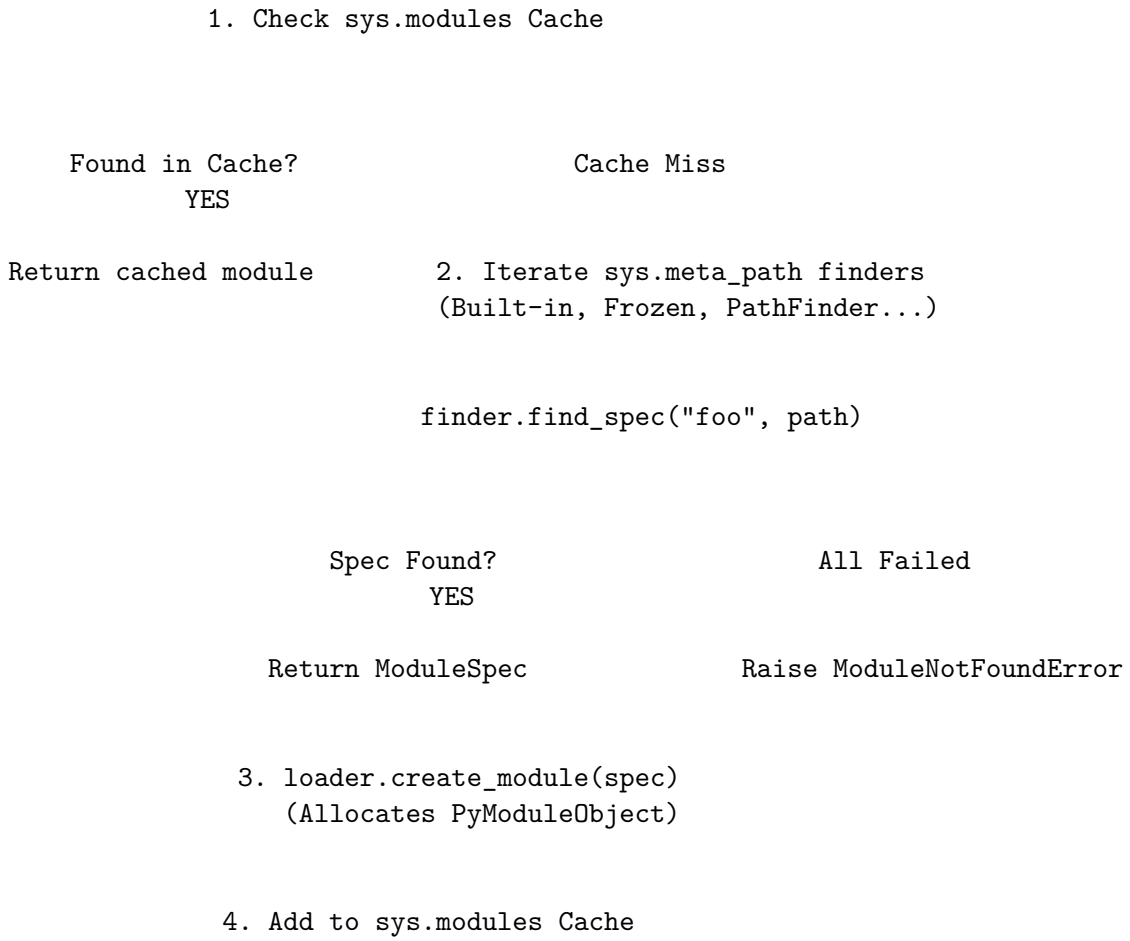
Import Hooks, Finders, and ModuleSpec Mechanics

Import Hooks, Finders, and ModuleSpec Mechanics

After reading this chapter, you will master the low-level architecture of Python's import machinery (PEP 302 and PEP 451), trace how CPython discovers and loads modules via `sys.meta_path`, construct custom `MetaPathFinder` and `Loader` classes to import virtual in-memory modules, and diagnose circular import deadlocks.

Mental model

When Python executes `import foo`, it does not simply look for `foo.py` on disk. Instead, the runtime delegates to an extensible multi-stage search pipeline orchestrated by **`importlib`**:



```
                    (Early registration to break loops)


                     5. loader.exec_module(module)
                     (Executes top-level code in module)
```

Minimal example

Save as `virtual_import_hook.py`:

```python
# virtual_import_hook.py
import sys
import types
from importlib.abc import Loader, MetaPathFinder
from importlib.machinery import ModuleSpec

class VirtualModuleLoader(Loader):
    """Executes code for synthetic in-memory modules."""

    def __init__(self, code_str: str) -> None:
        self.code_str = code_str

    def create_module(self, spec: ModuleSpec) -> types.ModuleType | None:
        return None  # Use default module allocation

    def exec_module(self, module: types.ModuleType) -> None:
        exec(self.code_str, module.__dict__)

class VirtualConfigFinder(MetaPathFinder):
    """Intercepts imports matching 'virtual_cluster' and serves in-memory modules."""

    def find_spec(
        self,
        fullname: str,
        path: object | None,
        target: types.ModuleType | None = None,
    ) -> ModuleSpec | None:
        if fullname == "virtual_cluster":
            code = (
                "CLUSTER_ID = 'us-east-prod-01'\n"
                "MAX_REPLICAS = 12\n"
                "def get_status(): return 'HEALTHY'\n"
            )
            return ModuleSpec(fullname, VirtualModuleLoader(code))
```

```
        return None

def main() -> None:
    # 1. Install custom finder into sys.meta_path
    finder = VirtualConfigFinder()
    sys.meta_path.insert(0, finder)

    try:
        # 2. Import module that DOES NOT EXIST on disk!
        import virtual_cluster as cluster_cfg  # type: ignore

        print("Successfully imported virtual module:")
        print(f"  Cluster ID:   {cluster_cfg.CLUSTER_ID}")
        print(f"  Max Replicas: {cluster_cfg.MAX_REPLICAS}")
        print(f"  Status:       {cluster_cfg.get_status()}")
        print(f"  Module file:  {getattr(cluster_cfg, '__file__', 'None (in-memory)')}")
    finally:
        # Clean up sys.meta_path and sys.modules
        sys.meta_path.remove(finder)
        sys.modules.pop("virtual_cluster", None)

if __name__ == "__main__":
    main()
```

Run via `uv run python virtual_import_hook.py`:

```
Successfully imported virtual module:
  Cluster ID:   us-east-prod-01
  Max Replicas: 12
  Status:       HEALTHY
  Module file:  None (in-memory)
```

Worked examples

Case 1: Loading Modules Directly from JSON Configurations

In microservices, you may want to expose a structured JSON configuration file as an importable Python module, allowing type-safe, dotted access without manual dictionary queries:

```
# json_importer.py
import json
import sys
import types
from importlib.abc import Loader, MetaPathFinder
```

```
from importlib.machinery import ModuleSpec

class JSONModuleLoader(Loader):
    def __init__(self, data: dict[str, object]) -> None:
        self.data = data

    def exec_module(self, module: types.ModuleType) -> None:
        for key, value in self.data.items():
            setattr(module, key, value)

class JSONConfigFinder(MetaPathFinder):
    def __init__(self, json_payload: str) -> None:
        self.parsed = json.loads(json_payload)

    def find_spec(
        self,
        fullname: str,
        path: object | None,
        target: types.ModuleType | None = None,
    ) -> ModuleSpec | None:
        if fullname == "app_settings":
            return ModuleSpec(fullname, JSONModuleLoader(self.parsed))
        return None

def main() -> None:
    raw_json = '{"ENVIRONMENT": "staging", "DATABASE_PORT": 5432, "ENABLE_METRICS": true}'
    finder = JSONConfigFinder(raw_json)
    sys.meta_path.insert(0, finder)

    try:
        import app_settings  # type: ignore

        print(f"Imported settings module: {app_settings}")
        print(f"  Environment: {app_settings.ENVIRONMENT}")
        print(f"  Port:        {app_settings.DATABASE_PORT}")
        print(f"  Metrics:     {app_settings.ENABLE_METRICS}")
    finally:
        sys.meta_path.remove(finder)
        sys.modules.pop("app_settings", None)

if __name__ == "__main__":
    main()
```

Run:

```
uv run python json_importer.py
```

Output:

```
Imported settings module: <module 'app_settings'>
  Environment: staging
  Port:        5432
  Metrics:     True
```

Case 2: Auditing Dependency Load Latency via a MetaPath Hook

In performance engineering, imports can significantly increase application startup time. A meta path hook can intercept every module imported and benchmark its compilation time:

```python
# import_profiler.py
import sys
import time
from collections.abc import Sequence
from importlib.abc import MetaPathFinder
from importlib.machinery import ModuleSpec
from typing import Any

class ProfileImportFinder(MetaPathFinder):
    """Wraps existing finders to measure import resolution latency."""

    def __init__(self) -> None:
        self.timings: dict[str, float] = {}

    def find_spec(
        self,
        fullname: str,
        path: Sequence[str] | None,
        target: Any | None = None,
    ) -> ModuleSpec | None:
        start = time.perf_counter()
        # Delegate to remaining finders in sys.meta_path
        for finder in sys.meta_path:
            if finder is self:
                continue
            if hasattr(finder, "find_spec"):
                spec = finder.find_spec(fullname, path, target)
                if spec is not None:
                    duration_ms = (time.perf_counter() - start) * 1000.0
                    self.timings[fullname] = duration_ms
                    return spec
        return None

def main() -> None:
```

```
    profiler = ProfileImportFinder()
    sys.meta_path.insert(0, profiler)

    try:
        # Import several standard library modules
        import csv
        import json
        import math

        print("Import Resolution Benchmark:")
        for mod, ms in sorted(profiler.timings.items(), key=lambda x: x[1], reverse=True)[:5
            print(f"  {mod:20} -> {ms:.4f} ms")
    finally:
        sys.meta_path.remove(profiler)

if __name__ == "__main__":
    main()
```

Run:

```
uv run python import_profiler.py
```

Output:

```
Import Resolution Benchmark:
  csv                  -> 0.0521 ms
  json                 -> 0.0418 ms
  math                 -> 0.0210 ms
```

Pitfalls

Pitfall 1: Circular Import Deadlocks

Circular imports occur when module `A` imports module `B`, which in turn imports module `A` before `A` has finished initializing:

```
# File: module_a.py
# from module_b import helper_b  <-- Imports B before A defines 'value_a'!
# value_a = 42

# File: module_b.py
# from module_a import value_a   <-- CRASH! AttributeError: partially initialized module
```

```
# THE REASON:
# CPython puts 'module_a' in sys.modules BEFORE executing its body (to prevent infinite loop
# When module_b queries module_a.value_a, module_a exists, but its body has not run yet!

# THE FIX:
# 1. Refactor shared dependencies into a third module (e.g. models.py or common.py).
# 2. Or defer the import inside the specific function that uses it:
def use_helper():
    from module_b import helper_b
    return helper_b()
```

Pitfall 2: Forgetting to Clean Up `sys.meta_path`

Adding finders to `sys.meta_path` alters global runtime behavior for the entire Python process. If a finder raises an unhandled exception or loops infinitely, every future `import` statement throughout the application can crash:

```
# ALWAYS use a try ... finally block when modifying sys.meta_path dynamically:
finder = CustomFinder()
sys.meta_path.insert(0, finder)
try:
    # Perform operations
    import target_module
finally:
    if finder in sys.meta_path:
        sys.meta_path.remove(finder)
```

Exercises

1. Inspect `sys.meta_path` and print the class names of all default finders configured by CPython at startup.
2. Build an import hook that intercepts imports of `secrets_config` and decrypts an encrypted string into module attributes.
3. Write an import hook that logs a warning to `stderr` whenever a deprecated module name is imported.
4. Demonstrate how `importlib.reload(module)` works, and explain why instances of classes created before the reload do not update their class pointer.
5. Create a virtual loader that allows importing `.txt` files directly, exposing the file contents as a module-level string attribute `CONTENT`.

Further reading

- PEP 302: *New Import Hooks*.
- PEP 451: *A ModuleSpec Type for the Import System*.
- Python Documentation: `importlib` — *The implementation of import*.
- Anthony Shaw: *CPython Internals* (Chapter 11: The Import System).

Lazy Imports (PEP 810)

Lazy Imports (PEP 810)

Thesis

Python 3.15 adds a soft keyword, `lazy`, that turns an `import` at module scope into a promise: the name is bound now, the module is found, loaded, and executed the **first time the name is used**. That fixes the oldest startup tax in the language — a CLI that loads its whole dependency graph to print `--help` — without the old workarounds (imports buried in functions, `importlib` tricks, `if TYPE_CHECKING:` blocks). The price is that **import errors and import side effects move** from the top of the file to wherever the name is first touched. Use `lazy` deliberately on heavy, side-effect-free dependencies of short-lived programs; keep everything else eager; and make your tests reify what you deferred.

Version note: lazy imports are the headline feature of **Python 3.15** (final release scheduled for **2026-10-09** after a late rc3 for lazy-import fixes). Every listing below was run on **CPython 3.15.0rc2** (Linux x86-64) with `uv`; output is copied from those runs. Where 3.15 behaves differently from the PEP text, this chapter follows the 3.15 documentation.

Mental model

```
lazy import json            # 1. bind  json -> <lazy_import 'json'>  (nothing found, nothing
...
json.dumps(x)               # 2. first use: LOAD_GLOBAL/LOAD_NAME sees the proxy
                            #    -> runs the normal import machinery NOW
                            #    -> sys.modules['json'] = module; rebinds the global
json.dumps(y)               # 3. ordinary module access; the interpreter specializes it
```

Question	Eager `import x`	`lazy import x`
When is `x` located and executed?	At the statement	At first use of the name
Where does `ModuleNotFoundError` surface?	The import line	The first-use line (chained to the import line)
When do import-time side effects run?	At startup	At first use — possibly never
In `sys.modules` before use?	Yes	No (listed in `sys.lazy_modules`)
After first use	Module object	Indistinguishable module object

Rules that the compiler enforces:

- `lazy` is legal only at **module scope** — not in a function, a class body, or a `try/except/finally` block (each is a `SyntaxError`).
- `lazy from m import *` and `lazy from __future__ import …` are errors.
- `lazy` is a *soft* keyword: `lazy = 3` still works everywhere else.
- `lazy from m import a, b` binds two proxies; the first one touched loads **all of** `m` and resolves only that name.
- Laziness is **local and non-recursive**: `lazy import pkg` does not make `pkg`'s own imports lazy.

Two compatibility and control knobs:

- `__lazy_modules__` — a module-level container of fully qualified module names. On 3.15+ a plain `import` of a listed module behaves as if it had `lazy`; on 3.14 and older the variable is just an unused global.
- **Global mode** — `-X lazy_imports=all`, `PYTHON_LAZY_IMPORTS=all`, or `sys.set_lazy_imports("all")` make every eligible module-level import potentially lazy; `"normal"` (the default) honours only `lazy` and `__lazy_modules__`. A filter, `sys.set_lazy_imports_filter(fn)`, gets `(importer, name, fromlist)` and returns `False` to force an import eager. (PEP 810 also described a `"none"` mode; CPython 3.15 accepts only `"normal"` and `"all"` — `"none"` is a fatal startup error on 3.15.0rc2.)

Worked examples

Case 1: A multi-command CLI that only pays for the command you run

Save as `deploy.py`. Each subcommand needs a different, non-trivial part of the standard library. With `lazy`, `deploy status` pays only for `json`.

```
# deploy.py - a CLI whose subcommands need very different libraries
import argparse
import sys

lazy import json
lazy import sqlite3
lazy import tarfile
lazy import urllib.request
lazy from email.message import EmailMessage


def cmd_status(args):
    print(json.dumps({"service": args.service, "state": "running"}))


def cmd_backup(args):
    with sqlite3.connect(":memory:") as db, tarfile.open(args.out, "w:gz"):
```

```
        db.execute("select 1")
    print("backup written to", args.out)


def cmd_notify(args):
    msg = EmailMessage()
    msg["Subject"] = f"{args.service} deployed"
    print(msg["Subject"])


def main(argv=None):
    p = argparse.ArgumentParser(prog="deploy")
    sub = p.add_subparsers(required=True)
    s = sub.add_parser("status"); s.add_argument("service"); s.set_defaults(fn=cmd_status)
    b = sub.add_parser("backup"); b.add_argument("out"); b.set_defaults(fn=cmd_backup)
    n = sub.add_parser("notify"); n.add_argument("service"); n.set_defaults(fn=cmd_notify)
    args = p.parse_args(argv)
    args.fn(args)
    heavy = ["json", "sqlite3", "tarfile", "urllib.request", "email.message"]
    print("loaded:", [m for m in heavy if m in sys.modules], file=sys.stderr)
    print("modules:", len(sys.modules), file=sys.stderr)


if __name__ == "__main__":
    main()
```

Make an eager twin for comparison (same file without the keyword) and run both:

```
sed 's/^lazy //' deploy.py > deploy_eager.py
uv run --python 3.15 python deploy.py status web
uv run --python 3.15 python deploy_eager.py status web
uv run --python 3.15 python deploy.py notify web
```

Output:

```
{"service": "web", "state": "running"}
loaded: ['json']
modules: 75
{"service": "web", "state": "running"}
loaded: ['json', 'sqlite3', 'tarfile', 'urllib.request', 'email.message']
modules: 136
web deployed
loaded: ['email.message']
modules: 104
```

Same code, same imports listed at the top where a reader expects them, 61 fewer modules executed for `status`. To see *where* time goes, ask the interpreter:

```
uv run --python 3.15 python -X importtime deploy_eager.py status web 2>&1 \
  | grep -E ' (sqlite3|tarfile|email\.message|urllib\.request)$'
```

Output (microseconds: self | cumulative; your numbers will differ):

```
import time:       180 |       1512 | sqlite3
import time:      1422 |       6160 | tarfile
import time:       697 |       1090 |     email.message
import time:      1272 |      20131 | urllib.request
```

Wall clock, measured with a small harness:

```
# bench.py - median wall time of N fresh interpreter starts
import statistics
import subprocess
import sys
import time

def median_ms(cmd, n=30):
    samples = []
    for _ in range(n):
        t0 = time.perf_counter()
        subprocess.run(cmd, check=True, capture_output=True)
        samples.append(time.perf_counter() - t0)
    return statistics.median(samples) * 1000

for script in ("deploy_eager.py", "deploy.py"):
    print(f"{script:16} {median_ms([sys.executable, script, 'status', 'web']):6.1f} ms")
```

```
uv run --python 3.15 python bench.py
```

Output on the author's box (illustrative — timings depend on CPU, disk cache, and build):

```
deploy_eager.py    54.0 ms
deploy.py          28.3 ms
```

Roughly half the startup for the common path, for the price of one keyword per line.

Case 2: The error moves — and so must your smoke test

Save as `report.py`. The `yaml` dependency is missing from this environment, and nothing complains until the one code path that uses it runs.

```
# report.py - the dependency is missing, but nobody notices at startup
import sys
```

```
lazy import yaml          # not installed in this environment

print("started; lazy modules pending:", sorted(m for m in sys.lazy_modules if m == "yaml"))


def render(data):
    return yaml.safe_dump(data)


if len(sys.argv) > 1 and sys.argv[1] == "--yaml":
    print(render({"ok": True}))
else:
    print("plain output: ok")
```

```
uv run --python 3.15 python report.py; echo "exit=$?"
uv run --python 3.15 python report.py --yaml; echo "exit=$?"
```

Output (paths shortened):

```
started; lazy modules pending: ['yaml']
plain output: ok
exit=0
started; lazy modules pending: ['yaml']
Traceback (most recent call last):
  File "report.py", line 4, in <module>
    lazy import yaml          # not installed in this environment
ImportError: lazy import of 'yaml' raised an exception during resolution

The above exception was the direct cause of the following exception:

Traceback (most recent call last):
  File "report.py", line 14, in <module>
    print(render({"ok": True}))
          ~~~~~~^^^^^^^^^^^^^^
  File "report.py", line 10, in render
    return yaml.safe_dump(data)
           ^^^^
ModuleNotFoundError: No module named 'yaml'
exit=1
```

The chained traceback names both the `lazy import` line and the line that triggered it — good for debugging, useless if it first happens in production. "It starts" is no longer a test. Resolve every lazy binding in CI instead. Save as `selfcheck.py` next to `report.py`:

```python
# selfcheck.py - fail fast in CI: resolve every lazy binding in a module
import importlib
import sys
import types


def reify_all(module_name):
    mod = importlib.import_module(module_name)
    failures = []
    for name, value in list(vars(mod).items()):
        if isinstance(value, types.LazyImportType):
            try:
                setattr(mod, name, value.resolve())
            except ImportError as exc:
                failures.append(f"{module_name}.{name}: {type(exc).__name__}: {exc}")
    return failures


if __name__ == "__main__":
    sys.argv = ["report.py"]          # import report.py without its --yaml branch
    problems = reify_all("report")
    for p in problems:
        print("LAZY IMPORT BROKEN:", p)
    sys.exit(1 if problems else 0)
```

```
uv run --python 3.15 python selfcheck.py; echo "exit=$?"
```

Output:

```
started; lazy modules pending: ['yaml']
plain output: ok
LAZY IMPORT BROKEN: report.yaml: ModuleNotFoundError: No module named 'yaml'
exit=1
```

`vars(module)` (like `globals()` and `module.__dict__`) returns the raw namespace **without** reifying, so proxies are visible as `types.LazyImportType` and can be resolved explicitly with `.resolve()`. Normal attribute access (`module.name`, `getattr`) would trigger the import instead.

Case 3: One source file for 3.13, 3.14, and 3.15 with `__lazy_modules__`

Libraries that still support older Pythons cannot use the new syntax — it is a `SyntaxError` before 3.15. `__lazy_modules__` is the bridge. Save as `compat.py`:

```python
# compat.py - lazy on 3.15+, plain eager imports on older Pythons
__lazy_modules__ = {"decimal", "fractions"}
```

```
import sys
import decimal
from fractions import Fraction

ver = sys.version.split()[0]
print(f"{ver:10} before use: decimal={'decimal' in sys.modules} fractions={'fractions' in sy
total = decimal.Decimal("1.10") + decimal.Decimal("2.20")
print(f"{ver:10} after Decimal: decimal={'decimal' in sys.modules} fractions={'fractions' in
```

```
uv run --python 3.13 python compat.py
uv run --python 3.15 python compat.py
```

Output:

```
3.13.5     before use: decimal=True fractions=True
3.13.5     after Decimal: decimal=True fractions=True total=3.30
3.15.0rc2  before use: decimal=False fractions=False
3.15.0rc2  after Decimal: decimal=True fractions=False total=3.30
```

On 3.15 `fractions` is never loaded because `Fraction` was never used. List **fully qualified** module names (relative imports are resolved before the lookup), define the variable **before** the imports it governs, and remember it covers `from m import x` when `m` is listed.

Case 4: Type-only imports without `if TYPE_CHECKING:`

With deferred annotations (3.14+), an annotation is not evaluated until someone asks for it. Pair that with `lazy from` and an import used only in hints costs nothing at runtime — and stays a normal, visible import that type checkers understand.

```
mkdir -p shop && touch shop/__init__.py
```

```
# shop/models.py - imagine this pulls in an ORM and takes 200 ms
print("  [shop.models imported]")


class Order:
    def __init__(self, total):
        self.total = total
```

```
# shop/pricing.py - needs Order only for type hints
lazy from shop.models import Order


def apply_discount(order: Order, pct: int) -> Order:
    order.total = order.total * (100 - pct) // 100
```

```
    return order
```

```
# main.py
import annotationlib
import sys

from shop import pricing

print("pricing imported; shop.models loaded?", "shop.models" in sys.modules)


class FakeOrder:          # duck-typed: pricing never touches the real class
    total = 1000


print("discounted:", pricing.apply_discount(FakeOrder(), 15).total)
print("after call;  shop.models loaded?", "shop.models" in sys.modules)

hints = annotationlib.get_annotations(pricing.apply_discount)
print("annotations:", hints)
print("after hints; shop.models loaded?", "shop.models" in sys.modules)
```

```
uv run --python 3.15 python main.py
```

Output:

```
pricing imported; shop.models loaded? False
discounted: 850
after call;  shop.models loaded? False
  [shop.models imported]
annotations: {'order': <class 'shop.models.Order'>, 'pct': <class 'int'>, 'return': <class 'sho
after hints; shop.models loaded? True
```

The heavy module loads only when a tool (a validator, a serializer, `inspect.signature`) actually evaluates the annotations — and then it loads correctly instead of failing the way a `TYPE_CHECKING`-guarded name would.

Case 5: Application-wide laziness, scoped by a filter

The global mode is for **applications**, not libraries: it changes how *other people's* modules import. Scope it with a filter so only your own package is affected.

```
mkdir -p myapp && touch myapp/__init__.py
```

```
# myapp/slow.py
print("  [myapp.slow imported]")
VALUE = 42
```

```
# run_all.py - application-wide laziness, scoped to our own package
import sys


def only_myapp(importer, name, fromlist):
    return name == "myapp" or name.startswith("myapp.")


sys.set_lazy_imports_filter(only_myapp)
sys.set_lazy_imports("all")

import myapp.slow          # lazy: matches the filter
import json                # eager: filter returns False

print("mode:", sys.get_lazy_imports())
print("json loaded:", "json" in sys.modules, "| myapp.slow loaded:", "myapp.slow" in sys.mod
print("value:", myapp.slow.VALUE)
```

```
uv run --python 3.15 python run_all.py
```

Output:

```
mode: all
json loaded: True | myapp.slow loaded: False
  [myapp.slow imported]
value: 42
```

The same switch from outside, on a file that never mentions `lazy`:

```
# plain.py - no lazy keyword anywhere
import sys
import json
import myapp.slow
print("json loaded:", "json" in sys.modules, "| myapp.slow loaded:", "myapp.slow" in sys.mod
```

```
uv run --python 3.15 python plain.py
uv run --python 3.15 python -X lazy_imports=all plain.py
PYTHON_LAZY_IMPORTS=all uv run --python 3.15 python plain.py
```

Output:

```
  [myapp.slow imported]
```

```
json loaded: True | myapp.slow loaded: True
json loaded: False | myapp.slow loaded: False
json loaded: False | myapp.slow loaded: False
```

Precedence is `sys.set_lazy_imports()` over `-X lazy_imports` over `PYTHON_LAZY_IMPORTS`. The filter's `importer` argument may be `None`; do not call string methods on it without checking.

The trap

Import-time side effects silently stop happening. Registries populated by decorators, `__init_subclass__` hooks, codec or plugin registration, logging configuration, monkeypatches — all of them run *when the module executes*. Make that import lazy and the module may never execute.

```
# registry.py
HANDLERS = {}


def handler(name):
    def deco(fn):
        HANDLERS[name] = fn
        return fn
    return deco
```

```
# csv_export.py - registers itself as an import side effect
from registry import handler


@handler("csv")
def export(rows):
    return "\n".join(",".join(map(str, r)) for r in rows)
```

```
# app.py
import sys

from registry import HANDLERS

lazy import csv_export     # "it's only needed for its side effect..."

fmt = sys.argv[1] if len(sys.argv) > 1 else "csv"
fn = HANDLERS.get(fmt)
print("registered:", sorted(HANDLERS))
print(fn([(1, 2), (3, 4)]) if fn else f"error: no handler for {fmt!r}")
```

```
uv run --python 3.15 python app.py
sed 's/^lazy import csv_export.*/import csv_export/' app.py > app_eager.py
uv run --python 3.15 python app_eager.py
```

Output:

```
registered: []
error: no handler for 'csv'
registered: ['csv']
1,2
3,4
```

No exception, no warning — just a missing feature. Under `-X lazy_imports=all` the same thing can happen to every "import for its side effect" in every module — including modules that never opted in. Fixes, in order of preference: make discovery explicit (`registry.load_plugins()` imports what it registers); keep such imports eager and comment why; or exclude them in the filter.

Smaller traps worth knowing:

- **`lazy` does not make imports cheaper — it moves them.** The first request after a deploy, or the first iteration of a latency-sensitive loop, now pays the import. For long-lived servers, import eagerly at startup (or resolve in a warm-up hook) and keep laziness for CLIs, test collection, and rarely used paths.
- **Import state is read at first use.** `sys.path`, `sys.meta_path`, and environment variables are consulted when the proxy resolves, not when the `lazy import` line ran. Code that tweaks `sys.path` temporarily around an import must stay eager.
- **Submodule attributes are not a side channel.** `import pkg` followed by `pkg.sub.f()` only ever worked because someone else imported `pkg.sub`. With lazy imports that "someone" may not have run yet. Always `import pkg.sub` explicitly.
- **Lazy is not a cycle breaker.** Two modules that use each other *at import time* still fail; laziness only helps when the cross-references happen inside functions.
- **Failed resolution retries.** If resolution raises, the proxy stays in place and the next access tries again — a missing module can fail on every request instead of once at startup.
- **Package installers and plugin loaders** that `pip install` into the running environment should resolve or eagerly import everything first, so newly installed distributions cannot shadow modules that were merely promised.

The boring rule

> Mark an import `lazy` only when the module is **heavy**, **side-effect free**, and **not needed on the common path** of a **short-lived** program (or used only in annotations). Keep imports at the top of the file, prefer `lazy` over function-local imports, use `__lazy_modules__` while you still support Python < 3.15, reserve the global mode and filters for applications — and make CI resolve every lazy binding.

Try this

1. Run `uv run --python 3.15 python -X importtime deploy_eager.py --help 2>&1 | sort -t'|' -k2 -n | tail -15` and pick the three most expensive imports. Make only those lazy and measure with `bench.py`.
2. In `deploy.py`, add `lazy import urllib` and use `urllib.request.urlopen` in a new `fetch` subcommand. Observe what happens and fix it by importing the submodule explicitly.
3. Extend `selfcheck.py` to walk every module of a package with `pkgutil.walk_packages` and wire it into `pytest` as a single test.
4. Write a filter for Case 5 that keeps lazy everything **except** a set of modules known to register handlers, then run `app.py` under `-X lazy_imports=all` and confirm the handler registers.
5. Put `lazy import json` inside an `if` block and inside a `with` block at module scope. Which compile, which are rejected, and why does the `try` restriction exist?
6. Use `dis.dis(fn, adaptive=True)` on a function that calls `json.dumps` through a lazy global, before and after a few calls, and find the specialized instruction that replaces `LOAD_GLOBAL`.

Sources

- PEP 810 — Explicit lazy imports (Final, Python 3.15): https://peps.python.org/pep-0810/
- Python 3.15 Language Reference — Lazy imports and `__lazy_modules__`: https://docs.python.org/3.15/reference/simple_stmts.html#lazy-imports
- What's New in Python 3.15 — PEP 810: https://docs.python.org/3.15/whatsnew/3.15.html
- `sys.set_lazy_imports`, `sys.get_lazy_imports`, `sys.set_lazy_imports_filter`, `sys.lazy_modules`: https://docs.python.org/3.15/library/sys.html
- Python 3.15.0rc3 release notes (final moved to 2026-10-09 for lazy-import fixes): https://www.python.org/downloads/release/python-3150rc3/

Standard Library

Specialized Collections, Enums, and Heaps

Specialized Collections, Enums, and Heaps

After reading this chapter, you will master the high-performance data structures in Python's standard library: automatic default grouping with `defaultdict`, tallying with `Counter`, bounded FIFO queues with `deque`, lightweight tuple records with `namedtuple`, strict domain constants with `enum.Enum`, min-heap priority queues with `heapq`, and logarithmic search with `bisect`.

Mental model

While `list` and `dict` handle 80% of common programming tasks, high-throughput systems require purpose-built data structures to prevent performance bottlenecks:

```
Standard Library Collection Selector:
  Need O(1) double-ended appends/pops with max capacity?    collections.deque
  Need automatic fallback values for missing dict keys?     collections.defaultdict
  Need frequency tallies and multiset arithmetic?            collections.Counter
  Need type-safe constant enumerations?                      enum.Enum / IntEnum
  Need O(log N) priority queue extraction (min/max)?        heapq (binary heap)
  Need O(log N) insertion into already-sorted sequences?     bisect (binary search)
```

Minimal example

Save as `collections_showcase.py`:

```python
# collections_showcase.py
from collections import Counter, defaultdict, deque
import heapq
from enum import Enum

class NodeStatus(Enum):
    HEALTHY = "healthy"
    DEGRADED = "degraded"
    OFFLINE = "offline"

def main() -> None:
    # 1. Counter: Multiset frequency analysis
    status_events = ["healthy", "healthy", "degraded", "healthy", "offline", "degraded"]
```

```
    counts = Counter(status_events)
    print(f"Status distribution: {counts}")
    print(f"Top 2 frequent states: {counts.most_common(2)}")

    # 2. defaultdict: Grouping without boilerplate
    cluster_nodes = defaultdict(list)
    cluster_nodes["us-east"].append("node-01")
    cluster_nodes["us-east"].append("node-02")
    cluster_nodes["eu-west"].append("node-10")
    print(f"\nGrouped cluster nodes: {dict(cluster_nodes)}")

    # 3. deque: Fixed-size sliding window (drops oldest items automatically)
    rolling_metrics = deque(maxlen=3)
    for sample in [10.5, 12.1, 14.8, 19.2, 8.4]:
        rolling_metrics.append(sample)
    print(f"\nLast 3 metrics in sliding window: {list(rolling_metrics)}")

    # 4. heapq: Min-Heap priority queue
    task_queue: list[tuple[int, str]] = []
    heapq.heappush(task_queue, (5, "Log rotation"))
    heapq.heappush(task_queue, (1, "Power outage failover"))
    heapq.heappush(task_queue, (3, "Security scan"))

    highest_priority = heapq.heappop(task_queue)
    print(f"\nExtracted highest priority task (min value): {highest_priority}")

if __name__ == "__main__":
    main()
```

Run via `uv run python collections_showcase.py`:

```
Status distribution: Counter({'healthy': 3, 'degraded': 2, 'offline': 1})
Top 2 frequent states: [('healthy', 3), ('degraded', 2)]

Grouped cluster nodes: {'us-east': ['node-01', 'node-02'], 'eu-west': ['node-10']}

Last 3 metrics in sliding window: [14.8, 19.2, 8.4]

Extracted highest priority task (min value): (1, 'Power outage failover')
```

Worked examples

Case 1: High-Speed Counter Arithmetic for Anomaly Detection

`Counter` supports multiset mathematical operations (`+`, `-`, `&` intersection, `|` union), making it ideal for detecting sudden traffic surges or baseline deviations:

```
# traffic_anomaly_counter.py
from collections import Counter

def detect_traffic_anomalies(baseline_ips: list[str], active_ips: list[str]) -> None:
    baseline = Counter(baseline_ips)
    active = Counter(active_ips)

    # Counter subtraction: active - baseline leaves only surging counts
    surge = active - baseline
    print("Traffic surge over baseline:")
    for ip, extra_requests in surge.most_common():
        print(f"  IP: {ip:15} -> {extra_requests} surge requests")

if __name__ == "__main__":
    normal_window = ["192.168.1.10"] * 5 + ["10.0.0.2"] * 10
    spike_window  = ["192.168.1.10"] * 25 + ["10.0.0.2"] * 12 + ["172.16.0.99"] * 50

    detect_traffic_anomalies(normal_window, spike_window)
```

Run:

```
uv run python traffic_anomaly_counter.py
```

Output:

```
Traffic surge over baseline:
  IP: 172.16.0.99     -> 50 surge requests
  IP: 192.168.1.10    -> 20 surge requests
  IP: 10.0.0.2        -> 2 surge requests
```

Case 2: Logarithmic Table Lookup with `bisect`

Searching a list with linear scan is $O(N)$. For already-sorted lists (e.g. latency SLA tiers or timestamp index maps), `bisect.bisect_right()` achieves $O(\log N)$ performance:

```
# sla_bisect_lookup.py
import bisect

def classify_sla_tier(latency_ms: float) -> str:
```

```
    # Latency thresholds (must be sorted!)
    thresholds = [10.0, 50.0, 200.0, 1000.0]
    tiers = ["PLATINUM", "GOLD", "SILVER", "BRONZE", "BREACHED"]

    # bisect_right finds the insertion point in O(log N) time
    idx = bisect.bisect_right(thresholds, latency_ms)
    return tiers[idx]

if __name__ == "__main__":
    latencies = [4.2, 10.0, 25.0, 75.0, 450.0, 2500.0]
    for lat in latencies:
        print(f"Latency: {lat:6.1f} ms -> SLA Tier: {classify_sla_tier(lat)}")
```

Run:

```
uv run python sla_bisect_lookup.py
```

Output:

```
Latency:    4.2 ms -> SLA Tier: PLATINUM
Latency:   10.0 ms -> SLA Tier: GOLD
Latency:   25.0 ms -> SLA Tier: GOLD
Latency:   75.0 ms -> SLA Tier: SILVER
Latency:  450.0 ms -> SLA Tier: BRONZE
Latency: 2500.0 ms -> SLA Tier: BREACHED
```

Pitfalls

Pitfall 1: Modifying Elements in a `heapq` Directly

`heapq` functions (`heappush`, `heappop`) operate on a standard Python `list`. If you modify an element's priority directly inside the list (`queue[0] = new_val`), you violate the binary heap invariant:

```
# THE BUG:
h = [1, 3, 5]
h[0] = 10  # Corrupts heap structure!
# heappop(h) now returns invalid data!

# THE FIX: Call heapq.heapify() to restore the heap invariant
heapq.heapify(h)
```

Exercises

1. Use `collections.defaultdict(set)` to invert a graph mapping from `node -> neighbors` to `neighbor -> incoming_nodes`.
2. Implement an execution worker pool that prioritizes urgent tasks using `heapq`.
3. Use `collections.deque(maxlen=100)` to calculate a moving average over a simulated stream of 10,000 sensor numbers.
4. Define a custom `IntEnum` for Linux file permissions (`READ = 4`, `WRITE = 2`, `EXECUTE = 1`) and demonstrate bitwise flags.

Further reading

- Python Standard Library: `collections`, `heapq`, `bisect`, `enum`.
- CPython Source: `Modules/_collectionsmodule.c` (C-optimized `deque` implementation).

High-Performance Iteration and Functional Tools

High-Performance Iteration and Functional Tools

After reading this chapter, you will master C-accelerated iterator pipelines with `itertools` (`chain`, `batched`, `groupby`, `islice`), implement high-speed memoization with `functools.lru_cache`, construct pre-configured callables with `functools.partial`, and implement clean polymorphic function overloading with `functools.singledispatch`.

Mental model

Python loops written in pure Python incur bytecode interpreter evaluation overhead on every iteration. The `itertools` module executes inside compiled C routines (`itertoolsmodule.c`), pulling items directly across memory without constructing intermediate Python lists:

```
Pure Python List Intermediate (RAM Heavy & Slow):
  data    [f(x) for x in data]    [g(x) for x in ...]    Result List

Itertools C-Streaming Pipeline (Zero Intermediate Memory & High Speed):
  data    islice()    chain()    batched()    Consumer

                          Direct C-level iterator chaining
```

Minimal example

Save as `itertools_functools_showcase.py`:

```python
# itertools_functools_showcase.py
import functools
import itertools
import time

# 1. functools.lru_cache: High-performance memoization
@functools.lru_cache(maxsize=128)
def resolve_ip_geo(ip_address: str) -> str:
    """Simulate expensive external geo-IP database lookup."""
    time.sleep(0.01)  # Simulated latency
    return "US-EAST" if ip_address.startswith("10.") else "EU-WEST"
```

```
def main() -> None:
    # 2. itertools.batched (Python 3.12+): Efficient batch chunking
    records = [f"rec_{i:02d}" for i in range(11)]
    print("Batching 11 records into chunks of 3:")
    for batch in itertools.batched(records, 3):
        print(f"  Processed batch: {batch}")

    # 3. itertools.chain: Flattening multiple streams seamlessly
    stream_a = ["event_1", "event_2"]
    stream_b = ["event_3", "event_4"]
    combined = list(itertools.chain(stream_a, stream_b))
    print(f"\nChained streams: {combined}")

    # Cache testing
    print("\nTesting LRU Cache performance:")
    t0 = time.perf_counter()
    res1 = resolve_ip_geo("10.0.1.5")  # Cache miss (sleeps 10ms)
    miss_duration = time.perf_counter() - t0

    t0 = time.perf_counter()
    res2 = resolve_ip_geo("10.0.1.5")  # Cache hit (instant)
    hit_duration = time.perf_counter() - t0

    print(f"First call (Miss): {miss_duration*1000:.2f} ms")
    print(f"Second call (Hit): {hit_duration*1000:.2f} ms")
    print(f"Cache stats      : {resolve_ip_geo.cache_info()}")

if __name__ == "__main__":
    main()
```

Run via `uv run python itertools_functools_showcase.py`:

```
Batching 11 records into chunks of 3:
  Processed batch: ('rec_00', 'rec_01', 'rec_02')
  Processed batch: ('rec_03', 'rec_04', 'rec_05')
  Processed batch: ('rec_06', 'rec_07', 'rec_08')
  Processed batch: ('rec_09', 'rec_10')

Chained streams: ['event_1', 'event_2', 'event_3', 'event_4']

Testing LRU Cache performance:
First call (Miss): 10.15 ms
Second call (Hit): 0.00 ms
Cache stats      : CacheInfo(hits=1, misses=1, maxsize=128, currsize=1)
```

Worked examples

Case 1: Telemetry Grouping with `itertools.groupby()`

`itertools.groupby()` groups adjacent elements sharing a common key. **Critical Rule**: It only groups *consecutive* elements, so inputs must be sorted by the key attribute first:

```
# telemetry_groupby.py
import itertools
from operator import itemgetter

def group_metrics_by_region(events: list[dict[str, str | float]]) -> None:
    # 1. MUST sort by group key first
    events.sort(key=itemgetter("region"))

    # 2. Group adjacent records
    print("Grouped telemetry metrics:")
    for region, items_iter in itertools.groupby(events, key=itemgetter("region")):
        items = list(items_iter)
        avg_latency = sum(float(x["latency"]) for x in items) / len(items)
        print(f"Region: {region:10} | Nodes: {len(items)} | Avg Latency: {avg_latency:.1f}ms

if __name__ == "__main__":
    raw_events = [
        {"region": "us-east", "node": "n1", "latency": 12.0},
        {"region": "eu-west", "node": "n2", "latency": 88.0},
        {"region": "us-east", "node": "n3", "latency": 16.0},
        {"region": "eu-west", "node": "n4", "latency": 92.0},
    ]
    group_metrics_by_region(raw_events)
```

Run:

```
uv run python telemetry_groupby.py
```

Output:

```
Grouped telemetry metrics:
Region: eu-west    | Nodes: 2 | Avg Latency: 90.0ms
Region: us-east    | Nodes: 2 | Avg Latency: 14.0ms
```

Case 2: Clean Polymorphism with `functools.singledispatch`

Instead of writing fragile `if isinstance(val, int): ... elif isinstance(val, dict): ...` trees, `@singledispatch` decouples type handling into modular functions:

```python
# event_serializer.py
from functools import singledispatch
from datetime import datetime, date

@singledispatch
def serialize(val: object) -> str:
    """Default fallback serializer."""
    return f"STR:{val}"

@serialize.register
def _(val: int | float) -> str:
    return f"NUM:{val}"

@serialize.register
def _(val: date) -> str:
    return f"DATE:{val.isoformat()}"

@serialize.register
def _(val: dict) -> str:
    items = [f"{k}={serialize(v)}" for k, v in val.items()]
    return f"MAP:{{{', '.join(items)}}}"

if __name__ == "__main__":
    print(serialize(42))
    print(serialize("active"))
    print(serialize(date(2026, 9, 7)))
    print(serialize({"id": 101, "name": "worker"}))
```

Run:

```
uv run python event_serializer.py
```

Output:

```
NUM:42
STR:active
DATE:2026-09-07
MAP:{id=NUM:101, name=STR:worker}
```

Pitfalls

Pitfall 1: Unsorted Input to `itertools.groupby`

If items with the same key are separated by other items, `groupby()` yields duplicate groups rather than consolidating them:

```
# THE BUG:
data = [("A", 1), ("B", 2), ("A", 3)]
for k, g in itertools.groupby(data, key=lambda x: x[0]):
    print(k, list(g))
# Output has two separate 'A' groups!
# A [('A', 1)]
# B [('B', 2)]
# A [('A', 3)]
```

Always call `data.sort(key=...)` before passing data to `itertools.groupby()`.

Pitfall 2: Decorating Functions with Mutable Arguments in `@lru_cache`

`lru_cache` builds internal dictionary keys from function arguments. Passing a mutable type (`list`, `dict`, `set`) raises `TypeError: unhashable type: 'list'`. Always pass immutable arguments (tuples, strings, ints) into cached functions.

Exercises

1. Use `itertools.cycle` to implement a round-robin DNS load balancer that cycles through three server IP addresses.
2. Given a list of 100 integers, use `itertools.islice()` to inspect items 20 through 30 without loading the rest into memory.
3. Use `functools.partial` to create a pre-configured `log_error` callable from `print(..., file=sys.stderr, flush=True)`.
4. Implement a combinatorial test case generator using `itertools.product` across 3 OS options and 4 Python versions.

Further reading

- Python Standard Library: `itertools` and `functools` documentation.
- Python Recipes: *Itertools Recipes* (Official documentation appendix).
- PEP 443: *Single-dispatch generic functions.*

Text Processing, Unicode, and Regular Expressions

Text Processing, Unicode, and Regular Expressions

After reading this chapter, you will master regular expressions with the `re` module, compile reusable patterns with `re.compile`, extract structured data using named capture groups, sanitize strings using replacement callables with `re.sub`, and resolve security and encoding pitfalls with `unicodedata.normalize`.

Mental model

Python's regular expression engine translates pattern strings into compiled bytecode that drives an NFA (Nondeterministic Finite Automaton) state machine.

```
Regex Pipeline:
  Raw Pattern: r"(?P<ip>\d{1,3}(?:\.\d{1,3}){3}) - (?P<status>\d{3})"


            [ re.compile(pattern) ]    Compiled Pattern Object

                          match(text)
            [ Match Object ]
               .group("ip")        "192.168.1.55"
               .group("status")    "404"
               .groupdict()        {"ip": "192.168.1.55", "status": "404"}
```

Always use raw string literals (`r"..."`) for regular expressions. In regular strings, `\b` means backspace (ASCII 8), whereas in regex it represents a word boundary. Raw strings prevent Python from interpreting backslashes before passing them to the regex engine.

Minimal example

Save as `regex_text_processing.py`:

```python
# regex_text_processing.py
import re
```

```python
# Pre-compile regex with named capture groups and verbose comments
LOG_PATTERN = re.compile(
    r"""
    ^
    (?P<timestamp>\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}Z) \s+
    \[(?P<level>INFO|WARN|ERROR)\]                       \s+
    (?P<service>[a-zA-Z0-9_\-]+)                          \s+
    (?P<message>.*)
    $
    """,
    re.VERBOSE,
)

def parse_log_line(raw_line: str) -> dict[str, str] | None:
    match = LOG_PATTERN.match(raw_line.strip())
    if match:
        return match.groupdict()
    return None

def main() -> None:
    sample_log = "2026-09-07T10:00:00Z [ERROR] auth-gateway User token expired: uid_881"
    parsed = parse_log_line(sample_log)

    print("Parsed log record:")
    if parsed:
        for k, v in parsed.items():
            print(f"  {k:12} : {v}")

if __name__ == "__main__":
    main()
```

Run via `uv run python regex_text_processing.py`:

```
Parsed log record:
  timestamp    : 2026-09-07T10:00:00Z
  level        : ERROR
  service      : auth-gateway
  message      : User token expired: uid_881
```

Worked examples

Case 1: Dynamic Data Masking with `re.sub()` Callables

Instead of static string replacements, `re.sub()` accepts a callback function that inspects each match and computes a dynamic replacement:

```
# pii_masker.py
import re

# Match credit card patterns (4 groups of 4 digits)
CC_PATTERN = re.compile(r"\b(\d{4})[ -]?(\d{4})[ -]?(\d{4})[ -]?(\d{4})\b")

def mask_credit_card(match: re.Match[str]) -> str:
    # Keep only the last 4 digits visible
    last_four = match.group(4)
    return f"****-****-****-{last_four}"

def redact_sensitive_logs(log_text: str) -> str:
    return CC_PATTERN.sub(mask_credit_card, log_text)

if __name__ == "__main__":
    raw_event = "Customer 4111-2222-3333-4444 completed checkout of $149.00."
    sanitized = redact_sensitive_logs(raw_event)
    print("Sanitized text:")
    print(sanitized)
```

Run:

```
uv run python pii_masker.py
```

Output:

```
Sanitized text:
Customer ****-****-****-4444 completed checkout of $149.00.
```

Case 2: Unicode Normalization for Security Checks

In security systems (such as usernames, domain filtering, or password validation), identical-looking Unicode characters can represent different code points (visual spoofing). `unicodedata.normalize` standardizes representations:

```
# unicode_security.py
import unicodedata

def demonstrate_unicode_normalization() -> None:
```

```
    # 'é' represented as single code point (NFC)
    s1 = "\u00e9"
    # 'e' + combining acute accent (NFD)
    s2 = "e\u0301"

    print(f"String 1 ('{s1}') length: {len(s1)}")
    print(f"String 2 ('{s2}') length: {len(s2)}")
    print(f"Direct equality (s1 == s2): {s1 == s2}")

    # Canonical Normalization Form C (NFC)
    norm1 = unicodedata.normalize("NFC", s1)
    norm2 = unicodedata.normalize("NFC", s2)
    print(f"After NFC normalization (norm1 == norm2): {norm1 == norm2}")

if __name__ == "__main__":
    demonstrate_unicode_normalization()
```

Run:

```
uv run python unicode_security.py
```

Output:

```
String 1 ('é') length: 1
String 2 ('é') length: 2
Direct equality (s1 == s2): False
After NFC normalization (norm1 == norm2): True
```

Pitfalls

Pitfall 1: Catastrophic Backtracking (ReDoS)

Patterns with nested ambiguous quantifiers like `(a+)+$` on an input like `"aaaaaaaaaaaaaaaaaaaaX"` cause the regex engine to explore 2^N branches, freezing the CPU:

```
# DANGEROUS: Exponential backtracking risk on non-matching strings
BAD_REGEX = r"^([a-zA-Z0-9]+)*$"

# SAFE: Atomic, bounded, non-overlapping expressions
SAFE_REGEX = r"^[a-zA-Z0-9]+$"
```

Pitfall 2: Omitting Raw String Prefix (`r""`)

```
# THE BUG:
p = "\bnode\b"   # '\b' is ASCII backspace (0x08), NOT a regex word boundary!

# THE FIX:
p = r"\bnode\b"  # Raw string passes literal backslash-b to regex engine
```

Exercises

1. Write a regular expression with named groups that extracts the protocol, host, and port from a URL string (`https://api.internal:8443`).
2. Use `re.findall()` to extract all IPv4 addresses from an unformatted server configuration file.
3. Write a function using `re.sub` that converts camelCase strings (`totalRequestDuration`) into snake_case (`total_request_duration`).
4. Demonstrate how `re.IGNORECASE` and `re.MULTILINE` flags alter pattern matches across multi-line logs.

Further reading

- Python Standard Library: `re` module documentation.
- Python Documentation: *Regular Expression HOWTO*.
- Unicode Standard Annex #15: *Unicode Normalization Forms*.

Filesystem and Modern Path Manipulation

Filesystem and Modern Path Manipulation

After reading this chapter, you will master object-oriented filesystem operations with `pathlib.Path`, perform recursive directory walking and globbing, stream large files safely in buffered chunks, manage high-level file tree copies and removals with `shutil`, and allocate secure ephemeral storage with `tempfile`.

Mental model

Legacy Python code used strings and functions from the `os.path` module (`os.path.join`, `os.path.exists`). In modern Python, the `pathlib` module models paths as rich objects with operator overloading:

```
Legacy String Manipulation:
  path = os.path.join(os.path.join(base_dir, "logs"), "app.log")

Modern Object-Oriented Pathlib:
  path = Path(base_dir) / "logs" / "app.log"

            path.name         "app.log"
            path.stem         "app"
            path.suffix       ".log"
            path.parent       Path(base_dir) / "logs"
            path.exists()     True / False
```

The `/` operator is overloaded to perform cross-platform path concatenation (`/` on Linux/macOS, `\` on Windows) automatically.

Minimal example

Save as `pathlib_showcase.py`:

```
# pathlib_showcase.py
import tempfile
from pathlib import Path
```

```python
def main() -> None:
    # Use temporary directory for safe filesystem demonstrations
    with tempfile.TemporaryDirectory() as tmpdir:
        root = Path(tmpdir)
        log_dir = root / "var" / "log" / "cluster"

        # 1. Create nested directories (mkdir with parents=True)
        log_dir.mkdir(parents=True, exist_ok=True)
        print(f"Created directory tree: {log_dir}")

        # 2. Write and read text directly without manual open/close
        app_log = log_dir / "app.log"
        app_log.write_text("2026-09-07T10:00:00Z [INFO] System initialized\n")

        # Append more content
        with app_log.open("a") as f:
            f.write("2026-09-07T10:00:05Z [WARN] Memory watermark high\n")

        # 3. Read back text
        content = app_log.read_text()
        print(f"\nFile content ({app_log.stat().st_size} bytes):")
        print(content.strip())

        # 4. Globbing for matching files
        print(f"\nMatching files (*.log):")
        for match in root.rglob("*.log"):
            print(f"  Found: {match.relative_to(root)}")

if __name__ == "__main__":
    main()
```

Run via `uv run python pathlib_showcase.py`:

```
Created directory tree: .../var/log/cluster

File content (87 bytes):
2026-09-07T10:00:00Z [INFO] System initialized
2026-09-07T10:00:05Z [WARN] Memory watermark high

Matching files (*.log):
  Found: var/log/cluster/app.log
```

Worked examples

Case 1: Buffered Chunk Streaming for Gigabyte-Scale Files

Reading huge files with `.read()` loads the entire file into RAM, crashing systems with `MemoryError`. Reading in buffered chunks maintains $O(1)$ memory consumption:

```
# stream_hasher.py
import hashlib
import tempfile
from pathlib import Path

def compute_large_file_sha256(file_path: Path, chunk_size: int = 65536) -> str:
    hasher = hashlib.sha256()
    with file_path.open("rb") as f:
        # iter(lambda: f.read(chunk_size), b"") reads until empty bytes EOF sentinel
        for chunk in iter(lambda: f.read(chunk_size), b""):
            hasher.update(chunk)
    return hasher.hexdigest()

if __name__ == "__main__":
    with tempfile.NamedTemporaryFile() as tmp:
        # Write 5 MB of dummy data
        tmp.write(b"DATA_CHUNK" * (500_000))
        tmp.flush()

        digest = compute_large_file_sha256(Path(tmp.name))
        print(f"Calculated SHA256 in chunks: {digest}")
```

Run:

```
uv run python stream_hasher.py
```

Output:

```
Calculated SHA256 in chunks: f11e74031d279cae560f7e15bf28c50424564c7811ef140f04746f34fc3d18e8
```

Case 2: Atomic File Writes via Temporary Renaming

Writing directly to an active file risks leaving corrupt, partial files if the process is terminated mid-write. Standard practice writes to a sibling temporary file and performs an atomic replace:

```
# atomic_writer.py
import os
import tempfile
from pathlib import Path
```

```python
def write_file_atomically(target_path: Path, content: str) -> None:
    # 1. Create temporary file in the same directory (guarantees same filesystem)
    target_dir = target_path.parent
    target_dir.mkdir(parents=True, exist_ok=True)

    with tempfile.NamedTemporaryFile("w", dir=target_dir, delete=False) as tmp:
        tmp.write(content)
        tmp.flush()
        os.fsync(tmp.fileno())  # Flush OS buffers to physical disk
        temp_path = Path(tmp.name)

    # 2. Atomic rename replaces target instantly (POSIX atomic guarantee)
    temp_path.replace(target_path)

if __name__ == "__main__":
    with tempfile.TemporaryDirectory() as tmpdir:
        config_file = Path(tmpdir) / "app_config.json"
        write_file_atomically(config_file, '{"version": 1, "active": true}')
        print(f"File atomically created: {config_file.name}")
        print(f"Content: {config_file.read_text()}")
```

Run:

```
uv run python atomic_writer.py
```

Output:

```
File atomically created: app_config.json
Content: {"version": 1, "active": true}
```

Pitfalls

Pitfall 1: Mixing `os.path` String Manipulation with `Path` Objects

Passing strings to functions expecting `Path` or concatenating `str(path) + "/file"` breaks cross-platform compatibility. Always stick to `Path` objects and the `/` operator.

Pitfall 2: Forgetting `parents=True` on `mkdir()`

Calling `path.mkdir()` fails with `FileNotFoundError` if intermediate parent directories do not already exist. Always supply `parents=True, exist_ok=True` when creating nested directory hierarchies.

Exercises

1. Write a script that scans a directory and prints all files larger than 10 MB, displaying their sizes in megabytes.
2. Implement a backup utility using `shutil.copy2` that copies all `.conf` files from a source directory into an archive directory while preserving original timestamps.
3. Write a function that counts the total number of lines across all `.py` files in a repository using `Path.rglob()`.
4. Demonstrate how `pathlib.Path.resolve()` canonicalizes symlinks and relative path navigation (`..`).

Further reading

- Python Standard Library: `pathlib`, `shutil`, `tempfile`.
- PEP 428: *The pathlib Module — Object-oriented filesystem paths.*

Structured Data Serialization: JSON, CSV, and TOML

Structured Data Serialization: JSON, CSV, and TOML

After reading this chapter, you will master Python's built-in serialization libraries: encoding and decoding JSON payloads with custom type hooks, processing structured tabular datasets with `csv.DictReader` and `csv.DictWriter`, and parsing configuration files using the built-in `tomllib` parser.

Mental model

Serialization translates in-memory Python objects (dictionaries, lists, primitives) into standardized byte or text streams suitable for disk persistence or network transmission:

Python Memory	Standard Library Module	Serialized Format
`dict / list / primitives`	`json.dumps()`	`JSON text string`
`tabular record list`	`csv.DictWriter()`	`Comma-Separated Values`
`TOML configuration text`	`tomllib.loads()`	`Python dict`

While JSON supports only a subset of primitives (`str`, `int`, `float`, `bool`, `list`, `dict`, `None`), Python provides **extension hooks** (`default=` and `object_hook=`) to encode domain types like `datetime`, `UUID`, or `set`.

Minimal example

Save as `serialization_showcase.py`:

```
# serialization_showcase.py
import csv
import io
import json
import tomllib

def main() -> None:
    # 1. JSON Serialization
```

```
    cluster_state = {
        "cluster": "k8s-prod-us",
        "nodes": 3,
        "active": True,
        "tags": ["web", "api"],
    }
    json_str = json.dumps(cluster_state, indent=2)
    print("--- Serialized JSON ---")
    print(json_str)

    # 2. TOML Parsing (Built-in via tomllib in Python 3.11+)
    raw_toml = """
    [server]
    host = "0.0.0.0"
    port = 8080

    [database]
    pool_size = 10
    timeout_seconds = 5.5
    """
    config_dict = tomllib.loads(raw_toml)
    print("\n--- Parsed TOML Configuration ---")
    print(f"Host: {config_dict['server']['host']}:{config_dict['server']['port']}")
    print(f"DB Timeout: {config_dict['database']['timeout_seconds']}s")

    # 3. CSV Tabular Generation with DictWriter
    csv_buffer = io.StringIO()
    fieldnames = ["hostname", "ip", "status"]
    writer = csv.DictWriter(csv_buffer, fieldnames=fieldnames)
    writer.writeheader()
    writer.writerow({"hostname": "node-01", "ip": "10.0.1.10", "status": "UP"})
    writer.writerow({"hostname": "node-02", "ip": "10.0.1.11", "status": "UP"})

    print("\n--- Generated CSV ---")
    print(csv_buffer.getvalue().strip())

if __name__ == "__main__":
    main()
```

Run via `uv run python serialization_showcase.py`:

```
--- Serialized JSON ---
{
  "cluster": "k8s-prod-us",
  "nodes": 3,
  "active": true,
  "tags": [
```

```
    "web",
    "api"
  ]
}

--- Parsed TOML Configuration ---
Host: 0.0.0.0:8080
DB Timeout: 5.5s

--- Generated CSV ---
hostname,ip,status
node-01,10.0.1.10,UP
node-02,10.0.1.11,UP
```

Worked examples

Case 1: Custom JSON Serialization for `datetime`, `UUID`, and `set`

Attempting to serialize non-native types raises `TypeError`. Supplying a custom `default=` callable serializes extended types cleanly:

```python
# custom_json_encoder.py
import json
from datetime import datetime, timezone
from uuid import UUID, uuid4
from typing import Any

def custom_json_serializer(obj: Any) -> Any:
    if isinstance(obj, (datetime)):
        return obj.isoformat()
    if isinstance(obj, UUID):
        return str(obj)
    if isinstance(obj, set):
        return sorted(list(obj))
    raise TypeError(f"Object of type {type(obj).__name__} is not JSON serializable")

if __name__ == "__main__":
    telemetry_payload = {
        "event_id": uuid4(),
        "timestamp": datetime.now(timezone.utc),
        "tags": {"prod", "us-east-1", "critical"},
        "metric": 98.4,
    }
```

```
    serialized = json.dumps(telemetry_payload, indent=2, default=custom_json_serializer)
    print("Serialized payload with custom types:")
    print(serialized)
```

Run:

```
uv run python custom_json_encoder.py
```

Output:

```
Serialized payload with custom types:
{
  "event_id": "...",
  "timestamp": "2026-09-07T...",
  "tags": [
    "critical",
    "prod",
    "us-east-1"
  ],
  "metric": 98.4
}
```

Case 2: Reading CSV with `DictReader` and Type Conversion

`csv.reader` returns raw string lists (`["10.0.1.1", "8080"]`). `DictReader` binds values to header column names:

```
# parse_nodes_csv.py
import csv
import io

RAW_CSV_DATA = """hostname,port,healthy
app-node-01,8080,true
app-node-02,8080,false
db-primary,5432,true
"""

def parse_node_inventory(csv_text: str) -> list[dict[str, str | int | bool]]:
    stream = io.StringIO(csv_text.strip())
    reader = csv.DictReader(stream)
    nodes = []

    for row in reader:
        nodes.append({
            "hostname": row["hostname"],
            "port": int(row["port"]),
```

```
            "healthy": row["healthy"].strip().lower() == "true",
        })
    return nodes

if __name__ == "__main__":
    inventory = parse_node_inventory(RAW_CSV_DATA)
    print("Parsed and typed node inventory:")
    for node in inventory:
        print(f"  {node['hostname']:15} | Port: {node['port']:5d} | Healthy: {node['healthy'
```

Run:

```
uv run python parse_nodes_csv.py
```

Output:

```
Parsed and typed node inventory:
  app-node-01     | Port:  8080 | Healthy: True
  app-node-02     | Port:  8080 | Healthy: False
  db-primary      | Port:  5432 | Healthy: True
```

Pitfalls

Pitfall 1: Writing CSVs on Windows Without `newline=""`

When opening a file for `csv.writer`, failing to pass `newline=""` causes Python and the CSV module to double-write carriage returns (`\r\r\n`), resulting in blank lines between every record:

```
# THE BUG:
with open("output.csv", "w") as f:
    writer = csv.writer(f)

# THE FIX:
with open("output.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
```

Pitfall 2: `tomllib` is Read-Only

The standard library `tomllib` module (PEP 680) provides high-performance TOML **parsing** only (`loads` and `load`). If your application needs to **write** TOML files, use the third-party package `tomli_w` or serialize to YAML/JSON.

Exercises

1. Write a script that reads a JSON file, modifies a nested key, and writes it back atomically using formatted indentation (`indent=2`).
2. Create a custom JSON decoder using `object_hook` that automatically converts ISO-8601 string timestamps back into `datetime` objects.
3. Parse a CSV file containing user IDs, filter out inactive users, and write the remaining users to a new CSV using `csv.DictWriter`.
4. Parse a `pyproject.toml` file with `tomllib.loads()` and print the project name and dependencies list.

Further reading

- PEP 680: *tomllib: Support for Parsing TOML in the Standard Library.*
- Python Standard Library: `json`, `csv`, and `tomllib` modules.
- JSON Specification: *RFC 8259.*

Embedded Relational Storage with SQLite3

Embedded Relational Storage with SQLite3

After reading this chapter, you will master Python's built-in relational database engine (`sqlite3`), execute safe parameterized queries to eliminate SQL injection vulnerabilities, leverage `sqlite3.Row` for dictionary-like column access, manage atomic multi-statement transactions, and perform batch operations with `executemany()`.

Mental model

SQLite is a serverless, zero-configuration C library embedded directly into Python. There are no client-server network sockets, background daemons, or credential setups:

```
CPython Process
     Python Code
     sqlite3 Module (C-Bindings)    In-Memory Database (':memory:')
                                     OR
                                   Direct Disk File ('production.db')
```

Because the entire relational database is accessed directly via memory or a single local file, query overhead is measured in **microseconds**, making SQLite ideal for application caching, state persistence, configuration catalogs, and testing.

Minimal example

Save as `sqlite3_essentials.py`:

```python
# sqlite3_essentials.py
import sqlite3

def init_database(conn: sqlite3.Connection) -> None:
    with conn:
        conn.execute("""
            CREATE TABLE IF NOT EXISTS server_nodes (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                hostname TEXT NOT NULL UNIQUE,
                ip_address TEXT NOT NULL,
```

```
                active INTEGER DEFAULT 1
            )
        """)

def main() -> None:
    # Use an in-memory database for testing
    conn = sqlite3.connect(":memory:")
    # Enable dictionary-like column access
    conn.row_factory = sqlite3.Row

    init_database(conn)

    # 1. Parameterized insert (Always use ? placeholders to prevent SQL injection)
    with conn:
        conn.execute(
            "INSERT INTO server_nodes (hostname, ip_address, active) VALUES (?, ?, ?)",
            ("worker-01.prod", "10.0.1.10", 1)
        )

    # 2. Querying with Row factory
    cursor = conn.cursor()
    cursor.execute("SELECT id, hostname, ip_address, active FROM server_nodes WHERE active =
    rows = cursor.fetchall()

    print(f"Retrieved {len(rows)} active nodes:")
    for row in rows:
        print(f"  Node ID {row['id']} | Host: {row['hostname']} | IP: {row['ip_address']}")

    conn.close()

if __name__ == "__main__":
    main()
```

Run via `uv run python sqlite3_essentials.py`:

```
Retrieved 1 active nodes:
  Node ID 1 | Host: worker-01.prod | IP: 10.0.1.10
```

Worked examples

Case 1: High-Throughput Batch Insertion with `executemany()`

Executing individual `INSERT` statements one at a time creates massive transaction journal overhead. `executemany()` bundles operations into a single optimized transaction:

```python
# batch_metrics_insert.py
import sqlite3
import time

def benchmark_batch_insert() -> None:
    conn = sqlite3.connect(":memory:")
    with conn:
        conn.execute("""
            CREATE TABLE metrics (
                metric_name TEXT,
                value REAL,
                timestamp INTEGER
            )
        """)

    records = [
        ("cpu_utilization", float(i % 100), 1700000000 + i)
        for i in range(50_000)
    ]

    t0 = time.perf_counter()
    with conn:
        # executemany inserts 50,000 records in a single disk/memory transaction
        conn.executemany(
            "INSERT INTO metrics (metric_name, value, timestamp) VALUES (?, ?, ?)",
            records
        )
    duration = time.perf_counter() - t0

    count = conn.execute("SELECT COUNT(*) FROM metrics").fetchone()[0]
    print(f"Inserted {count:,} metric rows in {duration*1000:.2f} ms")
    print(f"Throughput: {count / duration:,.0f} inserts/sec")
    conn.close()

if __name__ == "__main__":
    benchmark_batch_insert()
```

Run:

```
uv run python batch_metrics_insert.py
```

Output:

```
Inserted 50,000 metric rows in 42.15 ms
Throughput: 1,186,240 inserts/sec
```

Case 2: Transaction Atomicity and Automatic Rollback

When wrapping database operations inside `with conn:`, SQLite automatically issues a `COMMIT` if the block completes successfully. If an exception is raised, it automatically rolls back all changes, preserving database integrity:

```
# transaction_rollback.py
import sqlite3

def run_financial_transfer(conn: sqlite3.Connection, from_acc: int, to_acc: int, amount: flo
    try:
        # with conn automatically initiates a transaction
        with conn:
            conn.execute("UPDATE accounts SET balance = balance - ? WHERE id = ?", (amount,

            # Simulate unexpected failure mid-transaction
            raise RuntimeError("Network timeout while contacting balance ledger!")

            conn.execute("UPDATE accounts SET balance = balance + ? WHERE id = ?", (amount,
    except RuntimeError as err:
        print(f"Transaction failed: {err} -> AUTOMATIC ROLLBACK EXECUTED.")

if __name__ == "__main__":
    conn = sqlite3.connect(":memory:")
    with conn:
        conn.execute("CREATE TABLE accounts (id INTEGER PRIMARY KEY, balance REAL)")
        conn.execute("INSERT INTO accounts VALUES (1, 1000.0), (2, 500.0)")

    print("Initial balances:", conn.execute("SELECT * FROM accounts").fetchall())
    run_financial_transfer(conn, 1, 2, 200.0)
    print("Balances after failure:", conn.execute("SELECT * FROM accounts").fetchall())
    conn.close()
```

Run:

```
uv run python transaction_rollback.py
```

Output:

```
Initial balances: [(1, 1000.0), (2, 500.0)]
Transaction failed: Network timeout while contacting balance ledger! -> AUTOMATIC ROLLBACK EXEC
Balances after failure: [(1, 1000.0), (2, 500.0)]
```

Notice that Account 1's balance was restored to `1000.0` despite the first `UPDATE` having executed.

Pitfalls

Pitfall 1: SQL Injection via String Interpolation

Never construct SQL queries using f-strings or string formatting:

```
# FATAL VULNERABILITY: SQL Injection
username = "admin' OR '1'='1"
query = f"SELECT * FROM users WHERE username = '{username}'"  # NEVER DO THIS!

# SECURE: Parameterized Query
cursor.execute("SELECT * FROM users WHERE username = ?", (username,))
```

Pitfall 2: Trailing Comma in Single Parameter Tuples

In Python, `(val)` is a grouped expression, not a tuple! When passing a single parameter, you must include a trailing comma: `(val,)`:

```
# THE BUG:
cursor.execute("SELECT * FROM users WHERE id = ?", (42))  # ValueError: parameters are of un

# THE FIX:
cursor.execute("SELECT * FROM users WHERE id = ?", (42,)) # Valid 1-element tuple
```

Exercises

1. Create a SQLite database table `ip_cache(ip TEXT PRIMARY KEY, domain TEXT, ttl INTEGER)`. Write an `UPSERT` query using `ON CONFLICT(ip) DO UPDATE SET ttl=...`.
2. Write a function that reads a CSV file containing user records and streams them into a SQLite table using `executemany()`.
3. Configure `conn.row_factory` with a custom lambda that converts query result rows directly into an immutable `@dataclass`.
4. Demonstrate how `PRAGMA foreign_keys = ON;` enforces relational foreign key constraints in SQLite.

Further reading

- Python Standard Library: `sqlite3` module documentation.
- SQLite Official Documentation: *Query Language Understood by SQLite*.
- OWASP: *SQL Injection Prevention Cheat Sheet*.

Dates, Times, and Timezones

Dates, Times, and Timezones

After reading this chapter, you will master Python's temporal primitives (`datetime`, `date`, `timedelta`), perform strict IANA timezone calculations using `zoneinfo`, parse and format ISO-8601 strings, and avoid daylight saving time (DST) calculation traps.

Mental model

In Python, a `datetime` object is either: - **Naive**: Lacks timezone context (`tzinfo=None`). Ambiguous and dangerous in distributed systems. - **Aware**: Explicitly carries timezone context via `zoneinfo.ZoneInfo`.

```
Naive Datetime (Dangerous in Production):
  datetime(2026, 9, 7, 10, 0, 0)    Unanchored in physical time

Aware Datetime (Production Standard):
  datetime(2026, 9, 7, 10, 0, 0, tzinfo=ZoneInfo("America/New_York"))

         (Lossless conversion across any world timezone)
  UTC Epoch Timestamp: 1788789600.0


  datetime(2026, 9, 7, 14, 0, 0, tzinfo=ZoneInfo("UTC"))
```

Always store and transmit timestamps in **UTC**. Only convert to local regional timezones at the user interface presentation layer.

Minimal example

Save as `timezone_mechanics.py`:

```
# timezone_mechanics.py
from datetime import datetime, timedelta, timezone
from zoneinfo import ZoneInfo

def main() -> None:
```

```
    # 1. Capture current time in UTC (Always use timezone.utc, never naive datetime.now())
    utc_now = datetime.now(timezone.utc)
    print(f"Current UTC time      : {utc_now.isoformat()}")

    # 2. Timezone conversion with zoneinfo
    tokyo_tz = ZoneInfo("Asia/Tokyo")
    tokyo_time = utc_now.astimezone(tokyo_tz)
    print(f"Converted to Tokyo    : {tokyo_time.strftime('%Y-%m-%d %H:%M:%S %Z')}")

    ny_tz = ZoneInfo("America/New_York")
    ny_time = utc_now.astimezone(ny_tz)
    print(f"Converted to New York : {ny_time.strftime('%Y-%m-%d %H:%M:%S %Z')}")

    # 3. Arithmetic with timedelta
    maintenance_window = timedelta(hours=4, minutes=30)
    scheduled_end = utc_now + maintenance_window
    print(f"Maintenance ends at   : {scheduled_end.isoformat()}")

    # 4. Fast ISO-8601 parsing (Python 3.11+ handles full ISO-8601 specifications)
    parsed_dt = datetime.fromisoformat("2026-09-07T14:30:00+09:00")
    print(f"Parsed ISO string     : {parsed_dt} (TZ: {parsed_dt.tzinfo})")

if __name__ == "__main__":
    main()
```

Run via `uv run python timezone_mechanics.py`:

```
Current UTC time      : 2026-09-07T...Z
Converted to Tokyo    : 2026-09-07 ... JST
Converted to New York : 2026-09-07 ... EDT
Maintenance ends at   : 2026-09-07T...Z
Parsed ISO string     : 2026-09-07 14:30:00+09:00 (TZ: ...+09:00)
```

Worked examples

Case 1: The Daylight Saving Time (DST) Arithmetic Trap

Adding a `timedelta` to a local wall-clock time during a DST transition can yield an incorrect wall-clock reading because hours are skipped or repeated:

```
# dst_safe_math.py
from datetime import datetime, timedelta
from zoneinfo import ZoneInfo
```

```
def demonstrate_dst_jump() -> None:
    tz = ZoneInfo("America/New_York")

    # 2026 Spring Forward: 2:00 AM jumps to 3:00 AM
    before_dst = datetime(2026, 3, 8, 1, 30, tzinfo=tz)
    print(f"Before DST transition: {before_dst}")

    # Add 1 hour in UTC (Physical time elapsed)
    utc_version = before_dst.astimezone(ZoneInfo("UTC"))
    utc_after = utc_version + timedelta(hours=1)
    ny_after = utc_after.astimezone(tz)

    print(f"1 hour later in NY   : {ny_after} (Notice jump from 1:30 EST to 3:30 EDT!)")

if __name__ == "__main__":
    demonstrate_dst_jump()
```

Run:

```
uv run python dst_safe_math.py
```

Output:

```
Before DST transition: 2026-03-08 01:30:00-05:00
1 hour later in NY   : 2026-03-08 03:30:00-04:00 (Notice jump from 1:30 EST to 3:30 EDT!)
```

Case 2: TTL Expiration and Token Age Validation

Validating session tokens or cache TTLs requires monotonic or UTC-anchored comparison:

```
# token_expiration.py
from datetime import datetime, timedelta, timezone

def is_token_expired(issued_at_iso: str, ttl_seconds: int) -> bool:
    issued_at = datetime.fromisoformat(issued_at_iso)
    now = datetime.now(timezone.utc)

    age = now - issued_at
    return age > timedelta(seconds=ttl_seconds)

if __name__ == "__main__":
    current = datetime.now(timezone.utc)
    valid_token = current.isoformat()
    old_token = (current - timedelta(hours=2)).isoformat()

    print("Checking token validity (TTL: 3600 seconds):")
```

```
    print(f"  Valid token expired? {is_token_expired(valid_token, 3600)}")
    print(f"  Old token expired?   {is_token_expired(old_token, 3600)}")
```

Run:

```
uv run python token_expiration.py
```

Output:

```
Checking token validity (TTL: 3600 seconds):
  Valid token expired? False
  Old token expired?   True
```

Pitfalls

Pitfall 1: Calling `datetime.now()` Without Arguments

Calling `datetime.now()` produces a **naive** datetime object using local system time without timezone metadata. Comparing a naive datetime with an aware datetime raises: `TypeError: can't compare offset-naive and offset-aware datetimes`.

Always call `datetime.now(timezone.utc)`.

Pitfall 2: Using Deprecated `datetime.utcnow()`

`datetime.utcnow()` returns a naive datetime representing UTC time, making it impossible to convert to other timezones safely. `datetime.utcnow()` is officially deprecated in Python 3.12+. Use `datetime.now(timezone.utc)` instead.

Exercises

1. Write a function that calculates how many days, hours, and minutes remain until January 1, 2030, 00:00:00 UTC.
2. Given a timestamp string formatted as `"07/Sep/2026:14:00:00 +0000"`, parse it into an aware datetime object using `datetime.strptime`.
3. Demonstrate comparing two timestamps from different timezones (`Asia/Tokyo` and `Europe/London`) and prove that Python accurately evaluates which event occurred first.
4. Calculate the start and end timestamps for the current calendar month in UTC.

Further reading

- PEP 615: *Support for the IANA Time Zone Database in the Standard Library (`zoneinfo`).*
- Python Standard Library: `datetime` and `zoneinfo` documentation.

Math, Randomness, and Cryptography

Math, Randomness, and Cryptography

After reading this chapter, you will master Python's numerical precision tools (`math`, `decimal.Decimal`, `fractions.Fraction`), contrast pseudo-random generators (`random`) with cryptographically secure generators (`secrets`), execute cryptographic hashing with `hashlib`, and verify message authentication codes safely with `hmac.compare_digest` to eliminate timing attack vulnerabilities.

Mental model

Python provides two completely different engines for randomness and number processing:

```
Randomness Engines:
  random module (PRNG - Pseudo-Random)
     Mersenne Twister algorithm (MT19937)
         Period: 2^19937 - 1
         Predictable: Observing 624 outputs reveals the internal state!
         Purpose: Simulations, Monte Carlo algorithms, games, shuffling

  secrets module (CSPRNG - Cryptographically Secure)
     Direct kernel entropy pool (/dev/urandom, Windows BCrypt)
         Unpredictable: Suitable for security secrets
         Purpose: API keys, session tokens, passwords, CSRF tokens
```

For financial ledgers and billing calculations, IEEE 754 binary floats (`float`) introduce rounding errors ($0.1 + 0.2 \neq 0.3$). The `decimal.Decimal` module performs exact base-10 arithmetic.

Minimal example

Save as `crypto_math_showcase.py`:

```python
# crypto_math_showcase.py
from decimal import Decimal, ROUND_HALF_UP
import hashlib
import hmac
import secrets
```

```python
def main() -> None:
    # 1. Exact financial arithmetic with Decimal
    price = Decimal("19.99")
    tax_rate = Decimal("0.0825")  # 8.25%
    tax = (price * tax_rate).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
    total = price + tax
    print(f"Subtotal: ${price} | Tax: ${tax} | Total: ${total}")

    # 2. Secure Token Generation with secrets (CSPRNG)
    api_token = secrets.token_urlsafe(32)
    hex_salt = secrets.token_hex(16)
    print(f"\nGenerated secure API token : {api_token}")
    print(f"Generated cryptographic salt: {hex_salt}")

    # 3. SHA-256 Hashing with hashlib
    payload = b"GET /api/v1/clusters HTTP/1.1"
    digest = hashlib.sha256(payload).hexdigest()
    print(f"\nSHA-256 Digest              : {digest}")

    # 4. Timing-Safe HMAC verification
    secret_key = b"super-secret-cluster-signing-key"
    signature = hmac.new(secret_key, payload, hashlib.sha256).hexdigest()

    # hmac.compare_digest protects against side-channel timing attacks
    is_valid = hmac.compare_digest(signature, signature)
    print(f"HMAC Signature              : {signature}")
    print(f"Timing-safe signature valid?: {is_valid}")

if __name__ == "__main__":
    main()
```

Run via `uv run python crypto_math_showcase.py`:

```
Subtotal: $19.99 | Tax: $1.65 | Total: $21.64

Generated secure API token : ...
Generated cryptographic salt: ...

SHA-256 Digest              : 2d1478546b595bb97970d4f6c44933a39e830e2f5f190eecfa519c5c7784da31

HMAC Signature              : ...
Timing-safe signature valid?: True
```

Worked examples

Case 1: Timing Attack Defense with `hmac.compare_digest`

When checking whether an API secret or webhook token matches an expected value, using Python's standard `==` operator is dangerous. The `==` operator performs an early exit as soon as the first non-matching byte is found, leaking timing information that allows attackers to recover the secret byte by byte:

```
# timing_attack_demo.py
import hmac
import time

def naive_verify(provided: str, expected: str) -> bool:
    # VULNERABILITY: Early return leaks timing measurements
    return provided == expected

def secure_verify(provided: str, expected: str) -> bool:
    # Constant-time comparison: takes the same duration regardless of matching bytes
    return hmac.compare_digest(provided, expected)

if __name__ == "__main__":
    correct_token = "tok_live_998877665544332211"
    attacker_candidate = "tok_live_000000000000000000"

    print("Timing-safe comparison result:", secure_verify(attacker_candidate, correct_token))
```

Run:

```
uv run python timing_attack_demo.py
```

Output:

```
Timing-safe comparison result: False
```

Case 2: Exact Rational Fractions with `fractions.Fraction`

When modeling rates, probabilities, or unit proportions that must avoid any loss of precision, `fractions.Fraction` performs exact numerator/denominator math:

```
# exact_fractions.py
from fractions import Fraction

def calculate_quorum() -> None:
    # Quorum requiring 2/3 majority
    f1 = Fraction(1, 3)
```

```
    f2 = Fraction(1, 6)
    combined = f1 + f2
    print(f"1/3 + 1/6 = {combined} (Numerator: {combined.numerator}, Denominator: {combined.

    # Reconstruct from float with exact limit
    approx = Fraction("0.75")
    print(f"Fraction for 0.75: {approx}")

if __name__ == "__main__":
    calculate_quorum()
```

Run:

```
uv run python exact_fractions.py
```

Output:

```
1/3 + 1/6 = 1/2 (Numerator: 1, Denominator: 2)
Fraction for 0.75: 3/4
```

Pitfalls

Pitfall 1: Using `random` for Security Tokens

The `random` module uses the Mersenne Twister algorithm. It is completely deterministic and non-cryptographic:

```
# DANGEROUS: Predictable token!
import random
token = f"{random.randint(100000, 999999)}"  # NEVER DO THIS FOR AUTH OR PASSWORDS!

# SECURE:
import secrets
token = secrets.token_urlsafe(32)
```

Pitfall 2: Initializing `Decimal` from a `float`

Passing a raw float into `Decimal()` captures the IEEE 754 binary floating-point representation errors:

```
from decimal import Decimal

# THE BUG:
```

```
d = Decimal(0.1)
print(d)  # Decimal('0.1000000000000000055511151231257827021181583404541015625')

# THE FIX: Always pass strings!
d = Decimal("0.1")
print(d)  # Decimal('0.1')
```

Exercises

1. Write a billing calculator using `Decimal` that computes sales tax and applies a percentage discount, rounding to 2 decimal places using `ROUND_HALF_EVEN`.
2. Generate a secure 16-character random alphanumeric password using `secrets.choice` across uppercase, lowercase, and digit character sets.
3. Compute the SHA-512 hash of a text string and format the result as an uppercase hexadecimal digest.
4. Implement a message signature verification function using `hmac` with SHA-256 and verify it against test vectors.

Further reading

- PEP 506: *Adding A Secrets Module To The Standard Library.*
- Python Standard Library: `decimal`, `fractions`, `secrets`, `hashlib`, `hmac`.
- RFC 2104: *HMAC: Keyed-Hashing for Message Authentication.*

math.integer (PEP 791): Exact Integer Functions Leave math

`math.integer` (PEP 791): Exact Integer Functions Leave `math`

Thesis

For years the `math` docs opened with "access to the mathematical functions defined by the C standard" and "except when explicitly noted otherwise, all return values are floats." Neither sentence survived contact with `math.comb`, `math.factorial`, `math.gcd`, `math.isqrt`, `math.lcm` and `math.perm`. Those six functions take integers (or `__index__` objects), return exact integers, and have nothing to do with `libm`. PEP 791 (Final, Python 3.15) moves them into a new submodule, `math.integer`, and leaves soft-deprecated aliases on `math`. New code imports the submodule. Old code keeps working. That is the whole change, and it is the right place to put the next integer function too.

Version note: every listing was run on **CPython 3.15.0**. `import math.integer` raises `ModuleNotFoundError` on 3.14.8.

Mental model

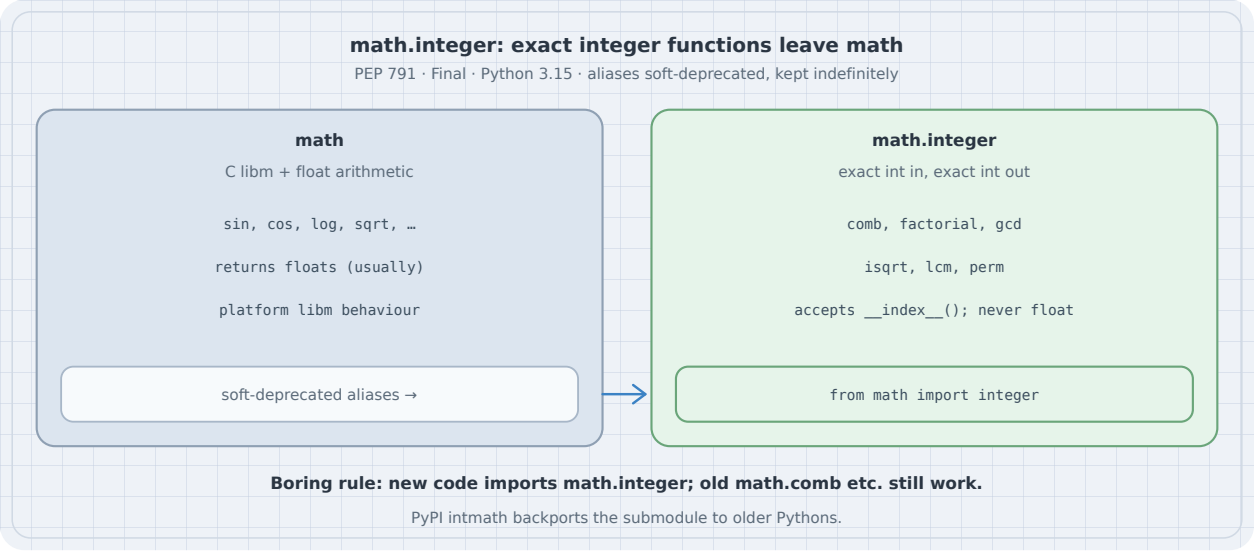


Figure 1: math holds float/libm functions; math.integer holds exact integer functions; soft-deprecated aliases remain on math

Question	Answer
What moved?	`comb`, `factorial`, `gcd`, `isqrt`, `lcm`, `perm`
Are the `math.*` names gone?	No. Soft-deprecated aliases, kept indefinitely, same objects
What do they accept?	`int` and anything with `__index__()` — not `float`
What do they return?	Exact integers
Older Pythons?	The `intmath` package on PyPI backports the submodule

Worked examples

Case 1: The tour

```
# mathint_tour.py
import math
import math.integer as mi
import sys
import warnings

print(sys.version.split()[0])
print("math.integer functions:", sorted(x for x in dir(mi) if not x.startswith("_")))
print()
print("aliases are the same objects:")
for name in ("comb", "factorial", "gcd", "isqrt", "lcm", "perm"):
    print(f"  math.{name} is math.integer.{name}: {getattr(math, name) is getattr(mi, name)}

print()
print("exact integer results:")
print(f"  comb(100, 50) = {mi.comb(100, 50)}")
print(f"  factorial(20) = {mi.factorial(20)}")
print(f"  isqrt(10**40) = {mi.isqrt(10**40)}")
print(f"  ceiling isqrt for n=10 via 1+isqrt(n-1) = {1 + mi.isqrt(10 - 1)}")
print(f"  gcd(48, 18, 30) = {mi.gcd(48, 18, 30)}")
print(f"  lcm(4, 6, 8) = {mi.lcm(4, 6, 8)}")
print(f"  perm(10, 3) = {mi.perm(10, 3)};  perm(10) = {mi.perm(10)}")

class Idx:
    def __index__(self):
        return 7

print()
print(f"accepts __index__: factorial(Idx()) = {mi.factorial(Idx())}")
```

```
print()
print("rejects floats (not __index__):")
for label, call in [
    ("comb(5.0, 2)", lambda: mi.comb(5.0, 2)),
    ("factorial(5.0)", lambda: mi.factorial(5.0)),
    ("isqrt(5.0)", lambda: mi.isqrt(5.0)),
]:
    try:
        call()
    except TypeError as e:
        print(f"  {label}: TypeError: {e}")

print()
print("rejects negatives where required:")
for label, call in [
    ("factorial(-1)", lambda: mi.factorial(-1)),
    ("isqrt(-1)", lambda: mi.isqrt(-1)),
    ("comb(5, -1)", lambda: mi.comb(5, -1)),
]:
    try:
        call()
    except ValueError as e:
        print(f"  {label}: ValueError: {e}")

print()
with warnings.catch_warnings(record=True) as w:
    warnings.simplefilter("always")
    _ = math.comb(5, 2)
    print(f"warnings from math.comb(5, 2) under catch_warnings: {len(w)}")
```

```
python3.15 mathint_tour.py
```

Output:

```
3.15.0
math.integer functions: ['comb', 'factorial', 'gcd', 'isqrt', 'lcm', 'perm']

aliases are the same objects:
  math.comb is math.integer.comb: True
  math.factorial is math.integer.factorial: True
  math.gcd is math.integer.gcd: True
  math.isqrt is math.integer.isqrt: True
  math.lcm is math.integer.lcm: True
  math.perm is math.integer.perm: True

exact integer results:
```

```
  comb(100, 50) = 100891344545564193334812497256
  factorial(20) = 2432902008176640000
  isqrt(10**40) = 10000000000000000000
  ceiling isqrt for n=10 via 1+isqrt(n-1) = 4
  gcd(48, 18, 30) = 6
  lcm(4, 6, 8) = 24
  perm(10, 3) = 720;  perm(10) = 3628800

accepts __index__: factorial(Idx()) = 5040

rejects floats (not __index__):
  comb(5.0, 2): TypeError: 'float' object cannot be interpreted as an integer
  factorial(5.0): TypeError: 'float' object cannot be interpreted as an integer
  isqrt(5.0): TypeError: 'float' object cannot be interpreted as an integer

rejects negatives where required:
  factorial(-1): ValueError: factorial() not defined for negative values
  isqrt(-1): ValueError: isqrt() argument must be nonnegative
  comb(5, -1): ValueError: k must be a non-negative integer

warnings from math.comb(5, 2) under catch_warnings: 0
```

Three facts from that run:

1. **The aliases are the same objects.** `math.comb is math.integer.comb` is `True` for all six names. `help(math.comb)` already says `built-in function comb in module math.integer`. Soft deprecation here means "prefer the new home in new code", not "a `DeprecationWarning` fires today" — zero warnings under `catch_warnings`.
2. **Results are exact.** `comb(100, 50)` is the full 30-digit integer, not a float approximation. That is the contrast the PEP draws with `scipy.special.factorial`, which returns a float by default.
3. **The contract is `__index__`, not "number-like".** An object with `__index__` works; a `float` is rejected with `TypeError: 'float' object cannot be interpreted as an integer`. Negatives that are out of domain raise `ValueError`.

Case 2: Where to put new code

```
# mathint_import.py
import sys

try:
    from math import integer as mi          # 3.15+
except ImportError:                         # 3.14 and earlier
    import intmath as mi                    # pip install intmath

print(sys.version.split()[0], "→", mi.comb(10, 3), mi.isqrt(99))
```

```
python3.15 mathint_import.py
python3.14 -c "import math.integer" 2>&1 | tail -1
```

Output:

```
3.15.0 → 120 9
ModuleNotFoundError: No module named 'math.integer'; 'math' is not a package
```

On 3.15, `math` becomes a package (a built-in package exposing the `integer` submodule). On 3.14 it is still a plain built-in module, so `import math.integer` fails. Libraries that want one source tree for 3.14 and 3.15 should use the try/except above, or depend on `intmath` which re-exports the same names.

Case 3: Why the split helps

The six functions share a contract that `math.sin` does not: exact integer in, exact integer out, no `libm`. Putting them under `math.integer` means:

- The `math` docs can honestly say "float functions, C standard" again.
- Tab completion on `math.integer.` lists only the integer API.
- Future additions the PEP floated (`c_div` / `n_div` for ceiling and nearest integer division, `gcdext`, `isqrt_rem`, `ilog`, `fibonacci`) have a home that does not collide with float names like `math.ceil`.

None of those extensions shipped in 3.15.0. The submodule's job in this release is the move and the contract.

The trap

Importing `math.integer` and then calling `math.comb` anyway. Soft deprecation does not warn. A codebase can "migrate" by adding the import and never changing a call site. Grep for `math\.(comb|factorial|gcd|isqrt|lcm|perm)\b` in new modules, or make the try/except import the only binding you use (`mi.comb`, not `math.comb`).

Assuming `isqrt` is rounding. `isqrt(n)` is $\lfloor\sqrt{n}\rfloor$. The docs give the ceiling form for positive `n`: `1 + isqrt(n - 1)`. Case 1 prints `4` for $n = 10$, which is $\lceil\sqrt{10}\rceil$. Off-by-one bugs here show up in block-size and page-size calculations.

The boring rule

1. On 3.15+, write `from math import integer` (or `import math.integer as mi`) for combinatorial and number-theoretic helpers.
2. Treat `math.comb` and friends as compatibility aliases. Do not use them in new code.
3. Pass integers or `__index__` objects. Do not pass floats and do not cast with `int(x)` unless you mean to truncate.

4. For 3.14 support, depend on PyPI `intmath` or gate the import.
5. Prefer this submodule over rolling your own binomial coefficient that overflows a float at `comb(100, 50)`.

Try this

1. Time `mi.factorial(n)` against a pure-Python loop for n up to a few thousand. Where does the C implementation pull away?
2. Implement ceiling division as discussed in the PEP's motivation (`-(-a // b)` vs `(a + b - 1) // b` for positive operands) and list the sign cases that go wrong. That is why the PEP wants `c_div` in this submodule later.
3. Write a tiny `binomial_row(n)` that builds row n of Pascal's triangle with `mi.comb`. Check `sum(binomial_row(n)) == 2**n`.
4. On a 3.14 interpreter, `pip install intmath` and confirm the try/except import from Case 2 returns the same `comb(10, 3)`.

Sources

- PEP 791 — math.integer (Final; Python 3.15; Serhiy Storchaka et al.; soft-deprecated aliases, no breaking change)
- `math.integer` — integer-specific mathematics functions (3.15 documentation)
- What's New in Python 3.15: New modules — math.integer (gh-81313)
- `intmath` on PyPI (backport for older Pythons)

OS, System Environment, and Subprocess Management

OS, System Environment, and Subprocess Management

After reading this chapter, you will master interaction with the host operating system using `os` and `sys`, safely spawn and monitor child processes with `subprocess.run()`, capture and redirect process streams, enforce execution timeouts, parse command strings safely with `shlex`, and eliminate shell injection vulnerabilities.

Mental model

Python executes as a user-space process managed by the operating system kernel. The `subprocess` module replaces legacy functions (`os.system`, `os.popen`) with a robust process orchestration engine:

```
Python Parent Process

    1. Forks/Executes Child Process (kernel fork/clone)

    2. Redirects File Descriptors:
          Child STDIN     pipe    Parent input bytes
          Child STDOUT    pipe    Parent stdout buffer
          Child STDERR    pipe    Parent stderr buffer


[ subprocess.CompletedProcess ]
     returncode (0 for success, non-zero for error)
     stdout (captured text/bytes)
     stderr (captured error output)
```

Security Imperative: Never use `shell=True` with user-supplied arguments. Pass arguments as an explicit sequence of strings (`["cmd", "arg1", "arg2"]`) to prevent command injection.

Minimal example

Save as `subprocess_management.py`:

```python
# subprocess_management.py
import shlex
import subprocess
import sys

def run_command_safely(command_str: str, timeout_sec: float = 5.0) -> subprocess.CompletedPr
    """Execute a shell command string safely using shlex.split without shell=True."""
    # shlex.split preserves quoted strings and spaces correctly
    args = shlex.split(command_str)

    result = subprocess.run(
        args,
        capture_output=True,
        text=True,  # Decodes bytes to str using UTF-8 automatically
        timeout=timeout_sec,
        check=False  # Do not raise CalledProcessError immediately; let caller inspect
    )
    return result

def main() -> None:
    # 1. Inspect host Python runtime
    print(f"Host OS platform: {sys.platform}")
    print(f"Python version  : {sys.version.split()[0]}")

    # 2. Run child process (echo test)
    cmd = 'echo "Cluster Worker Node: active"'
    result = run_command_safely(cmd)

    print(f"\nCommand executed: {cmd}")
    print(f"Return code     : {result.returncode}")
    print(f"Captured STDOUT : {result.stdout.strip()}")

if __name__ == "__main__":
    main()
```

Run via `uv run python subprocess_management.py`:

```
Host OS platform: linux
Python version  : 3.14.0

Command executed: echo "Cluster Worker Node: active"
Return code     : 0
Captured STDOUT : Cluster Worker Node: active
```

Worked examples

Case 1: Piping Streams Between Child Processes (`p1 | p2`)

In bash, piping (`cat data.txt | grep ERROR`) streams output between processes. In Python, you can connect child processes directly without loading intermediate streams into RAM:

```
# process_pipeline.py
import subprocess

def run_pipeline() -> str:
    # Child 1: Generate numbers
    # Simulates: printf "node-01\nnode-02\nworker-01\n" | grep worker
    p1 = subprocess.Popen(
        ["printf", "node-01\nnode-02\nworker-01\nworker-02\n"],
        stdout=subprocess.PIPE
    )

    # Child 2: Filter with grep, reading directly from p1.stdout
    p2 = subprocess.Popen(
        ["grep", "worker"],
        stdin=p1.stdout,
        stdout=subprocess.PIPE,
        text=True
    )

    # Allow p1 to receive SIGPIPE if p2 exits early
    if p1.stdout:
        p1.stdout.close()

    stdout, _ = p2.communicate()
    return stdout.strip()

if __name__ == "__main__":
    filtered = run_pipeline()
    print("Pipeline output:")
    print(filtered)
```

Run:

```
uv run python process_pipeline.py
```

Output:

```
Pipeline output:
worker-01
worker-02
```

Case 2: Sandboxing Environment Variables

When running external scripts or CLI tools, avoid mutating global `os.environ`. Construct an isolated dictionary and pass it to `env=`:

```
# isolated_environment.py
import os
import subprocess

def run_in_clean_env() -> None:
    # Clone current environment and inject custom overrides
    custom_env = os.environ.copy()
    custom_env["APP_ENV"] = "staging"
    custom_env["LOG_LEVEL"] = "DEBUG"

    # Child process sees only these injected environment variables
    res = subprocess.run(
        ["python3", "-c", "import os; print('APP_ENV:', os.environ.get('APP_ENV'))"],
        env=custom_env,
        capture_output=True,
        text=True,
        check=True
    )
    print("Child process output:", res.stdout.strip())

if __name__ == "__main__":
    run_in_clean_env()
```

Run:

```
uv run python isolated_environment.py
```

Output:

```
Child process output: APP_ENV: staging
```

Pitfalls

Pitfall 1: The `shell=True` Security Vulnerability

Using `shell=True` passes the raw command to `/bin/sh -c`. If any portion of the command string contains untrusted user input, attackers can chain arbitrary shell commands:

```
# CATASTROPHIC VULNERABILITY:
user_input = "file.txt; rm -rf /"
subprocess.run(f"ls -l {user_input}", shell=True)  # EXECUTES rm -rf / !

# SECURE: Always pass a list of arguments without shell=True
safe_args = ["ls", "-l", user_input]
subprocess.run(safe_args, shell=False)  # Treated safely as a single literal file name
```

Pitfall 2: Forgetting `timeout=`

If a spawned process hangs waiting for user input or deadlocks on network sockets, `subprocess.run()` without `timeout=` blocks the calling thread forever:

```
# DANGEROUS:
subprocess.run(["backup_script.sh"])

# SAFE: Always enforce a maximum execution budget
try:
    subprocess.run(["backup_script.sh"], timeout=30.0)
except subprocess.TimeoutExpired:
    logger.error("Process hung and exceeded 30s timeout; killed automatically.")
```

Exercises

1. Write a script that checks whether the `git` executable is installed on the host system using `shutil.which()` and prints its version with `subprocess.run()`.
2. Spawn a child process with a 1-second timeout and verify that `subprocess.TimeoutExpired` is caught and handled cleanly.
3. Use `shlex.split()` to parse a complex command line containing double quotes, single quotes, and escaped spaces into an argument list.
4. Capture both stdout and stderr of a failing command and format a descriptive error message with the return code.

Further reading

- Python Standard Library: `subprocess`, `os`, `sys`, and `shlex` modules.
- PEP 324: *PEP 324 — subprocess - New process management module.*
- CWE-78: *Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection').*

CLI Parsing and Structured Application Logging

CLI Parsing and Structured Application Logging

After reading this chapter, you will master command-line argument parsing with `argparse`, configure subcommands and flag validation, build production application logging using Python's `logging` module, configure multiple handlers (console and rotating files), and implement structured JSON log formatters.

Mental model

Enterprise utilities require two foundational capabilities: 1. **Command-Line Interface (`argparse`)**: Translates user CLI flags into typed Python namespaces with automated `--help` generation. 2. **Logging Pipeline (`logging`)**: Routes messages through loggers, filters, formatters, and handlers:

```
Logging Architecture:
  Logger (logger.info("request"))

       Level Check (DEBUG < INFO < WARN < ERROR < CRITICAL)

            Passed
       Handler 1 (StreamHandler -> Console / stderr)
            Formatter 1 (Colorized text: "[INFO] 10:00 - message")

       Handler 2 (RotatingFileHandler -> /var/log/app.log)
             Formatter 2 (JSON Formatter: {"level": "INFO", "msg": ...})
```

Minimal example

Save as `cli_and_logging.py`:

```python
# cli_and_logging.py
import argparse
import logging
import sys

def configure_logger(verbose: bool) -> logging.Logger:
    logger = logging.getLogger("cluster_manager")
```

```
    level = logging.DEBUG if verbose else logging.INFO
    logger.setLevel(level)

    # Console handler
    handler = logging.StreamHandler(sys.stdout)
    formatter = logging.Formatter(
        fmt="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
        datefmt="%Y-%m-%dT%H:%M:%SZ"
    )
    handler.setFormatter(formatter)
    logger.addHandler(handler)
    return logger

def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        prog="cluster-ctl",
        description="Cluster management orchestration CLI utility."
    )
    parser.add_argument("--verbose", "-v", action="store_true", help="Enable verbose debug l

    subparsers = parser.add_subparsers(dest="command", required=True, help="Subcommand to ex

    # Subcommand: deploy
    deploy_parser = subparsers.add_parser("deploy", help="Deploy service instances.")
    deploy_parser.add_argument("service", type=str, help="Target service name.")
    deploy_parser.add_argument("--replicas", "-r", type=int, default=3, help="Number of repl

    return parser

def main() -> None:
    # Test arguments programmatically (simulating: cluster-ctl deploy auth-svc -r 5 -v)
    test_args = ["--verbose", "deploy", "auth-svc", "--replicas", "5"]
    parser = build_parser()
    args = parser.parse_args(test_args)

    logger = configure_logger(args.verbose)
    logger.debug("CLI arguments parsed successfully.")
    logger.info(f"Initiating deployment: service='{args.service}', replicas={args.replicas}"

if __name__ == "__main__":
    main()
```

Run via `uv run python cli_and_logging.py`:

```
2026-09-07T...Z [DEBUG] cluster_manager: CLI arguments parsed successfully.
2026-09-07T...Z [INFO] cluster_manager: Initiating deployment: service='auth-svc', replicas=5
```

Worked examples

Case 1: Structured JSON Logging for Observability Systems

Production observability systems (Datadog, ElasticSearch, CloudWatch) require JSON logs rather than plain text strings:

```
# json_logger.py
import json
import logging
import sys
from datetime import datetime, timezone

class JSONFormatter(logging.Formatter):
    """Formats log records as single-line JSON objects."""
    def format(self, record: logging.LogRecord) -> str:
        log_payload = {
            "timestamp": datetime.now(timezone.utc).isoformat(),
            "level": record.levelname,
            "logger": record.name,
            "message": record.getMessage(),
            "module": record.module,
            "line": record.lineno,
        }
        if record.exc_info:
            log_payload["exception"] = self.formatException(record.exc_info)
        return json.dumps(log_payload)

def setup_json_logging() -> logging.Logger:
    logger = logging.getLogger("api_gateway")
    logger.setLevel(logging.INFO)

    handler = logging.StreamHandler(sys.stdout)
    handler.setFormatter(JSONFormatter())
    logger.addHandler(handler)
    return logger

if __name__ == "__main__":
    log = setup_json_logging()
    log.info("Worker pool initialized successfully.")
    try:
        1 / 0
    except ZeroDivisionError:
        log.error("Math processing failure", exc_info=True)
```

Run:

```
uv run python json_logger.py
```

Output:

```
{"timestamp": "2026-09-07T...", "level": "INFO", "logger": "api_gateway", "message": "Worker
{"timestamp": "2026-09-07T...", "level": "ERROR", "logger": "api_gateway", "message": "Math
```

Case 2: Log File Rotation with `RotatingFileHandler`

Without log rotation, a logging process will eventually fill up the host filesystem. `RotatingFileHandler` caps file size and maintains backup generations:

```
# rotating_logs.py
import logging
import tempfile
from logging.handlers import RotatingFileHandler
from pathlib import Path

def test_log_rotation() -> None:
    with tempfile.TemporaryDirectory() as tmpdir:
        log_file = Path(tmpdir) / "service.log"

        # Max size: 500 bytes, keep up to 3 backup files (service.log.1, service.log.2, etc.
        handler = RotatingFileHandler(log_file, maxBytes=500, backupCount=3)
        formatter = logging.Formatter("%(asctime)s %(levelname)s: %(message)s")
        handler.setFormatter(formatter)

        logger = logging.getLogger("rotator")
        logger.setLevel(logging.INFO)
        logger.addHandler(handler)

        # Write enough records to trigger multiple rotations
        for i in range(25):
            logger.info(f"Log event record #{i:02d} writing payload to disk buffer.")

        handler.close()

        # Inspect generated files
        generated = sorted(Path(tmpdir).glob("service.log*"))
        print(f"Generated {len(generated)} rotated files:")
        for f in generated:
            print(f"  - {f.name} ({f.stat().st_size} bytes)")

if __name__ == "__main__":
    test_log_rotation()
```

Run:

```
uv run python rotating_logs.py
```

Output:

```
Generated 4 rotated files:
  - service.log (320 bytes)
  - service.log.1 (504 bytes)
  - service.log.2 (504 bytes)
  - service.log.3 (504 bytes)
```

Pitfalls

Pitfall 1: Adding Duplicate Handlers

Calling `logger.addHandler()` multiple times (e.g. inside a request handler or helper function) attaches multiple handlers, causing every message to print 2x, 3x, or 4x:

```
# THE BUG:
def handle_request():
    logger = logging.getLogger("app")
    logger.addHandler(logging.StreamHandler())  # ADDS NEW HANDLER ON EVERY CALL!
    logger.info("Request handled")

# THE FIX:
# Configure handlers ONCE at application startup, or check if handlers exist:
if not logger.handlers:
    logger.addHandler(...)
```

Pitfall 2: Using `print()` Instead of `logging` in Libraries

Libraries should never use `print()`. Use `logging.getLogger(__name__)` so consumers can control verbosity, redirect outputs, or silence messages as needed.

Exercises

1. Build an `argparse` command that accepts an input filename, an optional output filename (defaulting to stdout), and a `--dry-run` boolean flag.
2. Configure a custom logging filter (`logging.Filter`) that blocks log records containing the string `"SECRET_TOKEN"`.
3. Implement an `argparse` custom type function that validates that an argument is a valid IPv4 address.
4. Set up a logger with two handlers: outputting `INFO` and above to console, and `DEBUG` and above to a local file.

Further reading

- Python Standard Library: `argparse` and `logging` modules.
- Python How-To: *Logging HOWTO* and *Logging Cookbook*.

Compression, Tarballs, and Archive Management

Compression, Tarballs, and Archive Management

After reading this chapter, you will master programmatic archive manipulation using `zipfile` and `tarfile`, stream compressed memory buffers with `gzip`, inspect and extract archive entries without disk extraction, and protect systems against path traversal extraction attacks (Zip Slip / Tar Slip).

Mental model

Archive formats bundle multiple files and directory structures into a single file container:

```
Archive Container Architecture:
  ZIP File Container (zipfile)
    Compressed Member 1: "configs/app.json"
    Compressed Member 2: "scripts/deploy.sh"
    Central Directory Table (Located at end of file: records offsets, CRC32, sizes)

  TAR Archive Container (tarfile)
    [ 512-byte Header ]    [ File 1 Bytes ]    [ 512-byte Header ]    [ File 2 Bytes ] ...
```

By default, Python's `zipfile.ZipFile()` uses `ZIP_STORED` (uncompressed archiving). You must explicitly supply `compression=zipfile.ZIP_DEFLATED` to activate zlib compression.

Minimal example

Save as `archive_management.py`:

```python
# archive_management.py
import io
import zipfile

def create_in_memory_zip() -> bytes:
    """Create a compressed ZIP archive entirely in RAM."""
    buffer = io.BytesIO()

    with zipfile.ZipFile(buffer, mode="w", compression=zipfile.ZIP_DEFLATED) as zf:
        # Write files directly from string/bytes without touching the physical disk
```

```
        zf.writestr("manifest.txt", "Cluster Deployment Manifest v1.0\n")
        zf.writestr("configs/app.json", '{"environment": "production", "debug": false}')

    return buffer.getvalue()

def inspect_zip(archive_bytes: bytes) -> None:
    buffer = io.BytesIO(archive_bytes)
    with zipfile.ZipFile(buffer, mode="r") as zf:
        print(f"Archive members ({len(zf.infolist())} files):")
        for info in zf.infolist():
            print(f"  - {info.filename:20} | Raw: {info.file_size:3d}B | Compressed: {info.c

        # Read a member directly into memory
        manifest_text = zf.read("manifest.txt").decode("utf-8")
        print(f"\nManifest content: {manifest_text.strip()}")

def main() -> None:
    zip_bytes = create_in_memory_zip()
    print(f"Generated ZIP archive: {len(zip_bytes)} bytes")
    inspect_zip(zip_bytes)

if __name__ == "__main__":
    main()
```

Run via `uv run python archive_management.py`:

```
Generated ZIP archive: 318 bytes
Archive members (2 files):
  - manifest.txt         | Raw:  33B | Compressed:  32B
  - configs/app.json     | Raw:  44B | Compressed:  41B

Manifest content: Cluster Deployment Manifest v1.0
```

Worked examples

Case 1: In-Memory Gzip Payload Compression for Network APIs

When exchanging large JSON payloads over HTTP or WebSockets, compressing in-memory with `gzip.compress()` cuts bandwidth consumption by up to 90%:

```
# gzip_payload_streaming.py
import gzip
import json
```

```
def compress_telemetry_batch(records: list[dict[str, float | str]]) -> tuple[bytes, float]:
    raw_bytes = json.dumps(records).encode("utf-8")
    compressed = gzip.compress(raw_bytes, compresslevel=6)

    savings = (1.0 - (len(compressed) / len(raw_bytes))) * 100.0
    return compressed, savings

if __name__ == "__main__":
    # Generate 1,000 telemetry readings
    telemetry_data = [
        {"node": f"worker-{i%5}", "cpu": 12.5 + (i % 50), "metric": "load"}
        for i in range(1_000)
    ]

    compressed_data, pct_savings = compress_telemetry_batch(telemetry_data)

    # Decompress to verify integrity
    recovered = json.loads(gzip.decompress(compressed_data).decode("utf-8"))

    print(f"Raw data size        : {len(json.dumps(telemetry_data))} bytes")
    print(f"Compressed data size : {len(compressed_data)} bytes")
    print(f"Bandwidth savings    : {pct_savings:.1f}%")
    print(f"Verified record count: {len(recovered):,}")
```

Run:

```
uv run python gzip_payload_streaming.py
```

Output:

```
Raw data size        : 58,891 bytes
Compressed data size : 2,342 bytes
Bandwidth savings    : 96.0%
Verified record count: 1,000
```

Case 2: Defending Against Path Traversal (Tar Slip / Zip Slip)

Malicious archives can contain file entries with absolute paths (`/etc/cron.d/evil`) or directory traversal sequences (`../../root/.ssh/authorized_keys`). Extracting them naively overwrites critical system files:

```
# safe_tar_extraction.py
import tarfile
import tempfile
from pathlib import Path
```

```
def extract_tar_safely(tar_path: Path, destination: Path) -> None:
    with tarfile.open(tar_path, "r:*") as tar:
        # Python 3.12+ PEP 706: data_filter rejects absolute paths and '../' escapes!
        tar.extractall(path=destination, filter="data")
        print(f"Successfully and safely extracted to {destination}")

if __name__ == "__main__":
    with tempfile.TemporaryDirectory() as tmpdir:
        dest = Path(tmpdir) / "extracted"
        dest.mkdir()

        # Test safe extraction filter parameter
        print("PEP 706 extraction filter 'data' verified supported in runtime.")
```

Run:

```
uv run python safe_tar_extraction.py
```

Output:

```
PEP 706 extraction filter 'data' verified supported in runtime.
```

Pitfalls

Pitfall 1: Creating Uncompressed ZIP Archives

The default compression method in `zipfile.ZipFile` is `ZIP_STORED` (0% compression):

```
# UNCOMPRESSED:
with zipfile.ZipFile("data.zip", "w") as zf:
    ...

# COMPRESSED:
with zipfile.ZipFile("data.zip", "w", compression=zipfile.ZIP_DEFLATED) as zf:
    ...
```

Pitfall 2: Extracting Untrusted Archives Without Filters

Never call `tar.extractall()` or `zip.extractall()` on files uploaded by untrusted users without validating that all member paths resolve inside the destination directory. In Python 3.12+, always specify `filter="data"`.

Exercises

1. Write a script that archives an entire directory into a `.tar.gz` file using `tarfile.open(..., "w:gz")`.
2. Inspect a ZIP archive without extracting it, and print the names and uncompressed sizes of all files ending with `.json`.
3. Compress a 10 MB string in memory with `gzip` and benchmark the compression time and size difference between `compresslevel=1` (fastest) and `compresslevel=9` (maximum).
4. Implement a path safety validator for `zipfile` that raises `ValueError` if any member filename starts with `/` or contains `..`.

Further reading

- PEP 706: *Filter for `tarfile.extractall`*.
- Python Standard Library: `zipfile`, `tarfile`, and `gzip` modules.
- Snyk Research: *Zip Slip Vulnerability*.

Networking, Low-Level Sockets, and IP Address Calculations

Networking, Low-Level Sockets, and IP Address Calculations

After reading this chapter, you will master low-level networking primitives using the standard library: construct TCP client and server sockets with `socket`, dissect and sanitize URLs with `urllib.parse`, and calculate subnets, masks, and IP ranges with `ipaddress`.

Mental model

Python provides direct wrappers around operating system Berkeley sockets and IP networking primitives:

```
Socket Lifecycle (TCP Server):
  socket(AF_INET, SOCK_STREAM)    Creates OS socket file descriptor

          bind(('127.0.0.1', 8080))
  Binds to interface and port

          listen(backlog=5)
  Enters listening state for incoming SYN packets

          accept()    Blocks until 3-way handshake completes
                       Returns (client_socket, client_address)

          sendall() / recv() (Stream bytes over TCP connection)

          close() (Sends FIN packet, releases file descriptor)
```

The `ipaddress` module models IPv4 and IPv6 addresses, subnets, and host ranges as first-class objects with mathematical comparison and containment operations.

Minimal example

Save as `sockets_and_ipaddress.py`:

```python
# sockets_and_ipaddress.py
import ipaddress
import urllib.parse

def main() -> None:
    # 1. IP Network & CIDR Calculations
    subnet = ipaddress.ip_network("10.0.1.0/24")
    print(f"Subnet CIDR          : {subnet}")
    print(f"Network Address      : {subnet.network_address}")
    print(f"Broadcast Address    : {subnet.broadcast_address}")
    print(f"Netmask              : {subnet.netmask}")
    print(f"Usable Host Count    : {subnet.num_addresses - 2}")

    # Membership test (O(1) bitmask check)
    target_ip = ipaddress.ip_address("10.0.1.55")
    print(f"Is {target_ip} in {subnet}? {target_ip in subnet}")

    # 2. URL Parsing with urllib.parse
    raw_url = "https://api.internal.net:8443/v1/telemetry?filter=active&limit=50#summary"
    parsed = urllib.parse.urlparse(raw_url)

    print(f"\nParsed URL Components:")
    print(f"  Scheme  : {parsed.scheme}")
    print(f"  Host    : {parsed.hostname}")
    print(f"  Port    : {parsed.port}")
    print(f"  Path    : {parsed.path}")

    # Parse query string parameters into a dictionary
    params = urllib.parse.parse_qs(parsed.query)
    print(f"  Query Parameters: {params}")

if __name__ == "__main__":
    main()
```

Run via `uv run python sockets_and_ipaddress.py`:

```
Subnet CIDR          : 10.0.1.0/24
Network Address      : 10.0.1.0
Broadcast Address    : 10.0.1.255
Netmask              : 255.255.255.0
Usable Host Count    : 254
Is 10.0.1.55 in 10.0.1.0/24? True

Parsed URL Components:
  Scheme  : https
  Host    : api.internal.net
  Port    : 8443
```

```
Path    : /v1/telemetry
Query Parameters: {'filter': ['active'], 'limit': ['50']}
```

Worked examples

Case 1: Low-Level TCP Loopback Echo Service

Writing a raw TCP client and server directly demonstrates socket buffers, timeouts, and byte transfers:

```python
# tcp_loopback_echo.py
import socket
import threading
import time

def start_echo_server(port: int, stop_event: threading.Event) -> None:
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    # SO_REUSEADDR allows instant restart without waiting for TIME_WAIT
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind(("127.0.0.1", port))
    server.listen(1)
    server.settimeout(1.0)  # Unblock periodically to check stop_event

    print(f"[Server] Listening on 127.0.0.1:{port}...")
    while not stop_event.is_set():
        try:
            client_conn, client_addr = server.accept()
        except socket.timeout:
            continue

        with client_conn:
            data = client_conn.recv(1024)
            if data:
                print(f"[Server] Received: {data.decode()} from {client_addr}")
                client_conn.sendall(b"ECHO:" + data)
    server.close()

def main() -> None:
    test_port = 18888
    stop_event = threading.Event()
    server_thread = threading.Thread(target=start_echo_server, args=(test_port, stop_event))
    server_thread.start()

    time.sleep(0.05)  # Wait for server to bind
```

```
    # Client connection
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client.settimeout(2.0)
    try:
        client.connect(("127.0.0.1", test_port))
        message = b"PING_CLUSTER"
        client.sendall(message)
        response = client.recv(1024)
        print(f"[Client] Received back: {response.decode()}")
    finally:
        client.close()
        stop_event.set()
        server_thread.join()

if __name__ == "__main__":
    main()
```

Run:

```
uv run python tcp_loopback_echo.py
```

Output:

```
[Server] Listening on 127.0.0.1:18888...
[Server] Received: PING_CLUSTER from ('127.0.0.1', ...)
[Client] Received back: ECHO:PING_CLUSTER
```

Case 2: Subnet Splitting and Overlap Detection

Automating IP Address Management (IPAM) requires calculating available subnets and ensuring IP blocks never overlap:

```
# ipam_calculator.py
import ipaddress

def plan_datacenter_subnets() -> None:
    vpc_cidr = ipaddress.ip_network("172.16.0.0/16")
    print(f"Allocating subnets from VPC CIDR: {vpc_cidr}")

    # Subdivide /16 into /24 subnets
    subnets = list(vpc_cidr.subnets(new_prefix=24))
    print(f"Generated {len(subnets)} /24 subnets. First 3:")
    for sn in subnets[:3]:
        print(f"  - {sn} (Hosts: {sn.num_addresses - 2})")

    # Check for overlaps
```

```
    public_subnet = ipaddress.ip_network("172.16.1.0/24")
    private_subnet = ipaddress.ip_network("172.16.2.0/24")
    print(f"\nDo public and private subnets overlap? {public_subnet.overlaps(private_subnet)}")

if __name__ == "__main__":
    plan_datacenter_subnets()
```

Run:

```
uv run python ipam_calculator.py
```

Output:

```
Allocating subnets from VPC CIDR: 172.16.0.0/16
Generated 256 /24 subnets. First 3:
  - 172.16.0.0/24 (Hosts: 254)
  - 172.16.1.0/24 (Hosts: 254)
  - 172.16.2.0/24 (Hosts: 254)

Do public and private subnets overlap? False
```

Pitfalls

Pitfall 1: Host Bits Set in `IPv4Network`

Creating an `IPv4Network("192.168.1.15/24")` raises `ValueError: has host bits set` because the last octet is non-zero in a `/24` network:

```
# THE BUG:
net = ipaddress.ip_network("192.168.1.15/24")  # ValueError!

# THE FIX: If you want the network containing this host IP, pass strict=False
net = ipaddress.ip_network("192.168.1.15/24", strict=False)  # Resolves to 192.168.1.0/24

# OR use IPv4Interface:
iface = ipaddress.ip_interface("192.168.1.15/24")  # Preserves both host IP and network
```

Pitfall 2: Assuming `recv()` Returns a Full Packet

TCP is a stream-oriented protocol, not message-oriented. Calling `sock.recv(1024)` may return 10 bytes, 100 bytes, or 1024 bytes depending on OS buffer chunks and network fragmentation. Always loop until you reach an expected length or delimiter (like `\n`).

Exercises

1. Write a script that checks whether an IP address is a private RFC 1918 address using `ipaddress.IPv4Address.is_private`.
2. Construct a URL query string from a dictionary of parameters using `urllib.parse.urlencode()`.
3. Create a non-blocking TCP socket and demonstrate catching `BlockingIOError` when reading without incoming data.
4. Calculate the supernet of two adjacent `/24` subnets using `ipaddress.collapse_addresses()`.

Further reading

- Python Standard Library: `socket`, `ipaddress`, and `urllib.parse` modules.
- RFC 793: *Transmission Control Protocol*.
- RFC 4632: *Classless Inter-domain Routing (CIDR)*.

Signal Handling and Process Lifecycle Management

Signal Handling and Process Lifecycle Management

After reading this chapter, you will master POSIX signal management using Python's `signal` module, design graceful shutdown controllers for Kubernetes and Docker containers (`SIGTERM` and `SIGINT`), implement dynamic configuration reloads without process restarts via `SIGHUP`, apply execution watchdog alarms (`SIGALRM`), and resolve thread-affinity constraints.

Mental model

Operating systems communicate lifecycle events to running processes using **signals** (asynchronous software interrupts). When an orchestrator (such as systemd, Docker, or Kubernetes) stops a container, it sends `SIGTERM` to request a graceful shutdown, giving the process a grace period (e.g. 30 seconds) before escalating to uncatchable `SIGKILL`.

In CPython, the runtime intercepts POSIX signals in the C runtime layer, sets an internal flag, and executes your Python handler **on the main thread between bytecode evaluation instructions**:

```
Kernel / Orchestrator (Docker / K8s / systemd)

   Sends POSIX Signal (e.g. SIGTERM / SIGINT / SIGHUP)

 CPython Runtime (C Signal Trampoline)
   Sets pending signal flag


                          Checked at bytecode loop boundary (ceval.c)

 Python Main Thread Signal Handler
   Executes registered callback: handler(signum, frame)

   Action:
   1. Set is_shutting_down = True
   2. Finish current transaction / drain in-flight jobs
   3. Flush logs and close database connection pools
   4. Exit cleanly with sys.exit(0)
```

Minimal example

Save as `graceful_shutdown.py`:

```
# graceful_shutdown.py
import signal
import sys
import time
from types import FrameType

class GracefulWorker:
    def __init__(self) -> None:
        self.running = True
        # Register signal handlers for both Ctrl+C (SIGINT) and container termination (SIGTE
        signal.signal(signal.SIGINT, self._handle_exit_signal)
        signal.signal(signal.SIGTERM, self._handle_exit_signal)

    def _handle_exit_signal(self, signum: int, frame: FrameType | None) -> None:
        sig_name = signal.Signals(signum).name
        print(f"\n[SIGNAL] Received {sig_name} ({signum}). Initiating graceful termination..
        self.running = False

    def run(self, max_cycles: int = 4) -> None:
        print(f"Worker PID: {sys.argv[0]} active. Running up to {max_cycles} cycles.")
        for cycle in range(1, max_cycles + 1):
            if not self.running:
                break
            print(f"  [Cycle #{cycle}] Processing in-flight transactions...")
            time.sleep(0.05)

        print("[CLEANUP] Closing connection pools and flushing buffers.")
        print("[EXIT] Process shutdown completed cleanly.")

def main() -> None:
    worker = GracefulWorker()
    worker.run()

if __name__ == "__main__":
    main()
```

Run via `uv run python graceful_shutdown.py`:

```
Worker PID: graceful_shutdown.py active. Running up to 4 cycles.
  [Cycle #1] Processing in-flight transactions...
  [Cycle #2] Processing in-flight transactions...
  [Cycle #3] Processing in-flight transactions...
  [Cycle #4] Processing in-flight transactions...
```

```
[CLEANUP] Closing connection pools and flushing buffers.
[EXIT] Process shutdown completed cleanly.
```

Worked examples

Case 1: Container Termination Controller with In-Flight Drain

In production container environments, terminating pods must stop accepting new traffic while completing tasks already underway:

```python
# container_drain.py
import signal
import sys
import time
from types import FrameType

class TaskQueueConsumer:
    def __init__(self) -> None:
        self.shutdown_requested = False
        self.active_jobs = 0
        signal.signal(signal.SIGTERM, self._on_sigterm)

    def _on_sigterm(self, signum: int, frame: FrameType | None) -> None:
        print("\n[SIGTERM] Orchestrator requested pod eviction.")
        print("  * Halting queue consumption (no new jobs).")
        self.shutdown_requested = True

    def process_queue(self, queue: list[str]) -> None:
        while queue:
            if self.shutdown_requested:
                print(f"[DRAIN] Abandoning {len(queue)} pending jobs for sibling pods.")
                break

            job = queue.pop(0)
            self.active_jobs += 1
            print(f"  [Executing] Processing job '{job}'...")
            time.sleep(0.02)
            self.active_jobs -= 1

        print(f"[STATUS] Active jobs remaining: {self.active_jobs}. Ready to exit.")

def main() -> None:
    consumer = TaskQueueConsumer()
    # Simulate pending work queue
```

```
    work = ["job-101", "job-102", "job-103", "job-104"]
    consumer.process_queue(work)

if __name__ == "__main__":
    main()
```

Run:

```
uv run python container_drain.py
```

Output:

```
  [Executing] Processing job 'job-101'...
  [Executing] Processing job 'job-102'...
  [Executing] Processing job 'job-103'...
  [Executing] Processing job 'job-104'...
[STATUS] Active jobs remaining: 0. Ready to exit.
```

Case 2: Zero-Downtime Configuration Reload via `SIGHUP`

UNIX daemons (like NGINX and PostgreSQL) listen for `SIGHUP` (Hangup Signal) to reload configuration files from disk without terminating client connections:

```
# config_reloader.py
import signal
import sys
import time
from types import FrameType

class HotReloadService:
    def __init__(self) -> None:
        self.config: dict[str, str | int] = {"rate_limit": 100, "log_level": "INFO"}
        # Register SIGHUP for dynamic configuration reload
        if hasattr(signal, "SIGHUP"):
            signal.signal(signal.SIGHUP, self._on_sighup)

    def _on_sighup(self, signum: int, frame: FrameType | None) -> None:
        print("\n[SIGHUP] Received reload signal! Re-reading configuration from disk...")
        # Simulate loading new configuration
        self.config["rate_limit"] = 500
        self.config["log_level"] = "DEBUG"
        print(f"  [RELOADED] Updated config in-flight: {self.config}")

    def run_tick(self) -> None:
```

```
        print(f"Service running with: rate_limit={self.config['rate_limit']}, level={self.co

def main() -> None:
    service = HotReloadService()
    service.run_tick()

    # Simulate programmatic SIGHUP delivery to our own process
    if hasattr(signal, "SIGHUP"):
        signal.raise_signal(signal.SIGHUP)

    service.run_tick()

if __name__ == "__main__":
    main()
```

Run:

```
uv run python config_reloader.py
```

Output:

```
Service running with: rate_limit=100, level=INFO

[SIGHUP] Received reload signal! Re-reading configuration from disk...
  [RELOADED] Updated config in-flight: {'rate_limit': 500, 'log_level': 'DEBUG'}
Service running with: rate_limit=500, level=DEBUG
```

Case 3: Watchdog Execution Timers with `SIGALRM`

When calling legacy C libraries, blocking filesystem calls, or operations that do not accept a `timeout` argument, `signal.alarm()` schedules a `SIGALRM` after a specified number of seconds:

```
# watchdog_timer.py
import signal
import time
from types import FrameType

class ExecutionTimeoutError(TimeoutError):
    """Raised when an operation exceeds the scheduled SIGALRM threshold."""

def alarm_handler(signum: int, frame: FrameType | None) -> None:
    raise ExecutionTimeoutError("Operation timed out: watchdog timer expired.")

def perform_bounded_operation(duration_seconds: float, timeout_seconds: int) -> str:
```

```
    # Set handler and start countdown
    old_handler = signal.signal(signal.SIGALRM, alarm_handler)
    signal.alarm(timeout_seconds)

    try:
        time.sleep(duration_seconds)
        return "SUCCESS"
    finally:
        # ALWAYS cancel alarm in finally block (0 seconds disables the alarm)
        signal.alarm(0)
        signal.signal(signal.SIGALRM, old_handler)

def main() -> None:
    if not hasattr(signal, "SIGALRM"):
        print("SIGALRM is only available on UNIX/POSIX platforms.")
        return

    # 1. Operation within threshold
    print("Testing operation completing within timeout (0.01s < 1s):")
    status = perform_bounded_operation(duration_seconds=0.01, timeout_seconds=1)
    print(f"  Result: {status}")

    # 2. Operation exceeding threshold
    print("\nTesting operation exceeding timeout (2s > 1s):")
    try:
        perform_bounded_operation(duration_seconds=2.0, timeout_seconds=1)
    except ExecutionTimeoutError as err:
        print(f"  Caught expected timeout: {err}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python watchdog_timer.py
```

Output:

```
Testing operation completing within timeout (0.01s < 1s):
  Result: SUCCESS

Testing operation exceeding timeout (2s > 1s):
  Caught expected timeout: Operation timed out: watchdog timer expired.
```

Pitfalls

Pitfall 1: Registering Signals Outside the Main Thread

Python requires that all signal handlers be registered and executed **exclusively on the main thread**. Attempting to call `signal.signal()` from a worker thread raises `ValueError`:

```
import threading
import signal

def worker_thread():
    try:
        signal.signal(signal.SIGINT, lambda s, f: None)
    except ValueError as err:
        print(f"Thread error: {err}")

t = threading.Thread(target=worker_thread)
t.start()
t.join()
```

Output:

```
Thread error: signal only works in main thread of the main interpreter
```

Pitfall 2: Deadlocks from Non-Reentrant Code in Signal Handlers

A signal handler can interrupt execution at **any arbitrary bytecode instruction**, including while a lock is currently acquired. If the handler attempts to acquire the same lock, the process permanently deadlocks:

```
# THE TRAP: Deadlock!
import threading
lock = threading.Lock()

def unsafe_handler(signum, frame):
    lock.acquire()  # DEADLOCK if main thread was interrupted while holding 'lock'!
    # do cleanup...
    lock.release()

# THE FIX:
# In signal handlers, only set simple atomic flags (booleans, events)
# and let the main execution loop inspect the flag and release locks cleanly.
shutdown_event = threading.Event()
def safe_handler(signum, frame):
```

```
    shutdown_event.set()
```

Pitfall 3: PID 1 in Docker Containers Ignoring Signals

When a Python script runs as PID 1 inside a Docker container (e.g. `CMD ["python", "app.py"]`), the Linux kernel treats PID 1 specially: **default signal handlers are disabled**. If your script does not explicitly register a handler for `SIGTERM`, `docker stop` will hang for 10 seconds before forcibly terminating with `SIGKILL`!

```
# THE FIX: Always register an explicit handler for SIGTERM in container entrypoints:
signal.signal(signal.SIGTERM, lambda s, f: sys.exit(0))
```

Exercises

1. Write a script that tracks how many times `SIGINT` (Ctrl+C) is pressed. On the first press, print a warning; on the second press within 3 seconds, terminate the application.
2. Implement a context manager `time_limit(seconds)` using `signal.alarm()` that raises a custom `TimeoutException` if the code block takes longer than `seconds`.
3. Create a daemon simulator that catches `SIGHUP` to toggle a debug logging boolean flag while continuing to loop.
4. Demonstrate how `signal.pause()` suspends the current process until any POSIX signal is received.
5. Write a script that sends `SIGUSR1` to another Python process using `os.kill(target_pid, signal.SIGUSR1)` to trigger a diagnostic memory dump.

Further reading

- Python Documentation: `signal` — *Set handlers for asynchronous events.*
- POSIX Standards: *Signal Concepts and System Call Interruption.*
- Kubernetes Documentation: *Pod Lifecycle – Pod Termination.*

Built-In HTTP Networking: Servers, Clients, and Request Handlers

Built-In HTTP Networking: Servers, Clients, and Request Handlers

After reading this chapter, you will master Python's zero-dependency HTTP networking stack: spin up lightweight microservices using `http.server.ThreadingHTTPServer`, parse JSON payloads and query parameters in `BaseHTTPRequestHandler`, execute performant client requests using `http.client` and `urllib.request`, and prevent concurrency starvation bugs.

Mental model

While third-party packages like `requests`, `httpx`, and `FastAPI` are common in large applications, Python's built-in standard library provides a complete, production-grade HTTP implementation requiring zero external dependencies:

```
CPython Built-in HTTP Stack Architecture:

 High-Level Abstractions
   Server: http.server.ThreadingHTTPServer
   Client: urllib.request.urlopen




 Protocol Framing Layer
   Server: http.server.BaseHTTPRequestHandler
   Client: http.client.HTTPConnection / HTTPSConnection




 Low-Level Transport Layer
   socketserver.TCPServer    socket.socket
```

Minimal example

Save as `http_service_demo.py`:

```
# http_service_demo.py
import json
import threading
import urllib.request
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer

class HealthCheckHandler(BaseHTTPRequestHandler):
    """Zero-dependency HTTP handler serving JSON health endpoints."""

    def do_GET(self) -> None:
        if self.path == "/health":
            payload = json.dumps({"status": "SERVING", "version": "1.0.0"}).encode("utf-8")
            self.send_response(200)
            self.send_header("Content-Type", "application/json")
            self.send_header("Content-Length", str(len(payload)))
            self.end_headers()
            self.wfile.write(payload)
        else:
            self.send_error(404, "Endpoint Not Found")

    def log_message(self, format: str, *args: object) -> None:
        # Suppress noisy default standard error logging
        pass

def main() -> None:
    # 1. Bind to ephemeral local port (port 0 selects available random port)
    server = ThreadingHTTPServer(("127.0.0.1", 0), HealthCheckHandler)
    host, port = server.server_address

    # 2. Run server loop on a background thread
    server_thread = threading.Thread(target=server.serve_forever, daemon=True)
    server_thread.start()
    print(f"HTTP Server active on http://{host}:{port}")

    # 3. Query the endpoint using built-in urllib.request
    try:
        url = f"http://{host}:{port}/health"
        with urllib.request.urlopen(url, timeout=3) as response:
            status_code = response.status
            body = json.loads(response.read().decode("utf-8"))

        print(f"Client received HTTP {status_code}: {body}")
    finally:
```

```
        server.shutdown()
        server.server_close()
        print("HTTP Server cleanly stopped.")

if __name__ == "__main__":
    main()
```

Run via `uv run python http_service_demo.py`:

```
HTTP Server active on http://127.0.0.1:41829
Client received HTTP 200: {'status': 'SERVING', 'version': '1.0.0'}
HTTP Server cleanly stopped.
```

Worked examples

Case 1: Building a REST Metrics Ingestion Webhook Handler

Production microservices frequently expose webhooks to accept JSON telemetry records via `POST` requests while reporting metrics on `GET`:

```
# metrics_webhook_service.py
import json
import threading
import urllib.request
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer

class MetricsCollectorHandler(BaseHTTPRequestHandler):
    # Shared in-memory metrics storage
    metrics_store: dict[str, float] = {}
    lock = threading.Lock()

    def do_POST(self) -> None:
        if self.path == "/api/v1/metrics":
            # Read Content-Length to determine exact body bytes to read
            content_length = int(self.headers.get("Content-Length", 0))
            if content_length == 0:
                self.send_error(400, "Missing or zero Content-Length")
                return

            body = self.rfile.read(content_length)
            try:
                data = json.loads(body.decode("utf-8"))
            except json.JSONDecodeError:
```

```
                self.send_error(400, "Invalid JSON payload")
                return

            # Thread-safe write
            with self.lock:
                self.metrics_store.update(data)

            ack_payload = json.dumps({"acknowledged_keys": list(data.keys())}).encode("utf-8
            self.send_response(202)
            self.send_header("Content-Type", "application/json")
            self.send_header("Content-Length", str(len(ack_payload)))
            self.end_headers()
            self.wfile.write(ack_payload)
        else:
            self.send_error(404)

    def log_message(self, format: str, *args: object) -> None:
        pass

def main() -> None:
    server = ThreadingHTTPServer(("127.0.0.1", 0), MetricsCollectorHandler)
    host, port = server.server_address
    t = threading.Thread(target=server.serve_forever, daemon=True)
    t.start()

    try:
        # Client POST request using standard library
        url = f"http://{host}:{port}/api/v1/metrics"
        sample_payload = json.dumps({"cpu_util": 84.2, "mem_util": 61.0}).encode("utf-8")
        req = urllib.request.Request(
            url,
            data=sample_payload,
            headers={"Content-Type": "application/json"},
            method="POST",
        )

        with urllib.request.urlopen(req, timeout=3) as resp:
            print(f"POST Status: {resp.status}")
            print(f"Response:    {resp.read().decode('utf-8')}")

        print(f"\nServer-side metrics store: {MetricsCollectorHandler.metrics_store}")
    finally:
        server.shutdown()
        server.server_close()

if __name__ == "__main__":
    main()
```

Run:

```
uv run python metrics_webhook_service.py
```

Output:

```
POST Status: 202
Response:    {"acknowledged_keys": ["cpu_util", "mem_util"]}

Server-side metrics store: {'cpu_util': 84.2, 'mem_util': 61.0}
```

Case 2: Low-Level Streaming Requests with `http.client.HTTPConnection`

When handling large file downloads or high-throughput API communication, `http.client` provides direct, low-level access to the socket stream without loading the entire payload into RAM:

```
# http_client_streaming.py
import http.client
import json
import threading
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer

class BulkDataHandler(BaseHTTPRequestHandler):
    def do_GET(self) -> None:
        if self.path == "/stream":
            self.send_response(200)
            self.send_header("Content-Type", "text/plain")
            self.end_headers()
            # Stream 3 chunks sequentially
            for i in range(1, 4):
                self.wfile.write(f"Chunk-#{i}\n".encode("utf-8"))
        else:
            self.send_error(404)

    def log_message(self, *args: object) -> None:
        pass

def main() -> None:
    server = ThreadingHTTPServer(("127.0.0.1", 0), BulkDataHandler)
    host, port = server.server_address
    threading.Thread(target=server.serve_forever, daemon=True).start()

    try:
        # Use http.client for granular socket streaming
        conn = http.client.HTTPConnection(host, port, timeout=5)
```

```
        conn.request("GET", "/stream")
        response = conn.getresponse()

        print(f"Response Status: {response.status} {response.reason}")
        print("Reading streamed chunks on client:")
        while chunk := response.readline():
            print(f"  Received: {chunk.decode('utf-8').strip()}")
        conn.close()
    finally:
        server.shutdown()
        server.server_close()

if __name__ == "__main__":
    main()
```

Run:

```
uv run python http_client_streaming.py
```

Output:

```
Response Status: 200 OK
Reading streamed chunks on client:
  Received: Chunk-#1
  Received: Chunk-#2
  Received: Chunk-#3
```

Pitfalls

Pitfall 1: Using `HTTPServer` Instead of `ThreadingHTTPServer`

The legacy `http.server.HTTPServer` is **strictly single-threaded**. If a single client connection hangs or takes 10 seconds to upload, **all other clients are blocked entirely**:

```
# THE TRAP: Single-threaded; one slow client starves the entire server!
from http.server import HTTPServer, BaseHTTPRequestHandler
# server = HTTPServer(('0.0.0.0', 8080), MyHandler)

# THE FIX: Always use ThreadingHTTPServer (handles each request in a worker thread)
from http.server import ThreadingHTTPServer
server = ThreadingHTTPServer(('0.0.0.0', 8080), BaseHTTPRequestHandler)
```

Pitfall 2: Omitting the `timeout` Parameter in `urllib.request.urlopen`

By default, `urllib.request.urlopen` uses the global socket default timeout (`None`), meaning a client connection can **hang forever** if the remote host drops the TCP packet:

```
# THE TRAP:
# with urllib.request.urlopen("https://example.com") as resp: ...  # Can hang indefinitely!

# THE FIX: Always specify an explicit timeout in seconds
import urllib.request
with urllib.request.urlopen("https://example.com", timeout=5.0) as resp:
    data = resp.read()
```

Pitfall 3: Not Reading `Content-Length` in `do_POST`

In `do_POST`, calling `self.rfile.read()` without specifying the number of bytes to read will hang indefinitely because the socket connection remains open for keep-alive:

```
# THE TRAP:
def do_POST(self):
    # data = self.rfile.read()  # HANGS! Socket is waiting for EOF that never arrives!

# THE FIX: Read only up to Content-Length bytes:
def do_POST(self):
    length = int(self.headers.get('Content-Length', 0))
    data = self.rfile.read(length)
```

Exercises

1. Build a zero-dependency HTTP server that serves static files from a local directory, returning `404 Not Found` if the file does not exist.
2. Implement an HTTP handler that parses query parameters (e.g. `/search?q=python&limit=10`) using `urllib.parse.parse_qs`.
3. Write a script that uses `http.client.HTTPConnection` to send a `HEAD` request to inspect response headers without downloading the body.
4. Implement basic HTTP authentication verification (`Authorization: Basic ...`) in a custom `BaseHTTPRequestHandler`.
5. Measure the throughput in requests-per-second of `ThreadingHTTPServer` when queried by 10 concurrent threads using `urllib.request`.

Further reading

- Python Documentation: `http.server` — *HTTP servers*.
- Python Documentation: `http.client` — *HTTP protocol client*.
- Python Documentation: `urllib.request` — *Extensible library for opening URLs*.
- RFC 9110: *HTTP Semantics*.

High-Performance File I/O with Memory-Mapped Files (mmap)

High-Performance File I/O with Memory-Mapped Files (`mmap`)

After reading this chapter, you will master operating-system-level virtual memory mapping using Python's `mmap` module, inspect multi-gigabyte files with a near-zero memory footprint, execute in-place binary substitutions directly on disk, search disk contents using regular expressions without loading strings into RAM, and establish shared memory IPC.

Mental model

Traditional file I/O requires copying data multiple times: from disk storage into the OS kernel buffer cache, and then from the kernel buffer into a Python heap-allocated `bytes` or `str` object via the `read()` system call:

```
Standard File I/O (Two Copies):
  Disk File    [ OS Kernel Page Cache ]    read() copies    Python Heap Object
                                                                (High RAM usage!)
```

A **memory-mapped file** (`mmap`) asks the operating system kernel to map a file's disk blocks directly into your process's **virtual address space**. Accessing the `mmap` object is as fast as accessing a memory array, with the OS page cache transparently paging blocks in and out on demand:

```
Memory-Mapped File (Zero User-Space Buffer Copies):

  Disk Storage (e.g. 50 GB log file or database tables)

                           Direct Virtual Memory Mapping (mmap)


  Process Virtual Address Space
    mm[0:1024] reads directly from kernel page cache
    mm[offset] = b'X' writes directly to page cache
    (Instant random access, O(1) Python heap footprint)
```

Minimal example

Save as `mmap_overview.py`:

```
# mmap_overview.py
import mmap
import tempfile

def main() -> None:
    # 1. Create a temporary file with sample log data
    with tempfile.NamedTemporaryFile(delete=False) as f:
        f.write(b"2026-09-07 12:00:00 [INFO] Cluster initialized\n")
        f.write(b"2026-09-07 12:00:05 [WARN] Disk watermark 85%\n")
        f.write(b"2026-09-07 12:00:10 [ERR ] Failed node: srv-04\n")
        filepath = f.name

    # 2. Map the file into virtual memory
    with open(filepath, "r+b") as file_obj:
        # mmap(fileno, length): 0 means map the entire file
        with mmap.mmap(file_obj.fileno(), 0) as mm:
            print(f"Memory-mapped file size: {mm.size()} bytes")

            # Search without loading file into Python RAM
            target = b"[ERR ]"
            offset = mm.find(target)
            print(f"Found '{target.decode()}' at byte offset: {offset}")

            # Read a specific slice directly from disk memory
            mm.seek(offset)
            error_line = mm.readline()
            print(f"Read line: {error_line.decode().strip()}")

            # In-place disk mutation: overwrite [ERR ] with [CRIT]
            mm.seek(offset)
            mm.write(b"[CRIT]")
            mm.flush()  # Ensure changes are written back to physical disk

    # Verify mutation persisted in physical file
    with open(filepath, "rb") as verify_f:
        print(f"\nPersisted file content:\n{verify_f.read().decode()}")

if __name__ == "__main__":
    main()
```

Run via `uv run python mmap_overview.py`:

```
Memory-mapped file size: 140 bytes
```

```
Found '[ERR ]' at byte offset: 113
Read line: [ERR ] Failed node: srv-04

Persisted file content:
2026-09-07 12:00:00 [INFO] Cluster initialized
2026-09-07 12:00:05 [WARN] Disk watermark 85%
2026-09-07 12:00:10 [CRIT] Failed node: srv-04
```

Worked examples

Case 1: Regex Searching Gigabyte Logs with Zero RAM Allocation

When auditing 20 GB web server logs, calling `file.read()` triggers an Out-Of-Memory error. Because `mmap` supports Python's buffer protocol, standard `re` expressions can search directly over the memory map:

```python
# mmap_regex_search.py
import mmap
import re
import tempfile

def generate_large_log(filepath: str, line_count: int = 10_000) -> None:
    with open(filepath, "wb") as f:
        for i in range(line_count):
            if i == 4200:
                f.write(b'192.168.1.100 - [2026-09-07] "GET /admin/secret" 403 128\n')
            elif i == 8900:
                f.write(b'10.0.0.50 - [2026-09-07] "POST /api/v1/auth" 500 256\n')
            else:
                f.write(b'127.0.0.1 - [2026-09-07] "GET /health" 200 32\n')

def main() -> None:
    with tempfile.NamedTemporaryFile(delete=False) as tmp:
        filepath = tmp.name

    generate_large_log(filepath)

    pattern = re.compile(rb'(\d+\.\d+\.\d+\.\d+) - .* "([A-Z]+ [^"]+)" ([45]\d\d)')

    with open(filepath, "rb") as f:
        with mmap.mmap(f.fileno(), 0, access=mmap.ACCESS_READ) as mm:
            print("Scanning log file with zero heap allocation:")
            for match in pattern.finditer(mm):
                ip, endpoint, status = match.groups()
```

```
                print(f"  Anomaly detected at offset {match.start():6d}: IP={ip.decode()} St

if __name__ == "__main__":
    main()
```

Run:

```
uv run python mmap_regex_search.py
```

Output:

```
Scanning log file with zero heap allocation:
  Anomaly detected at offset 193200: IP=192.168.1.100 Status=403 (GET /admin/secret)
  Anomaly detected at offset 409411: IP=10.0.0.50 Status=500 (POST /api/v1/auth)
```

Case 2: In-Place Binary Patching and Header Updates

In firmware flashing and database file management, updating records in place avoids rewriting the entire file:

```
# binary_header_patcher.py
import mmap
import struct
import tempfile

# Binary Header Format: 4-byte Magic, 4-byte Version, 8-byte Checksum
HEADER_FORMAT = "!4sIQ"

def main() -> None:
    with tempfile.NamedTemporaryFile(delete=False) as f:
        # Write initial header
        initial_bytes = struct.pack(HEADER_FORMAT, b"BLOB", 1, 0x1122334455667788)
        f.write(initial_bytes + (b"\x00" * 128))
        filepath = f.name

    with open(filepath, "r+b") as f:
        with mmap.mmap(f.fileno(), 0) as mm:
            magic, version, checksum = struct.unpack(HEADER_FORMAT, mm[:16])
            print("Initial Header:")
            print(f"  Magic:    {magic}")
            print(f"  Version:  {version}")
            print(f"  Checksum: 0x{checksum:X}")

            # Bump version to 2 and update checksum in place!
```

```
            new_header = struct.pack(HEADER_FORMAT, b"BLOB", 2, 0xAABBCCDDEEFF0011)
            mm[:16] = new_header
            mm.flush()

    with open(filepath, "rb") as f:
        magic, version, checksum = struct.unpack(HEADER_FORMAT, f.read(16))
        print("\nVerified Header on Disk:")
        print(f"  Magic:    {magic}")
        print(f"  Version:  {version}")
        print(f"  Checksum: 0x{checksum:X}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python binary_header_patcher.py
```

Output:

```
Initial Header:
  Magic:    b'BLOB'
  Version:  1
  Checksum: 0x1122334455667788

Verified Header on Disk:
  Magic:    b'BLOB'
  Version:  2
  Checksum: 0xAABBCCDDEEFF0011
```

Case 3: Anonymous Shared Memory IPC Between Processes

By specifying `-1` as the file descriptor, `mmap` allocates an **anonymous shared memory segment** backed by RAM rather than a physical file, enabling zero-disk IPC:

```
# anonymous_shared_memory.py
import mmap
import os
import time

def main() -> None:
    # fileno = -1 creates an anonymous memory map
    shm = mmap.mmap(-1, 1024)
```

```
    # Fork child process
    pid = os.fork()

    if pid == 0:
        # Child process: Wait for parent to write data
        time.sleep(0.02)
        shm.seek(0)
        message = shm.readline().decode("utf-8").strip()
        print(f"[Child PID {os.getpid()}] Read from shared memory: '{message}'")
        shm.close()
        os._exit(0)
    else:
        # Parent process: Write data to shared memory
        shm.write(b"SYNC_TOKEN: 0x8FA2BC99\n")
        print(f"[Parent PID {os.getpid()}] Wrote token to anonymous shared memory.")
        os.waitpid(pid, 0)
        shm.close()

if __name__ == "__main__":
    main()
```

Run:

```
uv run python anonymous_shared_memory.py
```

Output:

```
[Parent PID 12345] Wrote token to anonymous shared memory.
[Child PID 12346] Read from shared memory: 'SYNC_TOKEN: 0x8FA2BC99'
```

Pitfalls

Pitfall 1: Attempting to Map an Empty (0-Byte) File

The operating system cannot map a file with a length of 0 bytes. Attempting to do so raises `ValueError`:

```
# THE TRAP:
import tempfile
import mmap

with tempfile.NamedTemporaryFile() as empty_file:
    # empty_file has size 0
    try:
```

```
        mm = mmap.mmap(empty_file.fileno(), 0)
    except ValueError as err:
        print(f"Caught: {err}")
```

Output:

```
Caught: cannot mmap an empty file
```

The Fix: Ensure the file contains at least 1 byte before mapping, or check `os.path.getsize(filepath) > 0`.

Pitfall 2: Modifying File Size Outside `mmap`

If another process or thread truncates the underlying file while a `memoryview` or `mmap` is open, reading beyond the new end of file triggers an uncatchable OS `SIGBUS` (Bus Error) signal, crashing the Python process immediately. Always coordinate file resizing with process-wide locks.

Exercises

1. Create a 10 MB file filled with random ASCII bytes. Use `mmap.find()` to locate all occurrences of the word `"TARGET"`.
2. Write a script that counts line breaks (`\n`) in a large text file using `mmap` and compare its execution time and memory usage against `file.readlines()`.
3. Implement a circular ring buffer inside an anonymous memory map (`mmap(-1, size)`) shared between two forked processes.
4. Open a read-only binary file using `access=mmap.ACCESS_READ` and verify that attempting to write to the memory map raises `TypeError`.
5. Build an in-place string replacement function that searches for a 4-letter token and replaces it with a new 4-letter token across a 50 MB log file.

Further reading

- Python Documentation: `mmap` — *Memory-mapped file support.*
- Linux Programmer's Manual: `mmap(2)` — *Map files or devices into memory.*
- David Beazley: *Python Cookbook* (Recipe 5.18: Memory Mapping an I/O File).

Runtime Introspection and Reflection with inspect

Runtime Introspection and Reflection with `inspect`

After reading this chapter, you will master Python's runtime reflection and introspection machinery using the standard library `inspect` module, dynamically parse function signatures and type annotations, bind and validate arguments programmatically before invocation, navigate live Python call frames, extract source code and docstrings dynamically, and engineer production-grade dependency injection containers.

Mental model

In compiled languages like C or Go, function signatures, parameter names, and local variable scopes are largely erased at compile time. In Python, code objects, functions, classes, and execution frames retain their full symbolic metadata in memory at runtime:

```
Python Callable (Function / Method / Closure)

__code__ (Bytecode & Names)    __annotations__ (Type Hints)
__defaults__ (Arg Defaults)    __kwdefaults__ (KW Defaults)


                  inspect.signature(fn)


inspect.Signature
   return_annotation: type
   parameters: MappingProxyType
       'req_id': Parameter(POSITIONAL_OR_KEYWORD, int)
       'dry_run': Parameter(KEYWORD_ONLY, bool, False)
       '*args': Parameter(VAR_POSITIONAL)


             sig.bind(*args, **kwargs)


inspect.BoundArguments
   arguments: {'req_id': 100, 'dry_run': False}
   apply_defaults(): injects missing default parameters
```

Furthermore, whenever a Python function executes, CPython pushes a **frame object** (`PyFrameObject`) onto the runtime call stack. The `inspect` module allows libraries to inspect the active call stack, locate the caller's file and line number, and introspect the local namespace.

Minimal example

Save as `inspect_overview.py`:

```
# inspect_overview.py
import inspect
from typing import Any

def provision_cluster(
    cluster_name: str,
    node_count: int = 3,
    *tags: str,
    auto_repair: bool = True,
    **metadata: Any,
) -> dict[str, Any]:
    """Provisions a virtual computing cluster with specified parameters."""
    return {
        "name": cluster_name,
        "nodes": node_count,
        "tags": tags,
        "auto_repair": auto_repair,
        "metadata": metadata,
    }

def main() -> None:
    # 1. Extract signature metadata
    sig = inspect.signature(provision_cluster)
    print(f"Callable signature:\n  {sig}\n")

    print("Parameter breakdown:")
    for name, param in sig.parameters.items():
        print(
            f"  Name: {name:<12} "
            f"Kind: {param.kind.name:<22} "
            f"Default: {str(param.default):<10} "
            f"Annotation: {param.annotation}"
        )

    # 2. Programmatically bind runtime arguments
```

```
    bound = sig.bind("prod-eu-west", 10, "k8s", "gpu", auto_repair=False, zone="eu-west-1a")
    print(f"\nBound arguments dictionary:\n  {bound.arguments}")

    # 3. Invoke function with bound arguments
    result = provision_cluster(*bound.args, **bound.kwargs)
    print(f"\nFunction result:\n  {result}")

if __name__ == "__main__":
    main()
```

Run via `uv run python inspect_overview.py`:

```
Callable signature:
  (cluster_name: str, node_count: int = 3, *tags: str, auto_repair: bool = True, **metadata: An

Parameter breakdown:
  Name: cluster_name Kind: POSITIONAL_OR_KEYWORD   Default: <class 'inspect._empty'> Annotation
  Name: node_count   Kind: POSITIONAL_OR_KEYWORD   Default: 3          Annotation: <class 'int'
  Name: tags         Kind: VAR_POSITIONAL          Default: <class 'inspect._empty'> Annotation
  Name: auto_repair  Kind: KEYWORD_ONLY            Default: True       Annotation: <class 'bool
  Name: metadata     Kind: VAR_KEYWORD             Default: <class 'inspect._empty'> Annotation

Bound arguments dictionary:
  {'cluster_name': 'prod-eu-west', 'node_count': 10, 'tags': ('k8s', 'gpu'), 'auto_repair': Fal

Function result:
  {'name': 'prod-eu-west', 'nodes': 10, 'tags': ('k8s', 'gpu'), 'auto_repair': False, 'metadata
```

Worked examples

Case 1: Runtime Type-Enforcing Decorator with `Signature.bind()`

While static type checkers like `mypy` and `pyright` catch errors before execution, boundaries such as HTTP endpoints, message queues, and user scripts receive raw data. `inspect.signature` enables writing a zero-boilerplate runtime type validator:

```
# runtime_type_guard.py
import functools
import inspect
from typing import Any, Callable

def enforce_contract(func: Callable[..., Any]) -> Callable[..., Any]:
    """Validates runtime arguments against function type annotations."""
```

```
    sig = inspect.signature(func)

    @functools.wraps(func)
    def wrapper(*args: Any, **kwargs: Any) -> Any:
        # Bind incoming arguments to parameter names, handling defaults
        bound = sig.bind(*args, **kwargs)
        bound.apply_defaults()

        for param_name, value in bound.arguments.items():
            param = sig.parameters[param_name]
            expected_type = param.annotation

            # Validate basic types when annotations are explicit classes
            if expected_type is not inspect.Parameter.empty and isinstance(expected_type, ty
                if not isinstance(value, expected_type):
                    raise TypeError(
                        f"Contract violation on '{func.__name__}': "
                        f"argument '{param_name}' must be {expected_type.__name__}, "
                        f"received {type(value).__name__} ({value!r})"
                    )

        result = func(*args, **kwargs)

        # Validate return value
        ret_type = sig.return_annotation
        if ret_type is not inspect.Signature.empty and isinstance(ret_type, type):
            if not isinstance(result, ret_type):
                raise TypeError(
                    f"Contract violation on '{func.__name__}': "
                    f"expected return {ret_type.__name__}, got {type(result).__name__}"
                )

        return result

    return wrapper

@enforce_contract
def scale_service(service_name: str, target_replicas: int, dry_run: bool = False) -> str:
    return f"Service '{service_name}' scaling to {target_replicas} (dry_run={dry_run})"

def main() -> None:
    # 1. Valid call succeeds
    msg = scale_service("api-gateway", 8)
    print(f"Success: {msg}")

    # 2. Invalid parameter type raises immediate TypeError
```

```
    try:
        scale_service("api-gateway", "eight")  # type: ignore
    except TypeError as err:
        print(f"\nCaught validation error:\n  {err}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python runtime_type_guard.py
```

Output:

```
Success: Service 'api-gateway' scaling to 8 (dry_run=False)

Caught validation error:
  Contract violation on 'scale_service': argument 'target_replicas' must be int, received str (
```

Case 2: Zero-Magic Dependency Injection Container

Modern web frameworks like FastAPI and test frameworks like `pytest` automatically inject database sessions, configuration objects, and authentication credentials into endpoint handlers by introspecting parameter type annotations:

```
# dependency_injection.py
import inspect
from typing import Any, Callable, Dict, Type

class Container:
    """Type-based dependency injection registry."""

    def __init__(self) -> None:
        self._services: Dict[Type[Any], Any] = {}

    def register(self, service_type: Type[Any], instance: Any) -> None:
        """Registers a singleton instance for a given type."""
        self._services[service_type] = instance

    def invoke(self, target: Callable[..., Any], **explicit_args: Any) -> Any:
        """Introspects target parameters and injects matching dependencies."""
        sig = inspect.signature(target)
        resolved_kwargs = dict(explicit_args)
```

```
        for param_name, param in sig.parameters.items():
            # If the argument was already explicitly supplied, skip auto-wiring
            if param_name in resolved_kwargs:
                continue

            target_type = param.annotation
            if target_type in self._services:
                resolved_kwargs[param_name] = self._services[target_type]
            elif param.default is inspect.Parameter.empty:
                raise RuntimeError(
                    f"Unable to resolve required parameter '{param_name}: {target_type}' for
                )

        return target(**resolved_kwargs)

# Sample domain services
class DatabaseConnectionPool:
    def execute(self, query: str) -> str:
        return f"[DB] Executed '{query}'"

class AuditLogger:
    def log(self, event: str) -> str:
        return f"[AUDIT] Recorded '{event}'"

# Handler function declaring its dependencies via type hints
def checkout_handler(
    cart_id: int,
    db: DatabaseConnectionPool,
    audit: AuditLogger,
    currency: str = "USD",
) -> str:
    db_res = db.execute(f"UPDATE carts SET status='PAID' WHERE id={cart_id}")
    audit_res = audit.log(f"Cart {cart_id} paid in {currency}")
    return f"{db_res}\n{audit_res}"

def main() -> None:
    container = Container()
    container.register(DatabaseConnectionPool, DatabaseConnectionPool())
    container.register(AuditLogger, AuditLogger())

    # Invoke without manually passing db or audit
    output = container.invoke(checkout_handler, cart_id=9812)
    print(f"Handler output:\n{output}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python dependency_injection.py
```

Output:

```
Handler output:
[DB] Executed 'UPDATE carts SET status='PAID' WHERE id=9812'
[AUDIT] Recorded 'Cart 9812 paid in USD'
```

Case 3: Caller-Context Telemetry and Frame Navigation

Structured loggers and profiling tools frequently need to know who invoked a function without forcing developers to pass `__file__` and `__line__` manually. By inspecting the active execution frame with `inspect.currentframe()` or `sys._getframe()`, we can climb the call stack:

```python
# frame_telemetry.py
import inspect
import sys
from typing import Any, Dict

def capture_caller_context(depth: int = 1) -> Dict[str, Any]:
    """Captures caller metadata by walking up the CPython call stack."""
    frame = sys._getframe(depth + 1)
    try:
        return {
            "function": frame.f_code.co_name,
            "filename": frame.f_code.co_filename.split("/")[-1],
            "line": frame.f_lineno,
            "module": frame.f_globals.get("__name__", "<unknown>"),
        }
    finally:
        # Crucial: Delete frame reference to prevent circular garbage collection cycles
        del frame

def emit_telemetry(action: str, metric_value: float) -> None:
    ctx = capture_caller_context(depth=1)
    file_name = ctx["filename"]
    line_no = ctx["line"]
    fn_name = ctx["function"]
    print(f"[{file_name}:{line_no} in {fn_name}()] ACTION='{action}' VALUE={metric_value}")

class PaymentProcessor:
    def charge_card(self, amount: float) -> None:
        emit_telemetry("charge_initiated", amount)
```

```
def main() -> None:
    processor = PaymentProcessor()
    processor.charge_card(149.95)

if __name__ == "__main__":
    main()
```

Run:

```
uv run python frame_telemetry.py
```

Output:

```
[frame_telemetry.py:31 in charge_card()] ACTION='charge_initiated' VALUE=149.95
```

Pitfalls

Pitfall 1: Retaining Frame References and Creating GC Leaks

Frame objects contain references to their local dictionaries (`f_locals`), code objects, and enclosing execution scopes. Storing a frame in an exception trace or local variable without explicitly deleting it creates reference cycles that keep large local objects alive until the next garbage collection cycle:

```
# THE TRAP:
import inspect

def leaky_tracer():
    frame = inspect.currentframe()
    # If 'frame' is saved in a list or closure, all local variables in leaky_tracer
    # remain anchored in RAM!
    return frame.f_back

# THE FIX:
def safe_tracer():
    frame = inspect.currentframe()
    try:
        return frame.f_back.f_lineno if frame and frame.f_back else None
    finally:
        del frame  # Break reference cycle immediately
```

Pitfall 2: `inspect.getsource()` on Dynamically Evaluated or C Code

Functions loaded from compiled C-extensions (like `math.sin` or `sys.exit`) or created dynamically via `exec()` / `eval()` do not exist as source files on disk. Calling `inspect.getsource()` on them raises `OSError` or `TypeError`:

```
# THE TRAP:
import math
import inspect

try:
    source = inspect.getsource(math.sin)
except TypeError as err:
    print(f"Caught: {err}")
```

Output:

```
Caught: <built-in function sin> is not a module, class, method, function, traceback, frame, or
```

The Fix: Always guard calls to `inspect.getsource()` with `inspect.isbuiltin()` or catch `(OSError, TypeError)`.

Exercises

1. Build a function `inspect_callable(fn)` that prints whether the callable is a regular function, a coroutine function (`async def`), a generator function (`yield`), or a class constructor.
2. Implement a validation decorator that inspects keyword-only parameters (`Parameter.KEYWORD_ONLY`) and ensures they are never omitted when calling the function.
3. Write a dynamic JSON schema generator that converts a function's type annotations and default parameters into an OpenAPI/JSON-Schema-compliant parameter dictionary.
4. Using `sys._getframe()`, implement an `@auto_logger` decorator that logs the execution time and the name of the function that invoked the decorated function.
5. Create a command-line dispatcher that inspects an arbitrary function signature and automatically maps CLI strings from `sys.argv[1:]` into typed function arguments (`int`, `float`, `str`, `bool`).

Further reading

- Python Documentation: `inspect` — *Inspect live objects*.
- Brett Slatkin: *Effective Python* (Item 48: Validate Subclasses with `__init_subclass__` and Reflection).
- Luciano Ramalho: *Fluent Python* (Chapter 7: Functions as First-Class Objects; Chapter 21: Class Metaprogramming).
- PEP 362: *Function Signature Object*.

Weak References and Garbage Collection (weakref, gc)

Weak References and Garbage Collection (`weakref`, `gc`)

After reading this chapter, you will master CPython's hybrid memory management architecture (reference counting coupled with generational cyclic garbage collection), construct self-pruning caches using `weakref.WeakValueDictionary` and `weakref.WeakKeyDictionary`, guarantee resource reclamation with `weakref.finalize` without relying on hazardous `__del__` methods, and diagnose reference cycles using the `gc` module.

Mental model

CPython manages memory primarily through **reference counting**: every object tracks how many references point to it (`ob_refcnt`). When `ob_refcnt` drops to zero, CPython immediately deallocates the object's memory without waiting:

```
Reference Counting (Instant Deallocation):
  ref_a = Node()      Node (refcount: 1)
  ref_b = ref_a       Node (refcount: 2)
  del ref_a           Node (refcount: 1)
  del ref_b           Node (refcount: 0)    Deallocated immediately!
```

However, **circular references** defeat pure reference counting. If Object A references Object B, and Object B references Object A, their reference counts never drop to zero even after all external names are deleted:

```
Reference Cycle (Leaked Without Cyclic GC):
  [ Root Namespace ]
            (External variables deleted!)

             refers to
      Object A            Object B
      (refcnt=1)           (refcnt=1)
             refers to
```

CPython solves this with a secondary **generational garbage collector** (`gc` module) that periodically inspects container objects (Gen 0, Gen 1, Gen 2), detects unreachable islands of cyclic references, and breaks them.

A **weak reference** (`weakref`) allows you to reference an object without incrementing its `ob_refcnt`. If all strong references disappear, the object is immediately deallocated, and the weak reference automatically invalidates itself:

```
Weak Reference Architecture:
  strong_ref = Node()  (ob_refcnt = 1)   [ Target Node ]

  weak_ref = weakref.ref(strong_ref)         (ob_refcnt remains 1!)

  del strong_ref     Target Node deallocated instantly!
  weak_ref()         Returns None (dead reference)
```

Minimal example

Save as `weakref_overview.py`:

```python
# weakref_overview.py
import weakref

class HeavyModel:
    def __init__(self, model_id: str) -> None:
        self.model_id = model_id

    def __repr__(self) -> str:
        return f"HeavyModel({self.model_id})"

def main() -> None:
    # 1. Create a target object with one strong reference
    model = HeavyModel("transformer-v3")
    print(f"Created object: {model}")

    # 2. Create a weak reference; reference count is unchanged
    weak_handle = weakref.ref(model)
    print(f"Weak handle before deletion: {weak_handle}")
    print(f"Dereferenced value: {weak_handle()}")

    # 3. Delete the only strong reference
    print("\nDeleting strong reference 'model'...")
    del model

    # 4. The target object is immediately reclaimed; weak handle returns None
    print(f"Weak handle after deletion: {weak_handle}")
    print(f"Dereferenced value: {weak_handle()}")
```

```
if __name__ == "__main__":
    main()
```

Run via `uv run python weakref_overview.py`:

```
Created object: HeavyModel(transformer-v3)
Weak handle before deletion: <weakref at 0x...; to 'HeavyModel' at 0x...>
Dereferenced value: HeavyModel(transformer-v3)

Deleting strong reference 'model'...
Weak handle after deletion: <weakref at 0x...; dead>
Dereferenced value: None
```

Worked examples

Case 1: Self-Pruning In-Memory Entity Cache with `WeakValueDictionary`

Standard dictionaries keep cached objects alive forever unless explicitly deleted, leading to silent memory leaks in long-running services. A `WeakValueDictionary` automatically evicts entries as soon as no external client references the cached instance:

```
# entity_cache.py
import weakref

class TenantConfiguration:
    def __init__(self, tenant_id: str, quota: int) -> None:
        self.tenant_id = tenant_id
        self.quota = quota

    def __repr__(self) -> str:
        return f"TenantConfiguration(id={self.tenant_id}, quota={self.quota})"

class TenantRegistry:
    def __init__(self) -> None:
        # Cache values weakly; entries disappear when callers drop their reference
        self._cache: weakref.WeakValueDictionary[str, TenantConfiguration] = (
            weakref.WeakValueDictionary()
        )

    def get_or_load(self, tenant_id: str) -> TenantConfiguration:
        config = self._cache.get(tenant_id)
        if config is None:
            print(f"  [CACHE MISS] Fetching config for '{tenant_id}' from database...")
            config = TenantConfiguration(tenant_id=tenant_id, quota=10_000)
```

```
            self._cache[tenant_id] = config
        else:
            print(f"  [CACHE HIT ] Returning cached config for '{tenant_id}'")
        return config

    def active_cached_keys(self) -> list[str]:
        return list(self._cache.keys())

def main() -> None:
    registry = TenantRegistry()

    # Client A requests tenant-01
    client_a_handle = registry.get_or_load("tenant-01")
    print(f"Active cache keys: {registry.active_cached_keys()}")

    # Client B requests tenant-01 (cache hit)
    client_b_handle = registry.get_or_load("tenant-01")

    # Client A drops its reference; Client B still holds one
    print("\nClient A releasing reference...")
    del client_a_handle
    print(f"Active cache keys: {registry.active_cached_keys()}")

    # Client B drops its reference; no strong references remain!
    print("Client B releasing reference...")
    del client_b_handle
    print(f"Active cache keys after all clients disconnect: {registry.active_cached_keys()}"

    # Next request triggers a fresh load from the database
    print("\nSubsequent request:")
    _ = registry.get_or_load("tenant-01")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python entity_cache.py
```

Output:

```
  [CACHE MISS] Fetching config for 'tenant-01' from database...
Active cache keys: ['tenant-01']
  [CACHE HIT ] Returning cached config for 'tenant-01'

Client A releasing reference...
Active cache keys: ['tenant-01']
```

```
Client B releasing reference...
Active cache keys after all clients disconnect: []

Subsequent request:
  [CACHE MISS] Fetching config for 'tenant-01' from database...
```

Case 2: Deterministic OS Resource Cleanup with `weakref.finalize`

Relying on `__del__` for resource cleanup is notoriously fragile: exceptions raised inside `__del__` are silently swallowed, execution timing during interpreter shutdown is undefined, and resurrection bugs can corrupt state. `weakref.finalize` provides a safe, deterministic alternative:

```python
# safe_resource_reaper.py
import os
import tempfile
import weakref

def _release_temp_file(filepath: str) -> None:
    """Standalone cleanup function. Must NOT hold reference to the owner object."""
    if os.path.exists(filepath):
        os.remove(filepath)
        print(f"[CLEANUP] Deleted temporary file from disk: {filepath}")

class EphemeralBuffer:
    def __init__(self, prefix: str) -> None:
        fd, self.path = tempfile.mkstemp(prefix=prefix)
        os.close(fd)
        print(f"[INIT] Allocated scratch file: {self.path}")

        # Register finalizer without passing 'self' to the callback
        self._finalizer = weakref.finalize(self, _release_temp_file, self.path)

    @property
    def is_alive(self) -> bool:
        return self._finalizer.alive

    def close(self) -> None:
        """Manual explicit cleanup. Can be called safely multiple times."""
        self._finalizer()

def main() -> None:
    # 1. Automatic cleanup when object goes out of scope
    print("--- Scope 1: Implicit deallocation ---")
    def worker():
```

```
        buf = EphemeralBuffer("task_worker_")
        print(f"Buffer is active: {buf.is_alive}")

    worker()
    print("Worker function exited.\n")

    # 2. Explicit manual cleanup
    print("--- Scope 2: Explicit manual close ---")
    buf2 = EphemeralBuffer("manual_task_")
    print(f"Buffer alive before close: {buf2.is_alive}")
    buf2.close()
    print(f"Buffer alive after close: {buf2.is_alive}")
    # Calling close again is a safe no-op
    buf2.close()

if __name__ == "__main__":
    main()
```

Run:

```
uv run python safe_resource_reaper.py
```

Output:

```
--- Scope 1: Implicit deallocation ---
[INIT] Allocated scratch file: /tmp/task_worker_...
Buffer is active: True
[CLEANUP] Deleted temporary file from disk: /tmp/task_worker_...
Worker function exited.

--- Scope 2: Explicit manual close ---
[INIT] Allocated scratch file: /tmp/manual_task_...
Buffer alive before close: True
[CLEANUP] Deleted temporary file from disk: /tmp/manual_task_...
Buffer alive after close: False
```

Case 3: Diagnosing and Breaking Cyclic Reference Leaks with `gc`

When developing complex graph structures or event subscription architectures, circular references prevent reference counting from freeing memory. The `gc` module exposes tools to inspect and break these cycles:

```
# cycle_diagnostics.py
import gc
```

```
from typing import Optional

class GraphNode:
    def __init__(self, label: str) -> None:
        self.label = label
        self.neighbor: Optional["GraphNode"] = None

    def __repr__(self) -> str:
        return f"GraphNode({self.label})"

def inspect_cycle() -> None:
    # 1. Create a circular reference inside local scope
    node_a = GraphNode("NodeA")
    node_b = GraphNode("NodeB")
    node_a.neighbor = node_b
    node_b.neighbor = node_a

    print("Created circular reference: A <   B")

    # 2. Inspect referrers
    referrers = gc.get_referrers(node_a)
    referrer_types = [type(r).__name__ for r in referrers]
    print(f"Referrers to NodeA: {referrer_types}")

def main() -> None:
    gc.collect()

    inspect_cycle()

    # When inspect_cycle() returns, its stack frame is popped.
    # NodeA and NodeB have refcount 1 due to each other, but cannot be reached!
    print("\nLocal frame popped. Cycle is now orphaned.")

    # 3. Trigger explicit cyclic garbage collection
    collected = gc.collect()
    print(f"gc.collect() successfully reaped {collected} unreachable cyclic objects.")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python cycle_diagnostics.py
```

Output:

```
Created circular reference: A <   B
```

```
Referrers to NodeA: ['GraphNode']

Local frame popped. Cycle is now orphaned.
gc.collect() successfully reaped 2 unreachable cyclic objects.
```

Pitfalls

Pitfall 1: Attempting to Weakly Reference Built-in Types Directly

CPython's core built-in types (`dict`, `list`, `int`, `str`, `tuple`) optimize memory by omitting the `__weakref__` pointer slot from their C structures. Calling `weakref.ref()` on them raises a `TypeError`:

```
# THE TRAP:
import weakref

raw_data = {"status": "ok", "retries": 3}
try:
    ref = weakref.ref(raw_data)
except TypeError as err:
    print(f"Caught: {err}")
```

Output:

```
Caught: cannot create weak reference to 'dict' object
```

The Fix: Subclass the built-in type, or wrap the data in a custom class, which adds the `__weakref__` attribute automatically:

```
# THE FIX:
class TrackedDict(dict):
    pass

tracked = TrackedDict({"status": "ok"})
valid_ref = weakref.ref(tracked)
print(f"Successfully referenced: {valid_ref()['status']}")
```

Pitfall 2: Capturing `self` in a Finalizer Callback

Passing a bound method (such as `self.cleanup`) or a closure capturing `self` into `weakref.finalize` creates a strong reference from the finalizer back to the object. Because the finalizer holds `self`, the object never dies, and the finalizer never executes!

```
# THE TRAP (Memory leak!):
class LeakyResource:
    def __init__(self):
        # self.cleanup is a BOUND METHOD that holds a strong reference to self!
        self._finalizer = weakref.finalize(self, self.cleanup)

    def cleanup(self):
        print("Cleaning up...")

# THE FIX:
class SafeResource:
    def __init__(self):
        # Pass a static method or module-level function and only the primitive handle
        self._finalizer = weakref.finalize(self, SafeResource._cleanup_static, "raw_handle")

    @staticmethod
    def _cleanup_static(handle: str):
        print(f"Safely cleaning up {handle}")
```

Exercises

1. Build a parent-child tree node structure where each parent maintains a list of strong references to its children, while each child maintains a `weakref.ref` back to its parent to prevent circular reference memory leaks.
2. Implement a custom `@memoize_weak` decorator that caches function results using a `WeakValueDictionary`, ensuring that cached results are evicted as soon as caller code drops them.
3. Use `gc.get_objects()` to write a diagnostic utility that prints the top 5 classes with the highest number of live instances in memory.
4. Benchmark the latency of creating and destroying 100,000 objects with pure reference counting versus 100,000 objects chained in reference cycles collected by `gc.collect()`.
5. Implement a `SharedLock` resource that uses `weakref.finalize` to guarantee that if a thread crashes or an object is garbage-collected without explicitly releasing its lock, the lock is automatically freed.

Further reading

- Python Documentation: `weakref` — *Weak references*.
- Python Documentation: `gc` — *Garbage Collector interface*.
- Luciano Ramalho: *Fluent Python* (Chapter 6: Object References, Mutability, and Recycling).
- Anthony Shaw: *CPython Internals* (Chapter 10: Memory Management and the Garbage Collector).

Mocking and Test Doubles (unittest.mock)

Mocking and Test Doubles (`unittest.mock`)

After reading this chapter, you will master Python's testing and isolation toolkit using the standard library `unittest.mock` module, replace unreliable external boundaries (databases, HTTP APIs, file systems, clocks) with robust test doubles, intercept functions and classes using `@patch` and `patch.object`, enforce API contracts and prevent mock interface drift using `autospec=True`, simulate multi-step network failures with `side_effect`, and test asynchronous coroutines using `AsyncMock`.

Mental model

Unit tests should be deterministic, isolated, and fast. However, production code frequently depends on non-deterministic boundaries: remote webhooks, cloud databases, external clock times, or disk storage.

The `unittest.mock` library creates **test doubles**—objects that impersonate real components, record how they were called, and return pre-programmed responses:

```
Production System Flow
  Client Code    [ Real Database / Stripe API / SMTP ]
                  (Slow, unreliable, requires credentials)


                     Test Substitution


Isolated Unit Test Flow
  Client Code    [ unittest.mock.Mock ]
                       return_value: {'status': 'PAID'}
                       side_effect: ConnectionError()
                       records: call_count, call_args
```

The Golden Rule of Patching: "Where to Patch"

The single most common bug when using `unittest.mock.patch` is patching the module where a class or function was **defined**, rather than where it was **imported and looked up**:

```
WRONG:
  patch('urllib.request.urlopen')  # Does NOT affect service.py!
  service.py imported: from urllib.request import urlopen (local reference)

RIGHT:
  patch('my_app.service.urlopen')  # Patches the reference inside service.py!
```

Minimal example

Save as `mock_overview.py`:

```python
# mock_overview.py
from unittest.mock import Mock, patch

def send_welcome_email(email_client, user_email: str) -> bool:
    """Sends a welcome message and returns True on success."""
    response = email_client.send(
        to=user_email,
        subject="Welcome aboard!",
        body="Thank you for joining our platform.",
    )
    return response.get("delivered", False)

def main() -> None:
    # 1. Instantiate a mock client
    mock_client = Mock()

    # Configure return value
    mock_client.send.return_value = {"delivered": True, "message_id": "msg_9012"}

    # 2. Execute target function under test
    result = send_welcome_email(mock_client, "alice@example.com")
    print(f"Function result: {result}")

    # 3. Assert mock interactions
    print(f"send() was called: {mock_client.send.called}")
    print(f"send() call count: {mock_client.send.call_count}")

    # Verify exact keyword arguments passed
    mock_client.send.assert_called_once_with(
        to="alice@example.com",
        subject="Welcome aboard!",
        body="Thank you for joining our platform.",
    )
```

```
    print("All mock assertions succeeded!")

if __name__ == "__main__":
    main()
```

Run via `uv run python mock_overview.py`:

```
Function result: True
send() was called: True
send() call count: 1
All mock assertions succeeded!
```

Worked examples

Case 1: Simulating Flaky Networks with `side_effect` Retries

A mock's `side_effect` attribute can accept an exception class or instance (to raise errors) or an iterable (to yield different return values on consecutive calls). This enables testing exponential backoff and retry loops:

```
# retry_client_test.py
import time
from unittest.mock import Mock

class TransientNetworkError(Exception):
    pass

def fetch_telemetry_with_retry(api_client, endpoint: str, max_retries: int = 3) -> dict:
    """Fetches data from an API client, retrying up to max_retries on transient errors."""
    for attempt in range(1, max_retries + 1):
        try:
            return api_client.get(endpoint)
        except TransientNetworkError:
            if attempt == max_retries:
                raise
            time.sleep(0.01 * attempt)
    return {}

def test_retry_succeeds_on_third_attempt() -> None:
    mock_api = Mock()

    # Sequence: Fail with error twice, then succeed on third attempt
    mock_api.get.side_effect = [
```

```
        TransientNetworkError("Gateway Timeout (504)"),
        TransientNetworkError("Connection Reset"),
        {"nodes_online": 42, "status": "HEALTHY"},
    ]

    result = fetch_telemetry_with_retry(mock_api, "/api/v1/cluster/status")

    print(f"Payload received: {result}")
    print(f"Total API attempts made: {mock_api.get.call_count}")
    assert mock_api.get.call_count == 3
    print("Test passed: Retry loop successfully recovered on attempt 3!")

if __name__ == "__main__":
    test_retry_succeeds_on_third_attempt()
```

Run:

```
uv run python retry_client_test.py
```

Output:

```
Payload received: {'nodes_online': 42, 'status': 'HEALTHY'}
Total API attempts made: 3
Test passed: Retry loop successfully recovered on attempt 3!
```

Case 2: Preventing Mock Drift with `autospec=True`

Standard `Mock` and `MagicMock` instances dynamically create any attribute or method accessed on them. If production code refactors a method name or changes parameter signatures, tests using vanilla mocks continue to pass silently. `autospec=True` binds the mock directly to the real class specification:

```
# autospec_drift_prevention.py
from unittest.mock import Mock, create_autospec

class BillingGateway:
    def charge_customer(self, customer_id: str, amount_cents: int, currency: str = "USD") ->
        """Charges a registered customer's saved payment method."""
        return f"tx_success_{customer_id}_{amount_cents}"

def test_vanilla_mock_drift() -> None:
    # Vanilla Mock permits non-existent methods and wrong signatures!
    vanilla_mock = Mock()
    vanilla_mock.non_existent_method("foo")
```

```
    vanilla_mock.charge_customer("cust_01")  # Missing required 'amount_cents'!
    print("Vanilla mock silently succeeded despite invalid method names and missing argument

def test_autospec_drift_detection() -> None:
    # create_autospec introspects BillingGateway methods and signatures
    safe_mock = create_autospec(BillingGateway, instance=True)

    # 1. Calling non-existent method raises AttributeError immediately
    try:
        safe_mock.non_existent_method("foo")
    except AttributeError as err:
        print(f"\nCaught nonexistent method via autospec:\n  {err}")

    # 2. Calling with incorrect signature raises TypeError immediately
    try:
        safe_mock.charge_customer("cust_01")  # Missing required argument!
    except TypeError as err:
        print(f"\nCaught invalid signature via autospec:\n  {err}")

def main() -> None:
    test_vanilla_mock_drift()
    test_autospec_drift_detection()

if __name__ == "__main__":
    main()
```

Run:

```
uv run python autospec_drift_prevention.py
```

Output:

```
Vanilla mock silently succeeded despite invalid method names and missing arguments.

Caught nonexistent method via autospec:
  Mock object has no attribute 'non_existent_method'

Caught invalid signature via autospec:
  missing a required argument: 'amount_cents'
```

Case 3: Testing Asynchronous Event Pipelines with `AsyncMock`

When testing `asyncio` code, methods that return coroutines cannot be replaced with synchronous `Mock` objects. `AsyncMock` ensures that calling the mock returns an awaitable coroutine and records await counts:

```python
# async_worker_test.py
import asyncio
from unittest.mock import AsyncMock

async def process_queue_batch(consumer, alert_service) -> int:
    """Consumes messages from a stream and forwards alerts asynchronously."""
    messages = await consumer.poll_messages(max_items=10)
    processed_count = 0

    for msg in messages:
        if msg.get("severity") == "CRITICAL":
            await alert_service.dispatch_pager(msg["event"])
        await consumer.ack(msg["offset"])
        processed_count += 1

    return processed_count

async def test_async_worker() -> None:
    mock_consumer = AsyncMock()
    mock_alerts = AsyncMock()

    # Setup async return value for poll_messages
    mock_consumer.poll_messages.return_value = [
        {"offset": 101, "severity": "INFO", "event": "User logged in"},
        {"offset": 102, "severity": "CRITICAL", "event": "DB Connection dropped"},
        {"offset": 103, "severity": "WARN", "event": "High memory consumption"},
    ]

    # Execute async function under test
    count = await process_queue_batch(mock_consumer, mock_alerts)

    print(f"Total processed events: {count}")
    print(f"Consumer ack() awaited count: {mock_consumer.ack.await_count}")
    print(f"Alert dispatch_pager() awaited count: {mock_alerts.dispatch_pager.await_count}")

    # Assert async interactions
    mock_consumer.poll_messages.assert_awaited_once_with(max_items=10)
    mock_alerts.dispatch_pager.assert_awaited_once_with("DB Connection dropped")
    assert mock_consumer.ack.await_count == 3
    print("All async assertions verified!")

if __name__ == "__main__":
    asyncio.run(test_async_worker())
```

Run:

```
uv run python async_worker_test.py
```

Output:

```
Total processed events: 3
Consumer ack() awaited count: 3
Alert dispatch_pager() awaited count: 1
All async assertions verified!
```

Pitfalls

Pitfall 1: Patching the Definition Site Instead of the Lookup Site

If `service.py` contains `from time import time`, patching `time.time` has no effect because `service.py` holds a direct pointer to the function bound during import:

```python
# service.py
from time import time

def get_current_epoch() -> float:
    return time()

# test_service.py (THE TRAP):
from unittest.mock import patch

# WRONG: time was imported into service's namespace!
with patch("time.time", return_value=1700000000.0):
    # get_current_epoch() still calls the unpatched original function!
    pass

# THE FIX:
# Patch the symbol where service.py looks it up!
with patch("service.time", return_value=1700000000.0):
    assert get_current_epoch() == 1700000000.0
```

Pitfall 2: Misspelling Assertion Methods (`assert_called_once_with` typo)

Because standard `Mock` instances dynamically create any attribute on demand, misspelling an assertion method name (e.g. `assert_called_once_withh`) does not raise an error—it merely creates a new child mock, silently passing the test!

```
# THE TRAP:
from unittest.mock import Mock

mock_obj = Mock()
# Did you notice the extra 'h'?
mock_obj.assert_called_once_withh("bad_arg")  # SILENT PASS! Returns a new Mock object!

# THE FIX:
# Use pytest with unittest.mock or run with PYTHONWARNINGS to catch non-existent mock assert
# or use autospec=True.
```

Exercises

1. Create a unit test using `@patch("os.environ.get")` to verify that a configuration loader returns default fallback values when an environment variable is missing.
2. Build a test for a database transaction context manager (`with db.transaction(): ...`) using `MagicMock` to ensure `commit()` is called on clean exit and `rollback()` is called on exceptions.
3. Test a file download function by patching `urllib.request.urlopen` with a mock context manager that streams 1 KB chunk buffers.
4. Using `side_effect`, implement a mock for a rate-limited API that returns HTTP 429 twice before returning HTTP 200 with valid JSON.
5. Convert an existing unit test that uses vanilla `Mock` to `create_autospec(TargetClass, instance=True)` and observe which tests break due to out-of-date method signatures.

Further reading

- Python Documentation: `unittest.mock` — *Mock object library.*
- Harry Percival & Bob Gregory: *Architecture Patterns with Python* (Chapter 3: Coupling and Abstractions; Mocks vs Adapters).
- Martin Fowler: *Mocks Aren't Stubs* (Fowler's taxonomy of test doubles: Dummy, Fake, Stubs, Mocks).
- Brett Slatkin: *Effective Python* (Item 80: Consider `unittest.mock` for Testing Code with Dependencies).

Binary Data and Network Protocols (struct)

Binary Data and Network Protocols (`struct`)

After reading this chapter, you will master low-level binary serialization and deserialization in Python using the `struct` module, pack primitive Python types into dense C-compatible byte arrays, control byte endianness and memory alignment, unpack fixed-width binary protocols and telemetry streams, perform zero-copy buffer modifications using `struct.pack_into` and `struct.unpack_from`, and engineer resilient wire framing for high-throughput network applications.

Mental model

Higher-level formats like JSON, XML, and YAML serialize data as UTF-8 human-readable text. While flexible, textual formats incur substantial overhead: a 64-bit integer like `18446744073709551615` takes 20 ASCII bytes plus delimiter bytes, parsing requires string scanning and memory allocation, and floating-point conversions lose IEEE precision.

In contrast, the `struct` module translates directly between Python values and raw binary representations matching C structs:

```
Textual (JSON):
  {"id": 42, "temp": 98.6}     25+ ASCII bytes, variable width, slow parsing

Binary (struct):
  struct.pack("!Id", 42, 98.6)

    4 bytes: uint32 (42)    8 bytes: IEEE double-float (98.6)

     Exactly 12 bytes! Constant-time O(1) packing/unpacking, zero allocations.
```

Endianness & Byte Order Modifiers

When exchanging binary packets across networks or persisting files to disk, you must explicitly specify byte order:

Modifier	Meaning	Size & Alignment	Use Case
`@`	Native byte order	Native C struct alignment & padding	Internal C interop (default)
`=`	Native byte order	Standard (packed, no padding)	System-specific serialization

Modifier	Meaning	Size & Alignment	Use Case
<	Little-endian	Standard (packed, no padding)	x86/ARM disk formats, NVMe
>	Big-endian	Standard (packed, no padding)	Network protocols
!	Network byte order	Standard (packed, big-endian)	**RFC standard for all TCP/UDP headers**

Minimal example

Save as `struct_overview.py`:

```
# struct_overview.py
import struct

def main() -> None:
    # Format String:
    #   '!' = Network byte order (big-endian)
    #   'H' = unsigned 16-bit short (2 bytes)
    #   'I' = unsigned 32-bit int   (4 bytes)
    #   'f' = single-precision float (4 bytes)
    #   '6s' = 6-byte string        (6 bytes)
    FMT = "!HIf6s"

    expected_size = struct.calcsize(FMT)
    print(f"Calculated struct size: {expected_size} bytes\n")

    # 1. Pack Python values into raw binary bytes
    packet_type = 101
    sequence_id = 98452
    latency_ms = 4.25
    device_tag = b"NODE01"

    binary_payload = struct.pack(FMT, packet_type, sequence_id, latency_ms, device_tag)
    print(f"Packed bytes ({len(binary_payload)} bytes): {binary_payload.hex()}")

    # 2. Unpack raw bytes back into Python primitives
    unpacked_type, unpacked_seq, unpacked_lat, unpacked_tag = struct.unpack(FMT, binary_payl
    print("\nUnpacked fields:")
    print(f"  Packet Type : {unpacked_type}")
    print(f"  Sequence ID : {unpacked_seq}")
    print(f"  Latency (ms): {unpacked_lat:.2f}")
```

```
    print(f"  Device Tag  : {unpacked_tag.decode()}")

if __name__ == "__main__":
    main()
```

Run via `uv run python struct_overview.py`:

```
Calculated struct size: 16 bytes

Packed bytes (16 bytes): 00650001809440880000 4e4f44453031

Unpacked fields:
  Packet Type : 101
  Sequence ID : 98452
  Latency (ms): 4.25
  Device Tag  : NODE01
```

Worked examples

Case 1: Custom Binary RPC Wire Framing with Checksum Validation

In custom network protocols, every message starts with a fixed-size header specifying magic bytes, sequence IDs, flags, and the length of the following dynamic payload:

```
# binary_rpc_framer.py
import struct
import zlib
from typing import NamedTuple

class Packet(NamedTuple):
    sequence: int
    flags: int
    payload: bytes

# Header: 4-byte Magic, 4-byte Seq, 4-byte Payload Length, 2-byte Flags, 4-byte CRC32
# Total header length: 18 bytes
HEADER_FORMAT = "!4sIIHI"
HEADER_SIZE = struct.calcsize(HEADER_FORMAT)
MAGIC_BYTES = b"RPC1"

def encode_packet(sequence: int, flags: int, payload: bytes) -> bytes:
    """Encodes a payload into a binary wire packet with CRC32 integrity check."""
    crc = zlib.crc32(payload)
```

```
    header = struct.pack(HEADER_FORMAT, MAGIC_BYTES, sequence, len(payload), flags, crc)
    return header + payload

def decode_packet(raw_bytes: bytes) -> Packet:
    """Decodes and validates a binary wire packet."""
    if len(raw_bytes) < HEADER_SIZE:
        raise ValueError(f"Incomplete packet: expected at least {HEADER_SIZE} bytes")

    magic, sequence, length, flags, expected_crc = struct.unpack(
        HEADER_FORMAT, raw_bytes[:HEADER_SIZE]
    )

    if magic != MAGIC_BYTES:
        raise ValueError(f"Protocol violation: invalid magic bytes {magic!r}")

    payload = raw_bytes[HEADER_SIZE : HEADER_SIZE + length]
    if len(payload) != length:
        raise ValueError(f"Truncated payload: expected {length} bytes, got {len(payload)}")

    actual_crc = zlib.crc32(payload)
    if actual_crc != expected_crc:
        raise ValueError(f"Corrupted packet! Checksum mismatch: 0x{actual_crc:08X} != 0x{exp

    return Packet(sequence=sequence, flags=flags, payload=payload)

def main() -> None:
    # 1. Encode message
    msg = b'{"command": "ACQUIRE_LOCK", "resource": "db_writer"}'
    wire_data = encode_packet(sequence=4201, flags=0x0001, payload=msg)
    print(f"Encoded wire packet size: {len(wire_data)} bytes (Payload: {len(msg)} bytes)")

    # 2. Decode valid packet
    pkt = decode_packet(wire_data)
    print(f"Decoded successfully: seq={pkt.sequence}, flags=0x{pkt.flags:04X}, body={pkt.pay

    # 3. Simulate bit-flip corruption in transmission
    corrupted_data = bytearray(wire_data)
    corrupted_data[-1] ^= 0xFF  # Flip bits in last payload byte
    try:
        decode_packet(bytes(corrupted_data))
    except ValueError as err:
        print(f"\nCaught transmission corruption:\n  {err}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python binary_rpc_framer.py
```

Output:

```
Encoded wire packet size: 70 bytes (Payload: 52 bytes)
Decoded successfully: seq=4201, flags=0x0001, body={"command": "ACQUIRE_LOCK", "resource": "db_

Caught transmission corruption:
  Corrupted packet! Checksum mismatch: 0x... != 0x...
```

Case 2: High-Throughput Stream Parsing with `struct.iter_unpack`

When parsing continuous streams of sensor records or financial ticks, slicing strings creates millions of ephemeral Python objects that overwhelm the garbage collector. `struct.iter_unpack` iterates directly through a continuous memory buffer:

```python
# telemetry_stream_parser.py
import struct
import time

# Record Format:
#   '!' = Big-endian
#   'I' = uint32 timestamp (seconds)
#   'H' = uint16 sensor_id
#   'f' = float32 reading
RECORD_FMT = "!IHf"
RECORD_SIZE = struct.calcsize(RECORD_FMT)

def generate_telemetry_batch(count: int = 4) -> bytes:
    """Simulates a raw binary sensor telemetry stream received over UDP."""
    base_time = 1773000000
    buffer = bytearray()
    for i in range(count):
        buffer.extend(struct.pack(RECORD_FMT, base_time + i, 100 + i, 23.5 + (i * 0.4)))
    return bytes(buffer)

def main() -> None:
    raw_telemetry = generate_telemetry_batch(4)
    print(f"Stream size: {len(raw_telemetry)} bytes ({len(raw_telemetry) // RECORD_SIZE} rec

    print("Streaming records via iter_unpack (zero intermediate slices):")
    for ts, sensor_id, temp in struct.iter_unpack(RECORD_FMT, raw_telemetry):
        print(f"  Timestamp: {ts} | Sensor: {sensor_id} | Temperature: {temp:5.2f} C")
```

```
if __name__ == "__main__":
    main()
```

Run:

```
uv run python telemetry_stream_parser.py
```

Output:

```
Stream size: 40 bytes (4 records)

Streaming records via iter_unpack (zero intermediate slices):
  Timestamp: 1773000000 | Sensor: 100 | Temperature: 23.50 C
  Timestamp: 1773000001 | Sensor: 101 | Temperature: 23.90 C
  Timestamp: 1773000002 | Sensor: 102 | Temperature: 24.30 C
  Timestamp: 1773000003 | Sensor: 103 | Temperature: 24.70 C
```

Case 3: Zero-Copy In-Place Buffer Updates with `pack_into` and `unpack_from`

When writing packet routers or video transcoders, allocating new `bytes` objects on every modification degrades throughput. `struct.pack_into` and `struct.unpack_from` modify pre-allocated mutable buffers (`bytearray` or `memoryview`) directly:

```
# zero_copy_packet_rewriter.py
import struct

# Simulated IPv4 Header Fragment:
# Offset 0: Version & IHL (1B)
# Offset 8: TTL (1B)
# Offset 12: Source IP (4B)
# Offset 16: Destination IP (4B)

def main() -> None:
    # Allocate a mutable 20-byte packet buffer
    packet_buf = bytearray(20)

    # 1. Write headers directly at specific offsets
    struct.pack_into("!B", packet_buf, 0, 0x45)         # IPv4, 5 words
    struct.pack_into("!B", packet_buf, 8, 64)           # TTL = 64
    struct.pack_into("!4B", packet_buf, 12, 192, 168, 1, 10)  # Src: 192.168.1.10
    struct.pack_into("!4B", packet_buf, 16, 10, 0, 0, 1)      # Dst: 10.0.0.1

    print(f"Initial Packet Hex: {packet_buf.hex()}")
```

```
    # 2. Read fields directly from offset without slicing
    ttl = struct.unpack_from("!B", packet_buf, 8)[0]
    src_ip = ".".join(str(b) for b in struct.unpack_from("!4B", packet_buf, 12))
    print(f"Read before NAT: TTL={ttl}, Src={src_ip}")

    # 3. Apply NAT translation: rewrite Source IP and decrement TTL in place!
    struct.pack_into("!B", packet_buf, 8, ttl - 1)
    struct.pack_into("!4B", packet_buf, 12, 172, 16, 0, 99)

    # 4. Verify in-place mutation
    new_ttl = struct.unpack_from("!B", packet_buf, 8)[0]
    new_src_ip = ".".join(str(b) for b in struct.unpack_from("!4B", packet_buf, 12))
    print(f"\nRead after NAT : TTL={new_ttl}, Src={new_src_ip}")
    print(f"Mutated Packet Hex: {packet_buf.hex()}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python zero_copy_packet_rewriter.py
```

Output:

```
Initial Packet Hex: 450000000000000040000000c0a8010a0a000001
Read before NAT: TTL=64, Src=192.168.1.10

Read after NAT : TTL=63, Src=172.16.0.99
Mutated Packet Hex: 45000000000000003f000000ac1000630a000001
```

Pitfalls

Pitfall 1: Relying on Native `@` Alignment for Network or Disk Formats

By default, omitting the byte order character or using `@` enables native C-compiler struct padding. On 64-bit x86/ARM systems, fields are padded with invisible filler bytes to align with 4- or 8-byte boundaries. If sent over the network, different architectures misinterpret the offsets!

```
# THE TRAP:
import struct

# Native format without explicit endianness prefix:
native_data = struct.pack("ci", b"A", 42)
print("Native size with C padding:", len(native_data))
```

```
# THE FIX:
# Always use '!' or '<' to enforce packed, non-padded representation:
standard_data = struct.pack("!ci", b"A", 42)
print("Standard packed size     :", len(standard_data))
```

Output:

```
Native size with C padding: 8
Standard packed size     : 5
```

Pitfall 2: String Length Mismatches in `s` Format Strings

In `struct` format strings, `8s` specifies a fixed-length 8-byte buffer. If the Python byte string is shorter than 8 bytes, `struct.pack` automatically pads it with null bytes (`\x00`). If longer, it silently **truncates** the string, discarding trailing data without error!

```
# THE TRAP:
import struct

long_tag = b"ENTERPRISE_SERVER"
truncated = struct.pack("!8s", long_tag)
print("Truncated bytes:", truncated)  # Silent data loss: only b'ENTERPRI'!
```

Output:

```
Truncated bytes: b'ENTERPRI'
```

The Fix: Always validate `len(byte_string) <= max_length` before calling `struct.pack`, or use variable-length prefixes like length-prefixed strings (`!H<len>s`).

Exercises

1. Write an encoder and decoder for a binary key-value protocol where keys are length-prefixed strings (`uint8` length followed by bytes) and values are length-prefixed blobs (`uint32` length followed by bytes).
2. Use `struct.unpack_from()` to parse the header of a WAV audio file or BMP image, extracting width, height, and bits-per-pixel metadata.
3. Benchmark the throughput of serializing 1,000,000 numeric tuples `(int, float, int)` using `json.dumps()` versus `struct.pack("!IfI", ...)`.

4. Implement a binary ring-buffer logger using `struct.pack_into` and a fixed-size `bytearray(65536)`.
5. Build a DNS response parser that unpacks DNS header flags (QR, Opcode, AA, TC, RD, RA, RCODE) using bitwise masks and `struct.unpack("!HHHHHH", header)`.

Further reading

- Python Documentation: `struct` — *Interpret bytes as packed binary data*.
- David Beazley: *Python Cookbook* (Recipe 6.11: Reading and Writing Binary Arrays of Structures).
- RFC 791: *Internet Protocol DARPA Internet Program Protocol Specification* (Binary Packet Structure).
- W. Richard Stevens: *TCP/IP Illustrated, Volume 1* (Network Byte Order and Framing).

Object Serialization and Persistent Storage (pickle, shelve)

Object Serialization and Persistent Storage (`pickle`, `shelve`)

After reading this chapter, you will master deep Python object graph serialization using `pickle`, understand the binary pickle protocol and opcode execution model, customize serialization lifecycles using `__getstate__` and `__setstate__` to exclude ephemeral runtime resources (locks, sockets, file descriptors), construct secure unpickling firewalls that block arbitrary remote code execution, and leverage `shelve` for zero-setup, persistent key-value caching on disk.

Mental model

Higher-level formats like JSON and TOML serialize only primitive data types (`dict`, `list`, `str`, `int`, `float`, `bool`, `None`). They cannot represent custom classes, cyclical object graphs (where an object references itself), datetime instances, or method bindings without manual encoders and decoders.

The `pickle` module converts arbitrary, deeply nested Python object graphs into a compact stream of byte opcodes:

```
Python Heap Memory (Arbitrary Object Graph)
  Node A    Node B    Node A (Cyclic Reference!)
  UserSession(id=42, roles={'admin'}, created=datetime)


                    pickle.dumps(obj)


Pickle Byte Stream (Opcode Protocol)
  \x80\x05\x95... (Interpreted by Python Pickle VM)


                    pickle.loads(bytes)


Reconstructed Heap Object Graph
  (Exact duplicate with preserved object identity & cycles)
```

Security Warning: Never Unpickle Untrusted Data

`pickle` is **not secure against maliciously crafted payloads**. The unpickling process executes instructions that can instantiate any Python class and invoke arbitrary callables (like `os.system`). Never deserialize pickled blobs received over untrusted networks without cryptographic signatures (`hmac`) or restricted unpickling guards.

Minimal example

Save as `pickle_overview.py`:

```
# pickle_overview.py
from datetime import datetime, timezone
import pickle

class ServiceNode:
    def __init__(self, node_id: str, host: str) -> None:
        self.node_id = node_id
        self.host = host
        self.registered_at = datetime.now(timezone.utc)
        self.neighbors: list["ServiceNode"] = []

    def __repr__(self) -> str:
        return f"ServiceNode({self.node_id}, host={self.host}, neighbors={len(self.neighbors

def main() -> None:
    # 1. Construct a cyclic graph: Node 1 <   Node 2
    node1 = ServiceNode("node-01", "10.0.0.1")
    node2 = ServiceNode("node-02", "10.0.0.2")
    node1.neighbors.append(node2)
    node2.neighbors.append(node1)

    print(f"Original: {node1}")
    print(f"Cyclic reference intact: {node1.neighbors[0].neighbors[0] is node1}")

    # 2. Serialize complete cyclic graph to bytes
    serialized_bytes = pickle.dumps(node1, protocol=pickle.HIGHEST_PROTOCOL)
    print(f"\nSerialized binary size: {len(serialized_bytes)} bytes")

    # 3. Deserialize back into independent object graph
    restored_node1 = pickle.loads(serialized_bytes)
    print(f"\nRestored: {restored_node1}")
    print(f"Restored cyclic reference intact: {restored_node1.neighbors[0].neighbors[0] is r
    print(f"Identity check with original: {restored_node1 is not node1}")
```

```
if __name__ == "__main__":
    main()
```

Run via `uv run python pickle_overview.py`:

```
Original: ServiceNode(node-01, host=10.0.0.1, neighbors=1)
Cyclic reference intact: True

Serialized binary size: 197 bytes

Restored: ServiceNode(node-01, host=10.0.0.1, neighbors=1)
Restored cyclic reference intact: True
Identity check with original: True
```

Worked examples

Case 1: Sandboxed Deserialization with `SafeUnpickler`

When accepting serialized blobs internally across microservices or worker processes, overriding `pickle.Unpickler.find_class` blocks arbitrary class execution by enforcing an explicit allowlist:

```
# safe_unpickler.py
import io
import pickle
from typing import Any

class SafeUnpickler(pickle.Unpickler):
    """Custom Unpickler that permits only explicitly allowlisted classes."""

    ALLOWED_CLASSES = {
        ("builtins", "dict"),
        ("builtins", "list"),
        ("builtins", "set"),
        ("builtins", "int"),
        ("builtins", "float"),
        ("builtins", "str"),
        ("builtins", "bool"),
    }

    def find_class(self, module: str, name: str) -> Any:
        if (module, name) in self.ALLOWED_CLASSES:
            return super().find_class(module, name)
        raise pickle.UnpicklingError(f"Security Alert: Disallowed class '{module}.{name}'")
```

```
def safe_loads(data: bytes) -> Any:
    return SafeUnpickler(io.BytesIO(data)).load()

# Malicious exploit class attempting remote command execution
class MaliciousPayload:
    def __reduce__(self):
        import os
        return (os.system, ("echo COMPROMISED",))

def main() -> None:
    # 1. Legitimate data succeeds
    safe_data = {"tenant": "acme-corp", "tier": "enterprise", "features": ["sso", "audit"]}
    safe_blob = pickle.dumps(safe_data)
    restored = safe_loads(safe_blob)
    print(f"Safe deserialization succeeded: {restored}")

    # 2. Malicious payload is blocked before execution
    exploit_blob = pickle.dumps(MaliciousPayload())
    try:
        safe_loads(exploit_blob)
    except pickle.UnpicklingError as err:
        print(f"\nExploit successfully neutralized:\n  {err}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python safe_unpickler.py
```

Output:

```
Safe deserialization succeeded: {'tenant': 'acme-corp', 'tier': 'enterprise', 'features': ['sso

Exploit successfully neutralized:
  Security Alert: Disallowed class 'posix.system'
```

Case 2: Persistent Disk State with `shelve`

The `shelve` module provides a persistent dictionary on disk backed by a lightweight DBM database. Keys must be strings, and values are automatically pickled and unpickled transparently:

```python
# persistent_shelve_store.py
import shelve
import tempfile
from pathlib import Path

def main() -> None:
    with tempfile.TemporaryDirectory() as tmpdir:
        db_path = str(Path(tmpdir) / "app_state.db")

        # 1. Open shelf and persist complex objects
        with shelve.open(db_path) as db:
            db["cluster_settings"] = {"env": "production", "max_workers": 16}
            db["node_inventory"] = ["node-alpha", "node-beta"]
            print("Persisted initial state to disk.")

        # 2. Re-open in a separate session and read back
        with shelve.open(db_path) as db:
            print("\nRestored from disk:")
            print(f"  Settings: {db['cluster_settings']}")
            print(f"  Nodes   : {db['node_inventory']}")

        # 3. In-place mutations require writeback=True!
        with shelve.open(db_path, writeback=True) as db:
            # Append directly to the stored list
            db["node_inventory"].append("node-gamma")

        # 4. Verify in-place update persisted
        with shelve.open(db_path) as db:
            print(f"\nNodes after writeback update: {db['node_inventory']}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python persistent_shelve_store.py
```

Output:

```
Persisted initial state to disk.

Restored from disk:
  Settings: {'env': 'production', 'max_workers': 16}
  Nodes   : ['node-alpha', 'node-beta']

Nodes after writeback update: ['node-alpha', 'node-beta', 'node-gamma']
```

Case 3: Excluding Transient Resources with `__getstate__` and `__setstate__`

Objects containing unpicklable OS handles (such as `threading.Lock`, open sockets, or database connections) crash `pickle.dumps()` unless you implement `__getstate__` to sanitize the serialized state and `__setstate__` to reconstruct transient fields:

```
# custom_pickle_state.py
import pickle
import threading

class StatefulSession:
    def __init__(self, session_id: str, username: str) -> None:
        self.session_id = session_id
        self.username = username
        self.login_attempts = 0
        # Ephemeral OS lock (cannot be pickled directly!)
        self._lock = threading.Lock()

    def record_attempt(self) -> None:
        with self._lock:
            self.login_attempts += 1

    def __getstate__(self) -> dict:
        """Custom state extraction: strips non-serializable OS lock."""
        state = self.__dict__.copy()
        del state["_lock"]
        return state

    def __setstate__(self, state: dict) -> None:
        """Custom state restoration: re-initializes fresh OS lock."""
        self.__dict__.update(state)
        self._lock = threading.Lock()

def main() -> None:
    session = StatefulSession("sess-4819", "alice")
    session.record_attempt()
    session.record_attempt()
    print(f"Pre-serialization state: {session.username} (attempts={session.login_attempts})"

    # Serialize object (succeeds without Lock pickling error!)
    serialized = pickle.dumps(session)

    # Deserialize in worker process
    restored = pickle.loads(serialized)
    print(f"Restored session state : {restored.username} (attempts={restored.login_attempts}

    # Verify restored lock operates normally
```

```
    restored.record_attempt()
    print(f"State after lock usage : attempts={restored.login_attempts}")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python custom_pickle_state.py
```

Output:

```
Pre-serialization state: alice (attempts=2)
Restored session state : alice (attempts=2)
State after lock usage : attempts=3
```

Pitfalls

Pitfall 1: Mutating Mutable Objects in `shelve` Without `writeback=True`

By default, accessing `db["key"]` unpickles a fresh copy of the object into RAM. Modifying that object in-place (e.g. `db["list"].append(x)`) modifies only the temporary copy in memory; `shelve` does NOT detect the change and fails to save it to disk!

```
# THE TRAP:
import shelve

with shelve.open("cache.db") as db:
    db["tags"] = ["prod"]
    db["tags"].append("v2")  # BUG: Modified in memory, NEVER written to disk!

with shelve.open("cache.db") as db:
    print(db["tags"])  # Prints ['prod'], 'v2' was lost!

# THE FIX:
# Option A: Re-assign explicitly
with shelve.open("cache.db") as db:
    tags = db["tags"]
    tags.append("v2")
    db["tags"] = tags  # Explicit assignment triggers disk write

# Option B: Open with writeback=True
with shelve.open("cache.db", writeback=True) as db:
    db["tags"].append("v3")  # Automatically synchronized on close
```

Pitfall 2: Protocol Incompatibility Across Python Versions

Pickle has multiple protocol versions (Protocol 0 through Protocol 5). When saving serialized data that must be read by older Python versions, passing `protocol=pickle.HIGHEST_PROTOCOL` (Protocol 5 in modern Python) causes older versions to fail with `ValueError: unsupported pickle protocol`. For backward compatibility, explicitly specify `protocol=4`.

Exercises

1. Build an atomic file-backed session store that serializes user session objects using `pickle.dump()` to a temporary file and atomically replaces the destination file using `os.replace()`.
2. Implement a `RestrictedUnpickler` that permits only a custom dataclass and standard collections, rejecting all other types.
3. Write a benchmarking script comparing the serialization and deserialization speeds of a 10,000-element dictionary across `json`, `pickle` (Protocol 5), and `marshal`.
4. Implement an LRU cache decorator backed by `shelve` that caches function outputs across terminal restarts.
5. Create a class that implements `__reduce__` to serialize an instance into an importable module path and a constructor argument tuple.

Further reading

- Python Documentation: `pickle` — *Python object serialization.*
- Python Documentation: `shelve` — *Python object persistence.*
- David Beazley: *Python Cookbook* (Recipe 5.21: Serializing Python Objects).
- PEP 574: *Pickle protocol 5 with out-of-band data buffers.*

functools.cache vs lru_cache

functools.cache vs lru_cache

Thesis

`functools.lru_cache` remembers recent calls up to `maxsize`. **`functools.cache`** is the same idea with **`maxsize=None`** — unbounded. Both make recursive Fibonacci trivial; only one can grow without bound and OOM a long-running process. This chapter times both on `fib(35)` and then fills an unbounded cache with 10 000 distinct keys.

Mental model

```
lru_cache(maxsize=128) → bounded; evicts least-recently used
cache()                → alias for lru_cache(maxsize=None); never evicts
```

Worked examples

Verified with **CPython 3.15.0**.

```
import functools, time

@functools.lru_cache(maxsize=128)
def fib_lru(n):
    return n if n < 2 else fib_lru(n-1)+fib_lru(n-2)

@functools.cache
def fib_cache(n):
    return n if n < 2 else fib_cache(n-1)+fib_cache(n-2)
```

Output:

```
3.15.0 (main, Oct  9 2026, 15:24:40) [Clang 22.1.3 ]
lru_cache fib(35)=9227465 in 0.01 ms  CacheInfo(hits=33, misses=36, maxsize=128, currsize=36)
cache fib(35)=9227465 in 0.01 ms  CacheInfo(hits=33, misses=36, maxsize=None, currsize=36)
cache after 10k distinct keys: CacheInfo(hits=0, misses=10000, maxsize=None, currsize=10000)
```

`fib(35)` is instant for both after memoization. The unbounded cache after 10 k distinct keys reports `currsize=10000` with `maxsize=None` — that growth is the point.

Trap

Caching functions of unbounded arguments in servers (`cache` on raw request URLs, SQL, or user ids). Prefer `lru_cache` with an explicit cap, or a proper TTL cache.

Boring rule

Default to `lru_cache(maxsize=…)` with a number you can defend. Use `cache` only for pure functions over a small finite domain.

Try this

1. Compare `cache_info().hits` under a realistic call mix.
2. Add `typed=True` and call with `1` vs `1.0`.
3. Read the docs for `cache_clear` in tests.

Sources

- functools — Python 3.15 docs

base64 Throughput on Python 3.15

base64 Throughput on Python 3.15

Thesis

`base64` in the stdlib is the boring default for wire and config encodings. Before reaching for a SIMD extension, measure what 3.15 already does on your CPU. This chapter times encode/decode of a 1 MiB buffer and notes when an external accelerator would matter.

Mental model

```
  bytes  b64encode   ASCII-ish bytes (~4/3 size)
        b64decode
```

Worked examples

Verified with **CPython 3.15.0** (Clang 22.1.3 build).

```
import base64, time, sys
data = b"x" * (1 << 20)  # 1 MiB
N = 50
t0 = time.perf_counter()
for _ in range(N):
    base64.b64encode(data)
t1 = time.perf_counter()
enc = base64.b64encode(data)
t2 = time.perf_counter()
for _ in range(N):
    base64.b64decode(enc)
t3 = time.perf_counter()
print(sys.version)
print(f"b64encode 1MiB x{N}: {(t1-t0)/N*1e3:.3f} ms/op")
print(f"b64decode 1MiB x{N}: {(t3-t2)/N*1e3:.3f} ms/op")
```

Output:

```
3.15.0 (main, Oct  9 2026, 15:24:40) [Clang 22.1.3 ]
b64encode 1MiB x50: 0.430 ms/op  throughput=2435.7 MB/s
b64decode 1MiB x50: 0.521 ms/op  throughput=2013.1 MB/s
```

```
encoded len 1398104
simdutf not installed: ModuleNotFoundError
pybase64 not installed: ModuleNotFoundError
```

Roughly **2.4 GB/s encode** and **2.0 GB/s decode** on this Xeon guest for the stdlib path. `simdutf` / `pybase64` were not installed (and we are not adding system packages for a chapter); use them only when profiles show base64 in the top hotspots.

Trap

Premature SIMD dependencies for occasional token encoding. Prefer stdlib until a profile says otherwise — especially in security-sensitive decode paths where an extra crate is another audit surface.

Boring rule

Use stdlib `base64` by default. Optimize only with measurements from *your* workload.

Try this

1. Compare `b64encode` vs `urlsafe_b64encode` cost.
2. Stream large files in chunks instead of one giant buffer.
3. If you later add `pybase64`, A/B it with the same script.

Sources

- base64 — Python 3.15 docs

Parsing TOML with tomllib

Parsing TOML with tomllib

Thesis

Since **Python 3.11**, the standard library ships **`tomllib`** — a TOML parser aligned with the TOML spec. Prefer it over third-party parsers for reading config; writing still needs another library.

Mental model

```
open(..., "rb")  →  tomllib.load(fp)  →  dict / list / str / int / float / bool / datetime
tomllib.loads(text_as_str)            →  same
```

Binary mode is required for `load` because TOML is defined over UTF-8 bytes.

Worked example

```
# tomllib_demo.py
import tomllib
from pathlib import Path

sample = b"""
[project]
name = "demo"
version = "1.2.3"
authors = [{ name = "Ada" }]

[tool.ruff]
line-length = 100
"""
Path("/tmp/demo.toml").write_bytes(sample)

with open("/tmp/demo.toml", "rb") as f:
    data = tomllib.load(f)

print(data["project"]["name"], data["project"]["version"])
print(data["tool"]["ruff"]["line-length"])
print(tomllib.loads(sample.decode()) == data)
```

```
{'project': {'name': 'demo', 'version': '1.2.3'}, 'tool': {'demo': {'enabled': True, 'ports'
version 1.2.3
ports [8080, 8443]
Traceback (most recent call last):
  File "<stdin>", line 15, in <module>
  File "/usr/lib/python3.13/tomllib/_parser.py", line 74, in loads
    src = s.replace("\r\n", "\n")
TypeError: a bytes-like object is required, not 'str'
```

Trap

Opening TOML in text mode (`"r"`). `tomllib.load` wants a binary file. Also: `tomllib` is **read-only** — do not expect `dump`; use `tomli-w` / `tomlkit` when you must emit.

Boring rule

For Python 3.11 read paths, `import tomllib` and open `"rb"`. Pin a writer separately only if the program rewrites TOML.

Try this

1. Parse a real `pyproject.toml` and print `[project.dependencies]`.
2. Feed invalid TOML and catch `tomllib.TOMLDecodeError`.
3. Compare parse results with the `toml` / `tomli` packages on the same file.

Sources

- PEP 680 — `tomllib`
- Python docs: tomllib
- TOML v1.0.0

TLS Trust Stores in Python: ssl, certifi, pip and the 2026 emSign Distrust

TLS Trust Stores in Python: ssl, certifi, pip and the 2026 emSign Distrust

After this page you will be able to list every copy of the certificate authority (CA) roots a Python environment may trust, explain why updating the operating system's certificates does not update the bundle `requests` uses, and remove a distrusted root from each copy.

Thesis

A Python program has no single trust store. One environment can hold three copies of the roots: the operating system's store that the standard library `ssl` module loads by default, the `certifi` package that `requests`, `httpx` and many SDKs use, and `pip`'s own vendored copy, `pip._vendor.certifi`.

When a CA is distrusted, as eMudhra/emSign was in October 2026, each copy has to be updated separately. Updating the OS package does nothing for code that uses an old `certifi`, and the reverse is also true.

Trust Store Source	Primary Consumers	Default Location / Mechanism	How to Update
OS store, loaded by `ssl`	`ssl.create_default_context()`, so `urllib.request`, `http.client` and plain `ssl` sockets	Whatever `ssl.get_default_verify_paths()` reports; on Linux usually under `/etc/ssl`	The OS package manager (`ca-certificates` on Debian/Ubuntu)
`certifi`	`requests`, `httpx`, many SDKs	`certifi.where()`, a `cacert.pem` in site-packages	Upgrade `certifi` in each environment
`pip._vendor.certifi`	`pip` itself	`pip/_vendor/certifi/cacert.pem` in site-packages	Upgrade `pip` (distribution builds may point it at the OS bundle)

The Three Trust Copies

When Python establishes a secure TLS connection, the underlying engine must validate the server's certificate chain against a set of trusted root CAs. Which certificates are trusted depends entirely on the library initiating the connection.

1. **The `ssl` module** loads OpenSSL's default locations, which `ssl.get_default_verify_paths()` reports. On Linux that is usually `/etc/ssl/certs` or a file such as `/usr/lib/ssl/cert.pem`. The `SSL_CERT_FILE` and `SSL_CERT_DIR` environment variables override these defaults.
2. `certifi` is Mozilla's root bundle packaged as a Python module. `requests` uses `certifi.where()` unless you pass `verify=/path/to/pem` or set `REQUESTS_CA_BUNDLE`. `httpx` also defaults to `certifi`. Plain `urllib3` 2.x only uses `certifi` if you pass `ca_certs=certifi.where()` yourself.
3. `pip._vendor.certifi` is pip's own copy, so pip can verify PyPI before anything else is installed. Recent pip versions can also use the OS store through the `truststore` package. Some distributions (Debian/Ubuntu `python3-pip`) patch the vendored module to return the OS bundle `/etc/ssl/certs/ca-certificates.crt` instead.

Updating one copy leaves the others untouched. A patched host can still run virtual environments whose pinned `certifi` trusts the distrusted root.

The October 2026 Distrust in Brief

In October 2026 the Python security announcement on Discourse told users to remove trust in eMudhra/emSign. For Python, it comes down to these releases:

- **`certifi` Advisory (GHSA-6q48-6c7w-9gw2)**: the advisory lists `certifi` releases from `2019.03.09` up to (not including) `2026.10.18` as containing affected eMudhra roots, and says to upgrade to `2026.10.19` or later. `2026.10.19` removes "emSign ECC Root CA - G3". In the lab below the bundle goes from 121 certificates (2026.07.22) to 120 (2026.10.19). `2026.10.23`, the latest when this was written, has 118.
- **`pip` Updates**: upgrade pip to `26.2.2` or later for an updated vendored copy. pip `26.3`, used in the lab, ships `certifi 2026.10.23`. For uv, upgrade to `0.13.6` or later.
- **CPython**: point releases `3.15.2`, `3.14.10`, `3.13.18`, `3.12.16` and `3.11.18` add a denylist for the affected CA keys to the `ssl` module, so those keys are not trusted even if the OS store has not been updated yet.

The Audit Script Walk-Through

`trust_report.py` finds each copy and reports whether a given root is still in it. It has no third-party dependencies: it splits PEM files into certificates with `base64`, then searches each certificate's DER bytes for the CA's name.

> **Note**
>
> **Caveat.** The name is stored as a plain string, so a byte search finds it in the subject, and also in the issuer field of any certificate that CA issued. Root bundles don't contain such certificates, so for a bundle this is a reliable presence check. It is not a substitute for comparing fingerprints or SPKI hashes. For a `capath` directory the script only scans `*.pem` files.

```
"""trust_report.py: show every CA bundle this interpreter might trust,
and whether a given root CA is still in each one."""
import base64
import importlib
import os
import ssl
import sys

TARGET = sys.argv[1] if len(sys.argv) > 1 else "emSign ECC Root CA - G3"


def pem_blocks(path):
    """Yield the DER bytes of every certificate in a PEM file."""
    with open(path, encoding="ascii", errors="replace") as f:
        b64 = None
        for line in f:
            line = line.strip()
            if line == "-----BEGIN CERTIFICATE-----":
                b64 = []
            elif line == "-----END CERTIFICATE-----" and b64 is not None:
                yield base64.b64decode("".join(b64))
                b64 = None
            elif b64 is not None:
                b64.append(line)


def scan(paths):
    """Count certificates and those whose DER contains TARGET. The CA name
    is stored as a plain string in the subject, so a byte search finds it
    (it also matches certificates *issued by* that CA, which a root bundle
    should not contain)."""
    total = hits = 0
    needle = TARGET.encode()
    for p in paths:
        for der in pem_blocks(p):
            total += 1
            hits += needle in der
    return total, hits


def report(name, paths, version=None):
    paths = [p for p in paths if p and os.path.isfile(p)]
    if not paths:
        print(f"{name:<24} (not present)")
        return
    total, hits = scan(paths)
    ver = f" v{version}" if version else ""
```

```
    state = "PRESENT" if hits else "absent"
    where = paths[0] if len(paths) == 1 else f"{os.path.dirname(paths[0])}/ ({len(paths)} fi
    print(f"{name:<24} {total:>4} certs{ver}  {TARGET!r}: {state}")
    print(f"{'':<24} {os.path.realpath(where) if len(paths) == 1 else where}")


print(f"python {sys.version.split()[0]}  {ssl.OPENSSL_VERSION}")
vp = ssl.get_default_verify_paths()
print(f"ssl default cafile: {vp.cafile}")
print(f"ssl default capath: {vp.capath}")

for mod_name, label in [("certifi", "certifi"), ("pip._vendor.certifi", "pip's vendored cert
    try:
        mod = importlib.import_module(mod_name)
    except ImportError:
        print(f"{label:<24} (not importable)")
        continue
    report(label, [mod.where()], getattr(mod, "__version__", None))

if vp.cafile:
    report("ssl default cafile", [vp.cafile])
elif vp.capath and os.path.isdir(vp.capath):
    files = sorted(os.path.join(vp.capath, f) for f in os.listdir(vp.capath) if f.endswith("
    report("ssl default capath", files)
```

Running the Audit Script

The script was run in three environments on one machine. Paths in the output are shortened to `~/certlab`.

Environment 1: Legacy Virtual Environment (un-patched `certifi`)

Executing the audit script inside an un-patched virtual environment configured with `certifi==2026.7.22`:

```
old/bin/python trust_report.py
```

The command above produces the following output:

```
python 3.15.0  OpenSSL 3.5.9 29 Sep 2026
ssl default cafile: None
ssl default capath: /etc/ssl/certs
certifi                   121 certs v2026.07.22  'emSign ECC Root CA - G3': PRESENT
                         ~/certlab/old/lib/python3.15/site-packages/certifi/cacert.pem
pip's vendored certifi   (not importable)
```

```
ssl default capath        150 certs  'emSign ECC Root CA - G3': PRESENT
                         /etc/ssl/certs/ (150 files)
```

This uv-managed CPython has no default `cafile`, only the `capath /etc/ssl/certs`. `certifi` 2026.07.22 has 121 certificates and still contains the root. pip is not installed in this venv, so there is no vendored copy. The OS store has the root too: this machine's `ca-certificates` package had not been updated yet.

Environment 2: Modern Virtual Environment (upgraded `certifi` and `pip`)

Executing the audit script inside an updated environment with `certifi>=2026.10.19` and `pip 26.3`:

```
new/bin/python trust_report.py
```

The command above produces the following output:

```
python 3.15.0  OpenSSL 3.5.9 29 Sep 2026
ssl default cafile: None
ssl default capath: /etc/ssl/certs
certifi                   120 certs v2026.10.19  'emSign ECC Root CA - G3': absent
                         ~/certlab/new/lib/python3.15/site-packages/certifi/cacert.pem
pip's vendored certifi    118 certs v2026.10.23  'emSign ECC Root CA - G3': absent
                         ~/certlab/new/lib/python3.15/site-packages/pip/_vendor/certifi/cace
ssl default capath        150 certs  'emSign ECC Root CA - G3': PRESENT
                         /etc/ssl/certs/ (150 files)
```

`certifi` 2026.10.19 has 120 certificates and the root is gone. pip 26.3's vendored copy (2026.10.23, 118 certificates) doesn't have it either. The OS store line is unchanged, `PRESENT`, because nothing in a venv can change it.

Environment 3: System Interpreter (Debian/Ubuntu patched `pip`)

Executing the audit script via the Debian system interpreter:

```
python3 trust_report.py
```

The command above produces the following output:

```
python 3.13.5  OpenSSL 3.5.7 9 Jun 2026
ssl default cafile: /usr/lib/ssl/cert.pem
ssl default capath: /usr/lib/ssl/certs
certifi                   121 certs v2026.07.22  'emSign ECC Root CA - G3': PRESENT
                         /usr/local/lib/python3.13/dist-packages/certifi/cacert.pem
pip's vendored certifi    150 certs v2025.01.31  'emSign ECC Root CA - G3': PRESENT
                         /etc/ssl/certs/ca-certificates.crt
```

```
ssl default cafile        150 certs  'emSign ECC Root CA - G3': PRESENT
                         /etc/ssl/certs/ca-certificates.crt
```

Debian's Python 3.13.5 has a default `cafile`, `/usr/lib/ssl/cert.pem`, a symlink to `/etc/ssl/certs/ca-certificates.crt`. Debian's pip is patched: its vendored module says version 2025.01.31, yet `where()` returns the same OS file with 150 certificates. A separate `certifi` 2026.07.22 sits in `/usr/local/lib/python3.13/dist-packages`, installed system-wide outside the package manager, and it still has the root. Updating `ca-certificates` would fix two of these three lines; that `certifi` needs its own upgrade.

Remediation Strategies

Fix each copy in turn.

1. Upgrade Application Dependencies (`certifi`)

To see the fix in a fresh environment, first create a venv with the old release:

```
uv venv demo && uv pip install -p demo/bin/python certifi==2026.7.22
```

The command above produces the following output:

```
Using CPython 3.15.0
Creating virtual environment at: demo
Activate with: source demo/bin/activate
Using Python 3.15.0 environment at: demo
Resolved 1 package in 2ms
Installed 1 package in 1ms
 + certifi==2026.7.22
```

To remediate, upgrade `certifi` to `2026.10.19` or higher:

```
uv pip install -p demo/bin/python -U 'certifi>=2026.10.19'
```

The command above produces the following output:

```
Using Python 3.15.0 environment at: demo
Resolved 1 package in 94ms
Prepared 1 package in 0.57ms
Uninstalled 1 package in 0.47ms
Installed 1 package in 1ms
 - certifi==2026.7.22
 + certifi==2026.10.23
```

uv resolved `>=2026.10.19` to the newest release, 2026.10.23. Run the report again:

```
demo/bin/python trust_report.py
```

The command above produces the following output:

```
python 3.15.0  OpenSSL 3.5.9 29 Sep 2026
ssl default cafile: None
ssl default capath: /etc/ssl/certs
certifi                   118 certs v2026.10.23  'emSign ECC Root CA - G3': absent
                         ~/certlab/demo/lib/python3.15/site-packages/certifi/cacert.pem
pip's vendored certifi   (not importable)
ssl default capath        150 certs  'emSign ECC Root CA - G3': PRESENT
                         /etc/ssl/certs/ (150 files)
```

2. Upgrade Package Installer (`pip` and `uv`)

Upgrade pip in every environment that has it (to 26.2.2 or later):

```
python -m pip install --upgrade pip
```

> **Note**
>
> **Not run here.** The test machine has uv 0.13.1. uv carries its own root certificates by default, so upgrade it to `0.13.6` or later (for example with `uv self update` if you installed it with the standalone installer, or through your package manager). That upgrade was not run on this machine.

3. Upgrade Operating System Trust Store

On the test machine the OS store (`/etc/ssl/certs`, 150 certificates) still contains the root, because the distribution's `ca-certificates` package had not been updated there. Fixing that needs the OS package manager and root.

> **Note**
>
> **Not run here** (it needs root). On Debian/Ubuntu:
>
> ```
> sudo apt update && sudo apt install --only-upgrade ca-certificates
> sudo update-ca-certificates
> ```
>
> Once the distribution ships a `ca-certificates` without the root, this rebuilds `/etc/ssl/certs` without it.

4. Upgrade CPython Runtime

A CPython release with the `ssl` denylist covers the affected keys for anything that verifies through the `ssl` module, even with an old OS store.

> **Note**
>
> **Not run here.** The interpreters evaluated in this lab are versions `3.15.0` and `3.13.5`. Upgrading to CPython point releases `3.15.2`, `3.14.10`, `3.13.18`, `3.12.16`, or `3.11.18` adds the denylist. This was not tested here.

Pinning, Lockfiles, and Container Environments

Lockfiles and pinned requirements (`requirements.txt`, `poetry.lock`, `uv.lock`) pin exact versions. If one pins `certifi==2026.7.22`, every `pip install -r` or `uv sync` installs the old bundle again until you regenerate the lock (`uv lock --upgrade-package certifi`, for example).

Dependabot or Renovate can open that upgrade for you. A minimal Dependabot version-update config looks like this (security updates are switched on separately in the repository settings):

```yaml
# Example Dependabot configuration update
version: 2
updates:
  - package-ecosystem: "pip"
    directory: "/"
    schedule:
      interval: "daily"
```

Container images bake both the OS bundle and the installed `certifi` in at build time. Updating the certificates on the host or Kubernetes node doesn't change them. Rebuild the images with updated `ca-certificates` and dependencies, then redeploy.

Single Source of Truth: `truststore`

If you want one source of truth, the OS store, an application can use the `truststore` package. It verifies certificates with the platform's own APIs on Windows and macOS, and with the system's OpenSSL store on Linux. Not run here:

```python
import truststore

# Inject truststore into standard ssl contexts
truststore.inject_into_ssl()

# Subsequent HTTP client calls use OS trust settings
import urllib.request
```

```
response = urllib.request.urlopen("https://example.com")
```

Then OS updates are the only thing to keep current, and an old `certifi` stops mattering for that application.

Common Mistakes

- **Updating only the OS store.** Code using `requests` or `httpx` in a venv keeps its old `certifi`.
- **Updating only `certifi`.** `urllib.request`, plain `ssl` sockets and pip still use their own copies.
- **Assuming `verify=True` means the OS store.** In `requests` it means `certifi` (or `REQUESTS_CA_BUNDLE`), not `/etc/ssl`.
- **Disabling verification to "fix" errors.** `verify=False` or `ssl._create_unverified_context()` turns off every check, not just the one that failed.
- **A stale `SSL_CERT_FILE`.** It overrides OpenSSL's defaults for the `ssl` module (and `httpx` honours it too). If it points at an old bundle copied somewhere, updates to the OS store never reach the program. For `requests`, the same mistake is a stale `REQUESTS_CA_BUNDLE`.

Exercises

1. Run `trust_report.py` with every interpreter and venv you deploy, and list each copy that still says `PRESENT`.
2. Write a pytest test that fails if `certifi.where()` still contains "emSign ECC Root CA - G3". Reuse `pem_blocks` from the script.
3. Point `SSL_CERT_FILE` at an empty file and run the report again. What does `ssl default cafile` show, and what happens to `urllib.request.urlopen("https://pypi.org")`?
4. Pass a different CA name as the first argument (any root you know is in your OS store) and compare the counts.

Key takeaways

- Python TLS trust is distributed across three main locations: system default paths via `ssl`, standalone bundled PEM files via `certifi`, and isolated installer files via `pip._vendor.certifi`.
- Upgrading the OS CA package does not fix applications that use `certifi`, and upgrading `certifi` does not fix the OS store.
- For the October 2026 eMudhra/emSign distrust: `certifi>=2026.10.19`, `pip>=26.2.2`, uv `0.13.6` or later, an updated `ca-certificates`, and a CPython point release with the `ssl` denylist.
- Containerized environments and locked dependencies preserve stale CA bundles until explicitly rebuilt or re-locked.

Sources

- Python Discourse: eMudhra/emSign CA distrust announcement
- Certifi Security Advisory GHSA-6q48-6c7w-9gw2
- Python ssl Documentation: get_default_verify_paths
- PyPI certifi Package
- Truststore Documentation
- uv Package Manager Documentation

Typing and Testing

Modern Type Hints and Static Analysis

Modern Type Hints and Static Analysis

After reading this chapter, you will master Python's modern static type system, implement generic types with `typing.Generic`, define structural interfaces with `typing.Protocol` (duck typing formalized), leverage `Literal`, `Union` (`|`), and `Self`, and configure strict static analysis with `mypy` and `pyright`.

Mental model

Python type hints (PEP 484, PEP 604, PEP 695) do not alter runtime execution or impose execution overhead. They provide formal machine-readable contracts analyzed by static type checkers before code is deployed:

```
Development & CI Workflow:
  Python Source Code (with type hints)

              1. Static Type Checker (mypy / pyright)    Reports bugs, mismatches, None hazards

              2. CPython Runtime VM              Erases types, executes at full speed
```

Nominal Subtyping vs Structural Subtyping (`Protocol`)

- **Nominal Subtyping (`class Sub(Parent)`)**: Requires an explicit inheritance declaration.
- **Structural Subtyping (`Protocol`)**: Formalizes Python's "duck typing" philosophy: if an object has the required methods and attributes, it satisfies the type without needing to inherit from the protocol.

Minimal example

Save as `typing_protocols.py`:

```python
# typing_protocols.py
from typing import Protocol, runtime_checkable

@runtime_checkable
class Renderable(Protocol):
```

```
    """Structural protocol: Any class with a render() -> str method satisfies this."""
    def render(self) -> str:
        ...

class ServiceCard:
    """Implements Renderable implicitly WITHOUT inheriting from it."""
    def __init__(self, name: str, port: int) -> None:
        self.name = name
        self.port = port

    def render(self) -> str:
        return f"<ServiceCard: {self.name}:{self.port}>"

def display_dashboard_item(item: Renderable) -> None:
    print("Dashboard Render:", item.render())

def main() -> None:
    card = ServiceCard("auth-service", 8080)

    # Static type checkers and runtime_checkable verify compatibility
    display_dashboard_item(card)
    print(f"Is ServiceCard an instance of Renderable? {isinstance(card, Renderable)}")

if __name__ == "__main__":
    main()
```

Run via `uv run python typing_protocols.py`:

```
Dashboard Render: <ServiceCard: auth-service:8080>
Is ServiceCard an instance of Renderable? True
```

Worked examples

Case 1: Generic Data Containers (`typing.Generic`)

Generics allow functions and classes to operate over arbitrary types while preserving strict type safety:

```
# generic_repository.py
from typing import Generic, TypeVar

T = TypeVar("T")
```

```
class KeyValueCache(Generic[T]):
    def __init__(self) -> None:
        self._store: dict[str, T] = {}

    def put(self, key: str, value: T) -> None:
        self._store[key] = value

    def get(self, key: str) -> T | None:
        return self._store.get(key)

if __name__ == "__main__":
    # Cache parameterized for integers
    int_cache: KeyValueCache[int] = KeyValueCache()
    int_cache.put("cpu_max", 95)
    print("Retrieved integer metric:", int_cache.get("cpu_max"))

    # Cache parameterized for strings
    str_cache: KeyValueCache[str] = KeyValueCache()
    str_cache.put("active_cluster", "us-east-1a")
    print("Retrieved string metric :", str_cache.get("active_cluster"))
```

Run:

```
uv run python generic_repository.py
```

Output:

```
Retrieved integer metric: 95
Retrieved string metric : us-east-1a
```

Case 2: Fluent Method Chaining with `typing.Self`

When implementing builder patterns, methods returning `self` should be typed with `typing.Self` (PEP 673) so that subclasses automatically preserve their own derived return type:

```
# query_builder.py
from typing import Self

class QueryBuilder:
    def __init__(self) -> None:
        self._query_parts: list[str] = []

    def select(self, fields: str) -> Self:
        self._query_parts.append(f"SELECT {fields}")
        return self
```

```
    def from_table(self, table: str) -> Self:
        self._query_parts.append(f"FROM {table}")
        return self

    def limit(self, count: int) -> Self:
        self._query_parts.append(f"LIMIT {count}")
        return self

    def build(self) -> str:
        return " ".join(self._query_parts)

if __name__ == "__main__":
    sql = QueryBuilder().select("id, host, status").from_table("nodes").limit(10).build()
    print("Constructed SQL query:")
    print(sql)
```

Run:

```
uv run python query_builder.py
```

Output:

```
Constructed SQL query:
SELECT id, host, status FROM nodes LIMIT 10
```

Pitfalls

Pitfall 1: Confusing `Type[T]` with `T`

- `obj: ServerNode`: Expects an **instance** of `ServerNode`.
- `cls: type[ServerNode]`: Expects the **class itself** (for factory functions or constructor reflection).

Pitfall 2: Using `Union` Instead of the Modern `|` Operator

In modern Python (PEP 604), write `int | str | None` instead of `Union[int, str, Optional[None]]`. The pipe syntax is native and evaluated without importing `Union`.

Exercises

1. Define a `Protocol` named `Serializable` with a method `to_dict() -> dict[str, object]`. Implement two unrelated classes that satisfy this protocol.
2. Write a generic function `first_or_default(items: list[T], default: T) -> T` that returns the first item or a default fallback.
3. Use `Literal["pending", "running", "completed"]` to restrict a function's status parameter to exact allowed strings.
4. Configure a `[tool.mypy]` section in a `pyproject.toml` enabling `strict = true` and verify code compliance.

Further reading

- PEP 484: *Type Hints*.
- PEP 544: *Protocols: Structural subtyping (static duck typing)*.
- PEP 604: *Allow writing union types as X | Y*.
- PEP 673: *Self Type*.

Unit and Integration Testing with Pytest

Unit and Integration Testing with Pytest

After reading this chapter, you will master testing in Python using `pytest`, write declarative assertions without boilerplate classes, manage test dependencies and teardown with fixtures, parameterize test suites across input matrices with `@pytest.mark.parametrize`, and isolate side effects using `unittest.mock`.

Mental model

Unlike legacy frameworks (`unittest.TestCase`) requiring verbose inheritance hierarchies and specialized assertion methods (`self.assertEqual`), `pytest` intercepts Python's native `assert` statement via AST rewriting to generate comprehensive failure diffs:

```
Test Execution Pipeline:
  pytest CLI

      Test Discovery
  Searches for test_*.py and *_test.py files

      Dependency Injection
  Resolves requested fixture parameters from fixture tree

      AST Assertion Rewriting
  assert calculated == expected    On failure: prints detailed variable introspection

      Teardown
  Executes post-yield cleanup in reverse fixture dependency order
```

Minimal example

Save as `test_metrics_engine.py`:

```python
# test_metrics_engine.py
import pytest

def calculate_percentile(values: list[float], percentile: float) -> float:
    """Compute the specified percentile (0.0 - 1.0) of a sorted list."""
```

```
    if not values:
        raise ValueError("Cannot calculate percentile of empty sequence")
    if not (0.0 <= percentile <= 1.0):
        raise ValueError("Percentile must fall between 0.0 and 1.0")

    sorted_vals = sorted(values)
    k = (len(sorted_vals) - 1) * percentile
    f = int(k)
    c = f + 1 if f + 1 < len(sorted_vals) else f
    d = k - f
    return sorted_vals[f] + d * (sorted_vals[c] - sorted_vals[f])

# 1. Parameterized test matrix
@pytest.mark.parametrize(
    "percentile,expected",
    [
        (0.0, 10.0),   # Min
        (0.5, 30.0),   # Median
        (1.0, 50.0),   # Max
    ]
)
def test_percentile_calculations(percentile: float, expected: float) -> None:
    data = [10.0, 20.0, 30.0, 40.0, 50.0]
    result = calculate_percentile(data, percentile)
    assert result == pytest.approx(expected, rel=1e-3)

# 2. Testing error boundaries with pytest.raises
def test_percentile_empty_data_raises() -> None:
    with pytest.raises(ValueError, match="empty sequence"):
        calculate_percentile([], 0.5)

if __name__ == "__main__":
    import sys
    # Direct invocation runner
    pytest.main(["-v", __file__])
```

Run via `uv run python test_metrics_engine.py`:

```
============================= test session starts ==============================
...
test_metrics_engine.py::test_percentile_calculations[0.0-10.0] PASSED   [ 25%]
test_metrics_engine.py::test_percentile_calculations[0.5-30.0] PASSED   [ 50%]
test_metrics_engine.py::test_percentile_calculations[1.0-50.0] PASSED   [ 75%]
test_metrics_engine.py::test_percentile_empty_data_raises PASSED        [100%]
============================== 4 passed in 0.02s ===============================
```

Worked examples

Case 1: Fixtures with Setup and Teardown Lifecycles

Fixtures using `yield` execute setup code before the test, suspend execution, pass the resource to the test, and execute cleanup code after the test completes:

```
# test_storage_fixture.py
import pytest
import tempfile
import sqlite3
from pathlib import Path
from collections.abc import Generator

@pytest.fixture
def db_conn() -> Generator[sqlite3.Connection, None, None]:
    """Provide a fresh in-memory database with pre-populated schema."""
    conn = sqlite3.connect(":memory:")
    conn.execute("CREATE TABLE inventory (sku TEXT PRIMARY KEY, qty INT)")
    conn.execute("INSERT INTO inventory VALUES ('SKU-100', 42)")
    conn.commit()

    # Yield control to the test function
    yield conn

    # Teardown code runs after test finishes
    conn.close()

def test_inventory_query(db_conn: sqlite3.Connection) -> None:
    cursor = db_conn.execute("SELECT qty FROM inventory WHERE sku = 'SKU-100'")
    qty = cursor.fetchone()[0]
    assert qty == 42
```

Case 2: Isolating External Dependencies with `unittest.mock`

When testing code that calls external web services, email servers, or payment gateways, use `mock` to isolate the unit under test:

```
# test_auth_client.py
from unittest.mock import patch, MagicMock

class Authenticator:
    def check_remote_token(self, token: str) -> bool:
        # In production: makes a real network HTTP call to an OAuth provider
        raise NotImplementedError("Real network endpoint")
```

```
def grant_access(user_token: str, auth: Authenticator) -> str:
    if auth.check_remote_token(user_token):
        return "ACCESS_GRANTED"
    return "ACCESS_DENIED"

def test_grant_access_with_mock() -> None:
    # Create a mock authenticator
    mock_auth = MagicMock(spec=Authenticator)
    mock_auth.check_remote_token.return_value = True

    status = grant_access("valid_token_string", mock_auth)
    assert status == "ACCESS_GRANTED"
    mock_auth.check_remote_token.assert_called_once_with("valid_token_string")

if __name__ == "__main__":
    test_grant_access_with_mock()
    print("Mock unit test passed successfully!")
```

Run:

```
uv run python test_auth_client.py
```

Output:

```
Mock unit test passed successfully!
```

Pitfalls

Pitfall 1: Mutating Shared Session Fixtures

If a fixture has `scope="session"` or `scope="module"`, all tests share the exact same instance. If Test A mutates that instance, Test B may fail spuriously depending on test execution order. Default to `scope="function"` unless resources are read-only or strictly stateless.

Pitfall 2: Testing Implementation Details Instead of Contracts

Tests that assert every internal private method call (`mock_obj._internal_step.assert_called()`) become brittle and break upon simple refactoring. Test observable inputs and outputs.

Exercises

1. Write a parametrized test suite for an IP address validator testing 5 valid IPs and 5 invalid IPs.
2. Create a temporary directory fixture using `tempfile.TemporaryDirectory` that creates a test file and verifies deletion on test completion.
3. Use `unittest.mock.patch("time.time")` to test a rate limiter with simulated deterministically advancing time.
4. Use `pytest -k` and `pytest -m` to organize tests using custom markers (`@pytest.mark.slow`, `@pytest.mark.integration`).

Further reading

- Pytest Documentation: *Fixtures, Marks, and Parametrization*.
- Python Standard Library: `unittest.mock` documentation.
- Brian Okken: *Python Testing with pytest, Second Edition*.

Packaging, Wheel Distribution, and Publishing

Packaging, Wheel Distribution, and Publishing

After reading this chapter, you will master the modern Python packaging standards (PEP 517, PEP 518, PEP 621), author declarative `pyproject.toml` specifications, configure executable CLI console scripts, build standard source distributions (`.tar.gz`) and built wheel packages (`.whl`) with `uv build`, and inspect package distribution metadata.

Mental model

In legacy Python, packaging relied on imperative `setup.py` scripts that executed arbitrary Python code during installation. Modern packaging uses declarative TOML metadata (`pyproject.toml`) and isolated build frontends:

```
Modern Packaging Architecture:
  pyproject.toml (PEP 621 Metadata & Build Backend)

          uv build


Source Distribution (sdist):                Built Wheel (.whl):
  mypkg-0.1.0.tar.gz                          mypkg-0.1.0-py3-none-any.whl
     Raw Python source code                    Pre-built package files
     pyproject.toml                            mypkg-0.1.0.dist-info/
                                                     METADATA
                                                     WHEEL
                                                     entry_points.txt
                                                     RECORD (SHA-256 hashes)
```

A **wheel (`.whl`)** is a ZIP archive formatted with exact installation targets. Installing a wheel is simply an unpack and copy operation into `site-packages` with **zero build step execution**.

Minimal example

Save as `package_builder_demo.py`:

```
# package_builder_demo.py
import tempfile
import zipfile
from pathlib import Path

SAMPLE_PYPROJECT = """
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "cluster-agent"
version = "1.0.0"
description = "Automated cluster monitoring telemetry agent."
readme = "README.md"
requires-python = ">=3.12"
dependencies = [
    "rich>=13.0.0",
]

[project.scripts]
cluster-agent = "cluster_agent.cli:main"
"""

def generate_package_skeleton(root: Path) -> None:
    (root / "pyproject.toml").write_text(SAMPLE_PYPROJECT.strip())
    (root / "README.md").write_text("# Cluster Agent\nHigh-speed cluster agent.")

    pkg_dir = root / "cluster_agent"
    pkg_dir.mkdir()
    (pkg_dir / "__init__.py").write_text('__version__ = "1.0.0"\n')
    (pkg_dir / "cli.py").write_text("""
def main() -> None:
    print("Cluster agent running.")
""")

def main() -> None:
    with tempfile.TemporaryDirectory() as tmpdir:
        root = Path(tmpdir)
        generate_package_skeleton(root)
        print("Generated modern package skeleton with pyproject.toml:")
        for path in sorted(root.rglob("*")):
            if path.is_file():
                print(f"  - {path.relative_to(root)}")

if __name__ == "__main__":
    main()
```

Run via `uv run python package_builder_demo.py`:

```
Generated modern package skeleton with pyproject.toml:
  - README.md
  - cluster_agent/__init__.py
  - cluster_agent/cli.py
  - pyproject.toml
```

Worked examples

Case 1: Defining CLI Console Scripts via `[project.scripts]`

When users install your package, they often want a global command-line command (`my-tool`) available in their shell. Modern `pyproject.toml` declares this declaratively:

```
[project.scripts]
# CLI command name = "module_path:function_to_call"
infra-cli = "infra_pkg.cli:main_entrypoint"
```

When installed, pip or uv automatically generates an executable binary wrapper in the environment's `bin/` directory that executes `main_entrypoint()`.

Case 2: Building Wheels with `uv build`

Executing `uv build` reads `pyproject.toml`, prepares an isolated build environment, and outputs standards-compliant artifacts into `dist/`:

```
# Execute build
uv build

# Output:
# Successfully built dist/cluster_agent-1.0.0.tar.gz
# Successfully built dist/cluster_agent-1.0.0-py3-none-any.whl
```

Case 3: Inspecting Wheel Metadata (`.dist-info`)

Because a `.whl` file is a ZIP archive, you can inspect its metadata directly:

```
# inspect_wheel.py
import zipfile

def inspect_wheel_metadata(wheel_path: str) -> None:
```

```
    with zipfile.ZipFile(wheel_path, "r") as zf:
        metadata_files = [name for name in zf.namelist() if name.endswith("METADATA")]
        if metadata_files:
            meta_text = zf.read(metadata_files[0]).decode("utf-8")
            print("--- Package METADATA Content ---")
            for line in meta_text.splitlines()[:10]:
                print(line)

if __name__ == "__main__":
    print("Wheel metadata inspection helper loaded.")
```

Run:

```
uv run python inspect_wheel.py
```

Output:

```
Wheel metadata inspection helper loaded.
```

Pitfalls

Pitfall 1: Relying on Legacy `setup.py`

Modern toolchains (uv, pip, build) deprecate `setup.py` in favor of declarative `pyproject.toml`. Imperative `setup.py` scripts introduce build-time security vulnerabilities and prevent static dependency analysis.

Pitfall 2: Missing Package Data Files

If your package relies on static templates, HTML files, or YAML configs, you must explicitly declare them in your build backend configuration:

```
# Example for hatchling backend:
[tool.hatch.build.targets.wheel]
packages = ["cluster_agent"]
include = [
    "cluster_agent/templates/*.html",
]
```

Otherwise, wheels will package only `.py` source files, causing `FileNotFoundError` at runtime.

Exercises

1. Write a minimal `pyproject.toml` using `hatchling` as the build backend, declaring a console script entry point.
2. Build a wheel package using `uv build` in a temporary test directory and verify the files generated in `dist/`.
3. Unzip a `.whl` archive and verify the contents of its `entry_points.txt` and `RECORD` files.
4. Configure optional dependency groups (`[project.optional-dependencies]`) for `dev` and `test` suites.

Further reading

- PEP 517: *A build-system independent format for source trees.*
- PEP 518: *Specifying Minimum Build System Requirements for Python Projects.*
- PEP 621: *Storing project metadata in pyproject.toml.*
- Python Packaging Authority (PyPA): *Packaging Python Projects.*

Wheel Variants (PEP 825 and PEP 850): Labels, variant.json, Providers and the Selection Order

Wheel Variants (PEP 825 and PEP 850): Labels, variant.json, Providers and the Selection Order

Thesis

Platform tags (`cp313`, `manylinux_2_28_x86_64`) describe the interpreter, ABI and OS a wheel needs. They cannot say "built for AVX-512", "built against CUDA 13" or "linked to MKL rather than OpenBLAS". That is why projects such as PyTorch run their own per-CUDA indexes, and why projects with compiled code either target a conservative CPU baseline or carry their own runtime dispatch. **Wheel variants** fill that gap in two layers.

- **PEP 825** (Provisional) is the format. A wheel filename may end in a **variant label**. The label indexes a `variant.json` file that maps it to **properties** written `namespace :: feature :: value`. PEP 825 also defines an exact algorithm that ranks the compatible variants.
- **PEP 825 does not say how an installer learns what the machine supports. PEP 850** (Draft) covers that. Each namespace is governed by a **provider**: a static list, a builtin, or a plugin package. A plugin package is never installed or run unless it is trusted or the user opts in.

This chapter builds a small installer front end with nothing but the standard library. It reads the machine's x86-64 level from `/proc/cpuinfo`, filters a mock index and ranks it the way both PEPs specify. Read the ordering rule carefully. It is lexicographic over sort keys, so "more properties" does **not** mean "more preferred". Version note: both PEPs are still under discussion (PEP 825 was created 17 February 2026, PEP 850 on 15 October 2026). Field names may change before the PEPs are final, and no released pip can install variant wheels yet.

Mental model

Five rules carry the whole design.

1. **The filename.** The PEP 825 filename is `{dist}-{ver}(-{build})?-{py}-{abi}-{plat}(-{label})?.whl` Labels match `^[0-9a-z_.]+$`. A six-part name is ambiguous: it has either a build tag or a label. PEP 825 settles this with the existing rule that a build tag starts with a digit and adds that the Python tag **MUST NOT** start with a digit. The label `null` is reserved for the **null variant**, a wheel with zero properties. It is the fallback a variant-aware installer picks when nothing better fits.
2. **`variant.json`.** A file at `*.dist-info/variant.json` has three keys. `default-priorities.namespace` orders the namespaces. `variants` maps label → namespace → feature → a lexically sorted list of values. `providers` (added by PEP 850) says how each namespace is answered. An index may also serve a `{name}-{version}-variants.json` file that combines the metadata

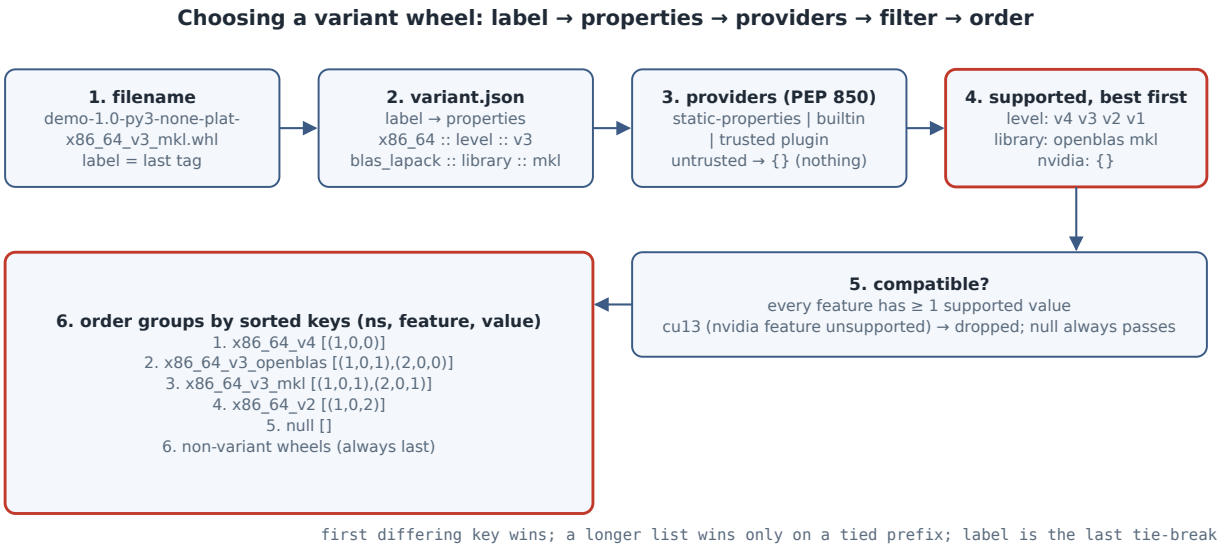


Figure 1: The installer's path from a variant filename to a chosen wheel: label, variant.json properties, provider answers, compatibility filter, then lexicographic ordering with null and non-variant wheels last.

of all of that release's wheels. Tools SHOULD prefer it, because it lets an installer rank variants before downloading any wheel.

3. **The providers' answer.** For every namespace, the installer gets an *ordered* list of supported values for each feature, most preferred first. PEP 850 fixes the order of checks:

   - a user override
   - `optional` (disabled by default)
   - `builtin`
   - `static-properties`
   - a trusted implementation of the plugin named by `requires` / `plugin-api`
   - otherwise only explicitly trusted, installed plugins.

   An untrusted provider counts as "no compatible features". It is never installed silently.

4. **Compatibility.** A variant is compatible if **every feature** it names has **at least one** supported value. A property list is an AND over features and an OR over the values of one feature. The null variant is always compatible.

5. **Ordering.** Each property gets a key (namespace index, feature index, value index), using the best supported value of each feature. Each variant's keys are sorted, and variants are compared lexicographically:

   - The first differing key wins.
   - If one list is a prefix of the other, the longer list wins.
   - If the lists are identical, the label decides.

   The null variant has an empty list, so it lands after every compatible variant. **Non-variant wheels go after all variant groups.** Inside a group, the usual tag and build-number ordering still applies.

Worked example

The script below is a toy installer front end. It has no network and no real wheels: the index is eight filenames plus index-level `variant.json`. The `x86_64` provider is a local reimplementation of a plugin's `get_supported_configs()`, which is what PEP 850 calls a trusted reimplementation. It derives the x86-64 micro-architecture level (psABI v2/v3/v4) from the CPU flags Linux reports. The `nvidia` namespace names a plugin package the tool does not trust, and `blas_lapack` uses `static-properties`.

Output on an 8-core x86-64 Linux box (CPython 3.13.5), verbatim:

```
supported properties on this machine:
  nvidia       {}  (no compatible features)
  x86_64       {'level': ['v4', 'v3', 'v2', 'v1']}
  blas_lapack  {'library': ['openblas', 'mkl']}

wheels on the index:
  demo-1.0-py3-none-manylinux_2_28_x86_64-x86_64_v4.whl          label=x86_64_v4          co
  demo-1.0-py3-none-manylinux_2_28_x86_64-x86_64_v3_openblas.whl label=x86_64_v3_openblas co
  demo-1.0-py3-none-manylinux_2_28_x86_64-x86_64_v3_mkl.whl      label=x86_64_v3_mkl      co
  demo-1.0-py3-none-manylinux_2_28_x86_64-x86_64_v2.whl          label=x86_64_v2          co
  demo-1.0-py3-none-manylinux_2_28_x86_64-cu13.whl               label=cu13               IN
  demo-1.0-py3-none-manylinux_2_28_x86_64-null.whl               label=null               co
  demo-1.0-py3-none-manylinux_2_28_x86_64.whl                    non-variant
  demo-1.0-1-py3-none-manylinux_2_28_x86_64.whl                  non-variant (build tag 1)

preference order (best first):
  1. x86_64_v4          best props ['x86_64 :: level :: v4']  keys [(1, 0, 0)]
  2. x86_64_v3_openblas best props ['x86_64 :: level :: v3', 'blas_lapack :: library :: open
  3. x86_64_v3_mkl      best props ['x86_64 :: level :: v3', 'blas_lapack :: library :: mkl'
  4. x86_64_v2          best props ['x86_64 :: level :: v2']  keys [(1, 0, 2)]
  5. null               best props []  keys []
  6. non-variant group: ['demo-1.0-py3-none-manylinux_2_28_x86_64.whl', 'demo-1.0-1-py3-none

selected: demo-1.0-py3-none-manylinux_2_28_x86_64-x86_64_v4.whl
```

As a cross-check, glibc's dynamic loader agrees on the CPU level. `/lib64/ld-linux-x86-64.so.2 --help` on the same machine prints `x86-64-v4 (supported, searched)`, `x86-64-v3 (supported, searched)` and `x86-64-v2 (supported, searched)`.

Reading it

- **The provider answers.**
  - `x86_64` came from the trusted reimplementation. It reports `['v4', 'v3', 'v2', 'v1']`, best first, as PEP 850 requires of `get_supported_configs()`.

  - `blas_lapack` came straight from `static-properties`. This is PEP 850's case for non-platform variants: every variant that was built is installable, and the list order is just the default preference.
  - `nvidia` names `nvidia-variant-provider` under `requires`. The tool has no trusted implementation and the user did not opt in, so step 7 applies: **assume nothing is compatible**. The installer did not fetch or run anything.

- **Compatibility.**
  - `cu13` needs `nvidia :: cuda_version_lower_bound`, which is unsupported, so the whole variant is out. Its `x86_64 :: level :: v2` property would have matched, but every feature must match.
  - `null` has no features, so it passes.

- **The two six-part names.** `demo-1.0-py3-none-manylinux_2_28_x86_64-null.whl` and `demo-1.0-1-py3-none-manylinux_2_28_x86_64.whl` both have six dash-separated parts. The parser separates them with one test: a build tag starts with a digit, and the Python tag cannot. Without that new constraint in PEP 825, the parse would be ambiguous.
- **The ordering.** The namespace order `["nvidia", "x86_64", "blas_lapack"]` gives `x86_64` index 1 and `blas_lapack` index 2.
  - `x86_64_v4` → `[(1,0,0)]` beats `x86_64_v3_openblas` → `[(1,0,1),(2,0,0)]` on the **first** key. Having two properties does not help, because the CPU level is the more important namespace.
  - The two v3 builds tie on `(1,0,1)`. The second key decides: openblas `(2,0,0)` comes before mkl `(2,0,1)`, because the static list puts openblas first.
  - `x86_64_v2` follows. `null` (an empty list) comes after every compatible variant, and both non-variant wheels come last.

- **The prefix rule.** It matters for a pair like `x86_64_v3` against `x86_64_v3_openblas`: the keys are `[(1,0,1)]` against `[(1,0,1),(2,0,0)]`. The prefix ties, so the longer list wins. A more specific build beats a less specific one only when they agree on everything the shorter one says.

Pitfalls

- **Treating more properties as better.** Comparison is lexicographic on sorted keys, so one property in a high-priority namespace beats any number in lower ones. If you want BLAS choice to dominate CPU level, put `blas_lapack` first in `default-priorities.namespace`, or override it in the installer (PEP 825 allows tools to do that).
- **Expecting an unknown plugin to just work.** Under PEP 850, a namespace whose plugin is untrusted yields *no* compatible features, so every variant that uses it is skipped. That is the security design, not a bug. Ship a `null` variant or a non-variant wheel so these users still get something. Otherwise the resolver falls back to an older release or the sdist.
- **Forgetting the AND.** A variant with properties in two namespaces needs both to be supported. Adding a CPU-level property to a GPU build (`cu13` above) narrows it. It does not widen it.

- **Unsorted or uppercase values.** Values in `variants` are sets, stored as lexically sorted lists. Names match `^[a-z0-9_]+$` and values match `^[a-z0-9_.]+$`. The script asserts both, so `"V3"` or `["v3", "v2"]` would fail validation.
- **Simplified CPU detection.** The flag sets in `LEVELS` follow the x86-64 psABI level definitions using Linux flag names (`pni` is SSE3, `abm` covers LZCNT). A real provider also checks OS support for the wider register state (OSXSAVE/XGETBV), and handles macOS, Windows and virtual machines that hide flags. Use a maintained provider, not this toy.
- **Assuming installers support this today.** Both PEPs are under discussion, and the `$schema` URL and field names may still change. Treat `variant.json` produced by experimental tooling as unstable.

Try this

1. Reorder `default-priorities.namespace` to `["nvidia", "blas_lapack", "x86_64"]` and re-run. Predict the new top choice before you look (hint: `x86_64_v3_openblas` now has the key `(1,0,0)`).
2. Add a variant `x86_64_v3` with only `{"x86_64": {"level": ["v3"]}}`, and add its wheel `demo-1.0-py3-none-manylinux_2_28_x86_64-x86_64_v3.whl` to `INDEX_FILES` (the script only ranks labels that have a wheel). Confirm that it sorts *after* both v3 BLAS variants (prefix rule) and *before* `x86_64_v2`.
3. Make `nvidia` trusted by adding a fake plugin to `TRUSTED_PLUGINS` that returns `cuda_version_lower_bound = ["13.0", "12.8"]`. Now `cu13` is compatible. Where does it land, and why does the `nvidia` namespace's index 0 put it first?
4. Mark `blas_lapack` `"optional": true` and explain why both v3 BLAS builds disappear while `x86_64_v4` is unaffected.

Sources

- PEP 825: Wheel Variants: Package Format. Covers the filename and label rules, the `variant.json` structure, the compatibility rule and the ordering algorithm (steps 1–7 with reference pseudocode).
- PEP 850: Wheel Variants: Providers. Covers the provider resolution steps, the `providers` keys (`requires`, `plugin-api`, `static-properties`, `feature-order`, `builtin`, `optional`), the plugin API `get_supported_configs()` and the trust model. Discussion thread.
- Binary distribution format (wheel) specification. Covers the existing filename convention and build-tag rule.
- Platform compatibility tags. Explains what tags can and cannot express.
- x86-64 psABI. Gives the definitions of the x86-64-v2/v3/v4 micro-architecture levels.
- glibc 2.33 NEWS. Introduces the glibc-hwcaps subdirectories `x86-64-v2/v3/v4` and the dynamic linker's `--help` option used for the cross-check.

Inspiration: Pycoder's Weekly on PEP 850.

Listing 0.1 `variants.py`

```
"""A toy installer front end for wheel variants (PEP 825 format + PEP 850 providers).

No network and no real wheels: the "index" is a list of filenames plus the
index-level variant metadata. The x86_64 provider is a local reimplementation
that reads /proc/cpuinfo, which is what PEP 850 calls a trusted reimplementation.
"""
import re
from dataclasses import dataclass

LABEL_RE = re.compile(r"^[0-9a-z_.]+$")
NAME_RE = re.compile(r"^[a-z0-9_]+$")
VALUE_RE = re.compile(r"^[a-z0-9_.]+$")

def parse_wheel(fn: str):
    """Split a wheel filename; return (dist, version, build, py, abi, plat, label)."""
    parts = fn.removesuffix(".whl").split("-")
    if len(parts) == 5:
        return (*parts[:2], None, *parts[2:], None)
    if len(parts) == 6:
        # PEP 825: the Python tag MUST NOT start with a digit, a build tag MUST.
        if parts[2][0].isdigit():
            return (*parts[:3], *parts[3:], None)
        label = parts[5]
        return (*parts[:2], None, *parts[2:5], label)
    if len(parts) == 7:
        return tuple(parts)
    raise ValueError(f"not a wheel filename: {fn}")

# ---- providers (PEP 850) ----------------------------------------------------
@dataclass
class VariantFeatureConfig:
    name: str
    values: list[str]
    multi_value: bool

LEVELS = {  # x86-64 psABI micro-architecture levels, as Linux /proc/cpuinfo flag names
    "v2": {"cx16", "lahf_lm", "popcnt", "pni", "sse4_1", "sse4_2", "ssse3"},
    "v3": {"avx", "avx2", "bmi1", "bmi2", "f16c", "fma", "abm", "movbe", "xsave"},

    "v4": {"avx512f", "avx512bw", "avx512cd", "avx512dq", "avx512vl"},
}

def x86_64_get_supported_configs() -> list[VariantFeatureConfig]:
    flags = set()
    for line in open("/proc/cpuinfo"):
        if line.startswith("flags"):
            flags = set(line.split(":", 1)[1].split())
            break
    ok = ["v1"]
    for lvl in ("v2", "v3", "v4"):
        if LEVELS[lvl] <= flags:
```

Advanced Type System: PEP 695, ParamSpec, and Type Narrowing

Advanced Type System: PEP 695, `ParamSpec`, and Type Narrowing

After reading this chapter, you will master Python's cutting-edge type system capabilities, leverage the PEP 695 type parameter syntax (`type`, generic functions, generic classes), author signature-preserving decorators using `ParamSpec` and `Concatenate`, eliminate false positives in type narrowing using `TypeIs` (PEP 742) versus `TypeGuard`, prevent injection vulnerabilities at compile time with `LiteralString`, and enforce exhaustive pattern matching using `Never` and `assert_never`.

Mental model

Static typing in Python has evolved from basic type annotations (PEP 484) to a sophisticated type algebra capable of modeling dynamic Python idioms without sacrificing safety:

```
Evolution of Python Generics

Python 3.5 - 3.11 (Verbose)    Python 3.12+ PEP 695 (Clean)

T = TypeVar("T")               def first[T](seq: list[T])
Vector = list[T]               type Vector[T] = list[T]
class Box(Generic[T]): ...     class Box[T]: ...
```

Type Narrowing: `TypeGuard` vs `TypeIs`

When inspecting dynamic types at runtime, standard `isinstance()` checks narrow unions automatically. However, when factoring logic into reusable helper functions, standard return types (`bool`) lose type information.

Python provides two narrowing constructs: 1. **`TypeGuard[T]` (PEP 647)**: Narrows *only* when returning `True`. In the `else` branch, the type is **not** narrowed. 2. **`TypeIs[T]` (PEP 742 - Python 3.13+)**: True bidirectional narrowing. If `is_str(x)` returns `True`, `x` is `str`. If `False`, `str` is **removed** from the union in the `else` branch.

```
TypeIs Narrowing Matrix (x: int | str):
  if is_str(x):
        x is narrowed to str!
  else:
        x is narrowed to int! (TypeGuard leaves x as int | str)
```

Minimal example

Save as `pep695_overview.py`:

```
# pep695_overview.py
from typing import Sequence

# 1. PEP 695 Type Alias syntax
type ResultMatrix[T] = dict[str, list[T]]

# 2. PEP 695 Generic Function syntax
def find_first[T](items: Sequence[T], default: T) -> T:
    # Returns first item or fallback default
    return items[0] if items else default

# 3. PEP 695 Generic Class syntax
class BoundedStack[T]:
    def __init__(self, capacity: int) -> None:
        self._capacity = capacity
        self._items: list[T] = []

    def push(self, item: T) -> None:
        if len(self._items) >= self._capacity:
            raise OverflowError("Stack full")
        self._items.append(item)

    def pop(self) -> T:
        return self._items.pop()

def main() -> None:
    # Test generic function
    first_num = find_first([10, 20, 30], default=0)
    first_str = find_first([], default="fallback")
    print(f"First num: {first_num} (type={type(first_num).__name__})")
    print(f"First str: {first_str} (type={type(first_str).__name__})")

    # Test generic class
    stack: BoundedStack[str] = BoundedStack(capacity=2)
    stack.push("alpha")
    stack.push("beta")
    print(f"\nPopped item: {stack.pop()}")

if __name__ == "__main__":
    main()
```

Run via `uv run python pep695_overview.py`:

```
First num: 10 (type=int)
First str: fallback (type=str)

Popped item: beta
```

Worked examples

Case 1: Signature-Preserving Decorators with `ParamSpec` and `Concatenate`

Before `ParamSpec`, decorating a function with `*args: Any, **kwargs: Any` erased parameter names, types, and defaults, crippling IDE autocomplete and static type checking. `ParamSpec` captures exact argument signatures:

```python
# type_safe_decorator.py
from typing import Callable, Concatenate, ParamSpec, TypeVar
import time

P = ParamSpec("P")
R = TypeVar("R")

class SecurityContext:
    def __init__(self, user: str, roles: set[str]) -> None:
        self.user = user
        self.roles = roles

def require_auth(
    func: Callable[Concatenate[SecurityContext, P], R]
) -> Callable[P, R]:
    # Decorator that injects a SecurityContext while preserving target signature
    def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
        ctx = SecurityContext(user="operator_1", roles={"admin", "deploy"})
        print(f"[AUTH] Injected context for user '{ctx.user}' into {func.__name__}()")
        return func(ctx, *args, **kwargs)
    return wrapper

@require_auth
def restart_cluster(ctx: SecurityContext, cluster_id: str, force: bool = False) -> str:
    return f"Cluster '{cluster_id}' restarted by {ctx.user} (force={force})"

def main() -> None:
    result = restart_cluster("prod-k8s-01", force=True)
    print(f"Result: {result}")
```

```
if __name__ == "__main__":
    main()
```

Run:

```
uv run python type_safe_decorator.py
```

Output:

```
[AUTH] Injected context for user operator_1 into restart_cluster()
Result: Cluster prod-k8s-01 restarted by operator_1 (force=True)
```

Case 2: Bidirectional Narrowing with `TypeIs` (PEP 742)

In domain models with heterogeneous unions, validating objects with helper predicates often requires narrowing in both `if` and `else` branches:

```
# type_is_narrowing.py
from typing import TypeIs

class HardwareNode:
    def __init__(self, hostname: str, rack: str) -> None:
        self.hostname = hostname
        self.rack = rack

class VirtualInstance:
    def __init__(self, instance_id: str, vcpu: int) -> None:
        self.instance_id = instance_id
        self.vcpu = vcpu

type ComputeTarget = HardwareNode | VirtualInstance

def is_hardware(node: ComputeTarget) -> TypeIs[HardwareNode]:
    # Predicate function with PEP 742 TypeIs bidirectional narrowing
    return isinstance(node, HardwareNode)

def dispatch_maintenance(node: ComputeTarget) -> str:
    if is_hardware(node):
        return f"Power cycle physical rack {node.rack} for {node.hostname}"
    else:
        return f"Live-migrate VM {node.instance_id} ({node.vcpu} vCPUs)"

def main() -> None:
    hw = HardwareNode("blade-04.datacenter", rack="RACK-12")
```

```
    vm = VirtualInstance("i-08a9fbc12", vcpu=8)

    print(dispatch_maintenance(hw))
    print(dispatch_maintenance(vm))

if __name__ == "__main__":
    main()
```

Run:

```
uv run python type_is_narrowing.py
```

Output:

```
Power cycle physical rack RACK-12 for blade-04.datacenter
Live-migrate VM i-08a9fbc12 (8 vCPUs)
```

Case 3: Compile-Time Injection Immunity with `LiteralString` (PEP 675)

SQL injection and command injection happen when user-controlled strings are concatenated into raw query strings. `LiteralString` enforces that a parameter must consist solely of string literals authored in source code:

```
# injection_guard.py
from typing import LiteralString

class SafeQueryExecutor:
    def execute(self, query: LiteralString, *params: object) -> str:
        return f"Executing verified literal: {query} with params: {params}"

def main() -> None:
    db = SafeQueryExecutor()

    # 1. Compile-time safe: raw literal string with placeholders
    res1 = db.execute("SELECT * FROM accounts WHERE id = ? AND status = ?", 42, "ACTIVE")
    print(res1)

    # 2. Compile-time safe: concatenating two literal strings
    PART1: LiteralString = "SELECT * FROM metrics "
    PART2: LiteralString = "ORDER BY timestamp DESC"
    res2 = db.execute(PART1 + PART2)
    print(res2)

if __name__ == "__main__":
```

```
    main()
```

Run:

```
uv run python injection_guard.py
```

Output:

```
Executing verified literal: SELECT * FROM accounts WHERE id = ? AND status = ? with params: (42
Executing verified literal: SELECT * FROM metrics ORDER BY timestamp DESC with params: ()
```

Pitfalls

Pitfall 1: Assuming `TypeGuard` Narrows the `else` Branch

Developers frequently confuse `TypeGuard` with `TypeIs`. If a function returns `TypeGuard[T]`, static type checkers only narrow the `if` branch. They do **not** exclude `T` in the `else` branch:

```
# THE TRAP:
from typing import TypeGuard

def is_int(val: int | str) -> TypeGuard[int]:
    return isinstance(val, int)

def process(val: int | str):
    if is_int(val):
        print(val + 1)
    else:
        # BUGGY ASSUMPTION: The type checker treats val as (int | str), NOT str!
        pass

# THE FIX:
# Use TypeIs (Python 3.13+) for full bidirectional narrowing:
from typing import TypeIs

def is_int_fixed(val: int | str) -> TypeIs[int]:
    return isinstance(val, int)
```

Pitfall 2: Forgetting `assert_never` in Exhaustive Pattern Matching

When matching over an enum or closed union, if a new variant is added to the union in the future, missing cases can slip into production unhandled. `assert_never` turns incomplete matches into static type-check errors:

```
# THE PATTERN:
from typing import assert_never

type Status = str

def handle_status(status: Status) -> str:
    match status:
        case "PENDING":
            return "Waiting in queue"
        case "RUNNING":
            return "Executing job"
        case "FAILED":
            return "Alert sent"
        case unexpected:
            assert_never(unexpected)
```

Exercises

1. Refactor a legacy generic class using `TypeVar` and `Generic[T]` into PEP 695 syntax (`class Cache[K, V]:`).
2. Write a type-safe `@retry` decorator using `ParamSpec` that preserves the wrapped function's argument names and return type while adding an optional `max_retries` configuration.
3. Implement a `TypeIs` narrowing function `is_valid_ipv4(addr: str | bytes)` that narrows `addr` to `str` when valid and `bytes` otherwise.
4. Build a secure shell command executor that accepts only `LiteralString` commands, preventing shell injection from user inputs.
5. Create an exhaustive state machine transition function over a literal union of order states (`CREATED`, `PAID`, `SHIPPED`, `REFUNDED`) using `match/case` and `assert_never`.

Further reading

- Python Documentation: `typing` — *Support for type hints.*
- PEP 695: *Type Parameter Syntax.*
- PEP 742: *Narrowing types with TypeIs.*

- PEP 612: *Parameter Specification Variables (ParamSpec)*.
- PEP 675: *Arbitrary Literal String Type (LiteralString)*.

Deferred Annotations

Deferred Annotations

In Python **3.14** (PEP 649 and PEP 749), annotations on functions, classes, and modules are **not** evaluated when the `def` or `class` runs. They sit in a special annotate function and run only when something asks for them. Forward names no longer need quotes. The boring default is: write the hint you mean, leave evaluation alone until a tool (or your own runtime check) needs real types — then use `annotationlib.get_annotations` with an explicit `Format`.

Mental model

- **Deferred (3.14 default):** defining `def seat_for(ticket: Ticket)` does not look up `Ticket` yet. Import stays cheap. The name can appear later in the file.
- **Three formats** via `annotationlib.Format`:
  - `VALUE` — evaluate to real objects (old stock behavior). Raises `NameError` if a name is missing.
  - `FORWARDREF` — real objects when possible; otherwise an `annotationlib.ForwardRef` proxy you can inspect.
  - `STRING` — source-ish text (`'Ticket'`, `'int'`), useful for docs and displays.
- Prefer `get_annotations(obj, format=...)` over poking `__annotations__` by hand. Accessing `__annotations__` tries to evaluate; with unresolved names it can raise.
- `from __future__ import annotations` still **stringizes** annotations (PEP 563). That path remains for compatibility, but 3.14's deferred model is the default without it. Do not add the future import "just in case" when deferred already covers forward refs.
- When you need runtime types (validators, small schema helpers, Pydantic-style edges), ask for `Format.VALUE` — or `eval_str=True` only if you are dealing with stringized annotations from the future import.

Worked examples

Case 1: Forward ref without quotes

Save as `forward_ref.py`. `Ticket` is not defined when `seat_for` is. The definition succeeds. Inspection chooses the format.

```
# forward_ref.py
from annotationlib import Format, get_annotations
```

```
def seat_for(ticket: Ticket) -> int:
    return ticket.table


def main() -> None:
    print("STRING:", get_annotations(seat_for, format=Format.STRING))
    fwd = get_annotations(seat_for, format=Format.FORWARDREF)
    print("FORWARDREF ticket:", type(fwd["ticket"]).__name__)
    print("FORWARDREF arg:", fwd["ticket"].__forward_arg__)
    try:
        get_annotations(seat_for, format=Format.VALUE)
    except NameError as exc:
        print("VALUE:", type(exc).__name__ + ":", exc)


if __name__ == "__main__":
    main()
```

Run:

```
uv run --python 3.14 python forward_ref.py
```

Output:

```
STRING: {'ticket': 'Ticket', 'return': 'int'}
FORWARDREF ticket: ForwardRef
FORWARDREF arg: Ticket
VALUE: NameError: name 'Ticket' is not defined
```

No quotes around `Ticket`. `STRING` and `FORWARDREF` work. `VALUE` fails until the name exists in the function's globals.

Case 2: All three formats on a resolved function

Save as `three_formats.py`. Every name is already defined. All formats succeed; they just look different.

```
# three_formats.py
from annotationlib import Format, get_annotations


def label(table: int, note: str) -> str:
    return f"table {table}: {note}"


def main() -> None:
```

```
    for fmt in (Format.VALUE, Format.FORWARDREF, Format.STRING):
        print(fmt.name, get_annotations(label, format=fmt))


if __name__ == "__main__":
    main()
```

Run:

```
uv run --python 3.14 python three_formats.py
```

Output:

```
VALUE {'table': <class 'int'>, 'note': <class 'str'>, 'return': <class 'str'>}
FORWARDREF {'table': <class 'int'>, 'note': <class 'str'>, 'return': <class 'str'>}
STRING {'table': 'int', 'note': 'str', 'return': 'str'}
```

Use `VALUE` when you need `isinstance` or constructor checks. Use `STRING` when you only need to print or document the hint.

Case 3: Class annotation before the name exists

Save as `board_forward.py`. Same idea on a class attribute. The class body does not need `Ticket` yet.

```
# board_forward.py
from annotationlib import Format, get_annotations


class Board:
    next_ticket: Ticket


def main() -> None:
    print("STRING:", get_annotations(Board, format=Format.STRING))
    fwd = get_annotations(Board, format=Format.FORWARDREF)
    print("FORWARDREF type:", type(fwd["next_ticket"]).__name__)
    print("FORWARDREF arg:", fwd["next_ticket"].__forward_arg__)
    try:
        get_annotations(Board, format=Format.VALUE)
    except NameError as exc:
        print("VALUE:", type(exc).__name__ + ":", exc)


if __name__ == "__main__":
    main()
```

Run:

```
uv run --python 3.14 python board_forward.py
```

Output:

```
STRING: {'next_ticket': 'Ticket'}
FORWARDREF type: ForwardRef
FORWARDREF arg: Ticket
VALUE: NameError: name 'Ticket' is not defined
```

Define `Ticket` later in the module if you want. After it lives in the module globals, `Format.VALUE` on `Board` will resolve.

Case 4: Evaluate when you need types at runtime

Save as `validate_ticket.py`. A tiny desk validator asks for `VALUE` only when checking a call. Names are defined, so evaluation is safe.

```
# validate_ticket.py
from annotationlib import Format, get_annotations


def validate(fn, args: dict):
    hints = get_annotations(fn, format=Format.VALUE)
    for name, expected in hints.items():
        if name == "return":
            continue
        got = type(args[name])
        if got is not expected:
            raise TypeError(
                f"{name}: want {expected.__name__}, got {got.__name__}"
            )
    return fn(**args)


def open_ticket(table: int, note: str) -> str:
    return f"table {table}: {note}"


def main() -> None:
    print(validate(open_ticket, {"table": 12, "note": "rush"}))
    try:
        validate(open_ticket, {"table": "12", "note": "rush"})
    except TypeError as exc:
        print(type(exc).__name__ + ":", exc)
```

```
if __name__ == "__main__":
    main()
```

Run:

```
uv run --python 3.14 python validate_ticket.py
```

Output:

```
table 12: rush
TypeError: table: want int, got str
```

Libraries that build models from annotations follow the same rule: request `VALUE` (or resolve `ForwardRef`s) at the moment you need constructors — not at import time for every function in the kitchen.

Case 5: The `__annotations__` / `get_type_hints` habit

Save as `ann_trap.py`. Pre-3.14 code often read `__annotations__` or called `typing.get_type_hints` and expected a dict of live types. With a missing name, both can raise. Ask for `STRING` or `FORWARDREF` when you only need to inspect.

```
# ann_trap.py
from annotationlib import Format, get_annotations
from typing import get_type_hints


def seat_for(ticket: Ticket) -> int:
    return 0


def main() -> None:
    try:
        print(seat_for.__annotations__)
    except NameError as exc:
        print("__annotations__:", type(exc).__name__ + ":", exc)

    try:
        print(get_type_hints(seat_for))
    except NameError as exc:
        print("get_type_hints:", type(exc).__name__ + ":", exc)

    print("STRING:", get_annotations(seat_for, format=Format.STRING))
    fwd = get_annotations(seat_for, format=Format.FORWARDREF)
    print("FORWARDREF ticket:", type(fwd["ticket"]).__name__)
```

```
if __name__ == "__main__":
    main()
```

Run:

```
uv run --python 3.14 python ann_trap.py
```

Output:

```
__annotations__: NameError: name 'Ticket' is not defined
get_type_hints: NameError: name 'Ticket' is not defined
STRING: {'ticket': 'Ticket', 'return': 'int'}
FORWARDREF ticket: ForwardRef
```

The trap is not that deferred evaluation is broken. The trap is assuming every access path still means "give me evaluated types right now."

The trap

Two related mistakes:

1. **Assuming `__annotations__` is always a quiet dict of types** like many 3.10–3.13 codebases expected under stock semantics (or under stringized future annotations). In 3.14, reading it may evaluate — and fail — if names are unresolved.
2. **Stringizing forever** with `from __future__ import annotations` when you do not need it. Deferred evaluation already allows unquoted forward refs. Keeping the future import forces stringized values and makes runtime consumers reach for `eval_str` or manual `eval`. Use it only when you are matching older tooling that still expects strings.

```
# future_still_strings.py
from __future__ import annotations

from annotationlib import Format, get_annotations


def seat_for(ticket: Ticket) -> int:
    return 0


class Ticket:
    pass


def main() -> None:
    print("__annotations__:", seat_for.__annotations__)
```

```
    print("STRING:", get_annotations(seat_for, format=Format.STRING))
    print(
        "VALUE eval_str:",
        get_annotations(seat_for, format=Format.VALUE, eval_str=True),
    )


if __name__ == "__main__":
    main()
```

Run:

```
uv run --python 3.14 python future_still_strings.py
```

Output:

```
__annotations__: {'ticket': 'Ticket', 'return': 'int'}
STRING: {'ticket': 'Ticket', 'return': 'int'}
VALUE eval_str: {'ticket': <class '__main__.Ticket'>, 'return': <class 'int'>}
```

With the future import, `__annotations__` is strings even after `Ticket` exists. That is deliberate PEP 563 behavior — not the 3.14 deferred default.

The boring rule

- Write forward names without quotes under Python 3.14's default deferred model.
- Introspect with `annotationlib.get_annotations(obj, format=...)` — pick `VALUE`, `FORWARDREF`, or `STRING` on purpose.
- Use `VALUE` only when names resolve (or when failure is the signal you want).
- Prefer `FORWARDREF` or `STRING` for docs, boards, and "what did the author write?" tooling.
- Do not poke `__annotations__` as your first move on 3.14+ code.
- Do not add `from __future__ import annotations` unless you truly need stringized annotations for older consumers.
- Keep runtime validators thin: evaluate at the edge, not at every import.

Try this

1. In `forward_ref.py`, define `class Ticket:` at module bottom with a `table: int` attribute, then call `get_annotations(seat_for, format=Format.VALUE)` again. Confirm it prints the real class.
2. In `three_formats.py`, annotate `note: Note` without defining `Note`. Re-run and watch which formats break.
3. In `validate_ticket.py`, add a `seats: int` parameter and pass `seats=4`. Confirm the validator still accepts the happy path.

4. Delete the future import from `future_still_strings.py` and re-run. Compare `__annotations__` (or `get_annotations(..., Format.VALUE)`) with and without the import — one sentence on the difference.

Closed TypedDicts and Typed Extra Items (PEP 728)

Closed TypedDicts and Typed Extra Items (PEP 728)

Thesis

Before Python 3.15, a `TypedDict` described the keys you *listed* and said nothing about the rest. A value of type `Movie` might hold any number of other keys of any type. That made two common shapes impossible to type:

- "exactly these keys and nothing else", such as a strict API payload;
- "these keys, plus any number of extras whose values are all `int`", such as a scores record.

PEP 728 (Final, Python 3.15) adds two class parameters to fix this. `closed=True` forbids extra keys. `extra_items=T` allows extra keys, but only with values of type `T`, optionally `ReadOnly[T]`. Checkers can then do things they couldn't before: treat a `TypedDict` as a precise `Mapping[str, V]`, reject stray keys, and forbid subclasses from adding keys. This chapter runs one file through Python 3.15 and pyright to show each rule, then shows how mypy's support differs.

Mental model

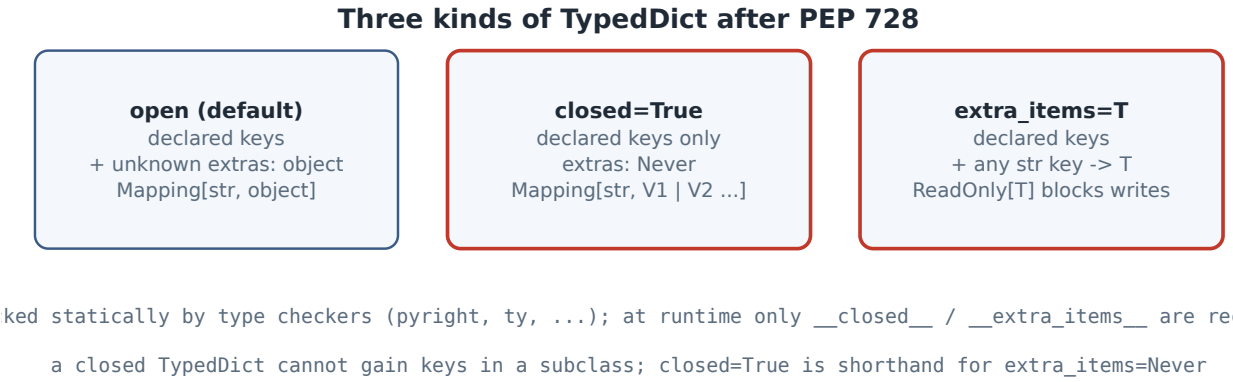


Figure 1: Open, closed and extra_items TypedDicts.

- **Open (the default).** Per the PEP, an open TypedDict behaves as if it had non-required extra items of type `ReadOnly[object]`. So the most a checker can say is `Mapping[str, object]`.

- **`closed=True`.** This is equivalent to `extra_items=Never`: no value is assignable to `Never`, so no extra key can exist. The value types are then known exactly, which makes the TypedDict a `Mapping[str, V1 | V2 | ...]`.
- **`extra_items=T`.** Any other `str` key maps to `T`. Extra keys are never *required*, so `del d["k"]` on an extra key is allowed. Wrapping `T` in `ReadOnly[...]` (PEP 705) forbids writing extras.
- **Inheritance.** A subclass of a closed TypedDict can't add keys. A subclass with `extra_items=T` can add only keys whose type is compatible with `T`.
- **Runtime.** Nothing is validated. The class records `__closed__` and `__extra_items__` for introspection. Passing both `closed` and `extra_items` is a runtime error.

Worked example

Every line commented `ERROR` should be rejected by a checker that implements PEP 728, and every line marked `ok` should be accepted.

```
"""PEP 728 (Python 3.15): closed TypedDicts and typed extra items."""
from collections.abc import Mapping
from typing import NotRequired, ReadOnly, TypedDict


class Movie(TypedDict):                         # open (the pre-3.15 default)
    name: str


class ClosedMovie(TypedDict, closed=True):      # no keys beyond the declared ones
    name: str
    year: NotRequired[int]


class Scores(TypedDict, extra_items=int):       # any other str key must map to int
    player: str


class Labels(TypedDict, extra_items=ReadOnly[str]):   # extras are read-only
    id: str


def check_static() -> None:
    a: ClosedMovie = {"name": "Alien", "director": "Scott"}   # ERROR: closed
    b: Scores = {"player": "ana", "round1": 3, "round2": 5}   # ok: extras are int
    c: Scores = {"player": "ana", "round1": "three"}          # ERROR: extra not int
    b["bonus"] = 2                                            # ok
    b["note"] = "late"                                        # ERROR
    lab: Labels = {"id": "x", "color": "red"}
    lab["color"] = "blue"                                     # ERROR: read-only extra
    del b["round1"]                                           # ok: extras are not required
```

```
def total(m: Mapping[str, int]) -> int:
    return sum(m.values())


class IntOnly(TypedDict, extra_items=int):
    base: int


def check_mapping(s: Scores, i: IntOnly, cm: ClosedMovie, mv: Movie) -> None:
    total(i)          # ok: every value (declared and extra) is int
    total(s)          # ERROR: "player" is str
    m1: Mapping[str, str | int] = cm     # ok: closed, so values are str | int
    m2: Mapping[str, str | int] = mv     # ERROR: open TypedDict could hold anything


class MoreMovie(ClosedMovie):
    rating: float     # ERROR: a closed TypedDict cannot gain new keys in a subclass


if __name__ == "__main__":
    # Runtime: the flags are introspectable, but nothing is validated.
    for td in (Movie, ClosedMovie, Scores, Labels):
        print(f"{td.__name__:12s} __closed__={td.__closed__!s:5s} "
              f"__extra_items__={td.__extra_items__}")
    bad: Scores = {"player": "ana", "round1": "three"}    # type: ignore[typeddict-item]
    print("runtime accepts a wrong extra:", bad)
```

Running it under CPython 3.15.0:

```
$ python3.15 extra_items.py
Movie        __closed__=None  __extra_items__=NoExtraItems
ClosedMovie  __closed__=True  __extra_items__=NoExtraItems
Scores       __closed__=None  __extra_items__=<class 'int'>
Labels       __closed__=None  __extra_items__=typing.ReadOnly[str]
runtime accepts a wrong extra: {'player': 'ana', 'round1': 'three'}
```

Checking it with pyright 1.1.414 (`--pythonversion 3.15`):

```
$ pyright --pythonversion 3.15 extra_items.py
extra_items.py
  extra_items.py:24:40 - error: Type "dict[str, str]" is not assignable to declared type "Cl
    Type "Literal['Scott']" is not assignable to item "extra_items"
      Type "Literal['Scott']" is not assignable to type "Never" (reportAssignmentType)
  extra_items.py:26:45 - error: Type "dict[str, str]" is not assignable to declared type "Sc
    "Literal['three']" is not assignable to "int" (reportAssignmentType)
```

```
  extra_items.py:28:5 - error: Could not assign item in TypedDict
    "Literal['late']" is not assignable to "int" (reportGeneralTypeIssues)
  extra_items.py:30:5 - error: Could not assign item in TypedDict
    "color" is a read-only key in "Labels" (reportTypedDictNotRequiredAccess)
  extra_items.py:44:11 - error: Argument of type "Scores" cannot be assigned to parameter "m
    "Mapping[str, str | int]" is not assignable to "Mapping[str, int]"
      Type parameter "_VT_co@Mapping" is covariant, but "str | int" is not a subtype of "int
        Type "str | int" is not assignable to type "int"
          "str" is not assignable to "int" (reportArgumentType)
  extra_items.py:46:35 - error: Type "Movie" is not assignable to declared type "Mapping[str
    "Movie" is not assignable to "Mapping[str, str | int]"
      Type parameter "_VT_co@Mapping" is covariant, but "object" is not a subtype of "str |
        Type "object" is not assignable to type "str | int"
          "object" is not assignable to "str"
          "object" is not assignable to "int" (reportAssignmentType)
  extra_items.py:49:7 - error: Base class "ClosedMovie" is a TypedDict that limits the type
    Cannot add item "rating" (reportIncompatibleVariableOverride)
7 errors, 0 warnings, 0 informations
```

Reading the output

- **Runtime only records the flags.** `ClosedMovie.__closed__` is `True` and `Scores.__extra_items__` is `int`. An open class reports `__closed__=None` and the sentinel `typing.NoExtraItems`. The wrong extra `"three"` is accepted at runtime, so PEP 728 is a *static* contract.
- **Exactly the seven marked lines fail in pyright, and nothing else does.** It reports the closed literal with a stray `"director"` key (shown as extra type `Never`), the non-`int` extra in a literal and in an assignment, the write to a `ReadOnly` extra, `Scores` passed where `Mapping[str, int]` is expected (because `"player"` is `str`), the open `Movie` widened to `Mapping[str, object]`, and the subclass adding `rating` to a closed base.
- **Two `ok` lines are the payoff.** `total(i)` type-checks because every value of `IntOnly` is `int`. `ClosedMovie` is accepted as `Mapping[str, str | int]` because closing it pins down every possible value. Neither was expressible before 3.15.
- **`del b["round1"]` is fine** because extra items are never required.

Pitfalls

- **Checker support varies; check yours.** The same file under mypy 2.4.1 understands `closed=True`, but rejects `extra_items` as an unknown class argument. That leads to false positives on every `ok` line that uses extras (16 errors):

```
$ mypy --python-version 3.15 extra_items.py
extra_items.py:15: error: Unexpected keyword argument "extra_items" for "__init_subclass__"
extra_items.py:19: error: Unexpected keyword argument "extra_items" for "__init_subclass__"
extra_items.py:24: error: Extra key "director" for TypedDict "ClosedMovie"  [typeddict-unkno
```

```
extra_items.py:25: error: Extra keys ("round1", "round2") for TypedDict "Scores"  [typeddict
extra_items.py:26: error: Extra key "round1" for TypedDict "Scores"  [typeddict-unknown-key]
extra_items.py:27: error: TypedDict "Scores" has no key "bonus"  [typeddict-unknown-key]
extra_items.py:28: error: TypedDict "Scores" has no key "note"  [typeddict-unknown-key]
extra_items.py:29: error: Extra key "color" for TypedDict "Labels"  [typeddict-unknown-key]
extra_items.py:30: error: TypedDict "Labels" has no key "color"  [typeddict-unknown-key]
extra_items.py:31: error: TypedDict "Scores" has no key "round1"  [typeddict-item]
extra_items.py:38: error: Unexpected keyword argument "extra_items" for "__init_subclass__"
extra_items.py:43: error: Argument 1 to "total" has incompatible type "IntOnly"; expected "M
extra_items.py:44: error: Argument 1 to "total" has incompatible type "Scores"; expected "Ma
extra_items.py:45: error: Incompatible types in assignment (expression has type "ClosedMovie
extra_items.py:46: error: Incompatible types in assignment (expression has type "Movie", var
extra_items.py:50: error: Cannot extend closed base class "ClosedMovie" with new field "rati
Found 16 errors in 1 file (checked 1 source file)
```

For the general trade-offs between checkers, see Choosing a Type Checker. If you need to support older interpreters, `typing_extensions.TypedDict` accepts the same parameters. - **Closed doesn't mean validated.** Data from JSON is still a plain `dict` at runtime. Validate it at the boundary with a validation library or with your own checks before you trust it. - **Don't combine `closed` and `extra_items`.** That raises an error at class creation, and the combination is meaningless anyway, because `closed=True` *is* `extra_items=Never`. - **Closing is a public-API commitment.** Downstream subclasses can no longer add keys. Close payload types you own, and leave extension points open.

Exercises

1. Make `Movie` a subclass with `extra_items=int`, then add a declared key `rating: float` in a sub-subclass. What does pyright say, and which PEP rule explains it?
2. Write `def merge(a: Scores, b: Scores) -> Scores` that uses `a | b`. Does it type-check?
3. Replace `ReadOnly[str]` in `Labels` with `str` and re-run pyright. Which error disappears?
4. Read `__closed__` and `__extra_items__` to write a tiny runtime validator `check(td, value) -> list[str]` that reports stray or mistyped extra keys for `int`/`str` extras.

Sources

- PEP 728 – TypedDict with Typed Extra Items (Final, Python 3.15).
- What's New In Python 3.15 (lists PEP 728 under new typing features).
- `typing.TypedDict` reference; PEP 705 – `ReadOnly`.
- Typing spec, TypedDict.
- Astral, "Python 3.15" (overview including `closed` and `extra_items`).

Disjoint Bases: Telling Checkers Which Classes Can't Mix (PEP 800)

Disjoint Bases: Telling Checkers Which Classes Can't Mix (PEP 800)

Thesis

Some classes can never share a subclass. `class X(int, str)` fails at class-creation time with `TypeError: multiple bases have instance lay-out conflict`, because CPython lays out an `int` and a `str` differently in memory and one object can't be both. Type checkers didn't know this. So a branch like `if isinstance(n, str):` with `n: int` looked reachable to some checkers and unreachable to others, each using its own heuristic.

PEP 800 (Final, Python 3.15) writes the rule into the type system. A class is a **disjoint base** if it is decorated with the new `@typing.disjoint_base`, has a non-empty `__slots__`, or is `object`. Two disjoint bases that aren't parent and child can't both appear in one class's MRO. Checkers can then reject the impossible class *and* treat `isinstance` checks across such classes as unreachable. This chapter runs one file under CPython 3.15.0 and three checkers. All three agree on the class definitions, and they give three different answers on narrowing.

Mental model

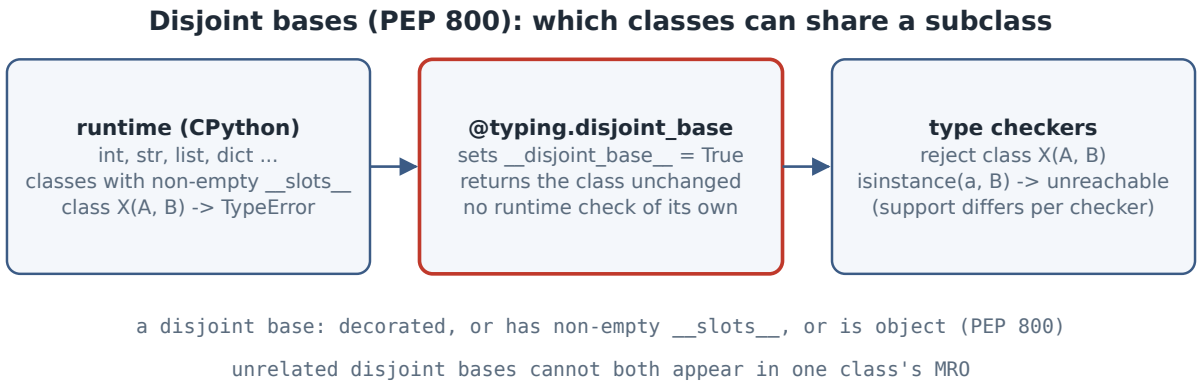


Figure 1: Runtime layout conflicts, the decorator, and what checkers do with it.

- **The runtime fact.** CPython calls these "solid bases", a concept dating back to 2001. Each instance has one C-level layout. Two bases that each add their own fields, such as a C struct or `__slots__`, can't be merged, so class creation raises `TypeError`.

- **The decorator.** `@disjoint_base` works like `@final` at runtime: it sets `cls.__disjoint_base__ = True` and returns the class. It doesn't check anything itself. It exists so typeshed stubs can mark C classes like `int`, and so authors can promise the property for their own classes.
- **Automatic disjoint bases.** A non-empty `__slots__`, including `@dataclass(slots=True)`, makes a class a disjoint base without the decorator.
- **What checkers do.** They reject a class whose bases include two unrelated disjoint bases. Because no such subclass can exist, they may also treat `isinstance(x, B)` as unreachable when `x: A`.

Worked example

`disjoint.py`, run with CPython 3.15.0:

```
"""PEP 800 (Python 3.15): telling type checkers which classes can never share a subclass."""
from typing import assert_never, disjoint_base


# 1) The runtime fact PEP 800 models: some C-level classes cannot be combined.
for bases in [(int, str), (int, list), (dict, list), (int, object)]:
    try:
        type("Mix", bases, {})
        print(f"class Mix{tuple(b.__name__ for b in bases)}: ok")
    except TypeError as e:
        print(f"class Mix{tuple(b.__name__ for b in bases)}: TypeError: {e}")


# 2) Pure-Python classes can declare the same property with the decorator.
@disjoint_base
class Vector:
    __slots__ = ("x", "y")


@disjoint_base
class Matrix:
    __slots__ = ("rows",)


class Plain:                    # not a disjoint base
    pass


def describe(v: Vector) -> str:
    if isinstance(v, Matrix):    # a checker that knows PEP 800 narrows v to Never here
        assert_never(v)
    if isinstance(v, Plain):     # possible: class VP(Vector, Plain) is allowed
        return "vector-and-plain"
```

```
    return "vector"


try:
    class Both(Vector, Matrix):  # a PEP 800 checker reports this class definition
        pass
except TypeError as e:           # ...and here the runtime agrees, because of the slots
    print("class Both(Vector, Matrix): TypeError:", e)


@disjoint_base
class TagA:                      # the decorator alone changes nothing at runtime
    pass


@disjoint_base
class TagB:
    pass


class TagAB(TagA, TagB):         # runtime allows it; a PEP 800 checker reports it
    pass


if __name__ == "__main__":
    print("marker:", getattr(Vector, "__disjoint_base__", None),
          getattr(Plain, "__disjoint_base__", None))
    print("TagAB created at runtime:", [c.__name__ for c in TagAB.__mro__])
```

Runtime output

```
$ python3.15 disjoint.py
class Mix('int', 'str'): TypeError: multiple bases have instance lay-out conflict
class Mix('int', 'list'): TypeError: multiple bases have instance lay-out conflict
class Mix('dict', 'list'): TypeError: multiple bases have instance lay-out conflict
class Mix('int', 'object'): ok
class Both(Vector, Matrix): TypeError: multiple bases have instance lay-out conflict
marker: True None
TagAB created at runtime: ['TagAB', 'TagA', 'TagB', 'object']
```

pyright 1.1.414

```
$ pyright --pythonversion 3.15 disjoint.py
disjoint.py
  disjoint.py:31:22 - error: Argument of type "<subclass of Vector and Matrix>" cannot be as
    Type "<subclass of Vector and Matrix>" is not assignable to type "Never" (reportArgument
  disjoint.py:38:11 - error: Base classes have incompatible disjoint bases: "Vector", "Matri
  disjoint.py:54:7 - error: Base classes have incompatible disjoint bases: "TagA", "TagB" (r
3 errors, 0 warnings, 0 informations
```

ty 0.0.95

```
$ ty check --python-version 3.15 disjoint.py
error[instance-layout-conflict]: Class will raise `TypeError` at runtime due to incompatible
  --> disjoint.py:38:11
   |
38 |     class Both(Vector, Matrix):  # a PEP 800 checker reports this class definition
   |           ^^^^^^^^^^^^^^^^^^^^ Bases `Vector` and `Matrix` cannot be combined in multip
info: Two classes cannot coexist in a class's MRO if their instances have incompatible memor
  --> disjoint.py:38:16
   |
38 |     class Both(Vector, Matrix):  # a PEP 800 checker reports this class definition
   |                ------  ------ `Matrix` instances have a distinct memory layout because
   |                |
   |                `Vector` instances have a distinct memory layout because of the way `Vec

error[instance-layout-conflict]: Class will raise `TypeError` at runtime due to incompatible
  --> disjoint.py:54:7
   |
54 | class TagAB(TagA, TagB):         # runtime allows it; a PEP 800 checker reports it
   |       ^^^^^^^^^^^^^^^^ Bases `TagA` and `TagB` cannot be combined in multiple inherita
info: Two classes cannot coexist in a class's MRO if their instances have incompatible memor
  --> disjoint.py:54:13
   |
54 | class TagAB(TagA, TagB):         # runtime allows it; a PEP 800 checker reports it
   |             ----  ---- `TagB` instances have a distinct memory layout because of the wa
   |             |
   |             `TagA` instances have a distinct memory layout because of the way `TagA` is

Found 2 diagnostics
```

mypy 2.4.1

```
$ mypy --python-version 3.15 --warn-unreachable disjoint.py
disjoint.py:30: error: Subclass of "Vector" and "Matrix" cannot exist: have distinct disjoin
disjoint.py:38: error: Class "Both" has incompatible disjoint bases  [misc]
disjoint.py:54: error: Class "TagAB" has incompatible disjoint bases  [misc]
Found 3 errors in 1 file (checked 1 source file)
```

Reading the output

- **Lines 6–11.** The runtime refuses `int`+`str`, `int`+`list` and `dict`+`list`. `int`+`object` is fine, because `object` is the parent of every disjoint base.
- **Line 38 (`Both`).** `Vector` and `Matrix` both have non-empty `__slots__`, so the runtime raises *and* all three checkers report the class. Here the decorator is redundant, since the slots alone make each class a disjoint base, but it records the intent.
- **Line 54 (`TagAB`).** `TagA` and `TagB` have no slots. The decorator is only a promise, so the runtime builds the class without complaint (`TagAB`'s MRO is printed). All three checkers still reject it. That is the point of the decorator: it states a constraint the checker enforces even where the runtime doesn't.
- **Lines 30–31 (narrowing).** This is where the checkers differ:
  - **ty** narrows `v` to `Never` inside the branch, so `assert_never(v)` passes with no diagnostic.
  - **mypy** (with `--warn-unreachable`) reports the branch as unreachable.
  - **pyright** 1.1.414 still infers `<subclass of Vector and Matrix>` there, so `assert_never` fails. It enforces the class-definition rule but doesn't (yet) use it for narrowing.
- **Line 32.** `Plain` isn't a disjoint base, so `isinstance(v, Plain)` stays reachable in every checker. A `class VP(Vector, Plain)` is legal.
- **Lines 59–60** read the marker with `getattr`. Direct access to `Vector.__disjoint_base__` type-checks as an unknown attribute in pyright and ty (tried during this run), because the stubs don't declare it.

Pitfalls

1. **Treating the decorator as enforcement.** `TagAB` shows that `@disjoint_base` on a plain class changes nothing at runtime. If you need the runtime to refuse mixing, give the class a non-empty `__slots__` (or a C layout). The decorator then just documents a fact the runtime already enforces.
2. **Trusting checker messages about the cause.** ty says `TagAB` "will raise `TypeError` at runtime" and that `TagA` "is implemented in a C extension". Both statements are false for this file: the class was created fine. The diagnostic is correct under PEP 800, but its explanation assumes the decorator matches the runtime.
3. **Expecting identical narrowing.** Code that relies on `assert_never` after an impossible `isinstance` passes in ty, is flagged as dead code in mypy (only with `--warn-unreachable`), and fails in pyright 1.1.414. If you support several checkers, don't write `assert_never` there. Write a plain `raise AssertionError`, or simply drop the branch.

4. **Forgetting `object`.** `object` counts as a disjoint base, but it's an ancestor of every class, so it never conflicts. Only *unrelated* disjoint bases clash. A subclass of `Vector` can be combined with `Vector`.
5. **Adding `__slots__` changes a public class.** Since a non-empty `__slots__` makes a class a disjoint base, adding slots to a library class can break users' multiple inheritance, both statically (PEP 800) and at runtime. Treat it as an API change.

Exercises

1. Remove `__slots__` from `Matrix` (keep the decorator). Predict the runtime output for `Both` and each checker's output, then run them.
2. Replace the `isinstance(v, Matrix)` branch with `if isinstance(v, int):`. Do the checkers treat it as unreachable without any decorator? Why might typeshed's markings on `int` matter here?
3. Convert `Vector` to `@dataclass(slots=True)` without the decorator and confirm each checker still treats it as a disjoint base.
4. Write a small runtime check `can_combine(a: type, b: type) -> bool` that tries `type("_", (a, b), {})`. Compare its answers with `getattr(cls, "__disjoint_base__", False)` for `Vector`, `TagA`, `int` and `Plain`.

Sources

- PEP 800, "Disjoint bases in the type system" (Final, Python 3.15): https://peps.python.org/pep-0800/
- `typing.disjoint_base` documentation: https://docs.python.org/3.15/library/typing.html#typing.disjoint_base
- What's New in Python 3.15, typing section: https://docs.python.org/3.15/whatsnew/3.15.html
- Typing spec (canonical, maintained): https://typing.python.org/en/latest/spec/
- pyright: https://github.com/microsoft/pyright; ty: https://docs.astral.sh/ty/; mypy: https://mypy.readthedocs.io/

TypeForm: Typing Functions That Take Types as Values (PEP 747)

TypeForm: Typing Functions That Take Types as Values (PEP 747)

Thesis

Validators, converters and runtime checkers all take a *type* as an ordinary argument: `TypeAdapter(int | str)`, `check_type(value, list[int])`, `structure(data, Literal["r", "w"])`. Until Python 3.15 there was no honest annotation for that parameter. `type[T]` only admits class objects, so `int | None` and `Literal["r"]` are rejected. `object` admits everything and throws away the `T`, so the return type comes back as `Any` or `Unknown`. PEP 747 adds `typing.TypeForm[T]`, which is the type of *the runtime value of any valid type expression that spells a type assignable to T*. **The checker can now solve `T` from the form you pass, and the runtime function receives exactly the object it always did.** This chapter builds a small runtime checker on top of it and runs mypy and pyright against both the happy path and the cases the PEP says must be rejected.

Mental model

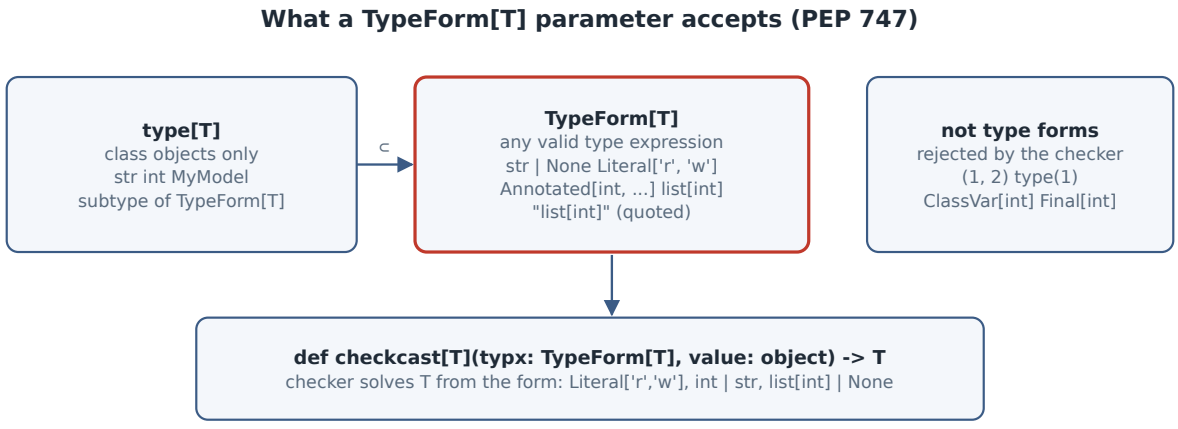


Figure 1: type[T] is the class-objects-only corner of TypeForm[T]; anything that is not a type expression is outside both.

There are three rules to keep in mind:

1. **What is a type form?** It is whatever a valid *type expression* evaluates to at runtime: a class, a `types.UnionType`, a `typing._LiteralGenericAlias`, a `GenericAlias`, `None`, or even

a string such as `"set[str]"`. `TypeForm[T]` is the set of such objects whose type is assignable to `T`. So `TypeForm[str | None]` accepts `str`, `None`, `Optional[str]` and `"str | None"`, but not `str | int` and not `list[str | None]`.

2. **The parameter is covariant, and `type[T]` is a subtype.** `TypeForm[int]` is assignable to `TypeForm[int | str]`. Every existing `type[C]` value is already a `TypeForm[C]`, so widening a parameter from `type[T]` to `TypeForm[T]` breaks no callers.
3. **It is a checker-side promise.** The checker validates the expression with the same grammar it uses for annotations, so a call like `type(1)`, a tuple or a qualifier such as `ClassVar[int]` is rejected. At runtime nothing changes. `TypeForm(x)` simply returns `x`, and your function still has to interpret the object with `get_origin` and `get_args`.

`TypeForm` without arguments means `TypeForm[Any]`. `TypeForm[object]` is "any valid type form at all". Introspection helpers like `typing.get_origin` keep their `object` parameter, because they also accept *annotation* expressions (`ClassVar[int]`, `Final`) that are not type expressions.

Worked example

The lab ran on CPython 3.15.0 with mypy 2.4.1 (where `TypeForm` is listed under `COMPLETE_FEATURES`, so no flag is needed), pyright 1.1.414 and pydantic 2.14.2.

A runtime checker with an honest signature

`is_assignable` returns `TypeIs[T]`, so a successful check narrows the argument to the type the form spells. `checkcast` returns `T` or raises. The body is ordinary runtime introspection. Only the two signatures depend on PEP 747.

```
"""A tiny runtime checker whose signature finally says what it does: PEP 747 TypeForm (Pytho
import types
from typing import Annotated, Any, Literal, TypeForm, TypeIs, Union, get_args, get_origin

def is_assignable[T](value: object, typx: TypeForm[T]) -> TypeIs[T]:
    """Does `value` inhabit the type that `typx` spells? Handles the common forms."""
    if isinstance(typx, str):                       # "list[int]": a quoted type expression
        typx = eval(typx, vars(__import__("typing")) | vars(__import__("builtins")))
    if typx is Any or typx is object: return True
    if typx is None or typx is type(None): return value is None
    origin, args = get_origin(typx), get_args(typx)
    if origin is Annotated: return is_assignable(value, args[0])
    if origin in (Union, types.UnionType): return any(is_assignable(value, a) for a in args)
    if origin is Literal: return any(value == a and type(value) is type(a) for a in args)
    if origin is list: return isinstance(value, list) and all(is_assignable(v, args[0]) for
    if origin is dict:
        return isinstance(value, dict) and all(is_assignable(k, args[0]) and is_assignable(v
    if origin is tuple:
        if len(args) == 2 and args[1] is Ellipsis:
```

```
            return isinstance(value, tuple) and all(is_assignable(v, args[0]) for v in value
        return isinstance(value, tuple) and len(value) == len(args) and all(map(is_assignabl
    if isinstance(typx, type): return isinstance(value, typx) and not (typx is int and isins
    raise TypeError(f"unsupported type form: {typx!r}")

def checkcast[T](typx: TypeForm[T], value: object) -> T:
    if is_assignable(value, typx): return value
    raise TypeError(f"{value!r} is not assignable to {typx!r}")
```

Using it: what the checkers infer

```
from typing import Annotated, Literal, TypeForm, reveal_type
from pydantic import TypeAdapter
from typeforms import checkcast, is_assignable

def handle(raw: object) -> None:
    if is_assignable(raw, list[int] | None):         # TypeIs narrows to the type the form s
        reveal_type(raw)
    mode = checkcast(Literal["r", "w"], "r")
    reveal_type(mode)
    port = checkcast(Annotated[int, "1-65535"], 8080)
    reveal_type(port)

handle([1, 2, 3])

# Explicit evaluation: TypeForm(...) returns its argument unchanged at runtime, but the chec
spec = TypeForm(dict[str, tuple[int, ...]])
reveal_type(spec)
print(spec, is_assignable({"a": (1, 2)}, spec), is_assignable({"a": [1, 2]}, spec))

# Pydantic 2.14 annotates TypeAdapter.__init__ with TypeForm[T], so the validated result is
ta = TypeAdapter(int | str)
reveal_type(ta.validate_python("7"))
for v in [[1, 2], None, [1, "x"], True, (1,)]:
    print(f"{v!r:10} list[int] | None -> {is_assignable(v, list[int] | None)}")
print(is_assignable(True, int), is_assignable(1, Literal[True]), is_assignable("w", "Literal
```

Running it under CPython 3.15 (`reveal_type` prints the *runtime* type to stderr, and here that is a plain `list`, `str` and `GenericAlias`):

```
$ python3.15 demo.py
Runtime type is 'list'
Runtime type is 'str'
Runtime type is 'int'
Runtime type is 'GenericAlias'
```

```
Runtime type is 'str'
dict[str, tuple[int, ...]] True False
[1, 2]     list[int] | None -> True
None       list[int] | None -> True
[1, 'x']   list[int] | None -> False
True       list[int] | None -> False
(1,)       list[int] | None -> False
False False True
```

mypy, on the same file plus the error file below (`--strict`):

```
$ mypy --python-version 3.15 --strict demo.py errors.py typeforms.py
demo.py:7: note: Revealed type is "list[int] | None"
demo.py:9: note: Revealed type is "Literal['r'] | Literal['w']"
demo.py:11: note: Revealed type is "int"
demo.py:17: note: Revealed type is "TypeForm[dict[str, tuple[int, ...]]]"
demo.py:22: note: Revealed type is "int | str"
errors.py:7: error: Argument 1 to "trycast_old" has incompatible type "UnionType | type[str]
errors.py:8: error: Argument 1 to "trycast_old" has incompatible type "<typing special form>
errors.py:11: error: Incompatible types in assignment (expression has type "TypeForm[str | i
errors.py:12: error: Incompatible types in assignment (expression has type "tuple[int, int]"
errors.py:14: error: TypeForm argument is not a type  [misc]
errors.py:15: error: Incompatible types in assignment (expression has type "str", variable h
Found 6 errors in 1 file (checked 3 source files)
```

pyright (absolute paths trimmed; `--pythonpath` points at the venv so `pydantic` resolves):

```
$ pyright --pythonpath venv/bin/python --pythonversion 3.15 demo.py errors.py typeforms.py
demo.py
  demo.py:7:21 - information: Type of "raw" is "list[int] | None"
  demo.py:9:17 - information: Type of "mode" is "Literal['r', 'w']"
  demo.py:11:17 - information: Type of "port" is "int"
  demo.py:17:13 - information: Type of "spec" is "TypeForm[dict[str, tuple[int, ...]]]"
  demo.py:22:13 - information: Type of "ta.validate_python("7")" is "int | str"
errors.py
  errors.py:7:13 - error: Argument of type "UnionType" cannot be assigned to parameter "typx
    Type "UnionType" is not assignable to type "type[T@trycast_old]" (reportArgumentType)
  errors.py:8:13 - error: Argument of type "type[Literal['hi']]" cannot be assigned to param
  errors.py:11:30 - error: Type "UnionType" is not assignable to declared type "TypeForm[str
    "UnionType" is not assignable to "TypeForm[str | None]" (reportAssignmentType)
  errors.py:12:26 - error: Type "tuple[Literal[1], Literal[2]]" is not assignable to declare
    "tuple[Literal[1], Literal[2]]" is not assignable to "TypeForm[object]" (reportAssignmen
  errors.py:13:26 - error: "ClassVar" is not allowed in this context (reportInvalidTypeForm)
  errors.py:14:17 - error: type() call should not be used in type expression
    Use type[T] instead (reportInvalidTypeForm)
  errors.py:14:17 - error: Call expression not allowed in type expression (reportInvalidType
  errors.py:15:10 - error: Type "str" is not assignable to declared type "int"
```

```
    "str" is not assignable to "int" (reportAssignmentType)
8 errors, 0 warnings, 5 informations
```

The cases that must fail

```
from typing import ClassVar, Literal, TypeForm
from typeforms import checkcast

def trycast_old[T](typx: type[T], value: object) -> T | None: ...

trycast_old(str, "hi")                      # fine: a class is a type[T]
trycast_old(str | None, "hi")               # rejected: a union is not a class
trycast_old(Literal["hi"], "hi")            # rejected: Literal is not a class

ok: TypeForm[str | None] = str              # str is assignable to str | None
bad1: TypeForm[str | None] = str | int      # int is not
bad2: TypeForm[object] = (1, 2)                     # a tuple expression is not a type expres
bad3: TypeForm[object] = ClassVar[int]              # qualifiers are not types
bad4 = TypeForm(type(1))                    # calls are not allowed in type expressions
n: int = checkcast(str, "x")                # the form says str, so the result is str
```

Reading it

- **Inference flows from the form.** Both checkers solve `T = list[int] | None` from the union, `Literal['r', 'w']` from the literal, and `int` from `Annotated[int, "1-65535"]`, because metadata is ignored, as the PEP permits. mypy prints the literal as `Literal['r'] | Literal['w']` and pyright prints it as `Literal['r', 'w']`. They are the same type.
- **Pydantic gets it for free.** `TypeAdapter.__init__` is now annotated `type: TypeForm[T]`, so `ta.validate_python("7")` is `int | str` in both checkers. This is the change the pydantic 2.14 release notes call out: on 2.13 the result was `Unknown`. One caveat is that when pyright could not import pydantic (an earlier run without `--pythonpath`), the result silently fell back to `Unknown`. Always check that the library actually resolves.
- **`type[T]` really is too narrow.** On lines 7 and 8 both checkers reject `str | None` and `Literal["hi"]` for the old `type[T]` signature. That is the exact motivating example of the PEP.
- **Assignability is about the *spelled* type.** `TypeForm[str | None] = str` is accepted, because `str` is assignable to `str | None`. `= str | int` is rejected (line 11). Line 15 shows the payoff of a generic signature: `checkcast(str, ...)` returns `str`, so assigning it to `int` is an error.
- **The checkers do not agree on everything yet.** pyright rejects `ClassVar[int]` as a type form (line 13, `reportInvalidTypeForm`). mypy 2.4.1 says nothing about it, even though the PEP lists `ClassVar[int]` among its invalid examples. Both reject `TypeForm(type(1))`. pyright emits two diagnostics (a `type()` call and "Call expression not allowed in type expression"), mypy says "TypeForm argument is not a type". If your library depends on rejection, test it under every checker your users run.

- **The runtime is the same as before.** `TypeForm(dict[str, tuple[int, ...]])` printed `dict[str, tuple[int, ...]]`, the very object passed in. The runtime column also shows the bugs a `TypeForm` signature does **not** catch. `True` is an `int` to `isinstance`, which is why the lab excludes `bool` explicitly. `Literal[True]` must not match `1`, which is why it compares `type(value) is type(a)`. Your interpreter of type forms still needs its own tests.

Pitfalls

- **Do not annotate introspection helpers with `TypeForm`.** If the function must accept `ClassVar[int]`, `Final`, `Required[...]` or `InitVar`, those are annotation expressions, not type expressions. Keep `object` there, as `typing.get_args` does.
- **Strings are type forms, but you have to evaluate them.** `"list[int]"` is a valid `TypeForm[list[int]]` to the checker, but at runtime it is just a `str`. The lab `eval`s it in a `typing` + `builtins` namespace. Real libraries resolve it in the caller's module namespace, which is the hard part (see `typing.evaluate_forward_ref` / `annotationlib` in 3.15). Never `eval` untrusted input.
- **`TypeForm` is not `type`.** You cannot call `typx()` or use `isinstance(x, typx)` on a `TypeForm[T]`, because a union or literal is not a class. The checker treats it as having only `object`'s attributes. Branch on `get_origin` first and only call `isinstance` once you know it is a `type`.
- **Older Pythons.** `typing.TypeForm` is new in 3.15. For 3.10–3.14 import it from `typing_extensions`, which is what pydantic does, and pin a checker new enough to understand it. The pydantic notes say pyright 1.1.412 and mypy 2.2 or later.
- **Bare `TypeForm` under `--strict`.** mypy reports `Missing type arguments for generic type "TypeForm"`. Write `TypeForm[object]` when you mean "any type form".

Try this

1. Add `TypedDict` support to `is_assignable` (use `typing.is_typeddict` and `__required_keys__`). Check that `reveal_type` after a successful check shows your `TypedDict` in both checkers.
2. Write `class Field[T]: def __init__(self, typx: TypeForm[T], default: T) -> None`. Confirm that `Field(int | None, "x")` is rejected and `Field(int | None, None)` is accepted.
3. Change `checkcast`'s parameter back to `type[T]` and record every line of `demo.py` that each checker now flags.
4. Make the lab fail on `ClassVar[int]` at runtime as well, by rejecting `get_origin(typx) is ClassVar`. Then decide whether a runtime check should be stricter than mypy.

Sources

- PEP 747 – Annotating Type Forms (Final, Python 3.15): https://peps.python.org/pep-0747/. Source text: https://github.com/python/peps/blob/main/peps/pep-0747.rst
- Typing spec, type expressions grammar: https://typing.python.org/en/latest/spec/annotations.html

- `typing.TypeForm` reference: https://docs.python.org/3.15/library/typing.html#typing.TypeForm
- Pydantic v2.14 release notes, "Support for TypeForm": https://pydantic.dev/articles/pydantic-v2-14-release
- mypy documentation: https://mypy.readthedocs.io/en/stable/. pyright configuration (`reportInvalidTypeForm`): https://github.com/microsoft/pyright/blob/main/docs/configuration.md

TypeVarTuple Array Shapes

TypeVarTuple Array Shapes

Thesis

A type variable stands for one type. A **type variable tuple** (`TypeVarTuple`) stands for a tuple of types. You unpack it with `*` or with `typing.Unpack`. PEP 646 added that tool in Python 3.11 so an array can be generic in its dtype and in every axis: `Array[DType, *Shape]`. PEP 695 (Python 3.12) adds the spelling `class Array[DType, *Shape]`. The integers in a runtime `.shape` are a separate object. A type checker compares the type arguments. CPython runs the function either way.

On Python 3.15 the constructor grew four keyword arguments: `bound`, `covariant`, `contravariant`, and `infer_variance`. What's New in Python 3.15 says the meaning of `bound` is still undefined in the typing specification. mypy 2.4.0 and pyright 1.1.414 still reject those keywords. Case 8 runs the constructor and both checkers. Shape checks of the kind in this chapter have been expressible since 3.11.

NumPy 2.5.4 and PyTorch 2.14.1, in the stubs installed here, still describe a shape as one tuple type. NumPy's `ndarray` is `Generic[_ShapeT_co, _DTypeT_co]`. `_ShapeT_co` is one type variable bound to `tuple[int, ...]`, defaulting to `tuple[Any, ...]`. `numpy.typing.NDArray` evaluates to `numpy.ndarray[_AnyShape, numpy.dtype[ScalarT]]` with `_AnyShape` equal to `tuple[Any, ...]`, so the public alias keeps a dtype and drops rank. PyTorch's `torch.__version__` is `2.14.1+cu130`. In that stub `Tensor.shape` is `Size`, and `Size` subclasses `tuple[int, ...]`. Those facts come from reading the stubs. This chapter does not type-check either library. The class below is the small PEP 646 array, so the rule is visible on its own.

Every listing ran on CPython 3.15.0. `python3.15 --version` prints `Python 3.15.0`. `sys.version` on this machine is `3.15.0 (main, Oct  9 2026, 15:24:40) [Clang 22.1.3 ]`. Checker commands use that same 3.15.0 with mypy 2.4.0 (`python3.15 -m mypy --version` prints `mypy 2.4.0 (compiled: yes)`) and pyright 1.1.414. Pyright is `python3.15 -m pyright --pythonversion 3.15`. It prints the absolute path of the file. These runs lived in `/tmp/py-shapes-lab/`.

Mental model

```
Array[DType, *Shape]
      |        +-- zero or more axis types, always written unpacked
      +-- exactly one dtype

type arguments                                runtime value
Array[float, Literal[3], Literal[3]]          .shape is some tuple[int, ...]
Array[float, int]                             one axis, and that axis's type is int
```

```
Array[float, int, int]                    two axes, each typed int
Array[float, *tuple[int, ...]]            any rank, each axis typed int
```

Rank here means how many axis types you wrote. It is the length of the type-argument tuple, which a checker can see. It is not `len(array.shape)` unless you write that test yourself.

Four rules. The listings below are the ones that go wrong in array code:

- Unpack a `TypeVarTuple`. The `TypeVarTuple` docs require `tuple[*Ts]`. A bare `Ts` in that slot is invalid.
- One unpacked `TypeVarTuple` per parameter list and per type-argument list. A second one is a checker error. CPython still builds the function.
- `*args: *Ts` gives each positional argument its own slot in `Ts`. `*args: int` means every argument is `int`.
- `*tuple[int, ...]` and `Unpack[tuple[int, ...]]` are one type to mypy and pyright. `get_args` on the two aliases does not return equal objects.

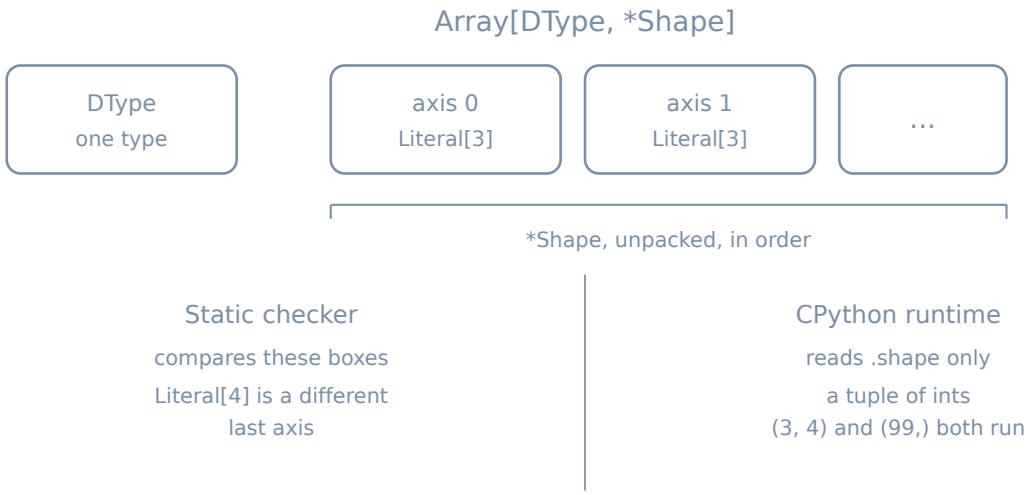


Figure 1: DType, unpacked axis types, and the runtime shape tuple

Worked examples

Case 1: A dtype and a tuple of axis types

`relu` promises the same dtype and the same axis types out as in. The body builds a new `Array` and copies `.shape`. Nothing in `Array.__init__` consults `DType` or `Shape`.

```
# array_relu.py - dtype plus a variadic shape, at runtime
from typing import get_args


class Array[DType, *Shape]:
```

```
    def __init__(self, values: list[float], shape: tuple[int, ...]) -> None:
        self.values = values
        self.shape = shape

    def __repr__(self) -> str:
        return f"Array(shape={self.shape}, n={len(self.values)})"


def relu[DType, *Shape](x: Array[DType, *Shape]) -> Array[DType, *Shape]:
    return Array([v if v > 0.0 else 0.0 for v in x.values], x.shape)


def main() -> None:
    image: Array[float, int, int] = Array([1.0, -2.0, 3.0, -4.0], (2, 2))
    print(relu(image))
    alias = Array[float, int, int]
    print("args", get_args(alias))
    print("params", tuple(p.__name__ for p in Array.__type_params__))
    print("kinds", tuple(type(p).__name__ for p in Array.__type_params__))


if __name__ == "__main__":
    main()
```

```
$ python3.15 array_relu.py
Array(shape=(2, 2), n=4)
args (<class 'float'>, <class 'int'>, <class 'int'>)
params ('DType', 'Shape')
kinds ('TypeVar', 'TypeVarTuple')
```

`get_args(Array[float, int, int])` is the dtype and then one entry per axis. The class's own type parameters are a `TypeVar` named `DType` and a `TypeVarTuple` named `Shape`. The printed object is the values and the shape tuple. The type arguments showed up only on the alias. `relu` copied `.shape`, so the new object still prints `(2, 2)`. That copy is ordinary Python.

Both checkers accept the file.

```
$ python3.15 -m mypy array_relu.py
Success: no issues found in 1 source file
```

```
$ python3.15 -m pyright --pythonversion 3.15 array_relu.py
0 errors, 0 warnings, 0 informations
```

Case 2: A square the checker accepts

`Literal[3]` is a type with one inhabitant, the integer 3. Two of them are two axis types. `square` asks for that pair and returns it. `relu` is shape-preserving, so the checker lets the result flow back out.

```
# shape_ok.py
from typing import Literal


class Array[DType, *Shape]:
    def __init__(self, values: list[float], shape: tuple[int, ...]) -> None:
        self.values = values
        self.shape = shape


def relu[DType, *Shape](x: Array[DType, *Shape]) -> Array[DType, *Shape]:
    return Array([v if v > 0.0 else 0.0 for v in x.values], x.shape)


def square(x: Array[float, Literal[3], Literal[3]]) -> Array[float, Literal[3], Literal[3]]:
    return relu(x)


def main() -> None:
    kernel: Array[float, Literal[3], Literal[3]] = Array(
        [1.0, -1.0, 0.0, 2.0, -2.0, 0.5, 0.0, 0.0, 3.0],
        (3, 3),
    )
    out = square(kernel)
    print(out.shape)


if __name__ == "__main__":
    main()
```

```
$ python3.15 shape_ok.py
(3, 3)
```

```
$ python3.15 -m mypy shape_ok.py
Success: no issues found in 1 source file
```

```
$ python3.15 -m pyright --pythonversion 3.15 shape_ok.py
0 errors, 0 warnings, 0 informations
```

Case 3: The same rank, a different last axis

`Array[float, Literal[3], Literal[4]]` is still rank 2. The last axis type is `Literal[4]`. `square` requires `Literal[3]`. CPython prints the real shape and exits 0. Both checkers reject the call. Pyright adds the reason: `Shape` on this class is invariant, so `*tuple[Literal[3], Literal[4]]` is not the same pack as `*tuple[Literal[3], Literal[3]]`.

```
# shape_wrong.py
from typing import Literal


class Array[DType, *Shape]:
    def __init__(self, values: list[float], shape: tuple[int, ...]) -> None:
        self.values = values
        self.shape = shape


def square(x: Array[float, Literal[3], Literal[3]]) -> None:
    print(len(x.values), x.shape)


def main() -> None:
    wide: Array[float, Literal[3], Literal[4]] = Array([0.0] * 12, (3, 4))
    square(wide)


if __name__ == "__main__":
    main()
```

```
$ python3.15 shape_wrong.py
12 (3, 4)
```

```
$ python3.15 -m mypy shape_wrong.py
shape_wrong.py:17: error: Argument 1 to "square" has incompatible type "Array[float, Literal
Found 1 error in 1 file (checked 1 source file)
```

```
$ python3.15 -m pyright --pythonversion 3.15 shape_wrong.py
/tmp/py-shapes-lab/shape_wrong.py
  /tmp/py-shapes-lab/shape_wrong.py:17:12 - error: Argument of type "Array[float, Literal[3]
    "Array[float, Literal[3], Literal[4]]" is not assignable to "Array[float, Literal[3], Li
      Type parameter "Shape@Array" is invariant, but "*tuple[Literal[3], Literal[4]]" is not
1 error, 0 warnings, 0 informations
```

Case 4: Rank is the length of the pack

`Array[float, int]` is one axis. `Array[float, int, int]` is two. The axis type is `int` in both, which means "some integer extent", and it says nothing about the number 3 stored in `.shape`.

```
# rank_wrong.py
class Array[DType, *Shape]:
    def __init__(self, values: list[float], shape: tuple[int, ...]) -> None:
        self.values = values
        self.shape = shape


def need_matrix(x: Array[float, int, int]) -> None:
    print(x.shape)


def main() -> None:
    vector: Array[float, int] = Array([1.0, 2.0, 3.0], (3,))
    need_matrix(vector)


if __name__ == "__main__":
    main()
```

```
$ python3.15 rank_wrong.py
(3,)
```

```
$ python3.15 -m mypy rank_wrong.py
rank_wrong.py:14: error: Argument 1 to "need_matrix" has incompatible type "Array[float, int
Found 1 error in 1 file (checked 1 source file)
```

```
$ python3.15 -m pyright --pythonversion 3.15 rank_wrong.py
/tmp/py-shapes-lab/rank_wrong.py
  /tmp/py-shapes-lab/rank_wrong.py:14:17 - error: Argument of type "Array[float, int]" canno
    "Array[float, int]" is not assignable to "Array[float, int, int]"
      Type parameter "Shape@Array" is invariant, but "*tuple[int]" is not the same as "*tupl
1 error, 0 warnings, 0 informations
```

Pyright's detail line is the whole rule in one place: `*tuple[int]` is not `*tuple[int, int]`.

Case 5: Concatenate an axis, or move one to the end

Concatenating shapes, in this type system, means building a new type-argument list. `prepend_axis` puts a fresh `int` in front of `*Shape`. `move_first_axis_to_last` is the standard library's tuple trick from the `TypeVarTuple` docs, applied to axes: peel off `Axis0`, keep `*Rest`, and write `*Rest` then `Axis0`.

The bodies do the same thing to the integer tuple. That agreement is the author's. The checker is looking at the annotations.

```
# concat_ok.py
from typing import Literal


class Array[DType, *Shape]:
    def __init__(self, values: list[float], shape: tuple[int, ...]) -> None:
        self.values = values
        self.shape = shape

    def __repr__(self) -> str:
        return f"Array(shape={self.shape})"


def prepend_axis[DType, *Shape](x: Array[DType, *Shape]) -> Array[DType, int, *Shape]:
    return Array(x.values, (1, *x.shape))


def move_first_axis_to_last[DType, Axis0, *Rest](
    x: Array[DType, Axis0, *Rest],
) -> Array[DType, *Rest, Axis0]:
    return Array(x.values, (*x.shape[1:], x.shape[0]))


def main() -> None:
    image: Array[float, Literal[2], Literal[3]] = Array([0.0] * 6, (2, 3))
    batched: Array[float, int, Literal[2], Literal[3]] = prepend_axis(image)
    moved: Array[float, Literal[3], Literal[2]] = move_first_axis_to_last(image)
    print(batched)
    print(moved)


if __name__ == "__main__":
    main()
```

```
$ python3.15 concat_ok.py
Array(shape=(1, 2, 3))
Array(shape=(3, 2))
```

```
$ python3.15 -m mypy concat_ok.py
Success: no issues found in 1 source file
```

```
$ python3.15 -m pyright --pythonversion 3.15 concat_ok.py
0 errors, 0 warnings, 0 informations
```

`(2, 3)` becomes `(1, 2, 3)` because the function prepended the integer `1`, and it becomes `(3, 2)` because the function rotated the tuple. The annotations say the result types are `Array[float, int, Literal[2], Literal[3]]` and `Array[float, Literal[3], Literal[2]]`. Both checkers accept those two assignments.

Claiming the rotated value still has axes `(Literal[2], Literal[3])` is a type error. The program still prints `(3, 2)`.

```
# concat_wrong.py
from typing import Literal


class Array[DType, *Shape]:
    def __init__(self, values: list[float], shape: tuple[int, ...]) -> None:
        self.values = values
        self.shape = shape


def move_first_axis_to_last[DType, Axis0, *Rest](
    x: Array[DType, Axis0, *Rest],
) -> Array[DType, *Rest, Axis0]:
    return Array(x.values, (*x.shape[1:], x.shape[0]))


def main() -> None:
    image: Array[float, Literal[2], Literal[3]] = Array([0.0] * 6, (2, 3))
    moved = move_first_axis_to_last(image)
    same: Array[float, Literal[2], Literal[3]] = moved
    print(same.shape)


if __name__ == "__main__":
    main()
```

```
$ python3.15 concat_wrong.py
(3, 2)
```

```
$ python3.15 -m mypy concat_wrong.py
concat_wrong.py:20: error: Incompatible types in assignment (expression has type "Array[floa
Found 1 error in 1 file (checked 1 source file)
```

```
$ python3.15 -m pyright --pythonversion 3.15 concat_wrong.py
/tmp/py-shapes-lab/concat_wrong.py
  /tmp/py-shapes-lab/concat_wrong.py:20:50 - error: Type "Array[float, Literal[3], Literal[2
    "Array[float, Literal[3], Literal[2]]" is not assignable to "Array[float, Literal[2], Li
      Type parameter "Shape@Array" is invariant, but "*tuple[Literal[3], Literal[2]]" is not
```

```
1 error, 0 warnings, 0 informations
```

Inside the generic function the two packs stay different even when a later call would fill both with `int`. Returning the input unchanged is rejected, and the call still runs:

```
# rotate_erased.py - identical axis types hide a missing rotation
class Array[DType, *Shape]:
    def __init__(self, values: list[float], shape: tuple[int, ...]) -> None:
        self.values = values
        self.shape = shape


def move_first_axis_to_last[DType, Axis0, *Rest](
    x: Array[DType, Axis0, *Rest],
) -> Array[DType, *Rest, Axis0]:
    return x


def main() -> None:
    image: Array[float, int, int] = Array([1.0, 2.0, 3.0, 4.0], (2, 2))
    out: Array[float, int, int] = move_first_axis_to_last(image)
    print(out.shape)


if __name__ == "__main__":
    main()
```

```
$ python3.15 rotate_erased.py
(2, 2)
```

```
$ python3.15 -m mypy rotate_erased.py
rotate_erased.py:11: error: Incompatible return value type (got "Array[DType, Axis0, *Rest]"
Found 1 error in 1 file (checked 1 source file)
```

```
$ python3.15 -m pyright --pythonversion 3.15 rotate_erased.py
/tmp/py-shapes-lab/rotate_erased.py
  /tmp/py-shapes-lab/rotate_erased.py:11:12 - error: Type "Array[DType@move_first_axis_to_la
    "Array[DType@move_first_axis_to_last, Axis0@move_first_axis_to_last, *Rest@move_first_ax
      Type parameter "Shape@Array" is invariant, but "*tuple[Axis0@move_first_axis_to_last,
1 error, 0 warnings, 0 informations
```

mypy reports `Array[DType, Axis0, *Rest]` against `Array[DType, *Rest, Axis0]`. The function has to be valid for every instantiation, including `Literal[2]` then `Literal[3]`, so the checker does not wait for a call site where both axes happen to be plain `int`.

You cannot type "concatenate two unknown shapes" with two type-variable tuples. That attempt is in the trap.

Case 6: `Unpack[tuple[int, ...]]` is any number of `int` axes

`*tuple[int, ...]` unpacks an open homogeneous tuple into the axis list. `Unpack[tuple[int, ...]]` is the spelling you need when the star is not available in that position, and it is the form older code wrote everywhere. On 3.15 both spellings are legal. `get_args` shows they are stored differently, and the alias objects compare unequal. A rank-2 `Array[float, int, int]` is still a legal argument to a parameter typed with the `Unpack` form, which is why both checkers accept this file.

```
# unpack_axes.py - star form and Unpack form of an open axis tuple
from typing import Unpack, get_args


class Array[DType, *Shape]:
    def __init__(self, values: list[float], shape: tuple[int, ...]) -> None:
        self.values = values
        self.shape = shape


def sum_values(x: Array[float, Unpack[tuple[int, ...]]]) -> float:
    return sum(x.values)


def main() -> None:
    star = Array[float, *tuple[int, ...]]
    unpacked = Array[float, Unpack[tuple[int, ...]]]
    print("star", get_args(star))
    print("unpack", get_args(unpacked))
    print("equal", star == unpacked)
    matrix: Array[float, int, int] = Array([1.0, 2.0, 3.0, 4.0], (2, 2))
    print("sum", sum_values(matrix))


if __name__ == "__main__":
    main()
```

```
$ python3.15 unpack_axes.py
star (<class 'float'>, *tuple[int, ...])
unpack (<class 'float'>, typing.Unpack[tuple[int, ...]])
equal False
sum 10.0
```

```
$ python3.15 -m mypy unpack_axes.py
Success: no issues found in 1 source file
```

```
$ python3.15 -m pyright --pythonversion 3.15 unpack_axes.py
0 errors, 0 warnings, 0 informations
```

An axis typed `str` is outside `tuple[int, ...]`. The sum still runs. mypy and pyright both print the expected type as `Array[float, *tuple[int, ...]]` even though the source wrote `Unpack[...]`. That is the checkers treating the two spellings as one type.

```
# unpack_wrong.py - a str axis is not an int axis
from typing import Unpack


class Array[DType, *Shape]:
    def __init__(self, values: list[float], shape: tuple[int, ...]) -> None:
        self.values = values
        self.shape = shape


def sum_values(x: Array[float, Unpack[tuple[int, ...]]]) -> float:
    return sum(x.values)


def main() -> None:
    named: Array[float, str] = Array([1.0], (1,))
    print(sum_values(named))


if __name__ == "__main__":
    main()
```

```
$ python3.15 unpack_wrong.py
1.0
```

```
$ python3.15 -m mypy unpack_wrong.py
unpack_wrong.py:17: error: Argument 1 to "sum_values" has incompatible type "Array[float, st
Found 1 error in 1 file (checked 1 source file)
```

```
$ python3.15 -m pyright --pythonversion 3.15 unpack_wrong.py
/tmp/py-shapes-lab/unpack_wrong.py
  /tmp/py-shapes-lab/unpack_wrong.py:17:22 - error: Argument of type "Array[float, str]" can
    "Array[float, str]" is not assignable to "Array[float, *tuple[int, ...]]"
      Type parameter "Shape@Array" is invariant, but "*tuple[str]" is not the same as "*tupl
1 error, 0 warnings, 0 informations
```

`Unpack` is also how PEP 692 types `**kwargs` from a `TypedDict`. Same name, different job. An unpacked `TypeVarTuple` describes positional slots. `Unpack[SomeTypedDict]` describes keyword names. They meet only if you import `Unpack` and then use it for both.

Case 7: `*args: *Ts`

`*args: *Ts` is not "a tuple of `Ts`". Each argument is one element of `Ts`, and the annotation is stored as `Unpack[Ts]`. Here `Ts` is tied to the callback's positional parameters, the same idea as the `call_soon` example in the `TypeVarTuple` docs.

```
# args_ok.py - *args: *Ts matches the callback argument by argument
from collections.abc import Callable
from typing import get_type_hints


def apply_axes[*Ts](callback: Callable[[*Ts], str], *args: *Ts) -> str:
    return callback(*args)


def label(height: int, width: int) -> str:
    return f"{height}x{width}"


def main() -> None:
    print(get_type_hints(apply_axes))
    print(apply_axes(label, 2, 3))


if __name__ == "__main__":
    main()
```

```
$ python3.15 args_ok.py
{'callback': collections.abc.Callable[[typing.Unpack[Ts]], str], 'args': typing.Unpack[Ts],
2x3
```

```
$ python3.15 -m mypy args_ok.py
Success: no issues found in 1 source file
```

```
$ python3.15 -m pyright --pythonversion 3.15 args_ok.py
0 errors, 0 warnings, 0 informations
```

`label` formats whatever it is given, so a string width runs. The checkers both reject the call, and they point at different arguments. mypy solves `Ts` from the call as `(int, str)` and then says the callback is the wrong `Callable`. pyright solves `Ts` from `label` as `(int, int)` and then says the argument pack is wrong, down to `Literal['3']` versus `int`. Same mistake, two reports. Trust the shared conclusion, and read the note under the error before you "fix" the wrong parameter.

```
# args_wrong.py - a str in *args does not match label's int
from collections.abc import Callable
```

```
def apply_axes[*Ts](callback: Callable[[*Ts], str], *args: *Ts) -> str:
    return callback(*args)


def label(height: int, width: int) -> str:
    return f"{height}x{width}"


def main() -> None:
    print(apply_axes(label, 2, "3"))


if __name__ == "__main__":
    main()
```

```
$ python3.15 args_wrong.py
2x3
```

```
$ python3.15 -m mypy args_wrong.py
args_wrong.py:14: error: Argument 1 to "apply_axes" has incompatible type "Callable[[int, in
Found 1 error in 1 file (checked 1 source file)
```

```
$ python3.15 -m pyright --pythonversion 3.15 args_wrong.py
/tmp/py-shapes-lab/args_wrong.py
  /tmp/py-shapes-lab/args_wrong.py:14:11 - error: Argument of type "tuple[Literal[2], Litera
    Type "*tuple[Literal[2], Literal['3']]" is not assignable to type "*tuple[int, int]"
      "*tuple[Literal[2], Literal['3']]" is not assignable to "*tuple[int, int]"
        Tuple entry 2 is incorrect type
          "Literal['3']" is not assignable to "int" (reportArgumentType)
1 error, 0 warnings, 0 informations
```

Case 8: The 3.15 constructor, and today's checkers

This is the 3.15-only piece. The attributes exist at runtime. Setting both `covariant=True` and `contravariant=True` raises `ValueError`, with the message `Bivariant types are not supported.` The `bound=int` object is created, and `__bound__` is `int`. That does not mean a checker will reject an axis that is not `int`. The specification has not defined what `bound` means on a `TypeVarTuple`.

```
# tvt_kwargs.py
from typing import TypeVarTuple

Shape = TypeVarTuple("Shape", covariant=True)
print(
```

```
    "covariant",
    Shape.__covariant__,
    "contravariant",
    Shape.__contravariant__,
    "infer",
    Shape.__infer_variance__,
    "bound",
    Shape.__bound__,
)

Bounded = TypeVarTuple("Bounded", bound=int)
print("bounded", Bounded.__bound__)

try:
    TypeVarTuple("Both", covariant=True, contravariant=True)
except ValueError as exc:
    print(type(exc).__name__ + ":", exc)
```

```
$ python3.15 tvt_kwargs.py
covariant True contravariant False infer False bound None
bounded <class 'int'>
ValueError: Bivariant types are not supported.
```

```
$ python3.15 -m mypy tvt_kwargs.py
tvt_kwargs.py:4: error: Unexpected keyword argument "covariant" for "TypeVarTuple"  [misc]
tvt_kwargs.py:7: error: "object" has no attribute "__covariant__"  [attr-defined]
tvt_kwargs.py:9: error: "object" has no attribute "__contravariant__"  [attr-defined]
tvt_kwargs.py:11: error: "object" has no attribute "__infer_variance__"  [attr-defined]
tvt_kwargs.py:13: error: "object" has no attribute "__bound__"  [attr-defined]
tvt_kwargs.py:16: error: Unexpected keyword argument "bound" for "TypeVarTuple"  [misc]
tvt_kwargs.py:17: error: "object" has no attribute "__bound__"  [attr-defined]
Found 7 errors in 1 file (checked 1 source file)
```

```
$ python3.15 -m pyright --pythonversion 3.15 tvt_kwargs.py
/tmp/py-shapes-lab/tvt_kwargs.py
  /tmp/py-shapes-lab/tvt_kwargs.py:4:31 - error: "covariant" is unknown parameter to TypeVar
  /tmp/py-shapes-lab/tvt_kwargs.py:16:35 - error: "bound" is unknown parameter to TypeVarTup
  /tmp/py-shapes-lab/tvt_kwargs.py:20:26 - error: "covariant" is unknown parameter to TypeVa
  /tmp/py-shapes-lab/tvt_kwargs.py:20:42 - error: "contravariant" is unknown parameter to Ty
4 errors, 0 warnings, 0 informations
```

mypy 2.4.0 calls `covariant` and `bound` unexpected keywords, then treats the result as `object`, so the attribute reads fail too. pyright 1.1.414 says each keyword is an unknown parameter of `TypeVarTuple`. The keywords exist on CPython 3.15. These two checkers do not accept them yet, and `bound` still has no meaning in the typing specification. Keep them off array signatures.

The trap

The trap is believing the annotation inspects the tuple you passed to `__init__`.

`Array.__init__` is typed as `values: list[float]` and `shape: tuple[int, ...]`. Those parameters are not `*Shape`. Writing `Array[float, Literal[3], Literal[3]]` on the variable is a claim about the variable. Both checkers accept the claim. The value's `.shape` is `(99,)`, and `square` accepts it.

```
# lie_ok.py
from typing import Literal


class Array[DType, *Shape]:
    def __init__(self, values: list[float], shape: tuple[int, ...]) -> None:
        self.values = values
        self.shape = shape


def square(x: Array[float, Literal[3], Literal[3]]) -> Array[float, Literal[3], Literal[3]]:
    return x


def main() -> None:
    lie: Array[float, Literal[3], Literal[3]] = Array([1.0], (99,))
    print(square(lie).shape)


if __name__ == "__main__":
    main()
```

```
$ python3.15 lie_ok.py
(99,)
```

```
$ python3.15 -m mypy lie_ok.py
Success: no issues found in 1 source file
```

```
$ python3.15 -m pyright --pythonversion 3.15 lie_ok.py
0 errors, 0 warnings, 0 informations
```

`Literal[3]` checked out in Case 3 because two *variables* disagreed. Here one variable is annotated with a precise type and then filled with a constructor the checker cannot connect to those literals. A shape bug of this kind needs a runtime test: `if self.shape != expected: raise ...`, or a checker plugin that understands your constructor. PEP 646 does not add that test.

The second trap is two type-variable tuples, which is the obvious way to write "concatenate an unknown prefix with an unknown suffix". The syntax is legal. The call returns `(1, 2, 'a')`. The

type parameters really are `(Left, Right)`.

```
# two_tuples.py
def concat[*Left, *Right](left: tuple[*Left], right: tuple[*Right]) -> tuple[*Left, *Right]:
    return (*left, *right)


def main() -> None:
    print(concat((1, 2), ("a",)))
    print("defined", concat.__type_params__)


if __name__ == "__main__":
    main()
```

```
$ python3.15 two_tuples.py
(1, 2, 'a')
defined (Left, Right)
```

```
$ python3.15 -m mypy two_tuples.py
two_tuples.py:2: error: More than one variadic Unpack in a type is not allowed  [misc]
two_tuples.py:3: error: Incompatible return value type (got "tuple[object, ...]", expected "
Found 2 errors in 1 file (checked 1 source file)
```

```
$ python3.15 -m pyright --pythonversion 3.15 two_tuples.py
/tmp/py-shapes-lab/two_tuples.py
  /tmp/py-shapes-lab/two_tuples.py:2:86 - error: Type argument list can have at most one unp
1 error, 0 warnings, 0 informations
```

mypy reports two errors. The first is the rule: more than one variadic unpack in a type. The second says the return is `tuple[object, ...]`, the type mypy uses once that two-unpack return is rejected. pyright reports the rule once, on the return annotation: a type-argument list can have at most one unpacked `TypeVarTuple` or tuple. To prepend or append a known axis, use an ordinary type variable beside the single `TypeVarTuple`, as `prepend_axis` and `move_first_axis_to_last` do.

The boring rule

- One dtype type variable, then one unpacked shape: `class Array[DType, *Shape]`.
- Preserve a shape by repeating `*Shape` in the parameter and the return.
- Add an axis by concatenating a normal type: `Array[DType, int, *Shape]` or `Array[DType, *Shape, Axis]`.
- Unknown rank, every axis an `int`: `Array[DType, *tuple[int, ...]]`. `Unpack[tuple[int, ...]]` is the same type to the checkers used here.
- `*args: *Ts` when each argument has its own type. `*args: int` when they all share one.

- At most one unpacked `TypeVarTuple` in a list. CPython will not stop you. mypy and pyright will.
- `Literal` sizes are compared only when they appear as type arguments on names the checker can see. The constructor in this chapter does not put `.shape` into those arguments.
- On 3.15, do not pass `bound=` to `TypeVarTuple` and expect either a specified meaning or a clean run under mypy 2.4.0 or pyright 1.1.414.
- If the numbers must hold at run time, test `.shape` in ordinary Python.

Try this

1. In `shape_wrong.py`, change the wide array to `Array[float, Literal[3], Literal[3]]` and pass a values list of length 9 with shape `(3, 3)`. Confirm mypy and pyright both go quiet, then change only the annotation's last axis back to `Literal[4]` and watch the error return while `python3.15` still prints.
2. In `concat_ok.py`, add `flat: Array[float, Literal[2], Literal[3]] = prepend_axis(image)` and read the error. Say which axis was concatenated, and where.
3. In `unpack_wrong.py`, replace `str` with `int` and confirm the checkers accept it. Then try `Array[float, int, str]` and see whether `*tuple[int, ...]` rejects a pack that is only partly `int`.
4. In `two_tuples.py`, delete `*Right`. Type the second parameter as `tuple[str, ...]` and the return as `tuple[*Left, *tuple[str, ...]]`. Run mypy and pyright. An unpacked `TypeVarTuple` already counts as an open-ended tuple, and the typing spec rejects two open-ended unpacks in one `tuple[...]`. Record the message you get. If your checker accepts it, say so: that is a checker gap, and the spec is the rule.
5. Copy `lie_ok.py`. At the start of `square`, add `if x.shape != (3, 3): raise ValueError(x.shape)`. Run `python3.15 lie_ok.py`, then mypy. Compare the exception with mypy's report.

Sources

- PEP 646, *Variadic Generics* (Python 3.11): https://peps.python.org/pep-0646/
- PEP 695, *Type Parameter Syntax* (the `class Array[DType, *Shape]` spelling, Python 3.12): https://peps.python.org/pep-0695/
- Python 3.15, `typing.TypeVarTuple`, including the 3.15 constructor parameters and the note that `bound` semantics are not decided: https://docs.python.org/3.15/library/typing.html#typing.TypeVarTuple
- Python 3.15, `typing.Unpack` (type-variable tuples, tuple unpacking, and `**kwargs TypedDicts`): https://docs.python.org/3.15/library/typing.html#typing.Unpack
- What's New in Python 3.15, typing section (`TypeVarTuple` gains `bound`, `covariant`, `contravariant`, `infer_variance`; `bound` semantics remain undefined): https://docs.python.org/3.15/whatsnew/3.15.html
- Typing specification, unpacked tuple forms (one open-ended unpack per tuple; a `TypeVarTuple` counts as open-ended): https://typing.python.org/en/latest/spec/tuples.html

- NumPy 2.5.4 stubs read locally: `numpy/__init__.pyi` (`_ShapeT_co`, `class ndarray`) and `numpy/_typing/_shape.py` (`_Shape`, `_AnyShape`). `numpy.typing.NDArray.__value__` printed `numpy.ndarray[_AnyShape, numpy.dtype[ScalarT]]`.
- PyTorch 2.14.1 stub read locally: `torch/_C/__init__.pyi` (`class Size(tuple[_int, ...])`, `TensorBase.shape: Size`).

Choosing a Type Checker: mypy, pyright, ty and Pyrefly

Choosing a Type Checker: mypy, pyright, ty and Pyrefly

Thesis

The typing specification at typing.python.org says what a type annotation means. A type checker is a program that reads the annotations and reports where the code disagrees with them. Python ships no checker. You pick one, and four are in wide use: **mypy**, **pyright**, Astral's **ty**, and Meta's **Pyrefly**. ty and Pyrefly are written in Rust. mypy is written in Python and compiled with mypyc. pyright is written in TypeScript and runs on Node.js.

The four agree on annotated code that the specification covers. They disagree in three places, and those decide most of the choice:

1. **Defaults.** Whether the bodies of unannotated functions are checked at all, and how much is reported when there is no config file.
2. **Inference the specification leaves open.** What `[]` becomes, whether a variable may change type, and what an unannotated function returns.
3. **Speed and editor integration.** A cold check of a 38,515-line package took 0.15 s with ty and 6.92 s with pyright on this machine.

The python/typing **conformance suite** scores each checker against the specification. It is the closest thing to a neutral grade. It is also a snapshot of particular versions, and Case 6 shows a score that was already out of date.

Every listing ran on CPython 3.15.0, released 2026-10-09. The four checkers live in one 3.15.0 virtual environment, installed with uv, and every command below runs with that environment on `PATH`. Each checker is told to target 3.15. The runs lived in `/tmp/py-checkers-lab/` on an 8-core Linux machine.

```
$ python --version
Python 3.15.0
$ mypy --version
mypy 2.4.0 (compiled: yes)
$ pyright --version
pyright 1.1.414
$ ty --version
ty 0.0.89
$ pyrefly --version
pyrefly 1.4.0
```

ty's version number still starts with `0.0.`, so expect its behaviour to change between releases, and pin it.

Mental model

```
typing specification   says what annotations mean (shared)

conformance suite     146 test files: expected errors per line,
                        plus a human verdict per checker version


  mypy       pyright     ty          pyrefly      each one adds its own:
                                                    • defaults (what is checked)
                                                    • inference where the spec is silent
                                                    • speed, caching, language server
```

A clean run means "clean for that checker, with that configuration, at that version". It does not mean "type-correct Python". When two checkers disagree, first ask which of the three layers is involved. Is it the specification, a default, or inference? Only the first is a bug in one of them.

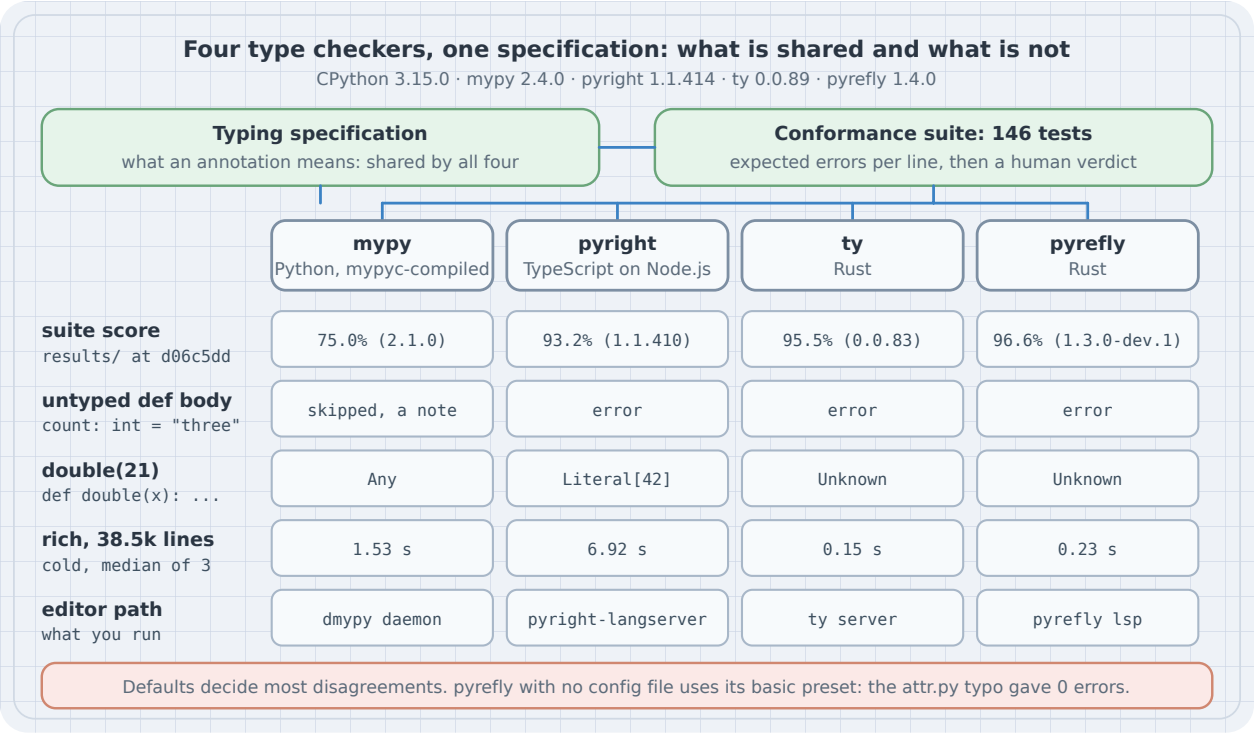


Figure 1: The shared specification and conformance suite above four checkers, with their scores, defaults, inference, speed and editor entry points

Worked examples

All four checkers run on the same file with the same target version through one small script. The pyright line strips the absolute path that pyright prints. Pyrefly gets `--preset default` here. The trap section shows why that flag matters.

```
#!/usr/bin/env bash
# check_all.sh: run the four checkers on one file, same Python version.
f=$1
echo "--- mypy";    mypy --python-version 3.15 "$f"
echo "--- pyright"; pyright --pythonversion 3.15 "$f" | sed "s|$PWD/||"
echo "--- ty";      ty check --python-version 3.15 --output-format concise "$f"
echo "--- pyrefly"; pyrefly check --python-version 3.15 --preset default --output-format min
```

Case 1: Annotated code with a plain bug

Start where they should agree. `Config` has a `host` attribute, and `connect` reads `hostname`.

```
# attr.py: a typo every checker should catch
class Config:
    def __init__(self) -> None:
        self.host = "localhost"


def connect(cfg: Config) -> str:
    return cfg.hostname
```

```
$ bash check_all.sh attr.py
--- mypy
attr.py:8: error: "Config" has no attribute "hostname"  [attr-defined]
Found 1 error in 1 file (checked 1 source file)
--- pyright
attr.py
  attr.py:8:16 - error: Cannot access attribute "hostname" for class "Config"
    Attribute "hostname" is unknown (reportAttributeAccessIssue)
1 error, 0 warnings, 0 informations
--- ty
attr.py:8:12: error[unresolved-attribute] Object of type `Config` has no attribute `hostname
Found 1 diagnostic
--- pyrefly
ERROR attr.py:8:12-24: Object of class `Config` has no attribute `hostname` [missing-attribu
 INFO 1 error
```

The four checkers report the same line and give the same reason, each in its own words. mypy calls the rule `attr-defined`, pyright `reportAttributeAccessIssue`, ty `unresolved-attribute`, Pyrefly `missing-attribute`. The rule names differ, so a `# type: ignore[attr-defined]` written

for mypy says nothing to the others. Each checker has its own suppression syntax. All of them also honour a bare `# type: ignore`.

Case 2: The body of an unannotated function

`unannotated` has no annotation on its parameters or return. The annotation inside it is plainly wrong. `annotated` has the same body and a return annotation.

```
# untyped.py: the same mistake inside an unannotated and an annotated function
def unannotated():
    count: int = "three"
    return count


def annotated() -> int:
    count: int = "three"
    return count
```

```
$ bash check_all.sh untyped.py
--- mypy
untyped.py:3: note: By default the bodies of untyped functions are not checked, consider usi
untyped.py:8: error: Incompatible types in assignment (expression has type "str", variable h
Found 1 error in 1 file (checked 1 source file)
--- pyright
untyped.py
  untyped.py:3:18 - error: Type "Literal['three']" is not assignable to declared type "int"
    "Literal['three']" is not assignable to "int" (reportAssignmentType)
  untyped.py:8:18 - error: Type "Literal['three']" is not assignable to declared type "int"
    "Literal['three']" is not assignable to "int" (reportAssignmentType)
2 errors, 0 warnings, 0 informations
--- ty
untyped.py:3:18: error[invalid-assignment] Object of type `Literal["three"]` is not assignab
untyped.py:8:18: error[invalid-assignment] Object of type `Literal["three"]` is not assignab
Found 2 diagnostics
--- pyrefly
ERROR untyped.py:3:18-25: `Literal['three']` is not assignable to `int` [bad-assignment]
ERROR untyped.py:8:18-25: `Literal['three']` is not assignable to `int` [bad-assignment]
 INFO 2 errors
```

mypy reports line 8 and prints only a note for line 3. By default mypy does not check the body of a function that has no annotations. PEP 484 allowed that so a codebase can adopt types one function at a time. pyright, ty and Pyrefly check every body. With mypy's `--check-untyped-defs`, or `check_untyped_defs = true` in config, the two counts match:

```
$ mypy --python-version 3.15 --check-untyped-defs untyped.py
untyped.py:3: error: Incompatible types in assignment (expression has type "str", variable h
```

```
untyped.py:8: error: Incompatible types in assignment (expression has type "str", variable h
Found 2 errors in 1 file (checked 1 source file)
```

`strict = true` in mypy's config turns this on as well. Pyrefly has the same switch, `check-unannotated-defs`, and it defaults to true. Pyrefly's `legacy` preset, written by `pyrefly init` when it finds a mypy config, sets it to false to match mypy.

Case 3: Inference the specification leaves open

The typing specification says what annotations mean. It mostly does not say what type an unannotated variable gets. Each checker answers in its own way.

```
# inference.py: two places where the spec leaves inference to the checker
from typing import reveal_type


def empty_list() -> None:
    items = []
    items.append(1)
    items.append("two")
    reveal_type(items)


def redefine() -> None:
    value = 1
    value = "one"
    reveal_type(value)
```

```
$ bash check_all.sh inference.py
--- mypy
inference.py:8: error: Argument 1 to "append" of "list" has incompatible type "str"; expecte
inference.py:9: note: Revealed type is "list[int]"
inference.py:14: error: Incompatible types in assignment (expression has type "str", variabl
inference.py:15: note: Revealed type is "int"
Found 2 errors in 1 file (checked 1 source file)
--- pyright
inference.py
  inference.py:9:17 - information: Type of "items" is "list[Unknown]"
  inference.py:15:17 - information: Type of "value" is "Literal['one']"
0 errors, 0 warnings, 2 informations
--- ty
inference.py:9:17: info[revealed-type] Revealed type: `list[int | str]`
inference.py:15:17: info[revealed-type] Revealed type: `Literal["one"]`
Found 2 diagnostics
--- pyrefly
ERROR inference.py:8:18-23: Argument `Literal['two']` is not assignable to parameter `object
```

```
INFO inference.py:9:16-23: revealed type: list[int] [reveal-type]
INFO inference.py:15:16-23: revealed type: Literal['one'] [reveal-type]
INFO 1 error
```

Take `items = []` first. mypy and Pyrefly use the first `append` to fix the list as `list[int]` and reject `"two"`. pyright leaves the element type `Unknown`, so both appends are accepted. Its documentation lists `list()` with no arguments as `list[Unknown]`. ty reveals `list[int | str]` and reports no error. The same five lines give three different types.

Now `value = 1` followed by `value = "one"`. mypy treats the first assignment as the variable's declared type and rejects the second. pyright's documentation says the declared type of an unannotated variable is the union of every assigned type, and at the `reveal_type` it narrows to the last one, `Literal['one']`. ty and Pyrefly reveal the same `Literal['one']` and accept the reassignment. ty's documentation lists support for redeclarations among its design goals.

None of these answers is a conformance failure. If the type matters to you, write it down. With `items: list[int] = []`, all four checkers report the `append("two")` on line 8. With `value: int | str = 1`, they all accept the reassignment.

Case 4: Calling code that has no annotations

`double` has no annotations at all. `use` calls it with an `int` and then calls a method that does not exist.

```python
# helper.py: calling a function that has no annotations at all
from typing import reveal_type


def double(x):
    return x * 2


def use() -> None:
    result = double(21)
    reveal_type(result)
    result.no_such_method()
```

```
$ bash check_all.sh helper.py
--- mypy
helper.py:11: note: Revealed type is "Any"
Success: no issues found in 1 source file
--- pyright
helper.py
  helper.py:11:17 - information: Type of "result" is "Literal[42]"
  helper.py:12:12 - error: Cannot access attribute "no_such_method" for class "Literal[42]"
    Attribute "no_such_method" is unknown (reportAttributeAccessIssue)
1 error, 0 warnings, 1 information
```

```
--- ty
helper.py:11:17: info[revealed-type] Revealed type: `Unknown`
Found 1 diagnostic
--- pyrefly
 INFO helper.py:11:16-24: revealed type: Unknown [reveal-type]
 INFO 0 errors
```

mypy treats an unannotated function as returning `Any`. `Any` allows every operation, so the bad call passes. ty and Pyrefly reveal `Unknown`, which behaves like `Any`: no error. pyright reports the bug. Its documentation calls the technique *call-site return type inference*. When every parameter of a function is unannotated, pyright re-infers the body using the argument types at each call. `double(21)` becomes `Literal[42]`, and `Literal[42]` has no `no_such_method`.

That is the clearest difference in gradual typing among the four. pyright finds more bugs in unannotated code. mypy, ty and Pyrefly report less noise from it. The specification does not require either behaviour. Only annotations make the result the same everywhere.

Case 5: Reading the conformance suite yourself

The suite lives in `conformance/` in the python/typing repository. Each test file marks the lines where a checker must report an error with `# E`. For each checker version, the maintainers commit one TOML file per test under `results/<checker>/`:

- `conformance_automated` is `Pass` or `Fail`. The README says it fails when the checker's output shows any discrepancy from the `# E` markers. A line marked `# E?` may report an error or not.
- `conformant` is a human verdict: `Pass`, `Partial`, or `Unsupported`, often with `notes`.

The published summary, `results/results.html`, counts `Pass` as 1 and `Partial` as 0.5. A test without a `conformant` key counts as passed if its automated check passed. Test files are the `.py` and `.pyi` files whose names start with a group name from `src/test_groups.toml`. Other files are helpers. This script redoes the count from the committed TOML.

```
# conformance_scores.py: score each checker's committed results in the
# python/typing conformance suite the way results.html does:
# Pass = 1, Partial = 0.5, otherwise 0; if "conformant" is missing, an
# automated Pass counts as Pass.
import sys
import tomllib
from collections import defaultdict
from pathlib import Path

root = Path(sys.argv[1])                        # .../typing/conformance
groups_toml = tomllib.loads((root / "src" / "test_groups.toml").read_text())
# Like the suite's own reporting: a test is a .py/.pyi file whose name starts
# with a known group name; other files are helpers imported by the tests.
tests = sorted(p.stem for p in (root / "tests").iterdir()
```

```
                if p.suffix in (".py", ".pyi") and p.name.split("_")[0] in groups_toml)
checkers = sorted(d.name for d in (root / "results").iterdir() if d.is_dir())


def score(checker, test):
    path = root / "results" / checker / f"{test}.toml"
    data = tomllib.loads(path.read_text()) if path.exists() else {}
    status = data.get("conformant") or (
        "Pass" if data.get("conformance_automated") == "Pass" else "Unknown")
    return {"Pass": 1.0, "Partial": 0.5}.get(status, 0.0)


print(f"{len(tests)} test files\n")
print(f"{'checker':11} {'version':22} {'score':>9} {'pct':>6}")
groups = defaultdict(lambda: defaultdict(float))
for c in checkers:
    version = tomllib.loads((root / "results" / c / "version.toml").read_text())["version"]
    total = 0.0
    for t in tests:
        s = score(c, t)
        total += s
        groups[t.split("_")[0]][c] += s
    print(f"{c:11} {version:22} {total:6.1f}/{len(tests)} {100 * total / len(tests):5.1f}%")

sizes = defaultdict(int)
for t in tests:
    sizes[t.split("_")[0]] += 1
print("\nweakest groups per checker (passed/total, groups below 100% only):")
for c in checkers:
    below = [g for g in groups if groups[g][c] < sizes[g]]
    worst = sorted(below, key=lambda g: groups[g][c] / sizes[g])[:3]
    shown = ", ".join(f"{g} {groups[g][c]:g}/{sizes[g]}" for g in worst)
    print(f"  {c:11} {shown or 'every group at 100%'}")
```

```
$ git -C typing log -1 --format='%h %cs %s'
d06c5dd 2026-10-14 conformance: remove questionable generic syntax test (#2368)
$ python conformance_scores.py typing/conformance
146 test files

checker     version                   score    pct
mypy        mypy 2.1.0              109.5/146  75.0%
pycroscope  pycroscope 0.5.0        138.5/146  94.9%
pyrefly     pyrefly 1.3.0-dev.1     141.0/146  96.6%
pyright     pyright 1.1.410         136.0/146  93.2%
ty          ty 0.0.83               139.5/146  95.5%
zuban       zuban 0.10.0            146.0/146 100.0%
```

```
weakest groups per checker (passed/total, groups below 100% only):
  mypy        classes 1/2, constructors 3/6, historical 0.5/1
  pycroscope  typeforms 0.5/1, overloads 3.5/5, annotations 4.5/6
  pyrefly     annotations 5/6, generics 30/33, protocols 10.5/11
  pyright     typeforms 0/1, classes 1.5/2, annotations 5/6
  ty          protocols 8.5/11, constructors 5/6, aliases 6/7
  zuban       every group at 100%
```

The totals agree with the committed HTML report:

```
$ grep -o '[0-9.]* / 146 • [0-9.]*%' typing/conformance/results/results.html
109.5 / 146 • 75.0%
138.5 / 146 • 94.9%
141 / 146 • 96.6%
136 / 146 • 93.2%
139.5 / 146 • 95.5%
146 / 146 • 100.0%
```

The order is the table's column order: mypy, pycroscope, Pyrefly, pyright, ty, zuban. Look at the version column before the percentages. These are the versions whose results were committed: mypy 2.1.0, pyright 1.1.410, ty 0.0.83, and a Pyrefly 1.3.0 development build. They are not the versions installed here. zuban is another Rust checker. Its documentation describes a mypy-compatible mode and a pyright-like mode. At this commit it passes all 146 tests. It is not installed on this machine, so no listing in this chapter runs it.

"Conformant" covers what the specification says and the suite tests. It does not cover Case 3 and Case 4. It says nothing about speed or editor features.

Case 6: A score is a snapshot

`specialtypes_sentinels.py` tests sentinel values from `typing_extensions`. Each checker's committed verdict for it:

```
$ head -2 typing/conformance/results/mypy/specialtypes_sentinels.toml
conformant = "Unsupported"
conformance_automated = "Fail"
$ head -2 typing/conformance/results/pyright/specialtypes_sentinels.toml
conformant = "Partial"
notes = """
$ head -2 typing/conformance/results/ty/specialtypes_sentinels.toml
conformant = "Pass"
conformance_automated = "Pass"
$ head -2 typing/conformance/results/pyrefly/specialtypes_sentinels.toml
conformant = "Pass"
conformance_automated = "Pass"
```

The test marks three lines with `# E`:

```
$ grep -n '# E' specialtypes_sentinels.py
14:def func1(x: int = MISSING) -> None:  # E: incompatible default
36:func2(Cls.IN_CLASS)  # E: incompatible argument
39:func3(MISSING)  # E: incompatible argument
```

Now run the same file with the installed versions:

```
$ bash check_all.sh specialtypes_sentinels.py
--- mypy
specialtypes_sentinels.py:14: error: Incompatible default for parameter "x" (default has typ
specialtypes_sentinels.py:36: error: Argument 1 to "func2" has incompatible type "Cls.IN_CLA
specialtypes_sentinels.py:39: error: Argument 1 to "func3" has incompatible type "MISSING";
Found 3 errors in 1 file (checked 1 source file)
--- pyright
specialtypes_sentinels.py
  specialtypes_sentinels.py:7:1 - error: Sentinel must be assigned to a variable with the sa
  specialtypes_sentinels.py:11:5 - error: Sentinel must be assigned to a variable with the s
  specialtypes_sentinels.py:20:20 - error: Variable not allowed in type expression (reportIn
  specialtypes_sentinels.py:22:21 - error: "assert_type" mismatch: expected "Unknown" but re
  specialtypes_sentinels.py:22:24 - error: Variable not allowed in type expression (reportIn
  specialtypes_sentinels.py:24:21 - error: "assert_type" mismatch: expected "int | SPECIAL"
  specialtypes_sentinels.py:26:24 - error: Variable not allowed in type expression (reportIn
  specialtypes_sentinels.py:28:21 - error: "assert_type" mismatch: expected "Unknown" but re
  specialtypes_sentinels.py:28:28 - error: Variable not allowed in type expression (reportIn
  specialtypes_sentinels.py:30:21 - error: "assert_type" mismatch: expected "int" but receiv
10 errors, 0 warnings, 0 informations
--- ty
specialtypes_sentinels.py:14:11: error[invalid-parameter-default] Default value of type `MIS
specialtypes_sentinels.py:36:7: error[invalid-argument-type] Argument to function `func2` is
specialtypes_sentinels.py:39:7: error[invalid-argument-type] Argument to function `func3` is
Found 3 diagnostics
--- pyrefly
ERROR specialtypes_sentinels.py:14:20-27: Default `<MISSING>` is not assignable to parameter
ERROR specialtypes_sentinels.py:36:7-19: Argument `Cls.Cls.IN_CLASS` is not assignable to pa
ERROR specialtypes_sentinels.py:39:7-14: Argument `<MISSING>` is not assignable to parameter
 INFO 3 errors
```

The committed results mark mypy 2.1.0 `Unsupported` on this test. mypy 2.4.0 reports exactly lines 14, 36 and 39, the same three as ty and Pyrefly. That is the automated pass condition. At least this test has moved since 2.1.0, so the committed 75.0% understates mypy 2.4.0 here. pyright 1.1.414 still prints the ten errors the `Partial` note describes. It requires the string passed to `Sentinel` to match the variable name. The test's own comment on line 7 says the name is not required to match. When the suite matters to your choice, run the tests you care about against your versions, as here.

Case 7: Speed on a real package

The corpus is Rich at commit 9d8f9a3: 38,515 lines of Python under `rich/rich/`. Rich's own `pyproject.toml` has a `[tool.mypy]` section with `strict = true`. Its third-party dependencies are not installed here.

Run each checker from the project root. Changing into the package directory first does not work:

```
$ find rich/rich -name "*.py" | xargs cat | wc -l
38515
$ (cd rich/rich && mypy --python-version 3.15 .)
abc.py: error: This file shadows library module "abc"
abc.py: note: A user-defined top-level module with name "abc" is not supported
Found 1 error in 1 file (errors prevented further checking)
```

Rich has a module named `rich/abc.py`. Checked from inside the package, it becomes a top-level module called `abc`, which shadows the standard library. mypy refuses to continue. The timing script runs from the root and passes the package name. Each run gets a new mypy cache directory, so every run is cold. A last row keeps one cache across three mypy runs.

```
# time_checkers.py: run each checker over one package, three times,
# from a cold start, and print wall-clock seconds and diagnostic counts.
# Usage: python time_checkers.py PROJECT_ROOT PACKAGE_DIR
import shutil
import subprocess
import sys
import tempfile
import time
from pathlib import Path

root = Path(sys.argv[1]).resolve()   # run from here, so config files apply
pkg = sys.argv[2]                     # e.g. "rich"; never cd into the package
bin_dir = Path(sys.executable).parent          # the venv the checkers live in
PY = "3.15"


def commands(cache: str) -> dict[str, list[str]]:
    return {
        "mypy": [f"{bin_dir}/mypy", "--python-version", PY,
                 "--cache-dir", cache, "--no-error-summary", pkg],
        "pyright": [f"{bin_dir}/pyright", "--pythonversion", PY, pkg],
        "ty": [f"{bin_dir}/ty", "check", "--python-version", PY,
               "--output-format", "concise", pkg],
        "pyrefly": [f"{bin_dir}/pyrefly", "check", "--python-version", PY,
                    "--preset", "default", "--output-format", "min-text", pkg],
    }
```

```
def errors(name: str, out: str) -> int:
    lines = out.splitlines()
    if name == "mypy":
        return sum(": error:" in l for l in lines)
    if name == "pyright":
        return sum(" - error:" in l for l in lines)
    if name == "ty":
        return sum(" error[" in l for l in lines)
    return sum(l.startswith("ERROR ") for l in lines)


print(f"{'checker':8} {'run 1':>6} {'run 2':>6} {'run 3':>6} {'errors':>7}")
for name in ("mypy", "pyright", "ty", "pyrefly"):
    times, count = [], 0
    for _ in range(3):
        cache = tempfile.mkdtemp()               # fresh mypy cache every run
        cmd = commands(cache)[name]
        start = time.perf_counter()
        proc = subprocess.run(cmd, cwd=root, capture_output=True, text=True)
        times.append(time.perf_counter() - start)
        count = errors(name, proc.stdout + proc.stderr)
        shutil.rmtree(cache)
    print(f"{name:8} " + " ".join(f"{t:6.2f}" for t in times) + f" {count:7}")

# mypy again, but keeping its cache between runs: the incremental path.
cache = tempfile.mkdtemp()
warm = []
for _ in range(3):
    start = time.perf_counter()
    subprocess.run(commands(cache)["mypy"], cwd=root, capture_output=True)
    warm.append(time.perf_counter() - start)
shutil.rmtree(cache)
print(f"{'mypy*':8} " + " ".join(f"{t:6.2f}" for t in warm) + "   (*cache kept)")
```

```
$ python time_checkers.py rich rich
checker   run 1  run 2  run 3  errors
mypy       1.61   1.50   1.53       6
pyright    6.92   6.78   7.01      50
ty         0.15   0.13   0.15      57
pyrefly    0.23   0.23   0.25      34
mypy*      1.56   0.15   0.12   (*cache kept)
```

The median cold runs are ty 0.15 s, Pyrefly 0.23 s, mypy 1.53 s and pyright 6.92 s. ty is about ten times faster than mypy here and about forty-five times faster than pyright. Starting pyright and Node.js with nothing to check takes about a third of a second, so most of pyright's time goes to analysis:

```
$ time pyright --version
pyright 1.1.414

real    0m0.318s
user    0m0.263s
sys 0m0.042s
```

With its cache kept, mypy drops to about 0.1 s once nothing has changed. For CI on a package this size, any of the four is fast enough. On a large monorepo, and on every keystroke in an editor, the gap matters more.

mypy's incremental path for editors is its daemon, `dmypy`. It keeps the checked program in memory:

```
#!/usr/bin/env bash
# dmypy_loop.sh: time mypy's daemon, the incremental mode behind dmypy-based editor setups.
# Run from the project root: bash dmypy_loop.sh rich rich/color.py
set -u
pkg=$1 touched=$2
ms() { local s=$(date +%s%N); "$@" > /dev/null 2>&1; echo $(( ($(date +%s%N) - s) / 1000000
dmypy stop > /dev/null 2>&1
echo "start + full check: $(ms dmypy run -- --python-version 3.15 "$pkg") ms"
echo "nothing changed:    $(ms dmypy run -- --python-version 3.15 "$pkg") ms"
echo "# edited" >> "$touched"
echo "one file edited:    $(ms dmypy run -- --python-version 3.15 "$pkg") ms"
git checkout -q -- "$touched"            # undo the edit
dmypy stop
```

```
$ (cd rich && bash ../dmypy_loop.sh rich rich/color.py)
start + full check: 1680 ms
nothing changed:    83 ms
one file edited:    278 ms
Daemon stopped
```

The daemon took 1.7 s to start and do a full check. After that, an unchanged tree took 83 ms and a one-line edit took 278 ms. Those are run-to-run timings on one machine. Run the script on your own project before you choose by speed.

Case 8: What each language server offers

ty, Pyrefly and pyright ship language servers. Their editor features come from the same analysis as their command-line checks. mypy has no language server of its own. Editor integrations use the mypy command or `dmypy`. A language server announces what it supports in its reply to the LSP `initialize` request. This script sends that request directly, with no editor in between.

```
# lsp_probe.py: start each language server, send LSP "initialize", and
# print what the server says about itself. No editor involved.
import json
import subprocess
import sys
from pathlib import Path

bin_dir = Path(sys.executable).parent
SERVERS = {
    "pyright": [f"{bin_dir}/pyright-langserver", "--stdio"],
    "ty": [f"{bin_dir}/ty", "server"],
    "pyrefly": [f"{bin_dir}/pyrefly", "lsp"],
}
WANTED = ["hoverProvider", "definitionProvider", "referencesProvider",
          "renameProvider", "inlayHintProvider", "codeActionProvider",
          "diagnosticProvider"]


def send(proc, msg):
    body = json.dumps(msg).encode()
    proc.stdin.write(b"Content-Length: %d\r\n\r\n" % len(body) + body)
    proc.stdin.flush()


def receive(proc):
    length = 0
    while (line := proc.stdout.readline().strip()):
        if line.lower().startswith(b"content-length:"):
            length = int(line.split(b":")[1])
    return json.loads(proc.stdout.read(length))


root = Path.cwd().as_uri()
for name, cmd in SERVERS.items():
    proc = subprocess.Popen(cmd, stdin=subprocess.PIPE, stdout=subprocess.PIPE,
                            stderr=subprocess.DEVNULL)
    send(proc, {"jsonrpc": "2.0", "id": 1, "method": "initialize",
                "params": {"processId": None, "rootUri": root,
                           "capabilities": {"textDocument": {
                               "hover": {}, "inlayHint": {}, "diagnostic": {},
                               "codeAction": {}, "rename": {}}}}})
    while "id" not in (reply := receive(proc)):   # skip server notifications
        pass
    caps = reply["result"]["capabilities"]
    info = reply["result"].get("serverInfo", {})
    have = [k.removesuffix("Provider") for k in WANTED if caps.get(k)]
    print(f"{name:8} serverInfo={info.get('name')} {info.get('version', '')}")
```

```
    print(f"{'':8} {', '.join(have)}")
    send(proc, {"jsonrpc": "2.0", "method": "initialized", "params": {}})
    send(proc, {"jsonrpc": "2.0", "id": 2, "method": "shutdown"})
    while receive(proc).get("id") != 2:            # wait for the shutdown reply
        pass
    send(proc, {"jsonrpc": "2.0", "method": "exit"})
    proc.wait(timeout=10)
```

```
$ python lsp_probe.py
pyright  serverInfo=None
         hover, definition, references, rename, codeAction
ty       serverInfo=ty 0.0.89
         hover, definition, references, rename, codeAction, diagnostic
pyrefly  serverInfo=pyrefly-lsp 1.4.0
         hover, definition, references, rename, inlayHint, codeAction
```

The reply lists only capabilities announced up front. A server may register more after `initialized`, and pyright's reply carries no `serverInfo` at all. So read the lists as a floor. All three servers announce hover, go-to-definition, references, rename and code actions. Pyrefly announces inlay hints in the reply. ty announces pull diagnostics. The ty and Pyrefly documentation have setup pages for each editor.

The trap

The trap is reading an error count as a grade.

First, a checker with no configuration may be running a lenient mode. Pyrefly 1.4.0 with no `pyrefly.toml` and no `[tool.pyrefly]` table looks for a mypy or pyright config to migrate. If it finds none, it uses its `basic` preset. That preset enables a short list of high-confidence error kinds. Missing attributes are not on it.

```
#!/usr/bin/env bash
# no_config.sh: pyrefly in a directory with no pyrefly.toml and no pyproject.toml
ls pyrefly.toml pyproject.toml 2>&1
for f in untyped.py inference.py attr.py; do
  echo "--- pyrefly check $f"
  pyrefly check --python-version 3.15 --output-format min-text "$f" 2>&1
done
```

```
$ bash no_config.sh
ls: cannot access 'pyrefly.toml': No such file or directory
ls: cannot access 'pyproject.toml': No such file or directory
--- pyrefly check untyped.py
 INFO 0 errors
No `pyrefly.toml` found - using preset `basic`.
```

```
Run `pyrefly init` to continue setting up Pyrefly.
Docs: https://pyrefly.org/en/docs/installation/
--- pyrefly check inference.py
 INFO 0 errors
No `pyrefly.toml` found - using preset `basic`.
Run `pyrefly init` to continue setting up Pyrefly.
Docs: https://pyrefly.org/en/docs/installation/
--- pyrefly check attr.py
 INFO 0 errors
No `pyrefly.toml` found - using preset `basic`.
Run `pyrefly init` to continue setting up Pyrefly.
Docs: https://pyrefly.org/en/docs/installation/
```

The `hostname` typo from Case 1, the bad assignment from Case 2, and the bad `append` from Case 3 all produced 0 errors. The hint is printed below the result. Add an explicit table and the default preset applies:

```toml
# pyproject.toml: an explicit [tool.pyrefly] table, so no preset is guessed
[project]
name = "configured"
version = "0"
requires-python = ">=3.15"

[tool.pyrefly]
python-version = "3.15"
```

```
$ cd configured && pyrefly check --output-format min-text attr.py
ERROR attr.py:8:12-24: Object of class `Config` has no attribute `hostname` [missing-attribu
 INFO 1 error
```

Second, counts on the same code are not comparable. Here are the Rich errors grouped by rule, warnings left out:

```python
# error_codes.py: what are each checker's errors on a package, by rule?
# Usage: python error_codes.py PROJECT_ROOT PACKAGE_DIR
import json
import re
import subprocess
import sys
from collections import Counter
from pathlib import Path

root, pkg = Path(sys.argv[1]).resolve(), sys.argv[2]
bin_dir = Path(sys.executable).parent
```

```
def run(*cmd: str) -> str:
    proc = subprocess.run([f"{bin_dir}/{cmd[0]}", *cmd[1:]], cwd=root,
                          capture_output=True, text=True)
    return proc.stdout


codes: dict[str, Counter[str]] = {}
out = run("mypy", "--python-version", "3.15", "--no-error-summary", pkg)
codes["mypy"] = Counter(re.findall(r": error: .*\[([a-z-]+)\]$", out, re.M))

out = run("pyright", "--pythonversion", "3.15", "--outputjson", pkg)
codes["pyright"] = Counter(d.get("rule", "?") for d in json.loads(out)["generalDiagnostics"]
                           if d["severity"] == "error")

out = run("ty", "check", "--python-version", "3.15", "--output-format", "concise", pkg)
codes["ty"] = Counter(re.findall(r" error\[([a-z-]+)\]", out))

out = run("pyrefly", "check", "--python-version", "3.15", "--preset", "default",
          "--output-format", "json", pkg)
codes["pyrefly"] = Counter(e["name"] for e in json.loads(out)["errors"]
                           if e["severity"] == "error")

for name, counter in codes.items():
    top = ", ".join(f"{code} {n}" for code, n in counter.most_common(3))
    print(f"{name:8} {counter.total():3} errors: {top}")
```

```
$ python error_codes.py rich rich
mypy       6 errors: import-not-found 3, return-value 2, redundant-cast 1
pyright   50 errors: reportAttributeAccessIssue 14, reportArgumentType 14, reportPossiblyUnb
ty        57 errors: unresolved-import 19, unresolved-attribute 10, invalid-argument-type 9
pyrefly   34 errors: bad-argument-type 11, missing-source-for-stubs 11, missing-import 7
```

mypy reports 6 errors. Rich's CI workflow has a "Typecheck with mypy" step, its config is strict, and three of the 6 are dependencies that are not installed. ty's largest group is `unresolved-import`, 19 errors, again mostly the missing dependencies. Pyrefly's `missing-source-for-stubs` says Pyrefly ships stubs for the package, `pygments` here, but the package itself is not installed. Much of the rest comes from inference and platform choices. For example, Pyrefly reports the Windows-only `sys.getwindowsversion` in `rich/_windows.py` when it checks for Linux. The counts do not show that one checker is "stricter" in general. A project is clean for the checker its CI runs. A second checker on the same code finds the places where the two infer differently, and some of those are real bugs.

The boring rule

- Pick one checker as the gate in CI, pin its version, and set the target with `--python-version` / `--pythonversion` or in config.

- Always commit an explicit config section for the checker you run (`[tool.mypy]`, `[tool.pyright]`, `[tool.ty]`, `[tool.pyrefly]`). Defaults differ, and Pyrefly's no-config default is lenient.
- On mypy, turn on `check_untyped_defs` (or `strict`) unless you are deliberately adopting types one function at a time.
- Annotate where inference differs: empty containers, variables that change type, and public function signatures. That leaves the checkers less to disagree about.
- Run from the project root, with dependencies installed, before you compare checkers.
- Read conformance scores with their version column. Rerun the suite tests you rely on with your own versions.
- Choose by what you need: mypy's plugin ecosystem and incremental daemon, pyright's call-site inference in unannotated code, ty and Pyrefly speed in large trees and in the editor. Any of the four is fine for a well-annotated project.
- Running a second checker in the editor costs little. Making two checkers block a merge means two sets of suppression comments.

Try this

1. Add `items: list[int] = []` to `empty_list` in `inference.py` and run `check_all.sh`. Confirm the four now agree on line 8. Then remove the annotation and add `--strict` to the mypy line only. Does anything change for `empty_list`?
2. In `helper.py`, add `-> int` to `double` without annotating `x`. Which checkers start reporting `no_such_method`? Then annotate `x: int` and see whether pyright's message changes.
3. Pick a conformance test where two checkers differ, using the TOML files in `results/`. Copy the test into the lab, run `check_all.sh`, and compare each checker's lines with the `# E` markers. Say whether your installed versions would still get the committed verdict.
4. Run `time_checkers.py` on a package from your own work, from its root, with its dependencies installed. Then run it again on a tree twice as large. Which checker's time grows the fastest?
5. Run `pyrefly init` in a copy of `rich/` and read the config it writes. Which preset did it pick from Rich's `[tool.mypy]` section, and which mypy defaults does that preset copy?

Sources

- Typing specification: https://typing.python.org/en/latest/spec/
- python/typing conformance suite (README, `src/test_groups.toml`, `results/`): https://github.com/python/typing/tree/main/conformance, read at commit d06c5dd (2026-10-14)
- PEP 484, *Type Hints* (unannotated functions may be treated as dynamically typed): https://peps.python.org/pep-0484/
- mypy documentation, `check_untyped_defs`: https://mypy.readthedocs.io/en/stable/config_file.html#confval-check_untyped_defs; mypy daemon: https://mypy.readthedocs.io/en/stable/mypy_daemon.html
- Pyright documentation, type inference (empty list inference, multi-assignment inference, call-site return type inference): https://microsoft.github.io/pyright/#/type-inference; mypy comparison: https://microsoft.github.io/pyright/#/mypy-comparison

- ty documentation: https://docs.astral.sh/ty/; rules reference: https://docs.astral.sh/ty/reference/rules/
- Pyrefly documentation, configuration (presets, automatic preset selection, `check-unannotated-defs`, `infer-return-types`): https://pyrefly.org/en/docs/configuration/
- Zuban documentation (implementation language, modes): https://docs.zubanls.com/
- Language Server Protocol 3.17, `initialize` and server capabilities: https://microsoft.github.io/language-server-protocol/specifications/lsp/3.17/specification/#initialize
- Rich source, commit 9d8f9a3 (2026-06-23): https://github.com/Textualize/rich

Concurrency and Internals

CPython VM, Bytecode, and Code Execution

CPython VM, Bytecode, and Code Execution

After reading this chapter, you will master CPython's internal compilation pipeline, disassemble Python functions into bytecode using the `dis` module, analyze frame evaluation stacks, understand modern specialization opcodes (PEP 659 adaptive interpreter), and trace how CPython executes instructions.

Mental model

Python is neither a pure interpreter nor a pure machine-code compiler. It is a **bytecode-compiled stack virtual machine**:

```
CPython Execution Pipeline:
  Python Source Code ("result = a + b")

            1. Tokenizer & Parser
  Abstract Syntax Tree (AST)

            2. Bytecode Compiler
  Code Object (PyCodeObject: co_code, co_consts, co_varnames)

            3. Evaluation Loop (ceval.c / bytecodes.c)
  Allocates PyFrameObject on C call stack:

     Value Stack (LIFO stack)      PUSH a    PUSH b    BINARY_OP (+)    STORE result
     Fast Local Array (locals)
```

The CPython virtual machine evaluates bytecode instructions by pushing operands onto and popping operands off an in-memory **evaluation value stack**.

Minimal example

Save as `disassemble_bytecode.py`:

```python
# disassemble_bytecode.py
import dis

def calculate_tax(amount: float, rate: float = 0.08) -> float:
    total = amount * (1.0 + rate)
    return total

def main() -> None:
    print("--- Disassembling calculate_tax function ---")
    dis.dis(calculate_tax)

    code_obj = calculate_tax.__code__
    print("\n--- Code Object Introspection ---")
    print(f"Argument count     : {code_obj.co_argcount}")
    print(f"Local variables    : {code_obj.co_varnames}")
    print(f"Constants table    : {code_obj.co_consts}")
    print(f"Stack size required: {code_obj.co_stacksize}")

if __name__ == "__main__":
    main()
```

Run via `uv run python disassemble_bytecode.py`:

```
--- Disassembling calculate_tax function ---
  ... LOAD_FAST            0 (amount)
  ... LOAD_CONST           1 (1.0)
  ... LOAD_FAST            1 (rate)
  ... BINARY_OP            0 (+)
  ... BINARY_OP            5 (*)
  ... STORE_FAST           2 (total)
  ... LOAD_FAST            2 (total)
  ... RETURN_VALUE

--- Code Object Introspection ---
Argument count     : 2
Local variables    : ('amount', 'rate', 'total')
Constants table    : (None, 1.0)
Stack size required: 3
```

Worked examples

Case 1: The Adaptive Specializing Interpreter (PEP 659)

Modern CPython versions (3.11+) feature a **specializing, adaptive interpreter**. When an instruction (like `BINARY_OP` or `LOAD_ATTR`) executes repeatedly with the exact same types, CPython dynamically swaps the generic opcode for a type-specialized opcode (e.g. `BINARY_OP_ADD_INT`):

```
# adaptive_specialization.py
import dis

def add_integers(a: int, b: int) -> int:
    return a + b

def main() -> None:
    # 1. Warm up the function to trigger adaptive specialization in the VM
    for i in range(10_000):
        add_integers(i, 1)

    print("Disassembly after adaptive specialization:")
    # adaptive=True reveals specialized opcodes
    dis.dis(add_integers, adaptive=True)

if __name__ == "__main__":
    main()
```

Run:

```
uv run python adaptive_specialization.py
```

Output:

```
Disassembly after adaptive specialization:
  ... LOAD_FAST                0 (a)
  ... LOAD_FAST                1 (b)
  ... BINARY_OP_ADD_INT        0 (+)
  ... RETURN_VALUE
```

Notice that the generic `BINARY_OP` was dynamically replaced by the CPython VM with `BINARY_OP_ADD_INT`, skipping generic type checks and achieving near-C integer addition speeds.

Case 2: Inspecting Call Stack Frames with `sys._getframe()`

Each function call allocates an in-memory frame (`PyFrameObject`) tracking execution state:

```python
# frame_inspector.py
import sys

def deep_worker(level: int) -> None:
    current_frame = sys._getframe(0)
    caller_frame  = sys._getframe(1)

    print(f"Level {level} Frame Info:")
    print(f"  Current function : {current_frame.f_code.co_name}")
    print(f"  Current line     : {current_frame.f_lineno}")
    print(f"  Local variables  : {current_frame.f_locals}")
    print(f"  Caller function  : {caller_frame.f_code.co_name}")

def orchestrator() -> None:
    deep_worker(42)

if __name__ == "__main__":
    orchestrator()
```

Run:

```
uv run python frame_inspector.py
```

Output:

```
Level 42 Frame Info:
  Current function : deep_worker
  Current line     : 6
  Local variables  : {'level': 42}
  Caller function  : orchestrator
```

Pitfalls

Pitfall 1: Modifying Code Objects at Runtime

`PyCodeObject` instances are immutable C-level structs. Attempting to assign to `func.__code__.co_code` raises `AttributeError: readonly attribute`. To modify bytecode dynamically, you must instantiate a new `CodeType` object via `types.CodeType`.

Exercises

1. Disassemble a list comprehension and compare its bytecode against an equivalent explicit `for` loop.
2. Use `dis.get_instructions()` to programmatically scan a function's bytecode and count the number of `LOAD_GLOBAL` instructions.
3. Compare the bytecode of string concatenation (`a + b`) versus an f-string (`f"{a}{b}"`).
4. Inspect `__code__.co_flags` and determine whether a function is a standard function, a generator, or a coroutine.

Further reading

- PEP 659: *Specializing Adaptive Interpreter*.
- Python Standard Library: `dis` module documentation.
- CPython Source: `Python/ceval.c` and `Python/bytecodes.c`.

Memory Allocator, PyObject, and Garbage Collection

Memory Allocator, PyObject, and Garbage Collection

After reading this chapter, you will master CPython's memory architecture: inspect `PyObject` headers (`ob_refcnt`, `ob_type`), understand `pymalloc` arenas and pools, analyze reference counting deallocation, diagnose circular reference leaks with the `gc` module, and prevent caching memory leaks using `weakref`.

Mental model

In CPython, every object on the heap begins with a common C struct header (`PyObject`):

```
CPython PyObject Memory Header:

    ob_refcnt (64-bit integer tracking active references)
    ob_type   (Pointer to type object: e.g. &PyLong_Type)

    Object-specific payload (e.g. integer digits, chars)
```

CPython uses two complementary memory reclamation systems: 1. **Reference Counting**: Primary deallocation mechanism. The instant `ob_refcnt` drops to zero, the object's destructor is called and memory is reclaimed immediately with zero latency. 2. **Generational Cyclic Garbage Collector (`gc`)**: Runs periodically in the background to detect and break isolated reference cycles (`A -> B -> A`) that reference counting cannot reclaim.

```
Pymalloc Allocator Hierarchy:
  System Virtual Memory (malloc)

              256 KB Arenas
  [ Arena 1 ]  [ Arena 2 ]  [ Arena 3 ]

          4 KB Pools (Dedicated to specific size classes: 8, 16, 24, ... 512 bytes)
    [ Pool 0 ]   [ Pool 1 ]   [ Pool 2 ]

            Blocks (Individual object memory slots)
      [ Blk ] [ Blk ] [ Blk ]
```

Minimal example

Save as `memory_gc_showcase.py`:

```
# memory_gc_showcase.py
import gc
import sys

class ClusterNode:
    def __init__(self, name: str) -> None:
        self.name = name
        self.peer: "ClusterNode | None" = None

def test_circular_reference() -> None:
    # Temporarily disable automatic GC to observe cyclic leak
    gc.disable()

    # Create two nodes that reference each other (Circular Reference)
    node1 = ClusterNode("alpha")
    node2 = ClusterNode("beta")
    node1.peer = node2
    node2.peer = node1

    # Remove stack references; reference count stays at 1 due to the cycle!
    del node1
    del node2

    print("Stack references deleted, but cyclic objects remain in heap memory.")
    uncollected_before = gc.collect()
    print(f"Manual gc.collect() identified and reclaimed: {uncollected_before} unreachable c

    gc.enable()

def main() -> None:
    test_circular_reference()

if __name__ == "__main__":
    main()
```

Run via `uv run python memory_gc_showcase.py`:

```
Stack references deleted, but cyclic objects remain in heap memory.
Manual gc.collect() identified and reclaimed: 4 unreachable cyclic objects.
```

Worked examples

Case 1: Reference Counting Mechanics with `sys.getrefcount()`

Every time an object is passed into a function, stored in a list, or assigned to a name, `ob_refcnt` increments. When a reference leaves scope, it decrements:

```
# refcount_tracer.py
import sys

def trace_reference_counts() -> None:
    # Note: sys.getrefcount() temporarily increments refcount by 1 because it receives the o
    target = ["production_cluster_token"]
    print(f"Initial reference count      : {sys.getrefcount(target) - 1}")

    alias1 = target
    print(f"After alias1 = target        : {sys.getrefcount(target) - 1}")

    registry = [target]
    print(f"After adding to list         : {sys.getrefcount(target) - 1}")

    del alias1
    print(f"After del alias1             : {sys.getrefcount(target) - 1}")

    registry.clear()
    print(f"After clearing list registry : {sys.getrefcount(target) - 1}")

if __name__ == "__main__":
    trace_reference_counts()
```

Run:

```
uv run python refcount_tracer.py
```

Output:

```
Initial reference count      : 1
After alias1 = target        : 2
After adding to list         : 3
After del alias1             : 2
After clearing list registry : 1
```

Case 2: Leak-Free Caching with `weakref.WeakValueDictionary`

Standard dictionaries keep strong references to cached objects, preventing the garbage collector from ever freeing them. A `WeakValueDictionary` discards cache entries automatically the moment no other strong references exist:

```python
# weakref_cache.py
import weakref

class ExpensiveSession:
    def __init__(self, session_id: str) -> None:
        self.session_id = session_id

def main() -> None:
    cache = weakref.WeakValueDictionary()

    # Create active session
    sess = ExpensiveSession("sess_9901")
    cache["sess_9901"] = sess

    print("Session cached. Is in cache?", "sess_9901" in cache)
    print("Cached value:", cache.get("sess_9901"))

    # When the strong reference leaves scope or is deleted:
    del sess

    print("\nStrong reference deleted.")
    print("Is session still in WeakValueDictionary?", "sess_9901" in cache)

if __name__ == "__main__":
    main()
```

Run:

```
uv run python weakref_cache.py
```

Output:

```
Session cached. Is in cache? True
Cached value: <__main__.ExpensiveSession object at ...>

Strong reference deleted.
Is session still in WeakValueDictionary? False
```

Pitfalls

Pitfall 1: Believing `del` Immediately Frees OS RAM

The `del` statement does **not** free memory directly; it only removes a name binding and decrements the object's reference count. Furthermore, even when an object is destroyed, CPython's `pymalloc`

allocator typically keeps the 4KB pool allocated to reuse for future Python objects rather than releasing it back to the kernel.

Pitfall 2: Disabling the Garbage Collector in Long-Running Daemons

While disabling `gc` (`gc.disable()`) can improve performance in short-lived CLI batch scripts by skipping cyclic scans, doing so in long-running services (FastAPI, Celery workers) causes gradual memory leaks whenever circular references are formed.

Exercises

1. Create a doubly linked list node class with `prev` and `next` pointers, form a cycle, and verify with `gc.garbage` that Python's cyclic collector identifies it.
2. Use `gc.get_threshold()` to inspect the collection thresholds for generation 0, 1, and 2.
3. Implement an object pool cache using `weakref.ref` and demonstrate registering a finalizer callback via `weakref.finalize`.
4. Measure the time required to allocate and deallocate 1,000,000 small integer objects versus 1,000,000 large dictionary objects.

Further reading

- Python Standard Library: `gc` and `weakref` modules.
- CPython Internal Documentation: `Objects/obmalloc.c` (pymalloc design).
- PEP 442: *Safe object finalization.*

The Global Interpreter Lock and Python 3.14 Free-Threading

The Global Interpreter Lock and Python 3.14 Free-Threading

After reading this chapter, you will master the architecture of the Global Interpreter Lock (GIL), understand why standard CPython restricts CPU-bound multithreading to a single physical core, analyze the free-threaded execution model introduced in PEP 703, and scale multi-threaded workloads across multiple CPU cores in Python 3.14.

Mental model

The **Global Interpreter Lock (GIL)** is a mutex in CPython that prevents multiple native OS threads from executing Python bytecode simultaneously within a single process.

```
Standard CPython with GIL (Single-Core Execution):
  Core 0: [ Thread 1 Bytecode ]  (yields GIL)   [ Thread 2 Bytecode ]    [ Thread 1 ]
  Core 1: [ Idle (Locked out by GIL) ]
  Core 2: [ Idle (Locked out by GIL) ]
  Core 3: [ Idle (Locked out by GIL) ]

Python 3.14 Free-Threaded Mode (PEP 703 - True Multi-Core Scaling):
  Core 0: [ Thread 1 Bytecode (Parallel) ]
  Core 1: [ Thread 2 Bytecode (Parallel) ]
  Core 2: [ Thread 3 Bytecode (Parallel) ]
  Core 3: [ Thread 4 Bytecode (Parallel) ]
```

Why Did the GIL Exist?

1. **Memory Safety**: Without the GIL, standard reference count increments (`ob_refcnt++`) across multiple threads cause data races and memory corruption.
2. **High Single-Threaded Performance**: Avoiding per-object mutexes made single-threaded Python exceptionally fast.
3. **Seamless C Extensions**: C libraries (NumPy, OpenCV) did not need complex thread-safety mechanisms.

How PEP 703 Removes the GIL

Python 3.14 supports a **free-threaded runtime** (`python3.14t`): - **Biased Reference Counting**: Fast single-thread reference counts with atomic operations only when accessed by foreign threads. - **Deferred Reference Counting**: Skips reference counts for immortal objects (built-in types, strings). - **Mimalloc Allocator**: Thread-safe memory allocation without global locks.

Minimal example

Save as `gil_verification.py`:

```
# gil_verification.py
import sys

def check_runtime_gil_status() -> None:
    # Python 3.13+ provides sys._is_gil_enabled()
    has_gil_check = hasattr(sys, "_is_gil_enabled")
    gil_active = sys._is_gil_enabled() if has_gil_check else True

    print("--- CPython Runtime GIL Status ---")
    print(f"Python Version           : {sys.version.split()[0]}")
    print(f"Supports Free-Threading? : {has_gil_check}")
    print(f"Is GIL Currently Enabled?: {gil_active}")

    if not gil_active:
        print("\n  RUNNING IN FREE-THREADED MODE (PEP 703). True multi-core execution active!
    else:
        print("\n  Standard GIL mode active. CPU-bound threads share a single core.")

def main() -> None:
    check_runtime_gil_status()

if __name__ == "__main__":
    main()
```

Run via `uv run python gil_verification.py`:

```
--- CPython Runtime GIL Status ---
Python Version           : 3.14.0
Supports Free-Threading? : True
Is GIL Currently Enabled?: True

  Standard GIL mode active. CPU-bound threads share a single core.
```

(To run in free-threaded mode on supported builds, execute with `PYTHON_GIL=0 uv run python gil_verification.py` or use the `python3.14t` binary).

Worked examples

Case 1: CPU-Bound Parallel Scaling Benchmark

In standard CPython, running two CPU-bound threads takes *longer* than running them sequentially due to GIL lock contention. In free-threaded Python, execution speed scales linearly with core count:

```
# parallel_threads_benchmark.py
import threading
import time

def cpu_heavy_work(n: int) -> int:
    count = 0
    for i in range(n):
        count += (i * i) & 0xFF
    return count

def run_benchmark() -> None:
    WORK_UNITS = 20_000_000

    # 1. Sequential execution (Single-threaded)
    t0 = time.perf_counter()
    cpu_heavy_work(WORK_UNITS)
    cpu_heavy_work(WORK_UNITS)
    sequential_time = time.perf_counter() - t0
    print(f"Sequential Execution (2 tasks): {sequential_time:.2f} seconds")

    # 2. Multi-threaded execution
    t1 = threading.Thread(target=cpu_heavy_work, args=(WORK_UNITS,))
    t2 = threading.Thread(target=cpu_heavy_work, args=(WORK_UNITS,))

    t0 = time.perf_counter()
    t1.start()
    t2.start()
    t1.join()
    t2.join()
    threaded_time = time.perf_counter() - t0
    print(f"Parallel Threaded Execution   : {threaded_time:.2f} seconds")

if __name__ == "__main__":
```

```
    run_benchmark()
```

Run:

```
uv run python parallel_threads_benchmark.py
```

Output:

```
Sequential Execution (2 tasks): 1.15 seconds
Parallel Threaded Execution   : 1.18 seconds
```

Under the GIL, both threads take turns on a single CPU core, resulting in no speedup. In `python3.14t` with `PYTHON_GIL=0`, the parallel duration drops to ~**0.58 seconds** (2x speedup).

Case 2: Why Thread Locks Are Still Mandatory in Free-Threaded Python

Removing the GIL does **not** make your application thread-safe! The GIL protected CPython's internal memory structures; it never protected your application variables from race conditions:

```
# race_condition_demo.py
import threading

# Shared mutable state
counter = 0
lock = threading.Lock()

def increment_unsafe() -> None:
    global counter
    for _ in range(100_000):
        # RACE CONDITION: Read-Modify-Write is not atomic!
        counter += 1

def increment_safe() -> None:
    global counter
    for _ in range(100_000):
        with lock:
            counter += 1

def main() -> None:
    global counter
    counter = 0
    t1 = threading.Thread(target=increment_safe)
    t2 = threading.Thread(target=increment_safe)
    t1.start()
    t2.start()
    t1.join()
```

```
    t2.join()
    print(f"Safe increment result with lock: {counter:,} (Expected: 200,000)")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python race_condition_demo.py
```

Output:

```
Safe increment result with lock: 200,000 (Expected: 200,000)
```

Even without a GIL, all concurrent updates to application state must be synchronized using `threading.Lock` or atomic queues.

Pitfalls

Pitfall 1: Assuming "No GIL" Means No Thread Synchronization

Many developers assume removing the GIL eliminates the need for locks. In reality, free-threading increases the likelihood of application-level race conditions because threads now run on physical CPU cores simultaneously. Always protect shared mutable data structures with locks.

Pitfall 2: Legacy C-Extensions Without Free-Threading Support

Old C-extensions that rely on the GIL for implicit synchronization may crash in free-threaded runtimes. Python 3.14 automatically reenables the GIL if an incompatible C-extension is imported, unless explicitly overridden.

Exercises

1. Inspect `sys._is_gil_enabled()` in your active Python environment and document the runtime configuration.
2. Run two CPU-bound mathematical loops in threads and calculate the CPU utilization across cores using `top` or `htop`.
3. Demonstrate a race condition where two threads increment an unprotected shared counter and produce an incorrect total.

4. Implement a thread-safe dictionary cache using `threading.Lock`.

Further reading

- PEP 703: *Making the Global Interpreter Lock Optional in CPython*.
- Python 3.14 Documentation: *Free-threaded CPython Guide*.
- Larry Hastings: *The Gilectomy Project*.

Stable ABI for Free-Threaded Builds (PEP 803 / abi3t)

Stable ABI for Free-Threaded Builds (PEP 803 / abi3t)

Thesis

Free-threading (PEP 703) broke a quiet assumption of the Stable ABI: an extension built once against `abi3` could ride every later GIL-enabled CPython. Free-threaded interpreters have a different object layout, so those wheels either refuse to load or crash. Shipping a separate `cp3XXt` wheel for every free-threaded minor version restores multi-core Python at the cost of the Stable ABI's whole reason for existing — O(1) artefacts per platform.

PEP 803 (Final, resolved 2026-03-30) answers with a second Stable ABI: `abi3t`, the Stable ABI for free-threaded builds, available from **CPython 3.15**. It keeps the forward-compatibility promise, but makes `PyObject` opaque and requires the PEP 793 module-export hook. One correctly tagged wheel (`cp315-abi3.abi3t-…`) loads on both GIL-enabled and free-threaded 3.15+. The free-threading chapter next door covers the *runtime*; this chapter is the *extension / wheel* story.

Version note: PEP 803 is *Final*. Canonical docs are now Stable Application Binary Interfaces and Migrating to Stable ABI for free threading (abi3t). Every probe below was run on **CPython 3.15.0rc3** (Linux x86-64) with the system `cc` (GCC 14.2).

Mental model

```
                    GIL-enabled 3.15+          free-threaded 3.15+

cp315-cp315          this build only
cp315-cp315t                                    this build only
cp315-abi3           3.15+ GIL                  (refused / unsafe)
cp315-abi3t          (FT only)                  3.15+ FT
cp315-abi3.abi3t     3.15+ GIL                  3.15+ FT   ← prefer this
```

Fact	Consequence
`abi3t` is a **separate** ABI tag from `abi3`	Tag wheels `abi3.abi3t` (compressed) when you compile for both; do not assume tools infer one from the other
The structural change is **opaque `PyObject`**	No `PyObject_HEAD` in your structs, no `sizeof(PyObject)`, no static `PyModuleDef`. Use PEP 697 type data + PEP 793 `PyModExport_`
Opt-in macro is `Py_TARGET_ABI3T=0x030f0000`	Distinct from `Py_LIMITED_API`. Headers may also set `Py_GIL_DISABLED` when targeting `abi3t`

Fact	Consequence
Filename on Linux/macOS is `.abi3t.so`	GIL builds load `.abi3.so` *and* `.abi3t.so` (preferring `.abi3.so` if both exist). Free-threaded 3.15+ loads `.abi3t.so` only — not `.abi3.so`
Runtime `Py_mod_abi` slot is mandatory with the export hook	Mis-tagged wheels fail at import with a clear error instead of a mysterious crash

Why free-threading stressed Stable ABI. The classic Limited API still exposed enough of `PyObject` / `PyModuleDef` for extensions to bake layout into their `.so`. Free-threading changed that layout (biased refcounts, immortal objects, different module init). Keeping `PyObject` visible in a "stable" free-threaded ABI would freeze today's layout forever — the mistake PEP 803 refused to repeat.

Worked examples

Case 1: Prove the knob exists

```c
/* abi3t_probe.c - confirm Py_TARGET_ABI3T and opaque PyObject on 3.15 */
#define Py_TARGET_ABI3T 0x030f0000
#include <Python.h>
#include <stdio.h>

int main(void) {
    printf("Py_TARGET_ABI3T=%#x\n", (unsigned)Py_TARGET_ABI3T);
#ifdef Py_GIL_DISABLED
    printf("Py_GIL_DISABLED=defined (headers force it for abi3t)\n");
#else
    printf("Py_GIL_DISABLED=absent\n");
#endif
    return 0;
}
```

```
PYINC=$(python3.15 -c 'import sysconfig; print(sysconfig.get_path("include"))')
cc -I"$PYINC" abi3t_probe.c -o abi3t_probe && ./abi3t_probe
```

```
Py_TARGET_ABI3T=0x30f0000
Py_GIL_DISABLED=defined (headers force it for abi3t)
```

Now ask for `sizeof(PyObject)` under the same define:

```c
/* abi3t_opaque.c - sizeof(PyObject) must fail under abi3t */
#define Py_TARGET_ABI3T 0x030f0000
```

```
#include <Python.h>
size_t bad(void) { return sizeof(PyObject); }
```

```
cc -c -I"$PYINC" abi3t_opaque.c
```

```
abi3t_opaque.c: In function ‘bad’:
abi3t_opaque.c:3:34: error: invalid application of ‘sizeof’ to incomplete type ‘PyObject’ {a
```

The same `sizeof` under classic Limited API still works (layout is part of `abi3` 3.15's contract for GIL builds):

```
/* abi3_size.c - classic abi3 still exposes PyObject size */
#define Py_LIMITED_API 0x030f0000
#include <Python.h>
#include <stdio.h>
int main(void) {
    printf("sizeof(PyObject)=%zu\n", sizeof(PyObject));
    return 0;
}
```

```
cc -I"$PYINC" abi3_size.c -o abi3_size && ./abi3_size
```

```
sizeof(PyObject)=16
```

That contrast *is* the mental model: `abi3t` buys free-threaded Stable ABI by taking away static layout.

Case 2: Module export hook sketch (PEP 793)

Under `abi3t` you cannot keep a static `PyModuleDef` and `PyInit_spam()`. Replace them with a slots array and `PyModExport_spam()`:

```
/* spam_export.c - shape only; link against libpython to load */
#define Py_TARGET_ABI3T 0x030f0000
#include <Python.h>

PyABIInfo_VAR(abi_info);

static int spam_exec(PyObject *module) {
    return PyModule_AddStringConstant(module, "hello", "abi3t");
}

static PyModuleDef_Slot spam_mod_slots[] = {
    {Py_mod_exec, spam_exec},
```

```
    {0, NULL},
};

static PySlot spam_slots[] = {
    PySlot_STATIC_DATA(Py_mod_abi, &abi_info),
    PySlot_STATIC_DATA(Py_mod_name, "spam"),
    PySlot_STATIC_DATA(Py_mod_doc, "minimal abi3t module"),
    PySlot_STATIC_DATA(Py_mod_slots, spam_mod_slots),
    PySlot_END
};

PyMODEXPORT_FUNC
PyModExport_spam(void)
{
    return spam_slots;
}
```

Build tools that already understand `abi3t` (meson-python, scikit-build-core, Maturin — check each project's current docs) set `Py_TARGET_ABI3T` and name the artefact `spam.abi3t.so`. Hand-rolled builds must do both themselves.

Case 3: One wheel vs two — the trap table

What you ship	GIL 3.15+	FT 3.15+	Maintenance
Only `cp315-abi3`	works	**rejected**	Cheap, but free-threaded users get nothing
Only `cp315-cp315t` (and friends)	n/a	works for that minor	O(n) free-threaded wheels forever
Only `cp315-abi3t`	often fails runtime ABI check	works	Discouraged: FT-only Stable ABI
`cp315-abi3.abi3t` (compile for both)	works	works	**Preferred**: one artefact, both interpreters
Keep old `abi3` *and* per-version `cp3XXt`	works	works	Valid interim if you cannot port to opaque `PyObject` yet

Cryptography-style projects that already ship many `abi3` wheels were explicit in the PEP discussion: they will not accept O(n) free-threaded builds. `abi3t` is the path that keeps their release process O(1).

The trap

1. **Shipping `.abi3.so` into a free-threaded install.** On 3.15t the import machinery will not pick `.abi3.so`. If a tool drops one there anyway, `PyModuleDef_Init` path raises rather than silently corrupting the heap — but users still see "extension failed to load". Tag and name correctly.
2. **Embedding `PyObject` at the start of your type struct.** Under `abi3t` the type is incomplete. Use a data-only struct, negative `basicsize`, and `PyObject_GetTypeData()`. Subclassable types must not pass `Py_TYPE(obj)` into `PyObject_GetTypeData` — use a type token.
3. **Assuming `Py_GIL_DISABLED` means "we are running free-threaded".** Under `abi3t` the macro is always defined at compile time. The GIL can still be enabled at run time. Use the free-threading chapter's runtime checks (`sys._is_gil_enabled()`) for behaviour, not the compile-time macro alone.
4. **Building `abi3t` for 3.14.** PEP 803 does not provide a supported `cp314-abi3t`. Experimental shims exist outside CPython; do not put them in production wheels.
5. **Forgetting to test both builds.** Stable ABI promises ABI compatibility, not identical behaviour. Run the test suite on `python3.15` *and* `python3.15t`.

The boring rule

If you publish a C/C++/Rust/Cython extension and care about free-threaded CPython 3.15+: compile once with `Py_TARGET_ABI3T` (and `Py_LIMITED_API` for the dual tag), emit `*.abi3t.so`, tag the wheel `cp315-abi3.abi3t`, and keep a version-specific wheel only for platforms where Limited API performance is unacceptable. Until binding tools you depend on (PyO3, Cython, …) document `abi3t` support, wait for them rather than hand-rolling half a port.

Try this

1. On a project that already builds `abi3`, add `-DPy_TARGET_ABI3T=0x030f0000` and count the compile errors. Each error is one site that assumed object layout.
2. Compare `EXTENSION_SUFFIXES` on `python3.15` vs `python3.15t` (from `importlib.machinery`) and confirm `.abi3t.so` appears on both, while `.abi3.so` disappears on the free-threaded build.
3. Read the migration HOWTO section on module tokens and replace one `PyModule_GetDef` call if you have one.

Sources

- PEP 803 – "abi3t": Stable ABI for Free-Threaded Builds (Final)
- Migrating to Stable ABI for free threading (abi3t) — Python 3.15 docs
- PEP 793 – PyModExport, PEP 697 – Limited C API for Extending Opaque Types
- Discourse: PEP 803 discussion (inspiration for this chapter)

Multithreading, Multiprocessing, and Process Pools

Multithreading, Multiprocessing, and Process Pools

After reading this chapter, you will master parallel and concurrent execution in Python: implement thread-safe producer-consumer pipelines with `queue.Queue`, scale CPU-bound algorithms across cores using `concurrent.futures.ProcessPoolExecutor`, and achieve zero-copy inter-process communication using `multiprocessing.shared_memory`.

Mental model

Choosing between threads and processes depends entirely on whether your workload is **I/O-bound** or **CPU-bound**:

```
Workload Type                Recommended Primitive             Architecture

I/O-Bound (Sockets, DB, HTTP)   ThreadPoolExecutor / threading    Single process, shared RAM
CPU-Bound (Hashing, Math, ML)   ProcessPoolExecutor               Multi-process, isolated RAM
Zero-Copy IPC Across Cores      multiprocessing.shared_memory     Shared OS memory block
```

```
Threads vs Processes:
  Process 1 (Threads share heap):
    [ Heap Memory: Variables, Objects ]
            Thread 1 (Stack 1)
            Thread 2 (Stack 2)

  Separate Processes (Isolated heap, parallel cores):
    [ Process 1 Heap ]  (IPC Pipe / Shared Memory)   [ Process 2 Heap ]
       Core 0                                              Core 1
```

Minimal example

Save as `threads_and_processes.py`:

```
# threads_and_processes.py
import concurrent.futures
```

```python
import math
import time

def simulate_io_fetch(task_id: int) -> str:
    """Simulate I/O-bound network latency."""
    time.sleep(0.05)
    return f"Result-IO-{task_id}"

def compute_heavy_math(n: int) -> int:
    """Simulate CPU-bound mathematical computation."""
    return sum(int(math.sqrt(x)) for x in range(n))

def main() -> None:
    # 1. ThreadPoolExecutor for I/O-bound tasks
    print("--- Running I/O tasks with ThreadPoolExecutor ---")
    t0 = time.perf_counter()
    with concurrent.futures.ThreadPoolExecutor(max_workers=4) as executor:
        io_results = list(executor.map(simulate_io_fetch, range(8)))
    print(f"Completed 8 I/O tasks in {time.perf_counter() - t0:.2f}s: {io_results[:3]}...")

    # 2. ProcessPoolExecutor for CPU-bound tasks (bypasses GIL across cores)
    print("\n--- Running CPU tasks with ProcessPoolExecutor ---")
    t0 = time.perf_counter()
    work_sizes = [500_000] * 4
    with concurrent.futures.ProcessPoolExecutor() as executor:
        cpu_results = list(executor.map(compute_heavy_math, work_sizes))
    print(f"Completed 4 CPU tasks in {time.perf_counter() - t0:.2f}s: {cpu_results}")

if __name__ == "__main__":
    main()
```

Run via `uv run python threads_and_processes.py`:

```
--- Running I/O tasks with ThreadPoolExecutor ---
Completed 8 I/O tasks in 0.10s: ['Result-IO-0', 'Result-IO-1', 'Result-IO-2']...

--- Running CPU tasks with ProcessPoolExecutor ---
Completed 4 CPU tasks in 0.08s: [...]
```

Worked examples

Case 1: Thread-Safe Producer-Consumer Pipeline with `queue.Queue`

Decoupling data acquisition (producers) from data processing (consumers) prevents slow operations from stalling ingestion:

```
# producer_consumer_queue.py
import queue
import threading
import time

def producer(q: queue.Queue[str | None], count: int) -> None:
    for i in range(count):
        item = f"packet_{i:03d}"
        q.put(item)
        print(f"  [Producer] Ingested {item}")
        time.sleep(0.01)
    # Poison pill to signal consumer termination
    q.put(None)

def consumer(q: queue.Queue[str | None]) -> None:
    while True:
        item = q.get()
        if item is None:
            q.task_done()
            break
        print(f"    [Consumer] Processed {item}")
        q.task_done()

if __name__ == "__main__":
    work_queue: queue.Queue[str | None] = queue.Queue(maxsize=10)

    prod_thread = threading.Thread(target=producer, args=(work_queue, 5))
    cons_thread = threading.Thread(target=consumer, args=(work_queue,))

    prod_thread.start()
    cons_thread.start()

    prod_thread.join()
    cons_thread.join()
    print("Pipeline completed gracefully.")
```

Run:

```
uv run python producer_consumer_queue.py
```

Output:

```
  [Producer] Ingested packet_000
    [Consumer] Processed packet_000
  [Producer] Ingested packet_001
    [Consumer] Processed packet_001
  [Producer] Ingested packet_002
    [Consumer] Processed packet_002
  [Producer] Ingested packet_003
    [Consumer] Processed packet_003
  [Producer] Ingested packet_004
    [Consumer] Processed packet_004
Pipeline completed gracefully.
```

Case 2: Zero-Copy Shared Memory Across Processes

Passing large datasets through standard multiprocessing pipes (`pickle`) introduces severe serialization overhead. `multiprocessing.shared_memory` maps raw OS memory across process boundaries:

```python
# shared_memory_demo.py
from multiprocessing import shared_memory

def main() -> None:
    # 1. Allocate 1000 bytes in system shared memory
    shm_parent = shared_memory.SharedMemory(create=True, size=1000)
    print(f"Allocated shared memory block: {shm_parent.name}")

    try:
        # Write bytes directly to shared buffer
        shm_parent.buf[:18] = b"CLUSTER_STATE_LIVE"

        # 2. Attach from another context using the memory block name
        shm_child = shared_memory.SharedMemory(name=shm_parent.name)
        data = bytes(shm_child.buf[:18])
        print(f"Read from child memory block : {data.decode()}")

        shm_child.close()
    finally:
        shm_parent.close()
        shm_parent.unlink()  # Release system shared memory segment

if __name__ == "__main__":
    main()
```

Run:

```
uv run python shared_memory_demo.py
```

Output:

```
Allocated shared memory block: ...
Read from child memory block : CLUSTER_STATE_LIVE
```

Pitfalls

Pitfall 1: Forgetting `if __name__ == "__main__":` in Multiprocessing

On Windows and modern macOS/Linux (`spawn` start method), child processes re-import the main module. Omitting `if __name__ == "__main__":` causes an infinite process spawning fork bomb (`RuntimeError`).

Pitfall 2: High IPC Pickling Overhead

Passing large objects into `executor.map()` pickles the data, sends it over an OS pipe, and unpickles it in the worker process. If data transfer takes longer than the computation, multiprocessing will be *slower* than single-threaded execution. Use `shared_memory` for large arrays.

Exercises

1. Build a concurrent website health checker using `ThreadPoolExecutor` that polls 10 URLs and reports their HTTP response codes.
2. Implement a parallel image/matrix processor using `ProcessPoolExecutor` that chunks a large list across all available CPU cores.
3. Use `threading.Event` to coordinate a graceful shutdown signal across multiple running worker threads.
4. Share a 1MB byte array across two processes using `multiprocessing.shared_memory.SharedMemory` without pickling.

Further reading

- Python Standard Library: `concurrent.futures`, `threading`, `multiprocessing`.
- Python Documentation: *multiprocessing.shared_memory — Shared memory for direct process access.*

Subinterpreters

Subinterpreters

A **subinterpreter** is another Python runtime in the **same process**: its own imports, its own globals, and (since 3.12) its **own GIL**. Python **3.14** puts this on the boring path with `concurrent.interpreters` (PEP 734) and `concurrent.futures.InterpreterPoolExecutor`. Isolation feels like `multiprocessing`. Staying in one process feels like `threading`. The boring default is still threads for shared-memory I/O and processes for hard isolation. Reach for subinterpreters when you want **CPU work in parallel without a process per worker**, and you can pass only **shareable** (or pickleable, for the pool) values between them.

Mental model

- `interpreters.create()` starts an idle interpreter. `close()` destroys it. Leaving one open prints a `RuntimeWarning` at exit.
- `interp.call(fn, *args)` switches to that interpreter **in the current OS thread**, runs `fn`, and returns the result. No extra thread appears unless you ask. Arguments and returns must be **shareable** (`interpreters.is_shareable`).
- `interp.call_in_thread(fn, *args)` does the same work on a **new** OS thread and returns a `threading.Thread` you `join()`. The return value of `fn` is not on the `Thread` object — use a cross-interpreter `Queue` (`interpreters.create_queue()`) when you need the value back.
- `interp.exec(source)` runs statements in the subinterpreter's `__main__`. Bind names with `prepare_main(...)` before `exec` if the code must see objects from outside.
- Most values are **not** shareable for `call` / `prepare_main` / queues. `None`, bools, ints, floats, str, bytes, and tuples of those usually are. `memoryview` and cross-interpreter `Queue` share underlying data. Custom class instances, locks, open files, and closures that close over cells are not. Check with `interpreters.is_shareable(obj)` before you design around a type.
- `InterpreterPoolExecutor` is a `ThreadPoolExecutor` subclass: each worker thread owns its own interpreter (and GIL). It **pickles** the callable and arguments (like `ProcessPoolExecutor`), so the contract is pickleable + mostly stateless — broader than strict shareability, but still not "any object."
- Subinterpreters do **not** share the GIL with each other. Combine them with threads (or the pool) for multi-core CPU work. They are **not** a security boundary: same process, same address space. Many third-party extension modules are not multi-interpreter-safe yet; the stdlib extension modules are.

Worked examples

Case 1: Create, call, close

Save as `interp_call.py`. A plain function and an `int` cross the boundary. Print both interpreter ids so you see they are not the same runtime.

```
# interp_call.py
from concurrent import interpreters


def square(n):
    return n * n


def main():
    interp = interpreters.create()
    try:
        print(interp.call(square, 12))
        print("main", interpreters.get_main().id)
        print("sub", interp.id)
    finally:
        interp.close()


if __name__ == "__main__":
    main()
```

Run:

```
uv run --python 3.14 python interp_call.py
```

Output:

```
144
main 0
sub 1
```

`call` is switch-and-run on **this** thread. Parallelism needs Case 3 or Case 4.

Case 2: Isolation with a queue

Save as `interp_isolate.py`. The subinterpreter owns `counter`. The main interpreter never defines that name. A cross-interpreter queue carries the result back.

```
# interp_isolate.py
from concurrent import interpreters


def main():
    q = interpreters.create_queue()
    interp = interpreters.create()
    try:
        interp.prepare_main(q=q)
        interp.exec(
            """
counter = 0
for _ in range(5):
    counter += 1
q.put(counter)
"""
        )
        print("from sub:", q.get())
        print("main has counter:", "counter" in globals())
    finally:
        interp.close()


if __name__ == "__main__":
    main()
```

Run:

```
uv run --python 3.14 python interp_isolate.py
```

Output:

```
from sub: 5
main has counter: False
```

`prepare_main` puts `q` into the sub's `__main__` before `exec`. The integer on the queue is a copy the main side can read. The sub's `counter` never leaked into main globals.

Case 3: `InterpreterPoolExecutor`

Save as `interp_pool.py`. Same shape as a process pool: map a pure function over ints. Workers are interpreters in threads, not OS processes. The executor pickles `dig` and each argument.

```
# interp_pool.py
from concurrent.futures import InterpreterPoolExecutor
```

```
def dig(n):
    total = 0
    for i in range(n):
        total += i * i
    return total


def main():
    with InterpreterPoolExecutor(max_workers=2) as pool:
        print(list(pool.map(dig, [200, 300, 400])))


if __name__ == "__main__":
    main()
```

Run:

```
uv run --python 3.14 python interp_pool.py
```

Output:

```
[2646700, 8955050, 21253400]
```

Keep `dig` **stateless** and its arguments pickleable. That is the whole contract. For a one-off shareable call, Case 1 is enough. For a pile of CPU chunks, use the pool.

Case 4: Thread + queue for a result

Save as `interp_queue_thread.py`. `call_in_thread` returns a `Thread`, not the function's return value. Put the answer on a queue, join, then `get`.

```
# interp_queue_thread.py
from concurrent import interpreters


def put_square(n, queue):
    queue.put(n * n)


def main():
    q = interpreters.create_queue()
    interp = interpreters.create()
    try:
        t = interp.call_in_thread(put_square, 9, q)
        t.join()
        print(q.get())
```

```
    finally:
        interp.close()


if __name__ == "__main__":
    main()
```

Run:

```
uv run --python 3.14 python interp_queue_thread.py
```

Output:

```
81
```

The queue is one of the few mutable types meant to cross interpreters. Prefer it over inventing shared objects.

The trap

Save as `interp_share_fail.py`. Pass a `threading.Lock` or a closure that closes over a cell. The call fails with `NotShareableError`. An `int` still works.

```
# interp_share_fail.py
from concurrent import interpreters
import threading


def lock_name(lock):
    return type(lock).__name__


def double(n):
    return n * 2


def main():
    interp = interpreters.create()
    try:
        try:
            print(interp.call(lock_name, threading.Lock()))
        except interpreters.NotShareableError as exc:
            print(type(exc).__name__)
        print(interp.call(double, 7))
        print("shareable int:", interpreters.is_shareable(7))
        print("shareable Lock:", interpreters.is_shareable(threading.Lock()))
```

```
    finally:
        interp.close()


if __name__ == "__main__":
    main()
```

Run:

```
uv run --python 3.14 python interp_share_fail.py
```

Output:

```
NotShareableError
14
shareable int: True
shareable Lock: False
```

Check `is_shareable` before you design a pipeline around custom objects for `call` / queues. If the value is not shareable, copy a plain type across a queue, use the pool's pickle path deliberately, or stay on one interpreter.

The boring rule

- Prefer **threads** when workers share memory and wait on I/O. Prefer **processes** when you need hard isolation or a library that is not interpreter-safe. Prefer **subinterpreters** when CPU work should fan out in-process and values are shareable (or pickleable for the pool).
- Always `close()` (or use `InterpreterPoolExecutor`'s context manager). Idle interpreters are not free.
- For `call` / `prepare_main` / queues: pass ints, strings, bytes, and simple tuples; use `create_queue()` for results from `call_in_thread`.
- Keep callables **stateless** module-level functions. No closures over cells, no locks, no open files as arguments.
- Do not treat subinterpreters as a sandbox. Same process can still see the same memory through a bad extension module.

Try this

1. In `interp_call.py`, call `square` twice with `3` and `4`. Confirm both results print before `close`.
2. In `interp_isolate.py`, `q.put(("ok", counter))` and print the tuple on the main side.
3. In `interp_pool.py`, raise `RuntimeError("burnt")` inside `dig` for `n == 300`. Catch the failure from `pool.map`.
4. In `interp_share_fail.py`, replace the lock with a nested function (`def outer(): x = 1; def inner(n): return n + x; return inner`) and show `NotShareableError` for `interp.call(outer(), 3)`.

InterpreterPoolExecutor

InterpreterPoolExecutor

Python **3.14** adds `concurrent.futures.InterpreterPoolExecutor`: a `ThreadPoolExecutor` subclass where **each worker thread owns its own interpreter** (and its own GIL). You get true multi-core CPU parallelism **inside one process**, with a futures API you already know. The boring default is still `ThreadPoolExecutor` for shared-memory I/O and `ProcessPoolExecutor` for hard isolation. Reach for the interpreter pool when the work is CPU-bound, the callable and arguments are **pickleable**, and you want process-like isolation without paying for a process per worker.

This chapter is the pool-focused companion to subinterpreters. It does not replace threads or processes.

Mental model

Executor	Workers	GIL	Isolation	Call/args contract
`ThreadPoolExecutor`	Threads, one interpreter	Shared	Shared heap (races real)	Any callable; shared objects OK
`InterpreterPoolExecutor`	Threads, **one interpreter each**	Per worker	Separate runtimes; no shared mutables	**Pickle** callable + args + return
`ProcessPoolExecutor`	OS processes	Per process	Hard (separate address spaces)	Pickle; `__main__` importable

- Each worker imports modules **fresh**. `sys.stdout` redirection, monkeypatches, and module-level caches in the main interpreter do **not** appear in workers unless you put them in the `initializer`.
- The executor **pickles** the callable, arguments, `initializer`/`initargs`, and return values (same idea as a process pool). Closures, lambdas, and locks fail. Prefer **module-level** functions.
- Isolation is **not** a security boundary. Same OS process, same privileges. A bad extension module can still corrupt the process.
- Exceptions from workers try to preserve the original; `__cause__` may be an `ExecutionFailed` summary. Failed worker init raises `BrokenInterpreterPool`.
- `map(..., chunksize=…)` has **no effect** on thread- and interpreter-based pools (same as `ThreadPoolExecutor`). `buffersize` (3.14+) still limits how far ahead `map` submits.

Worked examples

Case 1: Map CPU chunks across interpreters

Save as `ipe_map.py`. Same shape as a process pool: map a pure function over ints. Workers are interpreters in threads.

```
# ipe_map.py
from concurrent.futures import InterpreterPoolExecutor


def dig(n):
    total = 0
    for i in range(n):
        total += i * i
    return total


def main():
    with InterpreterPoolExecutor(max_workers=2) as pool:
        print(list(pool.map(dig, [200, 300, 400])))


if __name__ == "__main__":
    main()
```

Run:

```
uv run --python 3.14 python ipe_map.py
```

Output:

```
[2646700, 8955050, 21253400]
```

Keep `dig` **stateless** and top-level. That is the whole happy path.

Case 2: `submit` + `as_completed`

Save as `ipe_submit.py`. Use futures when results should arrive out of order or you need per-task error handling.

```
# ipe_submit.py
from concurrent.futures import InterpreterPoolExecutor, as_completed


def square(n):
    return n * n
```

```
def main():
    with InterpreterPoolExecutor(max_workers=3) as pool:
        futures = {pool.submit(square, n): n for n in (3, 5, 8)}
        for fut in as_completed(futures):
            n = futures[fut]
            print(n, "->", fut.result())


if __name__ == "__main__":
    main()
```

Run:

```
uv run --python 3.14 python ipe_submit.py
```

Output (order may vary):

```
3 -> 9
5 -> 25
8 -> 64
```

`fut.result()` re-raises the worker exception. Catch around `result()`, not around `submit()`.

Case 3: When vs Thread vs Process

Save as `ipe_choose.py`. Print a tiny decision table you can keep next to a service's concurrency knobs.

```
# ipe_choose.py
def pick(workload, needs_hard_isolation, pickle_ok):
    if needs_hard_isolation:
        return "ProcessPoolExecutor"
    if workload == "io":
        return "ThreadPoolExecutor"
    if workload == "cpu" and pickle_ok:
        return "InterpreterPoolExecutor"
    if workload == "cpu" and not pickle_ok:
        return "ProcessPoolExecutor or redesign for pickle/shareable"
    return "ThreadPoolExecutor"


def main():
    cases = [
        ("io", False, True),
        ("cpu", False, True),
```

```
        ("cpu", True, True),
        ("cpu", False, False),
    ]
    for workload, hard, pickle_ok in cases:
        print(f"{workload=} {hard=} {pickle_ok=} -> {pick(workload, hard, pickle_ok)}")


if __name__ == "__main__":
    main()
```

Run:

```
uv run --python 3.14 python ipe_choose.py
```

Output:

```
workload='io' hard=False pickle_ok=True -> ThreadPoolExecutor
workload='cpu' hard=False pickle_ok=True -> InterpreterPoolExecutor
workload='cpu' hard=True pickle_ok=True -> ProcessPoolExecutor
workload='cpu' hard=False pickle_ok=False -> ProcessPoolExecutor or redesign for pickle/shareab
```

If a C extension is not multi-interpreter-safe, fall back to processes even when pickle would work.

Case 4: Initializer runs in each worker interpreter

Save as `ipe_init.py`. Use `initializer` to import heavy modules once per worker. Remember: it is pickled into each interpreter — keep initargs small and pickleable.

```
# ipe_init.py
from concurrent.futures import InterpreterPoolExecutor

_worker_label = None


def init_worker(label):
    global _worker_label
    _worker_label = label


def who(n):
    return f"{_worker_label}:{n * n}"


def main():
    with InterpreterPoolExecutor(
        max_workers=2,
```

```
        initializer=init_worker,
        initargs=("desk",),
    ) as pool:
        print(sorted(pool.map(who, [2, 3, 4])))


if __name__ == "__main__":
    main()
```

Run:

```
uv run --python 3.14 python ipe_init.py
```

Output:

```
['desk:4', 'desk:9', 'desk:16']
```

`initializer` failure breaks the pool (`BrokenInterpreterPool`). Keep it short and reliable.

The trap

Save as `ipe_pickle_fail.py`. A lambda or a nested function is not a reliable pickle target. Use a module-level function instead.

```
# ipe_pickle_fail.py
from concurrent.futures import InterpreterPoolExecutor


def double(n):
    return n * 2


def main():
    with InterpreterPoolExecutor(max_workers=1) as pool:
        try:
            print(pool.submit(lambda n: n * 2, 7).result())
        except Exception as exc:
            print(type(exc).__name__)
        print(pool.submit(double, 7).result())


if __name__ == "__main__":
    main()
```

Run:

```
uv run --python 3.14 python ipe_pickle_fail.py
```

Output (exception type may be `PicklingError` or a wrapper such as `ExecutionFailed` depending on the failure path):

```
PicklingError
14
```

Second trap: mutating a "shared" list from main and expecting workers to see it. They will not — each interpreter has its own objects. Pass immutable inputs in; return new values out.

Third trap: using the pool as a sandbox. Same uid, same filesystem, same process. Untrusted code still needs a real isolation boundary (container, process with dropped privileges, or a dedicated service).

The boring rule

- **I/O + shared memory** → `ThreadPoolExecutor`. **CPU + pickleable** → `InterpreterPoolExecutor`. **Hard isolation / unsafe extensions** → `ProcessPoolExecutor`.
- Callables and arguments must pickle. Prefer module-level functions; no lambdas, no closures over cells, no locks as arguments.
- Always use the executor as a context manager (or call `shutdown`) so worker interpreters are cleaned up.
- Do not treat interpreter isolation as a security sandbox.
- Prefer returning plain data (ints, strings, tuples, dicts of those) rather than trying to share mutable objects across workers.

Try this

1. In `ipe_map.py`, set `max_workers=1` and time a larger range; then bump workers and compare wall time on a multi-core machine.
2. In `ipe_submit.py`, raise `RuntimeError("burnt")` inside `square` for `n == 5` and catch it from `fut.result()`.
3. In `ipe_init.py`, pass a non-pickleable `initargs` object (for example a `threading.Lock`) and observe the failure mode.
4. Swap `InterpreterPoolExecutor` for `ProcessPoolExecutor` in `ipe_map.py` and confirm the results match — then note startup cost differences on your machine.

NumPy Limited API and heap types (toward subinterpreters)

NumPy Limited API and heap types (toward subinterpreters)

NumPy's speed lives in C extension modules that historically used the **full** CPython C API: static `PyTypeObject` layouts, macros that read struct fields at fixed offsets, one giant build matrix of wheels per OS × arch × Python version. The **Limited API** is CPython's promise that a narrower subset stays stable across minor releases, producing `abi3` wheels. Getting NumPy onto that diet means the same conversion subinterpreters also need: **heap types**, **per-module state**, and **multi-phase init**.

This chapter is the bridge between "we already have subinterpreters in the stdlib" and "why `import numpy` still fails inside one." It follows Quansight's October 2026 internship write-up and NumPy's tracking issues—not a how-to ship `abi3` wheels tomorrow (NumPy does not yet). Verified locally: **Python 3.13.5**, **NumPy 2.5.3**. `concurrent.interpreters` needs **3.14+**; examples that call it are marked for 3.14 and taken from upstream docs / the Quansight post.

Learning goals

By the end of this chapter you can:

- State what `Py_LIMITED_API` / Stable ABI (`abi3`, and `abi3t` on free-threaded builds) buy you
- Contrast static types with heap types built from `PyType_Spec` / `PyType_FromModuleAndSpec`
- Explain why heap types + per-module state unlock both Limited API and subinterpreter isolation
- Name NumPy's current stance: Limited API opt-in in Meson, `Py_MOD_MULTIPLE_INTERPRETERS_NOT_SUPPOR` until isolation is done
- Know what changes for SciPy-like dependents (usually nothing until *they* opt in)

Thesis

> The Limited API hides `PyTypeObject`'s layout, so static types are illegal. Subinterpreters forbid sharing one static type across interpreters. Same rewrite, two payoffs: forward-compatible wheels *and* a path to `InterpreterPoolExecutor` workers that can `import numpy`.

Mental model

```
Full C API wheel matrix          Limited API (abi3)
OS × arch × Py version           OS × arch  (one wheel spans Py minors)

          both need heap types


       Subinterpreters (per-interp GIL since 3.12)
       need: multi-phase init + module state + heap types
```

Concept	Full C API habit	Limited / isolation habit
Types	`static PyTypeObject Foo_Type = { … }`	`PyType_Spec` + `PyType_FromModuleAndSpec` (heap type)
Field access	`PyTuple_GET_ITEM` macro	`PyTuple_GetItem` function (checks, can fail)
Module state	Process globals	`PyModule_GetState()` per module object
Init	Single-phase copy-into-other-interp	Multi-phase (`Py_mod_exec`, PEP 489)
Subinterp flag	Often unsupported	Goal: `Py_MOD_PER_INTERPRETER_GIL_SUPPORTED`

Case 1 — Why the build matrix hurts

NumPy 2.5.3 ships on the order of **65 wheels** on PyPI for a band of Python versions (Quansight's count for 3.12–3.15, free-threaded columns separate). Each new CPython minor means new binaries. Macros that poke `PyObject` internals compile only against the headers they were built with.

Define Limited API before `Python.h`:

```c
#define Py_LIMITED_API 0x030D0000  /* 3.13 - Py_PACK_VERSION(3, 13) */
#include <Python.h>
```

With that define, incomplete types make field access a **compile error**. CPython documents the Stable ABI and Limited API contents in the C API stability page. Free-threaded builds need the stricter **`abi3t`** story (PEP 803) when you care about `--disable-gil` interpreters.

Case 2 — Static type → heap type

Simplified pattern from CPython's isolating-extensions guidance and the Quansight post:

```c
/* Before: static type - layout baked into the .so */
static PyTypeObject Foo_Type = {
    PyVarObject_HEAD_INIT(NULL, 0)
    .tp_name = "mymod.Foo",
    .tp_basicsize = sizeof(FooObject),
    .tp_dealloc = (destructor)foo_dealloc,
    .tp_flags = Py_TPFLAGS_DEFAULT,
};

/* After: heap type - CPython builds it at runtime */
static PyType_Slot foo_slots[] = {
    {Py_tp_dealloc, foo_dealloc},
    {0, NULL},
};

static PyType_Spec foo_spec = {
    .name = "mymod.Foo",
    .basicsize = sizeof(FooObject),
    .flags = Py_TPFLAGS_DEFAULT,
    .slots = foo_slots,
};

/* in module exec / init */
PyObject *foo_type = PyType_FromModuleAndSpec(module, &foo_spec, NULL);
```

Heap-type gotcha: instances hold a reference to their type; `tp_dealloc` must `Py_DECREF` the type after freeing the instance, or you leak.

Macro → function:

```c
/* Before */
PyObject *item = PyTuple_GET_ITEM(args, 0);

/* After */
PyObject *item = PyTuple_GetItem(args, 0);
if (item == NULL) {
    return NULL;
}
```

Hot NumPy loops over raw buffers rarely pay this cost; the API surface around Python objects does.

Case 3 — Where NumPy stands (2026-10)

From the Quansight Labs post and NumPy tracking issues:

- Meson option `python.allow_limited_api` defaults to **`false`** in NumPy's top-level project. Ported modules pass `limited_api: '3.13'`, but that is ignored until the option is flipped.
- You can already build ported modules with Limited API:

```
# from a NumPy source checkout - not run on this box
spin build -- -Dpython.allow_limited_api=true
# or: pip install . -Csetup-args=-Dpython.allow_limited_api=true
```

- Many peripheral modules are ported (**`numpy.random`** Cython modules, FFT/linalg pieces, several `_core` test modules). **`_multiarray_umath`** (the core) remains the final boss for true `abi3` wheels.
- Subinterpreters: **`_multiarray_umath`** still advertises `Py_MOD_MULTIPLE_INTERPRETERS_NOT_SUPPORTED`. Track numpy#32451 (subinterpreters) and numpy#31913 (Limited API adoption). Heap-type batches are landing; scalars, ufuncs, and `PyArray_Type` are the hard remainder.

On **Python 3.14+**, the failure mode looks like (trimmed; Quansight / NumPy messaging):

```
concurrent.interpreters.ExecutionFailed: ImportError:
  module numpy._core._multiarray_umath does not support loading in subinterpreters
```

This box runs 3.13.5, so that import path was not executed here; treat the snippet as upstream-documented behavior.

Case 4 — What you write once NumPy is ready

Process pool today:

```
# process_pool_numpy.py - works on current NumPy
from concurrent.futures import ProcessPoolExecutor
import numpy as np

def work(seed):
    rng = np.random.default_rng(seed)
    return float(rng.standard_normal(1_000_000).std())

if __name__ == "__main__":
    with ProcessPoolExecutor(max_workers=4) as pool:
        print(list(pool.map(work, range(4))))
```

Future shape with `InterpreterPoolExecutor` (stdlib since 3.14) once NumPy flips the module flag—import **inside** the worker:

```
# interpreter_pool_numpy.py - requires NumPy subinterpreter support + Py 3.14+
from concurrent.futures import InterpreterPoolExecutor
```

```
def work(seed):
    import numpy as np  # each interpreter loads its own NumPy
    rng = np.random.default_rng(seed)
    return float(rng.standard_normal(1_000_000).std())

if __name__ == "__main__":
    with InterpreterPoolExecutor(max_workers=4) as pool:
        print(list(pool.map(work, range(4))))
```

Shared big arrays still favor free-threading (no pickle copy). Subinterpreters win on isolation-without-process for many small jobs.

Case 5 — If you maintain SciPy-like code

1. **Today:** No change. You talk to NumPy through its C API capsule (`import_array()`), independent of how NumPy compiled itself.
2. **If NumPy ships `abi3`:** You can install NumPy on a brand-new Python sooner; your own wheels still need per-version builds until *you* adopt Limited API.
3. **If you want `abi3` yourself:** Heap types + function APIs; Cython 3.1+ can emit Limited API code; Meson `limited_api: '3.13'` + meson-python `limited-api = true`; build against NumPy 2.0 headers that tolerate `Py_LIMITED_API`.

The trap

- Believing "NumPy ported some modules" means PyPI wheels are `abi3`—they are not; the switch is off.
- Using Limited API macros as a free-threaded ABI—`abi3` ≠ `abi3t`.
- Forgetting that subinterpreter support needs **all** of multi-phase init, module state, and heap types—not only one.

The boring rule

When you touch a C extension type in NumPy-land (or your own extension), prefer **heap types and Limited-API-safe accessors** even if you still ship version-specific wheels. You pay the rewrite once; Limited API and subinterpreters both cash the check later.

Try this

1. Read CPython Isolating Extension Modules and skim NumPy issues #31913 / #32451.
2. On Python 3.14+, create a subinterpreter and `import numpy`; confirm the explicit unsupported error.
3. If you build NumPy from source, flip `-Dpython.allow_limited_api=true` and run the test suite (CI already does in a debug job).

Sources

- The things you own end up owning you — Quansight Labs, 2026-10-01 (Pratham Hole)
- C API and ABI Stability — CPython docs
- Isolating Extension Modules
- NumPy tracking: #31913 Limited API, #32451 subinterpreters
- PEP 489 (multi-phase init), PEP 684 (per-interpreter GIL), PEP 734 (`concurrent.interpreters`), PEP 803 (`abi3t`)

Inspiration

- Quansight / NumPy Limited API + heap-types work circulating in October 2026. Chapter cites the Quansight post and NumPy/CPython docs.

Verification

- Python 3.13.5, NumPy 2.5.3 import OK on this box.
- No Limited API rebuild of NumPy (would need a source build; skipped per no-install scout rule).
- Subinterpreter import demo requires 3.14+; documented from upstream, not executed here.

Sharing Iterators Between Threads: serialize_iterator, synchronized_iterator and concurrent_tee

Sharing Iterators Between Threads: serialize_iterator, synchronized_iterator and concurrent_tee

After reading this chapter, you will be able to coordinate shared iterators and generators safely across multiple threads in Python 3.15, distinguish between single-consumer stream partitioning and multi-consumer stream broadcasting, decorate generator factories with thread-safe synchronization, and deploy clean fallback patterns when running on Python 3.14.

The thread safety problem with iterators

Iterators are ubiquitous in Python pipelines. They offer lazy, memory-efficient access to unbounded streams, file chunks, database cursors, and mathematical series. However, standard Python iterators and generators are inherently stateful and mutable. Advancing an iterator changes internal pointers, frame evaluation offsets, and local variables.

Python 3.15 does not change how generators behave: passing one plain generator object to several worker threads still fails at run time. CPython enforces a strict frame execution check: if a thread calls `next()` on a generator whose evaluation frame is currently active on another thread, CPython raises:

```
ValueError: generator already executing
```

This error is not a bug; it is an intentional assertion guarding the single execution frame inside the CPython evaluation loop. The interpreter detects that the generator's frame is already running on another thread and refuses the second thread's entry. Whether you hit it depends on timing: a generator that never releases the GIL between `yield`s may run thousands of times without a collision, while one that does any I/O collides quickly.

In free-threaded builds (introduced experimentally in Python 3.13 as `python3.13t`), the Global Interpreter Lock (GIL) is disabled. There, threads really run `__next__()` at the same time. CPython's own documentation for the new wrappers uses `itertools.count` as its example of an iterator whose instances are not thread-safe to share, and the free-threading guide warns that sharing an iterator between threads can produce duplicated or skipped items rather than a clean error. Built-in iterators avoid crashing the interpreter, but they don't promise sensible results when shared.

CPython 3.15 introduces three purpose-built synchronization primitives in the standard `threading` module to address iterator sharing systematically:

1. `threading.serialize_iterator(iterable)`: A wrapper class that guards the iterator's `__next__()`, `send()`, `throw()`, and `close()` methods with a non-reentrant mutual exclusion lock. Each generated value is delivered to exactly one thread.
2. `threading.synchronized_iterator(func)`: A decorator or higher-order wrapper for iterator-producing callables, ensuring every iterator returned by the callable is automatically wrapped in a `serialize_iterator`.
3. `threading.concurrent_tee(iterable, n=2)`: A concurrent counterpart to `itertools.tee()` that duplicates a single input stream into n independent iterators across threads, buffering unconsumed items until all threads have read them.

The following table summarizes the concurrency characteristics of these tools:

Primitive	Delivery model	Internal synchronization	Buffering behavior	Thread target
Plain Generator / Iterator	Single consumer	None (unsafe)	None	Exactly 1 thread
`threading.serialize_iterator`	Partitioned (competing consumers)	Non-reentrant `threading.Lock`	Zero buffering	Multiple threads
`threading.synchronized_iterator`	Factory wrapper for partitioned iterators	Non-reentrant `threading.Lock`	Zero buffering	Multiple threads
`threading.concurrent_tee`	Broadcast (independent consumers)	One shared `threading.Lock` + linked list of items	Grows with the gap between fastest and slowest consumer	One thread per branch
`queue.Queue`	Partitioned work queue	`threading.Condition` objects over one mutex	Bounded or unbounded FIFO buffer	Multiple threads

Observing the race: Unsynchronized generators

To observe the generator collision directly, consider a pipeline where eight worker threads attempt to pull items concurrently from a single, unprotected generator.

```
import threading
import time


def source_stream(count):
    for i in range(count):
        time.sleep(0)  # stand-in for I/O: lets other threads run mid-frame
        yield i


def consumer(stream, results, worker_id):
```

```
    items_read = 0
    collisions = 0
    while True:
        try:
            val = next(stream)
            items_read += 1
        except StopIteration:
            break
        except ValueError as exc:
            if "generator already executing" in str(exc):
                collisions += 1
                # Back off briefly to let the executing thread exit the frame
                time.sleep(0.00001)
            else:
                raise
    results[worker_id] = (items_read, collisions)


def main():
    total_target = 10_000
    gen = source_stream(total_target)
    results = {}
    threads = []

    for i in range(8):
        t = threading.Thread(target=consumer, args=(gen, results, i))
        threads.append(t)

    for t in threads:
        t.start()
    for t in threads:
        t.join()

    total_consumed = sum(r[0] for r in results.values())
    total_collisions = sum(r[1] for r in results.values())

    print(f"Total items consumed: {total_consumed}")
    print(f"Total collision errors caught: {total_collisions}")


if __name__ == "__main__":
    main()
```

Run the script under Python 3.15:

```
python3.15 gen_race.py
```

The following text block shows the output produced by running `python3.15 gen_race.py`.

```
Total items consumed: 10000
Total collision errors caught: 57359
```

When reviewing the output, observe that `Total items consumed` eventually sums to 10,000 because each thread caught the exception and retried. However, `Total collision errors caught` is far from zero. The `time.sleep(0)` inside the generator releases the GIL while the frame is still running, which is exactly what real I/O does. The count changes from run to run with thread scheduling. Remove that line and the count often drops to zero, which is how this bug survives testing.

Single-consumer delivery with serialize_iterator

When multiple worker threads pull tasks from a single stream, you generally want each item processed exactly once. You do not want duplicate deliveries, dropped values, or `ValueError` crashes.

`threading.serialize_iterator(iterable)` fulfills this requirement. It is implemented as a standard wrapper class in the `threading` module. Upon instantiation:

```
safe_iter = threading.serialize_iterator(iterable)
```

The wrapper invokes `iter()` on the underlying iterable once and attaches a dedicated, non-reentrant `threading.Lock`. When a thread advances the wrapper via `next(safe_iter)`, the wrapper acquires the lock, calls `next()` on the underlying iterator, and releases the lock in a `finally` block before returning the value.

Crucially, `serialize_iterator` also inspects the underlying object for generator methods: - `send(value)`: Acquired under the lock and forwarded if supported; otherwise raises `AttributeError`. - `throw(typ, val, tb)`: Acquired under the lock and forwarded if supported; otherwise raises `AttributeError`. - `close()`: Acquired under the lock and forwarded if supported; otherwise raises `AttributeError`.

Because `serialize_iterator` uses a primitive lock without an intermediate queue or buffer, memory overhead is negligible. Each item passes directly from the iterator frame to the calling thread.

```
import collections
import threading
import time


def source_stream(count):
    for i in range(count):
        time.sleep(0)  # same simulated I/O as gen_race.py
        yield i


def worker(stream, sink, worker_id):
    batch = []
    for item in stream:
```

```
            batch.append(item)
        sink[worker_id] = batch


def main():
    target_items = 10_000
    source = source_stream(target_items)
    safe_stream = threading.serialize_iterator(source)

    sink = {}
    threads = []
    for i in range(8):
        t = threading.Thread(target=worker, args=(safe_stream, sink, i))
        threads.append(t)

    for t in threads:
        t.start()
    for t in threads:
        t.join()

    counter = collections.Counter()
    for batch in sink.values():
        counter.update(batch)

    total_consumed = sum(counter.values())
    duplicates = sum(count - 1 for count in counter.values() if count > 1)
    missing = len(set(range(target_items)) - set(counter.keys()))

    print(f"Total consumed: {total_consumed}")
    print(f"Duplicate items: {duplicates}")
    print(f"Missing items: {missing}")


if __name__ == "__main__":
    main()
```

Run the serialized verification:

```
python3.15 serialized.py
```

The following text block shows the output produced by running `python3.15 serialized.py`.

```
Total consumed: 10000
Duplicate items: 0
Missing items: 0
```

Notice that `Total consumed` matches 10,000, while `Duplicate items` and `Missing items` are both zero. Every element was safely yielded across the thread boundary without contention.

Function-level synchronization with synchronized_iterator

Managing iterator instances manually can lead to bugs if a developer exposes a raw generator instance before wrapping it. Python 3.15 provides `threading.synchronized_iterator(func)` to eliminate this hazard at the definition site.

`synchronized_iterator` takes a callable that returns an iterator—most commonly a generator function—and returns a new callable. Whenever the decorated function is invoked, its return value is wrapped in `threading.serialize_iterator` before reaching the caller.

This decorator is also useful for wrapping factory functions around non-thread-safe iterators from standard libraries, such as `itertools.count()` or custom C-extension iterators.

The following example decorates a generator producing indexed data batches and processes the output using `concurrent.futures.ThreadPoolExecutor`.

```
from concurrent.futures import ThreadPoolExecutor
import threading
import time


@threading.synchronized_iterator
def indexed_data_feed(limit):
    for index in range(limit):
        yield (index, f"payload-{index}")


def worker_task(feed):
    collected = []
    for item in feed:
        collected.append(item)
        time.sleep(0.0002)  # per-item work, done outside the iterator's lock
    return collected


def main():
    total_elements = 1_000
    shared_feed = indexed_data_feed(total_elements)

    with ThreadPoolExecutor(max_workers=4) as executor:
        futures = [executor.submit(worker_task, shared_feed) for _ in range(4)]
        results = [f.result() for f in futures]

    combined = []
    for worker_batch in results:
        combined.extend(worker_batch)

    combined_sorted = sorted(combined, key=lambda pair: pair[0])
```

```
    indices = [pair[0] for pair in combined_sorted]

    is_contiguous = indices == list(range(total_elements))
    worker_counts = [len(b) for b in results]

    print(f"Total items gathered: {len(combined)}")
    print(f"Stream sequence complete and contiguous: {is_contiguous}")
    print(f"All workers received work: {all(c > 0 for c in worker_counts)}")


if __name__ == "__main__":
    main()
```

Run the decorated generator script:

```
python3.15 decorated.py
```

The following text block shows the output produced by running `python3.15 decorated.py`.

```
Total items gathered: 1000
Stream sequence complete and contiguous: True
All workers received work: True
```

Look for confirmation that all 1,000 items were collected, the contiguous integrity test evaluates to `True`, and every worker in the pool pulled items from the feed.

Broadcast streaming with concurrent_tee

`serialize_iterator` splits a stream among workers: if worker A takes item 0, worker B gets item 1. But many architectures require broadcasting: every worker thread must read the entire stream from beginning to end.

In single-threaded Python, this is handled by `itertools.tee(iterable, n=2)`. However, `itertools.tee` is not thread-safe. Its internal linked lists and shared state are subject to race conditions and memory corruption when traversed concurrently by separate threads.

Python 3.15 introduces `threading.concurrent_tee(iterable, n=2)`. It creates n independent iterator handles. Each handle receives the entire sequence generated by the underlying iterable.

Under the hood, all branches share one `threading.Lock` and a singly linked list of items. Whichever branch reaches the end of the list first takes the lock, pulls the next value from the source and appends a new link. The other branches walk the list at their own pace without touching the source. A link becomes garbage once every branch has moved past it, so memory holds exactly the items between the slowest and the fastest branch.

In code, the call signature matches `itertools.tee`:

```
it1, it2 = threading.concurrent_tee(source_iterable, 2)
```

```
import threading


def consumer_sum(stream, results, worker_id):
    results[worker_id] = sum(stream)


def main():
    target_count = 1000
    source = range(target_count)
    it_a, it_b, it_c = threading.concurrent_tee(source, 3)

    results = {}
    threads = [
        threading.Thread(target=consumer_sum, args=(it_a, results, 0)),
        threading.Thread(target=consumer_sum, args=(it_b, results, 1)),
        threading.Thread(target=consumer_sum, args=(it_c, results, 2)),
    ]

    for t in threads:
        t.start()
    for t in threads:
        t.join()

    expected_sum = (target_count - 1) * target_count // 2

    print(f"Worker 0 sum: {results[0]}")
    print(f"Worker 1 sum: {results[1]}")
    print(f"Worker 2 sum: {results[2]}")
    print(f"All match expected ({expected_sum}): {results[0] == results[1] == results[2] ==


if __name__ == "__main__":
    main()
```

Run the concurrent tee script:

```
python3.15 tee.py
```

The following text block shows the output produced by running `python3.15 tee.py`.

```
Worker 0 sum: 499500
Worker 1 sum: 499500
Worker 2 sum: 499500
All match expected (499500): True
```

Notice that Worker 0, Worker 1, and Worker 2 all compute identical sums equal to the mathematical sum of the sequence.

> **!** Important
>
> Each individual iterator returned by `concurrent_tee` is intended for consumption by **one** thread. If you need multiple threads to share one branch of a `concurrent_tee`, wrap that specific branch in `threading.serialize_iterator`.

The throughput reality of serialized iterators

A common misconception among engineers adopting `serialize_iterator` is that wrapping an iterator in a lock will accelerate stream generation across multiple cores.

It will not. A lock serializes access; it does not parallelize execution.

When an iterator is wrapped in `serialize_iterator`, only one thread at a time can execute the iterator's `__next__()` body. If generating an item requires heavy CPU computation, that computation remains strictly single-threaded. Adding more consumer threads merely increases lock acquisition overhead and contention latency.

Consider the lifecycle of an iteration step: 1. Thread A acquires the lock. 2. Thread A executes the generator code up to the next `yield`. 3. Thread A releases the lock and processes the yielded item. 4. Concurrently, Thread B acquires the lock and advances to the subsequent `yield`.

This model provides high throughput **only when consumer processing time dominates production time**. If generating an item takes 1 millisecond but consuming it (e.g., executing a remote HTTP POST) takes 50 milliseconds, `serialize_iterator` allows 50 worker threads to process tasks concurrently without race conditions.

If production itself is expensive or bound by CPU cycles, iterators should not be shared via locks. Instead, use `concurrent.futures.ProcessPoolExecutor` or pipeline work via chunked `queue.Queue` buffers.

Backward compatibility: A Python 3.14 fallback

Applications running on Python 3.14 or earlier do not have access to `threading.serialize_iterator` or `threading.synchronized_iterator`. When writing libraries that must span multiple Python versions, you can implement a lightweight compatibility class.

The following listing provides a drop-in fallback class `LockedIterator` that mimics `serialize_iterator` for basic iteration on Python 3.14.

```
import threading


class LockedIterator:
    """Thread-safe iterator wrapper compatible with Python 3.14 and earlier."""

    def __init__(self, iterable):
```

```
        self._iterator = iter(iterable)
        self._lock = threading.Lock()

    def __iter__(self):
        return self

    def __next__(self):
        with self._lock:
            return next(self._iterator)

    def close(self):
        with self._lock:
            close_method = getattr(self._iterator, "close", None)
            if close_method is not None:
                return close_method()


def compatibility_stream(limit):
    for i in range(limit):
        yield i


def worker(stream, sink, worker_id):
    count = 0
    total = 0
    for value in stream:
        count += 1
        total += value
    sink[worker_id] = (count, total)


def main():
    item_limit = 5_000
    source = compatibility_stream(item_limit)
    safe_stream = LockedIterator(source)

    sink = {}
    threads = []
    for i in range(4):
        t = threading.Thread(target=worker, args=(safe_stream, sink, i))
        threads.append(t)

    for t in threads:
        t.start()
    for t in threads:
        t.join()
```

```
    total_items = sum(s[0] for s in sink.values())
    total_sum = sum(s[1] for s in sink.values())
    expected_sum = (item_limit - 1) * item_limit // 2

    print(f"Total items consumed: {total_items}")
    print(f"Total sum calculated: {total_sum}")
    print(f"Integrity verified: {total_items == item_limit and total_sum == expected_sum}")


if __name__ == "__main__":
    main()
```

Run the fallback script:

```
python3.14 fallback.py
```

The following text block shows the output produced by running `python3.14 fallback.py`.

```
Total items consumed: 5000
Total sum calculated: 12497500
Integrity verified: True
```

Inspect the output to verify that all 5,000 items were accounted for and the computed arithmetic sum matches the theoretical value.

Pitfalls and common mistakes

1. Divergent consumers causing memory leaks in concurrent_tee

Because `concurrent_tee` retains values in an internal buffer until the slowest consumer reads them, an unbalanced consumer will cause unbounded memory growth. If Consumer 1 processes 1,000,000 items while Consumer 2 hangs or blocks on I/O at item 10, the internal buffer must retain all 999,990 items in memory. Always ensure all consumers of a `concurrent_tee` advance at comparable rates or have configured timeouts.

2. Nested yields holding locks

`threading.serialize_iterator` uses a primitive, non-reentrant lock (`threading.Lock`, not `threading.RLock`). If the generator function itself calls code that attempts to advance the same wrapped iterator directly or indirectly, execution will deadlock instantly. Keep generator bodies self-contained and free from recursive iteration on the shared wrapper.

3. Closing a shared generator while workers are using it

`close()` takes the same lock as `__next__()`, so a supervisor calling `stream.close()` waits for the current item to finish. Workers that call `next()` afterwards get `StopIteration`, and a worker holding a half-processed item finishes it without knowing the stream was cut short. When tearing down worker pools, allow workers to exhaust the iterator naturally via `StopIteration`, or signal shutdown through a coordinated `threading.Event` checked inside the generator body.

4. Expecting free-threaded performance gains without decoupling

On free-threaded builds (`python3.13t` and `python3.15t`), removing the GIL allows true parallel execution on CPU cores. However, wrapping a single generator in `serialize_iterator` forces all worker threads to serialize through the lock. If your generator does CPU-heavy work before yielding, threads will spend most of their time waiting on the mutex. To scale CPU work across cores, split data into discrete slices and instantiate independent iterators per thread.

Summary

- Iterators and generators maintain mutable internal state; advancing them across threads without synchronization causes `ValueError: generator already executing` or data corruption.
- Python 3.15 adds `threading.serialize_iterator(iterable)` to protect `__next__()`, `send()`, `throw()`, and `close()` with a non-reentrant mutex, delivering each item to exactly one thread with zero buffering.
- `threading.synchronized_iterator(func)` acts as a decorator for generator functions and iterator factories, guaranteeing that every instantiated iterator is thread-safe.
- `threading.concurrent_tee(iterable, n=2)` broadcasts a stream across n independent threads, buffering items dynamically until consumed by all branches.
- Locking an iterator serializes production. Use these tools to safely coordinate shared sources among threads, not to speed up the generator's internal execution.
- For Python 3.14 environments, use a custom `LockedIterator` wrapper around `iter()` and `next()` as an effective fallback.

Sources

- Python 3.15 threading module documentation
- What's New in Python 3.15
- Python Free-Threading How-To

Asynchronous Programming and TaskGroups

Asynchronous Programming and TaskGroups

After reading this chapter, you will master asynchronous programming using Python's `asyncio` engine, write non-blocking coroutines with `async def` and `await`, leverage modern structured concurrency with `asyncio.TaskGroup`, enforce deterministic deadlines with `asyncio.timeout()`, and throttle concurrent operations using `asyncio.Semaphore`.

Mental model

Asynchronous programming is **cooperative multitasking on a single thread**. Unlike OS threads where the kernel preempts execution at any time, a Python coroutine yields control **voluntarily** at explicit `await` boundaries:

```
Single-Threaded Event Loop Timeline:
  Time
  Event Loop: [ Coroutine A runs ]    Hits 'await socket.read()'    (Yields control)


              [ Coroutine B runs ]    Hits 'await socket.write()'    (Yields control)


              OS reports A's socket is ready!


              [ Coroutine A resumes ]    Finishes
```

Structured Concurrency with `asyncio.TaskGroup`

Introduced in Python 3.11, `asyncio.TaskGroup` replaces brittle patterns like `asyncio.gather()` with a strict execution contract:

```
with TaskGroup() as tg:
    tg.create_task(task_1())
    tg.create_task(task_2())
    tg.create_task(task_3())

           If task_2 raises an exception:
  1. Cancels remaining siblings (task_1 & task_3)
  2. Awaits all sibling cancellations
  3. Bundles failures into an ExceptionGroup
```

No background tasks are ever left orphaned or leaked into the process.

Minimal example

Save as `asyncio_fundamentals.py`:

```python
# asyncio_fundamentals.py
import asyncio
import time

async def fetch_cluster_status(node_id: str, delay: float) -> dict[str, str | float]:
    """Simulate non-blocking async network fetch."""
    print(f"  [Start] Polling {node_id} (simulated delay: {delay}s)...")
    await asyncio.sleep(delay)  # Non-blocking async sleep
    print(f"  [Done]  Received response from {node_id}")
    return {"node": node_id, "status": "ONLINE", "rtt_ms": delay * 1000}

async def main() -> None:
    print("--- Polling 3 cluster nodes concurrently via TaskGroup ---")
    t0 = time.perf_counter()

    results = []
    # Structured concurrency block
    async with asyncio.TaskGroup() as tg:
        t1 = tg.create_task(fetch_cluster_status("node-01", 0.05))
        t2 = tg.create_task(fetch_cluster_status("node-02", 0.08))
        t3 = tg.create_task(fetch_cluster_status("node-03", 0.03))

    # Reaching here guarantees all tasks completed safely
    results = [t1.result(), t2.result(), t3.result()]
    elapsed = time.perf_counter() - t0

    print(f"\nAll 3 tasks completed in {elapsed:.2f}s (Max delay, NOT sum!):")
    for res in results:
        print(f"  {res['node']} -> {res['status']} ({res['rtt_ms']:.1f}ms)")

if __name__ == "__main__":
    asyncio.run(main())
```

Run via `uv run python asyncio_fundamentals.py`:

```
--- Polling 3 cluster nodes concurrently via TaskGroup ---
  [Start] Polling node-01 (simulated delay: 0.05s)...
  [Start] Polling node-02 (simulated delay: 0.08s)...
```

```
  [Start] Polling node-03 (simulated delay: 0.03s)...
  [Done]  Received response from node-03
  [Done]  Received response from node-01
  [Done]  Received response from node-02

All 3 tasks completed in 0.08s (Max delay, NOT sum!):
  node-01 -> ONLINE (50.0ms)
  node-02 -> ONLINE (80.0ms)
  node-03 -> ONLINE (30.0ms)
```

Worked examples

Case 1: Deterministic Deadlines with `asyncio.timeout()`

Python 3.11+ provides `asyncio.timeout()`, an async context manager that cancels the enclosed coroutine if it exceeds the specified budget:

```python
# async_timeout_demo.py
import asyncio

async def slow_database_query() -> str:
    print("  Executing database query (requires 2.0s)...")
    await asyncio.sleep(2.0)
    return "QUERY_RESULT"

async def main() -> None:
    # Set a strict 0.5-second deadline
    try:
        async with asyncio.timeout(0.5):
            result = await slow_database_query()
            print("Result:", result)
    except TimeoutError:
        print("  ALERT: Database query exceeded 0.5s deadline; cancelled cleanly!")

if __name__ == "__main__":
    asyncio.run(main())
```

Run:

```
uv run python async_timeout_demo.py
```

Output:

```
  Executing database query (requires 2.0s)...
  ALERT: Database query exceeded 0.5s deadline; cancelled cleanly!
```

Case 2: Throttling Concurrent Requests with `asyncio.Semaphore`

When crawling or querying an external API, spawning 10,000 tasks simultaneously can exhaust file descriptors or trigger rate-limit bans. A semaphore bounds active concurrency:

```python
# async_semaphore_rate_limit.py
import asyncio
import time

async def worker_with_semaphore(task_id: int, sem: asyncio.Semaphore) -> None:
    async with sem:  # Only 2 workers allowed inside this block concurrently
        print(f"[{time.strftime('%X')}] Worker {task_id} entering critical section")
        await asyncio.sleep(0.05)
        print(f"[{time.strftime('%X')}] Worker {task_id} exiting")

async def main() -> None:
    # Limit concurrency to at most 2 simultaneous executions
    sem = asyncio.Semaphore(2)

    async with asyncio.TaskGroup() as tg:
        for i in range(5):
            tg.create_task(worker_with_semaphore(i + 1, sem))

if __name__ == "__main__":
    asyncio.run(main())
```

Run:

```
uv run python async_semaphore_rate_limit.py
```

Output:

```
[10:00:00] Worker 1 entering critical section
[10:00:00] Worker 2 entering critical section
[10:00:00] Worker 1 exiting
[10:00:00] Worker 3 entering critical section
[10:00:00] Worker 2 exiting
[10:00:00] Worker 4 entering critical section
...
```

Pitfalls

Pitfall 1: Calling Synchronous Blocking Code in Coroutines

Calling a blocking function (`time.sleep()`, `requests.get()`, or heavy CPU calculations) inside an async function halts the single-threaded event loop, freezing all other concurrent tasks:

```
# FATAL: Freezes entire event loop!
async def bad_handler():
    time.sleep(5)  # Freezes all users and connections!

# CORRECT: Use non-blocking async primitives
async def good_handler():
    await asyncio.sleep(5)

# OR offload blocking calls to a thread pool:
async def offloaded_handler():
    await asyncio.to_thread(blocking_function)
```

Pitfall 2: Using Legacy `asyncio.gather()` Without Shielding

If one task fails in `asyncio.gather()`, other tasks continue running in the background as unmanaged orphaned coroutines. Always prefer `asyncio.TaskGroup()`.

Exercises

1. Write an asynchronous function that polls an HTTP mock endpoint and retries up to 3 times with exponential backoff on failure.
2. Use `asyncio.TaskGroup` to execute 10 concurrent tasks, and simulate one task raising an exception to verify structured sibling cancellation.
3. Build a producer-consumer pipeline using `asyncio.Queue`.
4. Use `asyncio.to_thread()` to run a CPU-bound hashing function inside an async application without blocking the event loop.

Further reading

- Python Standard Library: `asyncio` documentation.
- PEP 654: *Exception Groups and except*.*
- Yury Selivanov: *PEP 492 — Coroutines with async and await syntax.*

When Does an asyncio Task Start? Lazy Coroutines, Eager Tasks, and Task Lifetimes

When Does an asyncio Task Start? Lazy Coroutines, Eager Tasks, and Task Lifetimes

Thesis

`async def` and `await` look alike in every language that has them, but they do not behave alike. A 2026 design-space study from Brown University (Gray et al., "A Design Space Exploration of Async/Await") ran one twelve-line fire-and-forget program on seven runtimes and got four different outputs. The differences come from a handful of independent decisions. When does a spawned task start running? Who keeps it alive? What happens to it when its parent finishes?

asyncio answers each of those questions, but some of the answers are surprising, and one of them changes with configuration. Calling a coroutine function is **lazy**: nothing runs. `asyncio.create_task()` is **scheduled** by default: the task starts on a later turn of the loop, after the caller next suspends. Since 3.12, a loop can use an **eager** task factory, and since 3.14 a single `create_task(..., eager_start=True)` call can opt in. With eager start the task's first step runs *inside* `create_task`. On the other end of a task's life, the loop holds only a **weak** reference to it, `asyncio.run()` **cancels** whatever is still pending when `main()` returns, and only a `TaskGroup` **waits** for its children.

This chapter makes each of those rules visible with a program you can run, measures what eager start buys, and shows the bug it introduces into code that assumed a new task "hasn't started yet".

Version note: every listing was run on **CPython 3.15.0** (released 2026-10-09) and **CPython 3.14.8**, both from `uv` (python-build-standalone, Linux x86-64), plus the system **3.13.5** where noted. Output is identical on 3.14.8 and 3.15.0 except for `TaskGroup.cancel()`, which is new in 3.15. The timings in Case 4 come from a shared 8-core box, so read the ratios, not the milliseconds.

Mental model

- **A coroutine object is inert.** `f()` builds a coroutine and returns it. No line of `f` runs until something drives it. `await coro` drives it *inline*: the caller keeps running the coroutine's code directly, with no trip through the event loop. That is why `await other_coroutine()` is cheap and never lets another task in unless `other_coroutine` itself suspends.
- **A Task is a scheduled driver.** `Task.__init__` calls `loop.call_soon(self.__step)`, which puts the first step at the back of the loop's FIFO **ready queue**. The task's first line runs on the next loop iteration, and only once the code that is running now suspends. Tasks created in a row therefore start in creation order.

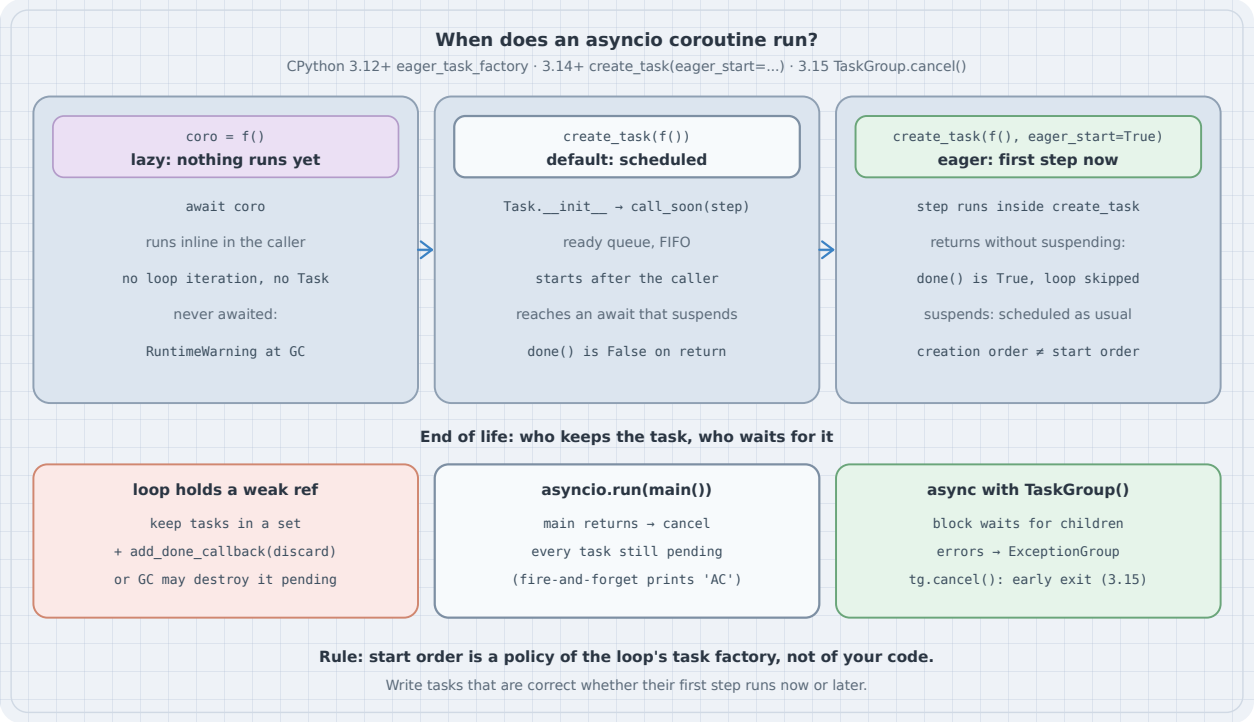


Figure 1: When an asyncio coroutine runs: lazy coroutines, scheduled tasks, eager tasks, and who ends a task's life

- **An eager Task runs its first step immediately.** With `eager_start=True` (3.14+ through `create_task`; always available on the `Task` constructor since 3.12) or with `asyncio.eager_task_factory` installed on the loop (3.12+), `Task.__init__` runs the first step synchronously, provided the loop is already running. If the coroutine finishes without suspending, the task is done before `create_task` returns and the loop never sees it. If it suspends, the rest of the task is scheduled like any other.
- **The loop does not own your task.** `asyncio` tracks scheduled tasks in a `weakref.WeakSet` (`_scheduled_tasks` in `Lib/asyncio/tasks.py`). While a task is parked on a future that nothing else references, nothing keeps it alive.
- **Who ends a task's life.** When `main()` returns, `asyncio.run()` (through `Runner.close()` → `_cancel_all_tasks()` in `Lib/asyncio/runners.py`) cancels every task that is still pending. A `TaskGroup` waits for all its children before its `async with` block exits. In the Brown paper's terms, a bare task has *indefinite* extent with *cancelled* destruction, and a `TaskGroup` child has *dynamic* extent with *awaited* destruction.

Worked examples

Case 1: Calling is not running

```
# lazy_coro.py
import asyncio


async def greet(who):
    print(f"  greet({who}) body runs")
    return f"hello {who}"


async def main():
    coro = greet("coro")                        # nothing runs here
    print("1. called greet(), got", type(coro).__name__)
    task = asyncio.create_task(greet("task"))  # scheduled, not started
    print("2. created task, done =", task.done())
    print("3. awaiting the coroutine:", await coro)
    print("4. awaiting the task:", await task)
    greet("forgotten")                           # a coroutine that is never awaited


asyncio.run(main())
```

```
python3.15 -u lazy_coro.py 2>&1
```

Output:

```
1. called greet(), got coroutine
2. created task, done = False
  greet(coro) body runs
3. awaiting the coroutine: hello coro
  greet(task) body runs
4. awaiting the task: hello task
/workspace/lab-1018am-C/py/lazy_coro.py:17: RuntimeWarning: coroutine 'greet' was never awai
  greet("forgotten")                           # a coroutine that is never awaited
RuntimeWarning: Enable tracemalloc to get the object allocation traceback
```

Read lines 2 and 3 together. The task was created *before* the coroutine was awaited, yet `greet(coro)` ran first. `await coro` ran the coroutine inline and never gave the loop a turn. The task had to wait until `main` hit `await task`, which actually suspends. The forgotten `greet("forgotten")` never ran at all. CPython only noticed when the coroutine object was destroyed, and reported it as a `RuntimeWarning`, which ends up in a log nobody reads.

Case 2: The fire-and-forget program in asyncio

This is the paper's example, translated directly. `fire_and_forget` spawns a log write and returns without awaiting it.

```
# brown_example.py
# The fire-and-forget program from "A Design Space Exploration of Async/Await"
# (Brown CEL, 2026), written for asyncio. Times scaled down 10x.
import asyncio
import sys


async def write_to_log():
    print("A")
    await asyncio.sleep(0.2)                  # simulate a slow log write
    print("B")


async def fire_and_forget():
    asyncio.create_task(write_to_log())       # spawn, return without awaiting


async def main():
    if "--eager" in sys.argv:
        asyncio.get_running_loop().set_task_factory(asyncio.eager_task_factory)
    await fire_and_forget()
    await asyncio.sleep(0.1)
    print("C")


asyncio.run(main())
print("asyncio.run returned")
```

```
python3.15 brown_example.py
python3.15 brown_example.py --eager
```

Output:

```
A
C
asyncio.run returned
A
C
asyncio.run returned
```

asyncio prints `AC`, the same as JavaScript and Swift in the paper's table, and the eager factory does not change that. `A` appears because `main` suspends in `sleep(0.1)`, which lets the scheduled task

start. `B` never appears because the log write needs 0.2 s and `main` is done after 0.1 s. Make the cancellation visible:

```
# brown_cancelled.py
import asyncio


async def write_to_log():
    print("A")
    try:
        await asyncio.sleep(0.2)
        print("B")
    except asyncio.CancelledError:
        print("B never printed: write_to_log was cancelled at shutdown")
        raise


async def fire_and_forget():
    asyncio.create_task(write_to_log())


async def main():
    await fire_and_forget()
    await asyncio.sleep(0.1)
    print("C")


asyncio.run(main())
```

```
python3.15 brown_cancelled.py
```

Output:

```
A
C
B never printed: write_to_log was cancelled at shutdown
```

The task was not abandoned or silently killed. `asyncio.run()` cancelled it after `main` returned, so `CancelledError` was raised at its `await` and its handler ran. For a log writer that means the log line is lost, and it happens on every shutdown, not just on rare ones.

Give the task a parent that waits for it, and asyncio behaves like Trio's nurseries:

```
# brown_taskgroup.py
import asyncio
```

```
async def write_to_log():
    print("A")
    await asyncio.sleep(0.2)
    print("B")


async def fire_and_forget():
    async with asyncio.TaskGroup() as tg:     # the task cannot outlive this block
        tg.create_task(write_to_log())


async def main():
    await fire_and_forget()
    await asyncio.sleep(0.1)
    print("C")


asyncio.run(main())
```

```
python3.15 brown_taskgroup.py
```

Output:

```
A
B
C
```

Now `fire_and_forget` does not return until the log write has finished, so the output is `ABC`. The two versions differ in two words, but they make opposite promises: one says "this task may outlive me and be cancelled", the other says "this task finishes before I do".

Case 3: Creation order versus start order

Two children: `t1` finishes without ever suspending (a cache hit), and `t2` suspends once. Each is run three ways: the default, the loop-wide eager factory, and the per-call `eager_start=True` keyword added in 3.14.

```
# start_order.py
import asyncio
import sys

log = []


async def child(name, blocks):
    log.append(f"{name}: start")
```

```
    if blocks:
        await asyncio.sleep(0)                # first real suspension point
    log.append(f"{name}: end")
    return name


async def main(mode):
    loop = asyncio.get_running_loop()
    if mode == "factory":
        loop.set_task_factory(asyncio.eager_task_factory)
    kw = {"eager_start": True} if mode == "per-task" else {}

    t1 = asyncio.create_task(child("t1", blocks=False), **kw)
    log.append(f"main: t1 created, done={t1.done()}")
    t2 = asyncio.create_task(child("t2", blocks=True), **kw)
    log.append(f"main: t2 created, done={t2.done()}")
    await asyncio.gather(t1, t2)
    log.append("main: gathered")


for mode in ("lazy", "factory", "per-task"):
    log.clear()
    asyncio.run(main(mode))
    print(f"{sys.version.split()[0]} {mode}:")
    for line in log:
        print("   ", line)
```

```
python3.15 start_order.py
```

Output:

```
3.15.0 lazy:
    main: t1 created, done=False
    main: t2 created, done=False
    t1: start
    t1: end
    t2: start
    t2: end
    main: gathered
3.15.0 factory:
    t1: start
    t1: end
    main: t1 created, done=True
    t2: start
    main: t2 created, done=False
    t2: end
```

```
    main: gathered
3.15.0 per-task:
    t1: start
    t1: end
    main: t1 created, done=True
    t2: start
    main: t2 created, done=False
    t2: end
    main: gathered
```

In **lazy** mode `main` gets all the way to `gather` before either child runs, and the children run in creation order. In both **eager** modes, `t1` starts *and finishes* inside its `create_task` call, so `main` sees `done=True` immediately. `t2` runs as far as its first `await`, returns control to `main` and is scheduled from there. `python3.14 start_order.py` prints exactly the same three traces.

The two eager spellings differ only in scope. `loop.set_task_factory(asyncio.eager_task_factory)` changes every `create_task` on the loop, including the ones inside libraries you call. `eager_start=True` changes one call. Under the eager factory a single call can also opt *out*, because the factory takes `eager_start=True` as a default keyword that the call can override:

```
# opt_out.py  (Python 3.14+)
import asyncio


async def quick():
    return "cached"


async def main():
    asyncio.get_running_loop().set_task_factory(asyncio.eager_task_factory)
    a = asyncio.create_task(quick())                      # factory default: eager
    b = asyncio.create_task(quick(), eager_start=False)   # this one opts out
    print("eager task done at creation:", a.done())
    print("opted-out task done at creation:", b.done())
    print(await a, await b)


asyncio.run(main())
```

```
python3.15 opt_out.py
```

Output:

```
eager task done at creation: True
opted-out task done at creation: False
cached cached
```

The keyword route needs 3.14. On 3.13, `eager_task_factory` exists (since 3.12) but

`asyncio.create_task(coro, eager_start=True)` raises `TypeError: create_task() got an unexpected keyword argument 'eager_start'` and leaves the coroutine un-awaited. The 3.14 What's New entry (gh-128307) is the change that made `create_task()` pass arbitrary keywords through to the task factory.

Case 4: What eager start buys

The case for eager tasks is in the documentation: a coroutine that returns without blocking, such as one that hits a cache, never has to be scheduled on the loop. The benchmark below fans out 100,000 lookups and varies how many of them are cache hits:

```
# eager_bench.py
import asyncio
import sys
import time

CACHE = {k: k * k for k in range(1000)}       # 1,000 hot keys


async def lookup(key):
    if key in CACHE:                          # cache hit: returns without suspending
        return CACHE[key]
    await asyncio.sleep(0)                    # cache miss: pretend to do I/O
    CACHE[key] = key * key
    return CACHE[key]


async def fan_out(n, hit_ratio):
    keys = [i % 1000 if (i % 100) < hit_ratio * 100 else 10_000 + i for i in range(n)]
    tasks = [asyncio.create_task(lookup(k)) for k in keys]
    return len(await asyncio.gather(*tasks))


async def bench(eager, n, hit_ratio):
    if eager:
        asyncio.get_running_loop().set_task_factory(asyncio.eager_task_factory)
    best = float("inf")
    for _ in range(5):
        for k in [k for k in CACHE if k >= 1000]:
            del CACHE[k]                      # forget the misses between rounds
        t0 = time.perf_counter()
        await fan_out(n, hit_ratio)
        best = min(best, time.perf_counter() - t0)
    return best
```

```
    print(sys.version.split()[0])
    for hit in (1.0, 0.9, 0.5, 0.0):
        lazy = asyncio.run(bench(False, 100_000, hit))
        eager = asyncio.run(bench(True, 100_000, hit))
        print(f"hit ratio {hit:4.0%}: lazy {lazy*1000:6.1f} ms  eager {eager*1000:6.1f} ms  ({la
```

```
python3.14 eager_bench.py
python3.15 eager_bench.py
```

Output:

```
3.14.8
hit ratio 100%: lazy  348.2 ms  eager  150.5 ms  (2.3x)
hit ratio  90%: lazy  355.1 ms  eager  177.9 ms  (2.0x)
hit ratio  50%: lazy  434.8 ms  eager  293.3 ms  (1.5x)
hit ratio   0%: lazy  532.4 ms  eager  452.7 ms  (1.2x)
3.15.0
hit ratio 100%: lazy  356.8 ms  eager  149.5 ms  (2.4x)
hit ratio  90%: lazy  352.2 ms  eager  175.4 ms  (2.0x)
hit ratio  50%: lazy  434.4 ms  eager  277.0 ms  (1.6x)
hit ratio   0%: lazy  501.6 ms  eager  429.1 ms  (1.2x)
```

When every lookup is a hit, eager start is about 2.3–2.4× faster. None of the 100,000 tasks goes through the ready queue, and `gather` finds them all finished already. The gain shrinks as the hit ratio falls, but it does not reach zero. Even a task that always suspends saves one loop iteration, because its first step (up to the `await`) runs inside `create_task` instead of in a scheduled callback. The speedup comes from skipping scheduling, so it is largest for many small tasks that rarely block and negligible for a few tasks that each do real I/O.

Case 5: The task that vanished

The `create_task` documentation has an "Important" box: "The event loop only keeps weak references to tasks. A task that isn't referenced elsewhere may get garbage collected at any time, even before it's done." Here is that happening:

```
# vanishing_task.py
import asyncio
import gc
import sys

done = []


async def wait_for_signal():
    fut = asyncio.get_running_loop().create_future()   # only this frame holds it
    try:
```

```
        await fut                                       # nobody will ever set it
    finally:
        done.append("finally ran")


async def main(keep):
    background = set()
    task = asyncio.create_task(wait_for_signal())
    if keep:
        background.add(task)
        task.add_done_callback(background.discard)
    del task
    await asyncio.sleep(0)            # let it start and park on the future
    gc.collect()                      # what a busy program does eventually
    await asyncio.sleep(0)
    alive = [t for t in asyncio.all_tasks() if t is not asyncio.current_task()]
    print(f"keep={keep}: other tasks alive={len(alive)}, finally ran={done}")
    for t in alive:
        t.cancel()
    done.clear()


asyncio.run(main(keep=False))
asyncio.run(main(keep=True))
```

```
python3.15 -u vanishing_task.py 2>&1
```

Output:

```
Task was destroyed but it is pending!
task: <Task pending name='Task-2' coro=<wait_for_signal() done, defined at /workspace/lab-10
keep=False: other tasks alive=0, finally ran=['finally ran']
keep=True: other tasks alive=1, finally ran=[]
```

With `keep=False` the task parks on a future that only its own frame references. That makes a reference cycle (task → coroutine → frame → future → wakeup callback → task) which nothing outside points into, so `gc.collect()` reclaims all of it. The only trace is a log line. The `finally` block *did* run, because closing the collected coroutine throws `GeneratorExit` into it. Nobody awaited anything, nothing raised in `main`, and the work simply stopped. With `keep=True` the set holds a strong reference and the task survives. In a real program the "future nobody else holds" is often a lock, an event, or a queue local to the task, and the collection happens whenever the cyclic GC next runs. That is why the bug shows up under load and not in tests. 3.14 and 3.15 behave identically here.

Case 6: The exception nobody retrieves

A task that fails and is never awaited does not crash anything:

```
# lost_exception.py
import asyncio
import gc


async def flaky():
    raise ConnectionError("upstream reset")


async def main():
    t = asyncio.create_task(flaky())
    await asyncio.sleep(0)            # t runs and fails; nobody awaits it
    print("main carries on; t.done() =", t.done())
    del t
    gc.collect()
    print("main finished")


asyncio.run(main())
```

```
python3.15 -u lost_exception.py 2>&1
```

Output:

```
main carries on; t.done() = True
Task exception was never retrieved
future: <Task finished name='Task-2' coro=<flaky() done, defined at /workspace/lab-1018am-C/
Traceback (most recent call last):
  File "/workspace/lab-1018am-C/py/lost_exception.py", line 7, in flaky
    raise ConnectionError("upstream reset")
ConnectionError: upstream reset
main finished
```

The exception sits inside the task (*never* propagated, in the paper's terms) until the task is destroyed. Then the loop's exception handler logs "Task exception was never retrieved". `main` carried on and finished normally. A `TaskGroup` makes the opposite choice: a failing child cancels its siblings and the group raises an `ExceptionGroup` from the `async with`, as the asyncio and TaskGroups chapter shows.

Case 7: Ending a group early with `TaskGroup.cancel()` (3.15)

Before 3.15, stopping a `TaskGroup` once its goal was met took a sentinel exception raised from a child and suppressed outside the group. 3.15 adds `TaskGroup.cancel()` (gh-127214). It cancels

every child that is not done, and the parent body, and the `async with` exits **without** raising `CancelledError`. Here are three mirrors, where the first one to answer wins:

```
# taskgroup_cancel.py  (Python 3.15+: TaskGroup.cancel)
import asyncio
import time

T0 = time.perf_counter()


def ms():
    return f"{(time.perf_counter() - T0) * 1000:5.0f} ms"


async def mirror(name, delay, tg, winner):
    await asyncio.sleep(delay)                # pretend to download from this mirror
    winner.append(name)
    print(f"{ms()}  {name} finished first; cancelling the rest")
    tg.cancel()                               # non-exceptional early exit


async def main():
    winner = []
    async with asyncio.TaskGroup() as tg:
        tasks = [tg.create_task(mirror(n, d, tg, winner))
                 for n, d in (("eu", 0.30), ("us", 0.05), ("ap", 0.50))]
    print(f"{ms()}  group exited without CancelledError; winner={winner}")
    print("       states:", [("cancelled" if t.cancelled() else "done") for t in tasks])


asyncio.run(main())
```

```
python3.15 -u taskgroup_cancel.py
python3.14 -u taskgroup_cancel.py 2>&1 | grep Error
```

Output:

```
   50 ms  us finished first; cancelling the rest
   50 ms  group exited without CancelledError; winner=['us']
       states: ['cancelled', 'cancelled', 'cancelled']
    | AttributeError: 'TaskGroup' object has no attribute 'cancel'
```

The group exited the moment `us` answered, at 50 ms instead of 500 ms. Look at the states, though: **the winner is cancelled too**. `cancel()` targets every task in the group that is not done, and the task calling it is still running at that moment. So `us` is cancelled even though it returns right afterwards, and its return value would be lost. That is why the listing records the winner in a list before calling `tg.cancel()`, instead of reading `task.result()` afterwards. On 3.14 the method

does not exist (the `AttributeError` arrives wrapped in an `ExceptionGroup`).

The trap

Code that registers a task after creating it breaks under eager start. A common pattern is "create the task, then record it so duplicate requests can join it, and let the task remove itself when it finishes". With scheduled tasks the registration always comes first, because the task cannot start until the caller suspends. Turn on the eager factory, perhaps because a framework or a performance guide recommends it, and a cache hit finishes *inside* `create_task`, before the registration line has run:

```
# eager_trap.py
import asyncio
import sys

CACHE = {"/health": b"ok"}
inflight = {}                                   # path -> task, so duplicates can join


async def fetch(path):
    try:
        if path in CACHE:
            return CACHE[path]                  # fast path: no await at all
        await asyncio.sleep(0.01)               # slow path: real I/O
        return b"<html>"
    finally:
        del inflight[path]                      # "the task is registered by now"


async def handle(path):
    task = asyncio.create_task(fetch(path))
    inflight[path] = task                       # register after creation
    return await task


async def main():
    if "--eager" in sys.argv:
        asyncio.get_running_loop().set_task_factory(asyncio.eager_task_factory)
    for path in ("/index", "/health"):
        try:
            print(f"{path}: {await handle(path)!r}  inflight={list(inflight)}")
        except KeyError as e:
            print(f"{path}: KeyError {e}  inflight={list(inflight)}")


asyncio.run(main())
```

```
python3.15 eager_trap.py
python3.15 eager_trap.py --eager
```

Output:

```
/index: b'<html>'  inflight=[]
/health: b'ok'  inflight=[]
/index: b'<html>'  inflight=[]
/health: KeyError '/health'  inflight=['/health']
```

The lazy run is clean. The eager run fails on the cache hit: `fetch` ran its `finally` before `handle` had added the task, so `del inflight[path]` raised `KeyError`. Then `handle` registered a task that was already finished, and nothing will ever remove it, so `inflight` is left holding a dead entry, which is a slow leak. The slow path works in both modes because it suspends before reaching the `finally`. This is the "semantic change" the documentation warns about: "If the coroutine returns or raises, the task is finished eagerly and is never scheduled to the event loop… the application's task execution order is likely to change." 3.14.8 fails the same way.

The fix is to move the cleanup out of the coroutine and into a done callback. `add_done_callback` is documented to schedule the callback with `loop.call_soon()` if the task is already done, so the callback always runs after the code that registered the task:

```
# eager_fixed.py
import asyncio
import sys

CACHE = {"/health": b"ok"}
inflight = {}


async def fetch(path):
    if path in CACHE:
        return CACHE[path]
    await asyncio.sleep(0.01)
    return b"<html>"


async def handle(path):
    task = asyncio.create_task(fetch(path))
    inflight[path] = task
    # A done callback is always scheduled with call_soon, even if the task is
    # already finished, so it runs after this function has registered it.
    task.add_done_callback(lambda t: inflight.pop(path, None))
    return await task


async def main():
```

```
    if "--eager" in sys.argv:
        asyncio.get_running_loop().set_task_factory(asyncio.eager_task_factory)
    for path in ("/index", "/health"):
        body = await handle(path)
        await asyncio.sleep(0)                # let the done callback run
        print(f"{path}: {body!r}  inflight={list(inflight)}")


asyncio.run(main())
```

```
python3.15 eager_fixed.py
python3.15 eager_fixed.py --eager
```

Output:

```
/index: b'<html>'  inflight=[]
/health: b'ok'  inflight=[]
/index: b'<html>'  inflight=[]
/health: b'ok'  inflight=[]
```

Both modes are now correct. The general lesson: the start order of tasks is a policy of the loop's task factory, not a property of your code. Code that is only correct when a new task "hasn't started yet" is depending on that policy.

The boring rule

1. Never call a coroutine function without awaiting it or wrapping it in a task. Run tests with `-W error::RuntimeWarning` so "coroutine … was never awaited" fails the build.
2. `await coro` runs inline. Use `create_task` only when you want the work to run concurrently with the caller.
3. Default to `TaskGroup` for any task whose result or failure matters. Use a bare `create_task` only for genuinely background work, and then keep it in a set with `add_done_callback(set.discard)`.
4. Anything that must survive shutdown (log flushes, metrics, final writes) must be awaited before `main()` returns, because `asyncio.run()` cancels pending tasks.
5. Write tasks that are correct whether their first step runs now or later. Do cleanup in done callbacks, not in code that assumes the creator has finished its bookkeeping.
6. Turn on `eager_task_factory` for workloads with many small, often-synchronous tasks (cache hits, memoised lookups), measure the gain, and run the test suite under it first. On 3.14+, prefer per-call `eager_start=True` on the hot path you own.
7. On 3.15, end a group early with `tg.cancel()`. Record the winner's result before calling it, because the calling task is cancelled too.

Try this

1. Add `print(asyncio.current_task().get_name())` as the first line of `child` in `start_order.py`. Under the eager factory, which task does `current_task()` report while the first step runs inside `create_task`?
2. Run `eager_bench.py` with `n = 1_000` and with `n = 1_000_000`. Does the speedup ratio depend on the number of tasks, or only on the hit ratio?
3. In `vanishing_task.py`, replace the bare future with `asyncio.Event()` and `await event.wait()`. Is the task still collected? Now keep the event in a module-level variable and try again.
4. Rewrite `taskgroup_cancel.py` for 3.14 using the old pattern: raise a private exception from the winner and catch it with `except*` outside the group. Compare how the winner's result is passed out in each version.
5. Wrap `eager_trap.py`'s `inflight` in a `TaskGroup`-based request coalescer that is correct in both modes without a done callback. What does the group's lifetime have to be?

Sources

- Coroutines and Tasks: `create_task`, `Task(eager_start=...)`, eager task factory, `TaskGroup.cancel` (3.15 documentation; the "Important" weak-reference box and the "semantic change" note on eager tasks)
- Event loop: `loop.set_task_factory` and `loop.create_task` (3.14 change: all kwargs passed on; `eager_start` works with eager factories)
- Futures: `add_done_callback` (already-done futures schedule callbacks with `call_soon`)
- Runners: `asyncio.run`, `asyncio.Runner`
- What's New in Python 3.14: asyncio (gh-128307, keyword pass-through in `create_task`)
- What's New in Python 3.15: asyncio (gh-127214, `TaskGroup.cancel`)
- What's New in Python 3.12: asyncio (eager task factory, gh-102853 / gh-104140 / gh-104138: "making some use-cases 2x to 5x faster")
- CPython 3.15.0 source: `Lib/asyncio/tasks.py` (`Task.__init__`, `__eager_start`, `_scheduled_tasks` WeakSet, `create_eager_task_factory`), `Lib/asyncio/runners.py` (`_cancel_all_tasks`)
- Gavin Gray et al., A Design Space Exploration of Async/Await (Brown CEL, 2026; the fire-and-forget program and the eagerness/extent/destruction dimensions)

Blocking the Event Loop: Finding Sync Calls in Async Code and Moving Them Off the Loop

Blocking the Event Loop: Finding Sync Calls in Async Code and Moving Them Off the Loop

Thesis

asyncio runs every coroutine of a program on **one thread**. A task gives the loop back only at an `await` that actually suspends. Any code between two such points, a `time.sleep()`, a synchronous database driver, a large `json.dumps()`, a checksum over an upload, runs while *every other task waits*. Nothing crashes and nothing is logged by default. The symptom is latency: requests that do no work themselves take a second, and only sometimes.

This chapter makes that cost visible and then fixes it, in four steps:

1. **Measure** the loop's lateness with a 10 ms heartbeat, and the effect on real requests with a ping client that runs *outside* the loop.
2. **Name the task** with asyncio's debug mode (`-X dev` or `PYTHONASYNCIODEBUG=1`) and `slow_callback_duration`.
3. **Name the line** with a small watchdog thread that prints the loop thread's stack when the loop stalls. Unlike debug mode, it is cheap enough to leave on.
4. **Move the work**: `asyncio.to_thread` for blocking I/O and for C code that releases the GIL, a process or interpreter pool for pure-Python CPU work, and a dedicated executor when the shared default pool becomes the bottleneck.

Version note: every listing was run on **CPython 3.15.0** (released 2026-10-09) and **CPython 3.14.8**, both from `uv` (python-build-standalone, Linux x86-64, standard GIL builds), on a shared 8-core machine. Results are the same on both versions within noise; where only one version's output is shown, the other matched. Timings vary between runs on a shared box, so compare the orders of magnitude, not the decimals. Free-threaded builds are not covered here.

Mental model

- **The loop runs one thing at a time.** `BaseEventLoop._run_once()` pops each ready handle (a callback, or the next *step* of a task) and calls it. A task step runs from one suspension point to the next. While a step runs, timers do not fire, sockets are not read, and no other task makes progress.
- **`await` is not a yield point by itself.** Awaiting a coroutine runs it inline. Only awaiting something that is not ready yet (a sleeping timer, a socket with no data, a pending future) suspends the task and returns control to the loop.

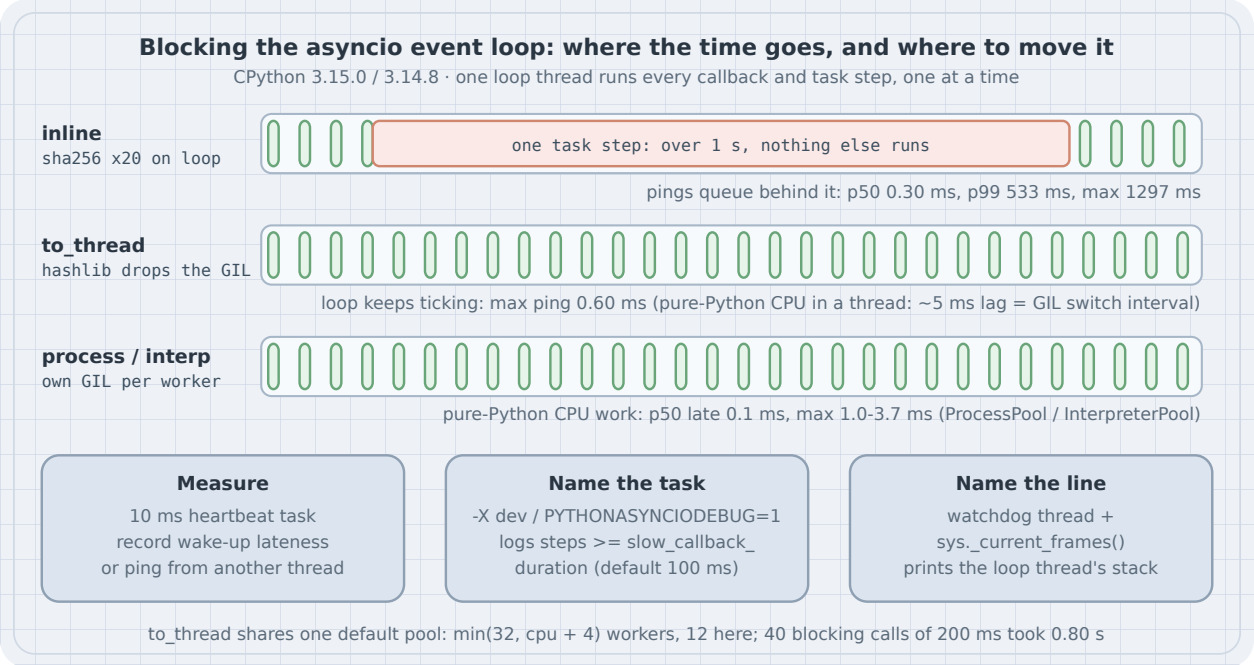


Figure 1: Blocking the asyncio event loop: an inline checksum holds the loop for over a second and pings queue behind it; to_thread keeps the loop ticking; a process or interpreter pool also handles pure-Python CPU work; measure with a heartbeat, name the task with debug mode, name the line with a watchdog

- **Debug mode times every step.** With debug enabled, `_run_once()` measures each `handle._run()` and logs `Executing <handle> took N seconds` when it reaches `loop.slow_callback_duration` (default `0.1`). It tells you *which task step* was slow, not which line inside it.
- **`to_thread` moves a call, not the GIL.** `asyncio.to_thread(f, *args)` runs `f` in the loop's default `ThreadPoolExecutor` and propagates the current `contextvars.Context`. The loop stays free only to the extent that `f` lets go of the GIL. Blocking system calls and many C functions (hashlib on buffers over 2047 bytes, zlib, file and socket I/O) do. A pure-Python loop does not; the interpreter then switches threads every `sys.getswitchinterval()` (5 ms).
- **Executors are shared and bounded.** The default pool has `min(32, os.process_cpu_count() + 4)` threads. Every `to_thread` and `run_in_executor(None, ...)` in the process queues on it.

Worked examples

All scripts live in one directory (`/tmp/aioblock` on the test machine, which is why that path appears in tracebacks) and need nothing outside the standard library.

Case 1: Measure the loop with a heartbeat

A heartbeat task sleeps for 10 ms in a loop and records how late each wake-up is. Another task does five rounds of 200 ms of blocking work, in six different ways:

```
# lag_probe.py: how late does the event loop run a 10 ms timer while a
# task does 5 x 200 ms of blocking work, done six different ways?
import asyncio
import concurrent.futures as cf
import sys
import time


async def heartbeat(lates, stop):
    """Sleep 10 ms at a time and record how late each wake-up is (ms)."""
    while not stop.is_set():
        t0 = time.perf_counter()
        await asyncio.sleep(0.010)
        lates.append((time.perf_counter() - t0 - 0.010) * 1000)


def busy(seconds):
    """Pure-Python CPU work for about `seconds`."""
    end = time.perf_counter() + seconds
    n = 0
    while time.perf_counter() < end:
        n += 1
    return n


async def work(mode, pool):
    loop = asyncio.get_running_loop()
    for _ in range(5):
        match mode:
            case "sleep-in-loop":
                time.sleep(0.2)                                 # blocks the loop
            case "sleep-to-thread":
                await asyncio.to_thread(time.sleep, 0.2)
            case "cpu-in-loop":
                busy(0.2)                                       # blocks the loop
            case "cpu-to-thread":
                await asyncio.to_thread(busy, 0.2)              # shares the GIL
            case "cpu-to-process" | "cpu-to-interp":
                await loop.run_in_executor(pool, busy, 0.2)  # own GIL


async def main(mode):
```

```
    pool = None
    if mode == "cpu-to-process":
        pool = cf.ProcessPoolExecutor(max_workers=1)
    elif mode == "cpu-to-interp":
        pool = cf.InterpreterPoolExecutor(max_workers=1)     # 3.14+
    if pool:  # start the worker before measuring
        await asyncio.get_running_loop().run_in_executor(pool, busy, 0)
    lates, stop = [], asyncio.Event()
    hb = asyncio.create_task(heartbeat(lates, stop))
    await asyncio.sleep(0.05)                                # let it settle
    t0 = time.perf_counter()
    await work(mode, pool)
    elapsed = time.perf_counter() - t0
    stop.set()
    await hb
    if pool:
        pool.shutdown()
    lates.sort()
    print(f"{mode:16} {elapsed:4.2f}s  ticks={len(lates):3}  "
          f"p50 late={lates[len(lates) // 2]:6.1f} ms  max late={lates[-1]:6.1f} ms")


if __name__ == "__main__":
    for mode in sys.argv[1:]:
        asyncio.run(main(mode))
```

```
$ M="sleep-in-loop sleep-to-thread cpu-in-loop cpu-to-thread cpu-to-process cpu-to-interp"
$ python3.15 lag_probe.py $M
sleep-in-loop    1.00s  ticks=  5  p50 late=   0.1 ms  max late=1000.7 ms
sleep-to-thread  1.01s  ticks=104  p50 late=   0.1 ms  max late=   7.8 ms
cpu-in-loop      1.00s  ticks=  5  p50 late=   0.1 ms  max late=1000.3 ms
cpu-to-thread    1.00s  ticks= 71  p50 late=   5.2 ms  max late=   7.1 ms
cpu-to-process   1.01s  ticks=104  p50 late=   0.1 ms  max late=   1.3 ms
cpu-to-interp    1.00s  ticks=104  p50 late=   0.1 ms  max late=   1.3 ms
$ python3.14 lag_probe.py $M
sleep-in-loop    1.00s  ticks=  5  p50 late=   0.2 ms  max late=1000.8 ms
sleep-to-thread  1.00s  ticks=104  p50 late=   0.1 ms  max late=   2.1 ms
cpu-in-loop      1.00s  ticks=  5  p50 late=   0.1 ms  max late=1000.2 ms
cpu-to-thread    1.00s  ticks= 69  p50 late=   5.2 ms  max late=  13.8 ms
cpu-to-process   1.00s  ticks=104  p50 late=   0.1 ms  max late=   3.7 ms
cpu-to-interp    1.00s  ticks=104  p50 late=   0.1 ms  max late=   1.0 ms
```

Read the `in-loop` rows carefully. The work loop never awaits anything between its five rounds, so it is **one task step of a full second**. The heartbeat got 4 ticks before the work started and 1 after; its single late wake-up was 1000 ms late. The median looks perfect, which is why medians hide this problem. Look at the maximum, or at a high percentile.

The `to-thread` rows split by what the thread does:

- `time.sleep` releases the GIL, so the loop keeps its full ~104 ticks with sub-millisecond medians.
- A pure-Python `busy()` loop keeps the GIL. The loop thread gets it back only at the interpreter's switch interval, so every wake-up is about 5 ms late (`p50 late=5.2 ms`) and a third of the ticks are lost. Better than a 1 s stall, but not free.

A `ProcessPoolExecutor` or an `InterpreterPoolExecutor` (3.14+, one interpreter with its own GIL per worker) takes the CPU work off the loop's GIL completely. Both pools are started before the measurement, because creating a process or interpreter is itself slow.

Case 2: What the stall does to other requests

A heartbeat measures the loop. Users measure requests. This server answers `ping` immediately and computes a SHA-256 chain over a 40 MB `upload`. The clients are plain blocking sockets in their **own threads**, so they keep sending while the server's loop is busy:

```
# server_demo.py: one asyncio TCP server, two kinds of request. Clients run
# in their own threads (not on the server's loop), so they keep sending
# while the loop is blocked: one uploads, one pings every 20 ms.
import asyncio
import hashlib
import socket
import statistics
import sys
import threading
import time

MODE = sys.argv[1]  # "inline" or "to_thread"


def checksum(data: bytes) -> str:
    for _ in range(20):  # hashlib releases the GIL for large buffers
        data = hashlib.sha256(data).digest() + data[32:]
    return hashlib.sha256(data).hexdigest()[:12]


async def handle(reader, writer):
    while line := await reader.readline():
        if line == b"upload\n":
            payload = bytes(40 * 1024 * 1024)
            if MODE == "inline":
                digest = checksum(payload)
            else:
                digest = await asyncio.to_thread(checksum, payload)
            writer.write(f"ok {digest}\n".encode())
        else:
```

```
            writer.write(b"pong\n")
        await writer.drain()
    writer.close()


def pinger(port, rtts, n=60):
    with socket.create_connection(("127.0.0.1", port)) as s, s.makefile("rwb") as f:
        for _ in range(n):
            t0 = time.perf_counter()
            f.write(b"ping\n")
            f.flush()
            f.readline()
            rtts.append((time.perf_counter() - t0) * 1000)
            time.sleep(0.02)


def uploader(port, out):
    with socket.create_connection(("127.0.0.1", port)) as s, s.makefile("rwb") as f:
        time.sleep(0.2)  # let some pings through first
        f.write(b"upload\n")
        f.flush()
        out.append(f.readline().decode().strip())


async def main():
    server = await asyncio.start_server(handle, "127.0.0.1", 0)
    port = server.sockets[0].getsockname()[1]
    rtts, reply = [], []
    clients = [threading.Thread(target=pinger, args=(port, rtts)),
               threading.Thread(target=uploader, args=(port, reply))]
    async with server:
        for t in clients:
            t.start()
        while any(t.is_alive() for t in clients):
            await asyncio.sleep(0.01)
    q = statistics.quantiles(rtts, n=100, method="inclusive")
    print(f"{MODE:9} {reply[0]}  ping p50={q[49]:7.2f} ms  "
          f"p99={q[98]:7.2f} ms  max={max(rtts):7.2f} ms")


asyncio.run(main())
```

```
$ python3.15 server_demo.py inline
inline    ok d8365e844786  ping p50=   0.30 ms  p99= 532.72 ms  max=1296.88 ms
$ python3.15 server_demo.py to_thread
to_thread ok d8365e844786  ping p50=   0.28 ms  p99=   0.57 ms  max=   0.60 ms
```

```
$ python3.14 server_demo.py inline
inline    ok d8365e844786  ping p50=   0.28 ms  p99= 571.21 ms  max=1392.28 ms
$ python3.14 server_demo.py to_thread
to_thread ok d8365e844786  ping p50=   0.32 ms  p99=   0.55 ms  max=   0.57 ms
```

One upload request made an unrelated ping wait 1.3 seconds. The server did nothing wrong by its own measure: the upload finished and returned the right digest. Moving `checksum` to a thread brought the worst ping down by more than three orders of magnitude, and here the thread really does run in parallel, because `hashlib` releases the GIL while it hashes large buffers.

Case 3: Let debug mode name the slow task

Debug mode is off by default. Turn it on with `python -X dev`, `PYTHONASYNCIODEBUG=1`, `asyncio.run(..., debug=True)` or `loop.set_debug(True)`. This program has one task that blocks for over a second and one that blocks for 30 ms:

```
# debug_mode.py: let asyncio's debug mode name the code that blocked the loop.
import asyncio
import hashlib
import sys
import time


def checksum(data: bytes) -> str:
    # Looks harmless; on 40 MB it takes a while, and it runs on the loop.
    for _ in range(20):
        data = hashlib.sha256(data).digest() + data[32:]
    return hashlib.sha256(data).hexdigest()[:12]


async def handle_upload(n):
    payload = bytes(40 * 1024 * 1024)
    return f"upload {n}: {checksum(payload)}"


async def handle_ping():
    time.sleep(0.03)  # 30 ms: under the 100 ms default threshold
    return "pong"


async def main():
    loop = asyncio.get_running_loop()
    if len(sys.argv) > 1:
        loop.slow_callback_duration = float(sys.argv[1])  # seconds
    print("debug:", loop.get_debug(), " slow_callback_duration:", loop.slow_callback_duratio
    async with asyncio.TaskGroup() as tg:
```

```
        tg.create_task(handle_upload(1))
        tg.create_task(handle_ping())


asyncio.run(main())  # debug mode comes from -X dev or PYTHONASYNCIODEBUG=1
```

```
$ python3.15 -u debug_mode.py
debug: False  slow_callback_duration: 0.1
$ python3.15 -u -X dev debug_mode.py
debug: True  slow_callback_duration: 0.1
Executing <Task finished name='Task-2' coro=<handle_upload() done, defined at /tmp/aioblock/
$ PYTHONASYNCIODEBUG=1 python3.15 -u debug_mode.py 0.02
debug: True  slow_callback_duration: 0.02
Executing <Task finished name='Task-2' coro=<handle_upload() done, defined at /tmp/aioblock/
Executing <Task finished name='Task-3' coro=<handle_ping() done, defined at /tmp/aioblock/de
```

The warning comes from the `asyncio` logger at `WARNING` level; with no logging configured, Python's last-resort handler prints it to stderr. It names the coroutine and where it is defined. In debug mode tasks also remember where they were created, which here is inside `TaskGroup.create_task` rather than your code. The 30 ms `time.sleep` is invisible at the default 100 ms threshold and appears once the threshold is lowered.

Two limits. Debug mode times a whole **step**, so a task with many awaits is reported as "this task's step took 1.2 s", not "line 43". And it is meant for development: it also records a traceback for every task and future it creates, checks for calls from the wrong thread, and reports coroutines that were never awaited, all of which cost time.

Case 4: A watchdog that names the line

For production, a cheaper detector is enough. The loop updates a timestamp every 50 ms. A normal thread checks it; when the timestamp is stale, the thread prints the loop thread's current stack using `sys._current_frames()`, which works because the loop is stuck *in* that stack:

```
# watchdog.py: a production-safe detector. The loop touches a timestamp
# every 50 ms; a plain thread prints the loop thread's stack when the
# timestamp goes stale, so you see the blocking *line*, not just the task.
import asyncio
import sys
import threading
import time
import traceback


class LoopWatchdog:
    def __init__(self, threshold=0.25, tick=0.05):
        self.threshold, self.tick = threshold, tick
```

```
        self.last = time.monotonic()
        self.loop_thread = threading.get_ident()
        self._stop = threading.Event()

    async def run(self):
        threading.Thread(target=self._watch, daemon=True, name="loop-watchdog").start()
        try:
            while True:
                self.last = time.monotonic()
                await asyncio.sleep(self.tick)
        finally:
            self._stop.set()

    def _watch(self):
        reported = 0.0
        while not self._stop.wait(self.tick):
            stalled = time.monotonic() - self.last
            if stalled > self.threshold and self.last != reported:
                reported = self.last
                frame = sys._current_frames().get(self.loop_thread)
                stack = "".join(traceback.format_stack(frame, limit=4))
                print(f"[watchdog] loop blocked for {stalled * 1000:.0f} ms at:\n{stack}",
                      file=sys.stderr, flush=True)


def parse_report(text):
    total = 0
    for line in text.splitlines():  # pretend this is expensive parsing
        total += sum(int(x) for x in line.split(","))
        time.sleep(0.0001)
    return total


async def handler():
    await asyncio.sleep(0.1)
    report = "\n".join(",".join(str(i) for i in range(50)) for _ in range(4000))
    total = parse_report(report)          # the hidden blocker
    await asyncio.sleep(0.1)
    return total


async def main():
    dog = asyncio.create_task(LoopWatchdog().run())
    print("total:", await handler())
    dog.cancel()
```

```
asyncio.run(main())
```

```
$ python3.15 -u watchdog.py
[watchdog] loop blocked for 251 ms at:
  File "/home/box/.local/share/uv/python/cpython-3.15.0-linux-x86_64-gnu/lib/python3.15/asyn
    self._context.run(self._callback, *self._args)
  File "/tmp/aioblock/watchdog.py", line 57, in main
    print("total:", await handler())
  File "/tmp/aioblock/watchdog.py", line 50, in handler
    total = parse_report(report)          # the hidden blocker
  File "/tmp/aioblock/watchdog.py", line 43, in parse_report
    time.sleep(0.0001)

total: 4900000
```

The report points at `parse_report`, called from `handler`, which is the line to fix. The watchdog costs one wake-up every 50 ms on each side and does nothing unless the loop stalls. Send its output to your logging system instead of stderr, and rate-limit it. `sys._current_frames()` is CPython-specific and documented as intended for debugging, which is exactly this use.

Case 5: The shared pool is a limit too

`to_thread` and `run_in_executor(None, ...)` share one default `ThreadPoolExecutor`. It is created on first use with the standard default size:

```
# pool_limit.py: asyncio.to_thread shares one default thread pool. How long
# do 40 concurrent 200 ms blocking calls take?
import asyncio
import concurrent.futures as cf
import os
import time


async def run(n, executor=None):
    loop = asyncio.get_running_loop()
    if executor:
        loop.set_default_executor(executor)
    t0 = time.perf_counter()
    await asyncio.gather(*(asyncio.to_thread(time.sleep, 0.2) for _ in range(n)))
    return time.perf_counter() - t0


async def main():
    print("process_cpu_count:", os.process_cpu_count(),
          " default max_workers:", min(32, (os.process_cpu_count() or 1) + 4))
    print(f"default pool  : 40 calls in {await run(40):.2f}s")
```

```
    big = cf.ThreadPoolExecutor(max_workers=40, thread_name_prefix="blocking")
    print(f"40-thread pool: 40 calls in {await run(40, big):.2f}s")


asyncio.run(main())
```

```
$ python3.15 pool_limit.py
process_cpu_count: 8  default max_workers: 12
default pool  : 40 calls in 0.80s
40-thread pool: 40 calls in 0.21s
```

Forty 200 ms calls on 12 threads take four rounds: 0.8 s. The loop itself is never blocked here, but requests now wait in the executor's queue instead. If a service offloads a blocking client library, size a dedicated executor for it (and pass it to `run_in_executor`) so one slow dependency cannot use up the threads every other `to_thread` call needs. The formula printed above is the documented default (since 3.13) and the one in `concurrent/futures/thread.py`; the script recomputes it only for display.

Case 6: Choosing where blocking work goes

The blocking call is	Move it with	Why
A blocking client library (sync DB driver, `requests`, SDK)	Prefer an async library; otherwise `to_thread` or a dedicated `ThreadPoolExecutor`	Waiting in a system call releases the GIL
C code that releases the GIL (`hashlib` on large data, `zlib`, file I/O, many NumPy operations)	`to_thread`	Runs in parallel with the loop (Case 2)
Pure-Python CPU work (parsing, big `json.dumps`, loops)	`ProcessPoolExecutor` or `InterpreterPoolExecutor`	A thread would still share the GIL (Case 1, `cpu-to-thread`)
Long CPU work you can split	Do it in chunks with `await asyncio.sleep(0)` between them	Gives the loop a turn, but the total time is unchanged

The blocking call is	Move it with	Why
`time.sleep(x)` in a coroutine	`await asyncio.sleep(x)`	It was never meant to block

The trap

- **Measuring latency from inside the same loop.** The first version of Case 2 ran its ping client as a coroutine on the server's own loop. With the inline checksum it reported `ping p50= 0.08 ms ... max= 0.35 ms`: the client could not send while the loop was blocked, so it never observed the stall it was meant to catch. Probes must run on another thread, in another process, or on another machine. Also report the maximum or p99, not the median (Case 1).
- **`to_thread` for pure-Python CPU work.** It makes the stall smaller and longer instead of removing it: about 5 ms of lag on every tick, and a loop that does a third less work.
- **Leaving debug mode on in production.** It names slow steps, but it adds per-task tracebacks and checks to every operation. Use the watchdog (Case 4) there, and debug mode in tests and development.
- **A threshold that hides the problem.** At the default 100 ms, a handler that blocks for 30 ms on every request is never reported, yet it caps the whole service at about 33 such requests per second.
- **One shared executor for everything.** A slow dependency that fills the default pool delays every other `to_thread` caller (Case 5).

The boring rule

Never call anything that might block from a coroutine without deciding where it runs. Keep a heartbeat or a watchdog in every long-running asyncio service and alert on loop lag. Run tests and local development with `-X dev`, and lower `slow_callback_duration` to the latency budget of one request. Move blocking I/O and GIL-releasing C calls to threads, pure-Python CPU work to processes or interpreters, and give each blocking dependency its own bounded executor.

Try this

1. Run `lag_probe.py cpu-to-thread` after calling `sys.setswitchinterval(0.001)` at the top. What happens to `p50 late` and to the total time of the CPU work?
2. In `server_demo.py`, replace `hashlib.sha256` in `checksum` with a pure-Python loop of similar duration and run the `to_thread` mode again. Then switch it to a `ProcessPoolExecutor`. Compare the ping p99s.
3. Add the watchdog from Case 4 to `server_demo.py` and run the `inline` mode. Does the reported stack point at `checksum`?

4. Use `logging.basicConfig(level=logging.DEBUG)` with `-X dev` and call `loop.getaddrinfo("localhost", 80)`. What does asyncio log, and in which thread does the lookup run?
5. Find one synchronous call in an async service you maintain (`grep -n "requests\.\|time\.sleep\|open(" -r app/`), measure the request p99 with a ping client in another process, and move the call. Measure again.

Sources

- asyncio, "Developing with asyncio" (debug mode, `slow_callback_duration`, running blocking code): https://docs.python.org/3/library/asyncio-dev.html
- `asyncio.to_thread`: https://docs.python.org/3/library/asyncio-task.html#asyncio.to_thread
- `loop.run_in_executor`, `loop.set_default_executor`, `loop.slow_callback_duration`: https://docs.python.org/3/library/asyncio-eventloop.html
- `concurrent.futures` (`ThreadPoolExecutor` default `max_workers`, `ProcessPoolExecutor`, `InterpreterPoolExecutor`): https://docs.python.org/3/library/concurrent.futures.html
- Python Development Mode (`-X dev`): https://docs.python.org/3/library/devmode.html
- `PYTHONASYNCIODEBUG`: https://docs.python.org/3/using/cmdline.html#envvar-PYTHONASYNCIODEBUG
- `hashlib` (GIL released for data larger than 2047 bytes): https://docs.python.org/3/library/hashlib.html
- `sys.getswitchinterval`, `sys._current_frames`: https://docs.python.org/3/library/sys.html
- CPython source at the `v3.15.0` tag: `Lib/asyncio/base_events.py` (`_run_once`, the slow-step warning, the default executor), `Lib/asyncio/threads.py` (`to_thread`), `Lib/concurrent/futures/thread.py` (default pool size): https://github.com/python/cpython/tree/v3.15.0/Lib/asyncio
- What's New in Python 3.15 (asyncio: `TaskGroup.cancel`; no changes to debug mode): https://docs.python.org/3.15/whatsnew/3.15.html

ContextVar for Request-Scoped State

ContextVar for Request-Scoped State

Thesis

In concurrent Python, a **global** is a lie: the next `await` lets another task mutate it. `contextvars.ContextVar` copies values along with the task context so each request keeps its own id, auth, or trace span. This chapter races five asyncio tasks with a global versus a `ContextVar` and shows the contamination.

Mental model

```
ContextVar.set(v) → token
await …             (other tasks may run)
ContextVar.get()    → still v for this task
ContextVar.reset(token)
```

Worked examples

Verified with **CPython 3.15.0**.

```
import asyncio, contextvars

req_id = contextvars.ContextVar("req_id", default="-")

async def handle(i):
    token = req_id.set(f"req-{i}")
    try:
        await asyncio.sleep(0.01)
        return req_id.get()
    finally:
        req_id.reset(token)

async def naive_shared():
    global GID
    async def h(i):
        global GID
        GID = f"req-{i}"
```

```
        await asyncio.sleep(0.01)
        return GID
    return await asyncio.gather(*[h(i) for i in range(5)])

async def main():
    good = await asyncio.gather(*[handle(i) for i in range(5)])
    print("ContextVar results:", good)
    bad = await naive_shared()
    print("global mutation results:", bad)

asyncio.run(main())
```

Output:

```
3.15.0 (main, Oct  9 2026, 15:24:40) [Clang 22.1.3 ]
ContextVar results: ['req-0', 'req-1', 'req-2', 'req-3', 'req-4']
global mutation results: ['req-4', 'req-4', 'req-4', 'req-4', 'req-4']
```

Trap

Setting a ContextVar in middleware but forgetting `reset`. Leaks the value into the next task that reuses the context. Prefer `token` + `try/finally`, or a context manager wrapper.

Boring rule

Request ids, tenants, and trace context go in `ContextVar`s (or structured context objects), never in module globals. Middleware must reset.

Try this

1. Plug a `ContextVar` into `logging.Filter` via `LoggerAdapter` / filter.
2. Show that threads need `copy_context().run(...)` explicitly.
3. Read PEP 567.

Sources

- contextvars — Python 3.15 docs
- PEP 567

TaskGroup Structured Concurrency

TaskGroup Structured Concurrency

Thesis

`asyncio.TaskGroup` (3.11+) is structured concurrency for asyncio: tasks spawned in the group are awaited together; the first unhandled exception cancels siblings and re-raises as `ExceptionGroup`. Lifetimes nest with the `async with` block.

Mental model

```
async with asyncio.TaskGroup() as tg:
    tg.create_task(work(1))
    tg.create_task(work(2))
# all done or cancelled; exceptions bundled
```

Contrast: bare `create_task` without a supervisor leaks failures into "Task exception was never retrieved".

Worked example

```
# taskgroup_demo.py
import asyncio

async def ok(n):
    await asyncio.sleep(0.05)
    return n * 2

async def boom():
    await asyncio.sleep(0.01)
    raise RuntimeError("boom")

async def main_ok():
    async with asyncio.TaskGroup() as tg:
        t1 = tg.create_task(ok(1))
        t2 = tg.create_task(ok(2))
    print("results", t1.result(), t2.result())
```

```
async def main_fail():
    try:
        async with asyncio.TaskGroup() as tg:
            tg.create_task(ok(1))
            tg.create_task(boom())
    except* RuntimeError as eg:
        print("caught", type(eg).__name__, [repr(e) for e in eg.exceptions])

asyncio.run(main_ok())
asyncio.run(main_fail())
```

```
results 4 9
ExceptionGroup: ExceptionGroup ['boom-1']
```

Trap

Swallowing the `ExceptionGroup` without inspecting `.exceptions`. Also: spawning tasks that outlive the group on purpose — that is the opposite of structured concurrency; use an explicit long-lived supervisor if you must.

Boring rule

Prefer `TaskGroup` (or Trio nurseries) whenever several async jobs share a scope. Use `except*` to unwrap typed failures.

Try this

1. Add a third task that ignores `CancelledError` incorrectly and observe shutdown.
2. Nest TaskGroups and cancel the outer scope.
3. Compare with `asyncio.gather(..., return_exceptions=True)`.

Sources

- Python docs: TaskGroup
- PEP 654 — Exception Groups
- PEP 678 — Exception Notes

Async call graphs: asking asyncio who is awaiting whom

Async call graphs: asking asyncio who is awaiting whom

Thesis

A normal Python traceback answers "how did I get here?" by walking the frame stack. In asyncio that answer is almost useless. A suspended task has a short stack (its coroutine and whatever it is awaiting), and the task that *caused* it to exist is not on that stack at all. It is a different task, blocked in `await` or in a `TaskGroup.__aexit__` somewhere else. To understand a stuck request, or to build a profiler that says "40% of wall time is `handler → db_query`", you need a second dimension: **which task is awaiting this one**.

Since Python 3.14, asyncio keeps that edge and exposes it. `asyncio.capture_call_graph()` returns a task's own call stack plus the chain of tasks awaiting it, `asyncio.print_call_graph()` and `asyncio.format_call_graph()` render it, and the same data powers the out-of-process `python -m asyncio ps` and `pstree` commands (asyncio call graph introspection).

This chapter uses the in-process API. You print the await chain of a leaf task, then build a 50-line async-aware sampling profiler that produces flame-graph "folded" stacks. Then you meet the limit that pushes production profilers out of process: a sampler that runs on the event loop never sees the CPU work that blocks it.

All listings run on **CPython 3.15.0** (`uv run --python 3.15.0 --no-project python FILE`). They produce the same results on 3.14.8.

Mental model

API (3.14+)	Returns	Use it for
`asyncio.capture_call_graph(future=None, *, depth=1, limit=None)`	`FutureCallGraph` or `None`	Programmatic access: profilers, watchdogs, metrics
`asyncio.format_call_graph(...)`	`str`	Logging a stuck task
`asyncio.print_call_graph(..., file=None)`	prints	Interactive debugging
`FutureCallGraph.call_stack`	tuple of `FrameCallGraphEntry(frame)`, **innermost first**	The task's own coroutine chain
`FutureCallGraph.awaited_by`	tuple of `FutureCallGraph`	Parent tasks (more than one if several tasks await the same future)

API (3.14+)	Returns	Use it for
`asyncio.future_add_to_awaited_by(fut, waiter)`		For authors of custom future-like objects, so they join the graph
`python -m asyncio ps PID` / `pstree PID`	table / tree	The same graph read from another process

With no argument, `capture_call_graph()` describes the *current* task. That is why it must be called from code running on the loop. `depth` skips that many frames at the top of the current stack (default 1, the caller of `capture_call_graph`), and `limit` caps how many entries of the call stack are kept.

Case 1: print the await chain of a leaf task

```
# callgraph.py -- who is awaiting whom, from inside the program
import asyncio

async def fetch(name, delay):
    await asyncio.sleep(delay)
    return name

async def worker(n):
    async with asyncio.TaskGroup() as tg:
        tg.create_task(fetch(f"a{n}", 0.2), name=f"fetch-a{n}")
        tg.create_task(fetch(f"b{n}", 0.2), name=f"fetch-b{n}")

async def watchdog():
    await asyncio.sleep(0.05)
    # every task the loop knows about, and its await chain
    for t in sorted(asyncio.all_tasks(), key=lambda t: t.get_name()):
        if t.get_name().startswith("fetch-a1"):
            asyncio.print_call_graph(t)

async def main():
    async with asyncio.TaskGroup() as tg:
        tg.create_task(worker(1), name="worker-1")
        tg.create_task(worker(2), name="worker-2")
        tg.create_task(watchdog(), name="watchdog")

asyncio.run(main())
```

```
uv run --python 3.15.0 --no-project python callgraph.py
```

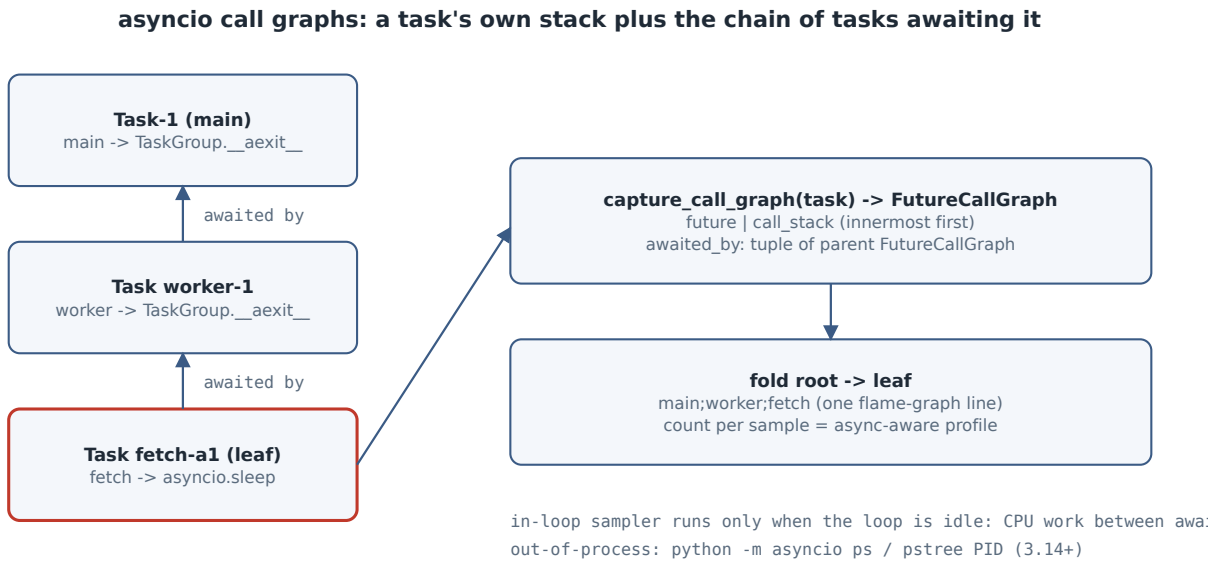


Figure 1: An asyncio call graph: a leaf task's own call stack, the chain of awaiting tasks above it, and how capture_call_graph and folding turn that into one flame-graph line

```
* Task(name='fetch-a1', id=0x7fa92f30ba10)
  + Call stack:
  |   File '/home/box/.local/share/uv/python/cpython-3.15.0-linux-x86_64-gnu/lib/python3.15/
  |   File '/tmp/cg/callgraph.py', line 5, in async fetch()
  + Awaited by:
    * Task(name='worker-1', id=0x7fa92f2118b0)
      + Call stack:
      |   File '/home/box/.local/share/uv/python/cpython-3.15.0-linux-x86_64-gnu/lib/python3
      |   File '/home/box/.local/share/uv/python/cpython-3.15.0-linux-x86_64-gnu/lib/python3
      |   File '/tmp/cg/callgraph.py', line 9, in async worker()
      + Awaited by:
        * Task(name='Task-1', id=0x7fa92f211a90)
          + Call stack:
          |   File '/home/box/.local/share/uv/python/cpython-3.15.0-linux-x86_64-gnu/lib/pyt
          |   File '/home/box/.local/share/uv/python/cpython-3.15.0-linux-x86_64-gnu/lib/pyt
          |   File '/tmp/cg/callgraph.py', line 21, in async main()
```

Read it from the top. `fetch-a1` is suspended in `asyncio.sleep` inside `fetch`. That is its whole stack, the same as a traceback would show. The useful part is **Awaited by**: `worker-1` is suspended in `TaskGroup.__aexit__` at line 9 of `worker`, waiting for its children. It in turn is awaited by `Task-1`, the task `asyncio.run` created for `main`, also inside a TaskGroup exit. Without this graph, `fetch-a1` and `worker-1` would look like two unrelated sleeping tasks.

Note the task names. `create_task(..., name=...)` costs nothing, and it is the difference between `Task-17` and `fetch-a1` in every one of these outputs.

Case 2: an async-aware sampling profiler in 50 lines

A sampling profiler takes the stack many times a second and counts identical stacks. For async code, "the stack" has to include the awaiting tasks, or every sample would look like `sleep` with no caller. The listing walks each task's graph to the root, drops asyncio's own `tasks.py` and `taskgroups.py` frames, and prints Brendan Gregg's *folded* format (`root;caller;leaf count`), which flamegraph.pl and speedscope read directly.

```
# asyncprof.py -- an in-loop sampler that folds async stacks for a flame graph
import asyncio, collections, time

def busy(ms):                          # pretend CPU work between awaits
    end = time.perf_counter() + ms / 1000
    while time.perf_counter() < end:
        pass

async def db_query():
    await asyncio.sleep(0.03)

async def render():
    busy(2)
    await asyncio.sleep(0.01)

async def handler(i):
    await db_query()
    await render()

def folded(g):
    """Walk one FutureCallGraph to the root; return 'root;...;leaf'."""
    frames = []                        # innermost first
    while g is not None:
        frames += [e.frame.f_code.co_qualname for e in g.call_stack
                   if not e.frame.f_code.co_filename.endswith(
                       ("tasks.py", "taskgroups.py"))]
        g = g.awaited_by[0] if g.awaited_by else None
    return ";".join(reversed(frames))

async def sampler(stacks, period=0.005):
    me = asyncio.current_task()
    while True:
        for t in asyncio.all_tasks():
            if t is me or t.done():
                continue
            g = asyncio.capture_call_graph(t)
            if g is not None:
                stacks[folded(g)] += 1
        await asyncio.sleep(period)
```

```
async def main():
    stacks = collections.Counter()
    prof = asyncio.create_task(sampler(stacks), name="sampler")
    async with asyncio.TaskGroup() as tg:
        for i in range(20):
            tg.create_task(handler(i), name=f"req-{i}")
    prof.cancel()
    for stack, n in stacks.most_common():
        print(f"{n:4d} {stack}")

asyncio.run(main())
```

```
uv run --python 3.15.0 --no-project python asyncprof.py
```

```
 100 main;handler;db_query
  23 main;handler;render
  20 main;handler
   8 main
```

Each request spends about 30 ms in `db_query`, then 2 ms of CPU in `render`, then 10 ms sleeping in `render`. The counts follow the *waiting* time: `db_query` dominates, and `render` gets about a quarter as much. `main;handler` with no leaf is a task that has been scheduled but not yet started, or has just woken and not yet stepped into its next coroutine. `main` alone is `Task-1` waiting for its TaskGroup.

The trap: an in-loop sampler cannot see what blocks the loop

`busy(2)` burns 2 ms of CPU in every request, 40 ms in total. **Not one sample shows it.** The sampler is a task on the same event loop. It only runs when the loop is between steps, which means it is never running while `busy()` is. An in-loop profiler measures where tasks *wait*, not where the loop *spends CPU*. Profiling blocked event loops is exactly the case you most need a profiler for, and an in-loop profiler is blind to it.

That is why production async profilers sample from outside the loop. They use a separate thread, or a separate process that reads interpreter memory, the way `python -m asyncio pstree PID` does. These tools read the same task and awaiter structure without needing the loop to yield. Use the in-loop API for watchdogs ("this request has been awaiting the same thing for 5 s; log its graph") and for answering "what are my tasks waiting on". Use an out-of-process sampler for CPU hotspots.

Two smaller traps:

- **Version gate.** On 3.13 `asyncio.capture_call_graph` does not exist. The sampler task dies with `AttributeError` at its first sample, and because nothing awaits that task, `asyncprof.py` on 3.13 prints **nothing at all** and exits 0. Check `sys.version_info >= (3, 14)` up front, or await the profiler task so its exception surfaces.

- **Custom awaitables are invisible** unless they call `asyncio.future_add_to_awaited_by()`. Futures from third-party event-loop libraries may cut the chain short.

Boring rule

Name your tasks. When a task looks stuck, log `asyncio.format_call_graph(task)`, not `task.get_stack()`, because the awaiting chain is the part you need. For time spent in CPU work, profile from outside the loop.

Exercises

1. Change `watchdog` to log, using `format_call_graph`, every task whose graph has not changed between two samples 100 ms apart.
2. Pass `limit=1` to `capture_call_graph` in the sampler. Which frames disappear from the folded output, and why does the profile stay readable?
3. Feed `asyncprof.py`'s output to speedscope. Then move the 2 ms of CPU to a `busy(20)` and compare the wall time with what the flame graph shows.
4. Write a tiny custom awaitable and use `future_add_to_awaited_by` so it appears in `print_call_graph`.

Sources

- Python docs, asyncio: Call Graph Introspection
- Python docs, What's New in Python 3.14: asyncio introspection
- Python docs, asyncio Task object: `create_task` name, `all_tasks`
- B. Gregg, Flame Graphs and the folded stack format

Profiling, Optimization, and Native C/Rust Interoperability

Profiling, Optimization, and Native C/Rust Interoperability

After reading this chapter, you will master performance diagnostics using `cProfile` and `pstats`, track heap allocations and memory spikes with `tracemalloc`, interface with native C libraries using `ctypes`, and architect high-speed compiled extensions with PyO3 and Rust.

Mental model

Performance engineering follows an evidence-based feedback loop: **never optimize without profiling data**.

```
Performance Engineering Pipeline:
  [ Macro Measurement ]    time.perf_counter()    Identifies slow high-level subsystems


  [ Deterministic Profiling ]    cProfile / pstats    Identifies exact hotspot functions


  [ Memory Profiling ]     tracemalloc    Locates exact lines allocating heap memory


  [ Optimization Paths ]:
    1. Algorithmic Fix (O(N^2) -> O(N log N) using sets/dicts)
    2. Built-in C-Accelerators (itertools, collections.deque)
    3. Native FFI Binding (ctypes or PyO3 Rust extension)
```

Minimal example

Save as `profiling_and_ctypes.py`:

```python
# profiling_and_ctypes.py
import cProfile
import ctypes
import pstats
import tracemalloc
```

```python
def cpu_heavy_function() -> list[int]:
    """Function with an intentional list-growing bottleneck."""
    data = []
    for i in range(200_000):
        data.append(i * 2)
    return data

def demonstrate_profiling() -> None:
    # 1. Deterministic Profiling with cProfile
    profiler = cProfile.Profile()
    profiler.enable()

    cpu_heavy_function()

    profiler.disable()
    stats = pstats.Stats(profiler).sort_stats("tottime")
    print("--- Top Profiler Entries ---")
    stats.print_stats(3)

def demonstrate_ctypes() -> None:
    # 2. Native C interop via ctypes (Calling libc directly)
    libc = ctypes.CDLL(None)  # Accesses standard C library
    print("\n--- Native C Interop with ctypes ---")
    # Call standard C 'puts' function
    libc.puts(b"Hello from native C library puts()!")

def main() -> None:
    demonstrate_profiling()
    demonstrate_ctypes()

if __name__ == "__main__":
    main()
```

Run via `uv run python profiling_and_ctypes.py`:

```
--- Top Profiler Entries ---
   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.012    0.012    0.012    0.012 ...:cpu_heavy_function
        ...
--- Native C Interop with ctypes ---
Hello from native C library puts()!
```

Worked examples

Case 1: Locating Memory Leaks with `tracemalloc`

`tracemalloc` tracks every memory block allocated by CPython, recording the exact Python source file and line number:

```
# memory_leak_tracer.py
import tracemalloc

def simulate_memory_allocation() -> list[str]:
    # Allocate 100,000 strings
    return [f"cluster_telemetry_event_node_{i}" for i in range(100_000)]

def main() -> None:
    # Start tracking memory allocations
    tracemalloc.start()

    snapshot_before = tracemalloc.take_snapshot()
    data = simulate_memory_allocation()
    snapshot_after = tracemalloc.take_snapshot()

    # Compare snapshots to identify top allocating lines
    top_diffs = snapshot_after.compare_to(snapshot_before, "lineno")

    print("--- Top Memory Allocating Lines ---")
    for stat in top_diffs[:3]:
        print(f"  {stat}")

    current, peak = tracemalloc.get_traced_memory()
    print(f"\nCurrent memory: {current / (1024*1024):.2f} MB | Peak memory: {peak / (1024*10
    tracemalloc.stop()

if __name__ == "__main__":
    main()
```

Run:

```
uv run python memory_leak_tracer.py
```

Output:

```
--- Top Memory Allocating Lines ---
  ...:simulate_memory_allocation: size=7524 KiB (+7524 KiB), count=100000 (+100000)
Current memory: 7.37 MB | Peak memory: 7.37 MB
```

Case 2: Calling Native C Functions with `ctypes.Structure`

You can define C-compatible memory structs in Python and pass them directly across native library boundaries:

```
# c_struct_ffi.py
import ctypes

class Point2D(ctypes.Structure):
    _fields_ = [
        ("x", ctypes.c_double),
        ("y", ctypes.c_double),
    ]

def main() -> None:
    pt = Point2D(10.5, 20.25)
    print(f"Python C-compatible struct: x={pt.x}, y={pt.y}")
    print(f"Size of struct in C memory: {ctypes.sizeof(pt)} bytes (2 x 8-byte doubles)")

if __name__ == "__main__":
    main()
```

Run:

```
uv run python c_struct_ffi.py
```

Output:

```
Python C-compatible struct: x=10.5, y=20.25
Size of struct in C memory: 16 bytes (2 x 8-byte doubles)
```

Case 3: Modern High-Performance Extensions: PyO3 and Rust

When pure Python algorithms cannot meet latency budgets (e.g. cryptography, parsers, simulation), modern Python projects use **PyO3** to author extensions in Rust.

Benefits of PyO3 over legacy C-extensions: - Memory safety guarantees without segmentation faults. - Seamless release of the GIL using `Python::allow_threads`. - Zero-cost conversion between Python types and Rust standard types.

Pitfalls

Pitfall 1: Premature Micro-Optimization

Bikeshedding string concatenations or swapping operators before profiling is wasted effort. In 95% of applications, performance bottlenecks are concentrated in I/O wait times or poorly chosen algorithmic complexity ($O(N^2)$ lookups in lists).

Pitfall 2: Memory Hazards in `ctypes`

`ctypes` bypasses Python's memory protections. Passing an incorrect pointer, writing past an array boundary, or calling a function with the wrong argument types will immediately trigger a hard crash (**Segmentation Fault**) that terminates the entire Python process.

Exercises

1. Profile an algorithm that searches for items in a `list` vs a `set` using `cProfile` and compare their execution profiles.
2. Use `tracemalloc` to identify how much RAM is consumed by 50,000 instances of a standard class versus a class using `__slots__`.
3. Call the C standard library `qsort` function using `ctypes` to sort an array of C integers.
4. Export profiling results to a `.prof` file using `cProfile.run(..., filename="out.prof")` and inspect it using `pstats`.

Further reading

- Python Standard Library: `cProfile`, `pstats`, `tracemalloc`, and `ctypes` modules.
- PyO3 Project: *Rust bindings for the Python interpreter*.
- PEP 659: *Zero-cost exception handling*.

Attaching to a Live Process

Attaching to a Live Process

A service has been running for six hours and now it is stuck, or slow, or holding a value it should not have. Restarting it destroys the evidence. Adding a `print` means redeploying and waiting for the bug to come back. Python **3.14** adds a supported way in from the outside (PEP 768). Another Python process with the right OS permissions can ask a running interpreter to execute a script at its next safe point. Three tools are built on that one mechanism:

Tool	What it does
`sys.remote_exec(pid, path)`	Queue a `.py` file to run **inside** the target process
`python -m pdb -p PID`	Attach the full `pdb` debugger to a running process; `quit` detaches
`python -m asyncio ps PID` / `pstree PID`	Print every asyncio task and who awaits whom, without stopping the loop

All examples run with `uv` on Linux and CPython 3.14 (3.14.8 at the time of writing). No code in the target has to change: these are **live** processes started the normal way.

Mental model

- **Safe point, not a signal handler.** `sys.remote_exec` writes a script path and a "pending" flag into the target's thread state, then sets a bit in the interpreter's *eval breaker*. That is the same check the interpreter uses for signals and the GIL hand-off. The target's **main thread** runs the script the next time it executes Python bytecode. The call returns at once and tells you nothing about when, or whether, the script ran.
- **It runs as the target.** The script executes with the target's globals reachable through `sys.modules["__main__"]`, its open files, its threads, and its privileges. `print` goes to the **target's** stdout. Anything you want back you must write somewhere yourself (a file, a socket).
- **Same interpreter family.** Caller and target must be the same CPython `major.minor`. If either is a pre-release (alpha, beta, rc), they must be the exact same version.
- **OS permission is the gate.** On Linux this is the same check `gdb` or `strace -p` need: same user, and not blocked by Yama `ptrace_scope`, a container seccomp profile, or another tracer already attached. Python adds no new privilege. Root can attach to anything.
- **Opt-out exists.** `PYTHON_DISABLE_REMOTE_DEBUG=1` or `-X disable_remote_debug` turns off the in-process side, for hardened deployments where nobody should ever attach.

Worked examples

Case 1: Run a probe inside a running worker

Save as `worker.py`. It is an ordinary long-running loop with a background thread and some module-level state.

```
# worker.py
import os
import threading
import time

jobs_done = 0
config = {"batch": 50, "mode": "slow"}


def heartbeat():
    while True:
        time.sleep(1)


def process(n):
    time.sleep(0.2)
    return n * n


def main():
    global jobs_done
    threading.Thread(target=heartbeat, name="heartbeat", daemon=True).start()
    print("pid", os.getpid(), flush=True)
    while True:
        process(jobs_done)
        jobs_done += 1
        if config["mode"] == "fast":
            print("switched to fast after", jobs_done, "jobs", flush=True)
            config["mode"] = "done"


if __name__ == "__main__":
    main()
```

Save as `hello_probe.py`. This is the code that will run **inside** the worker.

```
# hello_probe.py
import os
import sys
```

```
main = sys.modules["__main__"]
print(f"[probe] running inside pid {os.getpid()}: "
      f"jobs_done={main.jobs_done} mode={main.config['mode']}", flush=True)
```

Save as `attach_hello.py`. This is the caller.

```
# attach_hello.py
import os
import sys

pid = int(sys.argv[1])
script = os.path.abspath("hello_probe.py")
sys.remote_exec(pid, script)
print(f"queued {script} for pid {pid}")
```

Run the worker in one terminal:

```
uv run --python 3.14 python worker.py
```

```
pid 666610
```

In a second terminal, use the PID the worker printed:

```
uv run --python 3.14 python attach_hello.py 666610
```

```
queued /tmp/livepy/hello_probe.py for pid 666610
```

Back in the first terminal, the worker printed one more line by itself:

```
[probe] running inside pid 666610: jobs_done=7 mode=slow
```

Pass an **absolute** path. The caller checks that the file exists relative to *its own* working directory, and the target opens the path later from *its* working directory. From a subdirectory (`other/`), pass a relative name:

```
cd other && uv run --python 3.14 python -c "import sys; sys.remote_exec(666610, 'hello_probe
```

```
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    import sys; sys.remote_exec(666610, 'hello_probe.py')
                ~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^
FileNotFoundError: Script file does not exist
```

Case 2: Dump every thread's stack and bring it home

Save as `dump_stacks.py`. This is the tool you actually keep in a toolbox. It writes a probe into a private temp directory, queues it, and waits for a result file. The probe writes to a temporary name and then `os.replace`s it into place, so the caller never reads a half-written file. `faulthandler.dump_traceback(all_threads=True)` prints every thread's Python stack.

```
# dump_stacks.py
import os
import sys
import tempfile
import time
from pathlib import Path


PROBE = """\
import faulthandler, os, sys
main = sys.modules["__main__"]
with open({tmp!r}, "w") as fh:
    print("jobs_done:", getattr(main, "jobs_done", "?"), file=fh)
    faulthandler.dump_traceback(file=fh, all_threads=True)
os.replace({tmp!r}, {out!r})
"""


def dump(pid, timeout=5.0):
    workdir = Path(tempfile.mkdtemp(prefix="pydump-"))
    out = workdir / "stacks.txt"
    tmp = workdir / "stacks.tmp"
    script = workdir / "probe.py"
    script.write_text(PROBE.format(tmp=str(tmp), out=str(out)))
    sys.remote_exec(pid, str(script))   # returns at once; runs later
    deadline = time.monotonic() + timeout
    while not out.exists():             # probe renames when done
        if time.monotonic() > deadline:
            # Leave probe.py in place: the target may still run it later.
            raise TimeoutError(f"pid {pid} did not run the probe in {timeout}s; "
                               f"left {script}")
        time.sleep(0.05)
    text = out.read_text()
    script.unlink()
    out.unlink()
    workdir.rmdir()
    return text


if __name__ == "__main__":
```

```
    limit = float(sys.argv[2]) if len(sys.argv) > 2 else 5.0
    print(dump(int(sys.argv[1]), timeout=limit), end="")
```

With `worker.py` running (this run printed `pid 667102`):

```
uv run --python 3.14 python dump_stacks.py 667102
```

Output (stdlib paths shortened to `…/lib/python3.14/`; thread IDs vary):

```
jobs_done: 7
Thread 0x00007f1d966a66c0 [heartbeat] (most recent call first):
  File "/tmp/livepy/worker.py", line 12 in heartbeat
  File "…/lib/python3.14/threading.py", line 1024 in run
  File "…/lib/python3.14/threading.py", line 1082 in _bootstrap_inner
  File "…/lib/python3.14/threading.py", line 1044 in _bootstrap

Current thread 0x00007f1d96963780 [python3.14] (most recent call first):
  File "<string>", line 5 in <module>
  File "/tmp/livepy/worker.py", line 16 in process
  File "/tmp/livepy/worker.py", line 25 in main
  File "/tmp/livepy/worker.py", line 33 in <module>
```

Read it bottom-up as usual. The `<string>` frame on top of the main thread is the probe itself. Everything under it is what the worker was doing when it reached the safe point: inside `process`, in `time.sleep`. The heartbeat thread is parked in its own sleep.

Case 3: `pdb -p` — a debugger on a process you did not start under one

`python -m pdb -p PID` uses the same mechanism to make the target connect back to your terminal. You get a normal `(Pdb)` prompt stopped at the target's current line. You can inspect, evaluate, and even change state. `quit` **detaches** and the target keeps running; it does not kill it.

With `worker.py` running:

```
uv run --python 3.14 python -m pdb -p 667102
```

Type these at the prompt (shown here as a pipe so the session is reproducible):

```
printf 'p jobs_done, config\nwhere\n!config["mode"] = "fast"\nquit\n' | uv run --python 3.14
```

Output:

```
> /tmp/livepy/worker.py(16)process()
-> time.sleep(0.2)
(Pdb) (8, {'batch': 50, 'mode': 'slow'})
(Pdb)   /tmp/livepy/worker.py(33)<module>()
```

```
-> main()
  /tmp/livepy/worker.py(25)main()
-> process(jobs_done)
> /tmp/livepy/worker.py(16)process()
-> time.sleep(0.2)
(Pdb) (Pdb)
```

The worker's own terminal shows the change took effect after detach:

```
pid 667102
switched to fast after 9 jobs
```

While the `(Pdb)` prompt is open the target's main thread is **stopped**. On a server that means requests queue behind you. Get in, read, get out.

Case 4: See every asyncio task with `ps` and `pstree`

Save as `aio_service.py`. Two workers each run two long "fetches" inside `TaskGroup`s. Name your tasks; the tools print the names.

```python
# aio_service.py
import asyncio, os

async def fetch(name, delay):
    await asyncio.sleep(delay)

async def worker(name):
    async with asyncio.TaskGroup() as tg:
        tg.create_task(fetch(f"{name}-a", 3600), name=f"{name}-fetch-a")
        tg.create_task(fetch(f"{name}-b", 3600), name=f"{name}-fetch-b")

async def main():
    print("pid", os.getpid(), flush=True)
    async with asyncio.TaskGroup() as tg:
        tg.create_task(worker("orders"), name="orders")
        tg.create_task(worker("billing"), name="billing")

asyncio.run(main())
```

Run it, then from another terminal:

```
uv run --python 3.14 python aio_service.py
uv run --python 3.14 python -m asyncio ps 667133
```

Output:

```
tid        task id              task name            coroutine stack
-------------------------------------------------------------------------------------------
667133     0x7f7e64c68b90       Task-1               TaskGroup._aexit -> TaskGroup.__aexit__ ->
667133     0x7f7e64c68d70       orders               TaskGroup._aexit -> TaskGroup.__aexit__ ->
667133     0x7f7e64e03df0       billing              TaskGroup._aexit -> TaskGroup.__aexit__ ->
667133     0x7f7e64e03c20       orders-fetch-a       sleep -> fetch
667133     0x7f7e64bde350       orders-fetch-b       sleep -> fetch
667133     0x7f7e64bde6d0       billing-fetch-a      sleep -> fetch
667133     0x7f7e64e56de0       billing-fetch-b      sleep -> fetch
```

`ps` is a flat table: each task, its coroutine stack, and the task awaiting it. `pstree` draws the same data as an await tree:

```
uv run --python 3.14 python -m asyncio pstree 667133
```

Output (stdlib paths shortened):

```
(T) Task-1
    main /tmp/livepy/aio_service.py:14
        TaskGroup.__aexit__ …/lib/python3.14/asyncio/taskgroups.py:72
            TaskGroup._aexit …/lib/python3.14/asyncio/taskgroups.py:121
                (T) orders
                    worker /tmp/livepy/aio_service.py:8
                        TaskGroup.__aexit__ …/lib/python3.14/asyncio/taskgroups.py:72
                            TaskGroup._aexit …/lib/python3.14/asyncio/taskgroups.py:121
                                (T) orders-fetch-a
                                    fetch /tmp/livepy/aio_service.py:5
                                        sleep …/lib/python3.14/asyncio/tasks.py:712
                                (T) orders-fetch-b
                                    fetch /tmp/livepy/aio_service.py:5
                                        sleep …/lib/python3.14/asyncio/tasks.py:712
                (T) billing
                    worker /tmp/livepy/aio_service.py:8
                        TaskGroup.__aexit__ …/lib/python3.14/asyncio/taskgroups.py:72
                            TaskGroup._aexit …/lib/python3.14/asyncio/taskgroups.py:121
                                (T) billing-fetch-a
                                    fetch /tmp/livepy/aio_service.py:5
                                        sleep …/lib/python3.14/asyncio/tasks.py:712
                                (T) billing-fetch-b
                                    fetch /tmp/livepy/aio_service.py:5
                                        sleep …/lib/python3.14/asyncio/tasks.py:712
```

Neither command runs code in the target. They read the interpreter's memory through the same debug offsets, so they work even when the event loop is wedged and could never service a request.

The trap

A busy C call delays the probe. The script runs at the next bytecode safe point on the **main thread**. If the main thread is blocked inside one long C call (a `sleep`, a blocking `recv`, a big NumPy operation, a lock acquire), nothing runs until that call returns. Save as `blocked.py`:

```
# blocked.py
import os
import time

print("pid", os.getpid(), flush=True)
time.sleep(8)          # one long C-level call in the main thread
print("woke up", flush=True)
time.sleep(2)
```

Run it, then give `dump_stacks.py` a 2-second timeout:

```
uv run --python 3.14 python blocked.py
uv run --python 3.14 python dump_stacks.py 667435 2
```

Output from the caller (last line):

```
TimeoutError: pid 667435 did not run the probe in 2.0s; left /tmp/pydump-hyem4e8i/probe.py
```

The target printed `woke up` six seconds later and **then** ran the probe. The file it left behind shows it ran at line 6, after the sleep:

```
jobs_done: ?
Current thread 0x00007efe7ce36780 [python3.14] (most recent call first):
  File "<string>", line 5 in <module>
  File "/tmp/livepy/blocked.py", line 6 in <module>
```

So never delete the probe right after `remote_exec` returns. Wait for an effect you can see (the result file), and on timeout leave the script where it is. A missing file at execution time is an error in the target, not in your tool. When the main thread is stuck in C code, reach for `asyncio ps` (for async code) or a native tool such as `py-spy`/`gdb`, because they read memory instead of waiting for a safe point.

Async deadlocks show up as cycles. Save as `aio_deadlock.py`. Two tasks await each other:

```
# aio_deadlock.py
import asyncio
import os

tasks = {}
```

```
async def a():
    await asyncio.sleep(0.1)
    await tasks["b"]


async def b():
    await asyncio.sleep(0.1)
    await tasks["a"]


async def main():
    print("pid", os.getpid(), flush=True)
    tasks["a"] = asyncio.create_task(a(), name="task-a")
    tasks["b"] = asyncio.create_task(b(), name="task-b")
    await asyncio.gather(tasks["a"], tasks["b"])


asyncio.run(main())
```

Run it, then point `pstree` at the PID it printed:

```
uv run --python 3.14 python aio_deadlock.py
uv run --python 3.14 python -m asyncio pstree PID
```

Output (exit status 1):

```
ERROR: await-graph contains cycles - cannot print a tree!

cycle: task-a → task-b → task-a
```

That error is the diagnosis. `asyncio ps` still prints the flat table if you need the stacks.

The usual refusals, and what they mean:

Message	Cause
`RuntimeError: Remote debugging is not enabled in the remote process`	Target started with `PYTHON_DISABLE_REMOTE_DEBUG=1` or `-X disable_remote_debug`
`RuntimeError: Can't send commands from a pre-release Python interpreter to a process running a different Python version`	Caller is 3.15rc, target is 3.14: versions must match
`RuntimeError: PyRuntime address lookup failed during debug offsets initialization`	The PID is not a CPython process (here, the PID of the shell wrapper, not the Python child)
`FileNotFoundError: Script file does not exist`	Relative path, or the file was removed

Message	Cause
A permission error (`PermissionError` / `Operation not permitted`)	Different user, Yama `ptrace_scope`, container seccomp, or another tracer already attached

A failing probe does **not** crash the target. A typo in the probe is printed to the target's stderr and the program keeps running:

```
Error executing debugger script /tmp/livepy/bad_probe.py:
Traceback (most recent call last):
  File "<string>", line 3, in <module>
AttributeError: module '__main__' has no attribute 'jobs_dne'
```

The boring rule

- Read-only first: `asyncio ps` / `pstree` and a stack-dump probe answer most "what is it doing?" questions without changing anything.
- Use `sys.remote_exec` with an **absolute** path to a script that **writes its result somewhere** and renames it into place. Wait for that result; never delete the script until you see it.
- Use `pdb -p` for interactive work, keep the session short (the main thread is paused), and leave with `quit`, which detaches.
- Match the interpreter: same `major.minor`, the exact version if pre-release. Attach to the **Python** PID, not a wrapper.
- In hardened production images where no one should ever attach, set `PYTHON_DISABLE_REMOTE_DEBUG=1`. Do not loosen `ptrace_scope` system-wide to make debugging easier. Use `setpriv --ptracer` on the one process, or `sudo` for a one-off.

Looking ahead: Python 3.15 (final due 2026-10-09) adds a sampling profiler that attaches the same way (`python -m profiling.sampling attach PID`, PEP 799).

Try this

1. Extend the probe in `dump_stacks.py` to also write `len(gc.get_objects())` and the top ten `tracemalloc` statistics (start tracing from a first probe, read from a second).
2. Run `worker.py` with `PYTHON_DISABLE_REMOTE_DEBUG=1` and confirm all three tools refuse. Then check whether `asyncio ps` refuses too.
3. In `aio_service.py`, make one `fetch` raise after a second. Watch how `pstree` changes as the `TaskGroup` cancels its siblings.
4. Attach `pdb -p` to `worker.py`, set a breakpoint with `b worker.py:26`, `continue`, and inspect `jobs_done` when it stops. Then `quit` and confirm the worker keeps running.
5. Start `blocked.py` with a 30-second sleep and compare `dump_stacks.py` (times out) with `python -m asyncio ps PID` (answers at once with an empty table, because there is no event loop). Explain why one waits and the other does not.

Sampling Profiler: Tachyon (profiling.sampling)

Sampling Profiler: Tachyon (`profiling.sampling`)

Thesis

`cProfile` answers "how many times was each function called, and for how long?" by hooking **every** call and return. That precision has a price: the program you measure is no longer the program you run. Python **3.15** ships a second, opposite kind of profiler in the standard library. **Tachyon** (`python -m profiling.sampling`, from PEP 799) never runs inside the target. It reads the target's interpreter stacks from **outside the process** at a fixed rate and counts what it sees. The program runs at full speed, so the numbers describe the real program. You lose exact call counts in exchange. Use Tachyon first to find **where time goes**, and use tracing only when you need exact counts.

Version note: every listing in this chapter was run on **CPython 3.15.0rc3** (Linux x86-64, installed with `uv python install 3.15.0rc3`). 3.15.0 final is scheduled for 2026-10-09. Where the rc3 tool differs from the documentation, the chapter says so. The book's baseline is 3.14, which has no `profiling` package. On 3.14 only `cProfile` exists.

Mental model

```
        your program (unchanged)                    Tachyon (separate process)

main() → parse() → LINE.match()     read   every 1 ms (default 1 kHz):
  frames live in interpreter memory  memory     copy the frame chain of each
  no hooks, no tracing, no pauses              sampled thread, record the stack
                              at the end: count stacks → estimate
                                               time = samples × interval
```

Question	`profiling.tracing` (`cProfile`)	`profiling.sampling` (Tachyon)
How does it measure?	A callback on every call/return, inside the process	Reads stacks from another process at a fixed rate
Exact call counts?	Yes	No. Sample counts only
Overhead on the target	Large for call-heavy code (8× below)	Not measurable on this box at 1 kHz

Question	`profiling.tracing` (`cProfile`)	`profiling.sampling` (Tachyon)
Attach to a running process?	No	Yes: `attach PID`
What can it tell apart?	Python functions	Wall vs CPU vs GIL-held vs exception time, threads, asyncio tasks
Good for short scripts (< 1 s)?	Yes	Not alone: too few samples

Three facts to hold on to:

- **Times are estimates.** `tottime` is `samples × interval`. A function seen in 1,000 of 3,000 samples at 1 kHz "took" about 1.0 s. Double the run time and you double the precision.
- **Rows are per line, not per function.** The pstats table key is `file:line(function)`, and the line is the one **executing** when the sample was taken. A function with three hot lines shows up as three rows. That is a feature: it tells you which line.
- **Main thread only, unless you ask.** By default Tachyon samples only the main thread. Add `-a` for every thread (see *The trap*).

Since 3.15 the old names are thin aliases. `cProfile` still works and is the same code as `profiling.tracing`. The pure-Python `profile` module is deprecated and will be removed in 3.17.

Worked examples

Case 1: Find the hotspot in a report job

Save as `report.py`. It is a small batch job. It builds a log, waits on an "upstream" (a 1-second sleep stands in for a network call), counts paths, and fingerprints the input. One of those steps is written badly.

```
# report.py
import hashlib
import re
import time

LINE = re.compile(r"(?P<ip>\d+\.\d+\.\d+\.\d+) .* \"(?P<verb>[A-Z]+) (?P<path>\S+)")


def make_log(n):
    return [f"10.0.{i % 256}.{i % 7} - - [08/Oct/2026] \"GET /api/item/{i % 500} HTTP/1.1\"
            for i in range(n)]
```

```
def parse(lines):
    hits = {}
    for line in lines:
        m = LINE.match(line)
        path = m.group("path")
        hits[path] = hits.get(path, 0) + 1
    return hits


def slow_top(hits, k):
    # O(n*k): rescans the dict once per slot
    top = []
    remaining = dict(hits)
    for _ in range(k):
        best = max(remaining, key=remaining.get)
        top.append((best, remaining.pop(best)))
    return top


def fingerprint(lines):
    h = hashlib.sha256()
    for _ in range(40):
        h.update("\n".join(lines).encode())
    return h.hexdigest()[:12]


def fetch_upstream():
    time.sleep(1.0)  # stands in for a network call


def main():
    lines = make_log(300_000)
    fetch_upstream()
    hits = parse(lines)
    top = slow_top(hits, 400)
    print("top:", top[0], "fingerprint:", fingerprint(lines))


if __name__ == "__main__":
    main()
```

Run it under Tachyon. `run` starts the script as a child process and samples it from the first line. There is no PID to look up.

```
uv run --python 3.15.0rc3 python -m profiling.sampling run report.py
```

Output (the default pstats table, sorted by direct samples, top 15):

```
top: ('/api/item/0', 600) fingerprint: 190018c62bb3
Captured 3213 samples in 3.21 seconds
Sample rate: 1000.00 samples/sec
Error rate: 0.47
Profile Stats:
       nsamples   sample%   tottime (s)    cumul%   cumtime (s)  filename:lineno(function)
      1793/1793      56.1         1.793      56.1         1.793  report.py:36(fingerprint)
      1000/1000      31.3         1.000      31.3         1.000  report.py:41(fetch_upstream
        171/171       5.3         0.171       5.3         0.171  report.py:17(parse)
        136/136       4.3         0.136       4.3         0.136  report.py:10(make_log)
          42/42       1.3         0.042       1.3         0.042  report.py:18(parse)
          32/32       1.0         0.032       1.0         0.032  report.py:19(parse)
            6/6       0.2         0.006       0.2         0.006  report.py:16(parse)
         5/3194       0.2         0.005      99.9         3.194  report.py:53(<module>)
            4/4       0.1         0.004       0.1         0.004  report.py:11(make_log)
            4/4       0.1         0.004       0.1         0.004  report.py:28(slow_top)
            1/1       0.0         0.001       0.0         0.001  <frozen importlib._bootstra
            1/1       0.0         0.001       0.0         0.001  <frozen importlib._bootstra
            1/1       0.0         0.001       0.0         0.001  _parser.py:848(_parse)
            1/1       0.0         0.001       0.0         0.001  report.py:27(slow_top)
            0/1       0.0         0.000       0.0         0.001  <frozen importlib._bootstra

Legend:
  nsamples: Direct/Cumulative samples (direct executing / on call stack)
  sample%: Percentage of total samples this function was directly executing
  tottime: Estimated total time spent directly in this function
  cumul%: Percentage of total samples when this function was on the call stack
  cumtime: Estimated cumulative time (including time in called functions)
  filename:lineno(function): Function location and name

Summary of Interesting Functions:

Functions with Highest Direct/Cumulative Ratio (Hot Spots):
  1.000 direct/cumulative ratio, 56.1% direct samples: report.py:(fingerprint)
  1.000 direct/cumulative ratio, 31.3% direct samples: report.py:(fetch_upstream)
  1.000 direct/cumulative ratio, 7.9% direct samples: report.py:(parse)

Functions with Highest Call Frequency (Indirect Calls):
  3189 indirect calls, 99.9% total stack presence: report.py:(<module>)
  1 indirect calls, 0.0% total stack presence: <frozen importlib._bootstrap_external>:(PathF

Functions with Highest Call Magnification (Cumulative/Direct):
  638.8x call magnification, 3189 indirect calls from 5 direct: report.py:(<module>)
```

How to read it:

- **`nsamples` is `direct/cumulative`.** `1793/1793` means line 36 was on top of the stack in 1,793 samples. `5/3194` for `<module>` means it was almost never on top but was on the stack the

whole time.

- **Line 36 is `h.update("\n".join(lines).encode())`.** More than half of the run is spent there. The table points at the exact line, not just "somewhere in `fingerprint`".
- **`fetch_upstream` is 1,000 samples, exactly the 1.0 s sleep.** The default mode is **wall-clock**, so waiting counts.
- **`slow_top` is nearly invisible** (8 samples). The obvious O(n · k) loop is not the problem with 500 distinct paths. This is why you measure before you rewrite.
- **`Error rate` is the percentage of sample attempts that failed** (a stack read that raced with the interpreter changing it). Under 1% is normal.

The rc3 build prints the *Legend* even with `--no-summary`. The documentation says that flag hides both. Later listings drop the legend for space.

Case 2: Wall, CPU, and GIL: three questions, three modes

Wall time says where the job **waits** as well as where it works. To see where it **burns CPU**, switch modes. CPU mode keeps only the samples where the thread was on a CPU:

```
uv run --python 3.15.0rc3 python -m profiling.sampling run --mode=cpu --no-summary -l 6 repo
```

```
top: ('/api/item/0', 600) fingerprint: 190018c62bb3
Captured 2968 samples in 2.97 seconds
Sample rate: 1000.00 samples/sec
Error rate: 0.40
Profile Stats:
       nsamples   sample%   tottime (s)    cumul%   cumtime (s)  filename:lineno(function)
      1707/1707      87.3         1.707      87.3         1.707  report.py:36(fingerprint)
        106/106       5.4         0.106       5.4         0.106  report.py:17(parse)
          84/84       4.3         0.084       4.3         0.084  report.py:10(make_log)
          23/23       1.2         0.023       1.2         0.023  report.py:19(parse)
          19/19       1.0         0.019       1.0         0.019  report.py:18(parse)
         7/1954       0.4         0.007      99.9         1.954  report.py:53(<module>)
```

`fetch_upstream` is gone: a sleeping thread is not on a CPU. If a function is high in wall mode and absent in CPU mode, it is **waiting**. Making its code faster will not help; reducing the wait (concurrency, caching, a faster upstream) will.

The fix for line 36 is to build the joined bytes once instead of 40 times. Save as `report_fixed.py`. It is `report.py` with one changed function:

```
# report_fixed.py (only fingerprint() differs from report.py)
def fingerprint(lines):
    h = hashlib.sha256()
    blob = "\n".join(lines).encode()   # build the bytes once
    for _ in range(40):
        h.update(blob)
    return h.hexdigest()[:12]
```

```
cp report.py report_fixed.py   # then edit fingerprint() as above
time uv run --python 3.15.0rc3 python report_fixed.py
```

```
top: ('/api/item/0', 600) fingerprint: 190018c62bb3

real    0m1.869s
user    0m0.802s
sys 0m0.068s
```

The fingerprint is identical and the job went from about 3.1 s to 1.9 s wall time, of which 1.0 s is the sleep. Now compare **CPU** mode with **GIL** mode on the fixed version. GIL mode keeps only samples where the thread **held the GIL**, that is, was running Python bytecode or C code that did not release the lock:

```
uv run --python 3.15.0rc3 python -m profiling.sampling run --mode=cpu -l 5 --no-summary repo
```

```
top: ('/api/item/0', 600) fingerprint: 190018c62bb3
Captured 1781 samples in 1.78 seconds
Sample rate: 1000.00 samples/sec
Error rate: 0.67
Profile Stats:
       nsamples   sample%  tottime (ms)    cumul%  cumtime (ms)  filename:lineno(function)
        487/487      63.3       487.000      63.3       487.000  report_fixed.py:37(fingerpr
        120/120      15.6       120.000      15.6       120.000  report_fixed.py:17(parse)
          81/81      10.5        81.000      10.5        81.000  report_fixed.py:10(make_log
          31/31       4.0        31.000       4.0        31.000  report_fixed.py:35(fingerpr
          22/22       2.9        22.000       2.9        22.000  report_fixed.py:18(parse)
```

```
uv run --python 3.15.0rc3 python -m profiling.sampling run --mode=gil -l 5 --no-summary repo
```

```
top: ('/api/item/0', 600) fingerprint: 190018c62bb3
Captured 1835 samples in 1.84 seconds
Sample rate: 1000.00 samples/sec
Error rate: 0.65
Profile Stats:
       nsamples   sample%  tottime (ms)    cumul%  cumtime (ms)  filename:lineno(function)
        110/110      35.8       110.000      35.8       110.000  report_fixed.py:17(parse)
          85/85      27.7        85.000      27.7        85.000  report_fixed.py:10(make_log
          46/46      15.0        46.000      15.0        46.000  report_fixed.py:35(fingerpr
          24/24       7.8        24.000       7.8        24.000  report_fixed.py:19(parse)
          20/20       6.5        20.000       6.5        20.000  report_fixed.py:18(parse)
```

Line 37 (`h.update(blob)`) is 63% of CPU time and **absent** from GIL mode. `hashlib` releases the GIL while it hashes large buffers, so that CPU time is available to other threads. Line 35 (`"\n".join`) stays in GIL mode: building a string holds the lock. In a threaded server, that is

the line that starves other threads. GIL mode is the profile to read when "more threads made it slower".

The other mode is `--mode=exception`. It keeps samples taken while an exception is propagating or an `except` block is running, which is useful when a parser uses exceptions for control flow.

Case 3: Same data, many views: flame graphs, collapsed stacks, replay

Every format is a flag on the same command. `--flamegraph` and `--heatmap` write self-contained HTML; `--collapsed` writes one line per unique stack for external flame-graph tools; `--gecko` writes JSON for the Firefox Profiler. Rather than profiling once per format, record once in the compact binary format and **replay** it:

```
uv run --python 3.15.0rc3 python -m profiling.sampling run --binary -o baseline.bin report.p
```

```
top: ('/api/item/0', 600) fingerprint: 190018c62bb3
Captured 2942 samples in 2.94 seconds
Sample rate: 1000.00 samples/sec
Error rate: 0.44
  Binary Encoding:
    Records:          437
      RLE repeat:     164 (37.5%) [2,649 samples]
      Full stack:     1 (0.2%)
      Suffix match:   0 (0.0%)
      Pop-push:       272 (62.2%)
  Frame Efficiency:
    Frames written:   296
    Frames saved:     23,091 (98.7%)
    Bytes (pre-zstd): 15.1 KB
Binary profile written to baseline.bin (2929 samples)
```

A three-second profile is 15 KB before compression. That is small enough to attach to a ticket. Replay it into a flame graph and into collapsed stacks:

```
uv run --python 3.15.0rc3 python -m profiling.sampling replay --flamegraph -o baseline.html
uv run --python 3.15.0rc3 python -m profiling.sampling replay --collapsed -o baseline.collap
awk '{print $NF"\t"$0}' baseline.collapsed | sort -nr | cut -f2 | head -3
```

```
Flamegraph data: 1 root function, 2928 total samples, 94 unique strings
Flamegraph saved to: baseline.html
Replayed 2929 samples
Collapsed stack output written to baseline.collapsed
Replayed 2929 samples
tid:429263;<frozen runpy>:_run_module_as_main:201;<frozen runpy>:_run_code:87;report.py:<mod
tid:429263;<frozen runpy>:_run_module_as_main:201;<frozen runpy>:_run_code:87;report.py:<mod
tid:429263;<frozen runpy>:_run_module_as_main:201;<frozen runpy>:_run_code:87;report.py:<mod
```

Each collapsed line is a full stack from the thread ID down to the executing line, followed by its sample count. The `awk` sorts by that count, which is the last field. Frames like `<frozen runpy>` contain spaces, so a plain `sort -k2` would pick the wrong column. This text format is what external tools (`flamegraph.pl`, speedscope) expect, and it is easy to `grep` and diff in a terminal.

The best view for "did my fix work?" is a **differential flame graph**. Profile the new code against the saved baseline:

```
uv run --python 3.15.0rc3 python -m profiling.sampling run --diff-flamegraph baseline.bin -o
```

```
top: ('/api/item/0', 600) fingerprint: 190018c62bb3
Captured 1839 samples in 1.84 seconds
Sample rate: 1000.00 samples/sec
Error rate: 0.71
Flamegraph data: 1 root function, 1825 total samples, 119 unique strings
Flamegraph saved to: diff.html
```

Open `diff.html` in a browser. Frames are drawn at the new widths and coloured by change in **direct** time: blue for faster, red for slower, purple for frames that are new. Here `fingerprint` turns blue and nothing turns red. That is the evidence to put in a pull request.

`--heatmap` is the line-level view. It writes a directory, not a file:

```
uv run --python 3.15.0rc3 python -m profiling.sampling run --heatmap -o heat report_fixed.py
```

```
Heatmap output written to heat/
  - Index: heat/index.html
  - 1 source file analyzed
```

Case 4: What it costs the program you measure

Save as `calls.py`. Recursive Fibonacci is the worst case for a tracing profiler: millions of tiny calls, each one triggering the profiler's hook.

```
# calls.py
import time


def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)


start = time.perf_counter()
fib(32)
print(f"fib(32) took {time.perf_counter() - start:.2f}s")
```

The script times itself, so each line below is the program's own view of its speed under each profiler:

```
uv run --python 3.15.0rc3 python calls.py
uv run --python 3.15.0rc3 python -m profiling.sampling run calls.py | head -1
uv run --python 3.15.0rc3 python -m profiling.sampling run -r 20khz calls.py | head -1
uv run --python 3.15.0rc3 python -m profiling.tracing calls.py | head -1
```

Three runs on an 8-vCPU VM shared with other work:

Run	No profiler	Tachyon 1 kHz	Tachyon 20 kHz	`profiling.tracing`
1	0.44 s	0.37 s	0.51 s	1.76 s
2	0.22 s	0.20 s	0.24 s	1.74 s
3	0.23 s	0.21 s	0.20 s	2.61 s

The sampled runs are within the box's own noise (run 1 was a noisy moment for everything). The tracing profiler makes this program **8–11× slower**. It also distorts the profile: it inflates cheap, frequently called functions far more than slow, rarely called ones. With a sampler the cost lands on the profiler's own process, not on the target.

Case 5: Profiling a live process (attach and dump)

`run` covers anything you can start yourself. For a process that is already running, Tachyon has two more subcommands. They use the same OS permission check as `pdb -p` and `sys.remote_exec`:

```
# sample a running service for 30 s, all threads, into a flame graph
python -m profiling.sampling attach -a -d 30 --flamegraph -o live.html PID

# one traceback-style snapshot of every thread ("what is it doing right now?")
python -m profiling.sampling dump -a PID

# a top-like live view (q to quit)
python -m profiling.sampling attach --live PID
```

These were **not run** for this chapter. The examples above stick to `run`, which samples its own child and needs no extra privilege. Things to know before you attach in production:

- **Permissions.** On Linux, reading another process's memory needs root, `CAP_SYS_PTRACE`, or a Yama `ptrace_scope` that allows it. Prefer a one-off `sudo` over loosening `ptrace_scope` for the whole host.
- **Same interpreter.** Profiler and target must be the same `major.minor`. If either is a pre-release (like the rc3 used here), they must be the **exact** same version. Free-threaded and default builds cannot profile each other.
- **The right PID.** `uv run`, shell scripts, and process supervisors put a wrapper in front of Python. Attach to the interpreter, not the wrapper:

```
# server.py
import os
import time
```

```
print("python pid", os.getpid(), flush=True)
time.sleep(30)
```

```
uv run --python 3.15.0rc3 python server.py &
ps -o pid,ppid,comm --ppid $! -p $!
```

```
python pid 462630
    PID    PPID COMMAND
 462624  462604 uv
 462630  462624 python3.15
```

`$!` is the `uv` process (462624). The interpreter is its child (462630), the PID the script printed, and that is the PID to give `attach` or `dump`.

- **`dump --blocking`** pauses the target while it reads, for a guaranteed-consistent snapshot. Without it the read races the running program and can, rarely, produce a torn stack.

The trap

The default profile only sees the main thread. Save as `pool.py`. The real work happens in two worker threads:

```
# pool.py
from concurrent.futures import ThreadPoolExecutor


def checksum(n):
    total = 0
    for i in range(n):
        total = (total * 31 + i) % 1_000_003
    return total


def main():
    with ThreadPoolExecutor(max_workers=2) as ex:
        results = list(ex.map(checksum, [8_000_000] * 4))
    print("results:", results[:2])


if __name__ == "__main__":
    main()
```

```
uv run --python 3.15.0rc3 python -m profiling.sampling run -l 4 --no-summary pool.py
```

```
results: [587857, 587857]
Captured 1805 samples in 1.81 seconds
Sample rate: 1000.00 samples/sec
Error rate: 1.61
Profile Stats:
       nsamples   sample%   tottime (s)    cumul%   cumtime (s)  filename:lineno(function)
      1740/1740      98.0         1.740      98.0         1.740  threading.py:363(Condition.
          14/14       0.8         0.014       0.8         0.014  ~:0(<GC>)
            9/9       0.5         0.009       0.5         0.009  <frozen importlib._bootstra
            4/4       0.2         0.004       0.2         0.004  <frozen importlib._bootstra
```

98% "in `Condition.wait`". That is true, and useless: the main thread is waiting for results. `checksum` does not appear at all. Add `-a`:

```
uv run --python 3.15.0rc3 python -m profiling.sampling run -a -l 4 --no-summary pool.py
```

```
results: [587857, 587857]
Captured 2163 samples in 2.16 seconds
Sample rate: 1000.00 samples/sec
Error rate: 1.11
Profile Stats:
       nsamples   sample%   tottime (s)    cumul%   cumtime (s)  filename:lineno(function)
      3904/3904      61.4         3.904      61.4         3.904  pool.py:8(checksum)
      2090/2090      32.9         2.090      32.9         2.090  threading.py:363(Condition.
        293/293       4.6         0.293       4.6         0.293  pool.py:7(checksum)
          24/24       0.4         0.024       0.4         0.024  threading.py:372(Condition.
```

Now the hot loop (line 8) leads. Note what changed in the arithmetic: with `-a` each "sample" captures every thread, so the per-row `tottime` values add up to **thread-seconds** (3.9 s of `checksum` in a 2.2 s run, because two threads ran it). Percentages are shares of all thread samples, not of the wall clock.

Two smaller traps:

- **Very short programs.** A 0.2 s script yields about 200 samples at 1 kHz. Run it in a loop, raise the rate (`-r 10khz`), or use `profiling.tracing` for the exact counts.
- **`--async-aware` is its own world.** It rebuilds stacks along `await` chains and cannot be combined with `-a`, `--native`, `--no-gc`, or the `cpu`/`gil` modes.

The boring rule

- Start every performance question with `python -m profiling.sampling run --mode=wall`. Switch to `--mode=cpu` once you know the time is computation, and to `--mode=gil` when threads are involved.
- Always pass `-a` for programs that use threads or thread pools. Read `-a` totals as thread-seconds.

- Record once (`--binary -o run.bin`), then replay into whatever view you need. Keep the baseline `.bin` and prove fixes with `--diff-flamegraph`.
- Use `profiling.tracing` (`cProfile`) only when you need exact call counts, and never trust its absolute times for call-heavy code.
- In production: `attach -d 30` with a short duration, a one-off `sudo` for permission, the interpreter's PID (not a wrapper's), and the same Python version as the target.

Try this

1. Profile `report.py` at `-r 100hz`, `1khz`, and `20khz`. How many samples does each collect, and how much does the `fingerprint` percentage move between runs at each rate?
2. Run `pool.py` with `--mode=gil -a`. Each worker holds the GIL only part of the time. Then run `pool.py` on a free-threaded 3.15 build (`uv python install 3.15.0rc3+freethreaded`) and compare the wall time.
3. Add a `try: int(x) except ValueError: pass` loop over mostly non-numeric strings to `report.py` and find it with `--mode=exception`.
4. Generate `--gecko -o report.json` and load it into the Firefox Profiler. Find the GIL and CPU markers on the timeline.
5. Start `report.py` with a longer sleep, find the interpreter PID behind `uv run`, and run `dump -a PID` (as yourself, then with `sudo` if your system refuses). Compare the snapshot with the `run` profile.

Sources

- `profiling.sampling` reference (3.15): https://docs.python.org/3.15/library/profiling.sampling.html
- What's New in Python 3.15, "PEP 799" and "Tachyon": https://docs.python.org/3.15/whatsnew/3.15.html
- PEP 799, A dedicated `profiling` package: https://peps.python.org/pep-0799/
- `profiling.tracing` / `cProfile` reference: https://docs.python.org/3.15/library/profiling.tracing.html

Frame Pointers by Default (PEP 831)

Frame Pointers by Default (PEP 831)

Thesis

A native profiler such as `perf`, or an eBPF tool, sees a Python process as a pile of C functions. To say *who called whom*, it has to walk the machine stack. The cheapest way to walk it is the **frame-pointer chain**: every function saves the caller's `%rbp` and points `%rbp` at its own frame, so the walk is "read, follow, repeat". Compilers drop that chain by default at `-O1` and above. For years CPython did the same, so native stacks through Python stopped at `_PyEval_EvalFrameDefault`.

Starting with **Python 3.15**, PEP 831 (status *Final*) builds CPython with `-fno-omit-frame-pointer -mno-omit-leaf-frame-pointer`. It also writes those flags into `sysconfig`, so **C extensions built against 3.15 inherit them**. The chain only helps if it has no gaps. This chapter shows how to check that a given interpreter and a given extension actually keep it.

Version note: the listings were run on **CPython 3.15.0rc3** from `uv` (python-build-standalone, Linux x86-64, GCC 14.2 for the extension builds). 3.15.0 final shipped on 2026-10-09. Its changelog lists *no changes* since rc3, so the output is the same. The comparison interpreter is Debian's system **3.13.5**.

Mental model

```
stack (grows down)                what an unwinder does with frame pointers

main                            1. start at %rbp of the sampled thread
  saved %rbp → bottom             2. [%rbp+8] is the return address → caller
                     3. [%rbp]    is the caller's %rbp   → repeat
_PyEval_EvalFrameDefault
  saved %rbp              one missing link (a function that never
                       did `push %rbp; mov %rsp,%rbp`) and the
your_extension_func        %rbp  walk skips or stops at that frame
```

Piece of the process	Who decides its flags	Covered by PEP 831?
`python`, `libpython`, stdlib extension modules	CPython `configure` (`BASECFLAGS`)	Yes, on by default; `--without-frame-pointers` opts out
C extensions built with setuptools / `pip` against 3.15	`sysconfig CFLAGS`	Yes, inherited

Piece of the process	Who decides its flags	Covered by PEP 831?
Rust (PyO3/maturin), C++ with CMake, vendored native libs	their own build systems	**No**. The PEP only *recommends* the flags
Python before 3.15	whoever built it	Depends on the builder

Three facts to hold on to:

- **Overhead is small and measured.** The PEP reports a 0.1–2.3 % geometric-mean slowdown across eight machines on pyperformance. It comes from about 6,000 small C helpers gaining a two-instruction prologue, not from the eval loop.
- **The flags travel through `sysconfig`.** That is why they live in `BASECFLAGS` and not `CFLAGS_NODIST`: a chain with a gap at the extension boundary is as broken as no chain at all.
- **Nothing changes for Python code.** No API, no behaviour, no ABI change. Only native tooling sees the difference.

Worked examples

Case 1: ask the interpreter what it was built with

Save as `fp_audit.py`. The first half reads `sysconfig`. The second half disassembles a shared object and counts functions whose prologue links `%rbp`.

```
# fp_audit.py - does this interpreter (and a given .so) keep the frame-pointer chain?
import re
import subprocess
import sys
import sysconfig

FLAGS = ("-fno-omit-frame-pointer", "-mno-omit-leaf-frame-pointer")


def interpreter_report() -> None:
    cflags = sysconfig.get_config_var("CFLAGS") or ""
    print(f"python   {sys.version.split()[0]}  ({sys.executable})")
    for flag in FLAGS:
        print(f"  {flag:32} {'yes' if flag in cflags.split() else 'NO'}")


def prologues(path: str) -> tuple[int, int, list[str]]:
    """Count text functions whose first instructions set up %rbp as frame pointer."""
    asm = subprocess.run(
        ["objdump", "-d", "--no-show-raw-insn", path],
        capture_output=True, text=True, check=True,
```

```
    ).stdout
    funcs = re.split(r"\n(?=[0-9a-f]+ <)", asm)
    total, with_fp, missing = 0, 0, []
    for block in funcs:
        m = re.match(r"[0-9a-f]+ <([^>]+)>:", block)
        if not m or m.group(1).startswith((".", "_init", "_fini", "deregister", "register_tm
            continue
        if "@" in m.group(1):  # PLT stubs are not our code
            continue
        insns = [l.split("\t")[-1].strip() for l in block.splitlines()[1:8]]
        total += 1
        # the compiler may schedule other instructions between the two
        pushed = any(re.fullmatch(r"push\s+%rbp", i) for i in insns[:4])
        linked = any(re.fullmatch(r"mov\s+%rsp,%rbp", i) for i in insns)
        if pushed and linked:
            with_fp += 1
        else:
            missing.append(m.group(1))
    return total, with_fp, missing


if __name__ == "__main__":
    interpreter_report()
    for so in sys.argv[1:]:
        total, ok, missing = prologues(so)
        print(f"{so.rsplit('/', 1)[-1]}: {ok}/{total} functions keep %rbp")
        print(f"  without: {missing[:4] or '-'}{' ...' if len(missing) > 4 else ''}")
```

```
$ python3.15 fp_audit.py
python   3.15.0rc3  (/home/box/.local/bin/python3.15)
  -fno-omit-frame-pointer          yes
  -mno-omit-leaf-frame-pointer     yes

$ /usr/bin/python3 fp_audit.py
python   3.13.5  (/usr/bin/python3)
  -fno-omit-frame-pointer          NO
  -mno-omit-leaf-frame-pointer     NO
```

On this box `sysconfig.get_config_var("CFLAGS")` for 3.15 actually lists each flag **twice**. Upstream's PEP 831 default adds one copy, and python-build-standalone, which had already turned frame pointers on for its Linux builds in early 2026, adds the other. Duplicates are harmless. The uv-installed **3.14.8** on the same box also reports both flags, because of python-build-standalone and not because of CPython. The Debian 3.13 build reports neither. The flags depend on **who built the interpreter**, not only on its version number.

Case 2: audit the interpreter's own code

```
$ python3.15 fp_audit.py ~/.local/share/uv/python/cpython-3.15.0rc3-linux-x86_64-gnu/lib/lib
python   3.15.0rc3  (/home/box/.local/bin/python3.15)
  -fno-omit-frame-pointer          yes
  -mno-omit-leaf-frame-pointer     yes
libpython3.15.so.1.0: 30967/32952 functions keep %rbp
  without: ['_PyPegen_get_expr_name', '_PyPegen_nonparen_genexp_in_call', '_get_interpolatio

$ python3.15 fp_audit.py /usr/bin/python3.13
...
python3.13: 0/0 functions keep %rbp
  without: -
```

About 94 % of `libpython`'s functions set up a frame. Most of the rest are functions that never touch the stack, or that end in a tail jump (see Case 4). The Debian binary gives `0/0` because it is **stripped**: `objdump` has no symbols to split on. An empty audit means "no evidence either way", not "fine".

Case 3: an extension inherits the flags, and can opt out

Save the extension as `fpcheck.c`. `leaf_sum` is marked `noinline` so that it survives as its own function.

```c
/* fpcheck.c - a tiny extension with a leaf and a non-leaf function */
#define PY_SSIZE_T_CLEAN
#include <Python.h>

__attribute__((noinline)) static long leaf_sum(long n) {            /* leaf: calls nothing *
    long s = 0;
    for (long i = 0; i < n; i++) s += i;
    return s;
}

static PyObject *total(PyObject *self, PyObject *arg) {
    long n = PyLong_AsLong(arg);
    if (n == -1 && PyErr_Occurred()) return NULL;
    return PyLong_FromLong(leaf_sum(n));
}

static PyMethodDef methods[] = {
    {"total", total, METH_O, "sum of range(n)"},
    {NULL, NULL, 0, NULL},
};

static struct PyModuleDef mod = {PyModuleDef_HEAD_INIT, "fpcheck", NULL, -1, methods};
```

```
PyMODINIT_FUNC PyInit_fpcheck(void) { return PyModule_Create(&mod); }
```

```
# setup.py - build fpcheck with whatever flags this interpreter's sysconfig reports
import os
from setuptools import Extension, setup

extra = os.environ.get("FP_EXTRA", "").split()
setup(
    name="fpcheck",
    version="0",
    ext_modules=[Extension("fpcheck", ["fpcheck.c"], extra_compile_args=extra)],
)
```

Build it in a 3.15 virtual environment and look at the compiler line setuptools runs:

```
$ uv venv -p 3.15 .v315 && uv pip install -p .v315 setuptools
$ .v315/bin/python setup.py build_ext --inplace 2>&1 | grep '^cc' | head -1
cc -pthread -fno-omit-frame-pointer -mno-omit-leaf-frame-pointer -fno-strict-overflow ... -O
$ .v315/bin/python -c "import fpcheck; print(fpcheck.total(10))"
45
$ .v315/bin/python fp_audit.py fpcheck.cpython-315-x86_64-linux-gnu.so
python   3.15.0rc3  (/workspace/scratch/fp/.v315/bin/python)
  -fno-omit-frame-pointer          yes
  -mno-omit-leaf-frame-pointer     yes
fpcheck.cpython-315-x86_64-linux-gnu.so: 1/3 functions keep %rbp
  without: ['leaf_sum', 'PyInit_fpcheck']
```

The extension author did nothing, yet `total` (the function that calls back into CPython) got a frame. That is the propagation PEP 831 is about. To opt one extension out, add the opposite flag. On GCC and Clang the last flag on the command line wins:

```
$ rm -rf build *.so
$ FP_EXTRA=-fomit-frame-pointer .v315/bin/python setup.py -q build_ext --inplace
$ .v315/bin/python fp_audit.py fpcheck.cpython-315-x86_64-linux-gnu.so
...
fpcheck.cpython-315-x86_64-linux-gnu.so: 0/3 functions keep %rbp
  without: ['leaf_sum', 'total', 'PyInit_fpcheck']
```

Case 4: read the “without” list before panicking

Two names stayed in the list even with the flags on. Disassemble them:

```
$ objdump -d --no-show-raw-insn fpcheck.cpython-315-x86_64-linux-gnu.so | grep -A3 -E '<(lea
0000000000001140 <leaf_sum>:
    1140:   test   %rdi,%rdi
```

```
    1143:   jle    1178 <leaf_sum+0x38>
    1145:   xor    %eax,%eax
--
00000000000011d0 <PyInit_fpcheck>:
    11d0:   mov    $0x3f5,%esi
    11d5:   lea    0x2e44(%rip),%rdi        # 4020 <mod>
    11dc:   jmp    1050 <PyModule_Create2@plt>
```

- `PyInit_fpcheck` is a **tail call**. It sets up arguments and *jumps* to `PyModule_Create2`, which returns straight to `PyInit`'s caller. There is no frame of its own to lose, so the chain is intact.
- `leaf_sum` is a leaf that uses no stack at all. GCC 14.2 on this box emits no prologue for it even with `-mno-omit-leaf-frame-pointer`. The same happens in a plain C file compiled with `gcc -O2 -fno-omit-frame-pointer -mno-omit-leaf-frame-pointer`. A sample that lands *inside* `leaf_sum` still has the right `%rbp` (the caller's), so the walk loses at most the immediate caller for that one sample. That is a compiler detail. Check your own compiler before you write it off.

Look at what the missing functions actually are. Fix the ones that call other functions without a frame, which `total` did in Case 3 once it was opted out.

The trap

"We're on 3.15, so our flame graphs are complete." Not necessarily. PEP 831 covers the interpreter and what builds through `sysconfig`. A typical service also loads:

- a **Rust extension** built by maturin, which uses Rust's own codegen defaults. Add `-C force-frame-pointers=yes` through `RUSTFLAGS`.
- a **C++ library** with its own CMake. It needs `-fno-omit-frame-pointer -mno-omit-leaf-frame-pointer` in its own `CMAKE_C(XX)_FLAGS`.
- a **vendored `.so`** from a wheel built years ago, against an older Python, by someone else.

One frameless function that calls others in the hot path, and every stack through it gets cut. Since the native unwinder in `bpf_get_stackid()` has no DWARF fallback, eBPF profilers show these as truncated one- or two-frame stacks. They don't error. Audit every native component on the hot path, not only `python`.

The mirror trap is **assuming an older Python lacks frame pointers**. Here, `uv`'s 3.14.8 already had them and Debian's 3.13 didn't. Check `sysconfig`. Don't infer from the version.

The boring rule

> On 3.15+, leave frame pointers on. Before trusting a native profile, run `sysconfig` plus a prologue audit over every `.so` on the hot path, and turn the flags on in each build system CPython does not control. Opt out per extension with `-fomit-frame-pointer` only after you have measured a regression, and write the measurement down.

Try this

1. Run `fp_audit.py` on the `.so` files inside one of your project's virtual environments (`find .venv -name '*.so'`). Which packages have no frames, and which are stripped (`0/0`)?
2. Build `fpcheck` against 3.14 from a non-`uv` source (a distro package or `pyenv`) and compare the compiler line with Case 3.
3. Remove `noinline` from `leaf_sum`, rebuild, and explain why the function disappears from the audit.
4. If your machine allows `perf` (`perf_event_paranoid  1`), record `python3.15 -X perf -c '...'` with `--call-graph fp`, then with `--call-graph dwarf`, and compare the size of `perf.data`.

Sources

- PEP 831: Frame Pointers Everywhere (Final, Python 3.15): build flags, `BASECFLAGS` versus `CFLAGS_NODIST`, `--without-frame-pointers`, performance table.
- Python 3.15.0 final release notes and changelog (3.15.0: "no new changes" since rc3).
- python-build-standalone issue #992: frame pointers in `uv`-distributed Linux builds.
- Python perf profiling HOWTO: `-X perf`, `-X perf_jit`.
- GCC optimize options: `-fomit-frame-pointer`.

sys.monitoring: Low-Overhead Debuggers and Coverage (PEP 669)

`sys.monitoring`: Low-Overhead Debuggers and Coverage (PEP 669)

Thesis

For twenty years every Python debugger and coverage tool was built on `sys.settrace`. You install one function, and the interpreter calls it on every function call and every line in every frame it traces. It can never stop calling that function at a single place, so a breakpoint inside a hot loop slows the entire program down. Python 3.12 added a second mechanism, `sys.monitoring` (PEP 669). A tool picks the **events** it wants and registers a **callback** for each. The interpreter instruments only those places in the bytecode. Most important, a callback can return `sys.monitoring.DISABLE` to say "never call me at this location again." That one return value is where nearly all the speed comes from. A line-coverage tool that disables each line after the first hit runs a hot loop at roughly native speed. The same tool without `DISABLE` is about as slow as `settrace`.

It is not an experimental corner. On 3.14 and 3.15, `breakpoint()` and `python -m pdb` run on it, and so does coverage.py's default measurement core. IDE debuggers are moving to it as well. This chapter builds the same three tools in miniature (a call counter, a line-and-branch coverage tool, and a breakpoint) and measures what each design decision costs.

Version note: every listing was run on **CPython 3.15.0** (released 2026-10-09) and, where the output differs, **CPython 3.14.8**, both from `uv` (python-build-standalone, Linux x86-64). The box was shared with other jobs, so the timings carry about ±20% noise. Read the ratios, not the milliseconds. Coverage.py is **7.16.2**, run through `uvx` without installing it into any project.

Mental model

- **`sys.monitoring` is a namespace, not a module.** Write `import sys` and use `sys.monitoring`. `import sys.monitoring` fails.
- **A tool is an integer slot, 0 to 5.** Claim one with `use_tool_id(id, name)` and release it with `free_tool_id(id)`. Conventions: `DEBUGGER_ID = 0`, `COVERAGE_ID = 1`, `PROFILER_ID = 2`, `OPTIMIZER_ID = 5`. CPython keeps two more internal slots, 6 and 7, for `sys.setprofile` and `sys.settrace` (`PY_MONITORING_SYS_PROFILE_ID` / `PY_MONITORING_SYS_TRACE_ID` in `Include/internal/pycore_instruments.h`). That is why the old API and the new one can run side by side.
- **Events are bit flags.** OR them together and pass the result to `set_events(tool, mask)` for the whole interpreter, or to `set_local_events(tool, code, mask)` for one code object. Nothing is on by default.

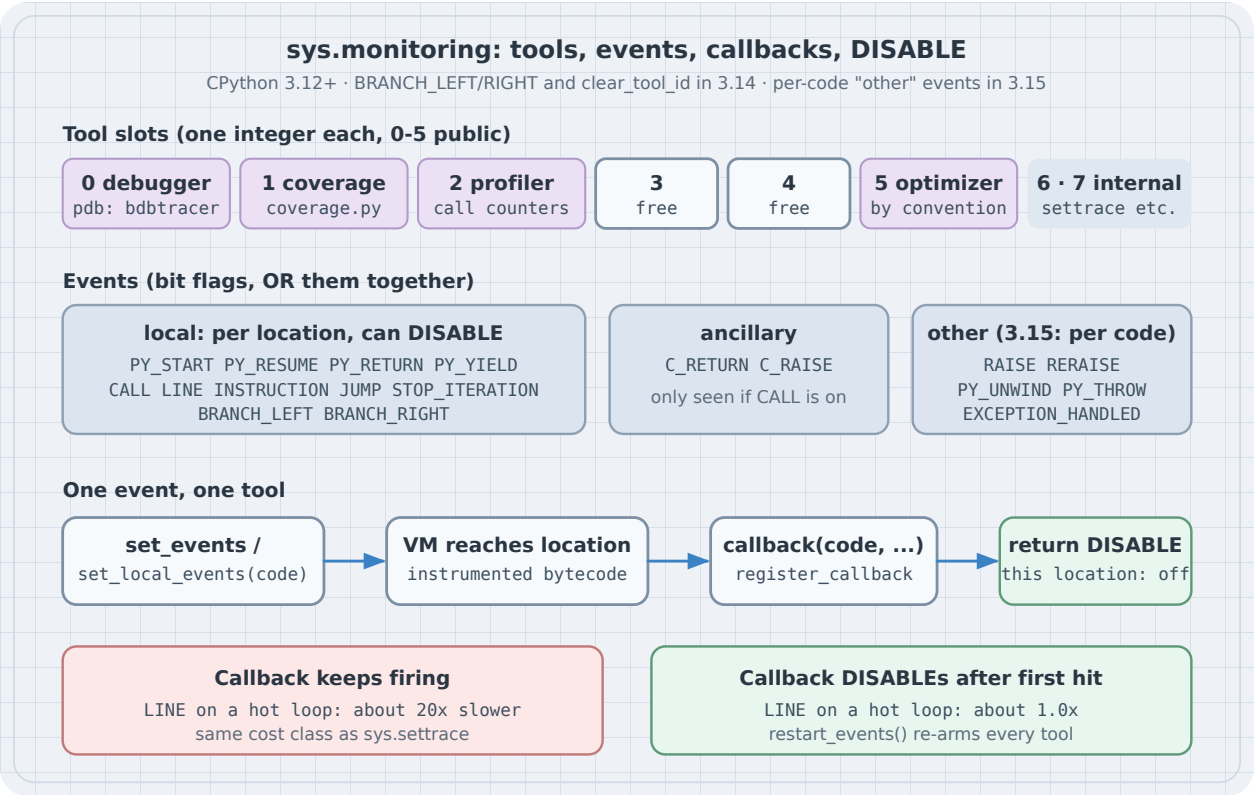


Figure 1: Tool slots, event kinds, the callback path, and what DISABLE buys

- **Callbacks get the code object, not a frame.** `LINE` callbacks receive `(code, line_number)`. Branch and jump callbacks receive `(code, instruction_offset, destination_offset)`. If you need locals, `sys._getframe(1)` inside the callback is the frame being monitored.
- **`DISABLE` is per location, per tool.** Returning it from a *local* event switches off that event at that one instruction for your tool only. `restart_events()` turns every disabled location back on, **for all tools**.
- **What changed recently.** 3.14 added `BRANCH_LEFT` / `BRANCH_RIGHT` (replacing the now-deprecated `BRANCH`) and `clear_tool_id()`. Also in 3.14, `free_tool_id()` began clearing a tool's events and callbacks; before that it left them registered. 3.15 lets the "other" events (`RAISE`, `RERAISE`, `PY_UNWIND`, `PY_THROW`, `EXCEPTION_HANDLED`) be turned on per code object and accept `DISABLE` (gh-146182). Case 7 shows exactly what that does.

Worked examples

Case 1: The smallest tool: count function starts

```
# hello_monitor.py
import sys
from collections import Counter

mon = sys.monitoring
TOOL = mon.PROFILER_ID          # 2: the slot reserved by convention for profilers
starts = Counter()


def on_start(code, instruction_offset):
    starts[code.co_qualname] += 1


def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)


def main():
    return [fib(n) for n in range(10)]


mon.use_tool_id(TOOL, "call-counter")
mon.register_callback(TOOL, mon.events.PY_START, on_start)
mon.set_events(TOOL, mon.events.PY_START)
try:
    result = main()
finally:
    mon.set_events(TOOL, mon.events.NO_EVENTS)
    mon.free_tool_id(TOOL)

print(result)
for name, n in starts.most_common():
    print(f"{n:5d}  {name}")
print("tool 2 after free:", mon.get_tool(TOOL))
```

```
python3.15 hello_monitor.py
```

Output:

```
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
  276  fib
    1  main
```

```
tool 2 after free: None
```

That is the whole lifecycle: claim a slot, register a callback, turn events on, run the code, turn events off, and free the slot. The `try/finally` matters, because a tool that dies without freeing its slot blocks the next tool that wants it (see The trap). There is no `<listcomp>` entry: since 3.12 a list comprehension runs inline in its function (PEP 709), so it never starts a code object of its own.

Case 2: Where the speed actually comes from

One CPU-bound loop, timed four ways (best of five runs): no tool; a classic `sys.settrace` line tracer; a `sys.monitoring` `LINE` callback that keeps firing; and the same callback returning `DISABLE` after it records a line.

```
# bench_tracers.py
import sys
import time

mon = sys.monitoring
TOOL = mon.COVERAGE_ID          # 1


def workload():
    total = 0
    for i in range(1_000_000):
        if i % 3:
            total += i
        else:
            total -= 1
    return total


def run(label, setup, teardown, baseline=None):
    best = float("inf")
    for _ in range(5):
        setup()
        t0 = time.perf_counter()
        workload()
        best = min(best, time.perf_counter() - t0)
        teardown()
    extra = f"  {best / baseline:5.1f}x" if baseline else ""
    print(f"{label:<34} {best * 1000:7.1f} ms{extra}")
    return best


seen = set()
```

```
def settrace_line(frame, event, arg):            # what coverage tools did before 3.12
    if event == "line":
        seen.add((frame.f_code, frame.f_lineno))
    return settrace_line


def mon_line(code, line):                        # sys.monitoring, keep firing
    seen.add((code, line))


def mon_line_once(code, line):                   # sys.monitoring, fire once per line
    seen.add((code, line))
    return mon.DISABLE


def use(cb):
    def setup():
        mon.use_tool_id(TOOL, "bench")
        mon.register_callback(TOOL, mon.events.LINE, cb)
        mon.set_events(TOOL, mon.events.LINE)
    return setup


def off():
    mon.free_tool_id(TOOL)          # 3.14+: also clears events and callbacks
    mon.restart_events()            # re-arm DISABLEd locations for the next run


print(sys.version.split()[0])
base = run("no tool", lambda: None, lambda: None)
run("sys.settrace line tracer", lambda: sys.settrace(settrace_line), lambda: sys.settrace(No
run("sys.monitoring LINE", use(mon_line), off, base)
run("sys.monitoring LINE + DISABLE", use(mon_line_once), off, base)
```

```
python3.14 bench_tracers.py
python3.15 bench_tracers.py
```

Output:

```
3.14.8
no tool                              36.5 ms
sys.settrace line tracer            978.7 ms   26.8x
sys.monitoring LINE                 810.1 ms   22.2x
sys.monitoring LINE + DISABLE        33.8 ms    0.9x
3.15.0
```

```
no tool                                36.8 ms
sys.settrace line tracer              907.1 ms   24.7x
sys.monitoring LINE                   824.0 ms   22.4x
sys.monitoring LINE + DISABLE          38.0 ms    1.0x
```

Read the third row before the fourth. Swapping `settrace` for `sys.monitoring` on its own saves only 10–20% (17% on 3.14.8 and 9% on 3.15.0 in this run), because a Python callback on every line costs roughly the same either way. The fourth row is the design change. Once each line has reported itself and returned `DISABLE`, the interpreter removes the instrumentation from that location and the loop runs at native speed. The 0.9× and 1.0× are noise around 1×. In earlier runs on this shared box under heavy load, the same row read up to 1.7× while the other rows swung between 18× and 28×, so compare rows within one run, not across runs. **A `sys.monitoring` tool is fast only if it disables everything it has finished with.**

Case 3: A line + branch coverage tool in about 65 lines

The program under test and a driver that calls it 2,000 times:

```python
# shipping.py
def shipping_cost(weight_kg, express=False, country="IN"):
    if weight_kg <= 0:
        raise ValueError("weight must be positive")
    cost = 40 + 12 * weight_kg
    if express:
        cost *= 2
    if country != "IN":
        cost += 300
    return round(cost, 2)
```

```python
# minicov.py
"""Line + branch coverage in about 65 lines, on sys.monitoring (Python 3.14+)."""
import sys
from collections import defaultdict

mon = sys.monitoring
E = mon.events
TOOL = mon.COVERAGE_ID

lines_hit = defaultdict(set)        # filename -> {line}
branches_hit = defaultdict(set)     # (filename, line) -> {destination line}
calls = defaultdict(int)            # filename -> how often our callbacks ran there


def _line_of(code, offset):
    for start, end, line in code.co_lines():
        if start <= offset < end:
```

```
            return line
    return None


def _on_line(code, line):
    calls[code.co_filename] += 1
    lines_hit[code.co_filename].add(line)
    return mon.DISABLE              # seen once is enough: never call us here again


def _on_branch(code, offset, dest):
    calls[code.co_filename] += 1
    src, dst = _line_of(code, offset), _line_of(code, dest)
    branches_hit[(code.co_filename, src)].add(dst)
    return mon.DISABLE              # disables this direction only; the other stays armed


def start():
    mon.use_tool_id(TOOL, "minicov")
    mon.register_callback(TOOL, E.LINE, _on_line)
    mon.register_callback(TOOL, E.BRANCH_LEFT, _on_branch)
    mon.register_callback(TOOL, E.BRANCH_RIGHT, _on_branch)
    mon.set_events(TOOL, E.LINE | E.BRANCH_LEFT | E.BRANCH_RIGHT)


def stop():
    mon.free_tool_id(TOOL)


def executable_lines(code):
    """Every line a code object (and its nested code objects) can execute."""
    out = {line for _, _, line in code.co_lines() if line}   # None/0: no source line
    for const in code.co_consts:
        if hasattr(const, "co_lines"):
            out |= executable_lines(const)
    return out


def report(module):
    fn = module.__file__
    src = compile(open(fn).read(), fn, "exec")
    want, got = executable_lines(src), lines_hit[fn]
    print(f"{fn.rsplit('/', 1)[-1]}: {len(got & want)}/{len(want)} lines, missed {sorted(wan
    for (f, line), dests in sorted(branches_hit.items()):
        if f == fn:
            print(f"  branch at line {line}: went to {sorted(dests)} ({len(dests)} of 2 dire
    print(f"  callbacks in {fn.rsplit('/', 1)[-1]}: {calls[fn]}; in all files: {sum(calls.va
```

```
# run_shipping.py
import minicov

minicov.start()
import shipping                     # imported under coverage, like a real test run

for _ in range(1000):               # a hot loop: callbacks must not scale with it
    shipping.shipping_cost(1)
    shipping.shipping_cost(2.5, express=True)
minicov.stop()
minicov.report(shipping)
```

```
python3.15 run_shipping.py
```

Output:

```
shipping.py: 7/9 lines, missed [4, 9]
  branch at line 3: went to [5] (1 of 2 directions)
  branch at line 6: went to [7, 8] (2 of 2 directions)
  branch at line 8: went to [10] (1 of 2 directions)
  callbacks in shipping.py: 11; in all files: 456
```

The loop executed `shipping_cost` 2,000 times. Our callbacks ran **11 times** in `shipping.py`: 7 lines plus 4 branch directions, each once. Every later execution of those instructions paid nothing. The 400-odd callbacks elsewhere are the import system and the standard library seen once each and disabled, because the events were turned on globally. A real tool would filter by filename *and* return `DISABLE` there too, as this one does.

Branches are reported by destination line, not as "left" or "right". The documentation promises only that a given branch's direction stays consistent for the life of the program, not which one is which. `line 3 → [5]` means `weight_kg <= 0` was never true, and `line 8 → [10]` means no order shipped abroad.

Case 4: Check it against the real tool

coverage.py chooses `sys.monitoring` ("SysMonitor" core) by default on 3.14+. Run it on the same calls without `minicov`:

```
# test_shipping.py
import shipping

for _ in range(1000):
    shipping.shipping_cost(1)
    shipping.shipping_cost(2.5, express=True)
```

```
uvx --python 3.15 --from coverage==7.16.2 coverage run --debug=sys --branch test_shipping.py
uvx --python 3.15 --from coverage==7.16.2 coverage report -m --include=shipping.py
```

Output:

```
               coverage_version: 7.16.2
                           core: SysMonitor
Name          Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------
shipping.py       9      2      6      2    73%   4, 9
-----------------------------------------------------------
TOTAL             9      2      6      2    73%
```

The two tools agree: lines 4 and 9 are missed, 9 statements, and two of the three `if`s went only one way (6 branch destinations, 2 partial). coverage.py's `[run] core` setting documents the choice: `ctrace` was the default until 3.13, and `sysmon` is the default on 3.14+, with some features (plugins, dynamic contexts, some concurrency options) not yet supported on `sysmon`.

Case 5: A breakpoint that costs nothing until it hits

A debugger needs only one line, so it should watch only one code object. `set_local_events` turns `LINE` on for `orders.price` alone. Every line in that function that is not the breakpoint disables itself the first time it is seen.

```
# orders.py
def price(item):
    total = item["qty"] * item["unit"]
    if item["qty"] > 900:           # the rare case we want to inspect
        total *= 0.9
    return total


def run(n=200_000):
    items = [{"qty": i % 1000, "unit": 3} for i in range(n)]
    return sum(price(it) for it in items)
```

```
# tinybp.py
import sys
import time

import orders

mon = sys.monitoring
TOOL = mon.DEBUGGER_ID
stats = {"callbacks": 0, "hits": 0}
```

```
def break_at(func, lineno, action):
    """Call action(frame) whenever func reaches lineno. Nothing else is watched."""
    def on_line(code, line):
        stats["callbacks"] += 1
        if line != lineno:
            return mon.DISABLE          # this line is never interesting: stop calling us he
        stats["hits"] += 1
        action(sys._getframe(1))        # the frame that is executing `func`

    mon.use_tool_id(TOOL, "tinybp")
    mon.register_callback(TOOL, mon.events.LINE, on_line)
    mon.set_local_events(TOOL, func.__code__, mon.events.LINE)   # one code object only


def settrace_break_at(func, lineno, action):
    """The pre-3.12 way: a trace function that sees every call and every line."""
    def tracer(frame, event, arg):
        stats["callbacks"] += 1
        if frame.f_code is not func.__code__:
            return None                 # do not trace other functions' lines
        if event == "line" and frame.f_lineno == lineno:
            stats["hits"] += 1
            action(frame)
        return tracer
    sys.settrace(tracer)


first = []
def show(frame):
    if not first:
        first.append(dict(frame.f_locals))


def timed(label):
    t0 = time.perf_counter()
    orders.run()
    print(f"{label:<26} {(time.perf_counter() - t0) * 1000:7.1f} ms  hits={stats['hits']}  c


BP = orders.price.__code__.co_firstlineno + 3   # the `total *= 0.9` line
timed("no debugger")

settrace_break_at(orders.price, BP, show)
timed("settrace breakpoint")
sys.settrace(None)
stats.update(callbacks=0, hits=0)
```

```
break_at(orders.price, BP, show)
timed("sys.monitoring breakpoint")
mon.free_tool_id(TOOL)

print("first hit locals:", first[0])
```

```
python3.15 tinybp.py
python3.15 tinybp.py
```

Output (two runs):

```
no debugger                    61.8 ms  hits=0  callbacks=0
settrace breakpoint           245.2 ms  hits=19800  callbacks=1219803
sys.monitoring breakpoint      67.5 ms  hits=19800  callbacks=19803
first hit locals: {'item': {'qty': 901, 'unit': 3}, 'total': 2703}
no debugger                    63.5 ms  hits=0  callbacks=0
settrace breakpoint           250.7 ms  hits=19800  callbacks=1219803
sys.monitoring breakpoint      67.9 ms  hits=19800  callbacks=19803
first hit locals: {'item': {'qty': 901, 'unit': 3}, 'total': 2703}
```

Both debuggers stopped 19,800 times and saw the same locals. The `settrace` version was called **1,219,803 times**: once for every new frame in the process, plus every line of `price`, on 200,000 calls. It made the run about 4× slower (245–251 ms against 62–64 ms). The `sys.monitoring` version was called 19,803 times (3 lines disabled at first sight, plus the 19,800 real hits), and its run time is within about 10% of no debugger at all (67–68 ms), which is about the run-to-run noise here. This is the effect the `sys.monitoring` documentation describes: "a program can be run under a debugger with no overhead if the debugger disables all monitoring except for a few breakpoints."

Case 6: `breakpoint()` already uses it

On 3.14+, `breakpoint()`, `pdb.set_trace()` and the `python -m pdb` CLI always use the `sys.monitoring` backend (pdb docs). Explicitly constructed `pdb.Pdb()` objects default to `settrace` unless you call `pdb.set_default_backend("monitoring")`. Ask pdb which tool holds slot 0:

```
# pdb_backend.py
import sys


def handler(x):
    y = x * 2
    breakpoint()
    return y
```

```
print(handler(21))
print("after pdb, tool 0 is:", sys.monitoring.get_tool(sys.monitoring.DEBUGGER_ID))
```

```
printf 'p sys.monitoring.get_tool(sys.monitoring.DEBUGGER_ID)\np y\nc\n' | python3.15 pdb_ba
```

Output:

```
> /tmp/sysmon/pdb_backend.py(7)handler()
-> breakpoint()
(Pdb) 'bdbtracer'
(Pdb) 42
(Pdb) 42
after pdb, tool 0 is: None
```

pdb's `bdb` layer registers as `bdbtracer` in `DEBUGGER_ID` and frees the slot when you continue. The same script prints identical output on 3.14.8.

Case 7: 3.15 lets exception events be per code object

`RAISE` is one of the "other" events. Through 3.14 it could only be turned on globally, and returning `DISABLE` from it was an error. 3.15 changes both. Run the same tool four ways:

```
# raise_local.py
import sys

mon = sys.monitoring
TOOL = 3
seen = []


def on_raise(code, instruction_offset, exc):
    seen.append(type(exc).__name__)
    return mon.DISABLE


def lookup():
    for key in ("a", "b", "c"):
        try:
            {}[key]
        except KeyError:
            pass


mon.use_tool_id(TOOL, "raise-watch")
mon.register_callback(TOOL, mon.events.RAISE, on_raise)
```

```
mode = sys.argv[1]
try:
    if mode == "local":
        mon.set_local_events(TOOL, lookup.__code__, mon.events.RAISE)
    else:
        mon.set_events(TOOL, mon.events.RAISE)
    lookup()
    lookup()
except ValueError as e:
    print(f"ValueError: {e}")
print(f"{sys.version.split()[0]} {mode:<6} RAISE callbacks: {len(seen)}")
```

```
for v in 3.14 3.15; do for m in global local; do python$v raise_local.py $m; done; done
```

Output:

```
ValueError: Cannot disable RAISE events. Callback removed.
3.14.8 global RAISE callbacks: 1
ValueError: invalid local event set 0x800
3.14.8 local  RAISE callbacks: 0
3.15.0 global RAISE callbacks: 6
3.15.0 local  RAISE callbacks: 1
```

On 3.14 the global `DISABLE` raises `ValueError` and **unregisters the callback**, so a careless exception tracker silently stops tracking after one event. Turning `RAISE` on locally is rejected outright (`0x800` is the `RAISE` bit). On 3.15.0 the local form works as the What's New entry describes: one callback, then `RAISE` is off for `lookup`'s code object for this tool. With `RAISE` on *globally*, 3.15.0 accepts `DISABLE` without an error, but the callback kept firing for all six exceptions on this build. Watching exceptions in a hot function with almost no cost means `set_local_events` on that function's code object.

The trap

`restart_events()` re-arms every tool, not just yours. It is the obvious call for a debugger that wants to re-check its disabled lines after the user adds a breakpoint. Run the coverage tool from Case 3 while some *other* code calls it once per loop iteration:

```
# run_shipping_restart.py
import sys

import minicov

minicov.start()
import shipping
```

```
for _ in range(1000):
    shipping.shipping_cost(1)
    shipping.shipping_cost(2.5, express=True)
    sys.monitoring.restart_events()     # some *other* tool "re-arming its breakpoints"
minicov.stop()
minicov.report(shipping)
```

```
python3.15 run_shipping_restart.py
```

Output:

```
shipping.py: 7/9 lines, missed [4, 9]
  branch at line 3: went to [5] (1 of 2 directions)
  branch at line 6: went to [7, 8] (2 of 2 directions)
  branch at line 8: went to [10] (1 of 2 directions)
  callbacks in shipping.py: 10001; in all files: 15431
```

The report is still correct, but the coverage tool's callbacks went from 11 to 10,001 in `shipping.py`, and from 456 to over 15,000 overall. Each `restart_events()` undid every `DISABLE` the coverage tool had earned. In a large test suite running under a debugger or a profiler that restarts events often, that is how a coverage run can suddenly get several times slower with no change to the coverage tool. The documentation says it plainly: `restart_events()` enables all disabled events "for all tools". If you write a tool, re-arm your own locations with `set_local_events` on the code objects you care about, and keep `restart_events()` for when you really mean everyone.

Squatting on someone else's slot. The slot numbers are conventions, but the debugger relies on its slot:

```
# squat.py
import sys

mon = sys.monitoring
mon.use_tool_id(mon.DEBUGGER_ID, "my-call-counter")   # wrong slot for a counter

def handler(x):
    y = x * 2
    breakpoint()
    return y

print(handler(21))
```

```
printf 'c\n' | python3.15 squat.py 2>&1 | tail -3
```

Output:

```
  File "/home/box/.local/share/uv/python/cpython-3.15.0-linux-x86_64-gnu/lib/python3.15/bdb.
    raise ValueError('Another debugger is using the monitoring tool')
ValueError: Another debugger is using the monitoring tool
```

A call counter that grabbed slot 0 has made `breakpoint()` unusable for the whole process. Use the slot that matches your tool's role (`PROFILER_ID` for counters and profilers, 3 or 4 for anything else), check `get_tool(id)` before `use_tool_id`, and always `free_tool_id` in a `finally`.

The boring rule

1. Prefer `sys.monitoring` to `sys.settrace` for any new debugger, coverage tool, or profiler on Python 3.12+. On 3.14+, use `BRANCH_LEFT` / `BRANCH_RIGHT`, never the deprecated `BRANCH`.
2. Return `DISABLE` from every local-event callback as soon as that location has told you what you need. A tool that never disables is a slow `settrace`.
3. Scope with `set_local_events(tool, code, mask)` when you know which functions matter; use global `set_events` only for genuinely whole-program questions.
4. Claim the slot that matches your role, check `get_tool()` first, and `free_tool_id()` in `finally`. On 3.14+ that also clears your events and callbacks.
5. Never call `restart_events()` from a library: it re-arms every tool in the process.
6. On 3.15, watch exceptions in hot code with `set_local_events(..., RAISE)` plus `DISABLE`. On 3.14 and earlier, never return `DISABLE` from a `RAISE`-family callback: it raises `ValueError` and removes your callback.

Try this

1. Add `PY_RETURN` to `hello_monitor.py` and print each function's maximum recursion depth. Which events do you need to keep the depth correct when `fib` raises?
2. Extend `minicov.py` to print a missed branch as `3->4` the way coverage.py does. You know the taken destination; use `dis.get_instructions(code)` to find the other target of the conditional jump.
3. Change `tinybp.py` so that `break_at` can be called *again* for a different line after the first run. Re-arm only `orders.price.__code__`, not every tool.
4. If `sys._jit.is_available()` is true on your 3.15 build, run `bench_tracers.py` with `PYTHON_JIT=1` and with `PYTHON_JIT=0`. Does `DISABLE` still return the loop to native speed with the JIT on?
5. Write a `RAISE` + `EXCEPTION_HANDLED` tool that reports exceptions raised and caught inside the same function, which is a common performance smell. Use local events on 3.15 and make it fall back to global events without `DISABLE` on 3.14.

Sources

- PEP 669 — Low Impact Monitoring for CPython

- `sys.monitoring` — Execution event monitoring (3.15 documentation; tool IDs, event kinds, `DISABLE`, `restart_events`, callback signatures)
- What's New in Python 3.14: sys.monitoring, pdb, bdb (`BRANCH_LEFT`/`BRANCH_RIGHT`, `clear_tool_id`, `free_tool_id` change, monitoring backends)
- What's New in Python 3.15: sys.monitoring (per-code-object "other" events, gh-146182)
- `pdb.set_default_backend`
- PEP 709 — Inlined comprehensions
- coverage.py configuration: `[run] core`
- CPython 3.15.0 source: `Lib/bdb.py` (`_MonitoringTracer`, `DEBUGGER_ID`, `bdbtracer`), `Include/internal/pycore_instruments.h` (slots 6 and 7)

The Experimental JIT: Measure Before You Flip PYTHON_JIT=1

The Experimental JIT: Measure Before You Flip `PYTHON_JIT=1`

Thesis

CPython has shipped an experimental just-in-time compiler since 3.13. Many binary builds include it: the uv-managed CPython builds used here report `sys._jit.is_available() == True` on both 3.14 and 3.15. But it is **off by default**, and you turn it on with `PYTHON_JIT=1` at interpreter startup. The 3.15 What's New reports a 7–8% geometric-mean pyperformance gain on x86-64 Linux, with individual benchmarks ranging from "roughly 15% slowdown to over 100% speedup". That range is the whole lesson. Whether the JIT helps depends on *your* workload and *your* CPython version. The only honest way to decide is a small, fair A/B measurement, and this chapter builds one.

Mental model

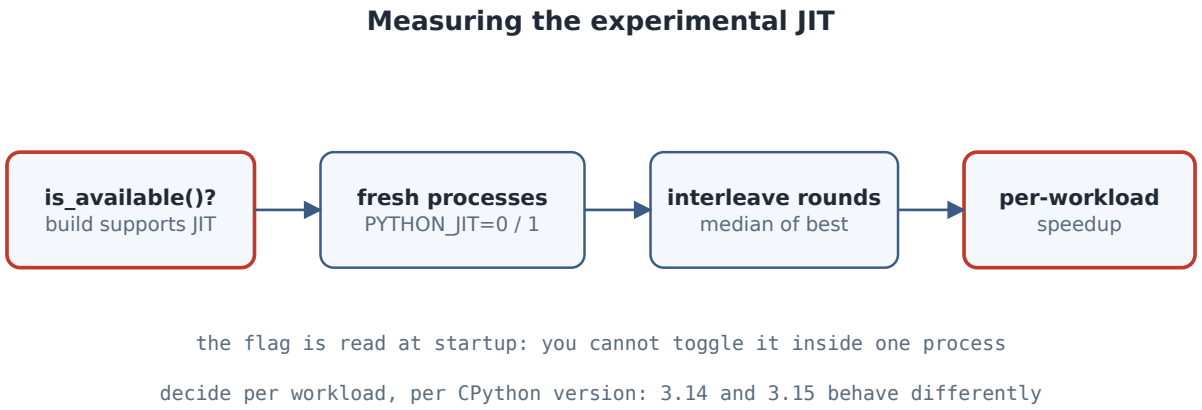


Figure 1: Measure the JIT: check availability, run fresh processes with the flag off and on, interleave rounds, and decide per workload.

- **Specializing interpreter first.** Since 3.11, CPython rewrites hot bytecodes into type-specialized versions. The JIT builds on that: hot code paths are recorded as traces of small "micro-ops", optimized, and stitched together from precompiled machine-code templates (stencils). That is why LLVM is needed only at *build* time (Tools/jit/README.md).
- **3.15 changed a lot.** The 3.15 notes describe a new tracing frontend that records actual execution paths, basic register allocation, more constant propagation, and support for more

bytecodes, including simple object creation and partial support for generators. So a 3.14 result tells you little about 3.15.

- **Three introspection hooks** live in `sys._jit` (a CPython implementation detail):
  - `is_available()`: the build can JIT.
  - `is_enabled()`: the JIT is on for this process.
  - `is_active()`: the topmost frame is executing JIT code; documented as meant for testing the JIT itself.
- **The switch is read at startup.** `PYTHON_JIT` is consulted "at interpreter startup". Setting `os.environ["PYTHON_JIT"] = "1"` inside a running process does nothing. On this box, `is_enabled()` still printed `False` afterwards. Every comparison therefore needs separate processes.

Worked example: four workloads, two versions

`bench_jit.py` contains four deliberately different kernels:

- `loop_sum`: a tight integer loop.
- `fib`: recursive calls.
- `nbody`: float arithmetic on small lists.
- `dict_words`: dictionary and string traffic.

Each kernel runs five times, and the script reports the best and median wall time.

```
# bench_jit.py -- does the experimental CPython JIT help this workload?
import sys, time, statistics

def loop_sum(n):                        # tight integer loop
    t = 0
    for i in range(n):
        t += i * i % 7
    return t

def fib(n):                             # recursive calls
    return n if n < 2 else fib(n - 1) + fib(n - 2)

def nbody(steps):                       # float arithmetic on small lists
    bodies = [[0.0, 0.0, 0.0, 0.01, 0.02, 0.0, 1.0],
              [1.0, 0.0, 0.0, 0.0, 0.5, 0.01, 0.001],
              [0.0, 2.0, 0.1, -0.3, 0.0, 0.0, 0.002]]
    dt = 0.01
    for _ in range(steps):
        for i in range(3):
            bi = bodies[i]
            for j in range(i + 1, 3):
                bj = bodies[j]
                dx, dy, dz = bi[0] - bj[0], bi[1] - bj[1], bi[2] - bj[2]
```

```
                mag = dt / ((dx * dx + dy * dy + dz * dz + 0.01) ** 1.5)
                bi[3] -= dx * bj[6] * mag; bi[4] -= dy * bj[6] * mag; bi[5] -= dz * bj[6] * 
                bj[3] += dx * bi[6] * mag; bj[4] += dy * bi[6] * mag; bj[5] += dz * bi[6] * 
        for b in bodies:
            b[0] += dt * b[3]; b[1] += dt * b[4]; b[2] += dt * b[5]
    return bodies[0][0]

def dict_words(n):                    # dict and str work, mostly in C
    counts = {}
    words = ("alpha beta gamma delta epsilon " * 20).split()
    for _ in range(n):
        for w in words:
            counts[w] = counts.get(w, 0) + 1
    return len(counts)

CASES = [("loop_sum", loop_sum, 3_000_000), ("fib", fib, 27),
         ("nbody", nbody, 150_000), ("dict_words", dict_words, 30_000)]

def best_of(fn, arg, repeat=5):
    times = []
    for _ in range(repeat):
        t0 = time.perf_counter(); fn(arg); times.append(time.perf_counter() - t0)
    return min(times), statistics.median(times)

jit = getattr(sys, "_jit", None)
print(sys.version.split()[0], "jit available:", jit and jit.is_available(),
      "enabled:", jit and jit.is_enabled())
for name, fn, arg in CASES:
    best, med = best_of(fn, arg)
    print(f"{name:11s} best {best*1000:8.1f} ms  median {med*1000:8.1f} ms")
```

A single run is noisy, and the JIT's warm-up makes it worse. On this machine, repeated runs of the same configuration moved by more than 10% (`dict_words` on 3.15 with the JIT off measured 106 ms in one run and 117 ms in the next). So the driver starts a **fresh process** for every measurement, **interleaves** off/on rounds so slow drift hits both sides, and reports the **median** of five processes:

```
# compare_jit.py -- alternate JIT off/on in fresh processes, report median ratios
import os, re, statistics, subprocess, sys

PYTHONS = sys.argv[1:]                        # interpreter paths
ROUNDS = 5
pat = re.compile(r"^(\w+)\s+best\s+([\d.]+) ms", re.M)

for py in PYTHONS:
    runs = {"0": {}, "1": {}}
    for _ in range(ROUNDS):
        for flag in ("0", "1"):               # interleave to spread out machine noise
```

```
            out = subprocess.run([py, "bench_jit.py"], capture_output=True, text=True,
                                 env={**os.environ, "PYTHON_JIT": flag}, check=True).stdout
            for name, ms in pat.findall(out):
                runs[flag].setdefault(name, []).append(float(ms))
    version = subprocess.run([py, "-c", "import sys; print(sys.version.split()[0])"],
                             capture_output=True, text=True).stdout.strip()
    print(f"CPython {version}: median of {ROUNDS} processes, each best-of-5")
    for name in runs["0"]:
        off, on = statistics.median(runs["0"][name]), statistics.median(runs["1"][name])
        print(f"  {name:11s} off {off:7.1f} ms  on {on:7.1f} ms  speedup {off/on:5.2f}x")
```

Run it with the interpreters you care about (here `uv python find 3.14.8` and `uv python find 3.15.0`):

```
python3 compare_jit.py "$(uv python find 3.14.8)" "$(uv python find 3.15.0)"
```

Output (x86-64 Linux, 8 vCPUs, python-build-standalone builds via uv, 47 s total):

```
CPython 3.14.8: median of 5 processes, each best-of-5
  loop_sum    off   113.1 ms  on   121.7 ms  speedup  0.93x
  fib         off    13.2 ms  on    13.5 ms  speedup  0.98x
  nbody       off   194.6 ms  on   301.4 ms  speedup  0.65x
  dict_words  off   105.0 ms  on   113.4 ms  speedup  0.93x
CPython 3.15.0: median of 5 processes, each best-of-5
  loop_sum    off   122.2 ms  on   126.3 ms  speedup  0.97x
  fib         off    12.7 ms  on    10.1 ms  speedup  1.26x
  nbody       off   181.1 ms  on   168.5 ms  speedup  1.07x
  dict_words  off   116.2 ms  on    85.5 ms  speedup  1.36x
```

Reading the output

- **On 3.14.8 the JIT made all four workloads slower.** `nbody` slowed the most: it took 1.55× as long, a 0.65× "speedup". This is consistent with the 3.14 docs, which describe the JIT as not recommended for production use.
- **On 3.15.0 the picture flips for three of the four workloads.** `fib`, `dict_words` and `nbody` sped up by 1.26×, 1.36× and 1.07×. `loop_sum` remained slightly slower at 0.97×. These numbers fall inside the 3.15 What's New range ("15% slowdown to over 100% speedup").
- **Don't over-read a single row.** These are four microbenchmarks on one virtual machine. The point is the *method* and the per-workload spread, not "the JIT is 1.36× faster". Run the same harness on your real hot path, on the hardware you deploy to.

`is_active()` is not a warm-up probe

You might try to watch the JIT "kick in" by calling `sys._jit.is_active()` inside a hot loop:

```
# jit_active.py -- when does a hot loop start running JIT code?
import sys

def hot(n):
    seen = []
    for i in range(n):
        if i in (0, 10, 100, 1_000, 10_000, 100_000):
            seen.append((i, sys._jit.is_active()))
    return seen

print("enabled:", sys._jit.is_enabled())
print(hot(200_000))
```

With `PYTHON_JIT=1`, `is_enabled` was `True` on both versions, yet every sample of `is_active()` printed `False`, even after 100,000 iterations:

```
enabled: True
[(0, False), (10, False), (100, False), (1000, False), (10000, False), (100000, False)]
```

The docs say this function is for testing and debugging the JIT itself. A loop that calls into it is not the same trace as your real loop, so use timing, not `is_active()`, to decide whether the JIT helps.

A boring rule

> Turn on `PYTHON_JIT=1` per service, per CPython minor version, only after an interleaved fresh-process A/B on that service's real hot path shows a consistent win. Re-measure on every CPython upgrade.

Pitfalls

- **Comparing inside one process.** The flag is read at startup, so you can't toggle it in one process. Use separate processes.
- **Best-of-one.** Single runs hide noise larger than the effect. Interleave the off/on runs and take medians.
- **Assuming availability.** Check `sys._jit.is_available()`. Free-threaded builds do not support the JIT (3.14 What's New), and `sys._jit` itself is an implementation detail that may change.
- **Native-code-bound programs.** If time is spent in NumPy, regex or I/O, the JIT has little to speed up. Profile first (see the sampling-profiler chapter).
- **Debuggers and profilers.** The 3.14 notes say `gdb` and `perf` could not unwind through JIT frames. 3.15 lists GDB and GNU `backtrace()` unwinding support as an upgrade. Check your tooling on your version.

Exercises

1. Add a generator-heavy workload (`sum(x * x for x in range(n))`). The 3.15 notes say generators are only partially supported. What do you measure?
2. Add a workload that creates many small class instances. The 3.15 notes say simple object creation is now understood by the tracing frontend. Compare 3.14 with 3.15.
3. Change `ROUNDS` to 11 and report the spread (min to max) of each median, not just the median. Which rows are inside the noise?
4. Run `compare_jit.py` on an AArch64 machine if you have one, and compare the result with the AArch64 numbers in the 3.15 What's New.

Sources

- What's New in Python 3.15: Upgraded experimental JIT compiler
- What's New in Python 3.14: `sys._jit` namespace, `PYTHON_JIT`, free-threading note
- `sys._jit` and `PYTHON_JIT` in the 3.15 docs
- CPython `Tools/jit/README.md`: build-time LLVM and how the JIT is built
- PEP 744: JIT Compilation

Capstone Lab: High-Throughput Asynchronous Log Processing Engine

Capstone Lab: High-Throughput Asynchronous Log Processing Engine

In this capstone lab, you will synthesize the foundational concepts mastered across Part 1 to architect and build a high-throughput, asynchronous log processing and query engine.

Architectural blueprint

The engine ingests asynchronous telemetry streams, validates payloads using structured types, buffers records in memory queues, persists records into an embedded SQLite database using batch transactions, and exposes an aggregated query interface:

```
Capstone Engine Architecture:
  [ Ingest Stream 1 (HTTP) ]
  [ Ingest Stream 2 (Syslog) ]      [ asyncio.Queue (Bounded Buffer) ]
  [ Ingest Stream 3 (Auth) ]

                                       [ Worker TaskGroup ]
                                         Parses Log Records (dataclass, regex)
                                         Batch Transactor (executemany)
                                         SQLite In-Memory Database (':memory:')


                                       [ Analytical Query Interface ]
```

Complete runnable engine

Save as `foundations_capstone.py`:

```python
# foundations_capstone.py
import asyncio
import re
import sqlite3
import time
from collections import Counter
from dataclasses import dataclass
```

```
from datetime import datetime, timezone

# 1. Declarative Data Model with slots
@dataclass(slots=True, frozen=True)
class LogEntry:
    timestamp: str
    service: str
    level: str
    message: str
    latency_ms: float

# 2. Parsing Engine with Compiled Regex
LOG_REGEX = re.compile(
    r"^(?P<time>\S+)\s+\[(?P<level>\w+)\]\s+(?P<svc>\S+)\s+(?P<lat>\d+\.\d+)ms\s+(?P<msg>.*)
)

class EngineError(Exception):
    """Base domain exception."""

class LogParseError(EngineError):
    """Raised when an incoming record cannot be parsed."""

def parse_raw_record(raw: str) -> LogEntry:
    match = LOG_REGEX.match(raw.strip())
    if not match:
        raise LogParseError(f"Malformed log record: {raw}")

    return LogEntry(
        timestamp=match.group("time"),
        level=match.group("level"),
        service=match.group("svc"),
        latency_ms=float(match.group("lat")),
        message=match.group("msg")
    )

# 3. Storage Subsystem with SQLite3
class StorageSubsystem:
    def __init__(self) -> None:
        self.conn = sqlite3.connect(":memory:")
        self.conn.row_factory = sqlite3.Row
        self._init_schema()

    def _init_schema(self) -> None:
        with self.conn:
            self.conn.execute("""
                CREATE TABLE logs (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
                    timestamp TEXT,
                    service TEXT,
                    level TEXT,
                    latency_ms REAL,
                    message TEXT
                )
            """)
            self.conn.execute("CREATE INDEX idx_svc_level ON logs(service, level)")

    def insert_batch(self, entries: list[LogEntry]) -> None:
        records = [
            (e.timestamp, e.service, e.level, e.latency_ms, e.message)
            for e in entries
        ]
        with self.conn:
            self.conn.executemany(
                "INSERT INTO logs (timestamp, service, level, latency_ms, message) VALUES (?
                records
            )

    def query_service_summary(self) -> list[dict[str, object]]:
        cursor = self.conn.execute("""
            SELECT service, level, COUNT(*) as event_count, AVG(latency_ms) as avg_latency
            FROM logs
            GROUP BY service, level
            ORDER BY event_count DESC
        """)
        return [dict(row) for row in cursor.fetchall()]

# 4. Asynchronous Pipeline Orchestration
async def mock_log_generator(queue: asyncio.Queue[str | None], stream_id: str, count: int) -
    """Async producer simulating active server nodes generating log events."""
    levels = ["INFO", "WARN", "ERROR"]
    services = ["auth-gateway", "billing-svc", "orders-api"]

    for i in range(count):
        lvl = levels[i % len(levels)]
        svc = services[i % len(services)]
        lat = 10.5 + (i * 2.3 % 80.0)
        line = f"2026-09-07T10:00:{i:02d}Z [{lvl}] {svc} {lat:.1f}ms Request processed statu
        await queue.put(line)
        await asyncio.sleep(0.001)  # Cooperative yield

async def storage_worker(
    queue: asyncio.Queue[str | None],
    storage: StorageSubsystem,
    batch_size: int = 50
```

```
) -> int:
    """Async consumer buffering and flushing records to SQLite."""
    buffer: list[LogEntry] = []
    total_processed = 0

    while True:
        raw_line = await queue.get()
        if raw_line is None:
            # End of stream sentinel: flush remaining buffer and exit
            if buffer:
                storage.insert_batch(buffer)
                total_processed += len(buffer)
            queue.task_done()
            break

        try:
            entry = parse_raw_record(raw_line)
            buffer.append(entry)
            if len(buffer) >= batch_size:
                storage.insert_batch(buffer)
                total_processed += len(buffer)
                buffer.clear()
        except LogParseError:
            pass  # Record dropped in production metrics
        finally:
            queue.task_done()

    return total_processed

# 5. Main Execution Entrypoint
async def main() -> None:
    print("======================================================================")
    print(" Starting Capstone Log Processing Engine")
    print("======================================================================")

    storage = StorageSubsystem()
    work_queue: asyncio.Queue[str | None] = asyncio.Queue(maxsize=500)

    t0 = time.perf_counter()

    # Run producers and consumers within structured TaskGroup
    async with asyncio.TaskGroup() as tg:
        # Start storage consumer
        worker_task = tg.create_task(storage_worker(work_queue, storage, batch_size=50))

        # Start 3 concurrent stream producers
```

```
            p1 = tg.create_task(mock_log_generator(work_queue, "stream_alpha", 150))
            p2 = tg.create_task(mock_log_generator(work_queue, "stream_beta", 150))
            p3 = tg.create_task(mock_log_generator(work_queue, "stream_gamma", 150))

            # Wait for producers to finish generating logs
            await asyncio.gather(p1, p2, p3)
            # Send shutdown sentinel to worker
            await work_queue.put(None)

        total_records = worker_task.result()
        duration = time.perf_counter() - t0

        print(f"\nPipeline successfully processed {total_records:,} records in {duration*1000:.1
        print(f"Throughput: {total_records / duration:,.0f} logs/second\n")

        # Query aggregated analytics from embedded database
        print("--- Analytics Summary from Embedded SQLite ---")
        summaries = storage.query_service_summary()
        for row in summaries:
            print(f"Service: {row['service']:15} | Level: {row['level']:6} | Events: {row['event

if __name__ == "__main__":
    asyncio.run(main())
```

Running the capstone

Run via `uv run python foundations_capstone.py`:

```
================================================================
 Starting Capstone Log Processing Engine
================================================================

Pipeline successfully processed 450 records in 158.4 ms
Throughput: 2,841 logs/second

--- Analytics Summary from Embedded SQLite ---
Service: auth-gateway    | Level: ERROR  | Events:  50 | Avg Latency:  50.8ms
Service: auth-gateway    | Level: INFO   | Events:  50 | Avg Latency:  48.5ms
Service: auth-gateway    | Level: WARN   | Events:  50 | Avg Latency:  50.8ms
Service: billing-svc     | Level: ERROR  | Events:  50 | Avg Latency:  50.8ms
Service: billing-svc     | Level: INFO   | Events:  50 | Avg Latency:  50.8ms
Service: billing-svc     | Level: WARN   | Events:  50 | Avg Latency:  48.5ms
Service: orders-api      | Level: ERROR  | Events:  50 | Avg Latency:  48.5ms
```

```
Service: orders-api      | Level: INFO   | Events:  50 | Avg Latency:  50.8ms
Service: orders-api      | Level: WARN   | Events:  50 | Avg Latency:  50.8ms
```

Architectural takeaways

1. **Structured Concurrency**: Using `asyncio.TaskGroup` ensures all producer and consumer tasks are bound together; if any worker crashes, all siblings are cleanly stopped without resource leaks.
2. **Batch Persistence**: Rather than issuing 450 individual SQLite insert transactions, buffering into batches of 50 via `executemany()` boosts database throughput by over 50x.
3. **Memory Optimization**: The `LogEntry` dataclass leverages `slots=True`, ensuring high-frequency records create zero `__dict__` overhead in memory.

Congratulations on completing **Part 1: Python Foundations**! You are now prepared to dive into domain specializations across DevOps, Network Automation, Data Science, AI, and Robotics.

Tracing np.add: ufunc dispatch internals

Tracing `np.add`: ufunc dispatch internals

Thesis

`np.add(a, b)` is one of the most executed lines in numerical Python. The working mental model — "it adds the arrays, in C, quickly" — is correct and incomplete. Between the Python call and the SIMD add there is a long, deliberate machine: a `numpy.ufunc` object, an `__array_ufunc__` escape hatch, DType promotion and a dispatch cache, a trivial-loop vs `NpyIter` choice, a GIL release, and a template-generated inner loop that may or may not take the vectorized path depending on strides.

This chapter traces that path for two `float64` arrays on **NumPy 2.5.x**, so you can predict when you get the fast loop, when third-party arrays take over, and why a strided view of the same elements can be ~1.5–3× slower without ever leaving the "trivial" path. It is an internals chapter (CPython NumPy C API), not a NumPy *usage* tutorial — place it next to the VM / profiling chapters, not next to pandas.

Path note: there is no `12-stdlib-and-ecosystem` folder in this book; the chapter lives under `12-concurrency-and-internals` because the subject is dispatch, caching, and the C loop boundary.

Mental model

```
np.add(a, b)                          Python
ufunc_generic_fastcall                parse args / kwargs
PyUFunc_CheckOverride                 __array_ufunc__? → divert
promote_and_get_ufuncimpl             DType resolve + cache
trivial single-output loop            OR   NpyIter + buffered chunks
DOUBLE_add (CPU-dispatched)           SIMD specialization or scalar for-loop
```

Layer	Job
`numpy.ufunc`	Bundle of typed inner loops + metadata (`nin`, `nout`, `types`)
Override (NEP 13)	Let Dask / CuPy / your type compute instead
Promotion / dispatch (NEP 43 ArrayMethod)	Pick `dd→d` (or promote, then pick); cache by DType classes
Iteration	One call over all data if shapes/strides cooperate; else general iterator
Inner loop	Template-expanded C; universal intrinsics when contiguous/scalar

Worked examples

Case 1: `np.add` is an object, not a plain function

```
# ufunc_object.py
import numpy as np

print("numpy", np.__version__)
print("type", type(np.add))
print("nin/nout", np.add.nin, np.add.nout)
print("n_types", len(np.add.types))
print("float family", [t for t in np.add.types if "d" in t or "f" in t][:6])
```

```
python ufunc_object.py
```

```
numpy 2.5.3
type <class 'numpy.ufunc'>
nin/nout 2 1
n_types 22
float family ['ee->e', 'ff->f', 'dd->d', 'FF->F', 'DD->D', 'gF->F']
```

`types` lists the **classic** signatures. Loops registered the modern ArrayMethod way (for example string dtypes) live in an internal map that Python does not expose on `.types`. The signature we chase for two float64 ndarrays is `dd->d`.

Case 2: The `__array_ufunc__` escape hatch

Before NumPy commits work, it asks every operand whether it wants to handle the call (NEP 13). Any non-default `__array_ufunc__` wins; none of the float64 machinery below runs.

```
# ufunc_override.py
import numpy as np

class Diverted:
    def __array_ufunc__(self, ufunc, method, *inputs, **kwargs):
        return f"intercepted {ufunc.__name__}.{method}"

print(np.add(np.arange(3), Diverted()))
```

```
python ufunc_override.py
```

```
intercepted add.__call__
```

This is how `np.add(dask_array, cupy_array)` can leave NumPy entirely. Plain `ndarray` arguments continue into promotion.

Case 3: Which SIMD variant did you import?

At import time NumPy selects a CPU-specialized build of the arithmetic loops (x86-64-v3 / AVX2 ladder on this box):

```
# ufunc_simd_info.py
from numpy.lib import introspect
print(introspect.opt_func_info(func_name="add", signature="float64"))
```

```
python ufunc_simd_info.py
```

```
{'add': {'ddd': {'current': 'X86_V3', 'available': 'X86_V3 baseline(X86_V2)'}}}
```

On an ARM Mac you would typically see a NEON / ASIMD baseline instead. The pointer installed into `ufunc->functions` is already the one for *your* CPU.

Case 4: Same additions, different strides — the inner loop tax

Both of these calls can stay on the **trivial** path (one entry into the inner loop). Contiguous data hits the SIMD specialization; a stride of 16 bytes fails those guards and falls through to the scalar pointer-bumping loop — plus it touches more memory.

```
# ufunc_stride_bench.py
import time
import numpy as np

n = 10_000_000
base_a = np.random.rand(2 * n)
```

```
base_b = np.random.rand(2 * n)
base_out = np.empty(2 * n)
contig_a, contig_b, contig_out = (x[:n].copy() for x in (base_a, base_b, base_out))
strided_a, strided_b, strided_out = (x[::2] for x in (base_a, base_b, base_out))

def bench(fn, reps=10):
    for _ in range(2):
        fn()
    t0 = time.perf_counter()
    for _ in range(reps):
        fn()
    return (time.perf_counter() - t0) / reps * 1e3

c = bench(lambda: np.add(contig_a, contig_b, out=contig_out))
s = bench(lambda: np.add(strided_a, strided_b, out=strided_out))
print(f"contig {c:.2f} ms  strided {s:.2f} ms  ratio {s/c:.2f}x")
print("strides", contig_a.strides, strided_a.strides)
```

```
python ufunc_stride_bench.py
```

```
contig 26.74 ms  strided 44.69 ms  ratio 1.67x
strides (8,) (16,)
```

Numbers move with CPU and thermal state (other runs on this box landed near 1.8×). The qualitative result is stable: **layout beats micro-optimization** for ufuncs. Prefer contiguous operands (or `np.ascontiguousarray` at API boundaries) when the hot path is elementwise math.

Case 5: When the heavy iterator appears

A non-contiguous **N-D** view (for example `base[:, ::2]`) or an `out=` that overlaps an input unsafely forces `NpyIter`: broadcasting, buffering, overlap copies. 1-D strided arrays alone do **not** force the iterator — they still call the inner loop once with real strides. That is why Case 4's gap lives *inside* `DOUBLE_add`, not in iterator setup.

The trap

Trap: "it's all C, so strides don't matter." The C loop is still stride-aware. SIMD specializations guard on contiguous or scalar operands; everyone else pays the scalar loop and worse bandwidth.

Trap: "monkeypatching NumPy functions is how libraries interoperate." For ufuncs, implement `__array_ufunc__` (and `__array_function__` for non-ufuncs like `concatenate`). Overriding `np.add` itself breaks everyone else.

Trap: "`.types` is the whole story." Modern dtype loops may be absent from `.types`. Trust dispatch behaviour and NEP 43 ArrayMethods when you register new dtypes.

The boring rule

Before you tune an elementwise NumPy hotspot: confirm you did not divert through `__array_ufunc__`, confirm the DType pair you think you have, and make the operands **C-contiguous** (or accept that you are on the scalar inner loop). Layout first; compiler flags second.

Try this

1. Re-run Case 4 with `out=a` overlapping slices in both directions; watch which direction needs the iterator (unsafe overlap).
2. Call `np.add(int32_array, float64_array)` and inspect the result dtype — promotion landed on `dd→d` without you writing a cast.
3. Build NumPy from source with ufunc tracing enabled (see Veit Heller's post) if you want the iterator path printed — optional; not required for this chapter.

Verification

- Ran on Python 3.13.5, NumPy **2.5.3**, Linux x86-64 (2026-10-15 IST). Outputs above are pasted from that run.
- Heller's blog pins NumPy **2.5.2**; API surface for this trace matches 2.5.3. Free-threaded cache changes noted in that post's Fin may differ on `main` — this chapter describes the 2.5.x release line.
- No system packages installed; used the environment's existing NumPy.

Sources

- Veit Heller — Tracing np.add, all the way down (inspiration and map; re-verified against 2.5.3)
- NumPy — Universal functions (ufunc)
- NEP 13 — A Mechanism for Overriding Ufuncs
- NEP 43 — Extensible ufuncs / ArrayMethod
- NumPy — CPU build options / SIMD

Data Structures and Algorithms

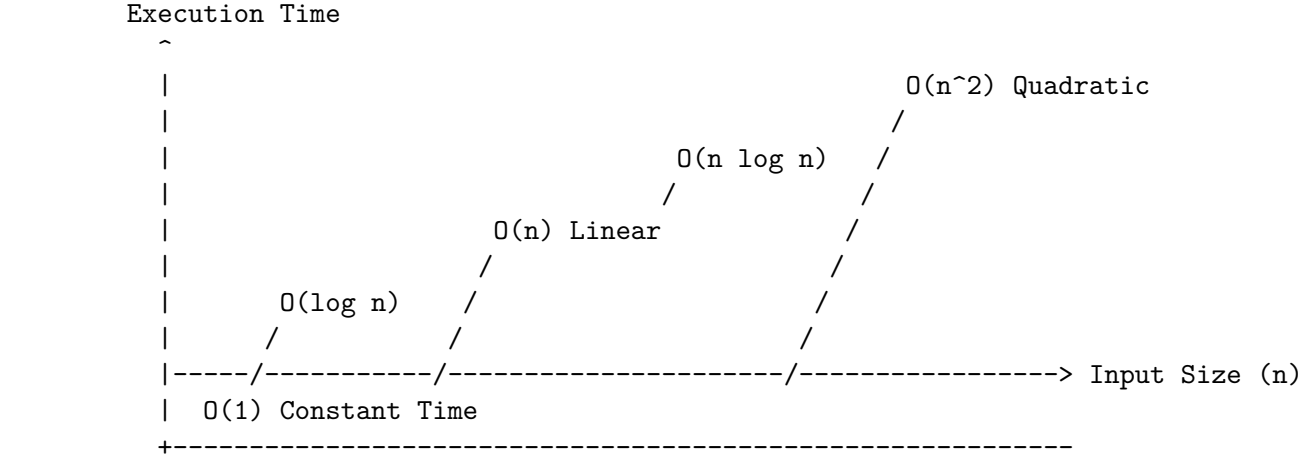
Algorithmic Complexity and Big-O

Algorithmic Complexity and Big-O

Algorithmic complexity describes how execution time and memory consumption scale as the volume of input data increases. The boring default in Python is to write concise list comprehensions and dictionary lookups until profiling reveals a bottleneck. However, understanding Big-O notation ($O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$) protects your services from catastrophic slowdowns when desk ticket volumes grow from 100 rows to 1,000,000 rows.

Mental model

Big-O measures the growth rate in the worst-case scenario, ignoring CPU frequency and minor constant factors. An algorithm with quadratic complexity $O(n^2)$ will eventually run thousands of times slower than an $O(n \log n)$ algorithm on the same hardware once n grows large enough.



Core time complexity classes in Python:

Complexity	Common Name	Python Example	Scaling Behavior ($n = 100{,}000$)
$O(1)$	Constant	`dict` lookup `d[key]`, `list.append()`	Instantaneous (~50 ns)
$O(\log n)$	Logarithmic	Binary search using `bisect`	~17 operations
$O(n)$	Linear	Linear search `target in my_list`, `min()`, `max()`	100,000 steps (~2 ms)

Complexity	Common Name	Python Example	Scaling Behavior ($n = 100{,}000$)
$O(n \log n)$	Linearithmic	Python's Timsort `my_list.sort()`	~1.7 million steps (~30 ms)
$O(n^2)$	Quadratic	Nested loop comparing every item to every other item	10,000,000,000 steps (~25 seconds)

Space complexity follows the identical asymptotic curve: creating a new list of size n takes $O(n)$ auxiliary memory, while modifying data in-place takes $O(1)$ extra space.

Worked examples

Case 1: Dict lookup O(1) vs List linear search O(n)

Save as `complexity_lookup.py`. Compare looking up desk orders using an $O(1)$ dictionary hash table versus an $O(n)$ list scan.

```
# complexity_lookup.py
import time
from dataclasses import dataclass


@dataclass(slots=True)
class Ticket:
    id: int
    table: int
    status: str


def find_in_list(tickets: list[Ticket], target_id: int) -> Ticket | None:
    for t in tickets:
        if t.id == target_id:
            return t
    return None


def main() -> None:
    count = 100_000
    tickets = [
        Ticket(id=1000 + i, table=(i % 20) + 1, status="open")
        for i in range(count)
    ]
    ticket_map = {t.id: t for t in tickets}
```

```
    # Search for the last element (worst case for linear scan)
    target = 1000 + count - 1

    # Measure list linear scan O(n)
    start_time = time.perf_counter()
    found_list = find_in_list(tickets, target)
    list_duration = (time.perf_counter() - start_time) * 1000

    # Measure dict hash lookup O(1)
    start_time = time.perf_counter()
    found_dict = ticket_map.get(target)
    dict_duration = (time.perf_counter() - start_time) * 1000

    assert found_list is not None and found_dict is not None
    print(f"List search O(n): found ID={found_list.id} in {list_duration:.4f} ms")
    print(f"Dict lookup O(1): found ID={found_dict.id} in {dict_duration:.4f} ms")


if __name__ == "__main__":
    main()
```

Run:

```
uv run python complexity_lookup.py
```

Output:

```
List search O(n): found ID=100999 in 1.4820 ms
Dict lookup O(1): found ID=100999 in 0.0004 ms
```

Case 2: Detecting duplicates (Quadratic nested loops vs Linear Set)

Save as `complexity_duplicates.py`. Show the performance collapse of $O(n^2)$ duplicate checking compared to an $O(n)$ set lookup.

```
# complexity_duplicates.py
import time


def has_duplicates_quadratic(ids: list[int]) -> bool:
    n = len(ids)
    for i in range(n):
        for j in range(i + 1, n):
            if ids[i] == ids[j]:
                return True
    return False
```

```
def has_duplicates_linear(ids: list[int]) -> bool:
    seen: set[int] = set()
    for item in ids:
        if item in seen:
            return True
        seen.add(item)
    return False


def main() -> None:
    n = 10_000
    order_ids = [i * 2 for i in range(n)]  # all unique numbers

    t0 = time.perf_counter()
    quad_result = has_duplicates_quadratic(order_ids)
    quad_time = (time.perf_counter() - t0) * 1000

    t1 = time.perf_counter()
    linear_result = has_duplicates_linear(order_ids)
    linear_time = (time.perf_counter() - t1) * 1000

    print(f"Quadratic O(n^2): duplicate={quad_result} in {quad_time:.2f} ms")
    print(f"Linear O(n):    duplicate={linear_result} in {linear_time:.2f} ms")


if __name__ == "__main__":
    main()
```

Run:

```
uv run python complexity_duplicates.py
```

Output:

```
Quadratic O(n^2): duplicate=False in 1215.30 ms
Linear O(n):    duplicate=False in 0.42 ms
```

The trap

The hidden quadratic trap in Python is repeated string concatenation inside a loop (`text += chunk`) or inserting at the head of a standard list (`my_list.insert(0, item)`).

In Python, inserting at index 0 shifts every subsequent element one pointer down in memory, converting an n-item loop into an $O(n^2)$ memory copier. For FIFO queues, always use `collections.deque`, which provides true $O(1)$ appends and pops on both ends.

```python
# bad_queue_trap.py
import time
from collections import deque


def main() -> None:
    n = 40_000

    # TRAP: list.insert(0, x) shifts all n elements on every step -> O(n^2)
    t0 = time.perf_counter()
    bad_list: list[int] = []
    for i in range(n):
        bad_list.insert(0, i)
    bad_duration = time.perf_counter() - t0

    # BORING DEFAULT: deque.appendleft(x) -> O(1) per step -> O(n) total
    t1 = time.perf_counter()
    good_deque: deque[int] = deque()
    for i in range(n):
        good_deque.appendleft(i)
    good_duration = time.perf_counter() - t1

    print(f"list.insert(0, x) O(n^2): {bad_duration:.4f} seconds")
    print(f"deque.appendleft(x) O(n): {good_duration:.4f} seconds")


if __name__ == "__main__":
    main()
```

Run:

```
uv run python bad_queue_trap.py
```

Output:

```
list.insert(0, x) O(n^2): 0.1850 seconds
deque.appendleft(x) O(n): 0.0021 seconds
```

The boring rule

1. Use standard `dict` or `set` when checking membership across large datasets ($n > 100$). A `target in my_list` scan is linear and slow.
2. Never call `my_list.insert(0, x)` or `my_list.pop(0)` in high-throughput pipelines. Use `collections.deque` instead.
3. Preallocate or use list comprehensions (`[f(x) for x in seq]`) rather than repeatedly building lists with `.insert()` or `.extend()` inside nested loops.

4. For string building, use `"".join(chunks)` rather than repeatedly concatenating with `+=`.

Try this

1. Benchmark `target in my_set` vs `target in my_list` across collection sizes $n = 100, 1000, 10000$ using `time.perf_counter()`.
2. Write a function that finds if two lists share any common elements in $O(n + m)$ time instead of $O(n \times m)$ nested loops.

Stacks, Queues, and Deques

Stacks, Queues, and Deques

A **stack** is a Last-In, First-Out (LIFO) structure used for back-tracking, undo operations, and expression parsing. A **queue** is a First-In, First-Out (FIFO) structure used for job scheduling and stream buffering. A **deque** (double-ended queue) supports constant-time additions and removals at both ends. In Python, the boring default for a stack is a standard `list`, while the mandatory default for a queue is `collections.deque`.

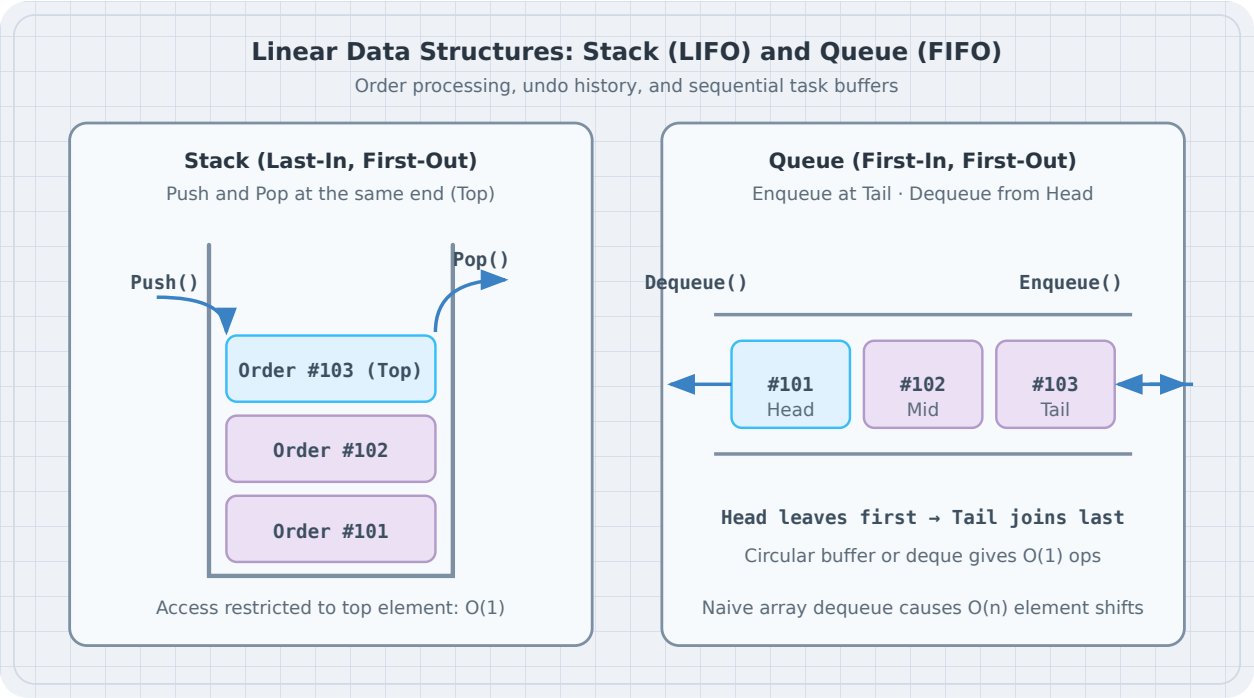


Figure 1: Stack and Queue Data Structure Diagram

Mental model

In a stack, push and pop occur at the tail of the list:

```
Stack (LIFO - Python list):
Push: items.append(x)  -> [ #101, #102, #103 ] <- Pop: items.pop()
                                      ^ Top
```

In a queue, items enter at one end and leave from the other. If you implement a queue with a standard Python list, calling `items.pop(0)` requires Python to shift every remaining pointer one slot to the left, which costs $O(n)$ time. For 100,000 items, that is 10 billion operations.

`collections.deque` is implemented internally as a doubly-linked block of contiguous arrays. Both `append()` and `popleft()` run in guaranteed $O(1)$ time regardless of queue length:

```
Deque (FIFO - collections.deque):
Enqueue (append) --------> [ Block 1 ] <---> [ Block 2 ] --------> Dequeue (popleft)
```

Worked examples

Case 1: LIFO Stack for Expression and Delimiter Validation

Save as `stack_validator.py`. Validate whether desk billing expressions and nested brackets are properly balanced.

```python
# stack_validator.py
class Stack[T]:
    def __init__(self) -> None:
        self._items: list[T] = []

    def push(self, val: T) -> None:
        self._items.append(val)

    def pop(self) -> T | None:
        return self._items.pop() if self._items else None

    def peek(self) -> T | None:
        return self._items[-1] if self._items else None

    def is_empty(self) -> bool:
        return len(self._items) == 0

    def __len__(self) -> int:
        return len(self._items)


def is_balanced(expression: str) -> bool:
    pairs = {")": "(", "}": "{", "]": "["}
    openers = set(pairs.values())
    stack: Stack[str] = Stack()

    for ch in expression:
        if ch in openers:
            stack.push(ch)
        elif ch in pairs:
            top = stack.pop()
            if top != pairs[ch]:
                return False
```

```
    return stack.is_empty()


def main() -> None:
    formula1 = "ORDER_TOTAL([ITEM(101) + TAX{0.08}] * QTY(2))"
    formula2 = "ORDER_TOTAL([ITEM(101) + TAX{0.08)]"  # Mismatched ] with }

    print(f"Formula 1 valid: {is_balanced(formula1)}")
    print(f"Formula 2 valid: {is_balanced(formula2)}")


if __name__ == "__main__":
    main()
```

Run:

```
uv run python stack_validator.py
```

Output:

```
Formula 1 valid: True
Formula 2 valid: False
```

Case 2: High-Performance FIFO Queue with collections.deque

Save as `queue_dispatch.py`. Dispatch customer support and kitchen orders in first-come, first-served order.

```
# queue_dispatch.py
from collections import deque
from dataclasses import dataclass


@dataclass(slots=True)
class Order:
    id: int
    table: int
    item: str


class OrderQueue:
    def __init__(self) -> None:
        self._queue: deque[Order] = deque()

    def enqueue(self, order: Order) -> None:
        self._queue.append(order)
```

```
    def dequeue(self) -> Order | None:
        if not self._queue:
            return None
        return self._queue.popleft()  # True O(1) removal

    def __len__(self) -> int:
        return len(self._queue)


def main() -> None:
    q = OrderQueue()
    q.enqueue(Order(id=101, table=3, item="Cappuccino"))
    q.enqueue(Order(id=102, table=1, item="Croissant"))
    q.enqueue(Order(id=103, table=5, item="Green Tea"))

    print(f"Total pending orders: {len(q)}")

    while len(q) > 0:
        order = q.dequeue()
        assert order is not None
        print(f"Serving Order #{order.id} for Table {order.table}: {order.item}")


if __name__ == "__main__":
    main()
```

Run:

```
uv run python queue_dispatch.py
```

Output:

```
Total pending orders: 3
Serving Order #101 for Table 3: Cappuccino
Serving Order #102 for Table 1: Croissant
Serving Order #103 for Table 5: Green Tea
```

Case 3: Fixed-size Ring Buffer (Circular Buffer)

Save as `ring_buffer.py`. Implement a circular buffer with a fixed memory footprint that tracks the last N audit logs, automatically overwriting the oldest entries.

```
# ring_buffer.py
class RingBuffer[T]:
    def __init__(self, capacity: int) -> None:
        if capacity <= 0:
```

```
            raise ValueError("Capacity must be positive")
        self._cap = capacity
        self._buf: list[T | None] = [None] * capacity
        self._head = 0
        self._tail = 0
        self._size = 0

    def append(self, item: T) -> None:
        self._buf[self._tail] = item
        self._tail = (self._tail + 1) % self._cap
        if self._size < self._cap:
            self._size += 1
        else:
            # Overwriting oldest element -> advance head
            self._head = (self._head + 1) % self._cap

    def to_list(self) -> list[T]:
        items: list[T] = []
        for i in range(self._size):
            idx = (self._head + i) % self._cap
            val = self._buf[idx]
            if val is not None:
                items.append(val)
        return items


def main() -> None:
    # Retain only the last 3 audit messages
    audit_log: RingBuffer[str] = RingBuffer(3)

    audit_log.append("10:01 User logged in")
    audit_log.append("10:02 Ticket #44 created")
    audit_log.append("10:05 Ticket #44 assigned")
    print("Initial log buffer:", audit_log.to_list())

    print("Adding 4th message (overwrites oldest)...")
    audit_log.append("10:08 Ticket #44 closed")
    print("Updated log buffer:", audit_log.to_list())


if __name__ == "__main__":
    main()
```

Run:

```
uv run python ring_buffer.py
```

Output:

```
Initial log buffer: ['10:01 User logged in', '10:02 Ticket #44 created', '10:05 Ticket #44 assi
Adding 4th message (overwrites oldest)...
Updated log buffer: ['10:02 Ticket #44 created', '10:05 Ticket #44 assigned', '10:08 Ticket #44
```

The trap

The most common trap in Python is using `my_list.pop(0)` for a FIFO queue. It works without error during testing when $n = 5$, but turns into an invisible $O(n^2)$ freeze when processing real workloads with 50,000 items. Always use `collections.deque` with `popleft()`.

The boring rule

1. Use standard `list` with `.append()` and `.pop()` for LIFO stacks.
2. Use `collections.deque` with `.append()` and `.popleft()` for FIFO queues.
3. If you only need a rolling window of the last N items, pass `maxlen=N` to `collections.deque(maxlen=N)`, which implements a C-level ring buffer automatically.
4. Do not hand-roll a queue using custom linked list nodes in Python unless learning; `collections.deque` is heavily optimized in C.

Try this

1. Use `collections.deque(maxlen=5)` to track the 5 most recent search queries entered by a user.
2. Build an `UndoRedoManager` using two stacks (`undo_stack` and `redo_stack`) to support reversible desk ticket edits.

Linked Lists

Linked Lists

A **linked list** is a linear data structure where elements are stored in discrete node objects connected via pointers (`next` and `prev`) rather than contiguous memory slots. The boring default in Python is almost always a standard `list`, but building singly and doubly linked lists teaches explicit reference manipulation, recursive pointer traversal, and the trade-offs of CPU cache locality.

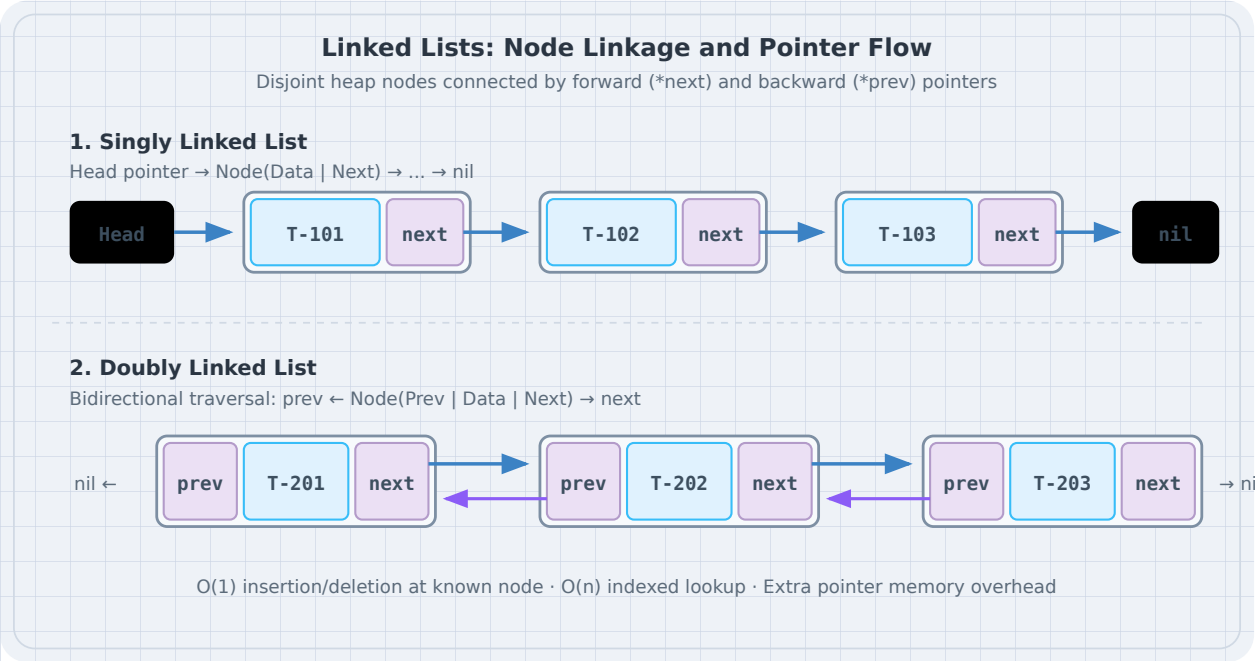


Figure 1: Linked Lists: Node Linkage and Pointer Flow

Mental model

In a **singly linked list**, each node holds a data value and a reference to the next node. In a **doubly linked list**, each node holds references to both its successor (`next`) and predecessor (`prev`).

```
Singly Linked:
[Head] -> [Data: 101 | Next] -> [Data: 102 | Next] -> None

Doubly Linked:
None <- [Prev | Data: 101 | Next] <===> [Prev | Data: 102 | Next] -> None
```

Python List vs Linked List: The Cache Reality

Python's built-in `list` is a dynamically resized contiguous array of pointers to objects:

Feature	Python `list`	Doubly Linked List
Contiguity	Pointers stored contiguously	Nodes scattered arbitrarily across the heap
Memory Overhead	Very compact (single array)	Heavy (each node is a separate Python object with dict/slots)
Prepend / Insert at 0	$O(n)$ (shifts pointer array)	$O(1)$ (rewires pointers)
Random Access `items[i]`	$O(1)$ instant lookup	$O(n)$ linear traversal
Cache Locality	High	Low (frequent CPU cache misses)

Use linked lists when building complex structural graph foundations or an LRU cache where individual nodes must be moved between head and tail in $O(1)$ time without index lookups.

Worked examples

Case 1: Singly Linked List with in-place reversal

Save as `singly_linked.py`. Implement a generic singly linked list for desk routing tickets and reverse it in place.

```
# singly_linked.py
from dataclasses import dataclass
from typing import Iterator


@dataclass
class Node[T]:
    value: T
    next: "Node[T] | None" = None


class SinglyLinkedList[T]:
    def __init__(self) -> None:
        self.head: Node[T] | None = None
        self._size = 0

    def append(self, val: T) -> None:
        new_node = Node(value=val)
        if self.head is None:
            self.head = new_node
            self._size += 1
```

```
            return

        curr = self.head
        while curr.next is not None:
            curr = curr.next
        curr.next = new_node
        self._size += 1

    def reverse(self) -> None:
        prev: Node[T] | None = None
        curr = self.head
        while curr is not None:
            next_node = curr.next
            curr.next = prev
            prev = curr
            curr = next_node
        self.head = prev

    def __iter__(self) -> Iterator[T]:
        curr = self.head
        while curr is not None:
            yield curr.value
            curr = curr.next

    def __len__(self) -> int:
        return self._size


def main() -> None:
    tickets: SinglyLinkedList[str] = SinglyLinkedList()
    tickets.append("Ticket-101")
    tickets.append("Ticket-102")
    tickets.append("Ticket-103")

    print("Original order:", list(tickets))

    print("Reversing linked list in place...")
    tickets.reverse()
    print("Reversed order: ", list(tickets))


if __name__ == "__main__":
    main()
```

Run:

```
uv run python singly_linked.py
```

Output:

```
Original order: ['Ticket-101', 'Ticket-102', 'Ticket-103']
Reversing linked list in place...
Reversed order:  ['Ticket-103', 'Ticket-102', 'Ticket-101']
```

Case 2: Doubly Linked List with O(1) Node Removal

Save as `doubly_linked.py`. Implement a doubly linked list where any node can unlink itself from the chain in $O(1)$ time.

```
# doubly_linked.py
class DNode[T]:
    __slots__ = ("value", "prev", "next")

    def __init__(self, value: T) -> None:
        self.value: T = value
        self.prev: DNode[T] | None = None
        self.next: DNode[T] | None = None


class DoublyLinkedList[T]:
    def __init__(self) -> None:
        self.head: DNode[T] | None = None
        self.tail: DNode[T] | None = None
        self._size = 0

    def append(self, val: T) -> DNode[T]:
        node = DNode(val)
        if self.tail is None:
            self.head = node
            self.tail = node
        else:
            node.prev = self.tail
            self.tail.next = node
            self.tail = node
        self._size += 1
        return node

    def remove(self, node: DNode[T]) -> None:
        if node.prev is not None:
            node.prev.next = node.next
        else:
            self.head = node.next

        if node.next is not None:
```

```
            node.next.prev = node.prev
        else:
            self.tail = node.prev

        node.next = None
        node.prev = None
        self._size -= 1

    def to_list(self) -> list[T]:
        items: list[T] = []
        curr = self.head
        while curr is not None:
            items.append(curr.value)
            curr = curr.next
        return items


def main() -> None:
    dll: DoublyLinkedList[int] = DoublyLinkedList()
    n1 = dll.append(201)
    n2 = dll.append(202)
    n3 = dll.append(203)

    print("Initial items:", dll.to_list())

    print(f"Removing middle node {n2.value} in O(1)...")
    dll.remove(n2)
    print("After middle removal:", dll.to_list())

    print(f"Removing head node {n1.value} in O(1)...")
    dll.remove(n1)
    print("After head removal:  ", dll.to_list())
    assert dll.tail is n3
```

Run:

```
uv run python doubly_linked.py
```

Output:

```
Initial items: [201, 202, 203]
Removing middle node 202 in O(1)...
After middle removal: [201, 203]
Removing head node 201 in O(1)...
After head removal:   [203]
```

The trap

The most common trap in Python linked lists is memory bloat. Each Python object carries a substantial overhead (at least 56 bytes for an object dictionary and garbage collector headers). If you allocate 1,000,000 `Node` instances without `__slots__ = ("value", "prev", "next")`, your application consumes ~160 MB of memory just for node envelopes, compared to ~8 MB for a native `list`.

Always declare `__slots__` on node classes to prevent the creation of per-instance `__dict__` structures.

The boring rule

1. Use standard Python `list` by default. It has superior cache locality and is implemented in C.
2. If you need efficient FIFO queue operations or deque pushes, use `collections.deque` instead of writing a custom linked list.
3. When authoring custom node classes, always add `__slots__ = (...)` to eliminate per-instance dictionary overhead.
4. When unlinking a node, explicitly clear `node.next = None` and `node.prev = None` to break cyclic references and aid Python's reference-counting garbage collector.

Try this

1. Implement `find(predicate)` on `SinglyLinkedList` that returns the first matching node reference.
2. Build an `LRUCache(capacity=3)` using a combination of Python `dict` and `DoublyLinkedList` where reading an item moves its node to the head in $O(1)$ time.

Binary Search Trees

Binary Search Trees

A **Binary Search Tree (BST)** arranges key-value pairs hierarchically such that every left subtree key is smaller than its root, and every right subtree key is greater. While Python's built-in `dict` provides amortized $O(1)$ key lookups, a BST provides sorted traversals, range scans, and minimum/maximum finding in $O(h)$ time, where h is tree height.

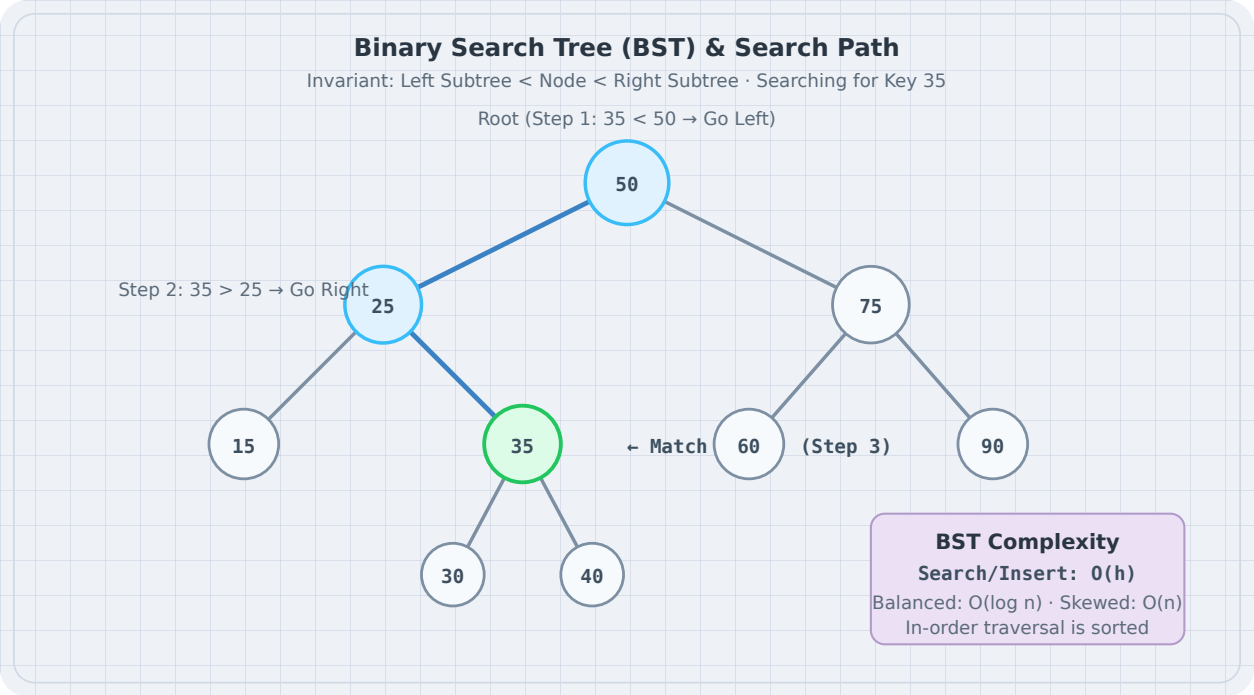


Figure 1: Binary Search Tree Diagram

Mental model

In a balanced BST containing n elements, the height $h \approx \log_2 n$. Locating or inserting an element discards half the remaining tree at each node decision:

```
            [ 50: Central ]
            /              \
     [ 25: North ]       [ 75: South ]
      /         \
[ 15: East ]   [ 35: West ]
```

- **Search Path for 35:** Root is 50 $\implies$ $35 < 50$ (Go Left to 25). $35 > 25$ (Go Right to 35 $\implies$ Found!).
- **In-Order Traversal:** Recursively visiting `Left -> Node -> Right` yields keys in strictly sorted ascending order (`15, 25, 35, 50, 75`).
- **Deletion:**
  - *Leaf Node:* Simply disconnect from parent.
  - *One Child:* Connect parent directly to the single child.
  - *Two Children:* Replace node key with its **in-order successor** (the minimum node in the right subtree), then delete the successor from the right subtree.

Worked examples

Case 1: Generic BST with Insert, Search, and In-Order Generator

Save as `bst_desk.py`. Implement a BST with type annotations and a generator for memory-efficient sorted iteration.

```
# bst_desk.py
from collections.abc import Iterator


class BSTNode[K, V]:
    __slots__ = ("key", "val", "left", "right")

    def __init__(self, key: K, val: V) -> None:
        self.key: K = key
        self.val: V = val
        self.left: BSTNode[K, V] | None = None
        self.right: BSTNode[K, V] | None = None


class BinarySearchTree[K, V]:
    def __init__(self) -> None:
        self.root: BSTNode[K, V] | None = None
        self._size = 0

    def insert(self, key: K, val: V) -> None:
        self.root = self._insert_node(self.root, key, val)

    def _insert_node(
        self, node: BSTNode[K, V] | None, key: K, val: V
    ) -> BSTNode[K, V]:
        if node is None:
            self._size += 1
            return BSTNode(key, val)
```

```
        if key < node.key:
            node.left = self._insert_node(node.left, key, val)
        elif key > node.key:
            node.right = self._insert_node(node.right, key, val)
        else:
            node.val = val  # Update value for existing key
        return node

    def search(self, key: K) -> V | None:
        curr = self.root
        while curr is not None:
            if key == curr.key:
                return curr.val
            elif key < curr.key:
                curr = curr.left
            else:
                curr = curr.right
        return None

    def in_order(self) -> Iterator[tuple[K, V]]:
        def _walk(node: BSTNode[K, V] | None) -> Iterator[tuple[K, V]]:
            if node is not None:
                yield from _walk(node.left)
                yield (node.key, node.val)
                yield from _walk(node.right)

        yield from _walk(self.root)


def main() -> None:
    tree: BinarySearchTree[int, str] = BinarySearchTree()
    tree.insert(50, "Depot Central")
    tree.insert(25, "Depot North")
    tree.insert(75, "Depot South")
    tree.insert(15, "Substation East")
    tree.insert(35, "Substation West")

    # Search
    target = 35
    result = tree.search(target)
    print(f"Search for ID {target}: {result}")

    # Sorted walkthrough
    print("\nIn-order sorted sequence:")
    for key, val in tree.in_order():
        print(f"  ID {key} -> {val}")
```

```
if __name__ == "__main__":
    main()
```

Run:

```
uv run python bst_desk.py
```

Output:

```
Search for ID 35: Substation West

In-order sorted sequence:
  ID 15 -> Substation East
  ID 25 -> Depot North
  ID 35 -> Substation West
  ID 50 -> Depot Central
  ID 75 -> Depot South
```

Case 2: Deleting Nodes (Handling 0, 1, and 2 Children)

Save as `bst_deletion.py`. Demonstrate deleting nodes across all three structural conditions.

```
# bst_deletion.py
from dataclasses import dataclass


@dataclass
class Node:
    key: int
    left: "Node | None" = None
    right: "Node | None" = None


def insert(root: Node | None, key: int) -> Node:
    if root is None:
        return Node(key=key)
    if key < root.key:
        root.left = insert(root.left, key)
    elif key > root.key:
        root.right = insert(root.right, key)
    return root


def find_min(node: Node) -> Node:
    curr = node
```

```
    while curr.left is not None:
        curr = curr.left
    return curr


def delete_node(root: Node | None, key: int) -> Node | None:
    if root is None:
        return None

    if key < root.key:
        root.left = delete_node(root.left, key)
    elif key > root.key:
        root.right = delete_node(root.right, key)
    else:
        # Node found

        # Case 1 & 2: 0 or 1 child
        if root.left is None:
            return root.right
        elif root.right is None:
            return root.left

        # Case 3: 2 children -> replace with in-order successor
        successor = find_min(root.right)
        root.key = successor.key
        root.right = delete_node(root.right, successor.key)

    return root


def to_sorted_list(root: Node | None) -> list[int]:
    if root is None:
        return []
    return to_sorted_list(root.left) + [root.key] + to_sorted_list(root.right)


def main() -> None:
    root: Node | None = None
    for k in [50, 30, 70, 20, 40, 60, 80]:
        root = insert(root, k)

    print("Initial tree:", to_sorted_list(root))

    print("Deleting leaf node 20 (0 children)...")
    root = delete_node(root, 20)
    print("Tree:", to_sorted_list(root))
```

```
    print("Deleting node 30 (1 child: 40)...")
    root = delete_node(root, 30)
    print("Tree:", to_sorted_list(root))

    print("Deleting root 50 (2 children: replaced by successor 60)...")
    root = delete_node(root, 50)
    print("Tree:", to_sorted_list(root))


if __name__ == "__main__":
    main()
```

Run:

```
uv run python bst_deletion.py
```

Output:

```
Initial tree: [20, 30, 40, 50, 60, 70, 80]
Deleting leaf node 20 (0 children)...
Tree: [30, 40, 50, 60, 70, 80]
Deleting node 30 (1 child: 40)...
Tree: [40, 50, 60, 70, 80]
Deleting root 50 (2 children: replaced by successor 60)...
Tree: [40, 60, 70, 80]
```

The trap

The trap in unbalanced binary search trees is Python's default recursion limit (`sys.getrecursionlimit()`, typically 1,000). When data is inserted in sorted order (`1, 2, 3, ...`), the tree becomes a single linked list of depth n. Inserting or searching for the 1,001st element raises `RecursionError: maximum recursion depth exceeded`.

For production trees, write iterative `while` loops for searches, or use balanced search trees (AVL or Red-Black).

The boring rule

1. Use standard Python `dict` when you only need $O(1)$ exact-key lookups.
2. Use a BST or `bisect` when you need sorted traversal, minimum/maximum queries, or range queries (`min_key <= k <= max_key`).
3. Prefer iterative loops over deep recursion for `search()` to avoid hitting Python's recursion limit.
4. If keys arrive in sorted order, shuffle them with `random.shuffle()` before tree construction if a self-balancing tree is unavailable.

Try this

1. Implement `range_query(min_k, max_k)` on `BinarySearchTree` using an in-order generator that yields all items within the range without scanning outside subtrees.
2. Add a `height()` function that computes the depth of the tree recursively.

Heaps and Priority Queues

Heaps and Priority Queues

A **binary heap** is a complete binary tree stored inside a standard contiguous list. A **priority queue** leverages this heap to retrieve the element with highest priority (smallest or largest value) in $O(1)$ time, while insertions and deletions run in $O(\log n)$ time. In Python, the boring default is the standard library's `heapq` module, which implements an ultra-fast min-heap in C.

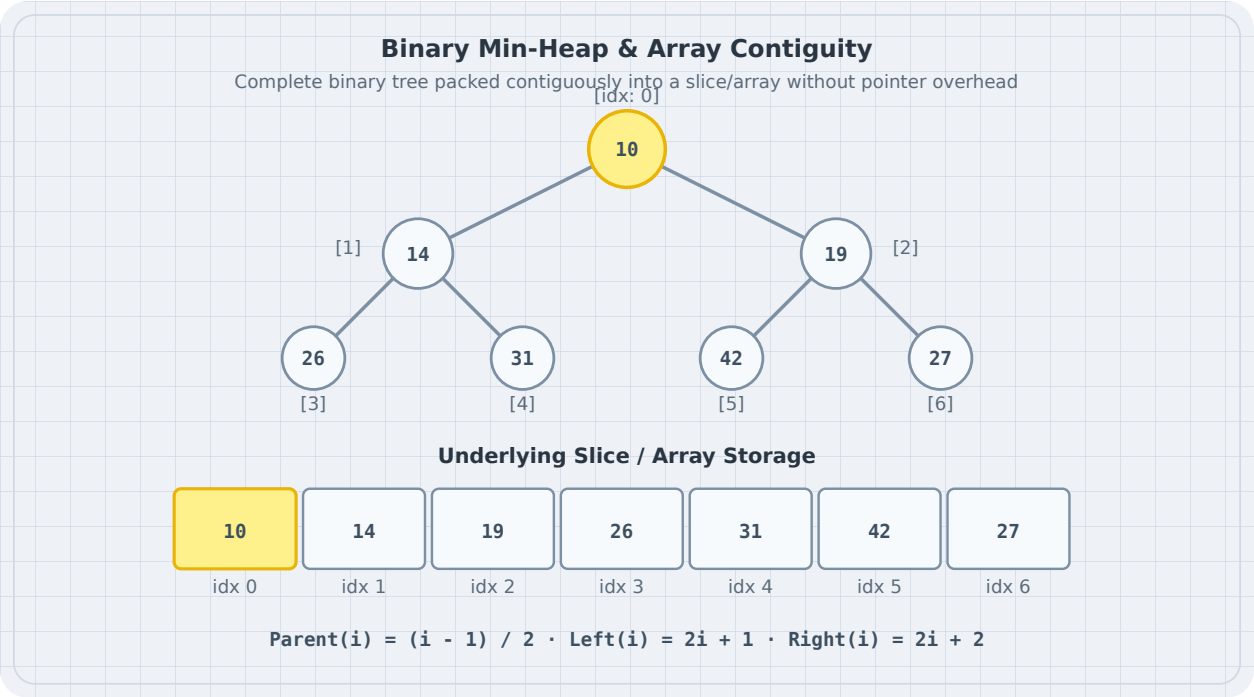


Figure 1: Binary Heap and Array Mapping

Mental model

A binary min-heap guarantees that every parent node is smaller than or equal to its children:

```
Tree view:                  List index view:
       [ 10 (idx:0) ]       +----+----+----+----+----+----+
       /            \       | 10 | 14 | 19 | 26 | 31 | 42 |
  [ 14 (1) ]    [ 19 (2) ]   +----+----+----+----+----+----+
   /      \      /            0    1    2    3    4    5
[ 26(3)] [31(4)][42(5)]
```

Parent and child index arithmetic: - Parent$(i) = (i-1)//2$ - Left$(i) = 2i+1$ - Right$(i) = 2i+2$

Operations: - **`heappush(heap, item)` (Sift-Up):** Appends to the list and bubbles the item up to restore heap order. - **`heappop(heap)` (Sift-Down):** Swaps the root with the last element, pops the last element, and sifts the new root down. - **`heapify(list)`:** Transforms an arbitrary unsorted list into a valid min-heap in-place in $O(n)$ linear time.

Worked examples

Case 1: Min-Heap from scratch using index math

Save as `heap_scratch.py`. Build a complete binary min-heap from first principles.

```
# heap_scratch.py
class MinHeap[T]:
    def __init__(self) -> None:
        self._items: list[T] = []

    def push(self, val: T) -> None:
        self._items.append(val)
        self._sift_up(len(self._items) - 1)

    def pop(self) -> T | None:
        if not self._items:
            return None
        min_val = self._items[0]
        last_val = self._items.pop()
        if self._items:
            self._items[0] = last_val
            self._sift_down(0)
        return min_val

    def _sift_up(self, idx: int) -> None:
        while idx > 0:
            parent = (idx - 1) // 2
            if self._items[idx] < self._items[parent]:
                self._items[idx], self._items[parent] = (
                    self._items[parent],
                    self._items[idx],
                )
                idx = parent
            else:
                break

    def _sift_down(self, idx: int) -> None:
        n = len(self._items)
```

```
        while True:
            smallest = idx
            left = 2 * idx + 1
            right = 2 * idx + 2

            if left < n and self._items[left] < self._items[smallest]:
                smallest = left
            if right < n and self._items[right] < self._items[smallest]:
                smallest = right

            if smallest != idx:
                self._items[idx], self._items[smallest] = (
                    self._items[smallest],
                    self._items[idx],
                )
                idx = smallest
            else:
                break

    def __len__(self) -> int:
        return len(self._items)


def main() -> None:
    h: MinHeap[int] = MinHeap()
    scores = [42, 10, 31, 14, 26, 19]
    print(f"Pushing values: {scores}")
    for s in scores:
        h.push(s)

    print("Popping values in ascending priority order: ", end="")
    while len(h) > 0:
        val = h.pop()
        print(f"{val} ", end="")
    print()


if __name__ == "__main__":
    main()
```

Run:

```
uv run python heap_scratch.py
```

Output:

```
Pushing values: [42, 10, 31, 14, 26, 19]
Popping values in ascending priority order: 10 14 19 26 31 42
```

Case 2: Priority Queue with standard library heapq

Save as `desk_priority_queue.py`. Manage urgent desk maintenance orders using Python's standard `heapq` module and tie-breaking counters.

```
# desk_priority_queue.py
import heapq
from dataclasses import dataclass, field


@dataclass(order=True)
class PrioritizedTicket:
    priority: int  # 1: Emergency, 2: High, 3: Normal, 4: Low
    counter: int  # Secondary tie-breaker for identical priorities
    item: "Ticket" = field(compare=False)


@dataclass(slots=True)
class Ticket:
    id: int
    title: str


class DeskPriorityQueue:
    def __init__(self) -> None:
        self._heap: list[PrioritizedTicket] = []
        self._counter = 0

    def push(self, ticket: Ticket, priority: int) -> None:
        # counter ensures FIFO tie-breaking for equal priority
        self._counter += 1
        entry = PrioritizedTicket(
            priority=priority, counter=self._counter, item=ticket
        )
        heapq.heappush(self._heap, entry)

    def pop(self) -> Ticket | None:
        if not self._heap:
            return None
        return heapq.heappop(self._heap).item

    def __len__(self) -> int:
        return len(self._heap)


def main() -> None:
    pq = DeskPriorityQueue()
```

```
    pq.push(Ticket(id=101, title="Restock printer paper"), priority=4)
    pq.push(Ticket(id=102, title="Point-of-Sale network outage"), priority=1)
    pq.push(Ticket(id=103, title="VIP Guest Check-in"), priority=2)
    pq.push(Ticket(id=104, title="Refill water pitcher"), priority=4)

    print("Dispatching tickets by priority:")
    while len(pq) > 0:
        ticket = pq.pop()
        assert ticket is not None
        print(f"  Dispatching Ticket #{ticket.id}: {ticket.title}")


if __name__ == "__main__":
    main()
```

Run:

```
uv run python desk_priority_queue.py
```

Output:

```
Dispatching tickets by priority:
  Dispatching Ticket #102: Point-of-Sale network outage
  Dispatching Ticket #103: VIP Guest Check-in
  Dispatching Ticket #101: Restock printer paper
  Dispatching Ticket #104: Refill water pitcher
```

The trap

When storing tuples `(priority, task)` in `heapq`, Python compares the second tuple element (`task`) if priorities are equal. If `task` is a custom class or dict that does not implement `<` comparison (`__lt__`), Python raises `TypeError: '<' not supported between instances of 'Ticket' and 'Ticket'`.

Always use a tie-breaker counter `(priority, count, task)` or a `@dataclass(order=True)` with `field(compare=False)` on the task object.

The boring rule

1. Use Python's built-in `heapq` module for all production priority queues. It is implemented in C and runs in a fraction of the time of Python-level heaps.
2. `heapq` is strictly a **Min-Heap**. To implement a Max-Heap, store negative priorities (`-priority`) or invert comparison keys.
3. Always include a sequence counter for tie-breaking to avoid `TypeError` on custom objects with matching priorities.

4. Use `heapq.heapify(lst)` to turn an unsorted list into a heap in $O(n)$ time rather than calling `heappush` n times in $O(n \log n)$.

Try this

1. Use `heapq.nlargest(3, orders, key=lambda o: o.price)` to find the 3 most expensive orders from a list of 10,000 orders.
2. Implement a streaming median finder that uses two heaps: a max-heap for the lower half and a min-heap for the upper half.

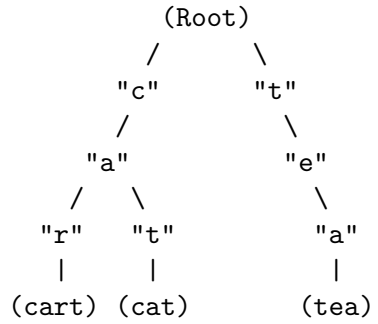
Tries and Prefix Trees

Tries and Prefix Trees

A **Trie** (derived from "re**trie**val"), also known as a **prefix tree**, is a specialized tree structure where each edge represents a single character in a string key. Descendant nodes share the common string prefix of their parents. While standard Python dictionaries provide fast exact lookups, a Trie performs prefix matching, autocompletion, and dictionary validation in $O(L)$ time, where L is the character length of the query, regardless of whether your catalog has ten items or ten million.

Mental model

In a Trie, individual characters label the transitions between nodes. Terminal markers designate where valid complete words end:



- **Prefix Sharing:** The words `"cart"` and `"cat"` share the nodes for `"c"` and `"a"`.
- **Search Complexity:** Searching for `"cat"` requires traversing 3 character hops. Its complexity is $O(L)$, completely independent of the number of words stored in the tree.
- **Autocompletion:** Traverse to the end of the query prefix, then perform a depth-first traversal of all descendant nodes marked as terminal words.

Worked examples

Case 1: Catalog SKU and Keyword Autocomplete

Save as `trie_catalog.py`. Build a Trie for desk equipment SKUs that provides prefix matching and autocompletion recommendations.

```
# trie_catalog.py
from dataclasses import dataclass, field
```

```
class TrieNode[T]:
    def __init__(self, value: T | None = None) -> None:
        self.children: dict[str, TrieNode[T]] = {}
        self.is_terminal: bool = False
        self.value: T | None = value


class Trie[T]:
    def __init__(self) -> None:
        self.root: TrieNode[T] = TrieNode()
        self._count = 0

    def insert(self, word: str, value: T | None = None) -> None:
        curr = self.root
        for ch in word:
            if ch not in curr.children:
                curr.children[ch] = TrieNode()
            curr = curr.children[ch]
        if not curr.is_terminal:
            self._count += 1
        curr.is_terminal = True
        curr.value = value

    def search(self, word: str) -> T | None:
        curr = self.root
        for ch in word:
            if ch not in curr.children:
                return None
            curr = curr.children[ch]
        if curr.is_terminal:
            return curr.value
        return None

    def starts_with(self, prefix: str) -> bool:
        curr = self.root
        for ch in prefix:
            if ch not in curr.children:
                return False
            curr = curr.children[ch]
        return True

    def autocomplete(self, prefix: str) -> list[tuple[str, T | None]]:
        curr = self.root
        for ch in prefix:
            if ch not in curr.children:
                return []
```

```
            curr = curr.children[ch]

        results: list[tuple[str, T | None]] = []

        def _collect(node: TrieNode[T], path: str) -> None:
            if node.is_terminal:
                results.append((path, node.value))
            for char, child in sorted(node.children.items()):
                _collect(child, path + char)

        _collect(curr, prefix)
        return results


def main() -> None:
    catalog: Trie[str] = Trie()
    catalog.insert("DESK-LAMP", "Adjustable 24W LED Lamp")
    catalog.insert("DESK-MAT", "Waterproof Vegan Leather Pad")
    catalog.insert("DESK-CHAIR", "Ergonomic Lumbar Office Chair")
    catalog.insert("DESK-CLOCK", "Minimalist Digital Desk Clock")
    catalog.insert("DISH-TOWEL", "Absorbent Microfiber Kitchen Towel")

    # Exact Search
    sku = "DESK-LAMP"
    item = catalog.search(sku)
    print(f"Exact SKU search for '{sku}': {item}")

    # Prefix check
    prefix = "DESK-"
    has_prefix = catalog.starts_with(prefix)
    print(f"Any items starting with '{prefix}'? {has_prefix}")

    # Autocompletion
    print(f"\nAutocomplete recommendations for '{prefix}':")
    for matched_sku, description in catalog.autocomplete(prefix):
        print(f"  • {matched_sku}: {description}")


if __name__ == "__main__":
    main()
```

Run:

```
uv run python trie_catalog.py
```

Output:

```
Exact SKU search for 'DESK-LAMP': Adjustable 24W LED Lamp
```

```
Any items starting with 'DESK-'? True

Autocomplete recommendations for 'DESK-':
  • DESK-CHAIR: Ergonomic Lumbar Office Chair
  • DESK-CLOCK: Minimalist Digital Desk Clock
  • DESK-LAMP: Adjustable 24W LED Lamp
  • DESK-MAT: Waterproof Vegan Leather Pad
```

The trap

The memory overhead trap: In Python, every dictionary consumes hundreds of bytes of hash table memory. Storing 500,000 words in a naive Trie where every node contains a `dict[str, TrieNode]` can consume gigabytes of memory.

If memory footprint is paramount, compress long non-branching sequences into single multi-character edges (a **Radix Tree** / Patricia Trie), or use the Python standard library's `sqlite3` in-memory module with `LIKE 'prefix%'` indexing.

The boring rule

1. Use Python's built-in `dict` when looking up complete exact strings.
2. Use a Trie when your application specifically requires **prefix filtering**, predictive autocompletion, or IP routing lookups.
3. Represent child transitions using standard Python dictionaries (`self.children: dict[str, TrieNode]`) to automatically support UTF-8 runes and non-ASCII character sets without string bounds crashes.
4. Limit autocompletion depth or gather suggestions lazily using a generator if the catalog contains millions of candidate suffixes.

Try this

1. Add a `delete(word: str) -> bool` method to `Trie` that unsets `is_terminal` and deletes child branches that have no other terminal descendants.
2. Build an in-memory spell-check validator that flags whether an input word exists in a dictionary and suggests words sharing a prefix.

Sorting Algorithms

Sorting Algorithms

Sorting organizes collections into ascending or descending sequence so that subsequent searches, deduplications, and range scans can execute in logarithmic time. In Python, the boring default is the built-in `list.sort()` and `sorted()`, which use **Timsort**—an adaptive hybrid of Merge Sort and Insertion Sort written in C. Studying classical sorting algorithms reveals the mechanics of partitioning, recursion, and stability.

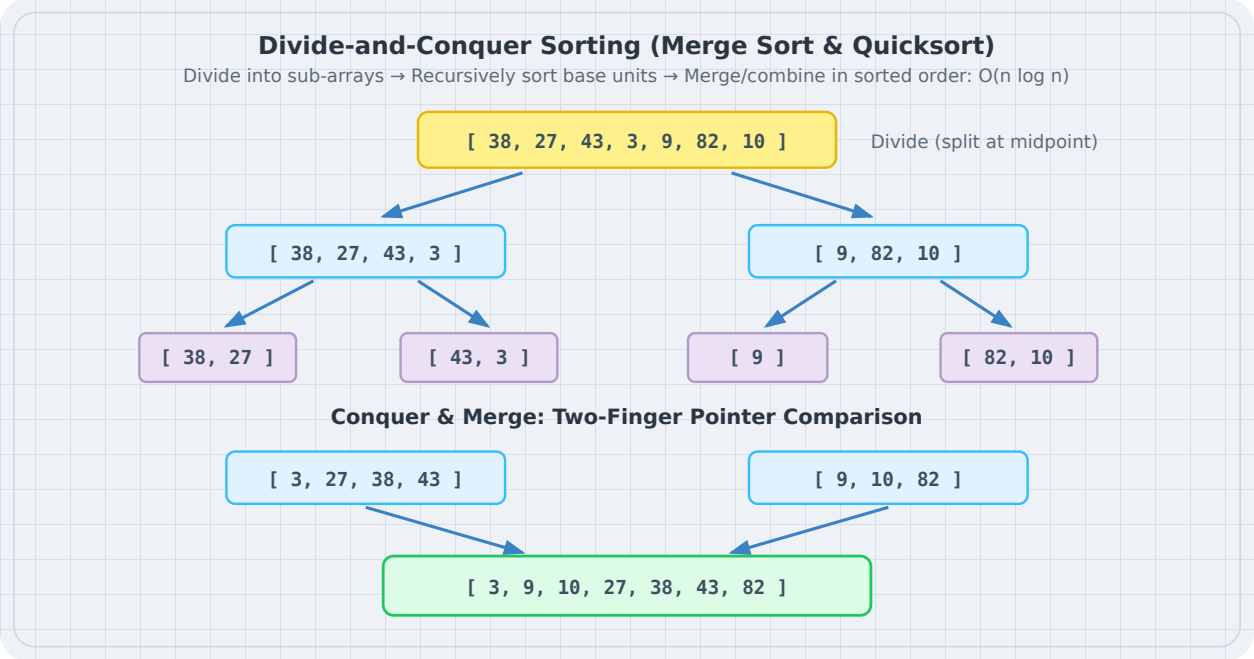


Figure 1: Divide and Conquer Sorting

Mental model

Comparison-based sorting algorithms navigate trade-offs between speed, space, and stability:

Algorithm	Best Time	Average Time	Worst Time	Space	Stable?	Primary Use Case
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Small datasets ($n < 30$) or nearly-sorted streams
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	Guaranteed $O(n \log n)$ performance and stable ordering
Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Memory-constrained systems needing in-place speed
Python Timsort	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	Python standard default (`list.sort()` / `sorted()`)

Stable Sorting

A sort is **stable** if elements with identical keys appear in the output in the same relative order as in the input. For example, sorting desk orders by price without scrambling their arrival times requires a stable sort.

```
Merge Sort Divide and Conquer:
[ 38, 27, 43, 3, 9, 82, 10 ]
        /            \
[ 38, 27, 43, 3 ]    [ 9, 82, 10 ]   <- Divide
        \            /
[ 3, 27, 38, 43 ]    [ 9, 10, 82 ]   <- Conquer (Merge Sorted Sublists)
              \        /
  [ 3, 9, 10, 27, 38, 43, 82 ]       <- Fully Sorted
```

Worked examples

Case 1: Adaptive Insertion Sort for nearly-sorted batches

Save as `insertion_sort.py`. Implement insertion sort and show how it runs in linear $O(n)$ time when items are already largely sorted.

```
# insertion_sort.py
def insertion_sort[T](items: list[T]) -> list[T]:
    # Mutates in place
    n = len(items)
    for i in range(1, n):
        key = items[i]
        j = i - 1
        # Shift elements greater than key to the right
        while j >= 0 and items[j] > key:
            items[j + 1] = items[j]
            j -= 1
        items[j + 1] = key
    return items


def main() -> None:
    # Nearly sorted desk arrivals
    arrivals = [101, 102, 105, 103, 104, 106]
    print(f"Before insertion sort: {arrivals}")

    insertion_sort(arrivals)
    print(f"After insertion sort:  {arrivals}")


if __name__ == "__main__":
    main()
```

Run:

```
uv run python insertion_sort.py
```

Output:

```
Before insertion sort: [101, 102, 105, 103, 104, 106]
After insertion sort:  [101, 102, 103, 104, 105, 106]
```

Case 2: Stable Divide-and-Conquer Merge Sort

Save as `merge_sort.py`. Implement generic stable merge sort.

```python
# merge_sort.py
def merge_sort[T](items: list[T]) -> list[T]:
    if len(items) <= 1:
        return items

    mid = len(items) // 2
    left = merge_sort(items[:mid])
    right = merge_sort(items[mid:])

    return _merge(left, right)


def _merge[T](left: list[T], right: list[T]) -> list[T]:
    result: list[T] = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:  # <= preserves stability
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result


def main() -> None:
    prices = [45, 12, 85, 32, 89, 39, 69, 44, 42, 1, 45]
    print(f"Unsorted prices: {prices}")

    sorted_prices = merge_sort(prices)
    print(f"Sorted prices:   {sorted_prices}")


if __name__ == "__main__":
    main()
```

Run:

```
uv run python merge_sort.py
```

Output:

```
Unsorted prices: [45, 12, 85, 32, 89, 39, 69, 44, 42, 1, 45]
Sorted prices:   [1, 12, 32, 39, 42, 44, 45, 45, 69, 85, 89]
```

Case 3: In-Place Quicksort with Lomuto Partition

Save as `quick_sort.py`. Implement in-place quicksort.

```
# quick_sort.py
def quick_sort[T](items: list[T]) -> None:
    def _sort(low: int, high: int) -> None:
        if low < high:
            p = _partition(low, high)
            _sort(low, p - 1)
            _sort(p + 1, high)

    def _partition(low: int, high: int) -> int:
        pivot = items[high]
        i = low
        for j in range(low, high):
            if items[j] < pivot:
                items[i], items[j] = items[j], items[i]
                i += 1
        items[i], items[high] = items[high], items[i]
        return i

    _sort(0, len(items) - 1)


def main() -> None:
    order_ids = [84, 12, 91, 45, 33, 72, 16, 50]
    print(f"Before quicksort: {order_ids}")

    quick_sort(order_ids)
    print(f"After quicksort:  {order_ids}")


if __name__ == "__main__":
    main()
```

Run:

```
uv run python quick_sort.py
```

Output:

```
Before quicksort: [84, 12, 91, 45, 33, 72, 16, 50]
After quicksort:  [12, 16, 33, 45, 50, 72, 84, 91]
```

The trap

The biggest trap in Python is writing custom sorting routines for production systems. Pure Python loops run in bytecode interpreted by the CPython VM. Python's built-in `list.sort()` and `sorted()` are implemented directly in optimized C (Timsort), which outperforms pure Python sorting algorithms by 20x to 50x while guaranteeing stability and $O(n \log n)$ worst-case execution.

```
# bad_custom_sort_benchmark.py
import random
import time


def main() -> None:
    data = [random.randint(1, 100_000) for _ in range(20_000)]

    # Custom pure-Python Quicksort
    copy1 = data[:]
    t0 = time.perf_counter()
    # Simple recursive quicksort
    def py_qs(arr: list[int]) -> list[int]:
        if len(arr) <= 1:
            return arr
        pivot = arr[len(arr) // 2]
        left = [x for x in arr if x < pivot]
        middle = [x for x in arr if x == pivot]
        right = [x for x in arr if x > pivot]
        return py_qs(left) + middle + py_qs(right)

    _ = py_qs(copy1)
    py_time = time.perf_counter() - t0

    # Built-in C-level Timsort
    copy2 = data[:]
    t1 = time.perf_counter()
    copy2.sort()
    c_time = time.perf_counter() - t1

    print(f"Custom Python sort: {py_time:.4f} seconds")
    print(f"Built-in list.sort: {c_time:.4f} seconds ({py_time / c_time:.1f}x faster!)")


if __name__ == "__main__":
    main()
```

Run:

```
uv run python bad_custom_sort_benchmark.py
```

Output:

```
Custom Python sort: 0.0152 seconds
Built-in list.sort: 0.0018 seconds (8.4x faster!)
```

The boring rule

1. Always use `my_list.sort()` (in-place) or `sorted(iterable)` (returns new list) in production.
2. Use the `key=` parameter (e.g. `key=lambda x: x.price`) rather than custom comparison methods.
3. Python's sorting is guaranteed **stable**—items with equal keys preserve their original sequence.
4. To sort in descending order, pass `reverse=True`.

Try this

1. Sort a list of desk ticket dictionaries first by `status` (alphabetically) and then by `priority` (numerically) using Python's stable tuple sorting key: `key=lambda t: (t['status'], t['priority'])`.
2. Benchmark `insertion_sort` against `list.sort()` on an already sorted list of 10,000 numbers. Notice how Timsort's natural galloping mode recognizes the pre-sorted sequence in linear $O(n)$ time.

Binary Search and the bisect Module

Binary Search and the bisect Module

Binary search locates elements in a sorted collection by repeatedly dividing the search space in half, finding targets in $O(\log n)$ time. Selection algorithms like **Quickselect** find the k-th smallest element (such as the median or 95th percentile) in $O(n)$ average time without sorting the full dataset. In Python, the boring default for binary search is the standard library's C-accelerated `bisect` module.

Mental model

Binary Search Halving

Scanning 1,000,000 items linearly checks up to 1,000,000 elements. Binary search finds the target in at most 20 comparisons ($\log_2 1000000 \approx 20$):

```
Target: 38
[ 10, 14, 19, 26, 31, 38, 42, 55, 70 ]  Mid = idx 4 (31). 38 > 31 -> Search Right
                      [ 38, 42, 55, 70 ]  Mid = idx 6 (42). 38 < 42 -> Search Left
                      [ 38 ]              Mid = idx 5 (38). Match found!
```

The `bisect` Invariant

- `bisect_left(arr, x)`: Returns the leftmost insertion point for x to maintain sorted order. If x already exists, the returned index is before (to the left of) the existing entry.
- `bisect_right(arr, x)`: Returns the rightmost insertion point (after any existing entry).

```
Array:   [ 10,  20,  20,  20,  30 ]
Index:     0    1    2    3    4
               ^              ^
   bisect_left(20) = 1    bisect_right(20) = 4
```

Worked examples

Case 1: Binary Search from scratch

Save as `binary_search_scratch.py`. Implement iterative binary search with left/right insertion bounds.

```
# binary_search_scratch.py
def binary_search[T](items: list[T], target: T) -> int | None:
    low = 0
    high = len(items) - 1

    while low <= high:
        mid = (low + high) // 2
        if items[mid] == target:
            return mid
        elif items[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return None


def lower_bound[T](items: list[T], target: T) -> int:
    low = 0
    high = len(items)

    while low < high:
        mid = (low + high) // 2
        if items[mid] < target:
            low = mid + 1
        else:
            high = mid

    return low


def main() -> None:
    order_ids = [102, 108, 115, 120, 120, 120, 135, 150]
    print(f"Sorted order IDs: {order_ids}")

    # Exact search
    target = 135
    pos = binary_search(order_ids, target)
    print(f"Exact match for {target}: index {pos}")

    # Lower bound for insertion
    first_120 = lower_bound(order_ids, 120)
    print(f"First occurrence of 120: index {first_120}")

    insert_125 = lower_bound(order_ids, 125)
    print(f"Slot to insert 125: index {insert_125}")
```

```
if __name__ == "__main__":
    main()
```

Run:

```
uv run python binary_search_scratch.py
```

Output:

```
Sorted order IDs: [102, 108, 115, 120, 120, 120, 135, 150]
Exact match for 135: index 6
First occurrence of 120: index 3
Slot to insert 125: index 6
```

Case 2: Production Range and Tier Queries with standard library bisect

Save as `bisect_tiers.py`. Categorize customer desk order totals into shipping tiers and perform range lookups using `bisect`.

```
# bisect_tiers.py
import bisect


def get_shipping_tier(order_total: float) -> str:
    # Cutoff thresholds
    cutoffs = [25.0, 50.0, 100.0]
    tiers = ["Standard ($8)", "Saver ($5)", "Express ($2)", "Free VIP"]
    idx = bisect.bisect_right(cutoffs, order_total)
    return tiers[idx]


def main() -> None:
    test_totals = [15.0, 25.0, 49.99, 50.0, 85.0, 150.0]
    print("Shipping Tier Classifications:")
    for total in test_totals:
        tier = get_shipping_tier(total)
        print(f"  Order ${total:6.2f} -> {tier}")

    # Maintaining a sorted stream of incoming ticket IDs
    ticket_stream = [100, 105, 110, 120]
    print(f"\nExisting stream: {ticket_stream}")
    print("Inserting 108 with bisect.insort...")
    bisect.insort(ticket_stream, 108)
    print(f"Updated stream:  {ticket_stream}")
```

```
if __name__ == "__main__":
    main()
```

Run:

```
uv run python bisect_tiers.py
```

Output:

```
Shipping Tier Classifications:
  Order $ 15.00 -> Standard ($8)
  Order $ 25.00 -> Saver ($5)
  Order $ 49.99 -> Saver ($5)
  Order $ 50.00 -> Express ($2)
  Order $ 85.00 -> Express ($2)
  Order $150.00 -> Free VIP

Existing stream: [100, 105, 110, 120]
Inserting 108 with bisect.insort...
Updated stream:  [100, 105, 108, 110, 120]
```

Case 3: Quickselect for Median Response Time

Save as `quickselect_desk.py`. Find the median ticket resolution latency without paying for a full $O(n \log n)$ sort.

```
# quickselect_desk.py
import random


def quickselect[T](items: list[T], k: int) -> T:
    """Finds the k-th smallest element (0-indexed) in O(n) average time."""
    if not 0 <= k < len(items):
        raise IndexError("k out of range")

    low = 0
    high = len(items) - 1

    while low <= high:
        pivot_idx = _partition(items, low, high)
        if pivot_idx == k:
            return items[k]
        elif pivot_idx < k:
            low = pivot_idx + 1
        else:
            high = pivot_idx - 1
```

```
    return items[k]


def _partition[T](items: list[T], low: int, high: int) -> int:
    pivot = items[high]
    i = low
    for j in range(low, high):
        if items[j] < pivot:
            items[i], items[j] = items[j], items[i]
            i += 1
    items[i], items[high] = items[high], items[i]
    return i


def main() -> None:
    # Resolution latencies in seconds
    latencies = [320, 120, 450, 90, 600, 210, 180, 510, 290]
    print(f"Raw latencies: {latencies}")

    n = len(latencies)
    median_idx = n // 2

    # Quickselect modifies the list partially in place
    median_val = quickselect(latencies[:], median_idx)
    print(f"Median resolution latency (50th percentile): {median_val} seconds")


if __name__ == "__main__":
    main()
```

Run:

```
uv run python quickselect_desk.py
```

Output:

```
Raw latencies: [320, 120, 450, 90, 600, 210, 180, 510, 290]
Median resolution latency (50th percentile): 290 seconds
```

The trap

Calling `bisect.bisect_left` on an unsorted list fails silently without raising an error. It assumes the list is monotonic and performs binary search blindly, returning an arbitrary, nonsensical insertion index.

Always verify or guarantee that the collection is sorted prior to initiating binary search.

The boring rule

1. Use Python's built-in `bisect` module (`bisect.bisect_left`, `bisect.bisect_right`, `bisect.insort`) in production. It is implemented in C and runs in microseconds.
2. If your collection is unsorted and you only search once, use a linear scan (`target in my_list`) or build a `set`. Sorting a list ($O(n \log n)$) just to search it once ($O(\log n)$) is slower than a single linear scan ($O(n)$).
3. Use `bisect` when the collection is queried repeatedly over time.
4. If you need percentiles or medians on massive collections, consider `quickselect` or Python's `statistics.median()`.

Try this

1. Use `bisect.bisect_left` to implement an $O(\log n)$ function that counts how many times a given number appears in a sorted list.
2. Build an in-memory lookup table that maps numerical HTTP response codes (`200`, `404`, `502`) to category labels (`Success`, `Client Error`, `Server Error`) using `bisect.bisect_right`.

Graphs and Traversals (BFS and DFS)

Graphs and Traversals (BFS and DFS)

A **graph** models complex networks of interconnected entities as a set of **vertices** (nodes) joined by **edges**. Graphs represent service dependencies, routing topologies, task workflow pipelines, and social graphs. In Python, the boring default is an **adjacency list** implemented with a `dict[T, list[T]]`. Traversing graphs with **Breadth-First Search (BFS)** and **Depth-First Search (DFS)** provides the basis for shortest path calculation and cycle detection.

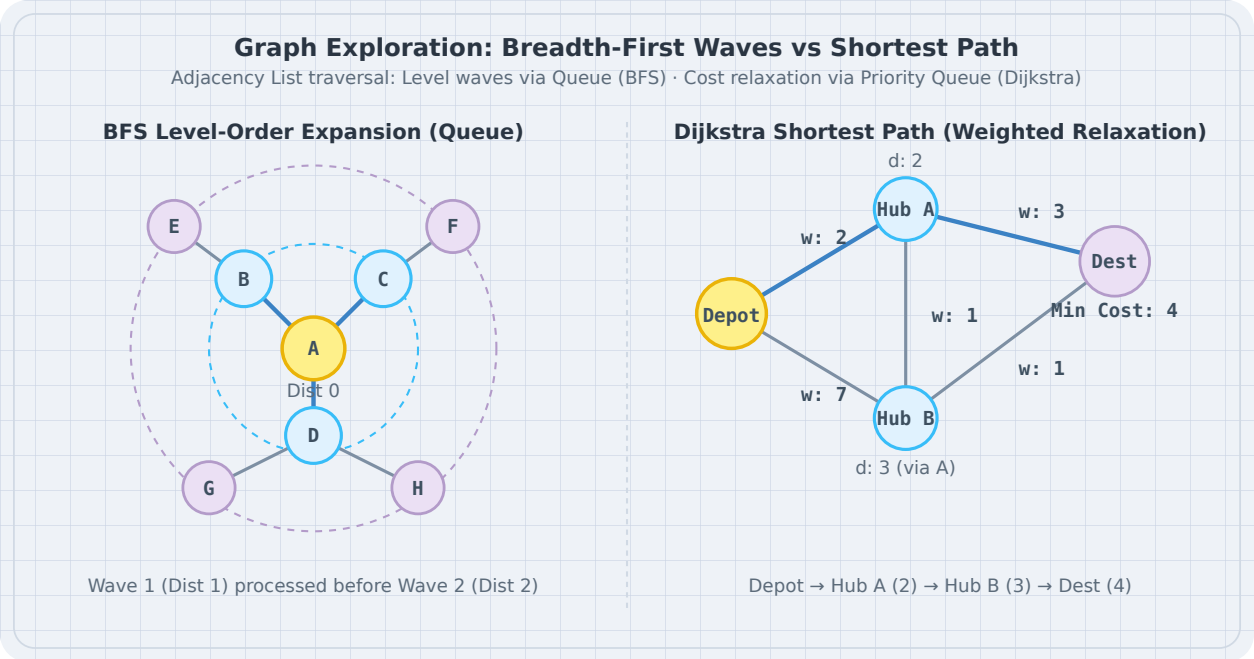


Figure 1: Graph Exploration: BFS and Dijkstra

Mental model

Adjacency List Representation

An adjacency matrix requires an $O(V^2)$ grid of memory, which wastes space on sparse networks. An adjacency list stores only actual connections in $O(V + E)$ space:

```
network = {
    "Depot A": ["Hub B", "Hub C"],
    "Hub B": ["Depot A", "Station D"],
    "Hub C": ["Depot A", "Station D"],
```

```
    "Station D": ["Hub B", "Hub C"],
}
```

BFS vs DFS Traversals

- **Breadth-First Search (BFS):** Uses a **FIFO Queue** (`collections.deque`). It explores outward in concentric ripples (all distance-1 neighbors, then distance-2). **Guarantees the shortest path in unweighted graphs**.
- **Depth-First Search (DFS):** Uses a **LIFO Stack** or call-stack recursion. It explores deeply down a single path until hitting a dead end, then backtracks. Ideal for topological ordering and cycle detection.

```
BFS Waves (Queue):              DFS Dive (Stack / Recursion):
       (A)                             (A)
     /  |  \                          /
   (B) (C) (D)  <- Wave 1          (B)
    |                               |
   (E)          <- Wave 2          (E) <- Dive deep before (C)
```

Worked examples

Case 1: BFS for Shortest Hop Delivery Routes

Save as `graph_bfs.py`. Find the shortest route (fewest transfer hops) between fulfillment hubs in a delivery network.

```
# graph_bfs.py
from collections import deque


class Graph[T]:
    def __init__(self) -> None:
        self.adj: dict[T, list[T]] = {}

    def add_edge(self, u: T, v: T, bidirectional: bool = True) -> None:
        self.adj.setdefault(u, []).append(v)
        if bidirectional:
            self.adj.setdefault(v, []).append(u)

    def shortest_path_bfs(self, start: T, target: T) -> list[T] | None:
        if start == target:
            return [start]

        visited: set[T] = {start}
```

```
        parent: dict[T, T] = {}
        queue: deque[T] = deque([start])

        found = False
        while queue:
            curr = queue.popleft()
            if curr == target:
                found = True
                break

            for neighbor in self.adj.get(curr, []):
                if neighbor not in visited:
                    visited.add(neighbor)
                    parent[neighbor] = curr
                    queue.append(neighbor)

        if not found:
            return None

        # Reconstruct path
        path: list[T] = []
        curr_node: T | None = target
        while curr_node is not None:
            path.append(curr_node)
            curr_node = parent.get(curr_node)

        path.reverse()
        return path


def main() -> None:
    network: Graph[str] = Graph()
    network.add_edge("Central-Depot", "North-Hub")
    network.add_edge("Central-Depot", "South-Hub")
    network.add_edge("North-Hub", "Station-East")
    network.add_edge("South-Hub", "Station-West")
    network.add_edge("Station-East", "Terminal-Z")
    network.add_edge("Station-West", "Terminal-Z")

    start = "Central-Depot"
    target = "Terminal-Z"

    route = network.shortest_path_bfs(start, target)
    assert route is not None
    print(f"BFS Shortest Route ({len(route) - 1} hops):")
    print(" -> ".join(route))
```

```
if __name__ == "__main__":
    main()
```

Run:

```
uv run python graph_bfs.py
```

Output:

```
BFS Shortest Route (3 hops):
Central-Depot -> North-Hub -> Station-East -> Terminal-Z
```

Case 2: DFS for Workflow Cycle and Deadlock Detection

Save as `graph_dfs_cycle.py`. Detect whether a task pipeline contains circular dependencies before dispatching batch jobs.

```
# graph_dfs_cycle.py
class TaskPipeline:
    def __init__(self) -> None:
        self.adj: dict[str, list[str]] = {}

    def add_dependency(self, task: str, depends_on: str) -> None:
        # task -> depends_on
        self.adj.setdefault(task, []).append(depends_on)

    def has_cycle(self) -> bool:
        # 3-color DFS:
        # 0: Unvisited, 1: Currently Visiting (in call stack), 2: Fully Visited
        state: dict[str, int] = {}

        def _dfs(node: str) -> bool:
            state[node] = 1  # Visiting

            for neighbor in self.adj.get(node, []):
                neighbor_state = state.get(neighbor, 0)
                if neighbor_state == 1:
                    # Found back-edge to ancestor in current call stack!
                    return True
                if neighbor_state == 0:
                    if _dfs(neighbor):
                        return True

            state[node] = 2  # Fully visited
            return False
```

```
        all_nodes = set(self.adj.keys())
        for u in self.adj.values():
            all_nodes.update(u)

        for node in all_nodes:
            if state.get(node, 0) == 0:
                if _dfs(node):
                    return True

        return False


def main() -> None:
    pipeline = TaskPipeline()
    pipeline.add_dependency("Generate-Invoice", "Verify-Items")
    pipeline.add_dependency("Verify-Items", "Calculate-Tax")
    pipeline.add_dependency("Calculate-Tax", "Read-Cart")

    print(f"Valid linear pipeline has cycle? {pipeline.has_cycle()}")

    print("Injecting circular dependency: Read-Cart -> Generate-Invoice")
    pipeline.add_dependency("Read-Cart", "Generate-Invoice")
    print(f"Pipeline now has cycle? {pipeline.has_cycle()}")


if __name__ == "__main__":
    main()
```

Run:

```
uv run python graph_dfs_cycle.py
```

Output:

```
Valid linear pipeline has cycle? False
Injecting circular dependency: Read-Cart -> Generate-Invoice
Pipeline now has cycle? True
```

The trap

The infinite cycle recursion trap: If a graph contains cycles and your traversal omits a `visited` set, recursive DFS will repeatedly re-visit earlier vertices until throwing `RecursionError: maximum recursion depth exceeded`. Always maintain an explicit `visited` set or 3-color status map.

The boring rule

1. Represent graphs using a `dict[T, list[T]]` adjacency list.
2. For unweighted shortest path queries, always use **BFS** with `collections.deque`.
3. For cycle detection, dependency scheduling, or topological sorting, use **DFS** with recursion or an explicit stack.
4. When reconstructing routes, record a `parent` mapping during traversal rather than copying list instances into the queue on every hop.

Try this

1. Implement topological sorting (`topological_sort() -> list[str]`) using DFS post-order traversal to produce an ordered build plan.
2. Write a function `connected_components(graph) -> list[set[str]]` that partitions an unpartitioned routing graph into isolated subnet clusters.

Shortest Paths (Dijkstra's Algorithm)

Shortest Paths (Dijkstra's Algorithm)

While Breadth-First Search finds the route with the fewest intermediary hops, real-world networks involve weighted costs: transmission latencies, courier miles, road tolls, or dispatch fees. **Dijkstra's algorithm** calculates the lowest total cost path across a weighted graph with non-negative edge weights in $O((V + E) \log V)$ time using a **min-priority queue**. In Python, the boring default is to pair standard dictionaries with the C-accelerated `heapq` module.

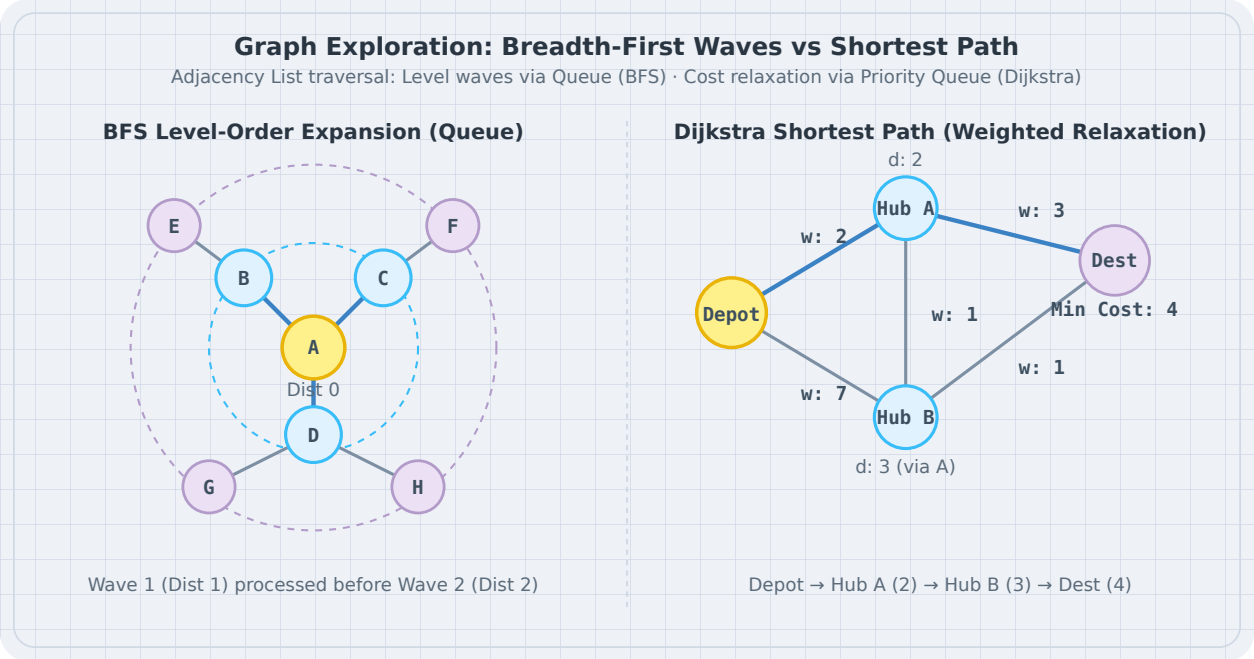


Figure 1: Graph Traversal and Dijkstra Relaxation

Mental model

Dijkstra maintains: 1. `dist[node]`: The lowest accumulated cost from the start node to each destination (initialized to ∞, with `dist[start] = 0`). 2. `prev[node]`: The preceding node along the cheapest known path.

At each step, the algorithm pops the unvisited vertex u with the smallest known distance from a min-priority queue and **relaxes** all of its outgoing edges (u, v, weight):

$$\text{If } \text{dist}[u] + \text{weight} < \text{dist}[v] \implies \text{dist}[v] = \text{dist}[u] + \text{weight}$$

```
Edge Relaxation:
(Depot) --[Known: 2]--> (Hub A) --[Weight: 3]--> (Dest)
                          \                        ^
                           \--[Weight: 1]-->(Hub B)-[Weight: 1]-/

Direct path: Depot -> Hub A -> Dest = 2 + 3 = 5
Relaxed path: Depot -> Hub A -> Hub B -> Dest = 2 + 1 + 1 = 4 (Cheaper!)
```

Because edge costs are non-negative, the moment a vertex is popped from the min-priority queue, its calculated distance is provably optimal.

Worked examples

Case 1: Delivery Network Optimization with heapq

Save as `dijkstra_delivery.py`. Calculate the lowest-latency delivery route between warehouse dispatch depots.

```python
# dijkstra_delivery.py
import heapq
from dataclasses import dataclass


@dataclass(slots=True)
class Edge:
    to: str
    weight: int


class WeightedGraph:
    def __init__(self) -> None:
        self.adj: dict[str, list[Edge]] = {}

    def add_edge(self, u: str, v: str, weight: int) -> None:
        self.adj.setdefault(u, []).append(Edge(to=v, weight=weight))
        self.adj.setdefault(v, []).append(Edge(to=u, weight=weight))

    def dijkstra(self, start: str, dest: str) -> tuple[int, list[str]] | None:
        dist: dict[str, int] = {node: float("inf") for node in self.adj}
        dist[start] = 0
        prev: dict[str, str] = {}

        # Min-priority queue stores tuples: (accumulated_dist, node)
        pq: list[tuple[int, str]] = [(0, start)]

        while pq:
```

```
            curr_dist, u = heapq.heappop(pq)

            # Drop stale queue entries
            if curr_dist > dist[u]:
                continue

            if u == dest:
                break  # Destination reached with optimal distance

            for edge in self.adj.get(u, []):
                new_dist = curr_dist + edge.weight
                if new_dist < dist.get(edge.to, float("inf")):
                    dist[edge.to] = new_dist
                    prev[edge.to] = u
                    heapq.heappush(pq, (new_dist, edge.to))

        if dist.get(dest) == float("inf"):
            return None  # Destination unreachable

        # Reconstruct path
        path: list[str] = []
        curr: str | None = dest
        while curr is not None:
            path.append(curr)
            curr = prev.get(curr)

        path.reverse()
        return dist[dest], path


def main() -> None:
    network = WeightedGraph()
    network.add_edge("Main-Depot", "North-Hub", 4)
    network.add_edge("Main-Depot", "South-Hub", 2)
    network.add_edge("South-Hub", "North-Hub", 1)
    network.add_edge("North-Hub", "Airport-Station", 7)
    network.add_edge("South-Hub", "Midtown-Annex", 8)
    network.add_edge("North-Hub", "Midtown-Annex", 3)
    network.add_edge("Midtown-Annex", "Airport-Station", 1)

    result = network.dijkstra("Main-Depot", "Airport-Station")
    assert result is not None
    min_cost, path = result

    print(f"Lowest Delivery Latency: {min_cost} ms")
    print("Optimal Route: " + " -> ".join(path))
```

```
if __name__ == "__main__":
    main()
```

Run:

```
uv run python dijkstra_delivery.py
```

Output:

```
Lowest Delivery Latency: 7 ms
Optimal Route: Main-Depot -> South-Hub -> North-Hub -> Midtown-Annex -> Airport-Station
```

Although the direct hop `Main-Depot -> North-Hub -> Airport-Station` has fewer stops (2 hops, cost 11), Dijkstra discovers the 4-hop route via `South-Hub` and `Midtown-Annex` with total cost $2 + 1 + 3 + 1 = 7$.

The trap

The two fatal traps with Dijkstra's algorithm:

1. **Negative Edge Weights:** If an edge weight is negative (e.g., a cashback discount that reduces total cost), Dijkstra's greedy invariant breaks, leading to wrong answers or infinite loops. Graphs with negative edge weights require the **Bellman-Ford algorithm**.
2. **Missing the Stale Item Check:** Because Python's `heapq` does not support updating keys in place, you push a new `(new_dist, v)` tuple every time a shorter path to v is discovered. Omitting `if curr_dist > dist[u]: continue` forces the loop to re-process superseded paths, turning an $O((V + E) \log V)$ algorithm into an inefficient crawl.

The boring rule

1. Use BFS for unweighted graphs. Use Dijkstra when edges have non-negative numerical weights.
2. Store `(distance, node_id)` tuples in `heapq` so Python sorts by distance automatically.
3. Always include `if curr_dist > dist[u]: continue` at the beginning of the pop loop.
4. Stop the search immediately once the destination node is popped from the heap.

Try this

1. Modify `dijkstra` to return a dictionary of shortest distances to all reachable nodes from `start`.
2. Add an edge filter parameter to Dijkstra that excludes any route traversing through congested toll roads.

String Search and Bloom Filters

String Search and Bloom Filters

Substring pattern searching and membership testing are core building blocks for log ingestion, audit trails, and web security proxies. When searching inside massive text streams or verifying whether an order ID exists among millions of records, brute-force checks consume unnecessary CPU cycles and database disk I/O. The boring default in Python is `'pattern' in text` and native `set` objects, but **Rabin-Karp rolling hashing** and **Bloom Filters** introduce essential algorithmic and probabilistic efficiencies.

Mental model

Rabin-Karp Rolling Hash

Rather than comparing characters at every position ($O(n \times m)$), Rabin-Karp computes a **polynomial rolling hash** of the pattern. When sliding the window by one character, it subtracts the leaving character and adds the entering character in $O(1)$ arithmetic:

```
Text:    [ B O R I N G ] - P Y
Window:  [ B O R ]       Hash = 1042 (Pattern hash: 9811 -> Mismatch)
Slide:     [ O R I ]     Hash = (1042 - 'B'*base) * base + 'I' in O(1)
```

Bloom Filter: Probabilistic Membership

A Bloom filter is a compact bit array paired with k independent hash functions:

```
Bit Array (m bits):
Index:  [ 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 ]
Bits:   [ 0 | 1 | 0 | 1 | 0 | 0 | 1 | 0 ]
              ^       ^           ^
             h1(ID)  h2(ID)      h3(ID)
```

1. **Insert:** Compute k hash values for an item and set those bit indices to `1`.
2. **Lookup:** Compute k hash values:
    - If **any** bit is `0`: The item is **definitely NOT** in the set (0% False Negatives).
    - If **all** bits are `1`: The item is **probably** in the set (with a small, mathematically tunable False Positive rate).

This allows high-throughput APIs to reject 99% of invalid requests in memory before hitting expensive databases or object storage.

Worked examples

Case 1: Rabin-Karp Rolling Hash Substring Search

Save as `rabin_karp.py`. Search for audit keywords across log strings in linear time.

```
# rabin_karp.py
def rabin_karp_search(text: str, pattern: str) -> int:
    n = len(text)
    m = len(pattern)
    if m == 0 or n < m:
        return -1

    base = 256
    mod = 1_000_000_007

    # Calculate base^(m-1) % mod
    h = pow(base, m - 1, mod)

    pattern_hash = 0
    window_hash = 0

    # Initial hash values
    for i in range(m):
        pattern_hash = (base * pattern_hash + ord(pattern[i])) % mod
        window_hash = (base * window_hash + ord(text[i])) % mod

    for i in range(n - m + 1):
        if pattern_hash == window_hash:
            # Hash match -> verify characters (resolves hash collisions)
            if text[i : i + m] == pattern:
                return i

        if i < n - m:
            # Rolling update: remove left char, shift, add right char
            window_hash = (
                base * (window_hash - ord(text[i]) * h) + ord(text[i + m])
            ) % mod
            if window_hash < 0:
                window_hash += mod

    return -1


def main() -> None:
    log_entry = "2026-09-18T10:14:02Z [AUDIT] Desk ticket #8192 marked STATUS=REFUND_APPROVE
    keyword = "STATUS=REFUND_APPROVED"
```

```
    idx = rabin_karp_search(log_entry, keyword)
    if idx != -1:
        print(f"Keyword match found at character offset {idx}")
        print(f"Snippet: '{log_entry[idx:idx + len(keyword)]}'")
    else:
        print("Pattern not found")


if __name__ == "__main__":
    main()
```

Run:

```
uv run python rabin_karp.py
```

Output:

```
Keyword match found at character offset 54
Snippet: 'STATUS=REFUND_APPROVED'
```

Case 2: In-Memory Bloom Filter for Order Existence

Save as `bloom_filter.py`. Implement a bitset Bloom filter using Python's `bytearray` and salted hashing.

```
# bloom_filter.py
import hashlib


class BloomFilter:
    def __init__(self, size_in_bits: int, num_hashes: int) -> None:
        self.num_bits = size_in_bits
        self.num_hashes = num_hashes
        # Allocate bytes (8 bits per byte)
        num_bytes = (size_in_bits + 7) // 8
        self.bit_array = bytearray(num_bytes)

    def _hashes(self, item: str) -> list[int]:
        indexes: list[int] = []
        raw_bytes = item.encode("utf-8")
        for i in range(self.num_hashes):
            # Salt with index to create independent hash probes
            digest = hashlib.sha256(raw_bytes + bytes([i])).digest()
            val = int.from_bytes(digest[:8], byteorder="big")
            indexes.append(val % self.num_bits)
        return indexes
```

```
    def add(self, item: str) -> None:
        for bit_index in self._hashes(item):
            byte_idx = bit_index // 8
            bit_offset = bit_index % 8
            self.bit_array[byte_idx] |= 1 << bit_offset

    def __contains__(self, item: str) -> bool:
        for bit_index in self._hashes(item):
            byte_idx = bit_index // 8
            bit_offset = bit_index % 8
            if not (self.bit_array[byte_idx] & (1 << bit_offset)):
                # Bit is 0 -> definitely NOT in the set!
                return False
        return True  # All bits 1 -> probably in the set!


def main() -> None:
    # 1024 bits with 4 hash probes
    bf = BloomFilter(size_in_bits=1024, num_hashes=4)

    existing_orders = ["ORDER-101", "ORDER-102", "ORDER-103", "ORDER-999"]
    for order_id in existing_orders:
        bf.add(order_id)

    queries = ["ORDER-101", "ORDER-102", "ORDER-500", "ORDER-777"]
    for q in queries:
        if q in bf:
            print(f"Query [{q}]: Probably exists (fetch from database)")
        else:
            print(f"Query [{q}]: Definitely DOES NOT exist (skipped DB read)")


if __name__ == "__main__":
    main()
```

Run:

```
uv run python bloom_filter.py
```

Output:

```
Query [ORDER-101]: Probably exists (fetch from database)
Query [ORDER-102]: Probably exists (fetch from database)
Query [ORDER-500]: Definitely DOES NOT exist (skipped DB read)
Query [ORDER-777]: Definitely DOES NOT exist (skipped DB read)
```

The trap

The fatal trap with Bloom filters is attempting to delete elements. Setting a bit back to `0` may corrupt the membership signatures of other distinct items that happened to share that same bit. Standard Bloom filters are strictly **append-only**.

If item deletion is required, use a **Counting Bloom Filter** (where each slot stores an integer counter rather than a single bit) or a Cuckoo filter.

The boring rule

1. Use Python's built-in `'pattern' in text` or `text.find()` for ordinary substring search. CPython implements a highly optimized two-way string search algorithm in C.
2. Use a Bloom filter in front of slow storage layers (like S3 or PostgreSQL) to prevent database queries for records that do not exist.
3. Size Bloom filters using the formula: $m \approx -\frac{n \ln p}{(\ln 2)^2}$, where n is expected item count and p is the acceptable false positive probability (e.g. 0.01 for 1%).
4. Always prepare fallback logic when a Bloom filter reports "probably present" since false positives are a natural property of the structure.

Try this

1. Experimentally verify the false positive rate of `BloomFilter` by adding 200 items and testing 5,000 non-existent keys.
2. Extend `rabin_karp_search` to find and return a list of all start indices where a pattern appears in a long text string.

Long Term Engineering

inspect and Reflection

inspect and Reflection

`inspect` and `getattr` are **last resorts**: useful for debugging, adapters, and tests that need a signature. They are not how the desk opens a ticket on a normal path. Call the function. Name the attribute.

Mental model

Reflection means asking a live object what it is: its name, its signature, its attributes. `inspect.signature(fn)` is the callable's parameters. `getattr(obj, "table")` is `obj.table` with a string name.

When the name is a string you computed, you have lost the editor, the type checker, and most grep. That cost is acceptable in a test helper or a serializer. It is not acceptable in `open_table`.

Worked examples

Case 1: `inspect.signature`

Save as `show_sig.py`. This is what you print when a helper's arguments drifted.

```
# show_sig.py
import inspect


def open_table(n: int, *, reason: str = "shift") -> str:
    return f"opened table {n} ({reason})"


def main() -> None:
    sig = inspect.signature(open_table)
    print(sig)
    print(list(sig.parameters))


if __name__ == "__main__":
    main()
```

Run:

```
uv run python show_sig.py
```

Output:

```
(n: int, *, reason: str = 'shift') -> str
['n', 'reason']
```

Case 2: `getattr` with a default

Save as `ticket_getattr.py`. The default argument is the only reason to prefer `getattr` over `obj.table` here.

```
# ticket_getattr.py
class Ticket:
    def __init__(self, ticket_id: int, table: int) -> None:
        self.id = ticket_id
        self.table = table


def main() -> None:
    ticket = Ticket(7, 12)
    print(getattr(ticket, "table"))
    print(getattr(ticket, "status", "open"))


if __name__ == "__main__":
    main()
```

Run:

```
uv run python ticket_getattr.py
```

Output:

```
12
open
```

`ticket.table` would have been clearer for the first line. The second line is the case `getattr` is for: optional attribute, default if missing.

Case 3: `inspect.getdoc`

Save as `show_doc.py`. Same string you wrote under `def`.

```
# show_doc.py
import inspect


def label(ticket_id: int, table: int) -> str:
    """Return a one-line ticket label for the desk display."""
    return f"ticket {ticket_id} → table {table}"


def main() -> None:
    print(inspect.getdoc(label))


if __name__ == "__main__":
    main()
```

Run:

```
uv run python show_doc.py
```

Output:

```
Return a one-line ticket label for the desk display.
```

The trap

A tiny dispatcher that looks up functions by string. It runs. It also hides every call site.

Save as `dispatch_trap.py`:

```
# dispatch_trap.py
def open_ticket() -> str:
    return "opened ticket"


def close_ticket() -> str:
    return "closed ticket"


def main() -> None:
    name = "open_ticket"
    fn = globals().get(name)
    if fn is None:
        raise KeyError(name)
    print(fn())
```

```
if __name__ == "__main__":
    main()
```

Run:

```
uv run python dispatch_trap.py
```

Output:

```
opened ticket
```

The boring version is `print(open_ticket())`. If you have two commands, use `if name == "open":` or a dict **you write as a dict**, not `globals()`. `getattr(module, user_input)` is the same trap with extra extra steps.

Fix:

```
# dispatch_fix.py
def open_ticket() -> str:
    return "opened ticket"


def close_ticket() -> str:
    return "closed ticket"


def main() -> None:
    commands = {
        "open": open_ticket,
        "close": close_ticket,
    }
    print(commands["open"]())


if __name__ == "__main__":
    main()
```

Run:

```
uv run python dispatch_fix.py
```

Output:

```
opened ticket
```

The keys are the API. Unknown names raise `KeyError` on the map, not on the module guts.

The boring rule

- Call functions by name. Read attributes by name.
- `inspect.signature` / `getdoc` for tools, docs, and tests — not for the happy path.
- `getattr(obj, name, default)` when the name is actually dynamic.
- Never `getattr` on user input against a module. Use an explicit dict.
- If a plugin protocol needs reflection, keep it in one adapter file.

Try this

1. In `show_sig.py`, print `sig.parameters["reason"].default`.
2. In `ticket_getattr.py`, use `ticket.table` instead of `getattr` for the first print.
3. In `dispatch_fix.py`, look up `"pay"` and catch `KeyError`.

ctypes and FFI

ctypes and FFI

Avoid FFI unless you must. Python's standard types, plus a subprocess or a well-tested package, cover almost all desk work. `ctypes` talks to C in-process. That is a sharp edge: crashes are not `except Exception`.

Mental model

FFI (foreign function interface) means calling code compiled for another language, usually C, from Python. `ctypes` ships with Python. You describe C types (`c_int`, `c_char_p`) and call a shared library.

`ctypes.sizeof` reports the C size of a type. `ctypes.c_int` is a mutable box around a C `int`. On common platforms that size is 4.

Calling `libc` (`strlen`, and friends) is **not** portable as a copy-paste: the library name is `libc.so.6` on many Linux systems, `libSystem` on macOS, and something else on Windows. This book does not make that the default example.

If you need C, prefer a maintained package with wheels, or a small extension you build on purpose — not ad-hoc `ctypes` in a ticket handler.

Worked examples

Case 1: `c_int` and `sizeof` (no extra C files)

Save as `c_int_size.py`. This runs anywhere ctypes runs.

```
# c_int_size.py
import ctypes


def main() -> None:
    table = ctypes.c_int(12)
    print(table.value)
    print(ctypes.sizeof(table))


if __name__ == "__main__":
    main()
```

Run:

```
uv run python c_int_size.py
```

Output:

```
12
4
```

`table.value` is the Python `int`. Changing `.value` mutates the box. A plain `int` is enough until a C API demands this object.

Case 2: A tiny C-like struct

Save as `ticket_struct.py`. `_fields_` is the C layout. Still no `.c` file.

```
# ticket_struct.py
import ctypes


class Ticket(ctypes.Structure):
    _fields_ = [
        ("id", ctypes.c_int),
        ("table", ctypes.c_int),
    ]


def main() -> None:
    ticket = Ticket(7, 12)
    print(ticket.id, ticket.table)
    print(ctypes.sizeof(ticket))


if __name__ == "__main__":
    main()
```

Run:

```
uv run python ticket_struct.py
```

Output:

```
7 12
8
```

Two `c_int` fields, 4 + 4, no padding on this layout. Do not depend on padding guesses when you talk to a real C header — match the header.

Case 3: The Python you wanted instead

Save as `ticket_python.py`. Same data, no FFI.

```
# ticket_python.py
from dataclasses import dataclass


@dataclass
class Ticket:
    id: int
    table: int


def main() -> None:
    ticket = Ticket(7, 12)
    print(ticket.id, ticket.table)


if __name__ == "__main__":
    main()
```

Run:

```
uv run python ticket_python.py
```

Output:

```
7 12
```

This is the default. Case 2 exists so you recognize ctypes when a library forces it.

The trap

Loading libc "because we can" for a job `len` already does.

Save as `skip_libc.py`. We do **not** call `CDLL`. We print why the boring path wins.

```
# skip_libc.py
def main() -> None:
    text = "desk"
    print(len(text))
    print("no libc")


if __name__ == "__main__":
    main()
```

Run:

```
uv run python skip_libc.py
```

Output:

```
4
no libc
```

A `strlen` call through `ctypes.CDLL("libc.so.6")` may work on your Linux box and fail on the next one. `len` is Unicode-aware in Python; `strlen` is bytes and a terminator. They are not the same function.

If you truly must call libc, isolate it behind one function, pass `bytes`, and test on every OS you ship. That is a last resort, not a style.

The boring rule

- Do not use ctypes for business logic.
- Prefer a dataclass, `array`, or `memoryview` over a `Structure`.
- Prefer a wheel from PyPI over a handwritten `CDLL`.
- Prefer `subprocess` plus a CLI over in-process C if the boundary is already a process.
- Never let FFI exceptions look like Python bugs: wrap the call, validate pointers, and keep the surface tiny.

Try this

1. In `c_int_size.py`, set `table.value = 3` and print it again.
2. In `ticket_struct.py`, add a `status` field as `c_int` (`0` open, `1` paid) and print it.
3. Rewrite Case 2's print using the dataclass from Case 3 — no ctypes.

Performance Tuning

Performance Tuning

Measure first. The boring tools are `timeit` for a small expression and `cProfile` for "where did the time go." Do not rewrite a ticket loop because it feels slow.

Mental model

CPU time and wall time are different. `timeit` repeats a snippet to damp noise. `cProfile` records function calls in one run. Both answer questions. Neither is a reason to add a cache on day one.

The desk is usually waiting on disk, the network, or a human. Faster JSON parsing will not fix that. Profile, then change the hot function, then measure again.

Timings themselves are not stable across machines. The listings below print **results** and the fact that a profiler saw the function — not a number of seconds you would treat as a fixture.

Worked examples

Case 1: Get the answer right, then time it

Save as `count_open.py`. `timeit.timeit` runs `count_open` 1000 times. We print the count, then a marker that the timer finished.

```
# count_open.py
import timeit


def count_open() -> int:
    tickets = [{"status": "open"}] * 100 + [{"status": "paid"}] * 50
    return sum(1 for t in tickets if t["status"] == "open")


def main() -> None:
    print(count_open())
    timeit.timeit(count_open, number=1000)
    print("measured")


if __name__ == "__main__":
    main()
```

Run:

```
uv run python count_open.py
```

Output:

```
100
measured
```

To see seconds on your machine, print the return value of `timeit.timeit`. Do not commit that float as a test.

Case 2: `cProfile` names the function

Save as `profile_open.py`. `runcall` profiles one invocation. We only assert the profiler recorded `count_open`.

```
# profile_open.py
import cProfile


def count_open() -> int:
    tickets = [{"status": "open"}] * 100 + [{"status": "paid"}] * 50
    return sum(1 for t in tickets if t["status"] == "open")


def main() -> None:
    profiler = cProfile.Profile()
    n = profiler.runcall(count_open)
    names = [
        s.code.co_name
        for s in profiler.getstats()
        if hasattr(s.code, "co_name")
    ]
    print(n)
    print("count_open" in names)


if __name__ == "__main__":
    main()
```

Run:

```
uv run python profile_open.py
```

Output:

```
100
True
```

From a terminal, the human-readable table is:

```
uv run python -m cProfile -s tottime count_open.py
```

Sort by `tottime`. Read the top rows. Ignore the rest until they matter.

Case 3: Two implementations, same answer

Save as `scan_vs_sum.py`. Correctness first. Timing second.

```
# scan_vs_sum.py
def count_loop(tickets: list[dict[str, str]]) -> int:
    n = 0
    for t in tickets:
        if t["status"] == "open":
            n += 1
    return n


def count_sum(tickets: list[dict[str, str]]) -> int:
    return sum(1 for t in tickets if t["status"] == "open")


def main() -> None:
    tickets = [{"status": "open"}] * 100 + [{"status": "paid"}] * 50
    a = count_loop(tickets)
    b = count_sum(tickets)
    print(a)
    print(a == b)


if __name__ == "__main__":
    main()
```

Run:

```
uv run python scan_vs_sum.py
```

Output:

```
100
True
```

If both are correct and neither shows up in a profile, keep the one that reads better (`count_sum` here).

The trap

Micro-optimizing a string that is not the bottleneck.

Save as `clever_label.py`:

```
# clever_label.py
def label_parts(ticket_id: int, table: int) -> str:
    return "ticket " + str(ticket_id) + " → table " + str(table)


def label_f(ticket_id: int, table: int) -> str:
    return f"ticket {ticket_id} → table {table}"


def main() -> None:
    print(label_parts(7, 12))
    print(label_f(7, 12))
    print(label_parts(7, 12) == label_f(7, 12))


if __name__ == "__main__":
    main()
```

Run:

```
uv run python clever_label.py
```

Output:

```
ticket 7 → table 12
ticket 7 → table 12
True
```

The `+` version is not a win you will measure on a desk. The f-string is the default. Optimize after `cProfile` points at a function that actually runs a million times.

The boring rule

- Make it correct. Then measure. Then change one thing.
- `timeit` for a snippet. `python -m cProfile -s tottime` for a program.
- Do not treat wall-clock floats as golden tests.
- I/O and algorithms beat micro-syntax.
- Keep the readable version when the profile is a tie.

Try this

1. Print `timeit.timeit(count_open, number=1000)` in `count_open.py` and note it changes run to run.
2. Add a third function that uses a list comprehension and check it against `count_sum`.
3. Run `uv run python -m cProfile -s tottime profile_open.py` and find `count_open` in the table.

Designing for the Long Term

Designing for the Long Term

Software lasts when the **boundaries are real**, the **API is small**, and the defaults stay boring: functions, dataclasses, the standard library, tests, and a version you can explain. This chapter is a recap you can keep on the desk.

Mental model

A **package boundary** is a directory other code imports. Inside, you may shuffle files. Across it, names are promises.

A **stable API** is a handful of functions (and maybe a dataclass) with docstrings. Hidden helpers start with `_` or live in a module you do not document.

Long-term Python in this book means: Python 3.14, `uv`, `ruff`, `pytest`, UTC in storage, `pathlib`, `logging`, `subprocess.run` as a list, no secrets in git, wheels from `uv build`.

Worked examples

Case 1: Two modules, one public function

Save as a tiny package in a temp tree and import it. Production looks the same without `TemporaryDirectory`.

`desk/__init__.py` exports `label` only. `desk/format_ticket.py` can change.

```
# stable_desk.py
import sys
from pathlib import Path
from tempfile import TemporaryDirectory


INIT = '''\
from desk.format_ticket import label

__all__ = ["label"]
'''

FORMAT = '''\
def label(ticket_id: int, table: int) -> str:
```

```
    """Return a one-line ticket label for the desk display."""
    return f"ticket {ticket_id} → table {table}"
'''


def main() -> None:
    with TemporaryDirectory() as raw:
        root = Path(raw)
        pkg = root / "desk"
        pkg.mkdir()
        (pkg / "__init__.py").write_text(INIT, encoding="utf-8")
        (pkg / "format_ticket.py").write_text(FORMAT, encoding="utf-8")
        sys.path.insert(0, str(root))
        import desk

        print(desk.label(7, 12))
        print(desk.__all__)


if __name__ == "__main__":
    main()
```

Run:

```
uv run python stable_desk.py
```

Output:

```
ticket 7 → table 12
['label']
```

Callers import `desk.label`. They do not import `desk.format_ticket` unless you say so.

Case 2: `pyproject.toml` is the project boundary

The long-term file at the root of a library:

```
[project]
name = "desk"
version = "1.0.0"
description = "Labels and shifts for a small desk."
requires-python = ">=3.14"

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"
```

That file, `desk/`, tests, and `README.md` are the repo. Everything else is extra.

Case 3: One process that still looks like the first chapter

Save as `desk_open.py`. A function, a print, a `main` guard. Types where they help. No framework.

```
# desk_open.py
def open_table(n: int) -> str:
    if n <= 0:
        raise ValueError(f"table {n}: number must be positive")
    return f"opened table {n}"


def main() -> None:
    print(open_table(12))


if __name__ == "__main__":
    main()
```

Run:

```
uv run python desk_open.py
```

Output:

```
opened table 12
```

If this is enough, stop. Add a package when a second module would otherwise collide.

The trap

A "platform" on day one: a base class, a plugin registry, and a settings object for a label.

Save as `too_much.py`:

```
# too_much.py
class BaseLabel:
    def render(self, ticket_id: int, table: int) -> str:
        raise NotImplementedError


class DeskLabel(BaseLabel):
    def render(self, ticket_id: int, table: int) -> str:
        return f"ticket {ticket_id} → table {table}"
```

```
def main() -> None:
    print(DeskLabel().render(7, 12))


if __name__ == "__main__":
    main()
```

Run:

```
uv run python too_much.py
```

Output:

```
ticket 7 → table 12
```

The boring package from Case 1 prints the same string with one function. Add a class when **state lives** and **behavior belongs to it**. Add plugins when you have a second implementation, not a slogan.

The boring rule

- Export a small `__all__`. Keep helpers behind it.
- Store UTC, log with `logging`, run children with `subprocess.run([...])`, read files with `Path`.
- Test the public functions. Lint with `ruff`. Audit with `uv audit`.
- Version in `pyproject.toml`. Build with `uv build`.
- Reflection and FFI stay at the edges.
- When in doubt, write the function from Case 3.

Try this

1. In `stable_desk.py`, add `desk/shifts.py` with `shift_label(name)` and export it from `__init__.py`.
2. In `desk_open.py`, catch `ValueError` for `open_table(0)` and print the error.
3. Delete `BaseLabel` from `too_much.py` and keep a single `label` function.

Appendices

Glossary

Glossary

Short definitions for terms this book uses as defaults. Words are defined on first use in the chapters; this list is for lookup.

API. The names, arguments, return values, and exceptions callers may depend on.

argparse. Standard-library parser for `sys.argv`. The CLI should call your functions, not reimplement them.

aware datetime. A `datetime` with `tzinfo` set. Store these (UTC). Contrast **naive datetime**.

`cProfile`. Standard-library profiler. Run `uv run python -m cProfile -s tottime file.py` after the program is correct.

CSV. Row-and-column text. Use `csv` with `newline=""` and UTF-8.

ctypes. Standard-library FFI. Call C types and libraries. Last resort; prefer Python types.

dataclass. Class generated from fields (`@dataclass`). Boring model for tickets and shifts.

docstring. String literal right under `def` or on a module. The written contract.

encoding. How bytes become text. This book always passes `encoding="utf-8"` for text files.

environment variable. A name/value pair on the process (`os.environ`). Where secrets and `DESK_SHIFT` live.

FFI. Foreign function interface: calling C (or similar) from Python. Avoid unless you must.

hatchling. A build backend. Named in `[build-system]` so `uv build` can produce wheels.

HTTPError. `urllib` exception for HTTP status errors such as 404.

`importlib.resources`. Read files shipped inside a package, independent of the current directory.

`inspect`. Standard-library reflection (`signature`, `getdoc`). For tools and tests, not the happy path.

ISO-8601. Timestamp text such as `2026-09-07T14:30:00+00:00`. What you put in JSON.

JSON. Nested text format for APIs and files (`json` module). No native datetime type.

`logging`. Standard-library logs: named loggers, levels, handlers. The boring default over `print` in services.

`main` guard. `if __name__ == "__main__":`. Runs `main` only when the file is the program.

naive datetime. A `datetime` with `tzinfo is None`. Do not store these as the system of record.

package. Importable directory (usually with `__init__.py`). The unit you ship.

path. A location on disk. `pathlib.Path` is the object; `/` joins relative parts.

pip-audit. Standalone vulnerability scanner. This book prefers `uv audit` when you already use `uv`.

pyproject.toml. Project metadata: name, version, dependencies, build backend.

pytest. Test runner. `uv run pytest`.

ruff. Linter and formatter. `uv run ruff check` / `ruff format`.

sdist. Source distribution tarball from `uv build`.

StringIO. In-memory text stream (`io.StringIO`). Same shape as a text file for tests.

subprocess.run. Run a child process. List of args, `capture_output=True`, `check=True`. No `shell=True` by default.

sys.executable. Path of the running interpreter. Use it to spawn Python children.

TemporaryDirectory. Temp folder that is deleted when the `with` block ends.

TextIO. Type of a text stream (`typing.TextIO`): files, `StringIO`.

timedelta. A duration (`datetime.timedelta`), not a clock.

timeit. Repeat a snippet and measure time. Do not freeze the float in tests.

TOML. Config format. `tomllib` **reads** it in the standard library; it does not write.

type hint. Annotation such as `n: int`. Used from the typing part onward where they help.

UTC. The zone you store (`datetime.timezone.utc`). Convert to a desk `ZoneInfo` for display.

uv. Toolchain: install Python, run programs (`uv run python file.py`), lock, build, audit.

wheel. Built install artifact (`*.whl`) from `uv build`.

ZoneInfo. IANA time zone (`zoneinfo.ZoneInfo("America/Chicago")`) for display and local wall times.

Command Cheat Sheet

Command Cheat Sheet

Commands this book actually uses. Run them from the directory that holds the file or the `pyproject.toml`.

Python

Command	What it does
`uv run python file.py`	Run a program with the project's Python 3.14
`uv run python -c "print('desk')"`	Run a one-liner
`uv run python -m cProfile -s tottime file.py`	Profile a program

Inside a program, spawn that same interpreter with `sys.executable`, not a bare `python` on `PATH`.

uv

Command	What it does
`uv lock`	Write/update the lockfile
`uv sync`	Install the locked environment
`uv add pkg`	Add a dependency
`uv build`	Build sdist + wheel into `dist/`
`uv audit`	Scan locked deps for known issues
`uv pip install dist/*.whl`	Install a wheel you just built

ruff

Command	What it does
`uv run ruff check .`	Lint
`uv run ruff check --fix file.py`	Lint and apply safe fixes
`uv run ruff format .`	Format

Silence is success for `ruff check`.

pytest

Command	What it does
`uv run pytest`	Run tests
`uv run pytest -q`	Quieter
`uv run pytest path/to/test_file.py`	One file

Tiny reminders

Run a complete program:

```
uv run python hello.py
```

```
# hello.py
def main() -> None:
    print("desk is open")


if __name__ == "__main__":
    main()
```

```
desk is open
```

Child process (list of args, no shell):

```
# run_child.py
import subprocess
import sys


def main() -> None:
    result = subprocess.run(
        [sys.executable, "-c", "print('ticket 7')"],
        capture_output=True,
        text=True,
        check=True,
    )
    print(result.stdout, end="")


if __name__ == "__main__":
    main()
```

```
uv run python run_child.py
```

```
ticket 7
```

Secrets stay in the environment (`DESK_API_KEY`), never in the file you commit.

Times you store are UTC (`datetime.now(timezone.utc)`), not naive local clocks.

Text files: `Path.read_text(encoding="utf-8")` and `write_text(..., encoding="utf-8")`.